# CẨM NANG NGÀNH

# DEVOPS



# MỤC LỤC

**PHẦN I — Máy tính và tiến trình** **4**

Chương 1 — Công việc, thời gian và đường đi dữ liệu . . . . . 5

Chương 2 — Chương trình, tiến trình và ranh giới nhân . . . . . 8

**PHẦN II — Linux và vận hành máy chủ** **12**

Chương 3 — Lời gọi hệ thống, đường dẫn và hệ thống tệp . . . . . 13

Chương 4 — Danh tính và quyền trong Linux . . . . . 16

Chương 5 — Tài nguyên và chẩn đoán hiệu năng Linux . . . . . 19

Chương 6 — systemd và vòng đời dịch vụ . . . . . 23

Chương 7 — Vận hành máy chủ và phục hồi có kiểm chứng . . . . . 27

**PHẦN III — Mạng và giao thức ứng dụng** **31**

Chương 8 — Địa chỉ, mạng con và đường đi gói IP . . . . . 32

Chương 9 — Tầng vận chuyển: TCP, UDP và NAT . . . . . 36

Chương 10 — Tên miền, TLS và ngữ nghĩa HTTP . . . . . 41

Chương 11 — Reverse proxy và điều tra yêu cầu qua hai chặng . . . . . 45

**PHẦN IV — Git và lịch sử thay đổi** **49**

Chương 12 — Đối tượng Git, vùng chờ và tham chiếu . . . . . 50

Chương 13 — Tích hợp nhánh và sửa lịch sử . . . . . 53

**PHẦN V — Tự động hóa hệ thống** **56**

Chương 14 — Bash và ngữ nghĩa lỗi của shell . . . . . 57

Chương 15 — Python và Go điều phối tiến trình . . . . . 61

Chương 16 — Quy trình tự động hóa an toàn và chạy lại . . . . . 68

**PHẦN VI — Build, kiểm thử và phát hành** **72**

Chương 17 — Dựng phần mềm và định danh sản phẩm . . . . . 73

Chương 18 — Cổng kiểm thử và CI/CD . . . . . 77

Chương 19 — Triển khai theo đợt và quyết định dừng . . . . . 80

Chương 20 — Phục hồi phiên bản và tương thích dữ liệu . . . . . 83

**PHẦN VII — Container và chuẩn OCI** **86**

Chương 21 — Namespace, cgroup v2 và ranh giới thực thi . . . . . 87

Chương 22 — Ảnh OCI, quy trình dựng và kho lưu trữ . . . . . 91

Chương 23 — Vòng đời container và dữ liệu . . . . . 94

Chương 24 — Compose và ứng dụng nhiều dịch vụ . . . . . 98

**PHẦN VIII — Điện toán đám mây và AWS** **101**

Chương 25 — Danh tính AWS và đánh giá chính sách IAM . . . . . 102

Chương 26 — VPC, mạng con và đường đi trong AWS . . . . . 105

Chương 27 — EC2, cân bằng tải và miền lỗi . . . . . 109

Chương 28 — Lưu trữ khối, tệp và đối tượng . . . . . 112

**PHẦN IX — Terraform và trạng thái hạ tầng** **115**

Chương 29 — HCL, nhà cung cấp và đồ thị phụ thuộc . . . . . 116

Chương 30 — Kế hoạch thay đổi, trạng thái và kho lưu trữ trạng thái . . . . . 121

Chương 31 — Sai lệch hạ tầng, nhập tài nguyên và tái cấu trúc . . . 124
Chương 32 — Thay đổi hạ tầng đang chứa dữ liệu . . . 127
**PHẦN X — Ansible và hội tụ cấu hình** **130**
Chương 33 — Danh sách máy và đường điều khiển Ansible . . . 131
Chương 34 — Hội tụ cấu hình, handler và lỗi từng máy . . . 134
Chương 35 — Vai trò, bí mật và triển khai theo đợt . . . 139
**PHẦN XI — Kubernetes và ứng dụng đám mây** **143**
Chương 36 — API, etcd và vòng điều khiển hội tụ . . . 144
Chương 37 — Pod, Deployment và trạng thái sẵn sàng . . . 148
Chương 38 — Lập lịch và ngân sách tài nguyên . . . 153
Chương 39 — Service, EndpointSlice và NetworkPolicy . . . 157
Chương 40 — PVC, CSI và dữ liệu của ứng dụng . . . 161
Chương 41 — HPA và điều khiển mở rộng . . . 165
Chương 42 — Gateway API và ranh giới yêu cầu ngoài cụm . . . 169
Chương 43 — Gói Helm (Helm chart) và vòng đời bản phát hành . . . 173
**PHẦN XII — Quan sát hệ thống và dữ liệu đo** **179**
Chương 44 — Phép đo, nhãn và chuỗi thời gian . . . 180
Chương 45 — Prometheus, PromQL và Grafana . . . 183
Chương 46 — Nhật ký, vết truy cập và OpenTelemetry . . . 188
Chương 47 — Cảnh báo và sức khỏe hệ thống quan sát . . . 194
**PHẦN XIII — Độ tin cậy và phục hồi dịch vụ** **198**
Chương 48 — Chỉ số, mục tiêu và ngân sách lỗi . . . 199
Chương 49 — Ứng phó sự cố và phân tích sau sự cố . . . 202
Chương 50 — Sao lưu, khôi phục thảm họa và dữ liệu . . . 206
Chương 51 — Hiệu năng, năng lực tải và thử giới hạn . . . 210
**PHẦN XIV — Bảo mật và DevSecOps** **214**
Chương 52 — Mô hình đe dọa và quyền tối thiểu . . . 215
Chương 53 — Bí mật và bảo vệ dữ liệu . . . 219
Chương 54 — Chuỗi cung ứng phần mềm . . . 223
Chương 55 — Gia cố môi trường đám mây và Kubernetes . . . 228
**PHẦN XV — Thiết kế và vận hành hệ thống thực tế** **232**
Chương 56 — Thiết kế một dịch vụ vận hành thực tế . . . 233
Chương 57 — Mở rộng và tối ưu chi phí có ràng buộc . . . 237
Chương 58 — Bài tổng hợp: thay đổi, sự cố và phục hồi . . . 241
**Lời giải bài tập** **246**

# PHẦN I — Máy tính và tiến trình

## Một dòng mã trở thành công việc có thể quan sát bằng cách nào?

Chương trình, tiến trình, luồng và tài nguyên là các thực thể khác nhau; thời gian là tổng của chạy, chờ và phối hợp.

I

**LỘ TRÌNH PHẦN**


Chương 1 — Công việc, thời gian và đường đi dữ liệu

Chương 2 — Chương trình, tiến trình và ranh giới nhân

# Chương 1 — Công việc, thời gian và đường đi dữ liệu

Phân tích một request thành tính toán, chờ và tác dụng phụ.

## Bài 1.1 — Máy thực thi

Một chương trình chỉ trở thành công việc khi hệ điều hành tạo tiến trình, cấp không gian địa chỉ và cho một hoặc nhiều luồng của nó cơ hội chạy. Tệp chương trình trên ổ lưu trữ là dữ liệu; lõi xử lý thực thi lệnh của luồng đã được bộ lập lịch chọn. Vì vậy, "máy có tám CPU" thường nói về tám đơn vị xử lý logic mà hệ điều hành có thể lên lịch, không hứa tám tác vụ sẽ tiến nhanh gấp tám lần. Hai luồng cùng sửa một dữ liệu còn phải phối hợp; nhiều lõi không xóa được thời gian chờ khóa hay thiết bị.

Ta có thể lần theo một request giả định: nhận byte từ mạng, giải mã trong bộ nhớ, đọc một tệp, tính kết quả rồi ghi câu trả lời. CPU thực hiện lệnh và di chuyển dữ liệu qua các tầng nhớ; RAM giữ mã, dữ liệu hoạt động và các trang cache; bộ điều khiển thiết bị cùng kernel đưa dữ liệu giữa bộ nhớ và mạng hoặc ổ lưu trữ. Khi tác vụ phải đợi dữ liệu chưa có, luồng có thể ngừng chạy để luồng khác dùng lõi. Một lần đọc sau có thể nhanh hơn vì dữ liệu đã nằm trong page cache; kết quả ấy không chứng minh ổ lưu trữ vừa nhanh hơn. Cũng không thể suy dung lượng bộ nhớ "bị ứng dụng dùng hết" chỉ từ lượng RAM đã được kernel dùng làm cache.

Đồng hồ trả lời một câu hỏi khác với bộ đếm CPU. Thời gian từ lúc bắt đầu tới lúc kết thúc request bao gồm cả chạy lẫn chờ; CPU time chỉ cộng thời gian được tính cho tiến trình hoặc luồng trong khi dùng CPU. Trên Linux, đồng hồ đơn điệu phù hợp để lấy hiệu giữa hai thời điểm của cùng máy vì không nhảy lùi khi chỉnh giờ hệ thống; đồng hồ thời gian thực phục vụ mốc lịch nhưng có thể được chỉnh. Đồng hồ đơn điệu thông thường không đếm thời gian máy suspend. Khi so log từ hai máy, phải xét đồng bộ giờ và vị trí đo; không lấy hiệu hai mốc từ những đồng hồ không có quan hệ đã kiểm chứng.

Ví dụ tính toán, không phải phép đo: request bắt đầu ở 0 ms, chờ lịch chạy 8 ms, chạy CPU 12 ms, chờ I/O 60 ms, chạy thêm 20 ms rồi hoàn tất. Wall time là 100 ms, CPU time theo mô hình này là 32 ms. Tăng gấp đôi tốc độ phần tính toán nhưng giữ nguyên hai khoảng chờ chỉ hạ wall time xuống 84 ms. Ngay cả con số đó là giả định: cache, tranh chấp và thời điểm hoàn tất I/O có thể đổi khi chương trình thật chạy. Bài kế tiếp dùng phân biệt này để đọc latency mà không nhầm nó với mức bận CPU.

### Bài tập 1.1 — Thời gian CPU và thời gian chờ

Trong timeline giả định 8 ms chờ lịch chạy, 12 ms CPU, 60 ms chờ I/O và 20 ms CPU, hãy tính wall time và CPU time. Nếu chỉ phần CPU nhanh gấp đôi, kết quả tối thiểu theo mô hình là bao nhiêu?

## Bài 1.2 — Đường thời gian

Hãy xét một ví dụ tính toán, không phải kết quả đo: yêu cầu A hoàn tất sau 100 ms theo đồng hồ, trong đó CPU xử lý 40 ms và các khoảng chờ cộng lại 60 ms. Độ trễ của A là 100 ms, còn CPU time gắn với phần xử lý của A là 40 ms. Nếu chỉ giảm thời gian CPU một nửa mà giữ nguyên mọi khoảng chờ, độ trễ mới tốt nhất trong mô hình đơn giản này là 80 ms. Kết quả đó không tiên đoán hệ thống thực; nó chỉ cho thấy không thể lấy tỷ lệ CPU bận để suy trực tiếp độ trễ của từng yêu cầu.

Để biết 60 ms chờ thuộc đoạn nào, tách trạng thái của **luồng**: `running` là đang dùng một lõi; `runnable` là đã đủ điều kiện chạy nhưng còn đợi bộ lập lịch cấp lõi; `blocked` là đang chờ một sự kiện như I/O hoàn tất. Trên Linux, cột trạng thái của `ps` dùng `R` cho cả đang chạy lẫn đang đợi trên run queue, `S` cho ngủ có thể ngắt khi đợi sự kiện, và `D` cho ngủ không thể ngắt, thường liên quan I/O. Một ảnh chụp `R` không cho biết luồng đã chạy CPU hay đợi lịch bao nhiêu mili giây; `D` cũng không xác định riêng ổ đĩa nào gây chậm. Ở timeline 100 ms của bài trước, 8 ms chờ lịch chạy và 60 ms chờ I/O đều làm tăng wall time nhưng không cộng vào 32 ms CPU time theo mô hình. Muốn phân biệt nút thắt phải đo các khoảng thời gian của đúng request, không suy từ một chữ trạng thái.

CPU time còn có **phạm vi**. Đồng hồ CPU của một luồng đếm phần luồng ấy tiêu thụ; đồng hồ CPU của tiến trình cộng các luồng. Ví dụ giả định hai luồng trong cùng tiến trình chạy song song trên hai lõi, mỗi luồng dùng 40 ms CPU trong 50 ms tường: CPU time tiến trình tăng 80 ms dù wall time chỉ qua 50 ms. Không thể gọi 80 ms ấy là độ trễ của request A khi tiến trình còn phục vụ request B. Ghép trace của request với counter CPU theo luồng/tiến trình và cửa sổ đo; nếu chỉ có CPU tổng, nêu rõ chưa quy được thời gian CPU cho từng request.

Thông lượng là số công việc hoàn thành trong một khoảng thời gian; độ trễ là thời gian một công việc mất từ đầu đến cuối. Hai đại lượng liên hệ nhưng không thay thế nhau. Một hệ thống có thể giữ thông lượng bằng cách xếp thêm yêu cầu vào hàng đợi, trong khi người dùng chờ lâu hơn. Vì thế, khi nói “cần thêm máy”, phải biết mục tiêu là tăng thông lượng, giảm độ trễ ở một mức tải hay giữ hệ thống ổn định khi có đột biến.

Một hàng đợi cho thấy sự khác nhau rõ hơn. Trong mô hình giả định, máy xử lý tối đa 120 yêu cầu mỗi giây, còn tải bình thường đến đều 100 yêu cầu mỗi giây. Một giây đột biến có 240 yêu cầu đến: nếu khả năng xử lý và chi phí mỗi yêu cầu không đổi, cuối giây đầu còn khoảng 120 yêu cầu chờ. Khi tải trở lại 100 yêu cầu mỗi giây, máy chỉ rút được khoảng 20 yêu cầu tồn mỗi giây; cần khoảng sáu giây để xóa backlog. Trong sáu giây ấy, thông lượng vẫn có thể xấp xỉ 120 yêu cầu mỗi giây, nhưng nhiều người dùng đợi lâu hơn. Phép tính bỏ qua phân bố thời điểm đến, thời gian xử lý và ưu tiên; nó không dự báo p95 thật.

Để kiểm chứng trên một dịch vụ thử nghiệm, đo đồng thời tốc độ đến, tốc độ hoàn tất, độ dài hàng đợi và phân vị latency theo từng cửa sổ ngắn. Nếu thông lượng hoàn tất nằm ngang khi tốc độ đến tăng còn hàng đợi và latency cùng tăng, giả thuyết bão hòa có cơ sở. Nếu hàng đợi không tăng nhưng một nhóm request vẫn chậm, phải tìm thêm các đường chờ như I/O hoặc khóa; chỉ tăng số replica theo CPU có thể không chạm đúng nguyên nhân.

### Bài tập 1.2 — CPU time, thời gian chờ và độ trễ

Trong ví dụ giả định, request mất 100 ms gồm 40 ms CPU và 60 ms chờ. Nếu chỉ giảm CPU xuống 20 ms, wall time thấp nhất theo mô hình cộng là bao nhiêu? Vì sao một lần thấy trạng thái R không chứng minh luồng chạy CPU suốt 100 ms? Nếu hai luồng cùng tiến trình mỗi luồng dùng 40 ms CPU trong 50 ms tường, CPU time của tiến trình tăng bao nhiêu và có được gọi đó là latency một request không?

## Bài 1.3 — Dữ liệu đi qua I/O

Một lời gọi ghi tệp đi qua nhiều ranh giới: bộ nhớ của tiến trình, lời gọi hệ thống, cache của kernel, filesystem, bộ điều khiển và thiết bị lưu trữ. “Ghi thành công” trước hết nghĩa là lời gọi đã chấp nhận số byte theo hợp đồng của nó; không đồng nghĩa mọi byte đã bền vững sau mất điện. Tầng đệm cho phép ứng dụng và thiết bị chạy với nhịp khác nhau. Page cache có thể giữ trang mới sửa ở trạng thái dirty, rồi kernel đưa chúng xuống thiết bị trong một lần writeback sau đó. Nếu chậm ở bước này, latency có thể xuất hiện muộn hơn lời gọi ghi ban đầu.

Hãy xét một thao tác thay cấu hình bằng tệp mới, với đường cũ vẫn còn cho tới khi thay tên. Ứng dụng ghi nội dung vào tệp tạm, yêu cầu đồng bộ tệp, đổi tên trong cùng filesystem, rồi yêu cầu đồng bộ thư mục chứa tên mới. Đây là một trình tự thiết kế độ bền trên hệ thống hỗ trợ các thao tác ấy; chỉ đồng bộ dữ liệu tệp chưa đủ để bảo đảm mục nhập thư mục mới đã bền. Ngược lại, thao tác đổi tên giúp người đọc tránh thấy một tệp đang ghi dở trong nhiều trường hợp, nhưng tính nguyên tử của tên không tự bảo đảm byte đã ở thiết bị. Cần xử lý từng lỗi trả về, ghi nhận filesystem và loại thiết bị, rồi thử khôi phục trong điều kiện mục tiêu; không biến một chuỗi lời gọi thành lời hứa tuyệt đối về mọi phần cứng.

Đọc cũng có hai đường. Nếu trang cần đọc đã ở page cache, kernel có thể phục vụ mà không chờ thiết bị. Nếu chưa có, tác vụ có thể phải đợi I/O vật lý; readahead và truy cập của tác vụ khác làm phép so “lần một so với lần hai” không còn là thí nghiệm sạch. Độ trễ thiết bị, độ trễ hàng đợi và thời gian hệ thống gọi lại ứng dụng là các thành phần khác nhau. Một counter tổng byte đọc từ ổ không chứng minh request cụ thể đã đọc byte ấy; cần tương quan thời điểm và tác vụ trước khi quy nguyên nhân.

Ví dụ tính toán: dịch vụ báo đã nhận 64 KiB và trả HTTP thành công sau 5 ms; sau đó máy mất điện. Nếu không có hợp đồng đồng bộ hoặc hệ lưu trữ giao dịch, câu trả lời HTTP chỉ là bằng chứng ứng dụng đã phát phản hồi, không là bằng chứng dữ liệu có thể đọc lại sau reboot. Với yêu cầu quan trọng, phải định nghĩa rõ điểm xác nhận bền vững, kiểm lỗi ghi và thử đọc lại từ bản phục hồi. Bài về Linux và backup sau này sẽ nối ranh giới này với snapshot nhất quán và phép thử restore.

### Bài tập 1.3 — Tên tệp sau mất điện

Một dịch vụ trả thành công ngay sau khi ghi tệp tạm và đổi tên. Sau mất điện, tên tệp mới biến mất. Nêu hai ranh giới độ bền cần kiểm và một điều chưa thể kết luận từ HTTP thành công.

# Chương 2 — Chương trình, tiến trình và ranh giới nhân

Lần theo một bản build đến hai tiến trình và kết quả từng nhánh.

## Bài 2.1 — Ba dạng ứng dụng

Một dịch vụ có thể có mã nguồn đúng mà vẫn không phục vụ được yêu cầu. Tệp thực thi có thể tồn tại, nhưng chưa có tiến trình nào đang chạy; tiến trình có thể chạy, nhưng không mở được cổng; cổng có thể mở, nhưng tiến trình không đọc được tệp cấu hình. Để chẩn đoán những tình huống này, trước hết cần biết hệ điều hành đã biến chương trình thành một thực thể đang hoạt động như thế nào. Chương này xây dựng mô hình ấy trước khi nói đến container, điều phối hay giám sát.

Mã nguồn là văn bản mô tả hành vi. Tệp thực thi hoặc chương trình được thông dịch là vật liệu để khởi chạy. Tiến trình là một lần thực thi cụ thể: nó có định danh, không gian địa chỉ, trạng thái xử lý, thông tin định danh người dùng và tập tài nguyên đang mở. Một chương trình có thể được chạy nhiều lần, tạo ra nhiều tiến trình độc lập. Ngược lại, một tiến trình có thể thay chương trình đang thực thi mà vẫn giữ định danh tiến trình, khi dùng cơ chế `execve`.

Sự phân biệt này giải thích vì sao “build thành công” không đồng nghĩa “dịch vụ sống”. Build xác nhận một số tính chất của artifact; nó không chứng minh hệ điều hành cấp đủ tài nguyên, địa chỉ mạng sẵn có hay cấu hình runtime hợp lệ. Đối với người vận hành, câu hỏi cần tách làm hai: artifact nào được đưa tới máy, và tiến trình nào đang thực thi artifact đó? Nếu hai câu trả lời khác nhau, một lần triển khai có thể đã thay tệp trên đĩa mà chưa thay phiên bản đang phục vụ.

CPU thực thi lệnh của tiến trình theo các khoảng thời gian; bộ nhớ chứa mã và dữ liệu cần cho lần chạy; nhân hệ điều hành quản lý tài nguyên và cung cấp các lời gọi hệ thống. Đó là mô hình khởi đầu, chưa phải toàn bộ chi tiết của Linux. Trong một tiến trình còn có thể có nhiều luồng thực thi cùng chia sẻ phần lớn không gian địa chỉ. Vì thế, số tiến trình không phải số công việc đồng thời, và mức dùng CPU không thể suy ra chỉ từ số PID.

Hãy lần theo một lần khởi động giả định thay vì gọi mọi thứ là “chạy app”. Lúc `t0`, service manager giữ đường dẫn tới artifact `A` và cấu hình `C`; chưa có PID của ứng dụng. Lúc `t1`, nó tạo tiến trình con và chuẩn bị UID, thư mục làm việc, biến môi trường, descriptor; lúc `t2`, lời gọi `execve` thành công thay hình ảnh chương trình trong tiến trình ấy. PID có thể giữ nguyên qua bước `execve`, nhưng mã, stack và ánh xạ bộ nhớ của chương trình mới được thiết lập theo quy tắc của hệ điều hành. Lúc `t3`, chương trình đọc cấu hình và bind cổng; chỉ sau khi đã sẵn sàng và đường request được kiểm tra mới có căn cứ nói dịch vụ phục vụ được. Một lỗi ở `t2` khác với lỗi parser cấu hình ở `t3`, dù bề mặt đều có thể là “không thấy port”.

Đổi artifact `A` trên đĩa sau `t2` không tự thay mã mà tiến trình đang chạy. Một lần restart có thể nạp `A` mới, nhưng cũng có thể nạp bản cũ nếu symlink, working directory hoặc lệnh khởi động trỏ nơi khác. Vì vậy khi triển khai, ghi cả digest của artifact dự kiến và identity của tiến trình đang phục vụ, không chỉ xem mốc thời gian của file. `ps` cho PID và dòng lệnh quan sát được; socket listener cho biết tiến trình nào giữ cổng; một request qua đúng đường người dùng cho biết instance nào trả lời. Từng phép quan sát giải một phần câu hỏi, không phép nào đơn lẻ chứng minh toàn bộ đường phục vụ.

Trong máy thử, có thể chạy một chương trình rất nhỏ in PID và phiên bản nhúng trong binary, giữ nó phục vụ một cổng cục bộ, rồi thay binary trên đĩa và gọi lại endpoint. Dự đoán phiên bản đang phục vụ trước restart và sau restart, đồng thời ghi PID cũ/mới. Không làm thí nghiệm trên service thật vì restart là thay đổi trạng thái. Nếu bản chạy mới vẫn báo phiên bản cũ, kiểm đường dẫn thực của executable và quy trình build/deploy trước khi đổ lỗi cho cache mạng. Mô hình artifact–process này là tiên quyết để hiểu container image, rollout và hậu kiểm release ở các Phần sau.

**Bài tập 2.1 — Thay tệp chạy khi dịch vụ còn hoạt động**

Một bản triển khai thay tệp thực thi trên đĩa nhưng không khởi động lại dịch vụ. Giải thích vì sao không được kết luận tiến trình đang phục vụ đã chạy phiên bản mới. Nêu phép kiểm tra cần làm.

**Bài 2.2 — Tạo và thay thế tiến trình**

Trên Linux, `fork` tạo tiến trình con. Tiến trình cha nhận PID của con, còn con nhận số 0; nếu không tạo được con, chỉ cha nhận `-1` và mã lỗi. Nhờ vậy hai tiến trình đi theo hai nhánh xử lý khác nhau. Không nên hình dung rằng toàn bộ bộ nhớ vật lý bị sao chép ngay lập tức: triển khai Linux dùng các trang copy-on-write, tạo bản riêng khi cần ghi. Đây là chi tiết triển khai Linux, không phải định nghĩa phổ quát của mọi hệ điều hành.

Sau bước tạo con, một cách khởi chạy chương trình khác là gọi `execve`. Lời gọi này thay process image hiện tại bằng chương trình mới; nó không tạo thêm một tiến trình mới. Nếu thành công, luồng lệnh cũ không tiếp tục ngay sau lời gọi. Nếu lỗi, lời gọi trả về `-1`, đặt `errno` và chương trình cũ còn cơ hội xử lý. Cặp cơ chế tạo con rồi thay chương trình giúp hiểu vì sao một shell có thể tiếp tục tồn tại sau khi khởi chạy lệnh khác, đồng thời vì sao tiến trình của lệnh có PID riêng.

Ví dụ chạy trên Linux bằng `python3 fork_exec.py`; thêm `fail` để cố ý chọn đường dẫn không tồn tại. PID do hệ điều hành cấp. Cha in trạng thái sau khi `waitpid` trả về.

```
"""Linux-only trace of fork and execve; run with optional argument 'fail'."""

import os
import sys

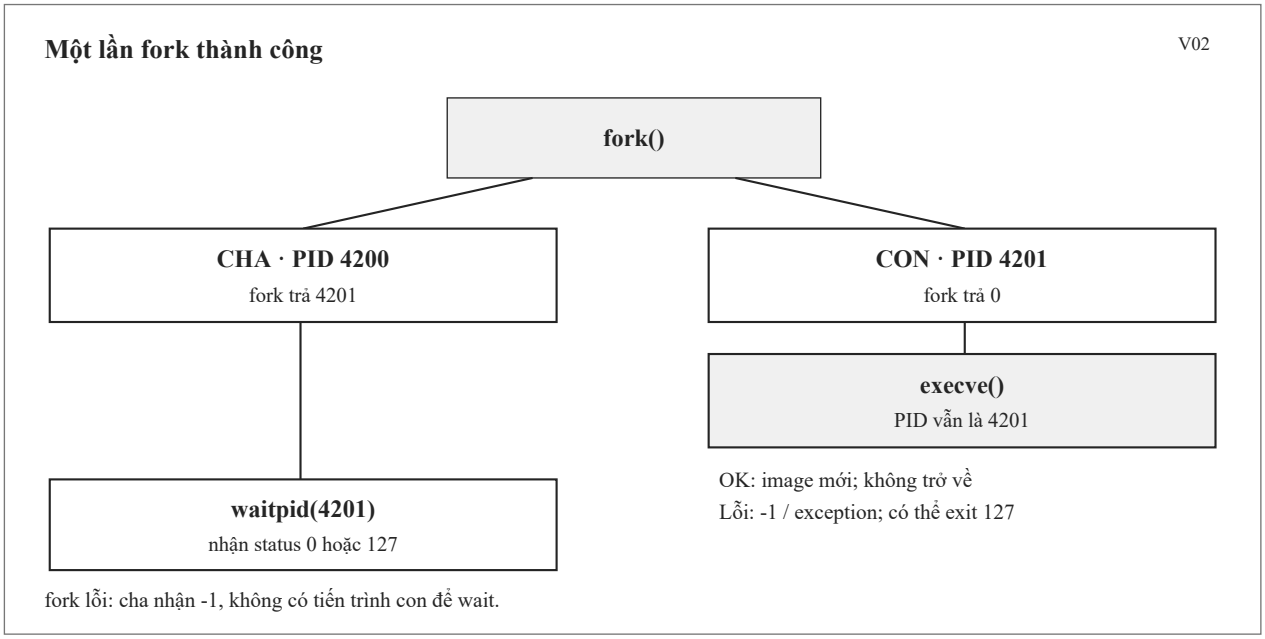

child = os.fork()
if child == 0:
    path = "/definitely/missing" if sys.argv[1:] == ["fail"] else "/bin/echo"
    os.write(2, f"child: fork returned {child}, PID {os.getpid()}\n".encode())
    try:
        os.execve(path, ["echo", "child: execve succeeded"], os.environ.copy())
    except OSError as exc:
        os.write(2, f"child: execve failed: {exc.strerror}\n".encode())
        os._exit(127)

print(f"parent: fork returned child PID {child}", flush=True)
_, status = os.waitpid(child, 0)
if os.WIFEXITED(status):
    print(f"parent: child exited with {os.WEXITSTATUS(status)}")
else:
    print("parent: child did not exit normally")
```

Ở cả hai lần chạy, `fork` thành công tạo hai PID. Trong cha, `child` là PID dương của con; trong con, `child` bằng 0. Nhánh thành công thay chương trình của con bằng `/bin/echo`, giữ nguyên PID của con và không vào khối `except`. Nhánh lỗi khiến lời gọi Python ném `OSError`; con in lỗi rồi thoát với mã 127 theo quy ước riêng của ví dụ, còn cha nhận mã ấy qua `waitpid`. Ở giao diện lời gọi hệ thống C, lỗi tương ứng là giá trị `-1` và `errno`; Python chuyển nó thành ngoại lệ. Nếu `fork` thất bại, Python cũng ném `OSError` và không tạo con để chờ. Mã 127 ở đây là lựa chọn của chương trình, không phải mã lỗi cố định do `execve` áp đặt.

Ví dụ chỉ dành cho tiến trình Python một luồng. Từ Python 3.12, `os.fork()` có thể phát `DeprecationWarning` nếu phát hiện tiến trình có nhiều luồng: các khóa do luồng khác giữ có thể khiến tiến trình con kẹt trước khi thay chương trình. Không dùng đoạn mã này làm mẫu khởi chạy trong ứng dụng Python đa luồng.

Mô hình này cũng giải thích một lỗi vận hành phổ biến: chương trình con kế thừa tài nguyên mà người gọi quên đóng. Theo mặc định, một file descriptor có thể còn mở qua `execve`; cờ close-on-exec thay đổi hành vi đó. Nếu vô tình giữ đầu ghi của một pipe ở tiến trình khác, đầu đọc có thể chưa nhìn thấy kết thúc luồng dù bên tạo dữ liệu đã xong. Kết luận này là hệ quả của vòng đời tài nguyên, không phải một quy tắc riêng của Bash.




Hình I.2.1 — fork/execve/wait: PID giả định; nhánh thành công và lỗi là hai lần chạy khác nhau.

### Bài tập 2.2 — Nhánh lỗi sau fork và execve

`fork` thành công, `execve` thất bại. Trong mô hình của chương này, có bao nhiêu tiến trình sau `fork`, và tiến trình nào có thể nhận biết lỗi `execve`? Vì sao không được coi `execve` là thao tác tạo tiến trình?

### Bài 2.3 — Bộ mô tả tệp và danh tính

Khi tiến trình gọi `open`, nó nhận một số nguyên nhỏ không âm: file descriptor. Số này là chỉ số trong bảng descriptor của chính tiến trình. Descriptor không phải tên tệp, cũng không phải bản thân nội dung tệp. Nó tham chiếu tới một open file description do nhân quản lý, nơi giữ vị trí đọc/ghi hiện tại và các cờ trạng thái. Hai descriptor có thể cùng tham chiếu một open file description; khi đó chúng chia sẻ vị trí đọc/ghi.

Sự khác biệt có ý nghĩa khi đọc log. Nếu một tiến trình đã mở tệp log và người vận hành đổi tên hoặc xóa đường dẫn, descriptor đang mở vẫn có thể trỏ tới đối tượng cũ. Tạo một tệp mới ở cùng tên đường dẫn không buộc tiến trình chuyển đầu ghi sang tệp mới. Phần thực hành về log rotation ở các chương sau sẽ dựa trên quan hệ này để giải thích vì sao có trường hợp dung lượng đĩa chưa được giải phóng và vì sao cần yêu cầu dịch vụ mở lại log.

Ba descriptor quen thuộc khi khởi chạy chương trình là đầu vào chuẩn, đầu ra chuẩn và đầu lỗi chuẩn. Chuyển hướng trong shell thay đổi các luồng mà tiến trình nhận được; nó không sửa ý nghĩa của chương trình. Khi một dịch vụ ghi thông báo lỗi vào đầu ra chuẩn, người thu thập log vẫn có thể lấy được dữ liệu, nhưng việc phân loại mức độ lỗi trở nên khó hơn. Do vậy, quy ước đầu ra là một phần của hợp đồng vận hành.

Chương này không yêu cầu thuộc lòng mọi lời gọi hệ thống. Điều cần giữ là ba câu hỏi: thứ gì tồn tại trên đĩa, tiến trình nào đang chạy, và tiến trình ấy đang nắm những tài nguyên nào? Phần sau sẽ dùng cùng ba câu hỏi khi đi từ một tiến trình trên máy tới container, rồi tới workload trong cụm.

Một quan sát an toàn trên Linux là tạo một tiến trình thử ghi vào tệp riêng trong thư mục tạm, giữ nó chạy, đổi tên tệp từ tiến trình khác rồi xem `/proc/<pid>/fd` của đúng PID. Liên kết của descriptor có thể còn chỉ tới inode cũ dưới tên đã đổi; nếu đường dẫn bị xóa khi descriptor vẫn mở, tệp chưa nhất thiết giải phóng dung lượng ngay. Thí nghiệm phải chỉ dùng dữ liệu tạm do mình tạo và phải đóng tiến trình thử sau khi kiểm. Khi chẩn đoán dịch vụ thật, so inode và dung lượng trước khi yêu cầu mở lại log; đừng xóa log đang mở để “giải phóng” dung lượng vì thao tác xóa tên có thể làm mất đường khôi phục mà chưa trả lại không gian.

# PHẦN II — Linux và vận hành máy chủ

## Nhân cho phép, giới hạn và duy trì một dịch vụ như thế nào?

Lời gọi hệ thống đi qua định danh, namespace tên, quyền, tài nguyên rồi tới supervisor; mỗi ranh giới có bằng chứng riêng.

II

**LỘ TRÌNH PHẦN**


- Chương 3 — Lời gọi hệ thống, đường dẫn và hệ thống tệp
- Chương 4 — Danh tính và quyền trong Linux
- Chương 5 — Tài nguyên và chẩn đoán hiệu năng Linux
- Chương 6 — systemd và vòng đời dịch vụ
- Chương 7 — Vận hành máy chủ và phục hồi có kiểm chứng

# Chương 3 — Lời gọi hệ thống, đường dẫn và hệ thống tệp

Giải thích tại sao cùng tên file có thể trỏ đối tượng khác.

## Bài 3.1 — Vào nhân

Một dịch vụ có thể chạy dưới đúng người dùng mà vẫn không đọc được cấu hình. Một tệp có bit đọc nhưng đường dẫn tới nó lại không thể đi qua. Một lệnh `systemctl start` có thể trả thành công trong khi ứng dụng chưa sẵn sàng nhận yêu cầu. Những hiện tượng này thuộc các lớp khác nhau: nhân kiểm tra lời gọi hệ thống và quyền truy cập, hệ thống tệp ánh xạ tên sang đối tượng, còn systemd điều hành vòng đời tiến trình. Chương này nối các lớp đó qua một trường hợp vận hành có thể tự kiểm chứng.

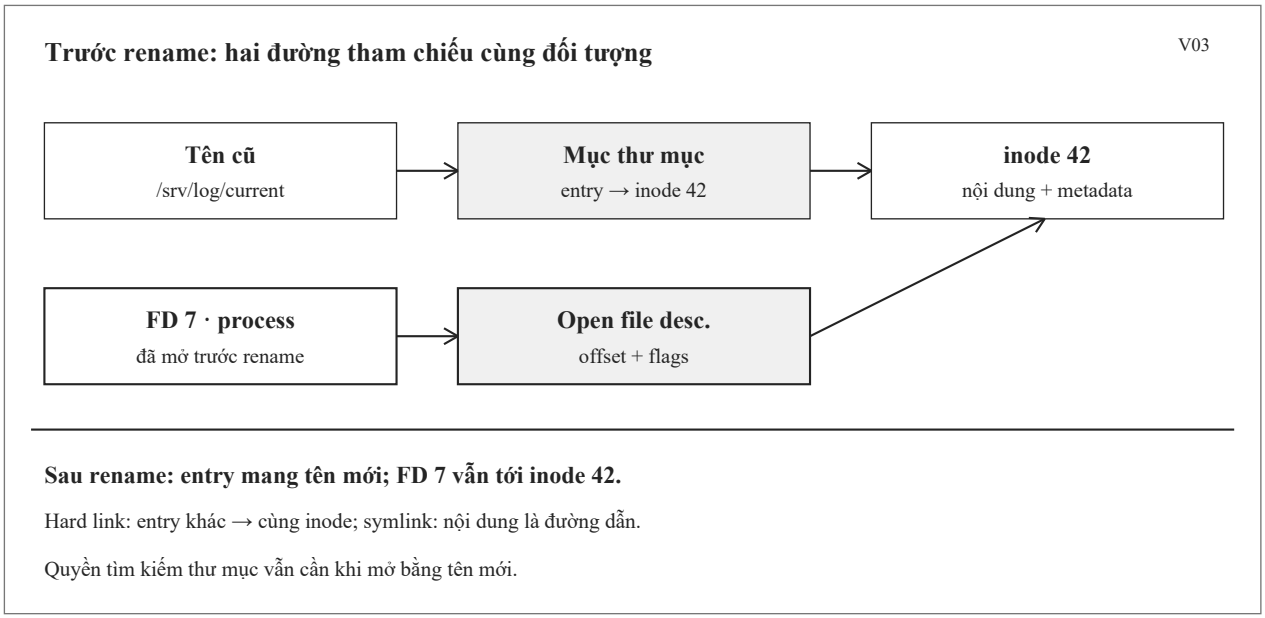
Tiến trình thực thi phần lớn mã của nó ở user space. Khi cần mở tệp, tạo socket hay đổi quyền, nó gọi một giao diện mà cuối cùng đi vào nhân. Ví dụ `openat` nhận đường dẫn, cờ mở và ngữ cảnh của tiến trình; nhân phân giải đường dẫn, kiểm tra quyền, rồi trả về file descriptor hoặc lỗi. Một hàm thư viện hay một lệnh shell có thể gọi thêm nhiều bước trước và sau lời gọi này. Vì vậy, thông báo “Permission denied” của ứng dụng cần được lần tới thao tác thực sự thất bại; nó không tự cho biết bit quyền nào sai.

Nhân là nơi thực thi các quy tắc về tiến trình, bộ nhớ, đối tượng tệp và thiết bị. systemd là một tiến trình ở user space dùng các giao diện ấy để khởi động và giám sát dịch vụ. `systemctl` lại là công cụ yêu cầu systemd thực hiện thao tác. Ba tên này không thay thế nhau: nhân không đọc mục `Restart=` trong unit, còn systemd không bỏ qua được phép kiểm tra của nhân chỉ vì unit đã nạp thành công.

Sau một lời gọi thất bại, chương trình có thể nhận `EACCES` khi thiếu quyền tìm kiếm hoặc truy cập, `ENOENT` khi thành phần đường dẫn không tồn tại, hay `ENOTDIR` khi một thành phần trung gian không phải thư mục. Đó là mã lỗi ở ranh giới lời gọi; ứng dụng có thể gộp hoặc đổi thông báo khi hiển thị. Trước khi sửa quyền, phải xác định đường dẫn thực, người dùng hiệu lực và thao tác đọc, ghi hay thực thi mà tiến trình đã yêu cầu.

## Bài 3.2 — Phân giải tên

Đường dẫn là chuỗi các thành phần được tra cứu từ một điểm bắt đầu. Mỗi thư mục tham gia phép tra cứu tên; tệp cuối cùng có metadata như loại, quyền và số inode. Tên không đồng nhất với inode: đổi tên thay quan hệ trong thư mục, còn một descriptor đang mở vẫn tham chiếu đối tượng đã mở như Bài 2.3 đã giải thích. Một hard link khác có thể trỏ tới cùng inode trên cùng hệ thống tệp, còn symbolic link chứa một đường dẫn cần được phân giải tiếp. Vì thế, xem riêng `ls -l` của tệp đích có thể bỏ sót vấn đề ở thư mục cha hoặc một liên kết trung gian.




Hình II.1.1 — Đổi tên thay mục thư mục; descriptor đang mở vẫn tới cùng inode.

Hãy xét `/srv/reports/daily.json` có mode `0644`, còn `/srv/reports` có mode `0700` và thuộc người dùng `deploy`. Một tiến trình chạy dưới người dùng `reader` vẫn không mở được tệp bằng đường dẫn này: nó cần quyền tìm kiếm trên từng thư mục cha trước khi quyền đọc ở tệp cuối có ý nghĩa. Đây là tình huống giả định để lần theo phép tra cứu, không phải output đã đo trên máy. Khi chẩn đoán, ghi lại UID/GID hiệu lực của tiến trình, từng thành phần đường dẫn và đích sau khi phân giải symlink; đổi mỗi mode của tệp cuối có thể không chữa đúng chỗ và có thể mở quyền quá rộng.

Có hai phép kiểm khác nhau dễ bị trộn lẫn. Quyền `x` trên thư mục cho phép đi qua nó để tra một tên đã biết; quyền `r` cho phép liệt kê tên trong thư mục. Một thư mục có `--x` đối với người gọi có thể cho phép mở một tệp đã biết tên, nhưng không cho phép liệt kê. Ngược lại, có `r` mà thiếu `x` thì nhìn thấy tên cũng chưa tra cứu được inode bên dưới. Khi gặp `Permission denied`, hãy xác định syscall mở đường dẫn nào trong namespace nào, rồi kiểm tra từng thư mục từ điểm bắt đầu đến tệp cuối. `namei -l` giúp trình bày chuỗi thành phần; `id` của shell điều tra chỉ có giá trị nếu cùng credential với tiến trình lỗi. Với service, xem UID/GID hiệu lực và supplementary groups của chính tiến trình, không suy từ người đã đăng nhập.

Thử suy luận với hai thay đổi riêng trong ví dụ trên. Cấp cho `reader` quyền đọc trên `daily.json` mà không cấp search trên `/srv/reports` vẫn thất bại trước khi kernel xét quyền cuối. Chỉ cấp search trên thư mục lại chưa đủ nếu tệp đích chuyển thành `0600` thuộc `deploy`. Nếu đường dẫn là symlink sang `/private/daily.json`, phải kiểm tra đường dẫn đích và các mount liên quan; mode hiển thị ở symlink không thay thế phép kiểm ở đích. Một file descriptor đã mở trước khi đổi tên hoặc gỡ tên vẫn có thể đọc đối tượng đã mở, nên phép thử "mở tên mới" không chứng minh mọi tiến trình đang dùng cùng đối tượng.

Trong môi trường tạm do chính mình sở hữu, có thể dựng hai thư mục và một tệp thử, đặt mode như trên, dùng một tài khoản không đặc quyền khác để kiểm tra cả liệt kê thư mục và mở tệp theo tên đã biết. Ghi riêng kết quả của từng thao tác và errno; sau khi chỉnh đúng quyền trên thư mục, thử lại cùng đường dẫn, cùng UID và cùng mount namespace. Nếu vẫn lỗi, chuyển sang ACL, mount

read-only, chính sách bảo mật hoặc thành phần đường dẫn khác. Không nới `0777` chỉ để làm phép thử xanh: nó thay đổi nhiều quyền hơn giả thuyết cần kiểm tra.

**Bài tập 3.1 — Tệp log cũ vẫn nhận dữ liệu**

Một chương trình vẫn ghi vào tệp log đã bị đổi tên; tệp mới cùng tên vẫn rỗng. Hãy dùng quan hệ giữa đường dẫn, descriptor và open file description để giải thích. Đề xuất một cách xử lý an toàn.

**Bài 3.3 — Ghi và bền vững**

Đọc một tệp thường đi qua page cache của Linux. Dữ liệu vừa đọc có thể được giữ trong RAM để lần truy cập sau không phải lấy lại từ thiết bị. Cache vì thế không đơn giản là "RAM bị chiếm vô ích"; nó là một cách đổi bộ nhớ lấy độ trễ I/O thấp hơn. Khi thiếu bộ nhớ, nhân có thể thu hồi những trang phù hợp. Tuy nhiên, khả năng thu hồi và chi phí thu hồi khác nhau giữa trang sạch của tệp, trang đã sửa nhưng chưa ghi xuống thiết bị, và vùng nhớ ẩn danh.

Ghi vào tệp cũng có thể hoàn tất ở tầng lời gọi ứng dụng trước khi mọi thay đổi đã nằm trên thiết bị bền vững. Trang đã sửa trong cache được đánh dấu dirty và sau đó cần được đồng bộ. Vì vậy, "lời gọi ghi trả về" và "dữ liệu sống sót qua mất điện" là hai hợp đồng khác nhau. Trên Linux, `fsync` yêu cầu đồng bộ dữ liệu cùng metadata của tệp tới thiết bị theo hợp đồng của lời gọi; khi tạo hoặc đổi tên tệp, việc đồng bộ riêng descriptor của tệp chưa nhất thiết bảo đảm mục thư mục chứa tên đó đã bền vững. Ứng dụng có dữ liệu quan trọng phải xác định chính xác mình cần bảo đảm điều gì, thay vì gọi chung là "đã lưu".

Một thao tác sao lưu an toàn vì thế không chỉ là sao chép bytes rồi thấy lệnh trả 0. Cần kiểm tra nhất quán dữ liệu đầu vào, kết quả ghi, ranh giới bền vững cần thiết và khả năng phục hồi. Hệ thống tệp, cơ sở dữ liệu và object storage cung cấp các hợp đồng khác nhau; chương về lưu trữ sẽ đi sâu vào từng loại. Ở đây điều cần nhớ là I/O có nhiều tầng và độ trễ ở từng tầng không hiện ra từ một phép đo CPU.

Xét việc thay `config.json` bằng một phiên bản mới trên **cùng hệ thống tệp**. Một trình tự cần phân tích là ghi đầy đủ `config.tmp`, kiểm mọi lỗi ghi, gọi `fsync` trên descriptor của tệp tạm, đổi tên tệp tạm thành `config.json`, rồi đồng bộ descriptor của thư mục chứa tên đó. Bước đầu bảo vệ byte của tệp theo hợp đồng `fsync`; đổi tên làm tên mới có hiệu lực trong namespace đang chạy; bước cuối xử lý độ bền của mục thư mục. Nếu bỏ bước cuối, `fsync` của tệp không tự chứng minh tên `config.json` sẽ còn sau crash. Mỗi bước có thể trả lỗi, kể cả hết dung lượng xuất hiện lúc đồng bộ; nếu lỗi, không báo "đã lưu". Đây là mô hình cho filesystem Linux phù hợp, không phải lời hứa tuyệt đối trước lỗi phần cứng, database transaction hay mọi filesystem từ xa. Thử phục hồi từ bản sao sau một lần ghi có kiểm soát vẫn cần thiết cho dữ liệu quan trọng.

**Bài tập 3.2 — Ghi cấu hình bền vững**

Dịch vụ ghi một tệp cấu hình mới rồi đổi tên tệp để thay bản cũ. Việc lời gọi ghi trả về thành công đã đủ chứng minh tên tệp mới sống sót sau mất điện chưa? Giải thích ranh giới cần kiểm tra.

# Chương 4 — Danh tính và quyền trong Linux

Chẩn đoán một lần mở file bị từ chối qua mọi thư mục cha.

## Bài 4.1 — Danh tính tiến trình

Một tiến trình mang thông tin định danh và chịu kiểm tra quyền khi truy cập tài nguyên. Có quyền đọc một đường dẫn không đồng nghĩa có quyền thực thi tệp, và có quyền thực thi một tệp không đồng nghĩa được đọc mọi tệp mà chương trình sẽ mở sau đó. Nhìn từ bên ngoài, cả hai tình huống có thể hiện thành "ứng dụng không khởi động"; bên trong, điểm lỗi nằm ở những bước khác nhau. Về sau, mô hình này sẽ nối với quyền Linux, IAM và quyền của tài khoản dịch vụ trong Kubernetes.

Tiến trình còn hoạt động trong các giới hạn tài nguyên. Một lần tạo tiến trình con có thể thất bại khi đạt giới hạn số tác vụ; `fork` báo lỗi thay vì bảo đảm thành công. Vì vậy, một bộ điều phối không thể chữa một chương trình tạo tiến trình vô hạn chỉ bằng cách tự khởi động lại nó. Tự phục hồi là chính sách phản ứng với lỗi; ngăn lỗi lặp lại đòi sửa nguyên nhân hoặc đặt giới hạn thích hợp.

Lưu ý rằng quyền truy cập và khả năng nhìn thấy tài nguyên là hai câu hỏi khác nhau. Một namespace có thể cho tiến trình một góc nhìn riêng về một loại tài nguyên của hệ thống. Nhưng góc nhìn riêng không tự chứng minh rằng tiến trình an toàn hay không thể tác động đến thứ bên ngoài ranh giới đó. Ở phần container, ta sẽ kết hợp namespace với giới hạn tài nguyên, quyền và chính sách bảo mật thay vì xem "cách ly" là một thuộc tính nhị phân.

Trên Linux, cần hỏi *tiến trình đang mang credential nào*, không chỉ hỏi người gõ lệnh là ai. Real UID ghi nhận danh tính gốc, còn effective UID thường quyết định quyền ở nhiều giao diện; kiểm tra quyền tệp trên Linux dùng filesystem UID/GID, thông thường trùng effective ID, cùng các supplementary groups. Một chương trình đổi danh tính khi khởi chạy có thể vì thế đọc được tài nguyên mà shell gọi nó không đọc được; chiều ngược lại cũng có thể xảy ra sau khi chương trình hạ quyền. Thao tác này vẫn chịu ACL, capability và các chính sách khác; không suy mọi quyền chỉ từ một số UID.

Ví dụ giả định: tệp cấu hình thuộc `root:atlas`, mode `0640`; tiến trình có filesystem UID 1001, filesystem GID 1001 và supplementary group `atlas`. Nếu mọi thư mục cha cho phép tìm kiếm, bit đọc của nhóm cho phép mở tệp; cùng tiến trình sau khi mất membership `atlas` có thể nhận `EACCES` dù đường dẫn và mode tệp không đổi. Khi điều tra, ghi UID/GID và nhóm *của PID đang lỗi* qua `/proc/<pid>/status`, rồi đối chiếu owner, mode, ACL và quyền đi qua từng thư mục. `id` chạy từ shell của người vận hành chỉ mô tả shell ấy, không chứng minh credential của dịch vụ.

### Bài 4.2 — Bước đi qua thư mục

Với chế độ quyền Linux thông thường, mỗi nhóm bit đọc, ghi, thực thi áp cho chủ sở hữu, nhóm và người khác. Trên tệp thường, bit `x` cho phép thực thi theo các điều kiện của định dạng tệp; trên thư mục, bit `x` là quyền tìm kiếm/đi qua thành phần tên, không phải "chạy thư mục". Bit `r` của thư mục cho phép liệt kê tên; biết sẵn một tên và tra cứu nó cần quyền tìm kiếm. Khi tạo hoặc xóa tên trong thư mục, quyền ở thư mục có vai trò riêng, không thể suy chỉ từ bit ghi trên tệp. ACL, mount option, namespace, chính sách bảo mật bổ sung và quyền đặc biệt có thể làm kết quả khác mô hình ba bộ bit cơ bản; mô hình đó là điểm bắt đầu để khoanh vùng, không phải toàn bộ chính sách.

Xét cây `/srv/atlas/config/app.toml`: `app.toml` là `0640`, thuộc `root:atlas`; tiến trình chạy với nhóm `atlas`. Tệp có vẻ đọc được qua bit nhóm, nhưng nếu `/srv/atlas` là `0700 root:root`, tiến trình không thể đi qua thư mục này và thao tác mở có thể trả `EACCES`. Cách sửa đúng phụ thuộc hợp đồng truy cập: có thể cấp quyền tìm kiếm tối thiểu trên thư mục cho nhóm phù hợp, hoặc đặt cấu hình ở một cây do dịch vụ được phép đi qua. Đổi `app.toml` thành `0666` không sửa được đường đi, lại mở rộng người có thể đọc và sửa bí mật trong tệp. Không dùng `chmod -R 777` như cách chẩn đoán.

Để kiểm chứng trên máy Linux thử nghiệm do mình kiểm soát, ghi nhận danh tính tiến trình bằng `id`, xem từng thành phần bằng `namei -l /srv/atlas/config/app.toml`, rồi thử đọc dưới đúng tài khoản dịch vụ, chẳng hạn `sudo -u atlas cat /srv/atlas/config/app.toml`. Trước khi thay quyền, xác nhận các đường dẫn và tài khoản thực sự tồn tại; các tên trong ví dụ là giả định. `namei` giúp nhìn chuỗi quyền nhưng kết quả đọc cuối cùng mới kiểm tra đúng tổ hợp tài khoản, nhóm, ACL và mount đang có hiệu lực. Nếu unit cấp thêm sandbox hoặc thay root directory, phải kiểm tra từ ngữ cảnh của dịch vụ thay vì chỉ dùng một shell bên ngoài.

Một thay đổi nhỏ giúp thấy ba phép kiểm độc lập. Giữ `app.toml` ở `0640 root:atlas` nhưng đổi `/srv/atlas` từ `0700 root:root` sang `0710 root:atlas`: tiến trình thuộc nhóm `atlas` có thể đi qua thành phần này nếu các thư mục còn lại cũng cho phép, dù không liệt kê được tên trong `/srv/atlas`. Nếu đổi tệp cuối thành `0600 root:atlas`, cùng tiến trình lại không đọc được; `x` ở thư mục không cấp `r` ở tệp. Ngược lại, một tệp `0666` sau thư mục không cho đi qua vẫn không mở được theo đường dẫn ấy. Hãy dự đoán từng kết quả trước khi thử trên fixture; mỗi lần chỉ đổi một mode, giữ UID/GID và đường dẫn cố định.

Khi `namei -l` trông đúng mà service vẫn báo `EACCES`, cần mở rộng giả thuyết thay vì chmod rộng hơn. Xác định credential hiệu lực của tiến trình và các nhóm bổ sung, kiểm ACL ở tệp và thư mục, mount namespace/root directory của unit, sau đó đọc log từ cơ chế bảo mật bổ sung nếu hệ có dùng. `id` chạy trong shell của người điều tra không nhất thiết là credential của service; `sudo -u` cũng không tái tạo mọi sandbox của unit. Hậu điều kiện là đúng tiến trình mục tiêu đọc được đúng tệp cần thiết, trong khi người dùng không được phép vẫn bị chặn. Quyền tối thiểu là một đặc tính cần kiểm chứng bằng cả phép thử cho phép lẫn phép thử từ chối.

### Bài tập 4.1 — Quyền đi qua thư mục cha

Tệp cấu hình có mode `0640 root:atlas`, tiến trình thuộc nhóm `atlas`, nhưng thư mục cha `/srv/atlas` có mode `0700 root:root`. Vì sao thao tác đọc có thể bị từ chối? Nêu một thay đổi tối thiểu cần kiểm tra và một thay đổi không nên dùng.

### Bài 4.3 — Điều tra và sửa

Ở bài trước, tệp cấu hình đọc được theo bit nhóm nhưng tiến trình không đi qua được thư mục cha. Phép sửa cần xuất phát từ lời gọi bị từ chối, chứ không xuất phát từ một ảnh chụp `ls -l` của tệp đích. Trước hết ghi đường dẫn mà ứng dụng thật sự mở, loại thao tác và mã lỗi; một đường dẫn tương đối còn phụ thuộc thư mục làm việc hoặc descriptor mà `openat` dùng. Sau đó xác định UID dùng để kiểm quyền tệp, GID chính và các nhóm phụ của tiến trình ở thời điểm lỗi. Tài khoản người vận hành trong terminal không tự đại diện cho tiến trình do systemd tạo.

Kernel lần lượt tra từng thành phần đường dẫn. Thiếu quyền tìm kiếm trên một thư mục trung gian có thể trả `EACCES` trước khi kernel xét bit đọc của tệp cuối. Một symbolic link có thể chuyển phép tra sang cây khác; mount namespace hoặc root directory của dịch vụ cũng có thể làm cùng chuỗi ký tự trỏ tới đối tượng khác. Vì vậy, phép chẩn đoán phải đọc cả cây, owner/group/ACL và ngữ cảnh chạy. Chỉ khi xác định được điểm chặn mới sửa một điều kiện nhỏ nhất, rồi thử lại chính thao tác đó dưới cùng danh tính. Sửa theo giả thuyết mà không thử hậu điều kiện có thể vừa để lỗi còn nguyên vừa mở rộng quyền ngoài ý muốn.

Tình huống giả định: `/srv/atlas` là `0700 root:root`, `/srv/atlas/config` là `0750 root:atlas`, còn `app.toml` là `0640 root:atlas`. Tiến trình thuộc nhóm `atlas` vẫn dừng ở `/srv/atlas`. Nếu chính sách cho phép nhóm này đi qua nhưng không liệt kê cây gốc, có thể cân nhắc cấp quyền tìm kiếm hẹp trên đúng thư mục đó hoặc chuyển tệp đến cây được quản lý cho dịch vụ; lựa chọn cuối cùng còn phụ thuộc ACL và chính sách bảo mật của máy. Chuyển cả cây thành `0777` hoặc chạy dịch vụ với quyền root phá ranh giới least privilege mà không chứng minh nguyên nhân. Nếu phép thử đọc thành công nhưng ứng dụng vẫn báo lỗi, quay lại kiểm đường dẫn, tài khoản và thời điểm thực tế thay vì tiếp tục tăng quyền.

Một bằng chứng đủ tốt ghi trạng thái trước, thay đổi được duyệt và trạng thái sau: lỗi `EACCES` ở phép mở nào, danh tính nào, quyền tra cứu ở thành phần nào, rồi cùng phép mở đã thành công hay chưa. Nó không khẳng định mọi yêu cầu ứng dụng đã khỏe; kiểm tra readiness và request ở bài vận hành host vẫn cần riêng.

### Bài tập 4.2 — Tìm điểm gây lỗi EACCES

Dịch vụ thuộc nhóm atlas vẫn nhận EACCES với cây 0700 root:root /srv/atlas, 0750 root:atlas /srv/atlas/config, 0640 root:atlas app.toml. Chỉ ra điểm chặn, cách kiểm lại và vì sao chmod 777 cho tệp không giúp.

# Chương 5 — Tài nguyên và chẩn đoán hiệu năng Linux

Phân biệt CPU runnable/quota, memory reclaim và I/O wait bằng counter.

## Bài 5.1 — CPU và bộ lập lịch

Một yêu cầu HTTP mất hai giây không có nghĩa CPU đã làm việc trong hai giây. Trong khoảng ấy, tiến trình có thể đợi lịch chạy, đợi ổ lưu trữ, đợi mạng hoặc chờ khóa do chính chương trình giữ. Nếu không phân biệt những khoảng thời gian này, ta rất dễ tăng số CPU cho một dịch vụ đang nghẽn ở dữ liệu, hoặc tăng bộ nhớ cho một tiến trình đang bị giới hạn thời gian CPU. Chương này xây dựng một cách phân rã độ trễ từ máy tính trước khi dùng công cụ giám sát.

CPU chỉ thực hiện lệnh của một luồng tại một thời điểm trên mỗi đơn vị xử lý logic. Khi nhiều luồng cùng sẵn sàng, bộ lập lịch chọn luồng nào được chạy tiếp. Một luồng có thể đang chạy, sẵn sàng nhưng chưa được chọn, hoặc không sẵn sàng vì đang chờ một sự kiện. Trạng thái "chờ" không đồng nhất: chờ dữ liệu mạng khác với chờ khóa, và cả hai khác với việc sẵn sàng nhưng bị giới hạn quota CPU. Những lý do này đòi cách sửa khác nhau.

Lập lịch công bằng là một mục tiêu chính sách, không phải lời hứa mỗi tiến trình luôn nhận đúng một phần CPU cố định. Chính sách, độ ưu tiên, số tác vụ sẵn sàng, ràng buộc nhóm và phiên bản nhân đều ảnh hưởng kết quả. Tài liệu Linux mô tả quá trình chuyển từ CFS sang EEVDF ở các nhân mới; cuốn sách này giữ mô hình chung "tác vụ sẵn sàng được chọn theo chính sách" ở phần nền tảng, và chỉ bàn thuật toán cụ thể khi có phiên bản nhân làm mốc.

Để phân biệt, hãy đặt câu hỏi theo chuỗi. Công việc có sẵn sàng chạy nhưng không nhận CPU không? Bộ nhớ nào đang tăng, và đó là cache có thể thu hồi hay dữ liệu hoạt động? Có thao tác I/O nào bị chặn? Giới hạn nào áp cho tiến trình, container hoặc cả máy? Nếu quan sát cho thấy CPU bị giới hạn theo quota, tăng số replica có thể chia tải, nhưng cũng có thể làm áp lực lên phụ thuộc chung. Nếu quan sát cho thấy ứng dụng chờ cơ sở dữ liệu, thêm CPU cho nó không loại bỏ thời gian chờ ấy.

Những câu hỏi trên là mô hình phân tích, không phải một runbook tự động. Chúng chuẩn bị cho các chương sau về đo lường, HPA và SLO. Trước khi dùng một metric để tự thay đổi số bản sao, phải biết metric đại diện cho tài nguyên nào, cửa sổ thời gian nào và điều gì xảy ra khi phép đo bị thiếu. Một bộ điều khiển ra quyết định từ tín hiệu sai vẫn có thể hoạt động đúng theo thuật toán của nó nhưng tạo kết quả vận hành xấu.

Muốn kiểm tra giả thuyết "thiếu CPU", lấy hai ảnh chụp `cpu.stat` và `cpu.pressure` của đúng cgroup, cách nhau cùng một khoảng thời gian, đồng thời ghi độ trễ yêu cầu. `usage_usec` là thời gian CPU tích lũy, còn `nr_throttled` đếm các kỳ cgroup bị giới hạn; cả hai phải lấy hiệu, không so một số tích lũy với phần trăm tức thời của `top`. Dòng `some total` của CPU pressure đếm thời gian có ít nhất một tác vụ sẵn sàng nhưng phải đợi CPU. Nó là thời gian có áp lực, không phải CPU time của ứng dụng và không xác định riêng tác vụ gây ra một request chậm.

Ví dụ giả định trên cửa sổ 10 giây: `usage_usec` tăng 3.000.000 µs, `nr_throttled` không tăng, còn CPU `some total` tăng 2.400.000 µs. Mức dùng trung bình là 0,3 lõi; có áp lực chờ CPU trong khoảng 24% cửa sổ. Không thể kết luận quota đã chặn workload vì counter throttle không đổi; cũng

không thể kết luận CPU không liên quan chỉ vì 0,3 lõi nhỏ. Cần kiểm số tác vụ runnable, cgroup cha, CPU được phép dùng và timeline của request. Nếu ở cửa sổ sau `nr_throttled` tăng cùng lúc `throttled_usec` tăng, giả thuyết quota có bằng chứng mạnh hơn, nhưng vẫn phải đối chiếu độ trễ đúng thời điểm. Đây là phép tính trên counter giả định, không phải một lần đo của host hiện tại.

### Bài 5.2 — Bộ nhớ

Mỗi tiến trình nhìn thấy một không gian địa chỉ ảo. Chương trình dùng các địa chỉ trong không gian ấy; phần cứng và nhân phối hợp ánh xạ chúng tới trang bộ nhớ vật lý khi cần. Hai tiến trình có thể dùng cùng một giá trị địa chỉ ảo mà không mặc nhiên đọc cùng dữ liệu. Điều này tạo ranh giới bảo vệ và cho phép quản lý bộ nhớ linh hoạt hơn một sơ đồ "biến nằm ở ô RAM cố định".

Một vùng địa chỉ được dành cho tiến trình chưa nhất thiết đã có một trang RAM riêng. Chẳng hạn, vùng ánh xạ ẩn danh có thể được tạo trước, còn trang vật lý được cấp khi truy cập hoặc ghi phù hợp. Khi một địa chỉ chưa có ánh xạ vật lý cần thiết, phần cứng báo page fault để nhân xử lý. Page fault không tự đồng nghĩa lỗi chương trình: nhiều fault là một phần của cấp phát theo nhu cầu. Lỗi thật xuất hiện khi truy cập không hợp lệ hoặc không thể đáp ứng điều kiện ánh xạ.

Vì vậy, một con số "bộ nhớ ảo" lớn không đủ để kết luận sắp hết RAM. Ngược lại, nhìn một con số bộ nhớ của riêng tiến trình cũng chưa cho biết áp lực bộ nhớ toàn máy: trang có thể được chia sẻ, nằm trong cache, hoặc chịu quy tắc tính của nhóm tài nguyên. Khi chẩn đoán, cần xác định phép đo đang đếm địa chỉ, trang cư trú, trang dùng chung hay bộ nhớ tính cho một cgroup. Nhầm tên đại lượng dẫn tới tăng giới hạn sai chỗ.

Giả sử một cgroup tăng `memory.current` từ 400 lên 600 MiB trong một phút, còn latency đọc tệp giảm. Nếu `memory.stat` cho thấy phần tăng chủ yếu ở `file`, một khả năng là page cache được làm ấm; chưa có căn cứ gọi đó là "memory leak". Nếu phần `anon` tăng đều theo số request rồi không giảm sau khi tải dừng, giả thuyết dữ liệu sống trong ứng dụng đáng kiểm hơn, nhưng vẫn phải xét allocator và thời điểm thu hồi. Với cả hai trường hợp, so `memory.events` (`high`, `max`, `oom`, `oom_kill`), `memory.pressure` và giới hạn ở cgroup tổ tiên trong cùng cửa sổ. Một snapshot `memory.current` ở cuối kỳ không cho biết đường tăng trước đó hay request nào gây ra nó.

Phép thử tại môi trường tạm có thể đọc lại cùng một tệp thử hai lần, đo latency từng lần và lấy delta bộ đếm I/O/cache giữa các lần; không dùng tệp dữ liệu production hay chủ ý ép máy thiếu RAM. Nếu lần sau nhanh hơn nhưng `memory.pressure` và `oom` không tăng, dữ kiện phù hợp với cache hữu ích hơn là thiếu RAM. Nó vẫn không chứng minh mọi lần đọc sau sẽ nhanh: cache có thể bị thu hồi và tải từ tiến trình khác làm thay đổi kết quả.

#### Bài tập 5.1 — Bộ nhớ ảo và bộ nhớ cư trú

Một tiến trình có không gian địa chỉ ảo lớn nhưng bộ nhớ cư trú thấp. Vì sao chưa thể kết luận nó đang chiếm gần hết RAM? Cần phân biệt ít nhất hai đại lượng nào?

## Bài 5.3 — I/O và độ bền

Một request ghi dữ liệu có thể trả nhanh khi kernel còn giữ trang dirty, trong khi writeback xuống thiết bị diễn ra sau đó. Cũng có thể request đọc bị chậm vì trang chưa ở cache, hoặc một luồng bị chặn bởi hàng đợi thiết bị. Cần phân biệt ba đại lượng: byte ứng dụng muốn đọc/ghi, byte kernel thực sự chuyển tới thiết bị và thời gian một tác vụ phải chờ. Chúng không nhất thiết tăng cùng lúc. `fsync` đặt thêm ranh giới: nó yêu cầu đồng bộ dữ liệu và metadata cần thiết của tệp, rồi trả lỗi nếu không thực hiện được; khi tạo tên mới còn phải xét thư mục chứa tên ấy.

Xét dữ liệu tính toán giả định trong cửa sổ mười giây, không phải output của máy đã chạy. Số byte ghi của ứng dụng tăng 120 MiB, số byte ghi của thiết bị tăng 40 MiB, lượng trang dirty tăng 80 MiB, và p95 latency của thao tác ghi tăng từ 6 lên 90 ms. Chênh lệch 80 MiB phù hợp giả thuyết writeback chưa theo kịp, nhưng không chứng minh tất cả 80 MiB thuộc ứng dụng này: máy còn tác vụ khác, phép đo có thể ở các thời điểm khác và các counter không cùng ranh giới kế toán. Nếu `fsync` của cùng request tăng thời gian chờ, giả thuyết thiết bị hoặc filesystem cần kiểm tiếp; nếu không có `fsync`, latency HTTP tăng có thể nằm ở khóa, CPU hoặc phụ thuộc mạng. Chỉ nhìn CPU idle không phân biệt được các đường đó.

Một quy trình đo có sức phân biệt lấy delta counter cùng cửa sổ, chỉ rõ process, cgroup và thiết bị, rồi ghép request ID với thời điểm gọi I/O. Trên Linux có thể xem `Dirty` và `Writeback` trong `/proc/meminfo` để thấy trạng thái toàn máy, nhưng đây không phải số byte của riêng request. Counter block I/O và pressure stall cho biết áp lực theo ranh giới đo của chúng; chúng không tự chỉ ra tệp hay câu lệnh SQL thủ phạm. Trong bài thử cục bộ, tạo một tệp riêng trên filesystem tạm có đủ dung lượng, đo trước và sau, đồng bộ tệp rồi đọc lại; không dùng môi trường lab hoặc dữ liệu thật để ép I/O.

Kết luận chẩn đoán phải ghi cả phép phủ định. Nếu dữ liệu đã đọc từ cache, không quy độ trễ đọc cho ổ chỉ vì ổ đang bận. Nếu `fsync` thành công, vẫn phải xét thư mục cho tên mới và đặc tính thiết bị trước khi tuyên bố phục hồi sau mất điện. Bài 1.3 đã đặt ranh giới bền vững; ở đây các counter giúp tìm nơi nào trong đường đi ấy cần kiểm chứng thực nghiệm.

### Bài tập 5.2 — Dữ liệu ghi và độ trễ I/O

Trong 10 giây, ứng dụng báo ghi 120 MiB, thiết bị báo ghi 40 MiB, Dirty tăng 80 MiB và latency tăng. Nêu một giả thuyết phù hợp, một phép đo bổ sung và điều chưa thể quy cho request.

## Bài 5.4 — Điều tra đa tín hiệu

Xét một dịch vụ đọc tệp trong một cgroup v2. Bảng sau là dữ liệu giả định để tập tính và phản biện, không phải đầu ra đo từ máy hay bằng chứng cho một sự cố có thật. Hai cửa sổ đều dài 60 giây, có cùng số yêu cầu và cùng giới hạn tài nguyên. Các giá trị `usage_usec`, `throttled_usec` và `io.pressure some total` là mức tăng của counter tích lũy giữa đầu và cuối mỗi cửa sổ; `memory.current` là giá trị ở cuối cửa sổ. Người đo phải lấy đúng cgroup chứa workload, cùng ranh giới thời gian với số liệu yêu cầu, và kiểm tra các tệp này có sẵn trên nhân đang chạy.

| Quan sát trong 60 giây | Bình thường | Độ trễ tăng |
|---|---|---|
| p95 yêu cầu | 120 ms | 870 ms |
| `cpu.stat usage_usec` tăng | 18.000.000 µs | 19.200.000 µs |
| `cpu.stat nr_throttled` tăng | 0 | 0 |
| `memory.current` cuối kỳ | 310 MiB | 315 MiB |
| `memory.events high / oom` tăng | 0 / 0 | 0 / 0 |
| `io.pressure some total` tăng | 1.200.000 µs | 18.000.000 µs |

Chia CPU time cho 60 giây cho ta mức dùng trung bình 0,30 rồi 0,32 lõi, không phải 30% rồi 32% độ trễ của từng yêu cầu. `nr_throttled` không tăng nên trong hai cửa sổ này chưa có dấu hiệu cgroup đó bị giới hạn bởi quota CPU; việc luồng phải đợi lịch chạy vì tác vụ khác vẫn có thể xảy ra. Bộ nhớ cuối kỳ tăng 5 MiB và không có sự kiện `high` hay `oom`, nên không có dấu vết vượt hai ngưỡng ấy trong counter đã xem; điều đó không loại trừ thu hồi bộ nhớ toàn máy, page fault hoặc một ngưỡng ở cgroup tổ tiên. Muốn xét áp lực bộ nhớ, đọc thêm `memory.stat`, `memory.pressure` và giới hạn thực tế theo cùng cửa sổ.

Với I/O, `some total` tăng 1,2 giây rồi 18 giây: lần lượt khoảng 2% và 30% của mỗi cửa sổ có ít nhất một tác vụ trong nhóm bị nghẽn I/O. Đây là dấu vết mạnh hơn cho giả thuyết chờ I/O trong cửa sổ chậm, nhưng nó không nói yêu cầu nào chờ, chờ thiết bị nào, hay I/O là nguyên nhân duy nhất của p95 tăng. Bước kế tiếp là ghép thời điểm yêu cầu chậm với thời gian thao tác đọc, bộ đếm bytes/thao tác và tình trạng thiết bị hoặc phụ thuộc tương ứng; cũng phải xét khóa trong ứng dụng và mạng. Nếu đó là đọc từ page cache, số bytes ở thiết bị có thể không phản ánh số lần ứng dụng đọc. Kết luận đúng ở giai đoạn này là ưu tiên kiểm tra đường I/O, chưa phải "ổ đĩa hỏng" hoặc "thêm CPU sẽ hết chậm".

### Bài tập 5.3 — Đọc nhiều tín hiệu tài nguyên

Dùng số liệu giả định ở Bài 5.4: tính mức dùng CPU trung bình và tỷ phần thời gian `io.pressure some` của hai cửa sổ; nêu dấu vết cho hoặc chống lại ba giả thuyết quota CPU, giới hạn bộ nhớ và chờ I/O. Chỉ ra một điều bảng chưa thể kết luận và phép đo kế tiếp.

# Chương 6 — systemd và vòng đời dịch vụ

Tách start thành công khỏi listener, readiness và dữ liệu đúng.

## Bài 6.1 — Đồ thị đơn vị systemd

Dịch vụ là một hợp đồng vận hành đặt lên tiến trình: khởi động khi nào, cấu hình từ đâu, báo trạng thái ra sao, dừng theo tín hiệu nào, có khởi động lại không và dữ liệu bền vững nằm ở đâu. Không phải mọi tiến trình đều là dịch vụ. Một công cụ chạy xong rồi thoát có vòng đời khác một HTTP server tồn tại lâu và tiếp nhận yêu cầu mới.

Khi một tiến trình thoát, hệ điều hành có thể ghi nhận mã kết thúc. Mã này cho biết chương trình kết thúc theo quy ước nào; tự nó chưa cho biết nguyên nhân gốc. Cùng một mã khác không có bảo đảm diễn giải giống nhau giữa mọi chương trình. Một hệ thống giám sát tốt cần xem thêm thời điểm, stderr, tình trạng tài nguyên và sự kiện từ bộ quản lý dịch vụ. Đọc mỗi trạng thái "đã dừng" rồi suy ra lỗi mạng, lỗi quyền hay thiếu bộ nhớ là bước nhảy thiếu chứng cứ.

Một dịch vụ cũng có thể còn tiến trình nhưng không còn khả năng làm việc có ích. Vòng lặp bị kẹt, hàng đợi đầy hoặc phụ thuộc phía sau không phản hồi đều có thể khiến PID vẫn tồn tại. Vì vậy, giám sát sự sống của tiến trình và kiểm tra khả năng phục vụ là hai phép đo khác nhau. Ta sẽ gặp lại khác biệt này khi phân tích healthcheck của Docker, probe của Kubernetes và SLI của SRE.

```
[Unit]
Description=Atlas example service
After=network.target

[Service]
Type=exec
User=atlas
Group=atlas
WorkingDirectory=/opt/atlas
ExecStart=/opt/atlas/bin/atlas --config /srv/atlas/config/app.toml
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

Dòng `After=network.target` trong unit chỉ đặt thứ tự nếu cả hai unit cùng được đưa vào giao dịch khởi động; nó không tự kéo `network.target` vào, cũng không xác nhận máy đã có địa chỉ IP. `Wants=` kéo một unit khác vào với quan hệ phụ thuộc yếu; `Requires=` mạnh hơn và có thể làm khởi động thất bại khi unit được yêu cầu thất bại, nhưng cả hai không tự đặt thứ tự nếu thiếu `After=`. `network.target` là mốc thụ động của ngăn xếp mạng, không phải tín hiệu mạng đã cấu hình đầy đủ. Dịch vụ thật sự cần mạng được cấu hình trước khi bắt đầu có thể dùng `Wants=network-online.target` cùng `After=network-online.target`; ý nghĩa "online" do dịch vụ quản lý mạng xác định và vẫn không bảo đảm máy ở xa hay ứng dụng của nó sẵn sàng. Với dịch vụ chạy lâu dài, còn phải thiết kế cách chịu mất mạng và kết nối lại sau khởi động, thay vì coi thứ tự boot là phép kiểm tra sức khỏe.

Đọc unit mẫu như một hợp đồng có giới hạn. `Type=exec` cho systemd biết khởi động đạt mốc khi thao tác thực thi đã thành công; nó không biết `atlas` đã đọc xong `app.toml`, bind cổng hay xử lý được request. `Restart=on-failure` yêu cầu thử khởi động lại ở những kiểu thất bại thuộc chính sách ấy, nhưng lỗi cấu hình cố định có thể tạo vòng lặp khởi động–chết. `RestartSec=5s` giảm nhịp thử; nó không sửa lỗi. `WantedBy=multi-user.target` chỉ liên quan cách unit được enable vào mục tiêu khởi động, không phải bằng chứng service đã ready. Trình tự suy luận đúng là unit được kích hoạt, lệnh thực thi thành công, process còn sống, endpoint sẵn sàng, rồi đường nghiệp vụ hoạt động.

Xét trường hợp giả định sau deploy: `systemctl start atlas` trả thành công, PID xuất hiện một giây rồi mất, journal ghi lỗi đọc `/srv/atlas/config/app.toml`. Có ít nhất hai giả thuyết: quyền trên đường dẫn như bài trước, hoặc cấu hình không tồn tại trong mount namespace của service. Kiểm log theo đúng invocation, UID/GID của unit, đường dẫn từng thành phần và cách namespace được thiết lập. Đừng tăng restart liên tục hay đổi `After=network-online.target` khi chứng cứ đang chỉ vào file cấu hình. Nếu PID còn sống nhưng request lỗi, chuyển sang listener, readiness và phụ thuộc; đừng gọi đó là “systemd không start được”.

Trong máy thử không có dữ liệu quan trọng, tạo một unit đơn giản trỏ tới chương trình phục vụ cục bộ rồi lần lượt cho lỗi đường dẫn executable, lỗi file cấu hình và ứng dụng phục vụ sai. Ghi trạng thái unit, journal và kết quả request ở từng lần; phục hồi fixture sau thử. Nếu không được phép quản lý unit trên host, có thể giải cùng chuỗi từ log và trạng thái mẫu, nhưng phải ghi đó là suy luận. Hậu kiểm cuối là một request qua đúng đường truy cập cho kết quả mong muốn sau khi điều kiện lỗi được sửa; trạng thái `active` một mình chỉ là một tín hiệu.

### Bài 6.2 — Vòng đời tiến trình dịch vụ

Một unit service mô tả chương trình chính, danh tính chạy, thứ tự phụ thuộc và chính sách khi nó kết thúc. Ví dụ minh họa sau dùng đường dẫn giả định; trước khi cài phải tạo tài khoản, triển khai chương trình và cấu hình đúng trên máy đích. `Type=exec` khiến thao tác khởi động chờ tới khi thực thi được binary; nó bắt được trường hợp executable hoặc user không tồn tại tốt hơn `Type=simple`. Nó vẫn không chứng minh mã ứng dụng đã đọc xong cấu hình, mở socket hoặc sẵn sàng phục vụ.

Trong ví dụ, systemd tạo tiến trình với `User=atlas` và `Group=atlas`, đặt thư mục làm việc ở `/opt/atlas`, rồi thực thi chương trình được chỉ ra ở `ExecStart`. Thư mục làm việc tách khỏi cây cấu hình để lỗi quyền ở `/srv/atlas` xuất hiện khi ứng dụng gọi `openat`, sau bước khởi chạy. Khi ấy nhân vẫn kiểm tra đường dẫn và quyền theo ngữ cảnh đã thiết lập. `Restart=on-failure` yêu cầu thử chạy lại khi tiến trình kết thúc theo điều kiện lỗi được systemd định nghĩa; nó không chữa lỗi quyền cố định. `RestartSec=5s` chỉ tạo khoảng chờ giữa các lần thử và giới hạn tần suất khởi động vẫn có thể chặn vòng lặp lỗi.

Đường lỗi có thể lần theo như sau. Nếu `/opt/atlas/bin/atlas` không tồn tại, bước thực thi thất bại và `Type=exec` báo lỗi khởi động. Nếu binary chạy được nhưng không đi qua `/srv/atlas`, chương trình nhận lỗi khi mở cấu hình và tự thoát với mã lỗi; systemd ghi nhận tiến trình chính đã thoát, áp chính sách restart, rồi có thể chạm giới hạn khởi động. Nếu binary còn chạy nhưng kẹt ở phụ thuộc khác, trạng thái `active` chỉ nói tiến trình được quản lý còn ở trạng thái ấy; nó không phải

healthcheck nghiệp vụ. Mỗi đường lỗi cần xem `systemctl status atlas.service`, `journalctl -u atlas.service` và phép thử khả năng phục vụ của chính ứng dụng. Không lấy riêng trạng thái `active` để tuyên bố đã triển khai thành công.

Khi yêu cầu dừng, systemd gửi tín hiệu theo cấu hình dừng và chờ tiến trình kết thúc trong thời hạn quy định; nếu tiến trình không dừng, chính sách có thể dẫn tới tín hiệu cưỡng bức. Ứng dụng phải xử lý tín hiệu phù hợp, ngừng nhận công việc mới, hoàn tất hoặc hủy công việc đang làm theo hợp đồng, đóng tài nguyên và trả mã kết thúc rõ ràng. `Restart=` mô tả phản ứng sau khi tiến trình mất; nó không bảo đảm dữ liệu được ghi bền vững hoặc yêu cầu đang xử lý được giữ lại. Muốn có ranh giới an toàn, phải thiết kế riêng giao thức dừng và kiểm tra nó dưới tải.

**Bài tập 6.1 — systemctl start đã chứng minh gì?**

Với `Type=exec`, `systemctl start` thành công. Điều này xác nhận được bước nào và chưa xác nhận được bước nào của một HTTP service? Nêu hai quan sát tiếp theo.

### Bài 6.3 — Quan sát dịch vụ

Giả sử sau triển khai, trình duyệt báo không kết nối được tới dịch vụ. Cách suy nghĩ theo lớp bắt đầu từ artifact và tiến trình: bản build đã đến máy chưa; tiến trình có được tạo không; nó còn tồn tại hay đã thoát? Nếu đã thoát, cần xem mã kết thúc và thông báo lỗi. Nếu còn tồn tại, xem nó đã tạo socket lắng nghe đúng địa chỉ và cổng chưa. Sau đó mới xem đường đi mạng và phụ thuộc ở xa. Trật tự này không bảo đảm tìm ra mọi lỗi, nhưng ngăn việc sửa DNS khi chương trình thực ra chưa mở cổng.

Khi kiểm tra, phải phân biệt quan sát với kết luận. “Không thấy PID trong danh sách” là quan sát tại thời điểm đo; “build lỗi” chỉ là một giả thuyết. “Có PID” cũng chưa chứng minh dịch vụ khỏe. Một hồ sơ sự cố tốt ghi mốc thời gian, lệnh hoặc nguồn quan sát, artifact đang chạy, cấu hình có hiệu lực và thay đổi vừa xảy ra. Nhờ thế, người khác có thể phản biện kết luận hoặc tái dựng chuỗi nhân quả thay vì tin vào trí nhớ của người trực hệ thống.

Unit có thể được kiểm tra cú pháp và một phần tham chiếu bằng `systemd-analyze verify` trên tệp unit. Sau khi cài unit vào nơi systemd đọc và chạy `daemon-reload`, cần kiểm tra user/group và đường dẫn đã tồn tại, rồi mới khởi động. Trên máy đang quản lý dịch vụ, quan sát `systemctl show` các thuộc tính như `MainPID`, `ExecMainStatus`, `ActiveState` và `SubState` giúp phân biệt “chưa tạo được process”, “process thoát” và “process còn sống”. Nhật ký đặt các bước vào trục thời gian; cùng một mã thoát chỉ là dấu vết, không phải lời giải thích nguyên nhân.

Một kiểm tra nhỏ nhưng có sức phân biệt là chủ động gây lỗi trong môi trường thử nghiệm: tạm trỏ unit thử tới một binary không tồn tại để quan sát lỗi ở bước exec, rồi trả lại; tiếp đó thử một đường dẫn cấu hình mà tài khoản dịch vụ không đi qua được để quan sát lỗi sau khi process đã khởi chạy. Hai phép thử không nên làm trên dịch vụ production. Chúng dạy cách đọc ranh giới: trường hợp đầu thuộc thiết lập tiến trình của systemd, trường hợp sau là lời gọi tệp trong ứng dụng bị nhân từ chối. Sau mỗi thay đổi phải hoàn nguyên cấu hình thử, kiểm tra trạng thái và dữ liệu; không dùng restart lặp vô hạn làm bằng chứng tự phục hồi.

Ở điểm này, một dịch vụ có bốn câu hỏi kiểm chứng khác nhau: artifact có ở đúng đường dẫn không; tiến trình có được tạo dưới đúng danh tính không; các lời gọi tài nguyên của nó có thành công

không; và người dùng có nhận được chức năng mong đợi không. Chúng lần lượt nối Chương 2, Chương 5 và chương này. Phần mạng sẽ bổ sung đường đi của yêu cầu qua địa chỉ và giao thức, nơi một PID khỏe vẫn có thể không phục vụ được người dùng ở xa.

**Bài tập 6.2 — Phân biệt lỗi khởi chạy dịch vụ**

Phân biệt hai lỗi: binary trong `ExecStart` không tồn tại và binary chạy được nhưng `openat` cấu hình trả `EACCES`. Thành phần nào ghi nhận lỗi đầu tiên, systemd có thể phản ứng ra sao, và vì sao tăng `RestartSec` không giải quyết nguyên nhân?

# Chương 7 — Vận hành máy chủ và phục hồi có kiểm chứng

Đặt thay đổi cấu hình, log rotation và backup vào quy trình có hậu điều kiện.

## Bài 7.1 — Thay đổi máy chủ

Một thay đổi host không chỉ là sửa tệp rồi khởi động lại dịch vụ. Host đang chứa tiến trình, cấu hình, quyền và dữ liệu có vòng đời khác nhau; mỗi bước có một hậu điều kiện riêng. Tình huống xuyên suốt là dịch vụ Atlas đang phục vụ request, nhưng một bản cấu hình mới đổi đường đọc dữ liệu. Người vận hành thấy lỗi mở tệp, trong khi unit vẫn `active`. Trước khi sửa, phải biết lỗi bắt đầu từ revision nào, request nào bị ảnh hưởng, trạng thái dữ liệu và liệu có đường quay lại hay không. Cửa sổ bảo trì dành cho những thao tác có thể ngắt phục vụ hoặc làm đổi dữ liệu, không phải tấm giấy chứng nhận rằng thay đổi chắc chắn an toàn.

Trình tự thực dụng là lưu cấu hình hiện hành và hash, kiểm bản sao có thể đọc, ghi điều kiện dừng, rồi so diff mong muốn với hiện trạng. Kiểm cú pháp unit và cấu hình bằng công cụ của chính ứng dụng nếu có. Xem danh tính mà systemd sẽ dùng, quyền trên toàn đường dẫn và biến môi trường cần thiết; không đưa bí mật vào log kiểm thử. Chỉ áp thay đổi khi đã có cách quay lại tương ứng với kiểu lỗi: hoàn nguyên config cho lỗi config, phục hồi dữ liệu cho lỗi dữ liệu. Một snapshot hạ tầng không thay được bản sao ứng dụng nhất quán nếu dữ liệu đang được sửa trong lúc chụp.

Trong cửa sổ triển khai, ghi thời điểm bắt đầu, người chịu trách nhiệm và các phép quan sát trước/sau. `systemctl restart` thành công chỉ chứng minh systemd đã xử lý yêu cầu theo hợp đồng của unit; tiến trình còn sống vẫn có thể chưa sẵn sàng hoặc đang đọc cấu hình sai. Cần thử request thực đi qua đường người dùng, đọc đúng dữ liệu và kiểm không có lỗi mới trong nhật ký. Nếu một điều kiện dừng đã định bị vi phạm, dừng triển khai trước khi tiếp tục sửa nhiều biến cùng lúc. Việc sửa quyền trên host phải ở mức tối thiểu đã kiểm chứng; chạy lại bằng root để "thử nhanh" làm phép chẩn đoán mất giá trị.

Không có một thứ tự thao tác bất biến cho mọi dịch vụ. Nếu thay đổi gồm schema dữ liệu không tương thích, hoàn nguyên binary/config có thể làm dữ liệu mới không đọc được. Khi ấy kế hoạch phải chỉ rõ phạm vi phục hồi dữ liệu và mức mất dữ liệu chấp nhận được trước khi bắt đầu, thay vì nghĩ rollback ứng dụng luôn đầy đủ.

Lần theo lỗi mở `/srv/atlas/data/orders.db` bằng **đường đi thực của tiến trình**. Giả sử unit đặt `User=atlas`, `/srv` có mode `0755`, `/srv/atlas` thuộc `root:atlas` mode `0750`, tệp `orders.db` thuộc `atlas:atlas` mode `0640`. Theo bit quyền cổ điển, user `atlas` cần quyền search (`x`) trên từng thư mục và quyền đọc tệp; cấu hình này cho phép đọc nếu membership/UID đúng. Nếu `/srv/atlas` đổi thành `root:root` mode `0750`, tiến trình không qua được thư mục dù tệp bên trong vẫn `0640`. Lỗi có thể hiện `EACCES` tại bước phân giải đường dẫn, không phải do bit đọc của tệp cuối. Kiểm danh tính của process, từng thành phần đường dẫn, ACL/LSM và symlink thực tế trước khi sửa. Không giải bằng `chmod -R 777`: việc ấy mở quyền cho nhiều tài khoản và không phân biệt lỗi đường dẫn với lỗi dữ liệu.

Phép thử trước triển khai nên dùng **cùng identity và cùng working directory** với unit: truy cập đúng đường dẫn trong fixture cô lập, kiểm config parser, rồi mới kiểm request. Một shell của

operator chạy bằng root có thể đọc file mà tiến trình `atlas` không thể; kết quả ấy không bác bỏ lỗi quyền. Nếu đường dẫn là symlink sang volume khác, xem cả đích cuối và mount đang gắn, vì đổi quyền ở link không sửa quyền ở các thư mục của đích. Tách kết quả "config parse được", "process mở được file" và "request trả dữ liệu đúng" thành ba hậu điều kiện; dừng ở ranh giới đầu tiên thất bại.

**Bài tập 7.1 — Hậu kiểm sau khi đổi máy chủ**

Atlas vẫn active nhưng request đọc /srv/atlas/data/orders.db lỗi. /srv/atlas đổi thành root:root mode 0750; file orders.db vẫn atlas:atlas mode 0640 và unit chạy User=atlas. Dự đoán điểm lỗi, nêu chứng cứ cần lấy và hai hậu điều kiện sau sửa.

**Bài 7.2 — Dữ liệu trên máy chủ**

Backup có ích khi nó phục hồi được một trạng thái có ý nghĩa cho ứng dụng. Một bản sao thư mục được tạo trong lúc dịch vụ đang ghi có thể chứa tệp A ở thời điểm trước và tệp B ở thời điểm sau, chưa từng cùng tồn tại ở trạng thái ấy. Với dữ liệu giao dịch, phải dùng cơ chế backup của ứng dụng, dừng ghi theo hợp đồng, hoặc snapshot nhất quán của toàn bộ tập dữ liệu cần thiết. Ví dụ PostgreSQL 17: sao chép trực tiếp các tệp dữ liệu bằng công cụ tệp khi server đang chạy không tạo bản backup dùng được; một snapshot nhất quán phải bao gồm dữ liệu liên quan và WAL theo điều kiện tài liệu nêu. Đây là ví dụ về tính nhất quán, không là hướng dẫn chạy backup trên cơ sở dữ liệu thật.

Đặt ba câu hỏi trước khi nhận một archive: nó chụp dữ liệu nào và ở mốc nào; có thể đọc lại từ nơi khác không; cần mất bao lâu và mất tối đa bao nhiêu thay đổi để phục hồi. Hash của archive giúp phát hiện byte đổi sau khi tạo, nhưng không chứng minh dữ liệu bên trong nhất quán hoặc đầy đủ. Phép thử restore trên đường dẫn tạm cô lập phải mở được tệp, kiểm cấu trúc hoặc truy vấn bản ghi đại diện và ghi kết quả. Không phục hồi đè lên dữ liệu đang chạy để "xác nhận nhanh". Nếu không có công cụ ứng dụng hay quyền cần thiết, ghi `SKIP` thay vì coi sự tồn tại của file `.tar` là PASS.

Log rotation là một thay đổi dữ liệu khác: tiến trình có thể giữ descriptor mở tới inode cũ sau khi tên tệp được đổi. Cần biết ứng dụng có hỗ trợ mở lại log khi nhận tín hiệu hay không. Chế độ `copytruncate` giữ tệp gốc rồi xóa phần đã copy, nhưng có khoảng giữa copy và truncate khiến bản ghi mới có thể mất. Do đó chỉ dùng khi chấp nhận rủi ro ấy và kiểm theo phiên bản công cụ; một hệ thống yêu cầu audit log đầy đủ cần cơ chế ghi và luân chuyển phù hợp hơn. Sau rotation, kiểm tiến trình vẫn ghi vào đích dự kiến và các bản lưu còn đọc được; không suy từ việc lệnh rotation trả mã 0.

Trong tình huống Atlas, bản backup trước đổi config phải gắn với cùng `change_id`, thời điểm, phạm vi dữ liệu và phép thử restore. Rotation log trong cửa sổ đó phải giữ đủ chứng cứ chẩn đoán trước/sau; xóa log ngay khi sự cố đang diễn ra có thể làm mất đường suy nguyên nhân.

Để hiểu **tính nhất quán của bản sao**, xét hai tệp giả định: `orders.db` giữ đơn O-7, `outbox.log` giữ sự kiện cần phát cho O-7. Một lệnh copy tuần tự có thể lấy `orders.db` lúc 10:00:00 rồi `outbox.log` lúc 10:00:02; nếu transaction chuyển trạng thái giữa hai thời điểm, cặp tệp sau copy có thể không là một trạng thái từng tồn tại. Hash từng tệp đều đúng vẫn không sửa được mâu thuẫn này. Nếu ứng dụng có giao thức backup nhất quán, dùng giao thức ấy; nếu không, xác định phạm vi cần

tạm dừng ghi và cách kiểm invariant sau restore. Chỉ snapshot một volume khi dữ liệu phụ thuộc nằm trên volume khác cũng có thể tạo cùng vấn đề.

Một bài diễn tập an toàn dùng **bản sao tạm không ghi đè nguồn**: tạo archive theo giao thức phù hợp, chép sang nơi restore tách biệt, xác nhận version/schema, mở bằng đúng account của ứng dụng và đọc O-7; sau đó kiểm outbox không có hai sự kiện trái nhau cho cùng operation ID. Đo thời gian tạo, chuyển, mở và xác minh riêng. Nếu chỉ có thể mở archive nhưng không khởi động được app với dữ liệu đó, kết luận phải là "byte đọc được", chưa phải "khôi phục dịch vụ". Nếu một khóa giải mã ở host gốc không có ở môi trường restore, đánh dấu phụ thuộc key và sửa kế hoạch trước khi dựa vào bản sao ấy cho RTO.

**Bài tập 7.2 — Bản sao tệp khi cơ sở dữ liệu đang ghi**

Một archive copy orders.db lúc 10:00:00 và outbox.log lúc 10:00:02 khi ứng dụng đang ghi; hash hai file đều khớp. Có thể gọi backup nhất quán không? Thiết kế phép kiểm restore an toàn.

### Bài 7.3 — Phục hồi

Phục hồi host bắt đầu từ việc đóng băng chứng cứ vừa đủ, không bắt đầu bằng restart lặp lại. Ghi revision artifact đang chạy, hash cấu hình, UID/GID hiệu lực, lỗi mở tệp, trạng thái dịch vụ, một request thất bại có ID và mốc thời gian. Không đưa nội dung secret vào bản ghi. Sau đó chọn ranh giới sửa nhỏ nhất: nếu chỉ một mode thư mục ngăn đọc, sửa quyền đã duyệt; nếu config mới trỏ sai đường, quay về bản config trước; nếu dữ liệu đã đổi, phải dùng kế hoạch phục hồi dữ liệu riêng. Các hành động này không hoán đổi được.

Ví dụ giả định, trước thay đổi request `r-01` đọc bản ghi `order-7` thành công. Sau đổi config, `r-02` trả lỗi vì tiến trình không đi qua `/srv/atlas`. Hoàn nguyên config khiến `r-03` trả HTTP thành công, nhưng hậu điều kiện chưa hoàn tất nếu `order-7` trả nội dung khác hoặc yêu cầu ghi mới đã mất. Phép kiểm phải đọc lại bản ghi cũ, ghi và đọc bản ghi thử mới trong phạm vi an toàn, xác nhận revision đang phục vụ, rồi so log lỗi theo cùng cửa sổ. ID ví dụ là dữ liệu giả định, không phải output thực nghiệm. Nếu cấu trúc dữ liệu đã thay, cần đánh giá tương thích trước khi chạy binary cũ.

Một bản phục hồi có thể được tuyên bố đạt khi bằng chứng sau sửa đáp ứng hợp đồng đã ghi trước: request thành công, dữ liệu đúng, quyền không bị mở rộng và log không còn đường lỗi ấy. `active` của systemd là một quan sát về vòng đời tiến trình; nó không thay phép thử request. Ngược lại, một request thành công đơn lẻ không chứng minh toàn bộ dữ liệu hoặc mọi đường nghiệp vụ đã phục hồi. Nếu còn thiếu dữ liệu, ghi phạm vi chưa kiểm và dừng ở trạng thái chưa đạt, thay vì sửa câu chữ báo cáo thành PASS.

Giữ bản ghi trước/sau cùng một trục thời gian giúp người khác phân biệt nhân quả với trùng hợp. Nếu thay nhiều biến cùng lúc, khó biết thay đổi nào sửa lỗi hoặc gây tác dụng phụ. Vì vậy, sau mỗi bước phải ghi diff, chạy phép kiểm tương ứng và chỉ tiến tiếp khi hậu điều kiện của bước đó được quan sát. Quy trình này chuẩn bị cho runbook sự cố nhiều dịch vụ ở phần sau, nhưng không biến host đơn lẻ thành toàn bộ hệ thống.

Một **cây quyết định phục hồi** cho Atlas có thể theo ba nhánh. Nếu parser báo config sai trước khi process đọc dữ liệu, hoàn nguyên đúng tệp config đã lưu và xác nhận revision/hash; không thay

quyền hay restore DB. Nếu parser qua nhưng process trả `EACCES` tại `/srv/atlas`, kiểm từng thành phần đường dẫn và identity, sửa đúng ACL/mode/owner đã bị đổi, rồi thử lại bằng user `atlas`. Nếu process mở được nhưng O-7 sai hoặc thiếu, giữ nguyên byte hiện tại làm chứng cứ và chuyển sang quy trình restore/đối soát dữ liệu; restart lặp lại có thể ghi thêm dữ liệu sai. Cây này ưu tiên phép quan sát phân biệt ba nguyên nhân, không tự động chạy ba loại sửa cùng lúc.

Sau một sửa quyền, hậu kiểm phải kiểm **quyền âm**: user phục vụ đọc được dữ liệu cần thiết, nhưng user khác không bỗng đọc được thư mục hay secret. Sau hoàn nguyên config, kiểm request từ client cùng đường định tuyến chứ không chỉ gọi localhost. Sau restore dữ liệu, kiểm một operation cũ và một operation mới với khóa idempotency để thấy hệ có thể tiếp tục ghi mà không nhân đôi tác dụng. Mọi kết quả đều gắn thời điểm và revision. Nếu một nhánh sửa xong mà triệu chứng còn, quay lại giả thuyết và bằng chứng; không nối thêm thay đổi tùy hứng trên cùng cửa sổ sự cố.

**Bài tập 7.3 — Dữ liệu sau hoàn nguyên cấu hình**

Sau hoàn nguyên config, r-03 trả HTTP 200 nhưng order-7 khác trước. Nên tiếp tục restart, sửa quyền hay phục hồi dữ liệu? Nêu ranh giới quyết định và phép xác nhận cuối.

# PHẦN III — Mạng và giao thức ứng dụng

## Một request đi từ tên tới byte phản hồi qua những ranh giới nào?

Mỗi tầng chọn một đích hoặc hợp đồng: route/neighbor, kết nối, bảo mật phiên, ngữ nghĩa HTTP và proxy.

III

**LỘ TRÌNH PHẦN**


- Chương 8 — Địa chỉ, mạng con và đường đi gói IP
- Chương 9 — Tầng vận chuyển: TCP, UDP và NAT
- Chương 10 — Tên miền, TLS và ngữ nghĩa HTTP
- Chương 11 — Reverse proxy và điều tra yêu cầu qua hai chặng

# Chương 8 — Địa chỉ, mạng con và đường đi gói IP

Từ đích IP tìm route và địa chỉ chặng kế tiếp.

## Bài 8.1 — Định danh logic

Một client báo "không kết nối được" chưa chỉ ra lỗi nằm ở đâu. Nó có thể chọn sai đích, không tìm thấy đường ra, không chuyển được gói tới gateway, bị lọc trên đường, hoặc đến đúng máy nhưng không có chương trình lắng nghe ở địa chỉ và cổng ấy. Muốn sửa đúng, ta phải tách địa chỉ IP của đích, địa chỉ ở từng chặng truyền, trạng thái của kết nối TCP và trạng thái mà Linux đang giữ. Chương này dùng IPv4 và Ethernet làm mô hình cụ thể; IPv6 có cơ chế tìm láng giềng và biểu diễn địa chỉ khác, không được suy nguyên xi từ ARP của IPv4.

Địa chỉ IP là định danh logic dùng để chuyển gói qua nhiều mạng. Nó không phải địa chỉ vật lý của card mạng, cũng không phải tên dịch vụ. Một giao diện có thể có nhiều địa chỉ; một máy có thể có nhiều giao diện; cùng một dịch vụ có thể được gọi bằng tên nhưng cuối cùng client vẫn phải chọn một địa chỉ và cổng để thử kết nối. Phần DNS và HTTP ở chương sau mới giải thích đầy đủ bước từ tên tới địa chỉ. Ở đây ta bắt đầu từ đích đã biết là `198.51.100.20:443`.

Giả sử client mang `10.20.1.10/24`. Prefix `/24` nghĩa là 24 bit đầu thuộc phần mạng; trong ví dụ, `10.20.1.10` và `10.20.1.1` cùng prefix `10.20.1.0/24`, còn `198.51.100.20` thì không. Phép tính này chỉ so phần bit theo mask; nó không tự chứng minh hai máy thực sự cùng một miền truyền Ethernet, cũng không cho biết đích có hoạt động. Những địa chỉ `198.51.100.20` và `203.0.113.5` trong chương là địa chỉ dành cho tài liệu, không phải endpoint để độc giả thử trên Internet. Địa chỉ `10.20.1.10` thuộc phạm vi riêng trong ví dụ, được dùng để minh họa một client phía trong gateway.

Linux giữ địa chỉ gán cho giao diện, prefix, trạng thái liên kết và bảng route theo network namespace. `ip address` cho biết cấu hình địa chỉ mà máy đang thấy; nó không chứng minh địa chỉ đã được quảng bá đúng ra mạng hoặc thiết bị phía kia có thể trả lời. Trước khi kết luận "mất mạng", cần kiểm tra client đang chạy trong namespace nào: một container có thể nhìn thấy giao diện và route khác host. Tên giao diện cũng là chi tiết của máy, không phải một hằng số của giao thức.

## Bài 8.2 — Chọn chặng

Với đích ngoài prefix trực tiếp, client tra bảng định tuyến để chọn đường ra và next hop. Trong ví dụ, route mặc định dẫn tới gateway `10.20.1.1` trên giao diện nội bộ. Khi có nhiều route khớp, prefix cụ thể hơn thắng prefix ngắn hơn; route mặc định `/0` chỉ dùng nếu không có đường phù hợp hơn theo các quy tắc chọn route có hiệu lực. Trên Linux, policy routing, nguồn được chọn, bảng route và namespace còn có thể thay đổi kết quả. `ip route get 198.51.100.20` hữu ích vì cho thấy quyết định route đối với một đích cụ thể, gồm gateway và địa chỉ nguồn được chọn trong ngữ cảnh gọi lệnh. Đó là phép xem quyết định ở máy, chưa phải bằng chứng gói đã tới gateway.

Trong bảng route giả định có `198.51.100.0/24 via 10.20.1.254` cùng `default via 10.20.1.1`, gói tới `198.51.100.20` đi qua `.254` vì `/24` cụ thể hơn `/0`. Nếu phép tra cứu vẫn báo `.1`, trước hết kiểm tra lệnh chạy trong đúng network namespace, địa chỉ nguồn và policy rule; không

kết luận gateway `.254` hỏng từ một phép tra cứu sai ngữ cảnh. Sau khi chọn được route, còn phải xác nhận phân giải neighbor cho next hop và quan sát ở hai đầu. Route được chọn, next hop có thể tới và ứng dụng nhận được phản hồi là ba mệnh đề khác nhau.

Đường đi thực có hai bước quyết định. Bảng route chọn next hop và giao diện; trên Ethernet, máy còn cần địa chỉ lớp liên kết của next hop qua ARP hoặc neighbor discovery. Với đích ngoài subnet, frame đầu tiên mang địa chỉ MAC của gateway, còn địa chỉ IP đích vẫn là `198.51.100.20`. Vì thế một entry route hợp lệ không tự tạo được neighbor reachable. Nếu neighbor của `10.20.1.254` chuyển sang `FAILED`, lỗi nằm trước chặng IP tiếp theo; đổi DNS không chữa được. Ngược lại, neighbor reachable chỉ chứng minh khả năng liên lạc cục bộ với next hop tại thời điểm quan sát, chưa chứng minh gateway có route về hoặc firewall phía xa chấp nhận gói.

Xét biến thể thứ hai: ứng dụng bind địa chỉ nguồn `10.20.1.50`, trong khi người điều tra chạy `ip route get` không chỉ rõ nguồn. Policy rule có thể chọn bảng khác cho nguồn ấy, nên hai phép tra cứu trả hai gateway khác nhau mà không mâu thuẫn. Phải ghi namespace, đích, nguồn, giao diện và rule áp dụng cùng một lượt kết nối. Trên Linux có thể đối chiếu `ip rule`, `ip route get 198.51.100.20 from 10.20.1.50` và `ip neigh show 10.20.1.254` trong namespace của workload; đây là hướng kiểm tra, không phải output của hệ thống ví dụ.

Khi client timeout, chia chuỗi bằng chứng theo chặng: quyết định route cục bộ, neighbor của next hop, gói SYN rời giao diện, gói đến máy đích, SYN-ACK đi về và socket ứng dụng. Bắt gói tại cả hai đầu nếu được phép và đồng bộ thời gian trước khi so dấu thời gian. Nếu SYN rời client nhưng không đến server, kết luận mới chỉ khoanh vùng đoạn giữa hai điểm quan sát; không thể quy lỗi chắc chắn cho một firewall cụ thể. Nếu SYN-ACK về đến client mà kết nối vẫn thất bại, quay lại state của socket, lọc gói và đường đi tại client. Mỗi phép đo phải phục vụ một giả thuyết, tránh chụp mọi counter rồi đoán.

**Bài tập 8.1 — Chọn tuyến đến địa chỉ đích**

Một máy có route mặc định và route `198.51.100.0/24` qua hai gateway khác nhau. Với đích `198.51.100.20`, route nào được ưu tiên? Nêu một lý do khiến kết quả thực tế trên Linux vẫn cần kiểm tra bằng `ip route get` trong đúng namespace.

**Bài 8.3 — Đóng gói khung truyền**

Sau khi chọn next hop `10.20.1.1`, client cần địa chỉ lớp liên kết của gateway để gửi frame Ethernet. ARP của IPv4 hỏi “ai giữ địa chỉ IP này?” trên liên kết cục bộ rồi lưu ánh xạ IP–MAC trong bảng láng giềng. Client hỏi MAC của gateway, không hỏi MAC của `198.51.100.20` ở mạng xa. `ip neigh` cho thấy cache láng giềng và trạng thái như `REACHABLE`, `STALE` hoặc `FAILED`; `STALE` không tự nghĩa là mất kết nối vì kernel có thể xác minh lại khi dùng. Một ánh xạ tốt chỉ chứng minh một phần đường cục bộ, không xác nhận route, NAT hay server phía sau.

Router nhận frame ở chặng đầu, bỏ lớp đóng gói liên kết cục bộ, xem địa chỉ IP đích, chọn chặng tiếp theo rồi đóng gói thành frame mới phù hợp với liên kết kế. IP đích thường vẫn là `198.51.100.20` qua các router thông thường, trong khi MAC nguồn và MAC đích đổi theo từng chặng. Nếu router cần chuyển qua nhiều mạng, mỗi chặng tự phân giải láng giềng của nó. Đường trả

lời cũng cần route hợp lệ theo chiều ngược; việc client gửi được SYN không có nghĩa SYN-ACK sẽ quay về được.

Xét một ảnh chụp **giả định** trên client: route tới `198.51.100.20` chọn `via 10.20.1.1 dev eth0`, còn bảng neighbor ghi `10.20.1.1 dev eth0 FAILED`. Ở thời điểm ấy, kernel đã chọn đường IP nhưng chưa có ánh xạ lớp liên kết dùng được cho chặng đầu. Hãy kiểm tra địa chỉ và trạng thái `eth0`, ARP trên đúng liên kết, rồi quan sát xem có request và reply tương ứng hay không. Nếu neighbor chuyển sang `REACHABLE` mà TCP vẫn timeout, giả thuyết lỗi ARP chặng đầu đã yếu đi; bước tiếp theo là kiểm tra gói rời client, đường trả về và bộ lọc ở các chặng. Một lần `FAILED` không chứng minh gateway chết vĩnh viễn: entry có thể đổi trạng thái sau phép thử lại, và ảnh chụp ở namespace khác không mô tả đường của ứng dụng.

Lần theo byte địa chỉ trong một lượt **giả định**: client `10.20.1.10` có MAC `02:00:00:00:01:10`, gateway `10.20.1.1` có MAC `02:00:00:00:01:01`. Khi chưa biết MAC gateway, client phát ARP request trên liên kết với Ethernet destination `ff:ff:ff:ff:ff:ff`, hỏi chủ của `10.20.1.1`; gateway trả lời ánh xạ `10.20.1.1 → 02:00:00:00:01:01`. Client mới đóng gói SYN với IP destination `198.51.100.20` nhưng Ethernet destination là `02:00:00:00:01:01`. MAC này không phải MAC của server và không đi xuyên qua router; router sẽ dùng địa chỉ lớp liên kết khác ở chặng sau. Nếu capture thấy ARP reply nhưng không thấy SYN, kiểm neighbor state và quyết định gửi tại client trước khi đổ lỗi cho server. Nếu thấy SYN rời client, ARP chặng đầu đã hoàn tất cho lần ấy, còn TCP vẫn có thể thất bại ở đường đi hoặc đường về. Các MAC là dữ liệu minh họa, không phải capture đã chạy.

**Bài tập 8.2 — MAC của chặng kế tiếp**

Client `10.20.1.10/24` gửi tới `198.51.100.20`. Route mặc định qua `10.20.1.1`. Client cần tìm MAC của địa chỉ nào trước khi gửi frame đầu tiên? Vì sao không hỏi trực tiếp MAC của server?

**Bài 8.4 — Kích thước đường**

Route chọn chặng kế, nhưng gói còn phải vừa khả năng truyền của từng chặng. MTU là kích thước gói IP tối đa mà một liên kết có thể mang theo cách nó được cấu hình; Path MTU là giới hạn nhỏ nhất trên đường đang dùng. Nó không phải băng thông và cũng không phải kích thước một request HTTP. Một đường ra có MTU 1500 byte vẫn có thể đi qua tunnel làm giới hạn chặng sau thấp hơn. Kết quả `ip route get` cho biết quyết định định tuyến tại nguồn và giao diện được chọn, chưa đo được giới hạn của mọi hop ở chiều đi hoặc chiều về.

Với IPv4, một gói có bit Don't Fragment vượt MTU chặng sau bị router bỏ và trả ICMP báo cần phân mảnh cùng thông tin MTU khi hỗ trợ. Nguồn có thể hạ ước lượng Path MTU rồi gửi gói nhỏ hơn. Nếu ICMP cần thiết bị chặn, nguồn không học được giới hạn; gói nhỏ vẫn qua nhưng gói lớn cứ mất là một kiểu “black hole” có thể quan sát. Cùng triệu chứng cũng có thể do firewall, lỗi tunnel hoặc phụ thuộc ứng dụng, nên không chẩn đoán chỉ bằng việc một ping nhỏ thành công. Nếu cho phép phân mảnh IPv4, các mảnh còn phải được lắp lại; mất một mảnh có thể làm mất cả datagram, vì vậy phân mảnh không phải cách sửa mặc định cho mọi đường.

Ví dụ tính toán, không phải capture thực: liên kết đầu cho phép 1500 byte, tunnel chặng sau chỉ cho 1400 byte. Một gói IPv4 dài 1480 byte với DF đặt có thể rời client nhưng bị bỏ ở chặng 1400. Nếu giả định header IPv4 20 byte và TCP 20 byte không có option, dữ liệu tối đa trong một segment không vượt 1360 byte khi Path MTU là 1400. Có IP/TCP option hoặc encapsulation khác thì con số giảm; MSS được thỏa thuận ở hai đầu và kích thước gói thực còn phụ thuộc cơ chế của stack. Không lấy 1360 làm hằng số cho mọi kết nối.

IPv6 có đường khác: router không phân mảnh gói đang chuyển tiếp; router gửi ICMPv6 Packet Too Big và nguồn điều chỉnh kích thước hoặc tự phân mảnh khi phù hợp. IPv6 yêu cầu mỗi liên kết hỗ trợ MTU ít nhất 1280 byte hoặc xử lý ở tầng dưới để cung cấp giới hạn ấy. Vì thế không áp nguyên xi phép thử DF của IPv4 cho IPv6. Khi điều tra, ghi địa chỉ nguồn/đích, family IP, route từ đúng namespace, kích thước phép thử và capture hoặc counter ở cả hai phía chặng nghi ngờ. Thấy Packet Too Big tới nguồn và lần gửi sau nhỏ hơn là bằng chứng mạnh hơn giả thuyết “MTU lỗi”; không thấy gói ICMP ở một điểm capture không chứng minh nó không tồn tại ở đường khác.

**Bài tập 8.3 — MTU và gói có cờ DF**

Gói IPv4 1480 byte có DF đi qua hop MTU 1400. Dự đoán hành vi router và tính payload TCP tối đa nếu IP/TCP header đều 20 byte, không option. Vì sao ping nhỏ thành công chưa bác giả thuyết PMTU?

# Chương 9 — Tầng vận chuyển: TCP, UDP và NAT

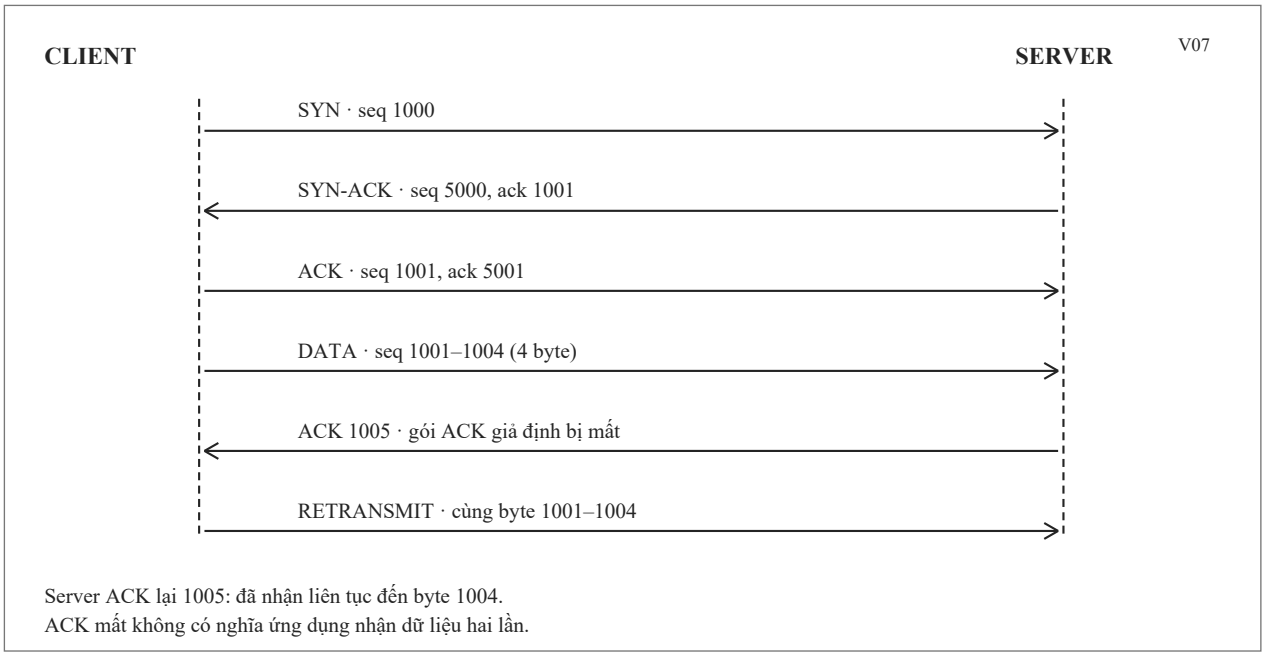
Lần theo SYN, seq/ack, retransmission, datagram UDP và dịch tuple hai chiều.

## Bài 9.1 — Bắt tay và luồng byte

TCP đặt cổng và trạng thái lên trên IP để hai đầu phân biệt các cuộc hội thoại. Với kết nối thường, client gửi SYN từ một cổng nguồn tạm thời tới cổng server, server trả SYN-ACK, rồi client gửi ACK. Các số thứ tự và xác nhận giúp hai đầu đồng bộ trạng thái và nhận biết byte nào đã được chấp nhận. Server phải có socket lắng nghe trên địa chỉ/cổng phù hợp; một tiến trình còn sống hoặc một unit `active` không đồng nghĩa đã có socket lắng nghe ở địa chỉ client đang gọi.

Có thể lần theo một trace tối giản với số khởi đầu giả định. Client gửi `SYN seq=1000`; server đáp `SYN-ACK seq=5000 ack=1001`; client gửi `ACK seq=1001 ack=5001`. Mỗi SYN chiếm một vị trí trong không gian số thứ tự dù không mang dữ liệu. Sau đó client gửi `ACK seq=1001 ack=5001 len=4`, tức các byte được đánh số 1001–1004; server đáp `ACK seq=5001 ack=1005 len=0`, nghĩa là đã nhận liên tục mọi byte trước 1005 và đang chờ byte 1005. Nếu riêng ACK này mất, client có thể truyền lại chính bốn byte với `seq=1001 ack=5001 len=4`; server nhận ra dữ liệu lặp, không giao lần hai cho ứng dụng và lại báo `ack=1005`. ACK là cộng dồn, không phải dấu xác nhận riêng từng packet. Trace bỏ qua TCP option, dữ liệu trong SYN, đảo thứ tự, vòng số thứ tự và cách bộ hẹn giờ quyết định truyền lại; capture ở một điểm cũng không đủ chứng minh vì sao bản truyền lại xuất hiện.

Sau bắt tay, TCP không chỉ "gửi byte rồi hy vọng". Bên gửi giữ dữ liệu chưa được xác nhận để truyền lại khi phát hiện mất; ACK xác nhận tiến độ của luồng byte. Cửa sổ nhận giới hạn lượng dữ liệu bên nhận còn nhận được, còn điều khiển nghẽn giới hạn lượng dữ liệu bên gửi đưa vào mạng theo tín hiệu của đường truyền. Hai giới hạn khác nhau: cửa sổ nhận nhỏ có thể phản ánh ứng dụng đọc chậm, còn điều khiển nghẽn phản ứng với tình trạng mạng theo thuật toán đang dùng. Việc bắt tay thành công không bảo đảm tốc độ truyền tiếp theo. TCP đóng có trạng thái riêng; reset chấm dứt đột ngột, còn đóng thông thường trao đổi FIN và có thể để lại `TIME-WAIT` nhằm xử lý các segment cũ. Không coi mọi socket không còn `ESTABLISHED` là lỗi.




Hình III.2.1 — TCP: sequence đếm byte, ACK cộng dồn; số liệu giả định.

### Bài tập 9.1 — ACK cộng dồn và truyền lại

Trong trace Bài 9.1, vì sao SYN của client được đáp bằng `ack=1001`, còn bốn byte ở `seq=1001` được đáp bằng `ack=1005`? Nếu ACK đầu cho dữ liệu bị mất, client truyền lại cùng bốn byte thì server nên báo số ACK nào, và vì sao ứng dụng không nhận dữ liệu hai lần?

### Bài 9.2 — Socket và lỗi

Một kết nối được nhận diện bởi giao thức và bộ địa chỉ, cổng ở hai đầu. `SYN-SENT` ở client cho biết nỗ lực bắt tay chưa hoàn tất; `ESTABLISHED` cho biết hai TCP endpoint đã thiết lập trạng thái, chưa chứng minh HTTP, TLS hay ứng dụng trả kết quả đúng. Nếu cổng đích không có người lắng nghe, client có thể nhận reset và `connect` báo `ECONNREFUSED`. Nếu SYN hoặc phản hồi không trở lại, TCP có thể thử lại và cuối cùng báo hết thời gian; `ETIMEDOUT` chỉ là triệu chứng, không tự chỉ ra firewall, mất route hay server quá tải. Một firewall `REJECT` cũng có thể tạo phản hồi rõ, còn `DROP` thường làm client chờ; cần xem gói thực tế và chính sách có hiệu lực trước khi quy nguyên nhân.

Đây là tình huống giả định có kết quả quan sát được định nghĩa trước để tập suy luận, không phải log của một hệ thống thật. Client `10.20.1.10/24` cần kết nối `198.51.100.20:443` qua gateway `10.20.1.1`; gateway dịch nguồn sang `203.0.113.5`. Ứng dụng báo `ECONNREFUSED`. Mục tiêu không phải thử mọi lệnh, mà tìm điểm đầu tiên nơi trạng thái quan sát khác đường đi mong đợi.

Trước hết, xác định lời gọi kết nối dùng đúng IP và cổng, rồi xem route trong namespace của client. Giả sử `ip route get` chọn `10.20.1.1` và địa chỉ nguồn `10.20.1.10`; `ip neigh` cho gateway có ánh xạ hợp lệ. Hai quan sát loại bớt giả thuyết thiếu route hoặc chưa tìm được MAC ở client tại thời điểm đo, nhưng không chứng minh gateway đang chuyển tiếp. Một capture ngắn ở giao diện ra của client thấy SYN `10.20.1.10:51514` tới `198.51.100.20:443`; như vậy lỗi không nằm ở việc client chưa phát SYN. Nếu không có quyền quan sát gateway, phải nói rõ khoảng đường giữa hai điểm đo vẫn chưa xác minh.

Trong tình huống này, người vận hành có quyền xem gateway và server. Capture ở phía ngoài gateway thấy SYN với nguồn đã đổi thành `203.0.113.5:40001`. Capture trên server thấy đúng SYN tới `198.51.100.20:443`, rồi một RST từ phía server quay lại. Điều đó cho thấy route đi và ánh xạ ra đã làm việc cho gói này; reset là manh mối mạnh cho việc kết nối bị từ chối tại hoặc gần server. Nó chưa tự phân biệt kernel vì không có listener với firewall chủ động từ chối. Ta kiểm tra đúng namespace của server: `ss -ltn` chỉ thấy `127.0.0.1:443`, không thấy listener trên `198.51.100.20:443` hay địa chỉ bao quát thích hợp. Đồng thời kiểm tra quy tắc lọc có hiệu lực để loại trừ reset do chính sách. Với bộ quan sát giả định này, nguyên nhân trực tiếp là ứng dụng chỉ bind vào loopback; sửa cấu hình bind theo hợp đồng truy cập rồi xác nhận `ss` và bắt tay SYN, SYN-ACK, ACK trên đúng đường. Sau đó mới thử một yêu cầu ứng dụng: bắt tay TCP thành công chưa kiểm tra được TLS hoặc HTTP.

Nếu trace khác đi, kết luận cũng đổi. Không có SYN tại client dẫn ta về địa chỉ đích, route hoặc lỗi cục bộ; SYN rời client nhưng không tới gateway dẫn ta về link, neighbor hoặc lọc tại chặng đầu. SYN tới server mà không có phản hồi cần kiểm tra lọc, trạng thái máy và việc server nhận gói; SYN-ACK rời server nhưng không về client hướng chú ý sang route trả, NAT và lọc theo chiều về. Những nhánh này là giả thuyết có thứ tự, không phải bảng ánh xạ một triệu chứng với một nguyên nhân duy nhất. Capture là ảnh chụp ở một điểm và một thời gian: mất gói khi capture, offload, namespace sai hoặc lưu lượng qua đường khác đều có thể làm nó thiếu bức tranh. Bằng chứng tốt nhất ghép cùng tuple kết nối, thời gian và hai đầu của đoạn đường đang nghi vấn.

`ss -ltn` trên đúng network namespace cho thấy socket TCP đang `LISTEN` và địa chỉ local mà socket bind; `127.0.0.1:443` chỉ nhận kết nối đi vào loopback của chính namespace đó, không nhận gói đến `198.51.100.20:443` qua giao diện ngoài.

**Bài tập 9.2 — SYN-ACK thất lạc trên đường về**

Client thấy SYN rời máy; server thấy SYN và gửi SYN-ACK; client không thấy SYN-ACK. Nêu ít nhất hai điểm cần kiểm tra theo chiều về và giải thích vì sao chưa thể quy lỗi cho ứng dụng HTTP.

**Bài tập 9.3 — Kết nối bị từ chối trên địa chỉ loopback**

`connect` trả `ECONNREFUSED`; server chỉ có `127.0.0.1:443` ở trạng thái `LISTEN`. Hãy nêu giả thuyết mạnh nhất, một nguyên nhân cạnh tranh vẫn phải kiểm tra, và phép xác nhận sau khi sửa. Vì sao `systemctl active` chưa đủ?

**Bài 9.3 — NAT và đối xứng đường**

Để client dùng địa chỉ riêng ra mạng khác, gateway trong ví dụ thực hiện NAPT: nó đổi nguồn `10.20.1.10:51514` thành `203.0.113.5:40001` khi gửi tới `198.51.100.20:443`, đồng thời giữ ánh xạ để dịch gói trả lời về đúng client. Số cổng `40001` chỉ là giá trị giả định; triển khai có thể giữ hoặc đổi cổng theo khả năng và xung đột. Server thấy nguồn sau dịch, không thấy địa chỉ riêng của client ở tầng IP. Nếu không có ánh xạ trả về còn hiệu lực, hoặc gói đáp đi qua một thiết bị khác không biết ánh xạ, phản hồi có thể thất lạc dù SYN ban đầu đã rời client.

NAT không phải định tuyến: route quyết định chặng chuyển tiếp, còn NAT thay địa chỉ hoặc cổng trong gói ở điểm dịch và cập nhật checksum cần thiết. Trạng thái dịch theo luồng của gateway cũng khác trạng thái TCP trong hai endpoint. Linux có thể dùng hệ thống theo dõi kết nối của netfilter để gắn các gói với cùng luồng và áp dụng NAT, nhưng việc thấy một mục theo dõi không tự chứng minh server đã chấp nhận kết nối. Ngược lại, một capture sau NAT có tuple khác capture trước NAT; phải ghi rõ mình quan sát ở giao diện nào và phía nào của điểm dịch trước khi so hai trace. NAT không phải cơ chế bảo mật thay cho lọc gói và chính sách truy cập.

Lần theo tuple hai chiều của ví dụ: client gửi `(10.20.1.10, 51514) → (198.51.100.20, 443)`; phía ngoài gateway thấy `(203.0.113.5, 40001) → (198.51.100.20, 443)`. Phản hồi của server đi tới `(203.0.113.5, 40001)`; gateway phải nhận đúng phản hồi ấy rồi đảo ánh xạ về `(10.20.1.10, 51514)`. Nếu server trả thẳng qua một đường khác không đi qua gateway giữ state, client không nhận được phản hồi đúng luồng. Nếu gateway mất state giữa chừng, các gói tiếp theo cũng có thể không còn được dịch như phiên trước. Đây là trace tính từ cấu hình giả định, không phải packet capture đã thu.

Một lỗi vận hành thường bị gọi chung là "NAT hỏng": SYN rời client, capture ngoài gateway thấy nguồn public, nhưng client timeout. Chia phép quan sát: capture trước và sau gateway để chứng minh chiều đi đã dịch; capture ở server để biết SYN có tới; capture chiều về để biết SYN-ACK có tới gateway và được chuyển vào client. Một entry conntrack chỉ cho thấy gateway ghi nhận luồng theo quy tắc của nó. Nếu capture ngoài thấy SYN-ACK tới nhưng phía trong không thấy, tập trung vào chiều về, state và chính sách tại gateway; nếu server chưa từng thấy SYN, chưa có cơ sở kết tội ánh xạ chiều về. Gắn timestamp và tuple đã chuẩn hóa trước/sau NAT, nếu không rất dễ so nhầm hai kết nối khác nhau.

Trong lab cô lập, có thể dựng client, gateway và server bằng network namespace và địa chỉ tài liệu, đo ở ba điểm trước khi bật NAT rồi sau khi bật NAT. Không cần đụng route host thật. Dự đoán trước tuple mà mỗi điểm sẽ thấy và kiểm trường hợp route phản hồi đi lệch gateway. Sau sửa, cần chứng minh một request ứng dụng hoàn tất, không chỉ có gói SYN hoặc mục conntrack. Chọn NAT vì cần chia sẻ địa chỉ/đường ra, không dùng nó làm lý do bỏ lọc gói hoặc bỏ cấu hình đường về.

**Bài tập 9.4 — NAT và đường trả gói**

Gateway đổi `10.20.1.10:51514` thành `203.0.113.5:40001`. Server trả gói về tuple nào, và gateway cần điều kiện gì để đưa gói về client? Phân biệt trạng thái NAT với trạng thái TCP endpoint.

## Bài 9.4 — UDP có điều kiện

UDP chuyển từng datagram giữa cặp địa chỉ và port; nó giữ ranh giới bản tin ở giao diện gửi/nhận, nhưng không thiết lập phiên như TCP, không tự đánh số để truyền lại và không hứa thứ tự hay giao hàng. Checksum giúp phát hiện hỏng dữ liệu trong phạm vi giao thức; nó không biến UDP thành cơ chế xác thực nguồn. Một ứng dụng cần timeout, retry, chống gửi trùng hoặc kiểm soát tải phải định nghĩa những hành vi ấy ở tầng của mình. QUIC chạy trên UDP nhưng cơ chế tin cậy và kiểm soát tắc nghẽn là của QUIC, không được quy ngược thành khả năng của UDP.

Xét một truy vấn DNS giả định: client `10.20.1.10:53000` gửi datagram tới resolver `10.20.1.53:53`. Thấy datagram rời client chỉ chứng minh nó đã tới điểm quan sát ấy. Nếu resolver nhận nhưng không trả, có thể nó đang đợi upstream hoặc bỏ yêu cầu theo chính sách; nếu nó trả từ `10.20.1.53:53` mà client không nhận, kiểm route và lọc gói chiều về. Một timeout ở client không phân biệt được hai trường hợp đó. ID của DNS và tuple giúp ghép yêu cầu với phản hồi, nhưng NAT có thể đổi port ở giữa; phải ghi phía trước hay sau điểm dịch. Retransmission của ứng dụng có thể tạo thêm datagram cùng nội dung; bộ đếm gói tăng không tự là bằng chứng resolver xử lý hai giao dịch độc lập.

Kích thước vẫn quan trọng. Một DNS response lớn có thể bị cắt ngắn theo cơ chế DNS và client dùng đường khác theo hợp đồng của phần mềm; một datagram UDP bị phân mảnh ở IP lại chịu rủi ro mất mảnh. Không đánh đồng "UDP nhanh" với "luôn ít độ trễ": nếu phải retry sau timeout, wall time có thể cao hơn đường có thiết lập kết nối. RFC hướng dẫn ứng dụng UDP xét kích thước, tắc nghẽn, retry và phát hiện lỗi; lựa chọn thực tế còn phụ thuộc giao thức tầng trên.

Trên IPv6, UDP checksum thông thường là bắt buộc, với ngoại lệ được định nghĩa riêng cho một số đường tunnel; IPv6 Neighbor Discovery thay ARP để tìm láng giềng cùng liên kết. Địa chỉ IPv6, route IPv6, firewall IPv6 và ICMPv6 Packet Too Big phải được quan sát như một đường riêng. Một lần thử IPv4 thành công không chứng minh IPv6 hoạt động, ngay cả khi hai tên DNS cùng trỏ tới một dịch vụ. Khi chẩn đoán, so hai phía bằng địa chỉ family, port, thời điểm và ID giao thức phù hợp; kết luận "mất ở đâu" chỉ tới ranh giới giữa điểm thấy gói cuối cùng và điểm không thấy gói kế tiếp, có ghi giới hạn capture.

**Bài tập 9.5 — Phản hồi DNS không về client**

Client thấy DNS datagram đi, resolver thấy request và gửi response, nhưng client timeout. Điều gì đã được loại trừ, điểm nào cần quan sát tiếp, và vì sao retry không chứng minh UDP tự truyền lại?

# Chương 10 — Tên miền, TLS và ngữ nghĩa HTTP

Tách lỗi DNS, TCP, chứng chỉ và HTTP bằng dữ kiện tối thiểu.

## Bài 10.1 — DNS và bộ nhớ đệm

Một trang không mở được có thể bắt đầu từ tên miền sai, nhưng cũng có thể thất bại sau khi client đã đến đúng IP và hoàn tất TCP. Ta sẽ theo một yêu cầu giả định tới `https://api.example.test/orders/42` qua một reverse proxy rồi tới ứng dụng phía sau. Đường chính của chương là DNS chọn địa chỉ, TCP nối tới địa chỉ ấy, TLS xác thực và bảo vệ kênh client–proxy, HTTP mang yêu cầu, proxy chọn upstream, rồi phản hồi đi ngược về. Đây là mô hình HTTPS trên TCP với HTTP/1.1 và TLS 1.3 để thấy rõ từng ranh giới; Không nên dùng mô hình này để khẳng định mọi lần tải trang đều mở một kết nối mới hoặc đi đủ mọi bước khi cache và kết nối tái sử dụng còn hiệu lực.

Tên `api.example.test` không tự mang địa chỉ IP. Ứng dụng nhờ bộ phân giải ở máy client tìm bản ghi phù hợp, thường là A cho IPv4 hoặc AAAA cho IPv6. Bộ phân giải có thể dùng thông tin cục bộ hoặc hỏi recursive resolver; resolver này dùng cache hay hỏi các máy chủ có thẩm quyền theo hệ phân cấp. Kết quả DNS là dữ liệu để client chọn địa chỉ thử kết nối, không phải phép thử cổng 443, chứng chỉ, hay tình trạng ứng dụng. Cùng một tên có thể trả nhiều địa chỉ; client và mạng có thể chọn các đường khác nhau. Khi đo, cần ghi rõ loại bản ghi, resolver, thời điểm và địa chỉ thật mà tiến trình đã dùng, thay vì coi một kết quả tra cứu trên máy khác là bằng chứng của request lỗi.

Trên đường DNS qua cổng 53, nhiều truy vấn dùng UDP nhưng triển khai DNS tổng quát cũng phải hỗ trợ TCP. Xét một ca giả định: resolver hỏi cùng tên và loại bản ghi, nhận phản hồi UDP có cờ `TC=1` báo nội dung bị cắt, rồi thử lại qua TCP tới cùng máy chủ DNS. Nếu mạng cho UDP/53 mà chặn TCP/53, một câu trả lời nhỏ vẫn có thể thành công trong khi câu trả lời cần thử lại bị timeout. Vì vậy, “DNS lúc được lúc không” chưa chứng minh bản ghi authoritative sai. Ghi tên, loại bản ghi, IP máy chủ và thời điểm của hai lượt; quan sát cờ `TC`, SYN/SYN-ACK trên TCP và phản hồi cuối. EDNS có thể cho payload UDP lớn hơn giới hạn ban đầu, nên không gán một ngưỡng kích thước cố định cho mọi truy vấn. Đây là tình huống suy luận về DNS qua cổng 53, không phải capture đã đo hay mô hình của mọi cơ chế DNS mã hóa.

Mỗi bản ghi có TTL giới hạn thời gian lưu ở cache DNS. Thay bản ghi ở authoritative server không buộc mọi cache và mọi tiến trình chuyển ngay sang địa chỉ mới; một số ứng dụng còn có cache hoặc giữ kết nối của riêng mình. Kết quả không tồn tại tên (NXDOMAIN) cũng có thể được cache theo quy tắc riêng; nó khác việc resolver chờ quá hạn vì không nhận được câu trả lời đáng tin. Vì thế “DNS sai” cần nói sai ở đâu: dữ liệu authoritative, cache của resolver, cấu hình resolver ở client, hay tên mà ứng dụng thật sự hỏi. Giảm TTL trước một thay đổi có kế hoạch có thể rút ngắn thời gian giữ dữ liệu cũ, đổi lại tăng số lần tra cứu; nó không xóa được cache đã nhận TTL trước đó. DNS thành công chỉ mở cửa cho bước kết nối tiếp theo.

**Bài tập 10.1 — Địa chỉ DNS cũ trong bộ nhớ đệm**

Client A vẫn lấy IP proxy cũ cho `api.example.test`, trong khi một resolver khác trả IP mới. Nêu ba dữ kiện cần ghi trước khi quy lỗi cho authoritative DNS. Vì sao một lần tra cứu từ máy của người vận hành không đủ?

**Bài tập 10.2 — DNS qua UDP được, qua TCP lỗi**

Cùng một client và resolver: truy vấn A nhỏ trả lời qua UDP/53, nhưng truy vấn khác nhận UDP `TC=1` rồi thử TCP/53 và timeout. Vì sao hai kết quả khác nhau? Nêu phép quan sát để khoanh vùng lỗi và hậu kiểm sau sửa. Có đủ căn cứ nói bản ghi authoritative sai không?

## Bài 10.2 — TLS

Với địa chỉ đã chọn, client dùng route và TCP của Chương 8 và Chương 9 để nối tới cổng 443 của proxy. Không có đường trả về hoặc SYN bị lọc có thể gây timeout trước khi có bất kỳ HTTP status nào. TCP `ESTABLISHED` chỉ chứng minh luồng byte đã mở giữa client và đầu nhận TCP; nó chưa nói đầu kia là đúng tên dịch vụ. Một phép thử trực tiếp bằng IP có thể bỏ qua DNS, nhưng nếu không giữ nguyên tên dịch vụ cho TLS và HTTP thì phép thử đó có thể đi vào cấu hình virtual host khác.

Trên TCP, client bắt đầu TLS bằng ClientHello, thường mang tên đích qua SNI để máy phục vụ chọn cấu hình và chứng chỉ phù hợp. Trong TLS 1.3 với xác thực bằng chứng chỉ, hai bên thỏa thuận khóa; server gửi chứng chỉ và chứng minh nắm khóa tương ứng, còn các thông điệp hoàn tất kiểm tra tính toàn vẹn của bắt tay. Client phải kiểm tra chuỗi tin cậy, thời hạn và danh tính trong chứng chỉ có khớp tên dịch vụ yêu cầu. Nếu tên không khớp hoặc chuỗi không đáng tin, client phải dừng theo chính sách xác thực của mình; tắt xác minh chỉ che lỗi và mở đường cho kẻ khác mạo danh. SNI giúp chọn chứng chỉ, không phải chứng cứ rằng chứng chỉ đã hợp lệ. Tên trong HTTP `Host` được dùng ở tầng định tuyến yêu cầu; nó cũng không thay thế phép xác minh TLS.

Lần theo một **full handshake TLS 1.3** bình thường giữa client và proxy, không dùng PSK hay 0-RTT. Client gửi `ClientHello` với key share và SNI `api.example.test`; proxy chọn tham số trong `ServerHello`, rồi gửi `EncryptedExtensions`, `Certificate`, `CertificateVerify` và `Finished`; client kiểm chuỗi và tên đã định từ URL, sau đó gửi `Finished` của mình rồi mới gửi HTTP request thông thường trong kênh đã xác thực. Các thông điệp bắt tay sau `ServerHello` được mã hóa theo khóa bắt tay, nhưng "đã mã hóa" không có nghĩa server được tin: nếu certificate chỉ trình tên `old.example.test`, phép so danh tính thất bại và client dừng trước request HTTP trong mô hình này. Vì vậy proxy không thể sinh một HTTP 502 từ request chưa hề được gửi; sửa binding/chứng chỉ cho tên công khai rồi kiểm lại từ cùng client. Trace này cố ý bỏ qua resumption và 0-RTT; các chế độ ấy có thứ tự và rủi ro riêng, nên không dùng trace rút gọn này để kết luận mọi phiên TLS có cùng số lượt trao đổi.

Khi TLS kết thúc ở reverse proxy, chỉ kênh client–proxy được bảo vệ bởi phiên TLS ấy. Proxy có thể mở một kết nối HTTP thường hoặc một phiên TLS khác tới upstream; cần xác định hợp đồng mạng và mối đe dọa của đoạn sau trước khi chọn. TLS tới proxy không tự mã hóa đoạn proxy–upstream. TLS handshake thành công cũng chưa chứng minh proxy chọn đúng ứng dụng hay

ứng dụng xử lý được yêu cầu. Tái sử dụng kết nối và phiên có thể giảm chi phí bắt tay, nhưng làm cho chẩn đoán "một request bằng một kết nối mới" sai; khi kiểm tra cần biết client vừa mở hay đang dùng kết nối cũ.

**Bài tập 10.3 — Chứng chỉ TLS không khớp tên**

TCP tới proxy đã `ESTABLISHED` nhưng client báo chứng chỉ không khớp tên. Nêu thông tin cần đối chiếu ở client và proxy. Vì sao đổi HTTP `Host` hoặc tắt xác minh TLS không phải cách sửa gốc?

**Bài 10.3 — HTTP**

HTTP/2 có thể ghép nhiều luồng trên một kết nối TCP; HTTP/3 vận chuyển cùng ngữ nghĩa HTTP trên QUIC. Vì vậy khi chẩn đoán cần biết giao thức đã thương lượng, thay vì giả định một request luôn ứng với một kết nối TCP mới.

Sau TLS, client gửi một HTTP request có phương thức, đích yêu cầu, các trường như `Host` và có thể có body. Với ví dụ này, `GET /orders/42` yêu cầu một tài nguyên; phản hồi có status, header và có thể có nội dung. Status mô tả kết quả ở tầng HTTP, nhưng một `200` không chứng minh nội dung đúng, còn một `404` có thể do proxy hoặc ứng dụng trả. Nếu HTTP request chưa hề được gửi vì DNS, TCP hoặc TLS hỏng thì không có status HTTP từ đích để diễn giải. Trong HTTP/1.1, `Host` giúp chọn authority khi nhiều tên cùng dùng một IP; một TCP connection tới đúng IP vẫn có thể bị route sang virtual host khác nếu tên yêu cầu sai.

Cache HTTP lưu phản hồi đủ điều kiện và dùng độ tươi hoặc xác minh lại để giảm độ trễ và tải upstream. Nó khác cache DNS: một bên lưu dữ liệu tên, bên kia lưu phản hồi ứng dụng. Cấu hình cache thiếu phân biệt người dùng có thể làm lộ nội dung riêng; việc thấy nội dung cũ cũng chưa đủ kết luận DNS còn cũ. Cần kiểm tra `Cache-Control`, khóa cache, `Age` và đường đi thực tế theo loại phản hồi. Không dùng cache để che một upstream hỏng nếu dữ liệu sai hoặc quá cũ có hậu quả nghiệp vụ.

Hãy đọc một chuỗi **giả định** của tài nguyên công khai `/catalog`, không phải capture mạng thật. Lượt đầu với `Accept-Language: vi`, origin trả `200`, body phiên bản `v7` và các trường `Cache-Control: public, max-age=60`, `ETag: "v7"`, `Vary: Accept-Language`. Giả sử response được lưu với tuổi ban đầu bằng 0 và không có chỉ thị nào khác: sau 30 giây, một GET cùng tên và cùng ngôn ngữ có thể dùng bản còn fresh mà không gọi origin. Một GET với `Accept-Language: en` không được dùng bản `vi` ấy khi chưa tái xác minh, vì trường `Vary` yêu cầu so giá trị header đã chọn. Sau 70 giây, bản `vi` đã stale theo giả định; cache có thể gửi `If-None-Match: "v7"` tới origin. Nếu origin trả `304 Not Modified`, cache cập nhật metadata rồi tái dùng body đã lưu cho một GET thông thường; nếu origin trả `200` với body `v8`, cache phải xét response mới thay vì phát body `v7` như còn fresh. Phép tính 30/70 giây chỉ đúng với giả định tuổi ban đầu bằng 0; trong hệ thật phải xét `Date`, `Age`, thời gian lưu và các chỉ thị khác.

Trace này cho một phép chẩn đoán cụ thể. Người dùng tiếng Anh nhận nội dung tiếng Việt: kiểm request header đi vào cache, `Vary` của response được lưu và khóa chọn bản, trước khi đổ cho DNS. Người dùng thấy `v7` lâu hơn 60 giây: kiểm tuổi response, cache nào phục vụ, cơ chế tái xác minh và chỉ thị cho phép dùng bản stale; đừng mặc định origin đang trả sai. Với dữ liệu theo tài khoản,

`public` như ví dụ là một quyết định nguy hiểm; thiết kế phải chọn chính sách cache theo danh tính, quyền và độ nhạy, và kiểm bằng hai phiên người dùng tách biệt. Một lần response đúng không chứng minh không rò dữ liệu giữa hai phiên. Tương tự, `304` chỉ xác nhận representation chưa đổi theo validator ở lượt đó, không xác nhận dữ liệu nghiệp vụ vốn đã đúng.

# Chương 11 — Reverse proxy và điều tra yêu cầu qua hai chặng

Xác định ai sinh status và chặng nào timeout.

## Bài 11.1 — Chọn máy phục vụ phía sau

Reverse proxy nhận request ở phía client, chọn route theo tên và đường dẫn, rồi gửi một request sang upstream đã cấu hình. Đây là hai cuộc trao đổi có thể dùng hai kết nối và hai bộ timeout khác nhau. Proxy có thể thêm thông tin về client ban đầu cho upstream, nhưng ứng dụng chỉ nên tin các trường chuyển tiếp do proxy đáng tin cậy đặt hoặc chuẩn hóa; client ngoài cũng có thể tự gửi header cùng tên. Proxy còn có thể cân bằng tải, giới hạn truy cập, đổi header hoặc cache theo cấu hình. Mỗi tính năng tạo thêm điểm cần quan sát: proxy trả lời từ cache thì upstream có thể không thấy request; upstream trả nhanh nhưng proxy gửi chậm thì độ trễ người dùng vẫn cao. Log nên ghép thời điểm, tên đích, route đã chọn, upstream, thời gian kết nối, thời gian chờ phản hồi và một định danh request không chứa bí mật.

Trong trace giả định, proxy nhận `GET /report` lúc 10:00:00, chọn upstream A, thiết lập kết nối trong 20 ms, rồi không nhận byte phản hồi trước timeout 2 s. Client thấy lỗi do proxy phát sinh; không thể suy upstream đã xử lý hay chưa chỉ từ status trả về. Nếu proxy thử lại trên B, request đọc có thể chấp nhận theo hợp đồng, còn request ghi có thể tạo tác dụng phụ kép khi A đã ghi nhưng chưa trả lời. Để chẩn đoán, ghép request ID qua hai chặng và so thời gian ở proxy với log upstream; việc không thấy log ở A cũng chưa chứng minh gói chưa tới A nếu log bị mất.

Đổi sang một `POST /orders` giả định để thấy thời hạn ở hai chặng không thể xét riêng. Tính từ khi proxy nhận request ở mốc 0, client chờ tối đa 1.500 ms, proxy cho upstream tối đa 2.000 ms. Upstream nhận request ở mốc 20 ms, commit đơn ở 1.700 ms và trả `201` ở 1.800 ms. Giả sử cấu hình proxy vẫn giữ chặng upstream sau khi client ngừng chờ: upstream đã đáp ứng trước timeout của proxy, nhưng client hết hạn từ mốc 1.500 ms nên không đọc được `201`. Nếu client tự gửi lại ở mốc 1.600 ms, hai lượt có thể cùng đang xử lý. Timeout phía client không phải lệnh hoàn nguyên giao dịch đã commit. Proxy cũng có thể hủy chặng upstream khi client rời đi theo cấu hình khác; không được suy đường hủy hay trạng thái database chỉ từ lỗi client.

Vì vậy, trước khi tăng timeout hoặc bật retry, ghi ngân sách thời gian ở client, proxy và upstream, điểm bắt đầu đếm của từng bộ đếm, cùng request ID và khóa idempotency nếu thao tác ghi hỗ trợ. Dùng log ở cả hai chặng và sổ đơn để phân biệt request chưa tới upstream, đã tới nhưng chưa commit, đã commit rồi mất phản hồi. Với `POST` có tác dụng phụ, chỉ thử lại tự động khi hợp đồng ứng dụng biết nhận diện cùng một thao tác hoặc chứng minh lượt đầu chưa áp dụng; đổi 2.000 thành 3.000 ms ở proxy không chữa được deadline 1.500 ms của client. Trong môi trường thử cô lập, có thể cho upstream cố ý trả sau 1.800 ms, đặt client deadline 1.500 ms và kiểm **cả kết quả client lẫn số đơn được tạo**. Các mốc này là dữ liệu suy luận, không phải log hay benchmark đã đo.

### Bài tập 11.1 — Client hết hạn nhưng đơn vẫn được ghi

Client đặt deadline 1.500 ms, proxy chờ upstream 2.000 ms. Upstream nhận POST ở 20 ms, commit ở 1.700 ms và trả 201 ở 1.800 ms; giả sử proxy vẫn đợi sau khi client ngừng chờ. Client thấy gì, trạng thái đơn có thể là gì, và vì sao retry ở 1.600 ms nguy hiểm? Nêu hai phép kiểm trước khi bật retry.

### Bài 11.2 — Hai chặng

Trace dưới đây là dữ liệu giả định để đọc một lần trao đổi thành công, không phải output thu từ mạng. Các dòng phân cách chặng là chú giải, dòng trống kết thúc phần header; byte trên dây HTTP/1.1 dùng CRLF. Client xác thực TLS với `api.example.test` rồi gửi request bên trong kênh mã hóa tới proxy. Proxy giải mã ở điểm kết thúc TLS và, theo cấu hình giả định này, mở HTTP thường tới upstream; đoạn sau cần biện pháp bảo vệ riêng nếu đi qua miền mạng không tin cậy.

```
client -> proxy (HTTP inside TLS)
GET /orders/42 HTTP/1.1
Host: api.example.test
Accept: application/json

proxy -> upstream (HTTP, illustrative)
GET /orders/42 HTTP/1.1
Host: orders.internal.test
X-Forwarded-Host: api.example.test
X-Forwarded-Proto: https

upstream -> proxy
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 9

{"id":42}

proxy -> client (HTTP inside TLS)
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 9

{"id":42}
```

`Host: api.example.test` giúp proxy chọn virtual host công khai. Ở chặng sau, proxy chủ động đổi `Host` thành `orders.internal.test` để upstream định tuyến, đồng thời ghi lại tên và giao thức phía client bằng hai trường chuyển tiếp do chính proxy ghi đè; upstream chỉ nên tin chúng khi request đến từ proxy được tin cậy. `200 OK` trong trace do upstream tạo và proxy chuyển tiếp nguyên trạng, không phải bằng chứng proxy luôn giữ nguyên header hoặc body. `Content-Length: 9` đếm chín byte ASCII của `{"id":42}`. Nếu upstream không trả header kịp ngưỡng chờ, proxy có thể tự sinh `504` thay cho phản hồi này; khi đó cần log của cả hai chặng để biết request có tới upstream và có còn chạy sau lúc proxy ngừng chờ hay không. Trace bỏ qua kết nối tái sử dụng, cache và cách đóng kết nối để tập trung vào ranh giới trách nhiệm.

Đổi một điều kiện trong trace để dự đoán nguồn status. Nếu TLS client–proxy thất bại vì certificate không khớp tên, chưa có HTTP request bên trong kết nối được xác lập, nên không thể gán một `502` cho upstream. Nếu TLS thành công nhưng proxy không mở được kết nối TCP tới upstream, proxy có thể tự phát phản hồi lỗi gateway tùy cấu hình; upstream không thể đã tạo `200` của request này. Nếu upstream nhận request rồi trả `500`, status ấy có thể được proxy chuyển qua hoặc biến đổi bởi policy, cần xem cả hai log. Cùng một `504` ở client không chứng minh upstream đã xử lý xong hay chưa: timeout của proxy chỉ đặt ranh giới chờ ở proxy, còn công việc upstream có thể tiếp tục và gây tác dụng phụ.

Hai chặng còn có hai hợp đồng định danh riêng. Certificate mà client kiểm thuộc tên công khai `api.example.test`; HTTP `Host` chọn virtual host sau khi kênh TLS tồn tại. Ở chặng proxy–upstream dùng HTTP thường trong ví dụ, upstream không xác thực proxy bằng TLS. Header `X-Forwarded-Host` và `X-Forwarded-Proto` chỉ có ý nghĩa khi proxy loại bỏ giá trị không tin cậy từ client và tự ghi đè trước khi chuyển tiếp. Nếu upstream dùng chúng để tạo redirect hoặc quyết định chính sách truy cập mà chấp nhận giá trị client tự gửi, ranh giới tin cậy bị phá. Với chặng nội bộ đi qua mạng không tin cậy, thay HTTP thường bằng TLS có xác thực phù hợp và kiểm tra tên đích; không lấy TLS chặng công khai làm bằng chứng mã hóa đến tận upstream.

Để tự kiểm tra trace, ghi request ID ở cả proxy và upstream, đánh dấu thời điểm nhận request, mở kết nối, nhận header phản hồi và đóng request. Một ca âm có thể cho upstream chờ lâu hơn proxy timeout: dự đoán client thấy lỗi do proxy sinh, upstream có thể vẫn ghi kết quả sau đó. Trước khi retry một POST có tác dụng phụ, kiểm idempotency key hoặc trạng thái giao dịch; status lỗi ở client không chứng minh server chưa xử lý. Hậu kiểm sau sửa là request thử đi qua đúng virtual host, trả body chín byte như kỳ vọng và log hai chặng cùng liên kết được một request, không chỉ thấy port 443 mở.

### Bài 11.3 — Đường lỗi

Sau bài về TCP, ta biết luồng byte giữa hai endpoint chưa bảo đảm người dùng mở được trang. Tên miền có thể trỏ sai, chứng chỉ bị từ chối, reverse proxy chọn sai upstream hoặc ứng dụng trả lỗi. Bài này ghép các ranh giới ấy thành một đường điều tra: không sửa route khi TCP đã bắt tay, và không sửa HTTP khi SYN chưa tới server.

Nếu proxy, trong vai gateway, nhận phản hồi không hợp lệ từ upstream, nó có thể tạo `502 Bad Gateway`; nếu không nhận phản hồi kịp để hoàn tất yêu cầu, nó có thể tạo `504 Gateway Timeout`. Đây là nghĩa của status, không phải bảng kết luận nguyên nhân: một upstream hoặc tầng trung gian cũng có thể tự trả mã giống vậy, và log proxy phải cho biết mã nào do nó tạo. Lỗi nối upstream, TLS tới upstream, header sai và ứng dụng đóng kết nối là những giả thuyết khác nhau cần phân biệt bằng log và phép đo ở đúng đoạn. Một timeout ngắn giảm thời gian chờ nhưng có thể loại yêu cầu hợp lệ chậm; timeout dài giữ tài nguyên và kéo dài trải nghiệm lỗi. Thử upstream khác có thể cải thiện khả dụng, nhưng gửi lại một yêu cầu thay đổi trạng thái khi chưa biết lần đầu đã thực thi chưa có thể tạo tác dụng phụ lặp. Chính sách retry phải dựa vào tính idempotent, khóa chống lặp hoặc hợp đồng ứng dụng, không chỉ dựa vào status.

Xét một mô hình giả định để luyện chẩn đoán, không phải trace từ hệ thống thật. Theo cấu hình mong đợi, `api.example.test` trỏ tới IP của proxy mới; proxy nhận HTTPS và chuyển `/orders/42` tới upstream đang phục vụ. Người dùng báo "không mở được đơn hàng" sau thay đổi cấu hình. Trước khi sửa, người vận hành ghi lại URL, thời điểm, client, lỗi hiển thị và request ID nếu có. Cùng một câu báo lỗi có thể tương ứng năm điểm dừng sau đây; mỗi phép quan sát chỉ chứng minh một đoạn đường.

Nếu máy bị lỗi phân giải tên ra IP của proxy cũ, còn authoritative server và một resolver khác đã trả IP mới, giả thuyết đầu là cache hoặc cấu hình resolver trên đường của máy ấy. Hỏi đúng resolver mà client dùng, xem TTL còn lại và địa chỉ tiến trình thực sự kết nối; không sửa upstream của proxy mới khi request chưa đến đó. Nếu DNS trả đúng IP mới nhưng SYN rời client mà không có SYN-ACK, chuyển sang route, lọc và đường trả TCP như Chương 8 và Chương 9. Một DNS answer đúng không chứng minh cổng mở; timeout TCP chưa phải lỗi TLS hay HTTP. Quan sát ở proxy xem SYN có đến hay không giúp thu hẹp đoạn lỗi, nhưng một capture im lặng tại sai giao diện không đủ kết luận gói chưa tới máy.

Nếu TCP hoàn tất rồi client báo certificate error, xem tên mà client yêu cầu, SNI, chứng chỉ proxy đã chọn, chuỗi tin cậy và đồng hồ của máy. Trong nhánh giả định này, proxy mới đưa chứng chỉ chỉ chứa `old.example.test`, nên phép xác minh tên `api.example.test` thất bại; sửa binding/chứng chỉ đúng tên là hành động phù hợp. Không chuyển sang HTTP hay bỏ xác minh chứng chỉ để "thử cho qua". Khi TLS hợp lệ và client nhận `502`, đối chiếu log theo request ID: proxy chọn upstream nào, có nối được không, có nhận header HTTP hợp lệ không, hay upstream tự trả `502`. Giả sử log cho thấy upstream chấp nhận TCP rồi đóng trước khi gửi header, nguyên nhân nằm ở giao tiếp proxy–upstream hoặc ứng dụng phía sau, chưa thể suy riêng từ mã `502` là tiến trình đã chết.

Cuối cùng, nếu proxy nhận request nhưng chờ upstream quá hạn rồi tự trả `504`, xem thời gian ở từng bước và log của cùng request ở upstream. Giả sử upstream ghi nhận request và kẹt ở truy vấn dữ liệu lâu hơn ngưỡng chờ của proxy: việc nâng timeout có thể giảm `504` trước mắt nhưng không sửa truy vấn chậm, đồng thời giữ tài nguyên lâu hơn. Cũng phải kiểm tra upstream có thể đã hoàn tất tác dụng phụ sau khi proxy bỏ chờ; retry tự động một lệnh ghi có thể tạo đơn hàng trùng. Mỗi nhánh cần một phép xác nhận sau sửa từ đúng đường người dùng: tên phân giải đúng, TCP bắt tay, TLS xác thực, proxy chọn đúng upstream và response có nội dung đúng. Status, log hay capture riêng lẻ chỉ là một phần chứng cứ.

**Bài tập 11.2 — Định vị lỗi theo từng tầng kết nối**

Cùng URL lần lượt gặp timeout trước TLS, certificate error, `502`, `504`. Với mỗi dấu hiệu, hãy chỉ ra ranh giới xa nhất chắc chắn đã vượt qua và một quan sát tiếp theo có giá trị phân biệt. Có thể kết luận upstream chết chỉ từ `502` không?

**Bài tập 11.3 — Thử lại yêu cầu tạo đơn hàng**

Proxy hết thời gian chờ một yêu cầu tạo đơn hàng rồi định retry sang upstream khác. Nêu rủi ro và điều kiện để retry an toàn. Phân biệt tác dụng của tăng timeout với sửa nguyên nhân upstream chậm.

# PHẦN IV — Git và lịch sử thay đổi

## Làm sao biết, tích hợp và khôi phục đúng thay đổi đã chia sẻ?

Blob/tree/commit là đồ thị bất biến; ref, index và working tree là trạng thái có thể dịch.

IV

**LỘ TRÌNH PHẦN**


- Chương 12 — Đối tượng Git, vùng chờ và tham chiếu
- Chương 13 — Tích hợp nhánh và sửa lịch sử

# Chương 12 — Đối tượng Git, vùng chờ và tham chiếu

Dự đoán byte nào sẽ nằm trong commit kế tiếp.

### Bài 12.1 — Đồ thị đối tượng Git

Khi hai người cùng sửa một dịch vụ, câu hỏi không chỉ là "tệp cuối cùng trông ra sao" mà còn là thay đổi nào đã được ghi, thay đổi nào chưa, và hai dòng phát triển có chung tổ tiên nào. Git lưu các snapshot có liên kết cha–con. Hiểu liên kết ấy giúp ta chọn cách tích hợp, đọc conflict và phục hồi sự cố mà không làm mất lịch sử của đồng đội. Các ký tự `A`, `B`, `C` trong chương là nhãn tượng trưng cho commit, không phải SHA thật; tình huống là mô hình giả định, không phải log của repo này.

Một blob chứa nội dung tệp, không chứa tên tệp. Một tree ghép tên, mode và định danh object của blob hoặc tree con thành snapshot thư mục. Commit trỏ tới một tree gốc, ghi tác giả, người tạo commit, thông điệp và không hoặc nhiều commit cha. Vì vậy đổi tên tệp có thể đổi tree mà blob nội dung vẫn giữ nguyên; hai commit khác nhau có thể trỏ tới cùng snapshot tệp nhưng khác cha hoặc metadata. Git lưu lịch sử bằng các quan hệ này, không phải bằng cách chép nguyên một thư mục có nhãn thời gian cho mỗi commit.

Định danh object là giá trị băm của loại, độ dài và nội dung object theo định dạng kho đang dùng. Nhiều kho dùng SHA-1; Git cũng hỗ trợ kho SHA-256. Vì commit chứa định danh tree và cha, thay nội dung một tệp làm đổi blob, tree chứa nó và các commit mới đi qua thay đổi ấy. Định danh cho phép kiểm tra tính nhất quán nội dung trong kho; nó không tự chứng minh ai là tác giả hay rằng mã an toàn. Tên branch và tên file không nằm trong blob. Để tự đối chiếu mô hình trên một kho được phép đọc, `git cat-file -p HEAD` cho thấy trường tree và parent của commit; `git ls-tree HEAD` cho thấy các mục ở tree gốc. Một SHA rút gọn chỉ là cách viết đủ phân biệt trong kho hiện tại, không phải một loại object khác.

Có thể tự dựng một phép thử trong repository tạm để thấy chính xác điều gì đổi. Commit đầu có `app.conf` chứa `port=8080` và một file `notes.txt`; gọi các object quan sát được là blob `B1`, tree gốc `T1`, commit `C1`. Đổi **chỉ tên** `app.conf` thành `service.conf`, giữ nguyên byte và mode: blob nội dung vẫn là `B1`, nhưng tree gốc thành `T2` vì entry tên khác; commit mới `C2` cũng khác vì trỏ `T2` và có cha `C1`. Sau đó sửa byte `8080` thành `9090`: blob mới `B2`, tree mới `T3`, commit mới `C3`. Các ký hiệu này là nhãn của thí nghiệm, không phải hash thật hay một quy tắc rằng Git luôn giữ object riêng cho từng tên file.

Trước khi chạy, dự đoán kết quả rồi dùng `git ls-tree -r HEAD`, `git cat-file -p HEAD` và `git hash-object` trên nội dung file để đối chiếu trong chính repository thử; ghi đủ object ID và loại object, không lấy dòng hiển thị rút gọn làm khóa toàn cục. Nếu lần đổi tên còn sửa mode hoặc nội dung, giả thuyết "blob không đổi" không còn đúng và phải đọc lại diff. Git có thể trình bày một cặp xóa/thêm như rename khi so hai snapshot theo độ giống nội dung; bản thân blob và tree không chứa một sự kiện "đổi tên" được lưu riêng. Bài tập này phân biệt dữ kiện có trong object với diễn giải mà công cụ diff tạo ra, trước khi dùng lịch sử để điều tra ai đã đổi cấu hình.

### Bài tập 12.1 — Blob, tree và commit khi đổi tên tệp

Trong repository thử, commit C1 có app.conf chứa port=8080. Commit C2 chỉ đổi tên tệp thành service.conf và giữ byte, mode; commit C3 đổi nội dung thành port=9090. Blob, tree và commit nào giữ nguyên hoặc đổi ở mỗi bước? Có thể khẳng định object Git lưu riêng một sự kiện rename không?

### Bài 12.2 — Trạng thái cục bộ

Working tree là những tệp đang thấy và sửa bằng trình biên tập. Index là snapshot dự kiến cho commit kế tiếp; `git add` chép nội dung hiện tại của đường dẫn vào index tại thời điểm chạy. `HEAD` thường là tham chiếu tượng trưng tới branch đang checkout, còn branch ref trỏ tới một commit. Khi tạo commit, Git ghi tree từ index, đặt commit mới có cha là commit tại `HEAD`, rồi dịch ref của branch hiện tại sang commit mới. Sửa tệp sau `git add` không tự cập nhật index; commit có thể chứa bản cũ trong khi working tree vẫn có sửa đổi mới.

Giả sử `timeout_ms` trong commit hiện tại là `1000`. Ta sửa working tree thành `2000`, stage, rồi sửa tiếp thành `3000`. Lúc ấy `HEAD` giữ `1000`, index giữ `2000`, working tree giữ `3000`. `git diff --cached` so `HEAD` với index, còn `git diff` mặc định so index với working tree. Nếu commit ngay, snapshot mới ghi `2000`; muốn ghi `3000` phải stage lại và kiểm tra diff. Đây là lý do `git status` và diff trước commit có giá trị hơn lời khẳng định “tôi đã lưu file”. Tệp chưa được theo dõi chưa tự có trong index.

Index còn lưu metadata đường dẫn và mode, không chỉ một “danh sách file đã chọn”. Một thao tác `git add -p` có thể stage một hunk sửa lỗi nhưng để hunk debug lại trong working tree. Khi đó `git diff --cached` là nội dung thực sự được đưa vào commit; `git diff` là phần chưa stage, và cả hai đều có thể cùng nhắc tới một tệp. Nếu test chạy trên working tree đang chứa hunk debug, test xanh chưa chứng minh snapshot sẽ commit chạy xanh. Muốn kiểm tra chính xác nội dung đã stage, xem diff staged rồi thử nó trong một checkout tạm hoặc dùng CI trên commit tạo ra từ index; không xóa working tree hiện tại để lấy bằng chứng.

Lần theo thêm một chuỗi có thể tự làm trên repository thử: tạo file `setting.txt` chứa `1000`, commit; sửa thành `2000`, `git add setting.txt`; sửa tiếp thành `3000`; quan sát `git show HEAD:setting.txt`, `git show :setting.txt` và nội dung file trên đĩa. Dự đoán lần lượt là `1000`, `2000`, `3000`. Sau commit, branch mới trỏ tới tree chứa `2000`; `3000` vẫn là thay đổi chưa stage. Có thể sửa lại bằng `git add` rồi commit kế tiếp, hoặc xem xét sửa commit nếu chưa chia sẻ lịch sử. Tránh dùng `reset --hard` để “dọn” trước khi đối chiếu vì nó làm mất bằng chứng trong working tree.

Sự phân tách này giải thích một lỗi review thực tế: tác giả nói đã sửa secret trong bản cấu hình, nhưng index vẫn chứa bản cũ. Kiểm tra file trên đĩa là chưa đủ; phải kiểm staged diff và sau commit kiểm tree của chính commit được đẩy. Nếu secret đã đi vào commit hoặc remote, chỉ xóa khỏi commit mới không thu hồi secret: cần quy trình thu hồi/đổi khóa và xử lý lịch sử theo phạm vi phân phối. Cơ chế index giúp ta biết byte nào sắp được ghi vào lịch sử, trước khi bàn tới merge hay rebase ở các bài sau.

**Bài tập 12.2 — HEAD, index và cây làm việc**

Commit `B` chứa `timeout_ms=1000`. Sau `git add` bản `2000`, người học sửa working tree thành `3000` nhưng chưa stage lại. `HEAD`, index và working tree lần lượt giữ gì? Commit ngay sẽ ghi giá trị nào và phép so sánh nào chứng minh điều đó?

**Bài 12.3 — Danh tính thay đổi**

Một commit có định danh từ nội dung object, còn tên branch là ref có thể di chuyển. `HEAD` thông thường trỏ gián tiếp qua branch hiện tại; một tag nhẹ thường trỏ tới object được đánh dấu, trong khi annotated tag là một object riêng chứa thông điệp và đích. Vì thế `release/1.4`, `v1.4` và một mã commit đầy đủ không phải ba tên đồng nghĩa theo hợp đồng vận hành. Muốn nói đã kiểm thử phiên bản nào, phải ghi commit đã resolve tại thời điểm kiểm và artifact đã tạo từ nó, không chỉ ghi tên có thể đổi.

Khi fetch, Git tải các object cần thiết và cập nhật các ref theo refspec; `origin/main` là remote-tracking ref trong kho cục bộ, không phải lời gọi trực tiếp tới server mỗi khi ta đọc nó. Nếu server di chuyển branch sau lần fetch cuối, `origin/main` có thể đã cũ. Ngược lại, một tag được fetch không tự bảo đảm nó do bên tin cậy tạo; chữ ký tag, chính sách bảo vệ ref và quyền đẩy là các kiểm soát riêng. Trước khi tích hợp, ghi rõ commit đích của branch local, remote-tracking ref và tag, rồi so ancestry và diff. Hai ref cùng tên hiển thị ở hai kho khác nhau có thể trỏ hai commit khác nhau.

Đồ thị ví dụ giả định: `A → B → C`; `main` đang trỏ C, `release/1.4` trỏ B và tag `v1.4` cũng đánh dấu B. Một commit D được tạo trên C khiến `main` chuyển D, còn B vẫn reachable từ D qua chuỗi cha D → C → B. Đó là lý do việc dời `main` không xóa ngay bản phát hành B. Nếu xóa mọi ref dẫn tới một commit riêng E trên nhánh thử, E có thể trở thành unreachable từ các ref hiện tại. Reflog, remote-tracking ref hoặc cơ chế giữ object vẫn có thể giữ E một thời gian; không hứa `gc` xóa ngay và không coi reflog là chính sách backup.

Điểm vận hành là phân biệt tên với đối tượng: `git rev-parse` giải tên sang object ID ở đúng kho, `git log --graph --all` cho thấy các ref được chọn đi tới commit nào, và `git merge-base --is-ancestor` kiểm giả thuyết ancestry. Những phép kiểm này nói về lịch sử nguồn, chưa chứng minh binary đang chạy được build từ commit ấy. Bài CI sau này nối commit đã resolve với digest artifact và bản ghi triển khai.

**Bài tập 12.3 — Commit, tag và nhánh từ xa**

Trong đồ thị giả định A→B→C→D, main trỏ D, tag v1.4 trỏ B và origin/main chưa fetch vẫn trỏ C. Những tên nào xác định cùng commit? Có thể kết luận server hiện trỏ C không?

# Chương 13 — Tích hợp nhánh và sửa lịch sử

Giải conflict mà vẫn giữ ý nghĩa thay đổi và lịch sử chia sẻ.

## Bài 13.1 — Gộp nhánh

Xét commit `B` có `timeout_ms=1000`. Nhánh `main` đi từ `B` tới `D`, đặt `timeout_ms=3000` để giảm timeout sai ở cổng vào. Nhánh `feature/retry` đi từ `B` tới `C`, đặt cùng dòng thành `timeout_ms=2000` để thử retry. Đồ thị trước tích hợp là `A → B → D` trên `main` và `B → C` trên `feature/retry`; `B` là tổ tiên chung gần nhất. Tên branch chỉ là ref có thể dịch chuyển; `D` và `C` là hai object commit vẫn còn nguyên khi ref đổi.

Nếu `main` vẫn ở `B`, tích hợp `C` có thể fast-forward: chỉ dịch ref `main` tới `C`, không tạo commit hai cha. Ở đây `main` đã tới `D`, nên fast-forward không đủ. Merge ba phía so snapshot ở `B`, `D` và `C`. Cả hai phía thay cùng dòng theo hai cách nên Git không thể chọn đúng ý định nghiệp vụ; nó dừng với conflict. Trước khi người vận hành giải quyết, `HEAD` vẫn là `D`, `MERGE_HEAD` ghi phía `C`, index có thể giữ ba stage cho tệp xung đột: stage 1 là bản ở `B`, stage 2 là phía `HEAD`/`D`, stage 3 là phía `MERGE_HEAD`/`C`. Dấu `<<<<<<<`, `=======`, `>>>>>>>` trong working tree chỉ là trình bày vùng chưa giải quyết, không phải cấu hình được phép đưa vào production.

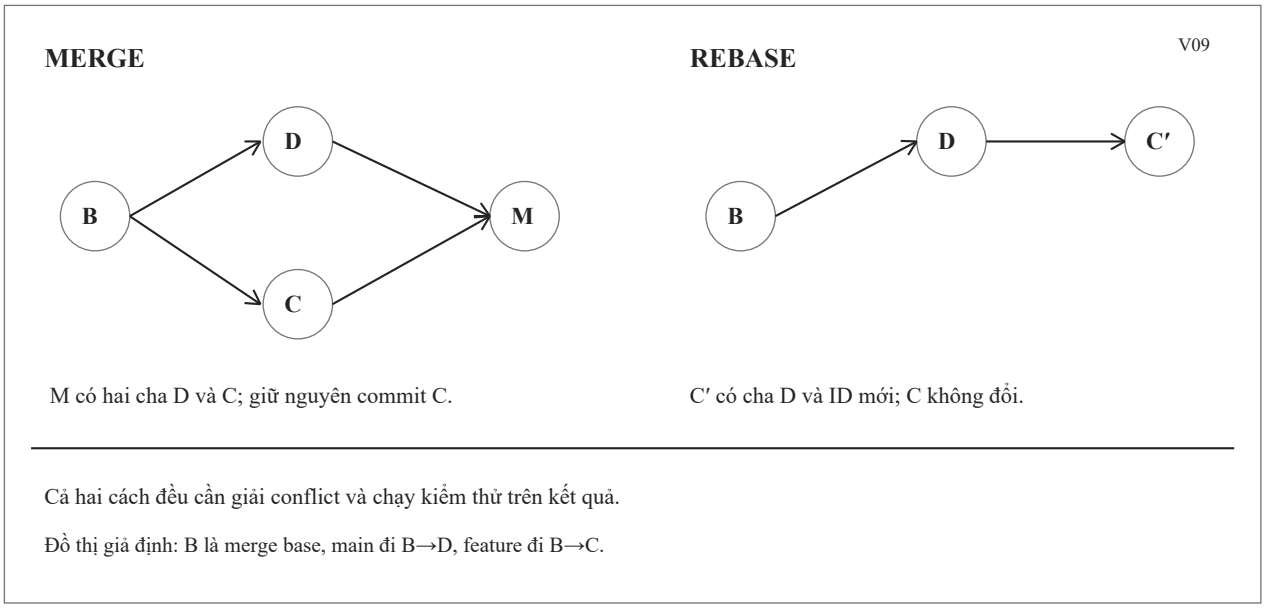
Quyết định đúng không thể suy từ câu "lấy ours" hay "lấy theirs". Nhóm phải kiểm tra hợp đồng timeout và retry. Giả sử kiểm thử cho thấy upstream cần ngưỡng `2500` và retry chỉ dành cho thao tác idempotent. Người tích hợp viết `timeout_ms=2500`, cập nhật kiểm thử liên quan, rồi stage tệp đã giải quyết. Khi đó index có bản thường thay cho ba stage xung đột; `git diff --cached` cho thấy snapshot sắp ghi, và `git ls-files -u` không còn mục chưa giải quyết. Chạy kiểm thử cấu hình và hành vi retry trên kết quả cuối, xem diff để bảo đảm không còn marker; chỉ sau đó tạo merge commit `M` với hai cha theo thứ tự `D` và `C`. Ref `main` dịch tới `M`, còn ref `feature/retry` vẫn ở `C` nếu chưa xóa. Nếu chưa hiểu cách giải, `git merge --abort` có thể đưa merge về trước khi bắt đầu, nhưng với thay đổi chưa commit trước merge không nên mặc định rằng mọi thứ luôn khôi phục hoàn hảo; hãy bắt đầu từ working tree sạch.

### Bài tập 13.1 — Xung đột khi gộp hai nhánh

Trên đồ thị `A → B → D` của `main` và `B → C` của feature, vì sao không thể fast-forward `main` tới `C` mà giữ `D`? Nếu giải conflict rồi merge thành `M`, nêu hai cha và trạng thái các branch ref. Index có gì khi conflict đang dở?

## Bài 13.2 — Sắp lại commit

Thay vì merge, trên nhánh feature chưa chia sẻ có thể rebase `C` lên `D`: Git lấy thay đổi mà `C` đưa vào so với `B`, thử áp dụng lên `D`, rồi tạo commit mới `C′` có cha là `D`. Vì `D` đã sửa cùng dòng, rebase cũng có thể dừng để con người giải conflict. Sau khi giải và tiếp tục, ref feature trỏ tới `C′`; `C` cũ không bị sửa tại chỗ. Ngay cả khi snapshot cuối cùng giống kết quả merge, đồ thị khác: `A → B → D → C′` là một đường, trong khi merge tạo `M` có hai cha `D` và `C`. Giá trị băm của `C′` khác `C` vì nội dung commit, đặc biệt parent, khác.




Hình IV.2.1 — Cùng nhánh ban đầu, merge thêm commit hai cha; rebase tạo commit mới.

Rebase thuận tiện để sắp xếp các commit của chính mình trước khi công bố. Nếu người khác đã dựa vào `C` trên một branch dùng chung, đẩy lịch sử rebase lên sẽ buộc ref từ xa rời đường cũ và làm bản sao của họ lệch nhau; cần phối hợp rõ trước mọi lần viết lại lịch sử đã chia sẻ. Với nhánh tình huống này đã có người cùng dùng, chọn merge để giữ cả `C` và `D` trong lịch sử công khai. Cả merge lẫn rebase chỉ xử lý cách ghép lịch sử và nội dung; không tự chứng minh bản `timeout_ms=2500` đúng. Chạy lại phép kiểm chứng trên snapshot tích hợp là điều bắt buộc.

Điểm cần theo dõi khi rebase dừng là commit nào đang được phát lại, không chỉ file nào có marker conflict. Giả sử nhánh feature có `C1` sửa timeout rồi `C2` thêm kiểm thử, còn main ở `D` đã sửa cùng timeout. Rebase lần lượt thử tạo bản mới của `C1` trên `D`; sau khi người học giải conflict và stage kết quả, quá trình tiếp tục mới thử phát lại `C2`. Nếu `C2` dựa trên hành vi cũ, nó có thể áp dụng sạch về mặt văn bản nhưng làm test sai về mặt nghiệp vụ. Vì thế `rebase --continue` không phải cổng chất lượng: sau khi hoàn tất, đọc đồ thị commit mới, diff tích hợp và chạy kiểm tra trên snapshot cuối. Lưu lại commit gốc trước thao tác để có đường quay lui nếu cần.

So hai phương án trên cùng dữ kiện. Merge giữ `C1`, `C2`, `D` nguyên identity và tạo commit kết hợp có hai cha; người đọc lịch sử thấy hai nhánh công việc. Rebase tạo `C1'`, `C2'` với cha mới, làm lịch sử tuyến tính nhưng thay các ID đã công bố nếu nhánh từng được chia sẻ. Với nhánh riêng chưa ai dựa vào, rebase có thể làm review dễ đọc; với nhánh chung hoặc release đã tham chiếu các SHA cũ, merge thường giữ dấu vết an toàn hơn. Không chọn theo hình đồ thị đẹp: chọn theo ai đang giữ bản sao lịch sử, chính sách repository và nhu cầu kiểm toán.

Thử trên repository tạm tự tạo: từ `B` tạo `C1`, `C2` ở feature và `D` ở main với sửa cùng dòng. Trước khi thao tác, vẽ cha của từng commit và dự đoán điểm dừng; sau thao tác, đọc `git log --graph --oneline --all`, kiểm nội dung cuối và chạy test. Nếu kết quả cuối giống nhau, giải thích vì sao SHA và quan hệ cha vẫn khác. Không rebase nhánh chung thật để làm bài thực hành; chỉ đổi lịch sử trong fixture. Sự khác nhau giữa merge và rebase là cơ chế ghi lịch sử, còn lựa chọn timeout đúng phải do yêu cầu ứng dụng và phép kiểm độc lập quyết định.

**Bài tập 13.2 — Viết lại commit bằng rebase**

Nếu rebase `C` lên `D` và giải cùng conflict, đồ thị mới và định danh commit feature thay đổi ra sao? Vì sao phương án này cần cân nhắc khác khi `C` đã được chia sẻ? Hai cách tích hợp có tự xác nhận timeout mới là đúng không?

**Bài 13.3 — Phục hồi**

Nếu một commit thường đã công bố gây lỗi, `git revert` tạo commit mới đảo tác động của commit được chọn trên trạng thái hiện tại. Commit lỗi vẫn có trong đồ thị, người khác có thể lấy commit đảo chiều bằng cách đồng bộ bình thường; revert cũng có thể xung đột nếu mã đã thay đổi thêm. Muốn revert chính merge commit `M` của ví dụ, phải chỉ định parent làm dòng chính: `git revert -m 1 M` xem `D` là mainline rồi đảo thay đổi mà merge đưa vào so với `D`. Cần xem diff và kiểm thử vì thao tác này không có nghĩa "quay toàn bộ repo về B", và về sau merge lại cùng nhánh có thể không tự mang lại các thay đổi đã bị đảo.

`git reset <commit>` ở dạng dịch `HEAD` đặt ref branch hiện tại về commit được chọn; chế độ `--soft`, mặc định `--mixed`, và `--hard` lần lượt ảnh hưởng khác nhau tới index và working tree. `--hard` còn có thể bỏ thay đổi local chưa commit ở các tệp được theo dõi. Dịch ref `main` từ `M` về `D` sẽ làm lịch sử branch công bố không còn đi qua `M`, dù object có thể còn tạm được tham chiếu ở nơi khác. Vì thế reset là công cụ sửa lịch sử cục bộ khi đã hiểu trạng thái; nó không thay revert trên nhánh đã chia sẻ. Khôi phục một tệp trong working tree hoặc bỏ stage là tác vụ khác với dịch branch ref; đừng dùng một lệnh dịch lịch sử để chữa nhầm một file đang sửa.

Khi nhận một conflict thật, trước hết xác định branch đang checkout, working tree có sạch trước khi tích hợp không, và hai commit đầu nhánh cùng merge base. Sau đó đọc ba phiên bản của tệp, xác định mỗi thay đổi phục vụ yêu cầu nào; conflict marker chỉ cho vị trí chồng lấn, không cho quyết định đúng. Kiểm tra diff đã stage và đồ thị cha sau commit, chạy test trên snapshot vừa tích hợp, rồi mới chia sẻ. Nếu test thất bại, phân biệt lỗi từ code đã commit với sửa đổi chỉ còn trong working tree; một `git status` sạch không thay thế kiểm thử. Nếu cần hủy ảnh hưởng trên branch chung, chuẩn bị một revert có review; nếu cần thử lại một merge đang dở, kiểm tra trạng thái trước khi abort. Sự an toàn đến từ việc biết ref, index và working tree nào đang thay đổi ở mỗi bước.

**Bài tập 13.3 — Hoàn nguyên merge trên nhánh chung**

Merge commit `M` đã lên branch chung rồi gây lỗi. So sánh mục tiêu và hậu quả của revert `M` với reset `main` về `D`; vì sao revert merge cần chọn mainline? Cần kiểm chứng gì trước khi push?

# PHẦN V — Tự động hóa hệ thống

## Một chuỗi tác vụ báo đúng thất bại và có thể chạy lại thế nào?

Parser, argv, status, deadline và tác dụng phụ qua nhiều tiến trình là các hợp đồng khác nhau.

V

**LỘ TRÌNH PHẦN**


- Chương 14 — Bash và ngữ nghĩa lỗi của shell
- Chương 15 — Python và Go điều phối tiến trình
- Chương 16 — Quy trình tự động hóa an toàn và chạy lại

# Chương 14 — Bash và ngữ nghĩa lỗi của shell

Dự đoán argv, output và exit status trước khi chạy script.

## Bài 14.1 — Phân tích cú pháp shell

Một script backup có thể in `BACKUP_OK` dù không tạo được bản sao dùng được. Lỗi không nhất thiết nằm ở lệnh sao lưu: shell có thể biến dữ liệu thành đối số khác, pipeline có thể che mã lỗi, và thao tác dọn dẹp có thể chạy sau một kết luận thành công sai. Chương này xem Bash như một chương trình điều phối tiến trình và file descriptor. Mọi đường dẫn và kết quả của tình huống bên dưới thuộc một thử nghiệm cô lập; chúng không mô tả trạng thái của lab hoặc máy production.

Bash đọc cú pháp trước: dấu nháy, toán tử `|`, `>`, `&&` và dấu kết thúc lệnh xác định cấu trúc. Sau đó shell mở rộng các word: biến, phép tính, command substitution, tách word và mở rộng tên tệp theo ngữ cảnh, rồi bỏ dấu nháy. Ký tự đặc biệt sinh ra từ giá trị biến không tự được phân tích lại thành toán tử shell; gọi `eval` để ép phân tích lần hai là mở một ranh giới thực thi mới. Dấu nháy kép giữ kết quả mở rộng biến thành một đối số, trừ những quy tắc riêng như `"$@"` giữ từng đối số ban đầu. Dấu nháy đơn giữ nguyên hầu hết ký tự bên trong, không mở rộng biến.

Giả sử `src='/tmp/du lieu'`. Với `tar -C $src`, Bash có thể đưa cho `tar` hai đối số `/tmp/du` và `lieu`; `tar -C "$src"` đưa đúng một đường dẫn. Nếu biến không được nháy chứa `*`, pathname expansion còn có thể thay một word thành nhiều tên file. Điều này xảy ra trước khi `tar` chạy; sửa tùy chọn của `tar` không chữa được lỗi truyền đối số. Phép gán đơn giản `src=$1` không tách word ở vế phải, nhưng dùng `$src` không nháy ở lời gọi sau đó vẫn nguy hiểm. Với tập tên file tùy ý, dùng mảng Bash và `"${files[@]}"`; không ghép danh sách thành một chuỗi rồi hy vọng shell tách lại đúng. `--` là quy ước được nhiều utility hỗ trợ để kết thúc phần tùy chọn, không phải phép bảo vệ do Bash tự áp cho mọi chương trình.

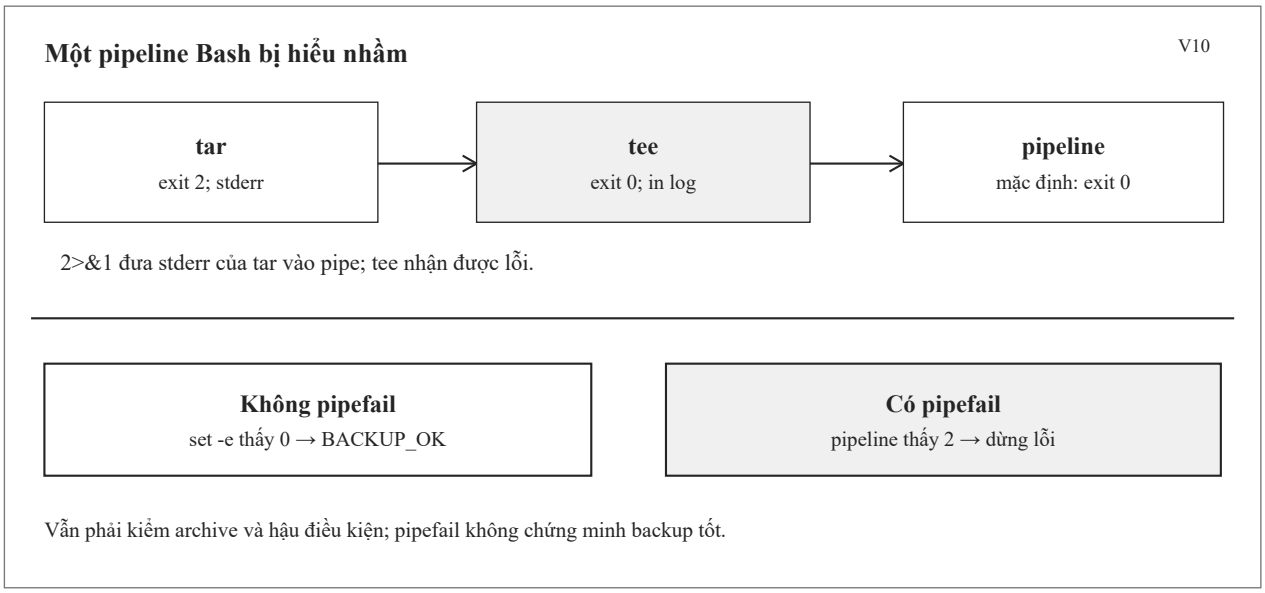
Một biến chưa đặt và một biến rỗng là hai trạng thái khác nhau. `set -u` báo lỗi khi mở rộng biến chưa đặt trong nhiều ngữ cảnh, nhưng không chứng minh đường dẫn tồn tại hoặc nội dung hợp lệ. Cú pháp `${1:?can tham so}` buộc có đối số thứ nhất và khác rỗng trước khi đi tiếp; nó vẫn không bảo đảm đối số ấy là một thư mục an toàn. Dữ liệu từ môi trường, tham số và file cấu hình phải đi qua kiểm tra theo hợp đồng của thao tác sắp làm.

### Bài tập 14.1 — Tách từ và ký tự đại diện trong Bash

Cho `src='/tmp/du lieu'`, giải thích số đối số đường dẫn mà `tar -C $src` và `tar -C "$src"` nhận sau mở rộng. Nếu tên file chứa `*`, vì sao chỉ thêm `set -u` không sửa lỗi? Nêu cách giữ từng phần tử của một danh sách tên file.

## Bài 14.2 — I/O và trạng thái thoát

Mỗi tiến trình nhận các file descriptor chuẩn: 0 để đọc, 1 để ghi thông thường, 2 để ghi lỗi. `cmd >file 2>&1` cho descriptor 1 trỏ tới file rồi nối descriptor 2 vào đích hiện tại của 1. Đảo thứ tự thành `cmd 2>&1 >file` khiến descriptor 2 giữ đích cũ của 1 trong khi 1 mới trỏ tới file. Bash thực hiện redirection trước khi thực thi lệnh; lỗi mở file có thể làm lệnh chính không chạy. Với pipeline, output của lệnh trái trở thành input của lệnh phải, và các phần thường chạy trong những môi trường tiến trình riêng. Đừng kỳ vọng biến gán bên trong một nhánh pipeline luôn trở lại shell cha; cấu hình `lastpipe` là một ngoại lệ có điều kiện.




Hình V.1.1 — Trạng thái pipeline không mặc nhiên là trạng thái của tar.

Hãy dự đoán hai đường đi lỗi của lệnh giả định `tar -cf - /srv/data | tee /tmp/data.tar >/dev/null`. Nếu `tar` không đọc được nguồn, `tee` vẫn có thể đóng file và trả 0; theo mặc định, shell báo pipeline thành công dù archive rỗng hoặc thiếu. Nếu `/tmp` hết dung lượng, `tee` trả lỗi ghi; ngay cả khi `tar` đã đọc xong, archive vẫn không dùng được. `pipefail` phát hiện pipeline có lệnh lỗi, còn `PIPESTATUS` giúp phân biệt thành phần nào báo lỗi. Nhưng đọc `PIPESTATUS` sau một lệnh `echo` sẽ thấy trạng thái của lệnh mới: phải lưu mảng ngay khi pipeline kết thúc rồi mới log. Cần kiểm tra kích thước, danh sách file và một lần giải nén thử vào thư mục tạm độc lập trước khi công bố backup hợp lệ. Giải nén thử vẫn chưa chứng minh ứng dụng khôi phục nhất quán nếu dữ liệu thay đổi trong lúc tạo archive.

Redirection cũng có thể gây "thành công giả" ở lớp quan sát. Với `cmd >out 2>&1`, cả stdout và stderr đi vào `out`; với `cmd 2>&1 >out`, stderr vẫn đi về đích stdout cũ, có thể là terminal hoặc log của CI. Một người chỉ đọc `out` rồi bảo "không có lỗi" đã kiểm tra sai kênh. Nếu thao tác mở `out` thất bại, chương trình đích có thể chưa từng chạy; phân biệt lỗi shell chuẩn bị môi trường với exit status do chương trình tự trả. Trong script tự động hóa, ghi lại lệnh nào được khởi động, mã thoát của từng công đoạn và trạng thái artifact đầu ra, nhưng không in đường dẫn chứa secret hoặc dữ liệu nhạy cảm.

Thực hành trên thư mục tạm với nguồn nhỏ do mình tạo và một đường dẫn nguồn cố ý không tồn tại. Dự đoán trước status của `tar`, `tee` và pipeline ở chế độ mặc định rồi ở chế độ `pipefail`; sau đó

đối chiếu giá trị thực từ phiên bản Bash và tar đang dùng. Thử thêm trường hợp đầu ra không ghi được. Điều kiện đạt là script không công bố `BACKUP_OK` khi bất kỳ công đoạn nào hỏng, và chỉ đổi tên file tạm thành backup cuối sau khi kiểm tra hậu điều kiện. Bài này nối cơ chế descriptor với ranh giới lỗi của chương tự động hóa sau: chính sách thành công do người viết script định nghĩa, không mặc nhiên mượn mã thoát của lệnh cuối.

Mặc định, exit status của `tar ... | tee log` là status của `tee`, không phải của `tar`. Nếu `tar` thoát 2 còn `tee` ghi log thành công và thoát 0, cả pipeline có status 0. `set -o pipefail` đổi status pipeline thành status khác 0 của lệnh lỗi nằm ngoài cùng bên phải; nó không nói lệnh nào hỏng nếu nhiều lệnh hỏng. Mảng `PIPESTATUS` trong Bash giữ status từng phần, nhưng phải sao chép ngay: chỉ cần chạy thêm một lệnh là trạng thái quan sát đã đổi. Status 0 nói rằng chương trình đã hoàn thành theo hợp đồng riêng của nó; không chứng minh bản backup khôi phục được. GNU `tar` phân biệt 0 thành công, 1 khi file thay đổi trong lúc tạo kho và 2 cho lỗi nghiêm trọng; `tee` thành công chỉ nói việc ghi dữ liệu nó nhận đã thành công theo hợp đồng của `tee`.

**Bài tập 14.2 — Pipeline báo thành công giả**

Trong script lỗi, giả sử `tar` trả 2 và `tee` trả 0. Dự đoán status pipeline, việc in `BACKUP_OK` và status cuối của script khi chưa bật `pipefail`. Bật `pipefail` đổi điều gì? Status 0 của `tar` sẽ và sẽ không chứng minh điều gì?

**Bài 14.3 — Luồng điều khiển**

`set -e` yêu cầu shell thoát khi một lệnh đơn giản thất bại trong những ngữ cảnh nhất định. Nó bỏ qua lỗi ở phép thử `if`/`while`, ở phần không cuối của chuỗi `&&`/`||`, ở lệnh không cuối của pipeline khi không bật `pipefail`, và ở một số ngữ cảnh phủ định. Nếu một hàm được gọi làm điều kiện `if`, các lệnh bên trong cũng chịu ngữ cảnh bỏ qua này: `false` có thể không chặn lệnh sau. Đây là quy tắc điều khiển của Bash, không phải một ngoại lệ của utility. `set -euo pipefail` có ích để bắt một số sơ suất, nhưng không định nghĩa thế nào là dữ liệu hợp lệ hoặc việc khôi phục thành công.

Command substitution `result=$(cmd)` chạy `cmd` trong môi trường subshell và bỏ các newline ở cuối output; status của phép gán đơn giản phản ánh substitution cuối. Bash không ở POSIX mode thường tắt `errexit` trong môi trường substitution, trừ khi bật `inherit_errexit`; vì vậy các lệnh bên trong cần xử lý status rõ. Một bẫy khác là `local result=$(cmd)`: status người gọi thấy có thể là của builtin `local`, che status của `cmd`. Tách `local result` và `result=$(cmd) || ...` khi kết quả quan trọng. Không nhét một thao tác có side effect vào `echo "$(...)"` rồi chỉ kiểm tra status của `echo`.

Có thể lần theo lỗi che khuất bằng một hàm giả định không đụng dữ liệu: `capture() { local out=$(false); printf 'ready\n'; }`. Dưới `set -e`, `false` trả 1 trong substitution, nhưng `local` khai báo biến thành công và trả 0; `printf` vẫn chạy, nên người gọi có thể thấy `ready` và status 0. Ý định đúng là `capture() { local out; out=$(false) || return $?; printf 'ready\n'; }`: phép gán đơn giản trả status của substitution, nhánh `||` lan truyền lỗi và dòng `ready` không chạy. Nếu lệnh thay `false` gồm nhiều bước, chính các bước bên trong cũng phải định nghĩa thất bại nào cần dừng; chỉ tách `local` không sửa được một chương trình con tự báo thành công giả. Trước khi đưa pattern vào script backup, thử cả lệnh thành công và thất bại trên dữ liệu

tạm, so stdout, status của hàm và sự tồn tại của artifact cuối.

Bash khởi tạo tiến trình con cho utility bên ngoài, chờ status và quyết định nhánh tiếp theo. `trap ... EXIT` chạy khi shell thoát bình thường hoặc do lỗi khiến shell thoát; `trap ... INT` và `TERM` xử lý tín hiệu tương ứng nếu shell có cơ hội nhận và xử lý chúng. `SIGKILL` không bắt được. Khi Bash đang chờ một lệnh đồng bộ ở foreground, trap có thể bị hoãn tới lúc lệnh đó kết thúc. Một trap trên shell cha không tự hủy toàn bộ cây tiến trình con. Vì vậy trap phù hợp để dọn file tạm đã biết, nhưng không thay cơ chế quản lý process group, timeout và shutdown của trình quản lý dịch vụ.

**Bài tập 14.3 — Giới hạn của set -e trong hàm**

Xét `f() { false; printf 'done\n'; }` rồi `if f; then printf 'ok\n'; fi` với `set -e`. Dự đoán các dòng in ra theo Bash và giải thích vì sao `false` không tự làm script thoát. Viết lại ý định “dừng nếu bước đầu thất bại” mà không dựa vào `set -e`.

# Chương 15 — Python và Go điều phối tiến trình

Đọc/viết CLI DevOps nhỏ bằng Python hoặc Go, xử lý argv, lỗi, timeout và test nhánh.

## Bài 15.1 — Chọn ngôn ngữ

Chương 14 cho thấy một pipeline Bash có thể che mã lỗi của bước sao lưu. Chuyển cùng việc ấy sang Python hoặc Go không tự sửa sai: chương trình vẫn có thể nhận lỗi rồi bỏ qua và in thông báo thành công. Khác biệt hữu ích nằm ở cách biểu diễn đối số, kết quả, thời hạn và trạng thái của từng bước. Ta sẽ điều phối ba bước nhỏ: kiểm tra điều kiện, áp dụng thay đổi giả, xác minh kết quả. Không bước nào chạm hệ thống thật; helper chỉ ghi vào thư mục tạm được đánh dấu cho demo.

Bash phù hợp cho vài lệnh tuyến tính trên một máy, với đầu vào nhỏ và hợp đồng lỗi rõ. Khi phải giữ trạng thái qua nhiều bước, đọc cấu hình có cấu trúc, kiểm tra dữ liệu, phân loại lỗi và kiểm thử nhánh phục hồi, Python thường cho mã dễ đọc hơn. Thư viện chuẩn của Python đã có subprocess, xử lý tệp, JSON và kiểm thử; đổi ngôn ngữ chỉ để gọi một lệnh đơn lẻ không mang lại an toàn. Go phù hợp khi cần một chương trình biên dịch triển khai trên nhiều máy, kiểu dữ liệu và luồng lỗi tường minh, hoặc nhiều tác vụ I/O đồng thời với cancellation chung. Một binary Go vẫn phụ thuộc hệ điều hành, kiến trúc, tùy chọn build và có thể phụ thuộc thư viện động; "một file" không có nghĩa chạy được ở mọi nơi. Python cần interpreter và môi trường package tương thích. Quyết định dựa trên độ phức tạp trạng thái, môi trường triển khai, năng lực nhóm và chi phí vận hành, không dựa trên nhãn "ngôn ngữ DevOps".

Trong cả ba ngôn ngữ, chương trình điều phối không đồng nhất với tác vụ nó khởi chạy. Một lệnh con có thể chạy xong và trả status khác 0; không khởi chạy được; bị timeout; hoặc trả 0 nhưng kết quả nghiệp vụ sai. Từng trạng thái cần chính sách riêng. Exit status là quan sát từ tiến trình con, còn "triển khai thành công" là kết luận sau khi kiểm tra hậu điều kiện. Không ghép hai ý ấy thành một biến boolean duy nhất.

Hai ví dụ chỉ dùng thư viện chuẩn. Với Python, ghi rõ phiên bản interpreter, tạo virtual environment cho package của dự án và khóa dependency theo công cụ đóng gói đã chọn; virtual environment là thứ tái tạo được, không nên coi thư mục của nó là artifact có thể di chuyển nguyên vẹn giữa máy. Không đưa secret vào package hoặc Git. Test nên chạy trong thư mục tạm có dấu hiệu nhận diện, xác minh cả status lẫn trạng thái cuối, rồi xóa đúng thư mục ấy sau khi đối chiếu. Với thay đổi thật, thêm kiểm thử hậu điều kiện từ góc nhìn người dùng, quan sát rollback và quyền thao tác; kết quả demo chỉ chứng minh các nhánh được thử trong môi trường thử.

## Bài 15.2 — Tiến trình con

Python `subprocess.run` nhận danh sách đối số; Go `exec.CommandContext` nhận tên chương trình cùng từng đối số. Hai API này mặc định không gọi shell. Giá trị như `"a; rm -rf ..."` đi thành một đối số, không thành lệnh thứ hai. Điều đó chặn một kiểu shell injection tại ranh giới này, nhưng utility con vẫn có thể hiểu dấu gạch đầu như tùy chọn, đọc file cấu hình độc hại hoặc tự gọi shell. Nếu cố dùng `shell=True` hay `sh -c` với chuỗi ghép từ đầu vào, ranh giới parsing của Bài 14.1 trở lại. Dùng argv tách rời, xác nhận giá trị theo schema và hợp đồng của chương trình con, chọn đường dẫn executable đáng tin, rồi dùng dấu kết thúc tùy chọn khi utility hỗ trợ.

Đường dẫn làm việc, PATH, biến môi trường, locale, quyền và tài nguyên của tiến trình con ảnh hưởng kết quả. Python env=None và Go Cmd.Env=nil cho con kế thừa môi trường hiện tại; điều này tiện trong demo nhưng có thể chuyển cả credential hoặc cấu hình ngoài dự kiến. Bí mật trong argv có thể lộ qua danh sách tiến trình hoặc log; bí mật trong environment cũng không phải két an toàn tuyệt đối. Đừng in nguyên command, exception, environment hoặc output khi chúng có thể chứa secret. Chỉ ghi mã bước, loại lỗi và request ID đã lọc; cấp secret qua cơ chế quản lý riêng với quyền tối thiểu. Trong ví dụ, output con bị bỏ đi vì ta đã biết helper nhỏ và chỉ cần status; công cụ thật cần log giới hạn kích thước, che dữ liệu nhạy cảm và đủ chứng cứ chẩn đoán.

Helper sau là tác vụ giả. Nó chỉ hoạt động khi thư mục đích có file .demo-only. Sau khi khởi tạo, nó phát READY rồi chờ lệnh go: nhờ vậy hạn tác vụ chỉ bắt đầu khi cả hai phía đã sẵn sàng. Biến DEMO_STARTUP_DELAY chỉ dùng trong kiểm thử để mô phỏng máy khởi tạo chậm. apply_fail ghi partial rồi trả 7. apply_slow ghi partial, ngủ hai giây, rồi mới định ghi ready. verify chỉ trả 0 nếu trạng thái là ready. Như vậy một lỗi hoặc timeout có thể để lại dấu vết của tác dụng phụ trước khi tiến trình con dừng. Không suy từ timeout rằng không có gì đã xảy ra.

```
"""Demo child: only write inside a marked temporary directory."""
import os
from pathlib import Path
import sys
import time

if len(sys.argv) != 3:
    sys.exit(2)
action, work_arg = sys.argv[1:]
work = Path(work_arg)
if not (work / ".demo-only").is_file():
    sys.exit(2)
state = work / "state.txt"
time.sleep(float(os.environ.get("DEMO_STARTUP_DELAY", "0")))
print("READY", flush=True)
if sys.stdin.readline() != "go\n":
    sys.exit(2)
if action == "precheck":
    sys.exit(0)
if action == "apply_ok":
    state.write_text("ready\n", encoding="utf-8")
    sys.exit(0)
if action in ("apply_fail", "apply_slow"):
    state.write_text("partial\n", encoding="utf-8")
    if action == "apply_fail":
        sys.exit(7)
    time.sleep(2)
    state.write_text("ready\n", encoding="utf-8")
    sys.exit(0)
if action == "verify":
    sys.exit(0 if state.is_file() and
             state.read_text(encoding="utf-8") == "ready\n" else 8)
sys.exit(2)
```

Chương trình Python gọi cùng helper theo thứ tự precheck → apply → verify. Với lệnh đơn giản, subprocess.run(check=False) trả CompletedProcess khi con kết thúc, kể cả status 7

hoặc 8; người gọi phải đọc `returncode`. Quá hạn tác vụ, chương trình kill và reap con trực tiếp; lỗi khởi chạy hoặc mất tín hiệu sẵn sàng được gắn nhãn khác với lỗi bước. Nhãn `TIMEOUT` không phải exit status của helper. Nhánh `bad` ghi nhận lỗi nhưng vẫn chạy bước sau và in `DEPLOY_OK`; nhánh `checked` dừng khác 0 ngay ở lỗi đầu. Cả hai nhánh cố ý để lại `partial` để người học thấy kết luận sai và giới hạn của việc dừng sớm.

```
"""Demo only: 'bad' deliberately ignores a failed step."""
import queue
import subprocess
import sys
import threading

STARTUP_LIMIT = 10  # Chỉ chờ tín hiệu READY, không tính vào hạn tác vụ.
TASK_LIMIT = 0.5


def run_step(probe, work, action):
    try:
        child = subprocess.Popen(
            [sys.executable, probe, action, work],
            stdin=subprocess.PIPE, stdout=subprocess.PIPE,
            stderr=subprocess.DEVNULL,
        )
    except OSError:
        return "START_ERROR"
    ready = queue.Queue(maxsize=1)
    threading.Thread(target=lambda: ready.put(child.stdout.readline()),
                     daemon=True).start()
    try:
        line = ready.get(timeout=STARTUP_LIMIT)
    except queue.Empty:
        child.kill()
        child.wait()
        return "START_TIMEOUT"
    if line.strip() != b"READY":
        child.kill()
        child.wait()
        return "START_ERROR"
    try:
        child.communicate(input=b"go\n", timeout=TASK_LIMIT)
    except subprocess.TimeoutExpired:
        child.kill()
        child.communicate()
        return "TIMEOUT"
    except OSError:
        child.kill()
        child.wait()
        return "IO_ERROR"
```

```
    return "OK" if child.returncode == 0 else f"EXIT {child.returncode}"

if len(sys.argv) != 5:
    sys.exit("usage: script.py bad|checked ok|fail|slow probe.py workdir")
policy, mode, probe, work = sys.argv[1:]
if policy not in ("bad", "checked") or mode not in ("ok", "fail", "slow"):
    sys.exit(2)

for action in ("precheck", "apply_" + mode, "verify"):
    status = run_step(probe, work, action)
    if status != "OK":
        print(f"{action}: {status}", file=sys.stderr)
        if policy == "checked":
            sys.exit(1)
print("DEPLOY_OK")
```

```
package main

import (
 "bufio"
 "context"
 "errors"
 "fmt"
 "io"
 "os/exec"
 "strings"
 "time"
)

func runStep(python, probe, work, action string) string {
 ctx, cancel := context.WithCancel(context.Background())
 defer cancel()
 cmd := exec.CommandContext(ctx, python, probe, action, work)
 input, err := cmd.StdinPipe()
 if err != nil {
  return "START_ERROR"
 }
 output, err := cmd.StdoutPipe()
 if err != nil {
  return "START_ERROR"
 }
 cmd.Stderr = io.Discard
 if err = cmd.Start(); err != nil {
  return "START_ERROR"
 }
 ready := make(chan string, 1)
 go func() {
  line, _ := bufio.NewReader(output).ReadString('\n')
  ready <- line
 }()
 select {
 case line := <-ready:
  if strings.TrimSpace(line) != "READY" {
   cancel()
   cmd.Wait()
   return "START_ERROR"
  }
 case <-time.After(10 * time.Second):
```

```
 cancel()
 cmd.Wait()
 return "START_TIMEOUT"
}
if _, err = io.WriteString(input, "go\n"); err != nil {
 cancel()
 cmd.Wait()
 return "IO_ERROR"
}
input.Close()
done := make(chan error, 1)
go func() { done <- cmd.Wait() }()
select {
case err = <-done:
 if err == nil {
  return "OK"
 }
 var exit *exec.ExitError
 if errors.As(err, &exit) {
  return fmt.Sprintf("EXIT %d", exit.ExitCode())
 }
 return "IO_ERROR"
case <-time.After(500 * time.Millisecond):
 cancel()
 <-done
 return "TIMEOUT"
}
}
```

Với API ngắn hơn, `Run` trả `nil` khi lệnh chạy và kết thúc thành công; ở đây `Wait` có cùng ranh giới lỗi sau `Start`.

**Bài tập 15.1 — Trạng thái từng bước của chương trình Python**

Với `orchestrate_python.py bad fail`, dự đoán thứ tự các bước, status của helper ở từng bước, output cuối và nội dung `state.txt`. Sau `Popen` và `communicate`, chương trình lấy kết quả thực thi từ đâu? Vì sao nhánh `bad` vẫn báo thành công dù đã nhận kết quả lỗi, còn nhánh `checked` dừng?

**Bài tập 15.2 — Đối số argv và nguy cơ chèn lệnh**

Một tên môi trường do người dùng nhập là `prod; echo LEAK`. So sánh truyền nó như một phần tử argv với ghép vào chuỗi `sh -c`. Vì sao argv tách rời vẫn phải kiểm tra schema và quy tắc tùy chọn của utility? Nêu một nơi có thể làm lộ secret dù không dùng shell.

**Bài 15.3 — Giới hạn thời gian**

Ở nhánh Python của ví dụ, ta dùng `Popen` để đợi `READY` riêng, rồi mới gửi `go` và gọi `communicate(timeout=0.5)`.

Go dùng `exec.CommandContext` nhưng chưa hủy context trong giai đoạn đợi `READY`. Sau tín hiệu đó, hàm gửi `go`, bắt đầu bộ đếm 0,5 giây và đợi `Wait`; nếu hết hạn, nó hủy context để kill con trực tiếp rồi reap. Status khác 0 thường cho `*exec.ExitError`, còn lỗi tìm executable hoặc I/O là loại

lỗi khác. Mã phải kiểm tra kết quả, không chỉ ghi log rồi bỏ qua. Nhánh `bad` dưới đây mắc đúng lỗi quyết định đó; nhánh `checked` dừng. Output con ngoài tín hiệu đồng bộ bị bỏ khỏi log demo, không phải mẫu quan sát phù hợp cho mọi dịch vụ.

```
// Demo only: policy "bad" deliberately ignores a failed step.
package main

import (
 "fmt"
 "os"
)

func main() {
 if len(os.Args) != 6 {
  fmt.Fprintln(os.Stderr, "usage: app bad|checked ok|fail|slow python probe workdir")
  os.Exit(2)
 }
 policy, mode := os.Args[1], os.Args[2]
 if (policy != "bad" && policy != "checked") ||
  (mode != "ok" && mode != "fail" && mode != "slow") {
  os.Exit(2)
 }
 python, probe, work := os.Args[3], os.Args[4], os.Args[5]
 for _, action := range []string{"precheck", "apply_" + mode, "verify"} {
  status := runStep(python, probe, work, action)
  if status != "OK" {
   fmt.Fprintf(os.Stderr, "%s: %s\n", action, status)
   if policy == "checked" {
    os.Exit(1)
   }
  }
 }
 fmt.Println("DEPLOY_OK")
}
```

Hàm chạy từng bước tách rời chính sách `bad/checked`: nó chỉ trả loại kết quả quan sát được. Cờ `TIMEOUT` do bộ đếm của cha xác định; lỗi trả về từ `Wait` sau khi kill không phải exit status nghiệp vụ của helper.

Python `run(timeout=...)` bao cả thời gian giao tiếp với con sau khi nó được tạo, nhưng khởi tạo tiến trình trên một số nền tảng có thể chưa bị ngắt ngay đúng deadline. Go `CommandContext` gắn context cho command và mặc định kill process khi context kết thúc; nó không tự rollback file đã ghi. Cả hai ví dụ chỉ quản lý tiến trình con trực tiếp. Nếu con khởi chạy cháu, giữ pipe mở hoặc gọi một API từ xa đã nhận yêu cầu, hết hạn ở phía điều phối không chứng minh mọi tác vụ bên ngoài đã dừng. Với Go, việc chờ pipe đóng và cách hủy có thể cần `WaitDelay` hoặc quy ước shutdown riêng; với Python, một process group có thể cần quản lý riêng. Không mặc định kill toàn bộ cây tiến trình bằng một lời gọi.

Sau lỗi, phải xác định trạng thái bền vững hiện tại rồi mới quyết định thử lại, tiếp tục hay quay lui. Retry một thao tác ghi có thể nhân đôi tác dụng nếu lần đầu đã hoàn tất ở hệ thống đích nhưng phản hồi bị mất, như tình huống HTTP ở Bài 10.3. Chỉ retry khi thao tác idempotent theo hợp đồng, hoặc có khóa chống lặp và khả năng truy kết quả cũ; dùng số lần và tổng thời gian hữu hạn. `finally` của

Python và `defer` của Go phù hợp giải phóng tài nguyên cục bộ, nhưng không thay giao dịch hay rollback phân tán; `os.Exit` trong Go còn bỏ qua các `defer` đang chờ ở cùng tiến trình. Việc xóa file `partial` mù quáng có thể che trạng thái cần điều tra hoặc xóa công việc hợp lệ của lượt khác. Demo dừng và giữ dấu vết, không tự tuyên bố đã khôi phục.

Ma trận kiểm thử có kiểm soát phải đối chiếu cả status lẫn file trạng thái: `ok` in `DEPLOY_OK`, thoát 0 và có `ready`; `bad fail` in `DEPLOY_OK` dù `apply` trả 7, `verify` trả 8 và file còn `partial`; `checked fail` thoát 1 trước `verify`; `checked slow` thoát 1 vì timeout sau khi đã ghi `partial`. `bad slow` vẫn in thành công giả. Ca `checked ok` còn ép khởi tạo helper chậm hơn hạn cũ 0,3 giây để chứng minh đồng hồ tác vụ chưa chạy trong lúc khởi tạo. Đây là phép thử của demo, không phải chứng nhận cho quy trình triển khai thật.

**Bài tập 15.3 — Timeout và trạng thái còn dở trong Go**

Với bản Go `checked slow`, hãy lần theo `READY`, `go`, lúc bắt đầu bộ đếm và kết quả `Wait`. Vì sao `TIMEOUT` khác exit status của helper, còn file vẫn chứa `partial` dù điều phối thoát 1? Nêu điều gì không thể kết luận về tiến trình cháu hoặc dịch vụ từ xa.

# Chương 16 — Quy trình tự động hóa an toàn và chạy lại

Thiết kế tác vụ nhiều bước có nhận diện partial failure.

## Bài 16.1 — Hợp đồng bước

Ví dụ lỗi dưới đây nhận thư mục nguồn và thư mục đích đã tồn tại. Nó được thiết kế để quan sát lỗi status, không có lệnh xóa nguồn. Không dùng nó cho backup thật.

```
#!/usr/bin/env bash
# Ví dụ lỗi: chỉ chạy trên dữ liệu thử nghiệm.
set -e
src=$1
out=$2
tar -cf "$out/backup.tar" -C "$src" . 2>&1 | tee "$out/backup.log"
printf 'BACKUP_OK\n'
```

Trong thử nghiệm trên Bash 5.2.21, GNU tar 1.35 và GNU coreutils 9.4, nguồn đầu vào là một thư mục không tồn tại, còn thư mục đích là thư mục tạm rỗng. `tar` không thể đổi sang nguồn và thoát 2; `tee` ghi thông báo lỗi vào log rồi thoát 0. Vì không bật `pipefail`, status pipeline bằng 0; `set -e` không chặn `printf`, script in `BACKUP_OK` và thoát 0. Dòng `BACKUP_OK` vì thế là kết luận của shell dựa trên status bị che, không phải chứng cứ backup tốt. Trên hệ thống khác, nội dung và ngôn ngữ thông báo lỗi, cũng như file tar dở dang, có thể khác. Trong một script có bước xóa nguồn phụ thuộc status này, hậu quả có thể là mất dữ liệu; ví dụ thực thi ở đây cố ý không có bước ấy.

Lần theo trạng thái của chính ví dụ backup: ban đầu đích rỗng; `tar` thất bại khi đọc nguồn, `tee` vẫn có thể ghi `backup.log`; Bash nhận status 0 từ lệnh cuối pipeline, nên đi tới `printf`. Kết quả là thông báo thành công và status 0 của script, trong khi không có bằng chứng archive dùng được. Tệp `backup.tar` nếu xuất hiện sau lỗi vẫn có thể dở dang; sự tồn tại hoặc kích thước dương của nó không thay phép thử đọc và khôi phục. Bật `pipefail` sẽ làm pipeline này trả khác 0 khi `tar` lỗi, nhưng chưa chứng minh dữ liệu nguồn nhất quán, archive bền sau mất điện hoặc bản khôi phục đúng nghiệp vụ.

Đối chiếu status với trạng thái đầu ra là phép kiểm chung của cả hai ví dụ. Ở ca backup, log do `tee` ghi không phải bản sao lưu; điều kiện tối thiểu là archive có thể liệt kê và trích xuất dữ liệu mong đợi trong môi trường riêng. Ca điều phối Python/Go ở chương trước có cùng hợp đồng **kết quả tiến trình** và **hậu trạng thái cần đọc lại**: `DEPLOY_OK` cũng không thay được trạng thái cuối `ready`. Hai ví dụ khác cơ chế lỗi nhưng đều cho thấy vì sao thông báo thành công của chương trình không đủ để xóa nguồn hay báo hoàn tất công việc.

### Bài 16.2 — Thử lại có điều kiện

Khi script chạy lại, phải định nghĩa rõ trạng thái nào có thể tái sử dụng. Việc `backup.tar` đã tồn tại không chứng minh nó đầy đủ; việc bản mới được tạo cũng không có nghĩa được phép xóa bản cũ. Một cách an toàn hơn là ghi vào file tạm trong chính thư mục đích, kiểm tra thành công của lệnh tạo, rồi công bố bằng thao tác không ghi đè tên cuối. Điều này xử lý xung đột tên, nhưng không giải quyết nguồn thay đổi trong lúc đọc, mất điện trước khi dữ liệu được đồng bộ, hay đối thủ đổi đường dẫn giữa kiểm tra và sử dụng. Quyền truy cập thư mục, snapshot nhất quán, kiểm thử khôi phục và chính sách giữ bản cũ là những hợp đồng riêng.

Xét hai lượt chạy giả định cùng dùng tên cuối `backup-2026-10-20.tar`. Lượt A đã ghi xong tệp tạm nhưng chết trước khi công bố; lượt B thấy tên cuối chưa có. B được phép tạo tệp tạm riêng, nhưng không được dùng sự vắng mặt của tên cuối để suy ra tệp tạm của A hợp lệ hay có thể xóa an toàn. Ngược lại, nếu A đã công bố rồi nhưng mất phản hồi, B phải xem tên cuối là một kết quả cần kiểm tra danh tính và khả năng khôi phục, không ghi đè để "thử lại". Hậu điều kiện hữu ích là đúng một bản cuối được chấp nhận, đọc lại được và có dấu vết của lượt đã tạo nó; script không thể tự bảo đảm tính nhất quán của nguồn nếu thiếu snapshot hoặc hợp đồng với ứng dụng.

Thiết kế lại thao tác theo trạng thái, không theo thông báo cuối của lệnh. Trạng thái `PREPARING` có tệp tạm riêng của lượt chạy; `VALIDATED` nghĩa là archive đã được đọc kiểm, kích thước và danh sách thành phần hợp lệ trong phạm vi bài thử; `PUBLISHED` nghĩa là tên cuối trỏ tới một archive đã chọn. Nếu process chết ở `PREPARING`, lượt sau không được dùng file ấy chỉ vì nó có tên giống backup. Nếu chết sau `PUBLISHED` nhưng trước khi trả lời người gọi, retry phải đọc và xác minh manifest/digest của bản cuối trước khi quyết định đây là kết quả của lần trước hay xung đột. Một tên cuối đã tồn tại và có digest khác là lỗi xung đột, không phải lời mời ghi đè.

Đặt cạnh nhau hai kiểu race. Kiểm `test -e final` rồi ghi `final` là check–then–act: A và B đều có thể thấy "chưa có" rồi cùng ghi. Trên Linux, `rename()` thông thường **có thể thay thế** tên đích đã tồn tại; muốn từ chối ghi đè trong một bước cần cơ chế phù hợp như `renameat2()` với `RENAME_NOREPLACE` trên filesystem hỗ trợ, hoặc một giao thức công bố khác có bảo đảm tương đương. Khi ấy đúng một bên có thể thắng tại thời điểm tạo tên cuối, nhưng vẫn cần kiểm tra thuộc tính file tạm, cùng filesystem, quyền thư mục và độ bền sau mất điện. Việc đổi tên nguyên tử đối với người đọc không đồng nghĩa byte đã được đồng bộ bền vững trên ổ. Một bản backup có tính phục hồi còn cần nhất quán nguồn, bảo vệ khỏi sửa/xóa trái phép, sao chép ra failure domain khác và bài restore định kỳ. Đây là các bảo đảm tách rời; không gộp chúng dưới nhãn "idempotent".

Thực hành trên thư mục tạm mới, với hai tiến trình cùng nhắm một tên cuối và input nhỏ đã biết. Tiêm lỗi bằng cách dừng một lượt trước công bố và một lượt ngay sau công bố; ghi kết quả của từng lượt cùng digest, file tạm còn lại và nội dung bản cuối. Tiêu chí đạt là retry không ghi đè bản cuối khác identity, không tuyên bố thành công khi archive không đọc được, và người vận hành có thể xác định tệp tạm nào an toàn để dọn sau khi điều tra. Không thử race bằng thư mục backup hoặc dữ liệu đang dùng thật.

### Bài tập 16.1 — Hai lượt công bố bản sao lưu

Hai lượt backup A và B cùng nhắm tên `backup.tar`. A đã tạo tệp tạm rồi chết trước khi công bố; B công bố một archive khác, sau đó A được chạy lại. Vì sao kiểm `test -e backup.tar` rồi dùng `rename()` thường vẫn có thể ghi đè bản của B? A cần quyết định gì khi thấy tên cuối đã tồn tại, và cần kiểm hậu điều kiện nào trước khi báo backup thành công?

### Bài 16.3 — Phục hồi

Phiên bản sau giữ từng đường lỗi ở vị trí có thể kiểm tra. Nó yêu cầu GNU `mktemp`, `tar`, `ln`, `rm`; tên cuối cố định và không cho ghi đè. `mktemp` tạo file riêng trong thư mục đích để tránh va chạm tên file tạm. Mọi nhánh trước `ln` đều thoát khác 0 và `EXIT` trap chỉ dọn đúng đường dẫn tạm đã tạo. `tar -tf` kiểm tra archive đọc được về mặt cấu trúc, không chứng minh dữ liệu có thể phục hồi đúng nghiệp vụ. `ln -T` tạo hard link tới file tạm trên cùng filesystem và buộc đối số đích được hiểu là đúng một tên file, kể cả khi tên ấy là symlink tới thư mục; nếu `backup.tar` đã tồn tại, GNU `ln` mặc định thất bại, nên lần chạy lại không âm thầm thay bản trước. Thành công của `ln` là điểm công bố tên, sau đó trap gỡ tên tạm và để hard link cuối giữ inode. Đích phải là thư mục tin cậy; một cây đường dẫn bị thay đổi đồng thời hoặc do đối thủ kiểm soát cần thiết kế an toàn hơn.

```
#!/usr/bin/env bash
# GNU tar/coreutils; đích phải là thư mục tin cậy cùng filesystem.
set -u
src=${1:?can thu muc nguon}
out=${2:?can thu muc dich}
[[ -d $src && -d $out ]] || exit 2

tmp=$(mktemp -- "$out/.backup.XXXXXX") || exit 3
cleanup() { rm -f -- "$tmp"; }
trap cleanup EXIT
trap 'exit 130' INT
trap 'exit 143' TERM

if tar -cf "$tmp" -C "$src" .; then
    :
else
    exit $?
fi
if tar -tf "$tmp" >/dev/null; then
    :
else
    exit 4
fi
ln -T -- "$tmp" "$out/backup.tar" || exit 5
printf 'BACKUP_OK\n'
```

Ví dụ kiểm chứng cô lập tạo một file `payload.txt` trong thư mục nguồn tạm, chạy script sửa lần đầu, rồi liệt kê archive và trích xuất sang một thư mục tạm khác để so nội dung. Lần thứ hai trả status 5 vì tên cuối đã tồn tại; bản cuối của lần đầu vẫn đọc được. Khi thay nguồn bằng đường dẫn không tồn tại, script dừng status 2 trước khi tạo file tạm. Những kết quả này kiểm tra đường thành công, lỗi đầu vào và xung đột tên cho môi trường thử; chúng không chứng minh tính bền vững sau mất điện, tính nhất quán của dữ liệu đang bị ghi, hay an toàn trước thay đổi đường dẫn có chủ ý. Một

backup production cần snapshot hoặc điểm dừng ghi phù hợp ứng dụng, checksum/manifest, thử khôi phục định kỳ, chính sách retention và đồng bộ tệp/thư mục theo hệ thống tệp thực tế. Bài học nối với Bài 3.3 về I/O bền vững, Bài 3.2 và Bài 4.2 về đường dẫn/quyền và Bài 12.2 về khác biệt giữa một snapshot đã ghi với working tree đang đổi.

**Bài tập 16.2 — Chạy lại bản sao lưu an toàn**

Trong script sửa, sau một lần thành công, lần chạy lại với cùng đích đi tới bước nào và status nào? Vì sao `tar -tf` cùng việc không ghi đè chưa đủ cho backup production? Phân biệt lỗi được `EXIT` trap dọn với rủi ro do `SIGKILL` hoặc nguồn thay đổi giữa chừng.

# PHẦN VI — Build, kiểm thử và phát hành

## Một commit trở thành artifact đang phục vụ người dùng bằng chứng cứ nào?

Source, build inputs, artifact digest, test gate, deployment state và dữ liệu là các vòng đời tách biệt.

VI

**LỘ TRÌNH PHẦN**


- Chương 17 — Dựng phần mềm và định danh sản phẩm
- Chương 18 — Cổng kiểm thử và CI/CD
- Chương 19 — Triển khai theo đợt và quyết định dừng
- Chương 20 — Phục hồi phiên bản và tương thích dữ liệu

# Chương 17 — Dựng phần mềm và định danh sản phẩm

Chứng minh production chạy đúng byte đã qua kiểm thử.

## Bài 17.1 — Đầu vào của bản dựng

Bài 12.3 cho ta một commit xác định mã nguồn; Bài 15.3 cho thấy chương trình điều phối có thể báo thành công dù hệ thống còn ở trạng thái từng phần. Phát hành một dịch vụ nối hai điều ấy bằng một chuỗi quyết định có chứng cứ: chọn commit, tạo artifact, kiểm thử chính artifact ấy, cho phép triển khai, chuyển lưu lượng, kiểm tra hành vi thật và chuẩn bị đường phục hồi. Mỗi mũi tên có thể thất bại độc lập. Một pipeline màu xanh không thay thế được câu hỏi người dùng đang nhận phiên bản nào và dữ liệu đã đổi ra sao.

Commit định danh một trạng thái của repository, không định danh duy nhất binary hay image. Hai lần build từ cùng commit có thể khác byte vì dependency được giải lại, toolchain khác, timestamp, thứ tự đóng gói hoặc cờ build khác. Muốn giảm biến thiên phải khóa đầu vào, phiên bản compiler, base image, cấu hình build và môi trường; nếu đòi tái lập từng byte, phải đo bằng cách build độc lập rồi so digest, thay vì suy từ một file lock. Tính tái lập là thuộc tính của toàn quá trình và những đầu vào được kiểm soát, không phải lời hứa của tên phiên bản.

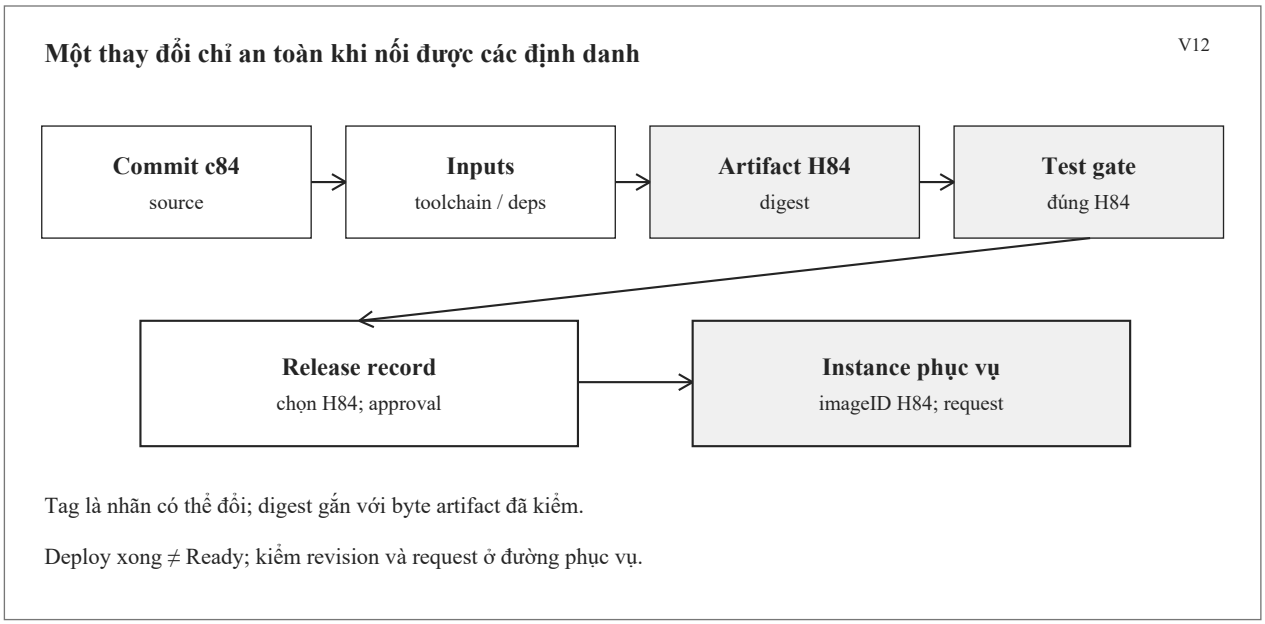
Với Go, module ghi yêu cầu ở `go.mod`, checksum dependency được quản lý ở `go.sum`; cần ghi cả phiên bản toolchain, target OS/architecture và cờ build khi muốn tái tạo binary.

Hãy lần theo một ca giả định: job kiểm thử lấy commit `c7`, chạy test trên mã nguồn và báo xanh. Job đóng image sau đó lại dùng base image qua tag có thể di chuyển; nó tạo digest `d8`, nhưng báo cáo test không ghi digest nào. Không có bằng chứng test đã chạy trên đúng các byte của `d8`. Nếu production lấy image bằng tag ấy ở một thời điểm khác, còn thêm một phép phân giải tag phải kiểm. Hồ sơ tối thiểu cần nối commit và các đầu vào build với build run, digest đầu ra, test run dùng chính digest đó, rồi revision triển khai và digest đang chạy. Các ký hiệu `c7`, `d8` chỉ là dữ liệu minh họa; một job xanh không tự điền được mắt xích còn thiếu.

Trước khi gọi đây là bản phát hành, kiểm từng ranh giới: compiler chấp nhận source chỉ xác nhận build được; test pass chỉ áp cho artifact và môi trường đã thử; deployment thành công chỉ nói hệ điều phối đã nhận hoặc thực hiện thay đổi theo hợp đồng của nó. Readiness và request nghiệp vụ sau chuyển lưu lượng mới cho bằng chứng phục vụ, còn dữ liệu đã ghi cần hậu kiểm riêng. Nếu thiếu digest của artifact được test, quyết định an toàn là giữ gate phát hành, ghi rõ bằng chứng thiếu và tạo lại đường test–promote có danh tính byte thống nhất; không suy từ cùng tên tag rằng hai lượt chạy là một artifact.

## Bài 17.2 — Định danh sản phẩm dựng

Artifact là tập byte đã được đóng gói để chạy. Digest tính từ byte cho phép kiểm tra ta đang dùng đúng nội dung đã kiểm thử; nhãn như `orders:1.8.4` chỉ là tên có thể bị trỏ lại nếu kho cho phép. Với image, descriptor chứa digest và kích thước để người nhận xác thực nội dung. Bản ghi provenance nối subject artifact với nguồn và cách build; nó giúp điều tra và kiểm tra chính sách, nhưng chỉ đáng tin trong phạm vi builder và quy trình tạo chứng cứ được tin cậy. Giữ riêng ba danh tính: commit nguồn, digest artifact và bản ghi triển khai ở môi trường. Khi quảng bá từ thử nghiệm sang production, dùng lại cùng digest; build lại từ cùng commit là một artifact mới, cần xét lại bằng chứng kiểm thử tương ứng.




Hình VI.1.1 — Từ commit đến tiến trình phục vụ: mỗi cổng giữ một bằng chứng riêng.

Giả sử commit `c1` tạo image có digest `sha256:d1` trong lần build được kiểm thử. Một lần build khác từ `c1` cho `sha256:d2` vì base image hoặc dependency đã đổi. Báo cáo "production chạy commit c1" không cho biết production đang chạy byte nào; cần liên kết bản ghi thử nghiệm với `d1` hoặc kiểm thử lại `d2`. Các chuỗi `c1`, `d1`, `d2` ở đây là ký hiệu giả định, không phải digest thực. Nếu hệ thống triển khai qua tag, chụp digest được tag phân giải ở thời điểm chọn và digest của image trên workload sau triển khai; so hai giá trị ấy trước khi dùng kết quả test làm bằng chứng phát hành.

Ba phép so thường bị gọi chung là "cùng phiên bản". So commit chỉ nói đầu vào mã nguồn được chọn; nó không cố định base image, dependency, toolchain, build args hay thời gian tạo. So digest artifact nói byte đóng gói giống nhau theo thuật toán và phạm vi hash, nhưng không nói bộ byte ấy đã vượt test nào. So deployment record với runtime nói artifact nào đang chạy ở một instance tại thời điểm quan sát; nó không đảm bảo mọi instance đã chuyển. Một hồ sơ phát hành cần nối ba phép so bằng cùng một khóa: `commit c1 → build run b7 → artifact d1 → test run t4 → deployment revision r9 → runtime digest d1`. Nếu mắt xích `t4` thực ra kiểm `d2`, kết luận "đã test bản đang chạy" sai dù các record riêng lẻ đều có thật.

Tính tái lập là một phép kiểm khác: với input được khóa và môi trường build được kiểm soát, hai build độc lập lý tưởng cho cùng digest. Nếu khác, truy ngược các nguồn không xác định như

dependency không khóa, timestamp nhúng, thứ tự file hoặc base tag có thể dịch. Tuy nhiên hai digest giống nhau cũng không chứng minh test đủ sâu hoặc builder không bị xâm nhập. Provenance cho biết ai hoặc hệ nào tuyên bố đã build từ input nào; chữ ký chỉ hữu ích khi identity builder và khóa ký được tin cậy. Khi quyết định promote, lấy chính digest đã qua gate; đừng build lại chỉ để đổi nhãn production rồi dựa vào kết quả test của byte cũ.

Người học có thể thử với artifact nhỏ: ghi hash của input, dependency khóa, output và metadata build; tạo lại output hai lần trong môi trường tạm, đối chiếu byte và giải thích mọi sai khác. Nếu thiếu container runtime, dùng một gói file thử với build script tối giản vẫn dạy được ranh giới commit–artifact–runtime, nhưng phải ghi đó là mô hình giới hạn. Hậu kiểm của một triển khai thật là hỏi runtime digest của từng instance đang nhận traffic và so với `d1`, kể cả instance cũ còn sót trong rollout. Không lấy tag trong cấu hình làm bằng chứng thay cho byte ở nơi chạy.

**Bài tập 17.1 — Định danh artifact khi chuyển môi trường**

Một pipeline build lại từ cùng commit khi đưa từ staging sang production, rồi chỉ so tên tag. Hãy chỉ ra mắt xích chứng cứ bị đứt và thiết kế bản ghi tối thiểu để chứng minh production chạy đúng artifact đã kiểm thử. Trường hợp nào hai lần build cùng commit có thể khác byte?

### Bài 17.3 — Tái lập

Build có thể được lặp lại mà chưa tái lập được. “Lặp lại” chỉ nói lệnh build chạy thêm một lần; “tái lập byte” yêu cầu cùng nguồn, môi trường và chỉ dẫn build đã xác định tạo cùng byte cho các artifact đã khai báo. Commit `c83` không đủ để suy digest `H83`: dependency tải động, phiên bản compiler, locale, đường dẫn build, thời điểm đóng gói và thứ tự duyệt tệp đều có thể ảnh hưởng. Nếu cache trả artifact cũ trong một lần nhưng rebuild sạch tạo byte khác, chỉ việc pipeline xanh hai lần không phát hiện được sai khác.

Một phép kiểm độc lập bắt đầu bằng ghi commit object ID, lockfile, toolchain, biến môi trường liên quan và chính xác tập đầu ra cần so. Dùng hai môi trường build tách biệt theo cùng mô tả, tính digest từ byte của từng artifact, rồi so từng digest. Ví dụ giả định: từ `c83`, build A và B cùng ra `H83`; đó là bằng chứng có giới hạn cho cấu hình và lần thử ấy. Từ `c84`, build A ra `H84` nhưng build B ra `H84b`; không được tuyên bố artifact của `c84` tái lập chỉ vì cả hai chương trình khởi động được. Lấy diff byte hoặc kiểm metadata archive để khoanh vùng; nếu trường timestamp biến thiên, chuẩn hóa đầu vào thời gian theo hợp đồng build rồi thử lại. Không sửa trực tiếp artifact sau khi ký hoặc test để làm hash khớp.

Digest bằng nhau chứng minh byte bằng nhau trong phạm vi thuật toán hash và tệp đã so; nó không chứng minh source an toàn, builder không bị xâm nhập hay test bao phủ đủ hành vi. Ngược lại, digest khác nhau không tự chứng minh có mã độc: có thể do timestamp hoặc nondeterminism vô hại, nhưng vẫn phá khả năng xác nhận chính xác byte nào được thử. Provenance có thể ghi nguồn, tham số, builder và subject digest; độ tin cậy của bản ghi phụ thuộc cách bảo vệ builder và khóa ký. Cần phân biệt bằng chứng build lặp với bằng chứng production đang chạy đúng digest đó.

Khi phát hành, promotion nên tham chiếu digest bất biến đã kiểm thử. Nếu phải rebuild cho production, coi đó là artifact mới và chạy lại gate phù hợp. Không lấy tag như `stable` hoặc nhãn

job thành bằng chứng byte identity vì chúng có thể bị cập nhật. Bài container sau sẽ phân biệt digest của image manifest, layer và writable layer của lần chạy; chúng là các ranh giới byte khác nhau.

**Bài tập 17.2 — Cùng commit nhưng khác digest**

Hai build cùng commit c84 đều chạy được nhưng cho digest H84 và H84b. Có thể promote H84b vì test trên H84 đã xanh không? Nêu bước kiểm tiếp.

# Chương 18 — Cổng kiểm thử và CI/CD

Thiết kế test kiểm đúng ranh giới của thay đổi và quyền release.

## Bài 18.1 — Tích hợp và triển khai liên tục

CI tích hợp thay đổi bằng build, kiểm tra và phản hồi sớm cho mỗi commit hoặc đề nghị hợp nhất. CD đưa một artifact đã qua cổng tới một hay nhiều môi trường; continuous delivery có thể dừng ở phê duyệt trước production, còn continuous deployment tự đi tiếp khi chính sách cho phép. Build tạo đầu ra; release là quyết định gắn đầu ra ấy với một phiên bản được phép dùng; deployment thay đổi cái đang chạy. Ba sự kiện có thể cách nhau nhiều ngày. Phân biệt chúng giúp biết chính xác cổng nào đã kiểm tra byte nào.

Môi trường triển khai có schema dữ liệu, cấu hình, quyền, secret, quota và lưu lượng riêng.

Hãy theo một thay đổi giả định: CI kiểm thử commit `c1`, tạo artifact `d1`; cổng release chấp nhận `d1`; CD đặt `d1` lên staging, rồi chờ người chịu trách nhiệm duyệt production. CI xanh chỉ chứng minh các phép kiểm đã chạy trên đầu vào ấy, trong điều kiện ấy. Nếu production kéo lại một tag đã trỏ sang `d2`, chuỗi chứng cứ bị đứt dù tên phiên bản vẫn giống; nếu schema production chưa tương thích, cùng `d1` cũng có thể thất bại. Vì vậy bản ghi của mỗi cổng phải giữ artifact digest, môi trường, bộ kiểm tra và hậu điều kiện. Trạng thái "đã triển khai" là một bước trong chuỗi, chưa phải kết quả kinh doanh hoặc readiness.

Đặt các gate ở đúng chỗ của hành trình thay đổi. Trước build, xác nhận source revision và quyền người thay đổi. Sau build, kiểm artifact digest và kết quả test trên chính digest ấy. Trước production, kiểm môi trường đích, schema, secret/quyền tối thiểu, capacity, phương án dừng và chủ thể phê duyệt. Sau deployment, kiểm controller đã chuyển phiên bản và request nghiệp vụ đã đi qua bản mới. Một pipeline có thể PASS mọi unit test nhưng FAIL ở readiness vì thiếu biến môi trường; không nên gọi đó là "CI bị lỗi" nếu bằng chứng chỉ ra cổng môi trường hoặc vận hành.

Ví dụ tính trạng thái: 10 instance đang phục vụ `d0`, rolling update định thay mỗi lượt 2 instance sang `d1`. Sau hai lượt, controller báo 4 instance mới Ready, 6 instance cũ vẫn nhận traffic; một request thành công từ `d1` chưa chứng minh release hoàn tất. Nếu một trong 4 instance mới ghi dữ liệu theo schema mới, rollback traffic về `d0` còn phải chứng minh 6 instance cũ đọc được dữ liệu ấy. Một dashboard gộp 10 instance có thể che lỗi chỉ xuất hiện ở nhóm mới; chia metric và log theo revision rồi so cùng khoảng thời gian, cùng loại request. Con số 4/10 là dữ kiện giả định để suy luận, không phải output deployment thực.

Khi một gate FAIL, lưu artifact và bằng chứng đã có, dừng quảng bá, xác định trạng thái đang phục vụ và tác dụng phụ đã xảy ra rồi mới quyết định rollback hoặc sửa tiến. Nếu gate SKIP vì thiếu môi trường, ghi đúng phép chưa chạy và người chấp nhận rủi ro; không đổi nhãn SKIP thành PASS. Người học nên trả lời được bốn câu cho bất kỳ trạng thái pipeline nào: byte nào được kiểm, môi trường nào được đổi, điều gì đang phục vụ request, và dữ liệu sau đổi có thể đọc/khôi phục ra sao. Cùng một pipeline xanh có thể trả lời đủ bốn câu hoặc không câu nào, tùy chất lượng record của nó.

### Bài 18.2 — Mô hình kiểm thử

Một bộ test chỉ bảo đảm những đường mà nó thực sự chạy với dữ kiện cụ thể. Trong tình huống `orders-api`, unit test và readiness đều xanh, nhưng một thay đổi làm `amount_cents` sai. Unit test không có trường hợp giá trị gây lỗi; readiness chỉ gọi đường probe. Hai phép kiểm này vì thế không kiểm tra hậu điều kiện quan trọng: số tiền của đơn đã ghi có bằng giá trị đúng theo input và quy tắc nghiệp vụ hay không.

Đặt hai cổng kiểm vào cùng một dữ kiện giả định sẽ thấy lỗ hổng. Bản `H84` có lỗi nhân `amount_cents` thêm 100 trước khi ghi; unit test hiện chỉ tạo đơn có giá trị 0 nên vẫn thấy `0 → 0`. Probe `/health` trả 200 vì tiến trình và dependency nó kiểm tra còn đáp ứng. Một request tạo đơn mang `amount_cents=1234` có thể nhận HTTP thành công nhưng bản ghi lại thành `123400`, trong khi sổ đối soát độc lập vẫn là `1234`. Màu xanh của hai cổng không hề quan sát phép biến đổi sai ấy. Các ID và con số ở đây là fixture dạy học, không phải kết quả đo production.

Thiết kế cổng kiểm thử bắt đầu từ hợp đồng dữ liệu. Với một fixture có input, kết quả mong đợi và định danh request, kiểm tra riêng phép tính, API và bản ghi sau khi giao dịch hoàn tất. Nếu dùng môi trường tích hợp, phải biết schema, cấu hình và dependency nào tham gia; một test chạy trên mock không chứng minh database thật có cùng hành vi. Khi canary nhận một phần lưu lượng, gửi phép thử qua đúng đường client, gắn nó với release nhận request và đối chiếu dữ liệu đã lưu. Tách kết quả của phép thử chủ động khỏi tỷ lệ lỗi trong traffic tự nhiên.

Phép thử phát hiện lỗi phải chỉ rõ byte được thử và điểm đọc. Gửi request có ID duy nhất qua canary `H84`, lưu input `1234`, response và revision phục vụ; sau khi giao dịch đã commit theo hợp đồng của ứng dụng, đọc bản ghi theo ID từ nguồn có mức nhất quán đã biết và so với giá trị mong đợi cùng nguồn đối soát. Đọc ngay từ một replica có thể trễ rồi gán sai khác cho phép tính là kết luận vội. Nếu phép thử tạo dữ liệu thật, dùng fixture và quyền phù hợp với môi trường thử; không tạo đơn hay thanh toán production chỉ để chứng minh ví dụ. Cổng này kiểm một đường dữ liệu cụ thể, không chứng minh mọi giá trị đầu vào đều đúng.

Một mẫu canary nhỏ có thể không gặp input gây lỗi, dù tất cả phép kiểm vừa chạy đều xanh. Ngược lại, bốn đơn quan sát thấy sai đủ để dừng mở rộng, nhưng không cho phép ước lượng tỷ lệ lỗi của toàn bộ traffic khi cách chọn mẫu chưa được biết. Giữ request ID, digest, thời điểm, giá trị trước/sau và đường đọc để khoanh tập dữ liệu bị ảnh hưởng; không sửa hàng loạt bằng cách chia mọi bản ghi cho 100 vì có thể lẫn đơn đúng do bản cũ ghi. Kết luận phải nêu phạm vi: điều gì đã được test, điều gì chưa được test, và bằng chứng nào cần thêm trước khi tăng lưu lượng.

#### Bài tập 18.1 — Kiểm thử bỏ sót sai lệch nghiệp vụ

Trong tình huống `orders-api`, unit test và readiness đều xanh. Vì sao hai tín hiệu ấy không bác bỏ lỗi `amount_cents`? Nêu phép kiểm tra hậu triển khai phát hiện sai lệch và giới hạn của mẫu canary nhỏ.

#### Bài tập 18.2 — Readiness xanh nhưng dữ liệu sai

Nếu readiness xanh nhưng amount_cents sai, phép kiểm nào còn thiếu?

### Bài 18.3 — Cổng quyết định

Các job là đồ thị phụ thuộc: kiểm tra tĩnh và unit test có thể chạy song song, nhưng kiểm thử tích hợp cần binary, database và cấu hình đã chuẩn bị; triển khai chỉ được dùng artifact mà các cổng đã kiểm tra. Unit test kiểm tra logic hẹp; integration test kiểm tra hợp đồng với thành phần thật hoặc mô phỏng có giới hạn; smoke test sau triển khai chỉ thử vài đường trọng yếu. Một test pass chứng minh những trường hợp đã chạy, không chứng minh không có lỗi trên dữ liệu production. Ghi input, phiên bản, kết quả và lý do bỏ qua cho từng cổng. Nếu thay artifact hoặc cấu hình có ảnh hưởng, không gắn nhãn "đã kiểm thử" từ lượt khác.

Secret được cấp ở môi trường cần dùng và không nằm trong source, artifact hay log. Cổng phê duyệt production cần cho người duyệt thấy digest, khác biệt cấu hình, kết quả test, kế hoạch dừng và phương án phục hồi; nút bấm phê duyệt không tự kiểm tra những điều đó. Hai bản phát hành chạy đồng thời vào cùng môi trường có thể ghi đè nhau: dùng một khóa điều phối theo đích, xác nhận phiên bản hiện tại trước khi thay và định nghĩa rõ yêu cầu mới sẽ chờ hay hủy lượt cũ. Khóa pipeline không thay thế khóa ở hệ thống đích nếu còn công cụ triển khai khác.

Một đồ thị cổng **giả định** có `build(c1) → d1`, rồi hai nhánh `unit(d1)` và `integration(d1)`; `release(d1)` chỉ mở khi cả hai nhánh đạt. Nếu integration fail nhưng job deploy chỉ phụ thuộc unit, pipeline vẫn có thể chuyển `d1` dù UI hiển thị một ô đỏ ở nơi khác: lỗi nằm ở cạnh phụ thuộc, không nằm ở kết quả unit. Nếu build lại thành `d2` sau khi hai test đã đạt trên `d1`, kết quả cũ cũng không áp cho `d2`. Khi chẩn đoán, lần theo digest ở đầu vào và đầu ra của từng job, trạng thái skipped/fail và điều kiện mở cổng. Sau deploy, xác nhận digest đang chạy và một request đại diện; cạnh cuối của đồ thị là hậu điều kiện ở môi trường đích chứ không phải màu của workflow.

# Chương 19 — Triển khai theo đợt và quyết định dừng

Chọn rolling, blue-green hay canary theo năng lực và khả năng quan sát.

## Bài 19.1 — Chiến lược

Rolling update thay dần instance cũ bằng mới, điều chỉnh số instance có thể thiếu và số được tăng thêm. Nó tiết kiệm tài nguyên hơn việc giữ hai cụm đầy đủ, nhưng cũ và mới cùng phục vụ trong một khoảng; schema, thông điệp và cache phải tương thích hai chiều trong thời gian đó. Blue-green chuẩn bị một tập mới rồi đổi tuyến lưu lượng; chuyển lại tuyến có thể nhanh nếu tập cũ còn khỏe, đổi lại phải trả thêm tài nguyên và giữ đồng bộ dữ liệu dùng chung. Canary gửi một phần lưu lượng tới bản mới, so cùng thời điểm với nhóm cũ rồi tăng dần. Nó giới hạn phạm vi ban đầu, nhưng mẫu nhỏ hoặc lưu lượng không đại diện có thể bỏ sót lỗi; sự kiện hiếm vẫn có thể gây hại ngay ở 1%. Chiến lược nào cũng cần xác định lưu lượng nền, phiên dài, queue, job nền và thao tác ghi không đi qua bộ cân bằng tải.

Rollback ứng dụng là đưa lưu lượng hoặc cấu hình chạy về artifact trước đó.

Giả sử bản mới bắt đầu ghi thêm một trường bắt buộc vào cùng bảng dữ liệu sau khi nhận 5% lưu lượng canary. Quay traffic về bản cũ có thể dừng lỗi ở request mới, nhưng bản cũ có đọc được các bản ghi đã được bản mới sửa hay không là một câu hỏi riêng. Trước khi tăng traffic, kiểm schema và định dạng sự kiện có tương thích hai chiều, định nghĩa phép đo lỗi theo cohort canary, ngưỡng dừng và thời gian quan sát. Nếu phát hiện dữ liệu đã thay đổi không tương thích, rollback image đơn thuần không khôi phục dữ liệu; cần phương án migration tiến/lùi hoặc phục hồi đã diễn tập, cùng ranh giới mất dữ liệu được chấp thuận.

Giả sử Deployment có bốn Pod cũ Available, `replicas: 4`, `maxSurge: 1`, `maxUnavailable: 0`. Controller có thể tạo Pod mới thứ năm nhưng phải giữ ít nhất bốn Pod Available. Nếu Pod mới chưa Ready, controller chưa thể giảm Pod cũ để tiếp tục; `apply` xanh không phải rollout hoàn tất. Khi Pod mới Available, controller mới thay dần bản cũ. Số Pod không quyết định tỷ lệ request: một Pod mới trong tập năm Pod không tự tạo canary 5%; cần định tuyến phù hợp và đo request theo revision. Pod đang terminating còn có thể chiếm tài nguyên ngoài mức surge danh nghĩa.

### Bài tập 19.1 — Chọn chiến lược triển khai khi thiếu tài nguyên

So sánh rolling, blue-green và canary cho dịch vụ vừa nêu khi chỉ có đủ tài nguyên cho thêm một Pod. Chiến lược nào thực tế hơn, điều kiện tương thích dữ liệu nào phải giữ, và khi nào cần dừng dù tỷ lệ lỗi tổng thể chưa tăng rõ?

### Bài 19.2 — Trạng thái sẵn sàng

Một Pod hoặc process đã chạy chỉ chứng minh scheduler và runtime khởi động được nó. Readiness phải phản ánh khả năng phục vụ đường yêu cầu cần thiết; một probe chỉ trả 200 từ bộ nhớ có thể xanh dù kết nối database hỏng. Trong Kubernetes, Pod chưa ready không nhận lưu lượng qua Service tương ứng, nhưng client đi đường khác hoặc yêu cầu đã bay tới vẫn cần xử lý riêng. Sau chuyển lưu lượng, kiểm tra từ phía client: phiên bản trả về, lỗi, latency và kết quả nghiệp vụ; gắn quan sát với digest hoặc release ID, không chỉ đếm process còn sống. Giới hạn thời gian quan sát và tiêu chí dừng phải được chọn trước khi thay đổi.

Trong một rollout giả định, ba Pod bản mới đều `Running`, nhưng chỉ hai Pod `Ready`; EndpointSlice của Service có hai endpoint sẵn sàng. Đây là bằng chứng đường chọn backend chưa nhận Pod thứ ba, chứ chưa chứng minh hai Pod còn lại xử lý đúng thao tác ghi. Tách ba câu hỏi: tiến trình có sống, đường chọn lưu lượng có đưa request vào Pod, và một request đại diện có trả đúng trạng thái dữ liệu. Nếu liveness probe liên tục giết Pod vì dependency tạm chậm, việc tăng số replica có thể làm chu kỳ lỗi nhanh hơn; cần đọc probe failure và trạng thái dependency trước khi chọn rollback hay thay ngưỡng.

Thiết kế probe là định nghĩa một hợp đồng ở ranh giới Pod. Giả sử bản mới trả `200` ở `/ready` ngay sau khi mở HTTP listener, nhưng credential database sai. Cả ba Pod có thể Ready và nhận traffic; `POST /orders` vẫn lỗi ở bước ghi. Một phép thử chủ động qua đúng Service với request ID và hậu kiểm bản ghi bác bỏ kết luận "Ready nghĩa là release dùng được". Ngược lại, nếu sửa `/ready` thành truy vấn database chung ở mọi lần probe, một đợt database chậm có thể làm cả ba Pod bị loại khỏi endpoint dù `GET /catalog` từ cache vẫn có thể trả đúng. Đây là đánh đổi về việc Pod được phép phục vụ **loại traffic nào**, không thể giải bằng một giá trị 200 chung cho mọi đường nghiệp vụ.

Với dịch vụ có cả đường đọc suy giảm được và đường ghi bắt buộc database, hãy quyết định rõ: mất database thì loại toàn Pod khỏi traffic, hay giữ phần đọc và tách định tuyến/triển khai để không hứa sai cho đường ghi. Một readiness probe duy nhất chỉ điều khiển khả năng tham gia endpoint của Pod trong Service, không định tuyến riêng từng route HTTP. Liveness nên kiểm khả năng tiến trình tự phục hồi bằng restart; nếu liveness cũng phụ thuộc database chung, restart hàng loạt không sửa database và có thể làm sự cố rộng thêm. Startup probe có thể cho ứng dụng khởi động chậm hoàn tất trước khi liveness/readiness chạy, nhưng không thay hậu kiểm nghiệp vụ sau rollout. Trong fixture cô lập, giả lập hai trạng thái database `available/unavailable`, ghi kết quả probe, số endpoint, response của một GET và một POST cùng trạng thái bản ghi; đó mới là dữ liệu để chọn hợp đồng probe. Chưa có phép chạy cluster thật thì đây vẫn là thiết kế thí nghiệm, không phải kết quả đo.

### Bài 19.3 — Điều phối

Phê duyệt một release là quyết định cho phép thử thay đổi, không xếp thứ tự ghi lên môi trường. Giả sử `R1` và `R2` cùng được duyệt: `R2` hoàn tất trước, `R1` ghi đè sau. Hai pipeline đều xanh nhưng production chạy bản cũ. Phải kiểm phiên bản đang phục vụ, không chỉ trạng thái CI.

Có thể xếp hàng các release cùng đích, hoặc so sánh phiên bản hiện tại với phiên bản dự kiến ngay trước khi thay. Nếu khác, dừng và lập lại kế hoạch. Khóa trong CI chỉ ràng buộc tác nhân đi qua CI đó; công cụ khác hoặc thao tác tay vẫn có thể chen vào. Ghi actor, thời điểm, phiên bản hiện tại và

bản mục tiêu cho mỗi lần chuyển lưu lượng.

Sau triển khai, kiểm tra artifact thực nhận request, readiness và một đường nghiệp vụ. Nếu metric hoặc dữ liệu báo hỏng, dừng tăng traffic. Rollback ứng dụng chỉ thay mã chạy; schema và dữ liệu phát sinh cần được xử lý riêng.

Xét lịch sự kiện có ba trạng thái tách biệt: hàng chờ đã duyệt, revision mong muốn và revision đang phục vụ. Lúc 10:00 môi trường phục vụ digest `H0`; `R1` xây `H1` và được duyệt, `R2` xây `H2` từ commit mới hơn và cũng được duyệt. `R2` chuyển lưu lượng sang `H2` lúc 10:05; `R1` hoàn tất bước chậm lúc 10:07 và chuyển lại `H1`. Cả hai job có thể báo success vì mỗi job chỉ biết lệnh của mình đã chạy. Đối chiếu thời điểm duyệt, thời điểm ghi đích và digest tại đích mới tìm ra sự đảo phiên bản. Không dùng số run lớn hơn hoặc tag `latest` thay cho identity của artifact đã kiểm thử.

Một biện pháp là xếp tuần tự mọi deployment tới cùng môi trường và từ chối run cũ khi đã có revision mới hơn. Nhưng chỉ khóa trong một hệ CI chưa bao trùm thao tác tay hoặc một controller khác. Tại ranh giới ghi, kiểm tra revision hiện tại phải bằng revision mà kế hoạch dựa trên, artifact được chọn phải đúng digest đã qua gate, và thao tác cập nhật phải có cơ chế chống tranh chấp phù hợp với API đích. Nếu điều kiện không còn đúng, lập kế hoạch lại. Với rolling hoặc canary, xác nhận controller đã hội tụ và quan sát instance phục vụ thật, vì trạng thái pipeline hoàn tất có thể đi trước trạng thái ứng dụng.

Để kiểm chứng một release, lấy mẫu request mang `request_id` qua đường người dùng, ghi revision/digest ở backend trả lời và đối chiếu với manifest triển khai. Kiểm thêm một thao tác có đọc/ghi dữ liệu, không chỉ `/health`. Nếu kết quả xấu, quyết định dừng dựa trên cửa sổ đo và ngưỡng đã định trước; lần rollback phải kiểm lại cả code đang chạy lẫn khả năng đọc dữ liệu tạo bởi bản mới. Trường hợp migration đã đổi schema hoặc tác dụng phụ đã ra hệ thống ngoài, trỏ traffic về `H0` có thể làm tăng lỗi. Khi ấy một hotfix tương thích tiến có thể an toàn hơn đảo mã ngay; cần chứng minh hậu điều kiện của phương án chọn.

Nếu log CI chỉ có hai dòng `R2 success` và `R1 success`, vẫn chưa đủ kết luận production đang chạy `H1`: một controller khác có thể ghi sau 10:07 hoặc các instance chưa đồng nhất. Cần mốc thời gian, target environment, digest mong muốn, trạng thái rollout và phép quan sát qua traffic thật. Đây là khác biệt giữa phê duyệt, điều phối và xác minh hậu quả của thay đổi.

**Bài tập 19.2 — Hai bản phát hành ghi đè nhau**

Hai lượt release `R1` và `R2` cùng được phê duyệt vào production; `R2` hoàn tất trước nhưng `R1` ghi đè sau. Cổng phê duyệt thiếu điều kiện nào? Nêu cách điều phối theo môi trường và phép kiểm tra phiên bản ngay trước khi thay, đồng thời giải thích vì sao khóa ở CI chưa đủ nếu có người triển khai bằng công cụ khác.

**Bài tập 19.3 — Phê duyệt không thay thế điều phối**

Vì sao hai release được phê duyệt vẫn có thể ghi đè nhau?

# Chương 20 — Phục hồi phiên bản và tương thích dữ liệu

Chứng minh hậu điều kiện sau khi lùi ứng dụng.

## Bài 20.1 — Dữ liệu qua bản phát hành

Việc chuyển về artifact cũ chỉ thay phiên bản ứng dụng đang phục vụ. Nó không tự đảo một migration database đã commit, hàng đợi đã phát, email đã gửi hay giá trị nghiệp vụ đã ghi sai. Một migration kiểu mở rộng, chẳng hạn thêm cột nullable trong khi phiên bản cũ vẫn đọc được bảng, thường cho đường quay về ứng dụng dễ hơn việc đổi tên hoặc xóa cột đang được bản cũ dùng. “Rollback database” là một hành động khác: lệnh `ROLLBACK` chỉ hủy thay đổi của giao dịch còn mở; sau `COMMIT` cần migration bù, khôi phục dữ liệu có kiểm soát hoặc một quy trình sửa riêng. Không chạy phép đảo cơ học trên dữ liệu sống khi chưa biết các ghi hợp lệ đã xen vào.

Tình huống sau là dữ liệu giả định để suy luận, không phải log production hay benchmark. Dịch vụ `orders-api` đang chạy 1.8.3, build từ commit `c83`, artifact `A83` có digest `H83`. Lượt mới lấy commit `c84`, tạo `A84` với digest `H84` khác `H83`. Pipeline khóa dependency và toolchain, unit test pass; test tích hợp chỉ thử tạo đơn có tổng bằng 0 nên không bắt lỗi đổi đơn vị tiền. Artifact `A84` được đưa qua staging, nhưng production dùng dữ liệu và lưu lượng khác. Bản ghi release lưu `c84 → H84 → test-run T84 → production`, cùng người duyệt và thời điểm; dấu mũi tên chỉ quan hệ, không hàm ý các chuỗi ký hiệu này là digest SHA-256 thật.

Trước khi chuyển lưu lượng, migration thêm `receipt_token` nullable vào bảng `orders` rồi commit. 1.8.3 bỏ qua cột mới nên vẫn chạy với schema đã mở rộng. 1.8.4 có lỗi: đầu vào `amount_cents=1234` bị nhân thêm 100 trước khi ghi, thành `123400`. Bản canary nhận 5% lưu lượng. Trong một cửa sổ giả định, kiểm tra một đơn canary có giá trị ghi `123400` trong khi yêu cầu và sổ đối soát độc lập đều là `1234`; ba đơn khác cũng có chênh lệch. Không suy rằng mọi request trong 5% đều sai, cũng không suy tổng thiệt hại từ bốn mẫu; chúng đủ để dừng tăng canary và điều tra.

## Bài 20.2 — Triển khai dở dang

Readiness của Pod vẫn xanh vì endpoint probe không đọc đường ghi đơn.

Trong tình huống này, trạng thái triển khai và khả năng phục vụ của một đường nghiệp vụ là hai phép quan sát khác nhau. Trước tiên phải gắn mỗi request với artifact đang nhận nó; tiếp đó so kết quả của đường ghi với hậu điều kiện dữ liệu mà thay đổi hứa bảo toàn. Nếu chỉ thấy probe trả thành công, ta mới biết đúng đường probe còn phản hồi. Không thể từ đó suy rằng thao tác ghi đơn đã hoàn tất hoặc dữ liệu vẫn ở trạng thái trước triển khai.

Khi phát hiện sai khác, cần dừng mở rộng lưu lượng, ghi lại phiên bản, khoảng thời gian và tập request chịu ảnh hưởng. Quyết định quay về artifact cũ phải đi kèm đánh giá schema và dữ liệu đã ghi trong thời gian phiên bản mới chạy. Một lần đổi image có thể phục hồi mã xử lý nhưng không tự đảo các bản ghi đã thay đổi. Sau thao tác giảm thiểu, kiểm tra lại đường ghi và dữ liệu đại diện; chỉ báo đã phục hồi trong phạm vi đã được đo.

Lần theo một rollout giả định có hai instance: A còn chạy bản `v1`, B đã chạy `v2`. Trước rollout, cả hai đọc và ghi trường `price_cents`. Bản `v2` bắt đầu ghi duy nhất trường mới `amount_minor`; migration đồng thời xóa `price_cents`. Readiness của B vẫn xanh vì probe chỉ kiểm `/health`, và A vẫn nhận một phần lưu lượng. Khi A đọc bản ghi mới, nó có thể lỗi dù B vừa ghi thành công. Nếu quay toàn bộ về `v1`, lỗi còn rộng hơn: binary cũ không thể đọc những bản ghi mà `v2` đã tạo. Việc giảm lưu lượng về A không phải rollback dữ liệu.

Một kế hoạch an toàn phải định nghĩa trước cửa sổ tương thích. Chẳng hạn, giai đoạn mở rộng schema giữ cả hai trường, bản mới biết đọc dữ liệu cũ và ghi theo hợp đồng đã kiểm; backfill có kiểm đếm và phép so giá trị trước khi bỏ trường cũ ở lần phát hành riêng. Chi tiết dual write phải có quy tắc xử lý lỗi từng ghi, không giả định hai thao tác riêng tự thành giao dịch. Trong sự cố trên, dừng rollout, đo số bản ghi chỉ có `amount_minor`, kiểm cách phục hồi hoặc chuyển đổi chúng trong bản sao tách biệt, rồi mới quyết định binary nào được phục vụ. Hậu điều kiện gồm đọc/ghi đơn, tính đúng dữ liệu và tập request không còn lỗi theo phiên bản, không chỉ hai Pod Ready.

**Bài tập 20.1 — Pod Ready nhưng đơn hàng sai**

Pod Ready nhưng đơn ghi sai: bằng chứng đầu tiên cần thêm là gì?

### Bài 20.3 — Phục hồi

Trước hết đóng đường ghi của `H84` hoặc chuyển toàn bộ lưu lượng ghi về `H83`, tạm dừng lượt triển khai đang chạy và xác nhận không có worker/queue consumer `H84` tiếp tục ghi. Xác minh `H83` đáp ứng request mới với số tiền đúng và 1.8.3 tương thích schema có `receipt_token`. Giữ cột mới vì việc rút schema không cần cho phục hồi phục vụ; nó đã commit và có thể chứa dữ liệu hợp lệ. Sau đó khoanh tập bản ghi do `H84` tạo trong khoảng canary bằng release ID, thời gian, request ID và đối soát độc lập; sửa qua quy trình có phê duyệt, transaction và audit trail. Không chia mọi `amount_cents` cho 100: có thể có đơn hợp lệ do `H83` ghi xen vào. Hậu điều kiện gồm digest phục vụ hiện tại là `H83`, không còn đường ghi `H84`, đơn mới khớp nguồn đối soát và toàn bộ đơn bị ảnh hưởng đã được tìm và xử lý hoặc ghi rõ còn chờ. Một thông báo “rollback succeeded” chỉ chứng minh controller đã đổi cấu hình, không chứng minh bốn điều ấy.

Nếu migration đã xóa cột 1.8.3 cần, đưa `H83` trở lại sẽ không phục vụ được; phải khôi phục tính tương thích schema hoặc phát hành bản sửa tiến lên trước khi chuyển. Nếu yêu cầu từ `H84` đã gọi hệ thống thanh toán bên ngoài, việc đổi image không hủy giao dịch ở đó; cần truy trạng thái bằng khóa idempotency và quy trình bù theo hợp đồng của hệ thống đích. Vì vậy kế hoạch release phải nêu cả “quay lại code” lẫn “xử lý hậu quả đã commit”.

Để không biến bốn đơn đã thấy thành lời khẳng định “chỉ có bốn đơn sai”, hãy lập một tập kiểm kê theo **toàn bộ** cửa sổ `H84` có thể nhận ghi, kể cả worker chạy sau khi ngừng canary. Ví dụ đối soát giả định ban đầu tìm bảy đơn: bốn đơn có request ID nối được với `H84` và giá trị sai; hai đơn nối được với `H83` và đúng; một đơn thiếu release ID nên chưa phân loại được. Bốn đơn đầu đủ để dừng rollout và sửa từng trường hợp sau khi so với nguồn tiền độc lập, nhưng dù sửa xong chúng, trạng thái của đơn thứ bảy vẫn là `UNKNOWN`. Tổng `4+2+1=7` chỉ kiểm sự phân loại của **tập đã tìm**, chưa chứng minh không còn đơn khác ngoài tập ấy. Phải so tập ID từ log nhận request, bản ghi database, hàng đợi và sổ thanh toán theo cùng khoảng thời gian; một nguồn log có lỗ hổng cần được ghi là lỗ hổng

chứng cứ, không được biến thành số không.

Với từng bản ghi cần sửa, giữ giá trị gốc, giá trị đối soát, lý do và người duyệt; trong giao dịch sửa có điều kiện trên ID cùng phiên bản hoặc giá trị đã đọc để không ghi đè một chỉnh sửa hợp lệ đến sau. Đọc lại bản ghi, sổ đối soát và trạng thái thanh toán sau commit. Nếu điều kiện cập nhật không khớp, dừng bản ghi ấy để điều tra thay vì ép phép chia cho 100. Chỉ công bố phục hồi dữ liệu khi tập chịu ảnh hưởng đã được khoanh đầy đủ theo các nguồn quan sát có sẵn, mọi mục được phân loại và hậu kiểm đạt; nếu còn `UNKNOWN` hoặc log không đầy đủ, báo phạm vi đã sửa và phần chưa chứng minh. Đây là quy trình suy luận trên fixture, không phải câu lệnh đã chạy trên database thật.

**Bài tập 20.2 — Rollback ứng dụng và dữ liệu**

Sau khi chuyển lưu lượng về `H83`, người trực định chạy `ROLLBACK` SQL và chia tất cả số tiền cho 100. Phân tích vì sao hai thao tác đó không chứng minh phục hồi. Đề xuất hậu điều kiện đo được cho phần ứng dụng, database và tác dụng phụ bên ngoài.

# PHẦN VII — Container và chuẩn OCI

## Image trở thành tiến trình cô lập ra sao, và dữ liệu sống ở đâu?

Namespace đổi góc nhìn, cgroup điều tiết tài nguyên; image là artifact, container là lần chạy, mount tách vòng đời dữ liệu.

VII

**LỘ TRÌNH PHẦN**

- Chương 21 — Namespace, cgroup v2 và ranh giới thực thi
- Chương 22 — Ảnh OCI, quy trình dựng và kho lưu trữ
- Chương 23 — Vòng đời container và dữ liệu
- Chương 24 — Compose và ứng dụng nhiều dịch vụ

# Chương 21 — Namespace, cgroup v2 và ranh giới thực thi

Chỉ ra cơ chế nào đổi góc nhìn, cơ chế nào giới hạn CPU/RAM.

## Bài 21.1 — Tạo/tham gia

Namespace mới phải được tạo hoặc tiến trình phải tham gia namespace đã có. `clone`/`clone3` có thể tạo tiến trình con ngay trong namespace mới theo cờ `CLONE_NEW*`; `unshare` tách một phần ngữ cảnh của tiến trình gọi. Trường hợp PID có điểm khác dễ bỏ sót: `unshare(CLONE_NEWPID)` chỉ đặt namespace PID cho những đứa con tạo sau đó, không chuyển PID của tiến trình gọi; đứa con đầu tiên thành PID 1. `setns` dùng file descriptor của namespace đã tồn tại để tham gia; với network namespace nó đổi góc nhìn mạng của tiến trình gọi, còn với PID namespace nó chỉ đổi nơi những đứa con sau này được tạo. `fork` thông thường kế thừa namespace, `execve` thay chương trình đang chạy mà không tự tạo namespace mới. Vì thế chuỗi tạo hoặc tham gia namespace, tạo con, rồi `execve` chương trình ứng dụng có thể dẫn tới PID 1 bên trong; chỉ gọi `execve` trên tiến trình hiện tại không tạo ra kết quả ấy.

Tạo hoặc tham gia PID/network namespace cần capability phù hợp, thường là `CAP_SYS_ADMIN` trong user namespace liên quan; cấu hình user namespace và chính sách host cũng ảnh hưởng việc cho phép. Lỗi `EPERM` là kết quả cần xử lý, không phải lý do tự nâng quyền trên máy đang phục vụ. Một phép quan sát chỉ đọc trên Linux là chạy `readlink /proc/$$/ns/pid` và `readlink /proc/$$/ns/net` trong shell, rồi chạy hai lệnh ấy trong một shell con `sh -c`; mặc định danh tính namespace của cha và con trùng nhau. Phép thử chỉ chứng minh lượt `fork`/`exec` ấy kế thừa góc nhìn, không chứng minh có thể tạo hay tham gia namespace, và `/proc` phải phản ánh đúng tiến trình đang xét.

Góc nhìn phải gắn với tiến trình đang xét. Từ host Linux có quyền phù hợp, `/proc/3142/status` có thể ghi `NSpid: 3142 1`, cho biết hai PID theo thứ tự các cấp namespace; `/proc/3142/ns/pid` và `/proc/3142/ns/net` cho biết danh tính namespace. Một số container có PID namespace riêng nhưng chia sẻ network namespace với host; cũng có thể chia sẻ PID namespace. Cấu hình runtime quyết định từng loại: loại namespace không khai báo có thể được kế thừa. Không thể suy từ nhãn "container" rằng mọi namespace đều tách. PID 1 trong namespace còn là điểm cần xem khi xử lý tín hiệu và tiến trình con; một ứng dụng bỏ qua việc thu hồi con có thể tích lũy zombie dù host vẫn hoạt động bình thường.

## Bài 21.2 — Góc nhìn tài nguyên

Chương 2 đặt tiến trình làm đơn vị chạy; Chương 5 đo tài nguyên mà các tiến trình tranh dùng. Container ghép hai cơ chế của nhân Linux quanh một nhóm tiến trình: namespace quyết định chúng nhìn thấy phần nào của hệ thống, còn cgroup tổ chức và kiểm soát mức dùng tài nguyên. Image ở Bài 22.2 là đầu vào bất biến để dựng môi trường chạy, không phải một tiến trình. Hiểu ba thực thể ấy giúp ta điều tra một dịch vụ chậm hoặc chết mà không gán mọi lỗi cho "container".

Namespace gắn với tiến trình và bọc một loại tài nguyên bằng góc nhìn riêng. Trong PID namespace, cùng một tiến trình có thể là PID 1 khi nhìn từ bên trong và PID 3142 từ namespace tổ tiên; danh sách tiến trình nhìn thấy vì thế khác nhau. Trong network namespace, các interface, địa

chỉ, route và socket thuộc một ngăn mạng riêng. `127.0.0.1` của tiến trình bên trong trỏ về loopback của ngăn ấy, không tự trỏ về host hay một container khác. Mount namespace thay bảng mount nhìn thấy; user namespace ánh xạ UID/GID; còn UTS và IPC tách các không gian tên tương ứng. Đây là phạm vi nhìn và định danh, không phải hạn mức CPU hoặc RAM.

Namespace không tạo tài nguyên từ hư không và không tự làm một sandbox hoàn chỉnh. Tiến trình vẫn dùng cùng nhân Linux với host; quyền, capability, seccomp, chính sách bảo mật nhân, mount và cấu hình runtime quyết định thêm bề mặt truy cập. Máy ảo thông thường chạy một nhân khách trên phần cứng ảo do hypervisor cung cấp. Ranh giới và chi phí vận hành khác nhau; trên máy để bàn, một hệ container Linux thậm chí có thể chạy bên trong máy ảo Linux. Vì vậy phải nói rõ đang xét ranh giới nào trước khi so sánh tính cô lập.

Hãy phản biện một câu điều tra giả định: “PID bên trong là 1, nên container đã bị tách khỏi host và không thể chạm dịch vụ host”. `NSpid: 3142 1` chỉ hỗ trợ kết luận rằng tiến trình có PID khác nhau ở hai góc nhìn; nó không nói network namespace, mount hay quyền đã được cấu hình thế nào. Nếu liên kết `/proc/3142/ns/net` trùng với tiến trình host được chọn để so, hai tiến trình có thể chia sẻ góc nhìn mạng dù PID tách. Khi ấy `127.0.0.1` ở hai phía có thể chỉ cùng loopback network namespace; nếu liên kết net khác nhau, `127.0.0.1` lại là hai loopback riêng. Phép so phải dùng đúng PID của workload còn sống và cùng thời điểm, vì container bị tạo lại có thể mang PID host khác. Quan sát cùng network namespace vẫn chưa chứng minh firewall, socket hoặc quyền cho phép một request cụ thể.

Ngay cả khi PID và network namespace đều khác host, một bind mount do runtime cấp có thể đặt một đường host vào góc nhìn filesystem của tiến trình; namespace không tự biến đường đó thành dữ liệu vô hại. Ngược lại, có cùng kernel không đồng nghĩa tiến trình được phép gọi mọi syscall: credential, capability, seccomp và chính sách truy cập còn được kiểm khi nó thực hiện thao tác. Khi đánh giá rủi ro, lập hồ sơ theo từng cạnh mà ứng dụng cần: namespace nào tách/chia sẻ, mount nào đưa dữ liệu vào, UID và quyền tại đích, syscall/capability nào cần, rồi thử một thao tác đọc hoặc kết nối không phá hoại từ đúng vị trí tiến trình. `docker inspect` hoặc cấu hình Pod chỉ là ý định; so với `/proc/<pid>/ns/*`, bảng mount và kết quả thao tác thực trên lần chạy ấy trước khi kết luận cô lập đã có hiệu lực. Đây là quy trình quan sát, không phải kết quả đã chạy trên Docker hay cluster của người học.

**Bài tập 21.1 — Quan sát PID và namespace mạng**

Từ `NSpid: 3142 1`, namespace mạng khác host và listener `127.0.0.1:8080`, hãy nêu ba điều có thể kết luận và hai điều chưa thể kết luận về khả năng client kết nối. Phép quan sát nào phân biệt không có đường chuyển tiếp với ứng dụng đã nhận kết nối nhưng trả HTTP 500?

**Bài 21.3 — Giới hạn tài nguyên**

Trong môi trường container, cgroup là một cơ chế Linux dùng để tổ chức và kiểm soát tài nguyên của nhóm tác vụ. Một giới hạn CPU có thể khiến nhóm bị trì hoãn khi dùng hết quota trong chu kỳ; một giới hạn bộ nhớ có thể làm tăng thu hồi và cuối cùng dẫn tới lỗi cấp phát hoặc tiến trình bị kết thúc theo chính sách. Hai tình huống này có thể cùng biểu hiện thành tăng độ trễ, nhưng dấu vết và cách xử lý khác nhau. Không nên gọi mọi trường hợp “container chậm” là thiếu CPU.

Cgroup v2 đặt tiến trình trong một cây phân cấp. Controller ở cha phân phối hoặc giới hạn tài nguyên cho các nhóm con; giới hạn ở nhóm con không vượt được ràng buộc của tổ tiên. Đây là cơ chế kiểm soát mức dùng, không thay đổi PID, route hay danh sách file mà tiến trình nhìn thấy. `cpu.weight` là trọng số cạnh tranh khi có nhóm cùng cần CPU, còn `cpu.max` là quota trên mỗi chu kỳ. Chẳng hạn `cpu.max` bằng `50000 100000` cho phép tối đa 50.000 microsecond CPU trong chu kỳ 100.000 microsecond: xấp xỉ nửa lõi trung bình khi luôn có việc, không hứa mỗi request sẽ hoàn tất trong 50 ms. Khi hết quota trước khi hết chu kỳ, tiến trình có thể phải chờ dù host còn CPU nhàn rỗi ngoài cgroup.

`cpu.stat` có `usage_usec`; khi CPU controller được bật, nó còn có `nr_periods`, `nr_throttled` và `throttled_usec`. Muốn quy chậm cho quota, đọc hai mẫu trên cùng cgroup và cùng khoảng thời gian, tính hiệu số, đồng thời so với thời điểm request chậm. Một mẫu sau sự cố không cho biết counter đã tăng khi nào. `memory.current` là mức dùng hiện tại, còn `memory.max` là giới hạn cứng. Khi không thể thu hồi đủ bộ nhớ dưới giới hạn, nhân có thể kích hoạt OOM trong cgroup. `memory.events.local` cho biết sự kiện tại cgroup ấy; `oom_kill` tăng nghĩa là đã có tiến trình thuộc nhóm bị OOM killer giết. `max` tăng chỉ nói đã chạm giới hạn, không đồng nghĩa đã có tiến trình chết. Bộ nhớ hiện tại có thể giảm mạnh sau khi nạn nhân bị giết, nên một ảnh chụp thấp không bác bỏ OOM vừa xảy ra. `memory.high` là ngưỡng thúc đẩy thu hồi và gây chậm; nó khác `memory.max` và không phải điểm kích hoạt OOM trực tiếp.

Tình huống giả định để tính toán, không phải output đo trên máy người học. Dịch vụ `orders` chạy trong một PID và network namespace riêng, dùng cgroup v2 `/service/orders`. Client nhận timeout xen kẽ; sau đó runtime báo một lần khởi động thất bại. Người vận hành có quyền đọc trạng thái từ host Linux. Trước hết họ gắn đúng PID host của lượt chạy và thời gian quan sát, tránh trộn counter của container cũ với lượt chạy mới. Với PID host 3142, `NSpid: 3142 1` và namespace `net:[4026533001]` khác host `net:[4026531992]`: đó là chứng cứ về góc nhìn PID và mạng khác nhau, chưa phải chứng cứ lỗi mạng. Từ góc nhìn network namespace của tiến trình, ứng dụng nghe `127.0.0.1:8080`; client ở namespace khác không thể dùng trực tiếp loopback ấy. Muốn chốt nhánh này phải kiểm tra đường publish/forward, địa chỉ bind, route và phép kết nối từ chính vị trí client. Nếu đường publish được cấu hình tới địa chỉ khác mà không có listener tương ứng, timeout có thể đến từ mạng; nếu TCP đã kết nối rồi nhận HTTP 500, lỗi nằm sau bước kết nối.

Trong một cửa sổ 10 giây của cùng lượt chạy, cấu hình là `cpu.max = 50000 100000`, `memory.max = 268435456` byte. Hai mẫu đọc từ cùng cgroup cho bảng sau; số là dữ liệu giả định, công thức hiệu số là phần cần kiểm chứng.

| Counter | Đầu cửa sổ | Cuối cửa sổ |
|---|---|---|
| `cpu.stat usage_usec` | 20.000.000 | 24.700.000 |
| `cpu.stat nr_periods` | 1.000 | 1.100 |
| `cpu.stat nr_throttled` | 40 | 120 |
| `cpu.stat throttled_usec` | 4.000.000 | 6.000.000 |
| `memory.events.local max` | 3 | 5 |

| Counter | Đầu cửa sổ | Cuối cửa sổ |
|---|---|---|
| `memory.events.local oom` | 0 | 1 |
| `memory.events.local oom_kill` | 0 | 1 |

Hiệu `usage_usec` là 4,7 triệu microsecond, tức trung bình 0,47 lõi trong 10 giây; quota cho tối đa xấp xỉ 0,5 lõi trung bình. Có 100 chu kỳ mới, 80 chu kỳ bị throttle, và `throttled_usec` tăng 2 triệu microsecond. Đây là bằng chứng quota đã tác động trong cửa sổ, phù hợp giả thuyết latency tăng; nó không cho biết request cụ thể nào chờ bao lâu, cũng không loại trừ I/O, khóa hoặc upstream chậm. Cần đặt timestamp request cạnh counter và, khi cần, đo thêm CPU pressure, hàng đợi và thời gian từng chặng. Không lấy tỷ số 80/100 làm "80% CPU đã dùng": nó đếm chu kỳ có throttle, không đếm CPU time.

`memory.events.local max` tăng 2, `oom` tăng 1 và `oom_kill` tăng 1. Ít nhất một tiến trình trong cgroup này đã bị OOM killer giết trong cửa sổ; không thể từ counter ấy tự biết PID nào, request nào hoặc lỗi cấu hình nào làm bộ nhớ tăng. Phải đối chiếu thời điểm, danh tính tiến trình, log kernel/runtime và mức dùng trước khi bị giết. Nếu runtime báo exit 137, mã ấy riêng lẻ chỉ tương thích với việc tiến trình bị `SIGKILL` theo quy ước shell; OOM counter và log cùng thời điểm mới làm giả thuyết OOM mạnh hơn. Một lần khởi động sau đó thoát 2 với log `config missing`, trong khi `oom_kill` của cgroup mới không đổi: đó là bằng chứng của nhánh lỗi ứng dụng, không được gọi nhầm là OOM chỉ vì lượt trước từng OOM.

Để tái lập phép phân tích mà không gây OOM trên môi trường đang phục vụ, người học lấy hai bản chụp chỉ đọc của `cpu.max`, `cpu.stat`, `memory.max`, `memory.events.local` trên cùng đường cgroup và lưu thời điểm cùng PID/lượt chạy; rồi tính hiệu từng hàng như bảng. Trên Linux có quyền phù hợp, đọc `NSpid` và liên kết `/proc/<pid>/ns/{pid,net}`; quan sát route, listener từ đúng network namespace và thử kết nối từ vị trí client. Nếu đang ở cgroup namespace khác hoặc controller không được bật, đường dẫn/counter nhìn thấy có thể khác; ghi rõ ngữ cảnh hoặc dừng kết luận. Bảng này tái lập được phép tính, không tái lập một sự cố thật và không chứng minh cấu hình nào là an toàn cho production.

**Bài tập 21.2 — Quota CPU và số chu kỳ bị giới hạn**

Tính hiệu `usage_usec`, `nr_periods`, `nr_throttled`, `throttled_usec` trong bảng. Vì sao quota 0,5 lõi và 80 chu kỳ throttled ủng hộ nhưng chưa chứng minh CPU là nguyên nhân duy nhất của timeout?

**Bài tập 21.3 — Sự kiện bộ nhớ và nguyên nhân OOM**

Phân biệt ý nghĩa `memory.events.local max`, `oom` và `oom_kill` tăng; đề xuất chứng cứ để gắn nạn nhân OOM với lượt chạy. Vì sao lần thoát 2 có log `config missing` không thể gộp với sự kiện OOM trước?

# Chương 22 — Ảnh OCI, quy trình dựng và kho lưu trữ

Lần từ Dockerfile tới manifest/digest và byte đã pull.

## Bài 22.1 — Dựng ảnh container

Build cache tái dùng kết quả bước trước khi các đầu vào mà builder theo dõi được coi là không đổi. Khi một bước thay đổi, các bước phụ thuộc sau nó có thể phải chạy lại. Cache giúp tiết kiệm thời gian, nhưng cache hit không chứng minh chương trình đúng, dependency ngoài đã được cập nhật hay hai lượt build tạo byte giống nhau. Chương 17 đã yêu cầu ghi commit, toolchain, đầu vào và digest thực tế; ở đây cần thêm base image theo digest và các đầu vào build ngoài repository nếu muốn kiểm soát biến thiên. Build tạo image ở nơi build; push phân phối nội dung lên registry; pull lấy manifest và blob về nơi chạy. Ba hành động ấy khác nhau, và "đã pull" không có nghĩa container đang chạy đã được thay.

Ví dụ giả định một Dockerfile sao chép tệp khai báo dependency và cài gói trước khi sao chép mã ứng dụng. Sửa mã mà không sửa tệp dependency có thể tái dùng layer cài gói; sửa tệp dependency làm cache của bước ấy và các bước phụ thuộc mất hiệu lực. Nhưng package index hoặc bản base image được gọi bằng tag có thể thay đổi bên ngoài đầu vào mà ta tưởng đã khóa. Để so hai build, lưu digest base, lockfile, tham số build và digest image đầu ra; nếu các digest đầu ra khác, tìm nguồn sai khác trước khi gán nhãn "reproducible". Kết quả cache hit chỉ giải thích công việc builder đã bỏ qua, không thay cho phép so artifact.

Lần theo một đồ thị build giả định: `FROM` chọn base B; bước A sao chép lockfile; bước B chạy `RUN apt-get update && apt-get install ...`; bước C sao chép source; bước D biên dịch. Nếu chỉ source đổi, C và D phải tính lại, còn A và B có thể lấy cache. Ngày hôm sau kho gói công bố bản sửa bảo mật nhưng Dockerfile, lockfile và base không đổi: builder có thể lấy lại B từ cache, không gọi kho gói, nên image mới vẫn chứa gói cũ. `--no-cache` buộc thực thi lại bước B trong một lượt thử, nhưng nếu dependency chưa khóa thì byte đầu ra có thể đổi theo kho bên ngoài; đó không phải phép chứng minh build tái lập. Cách xử lý phụ thuộc mục tiêu: để tái lập byte thì khóa nguồn và phiên bản gói, để cập nhật bảo mật thì đổi đầu vào đã khóa theo quy trình, dựng và thử artifact mới. Hai mục tiêu không đạt được bằng một lệnh "build lại" mơ hồ.

Một bí mật dùng trong `RUN --mount=type=secret` cũng cho thấy ranh giới của cache: đổi giá trị secret không tự làm cache của bước ấy mất hiệu lực theo quy tắc Docker Build; ID và đường mount của secret có tham gia. Nếu output thực sự phụ thuộc giá trị secret, thiết kế build cần một cách làm mới cache có kiểm soát mà không ghi chính secret vào Dockerfile, build arg, layer hay log. Đừng đưa secret vào cache key công khai để chữa cache cũ. Khi điều tra một image nghi còn gói lỗi, ghi base digest đã phân giải, input dependency, builder và chính sách cache, log bước nào được thực thi, phiên bản gói trong artifact đã tạo và digest image cuối. So những byte này với artifact đã qua kiểm thử và đang chạy; tên job xanh hoặc tag không trả lời được liệu patch đã có mặt. Ví dụ là phép suy luận theo hợp đồng cache, chưa được chạy với Docker Engine trong môi trường này.

### Bài 22.2 — Định dạng ảnh OCI

Chương 21 cho thấy container đang chạy là một nhóm tiến trình với góc nhìn và giới hạn tài nguyên được cấu hình. Chương này đi ngược dòng tới thứ được phân phối để tạo tiến trình ấy, rồi theo dữ liệu đi qua nhiều lần chạy. Một câu hỏi vận hành thường bị trả lời sai là "đã đổi image, sao dữ liệu cũ vẫn ở đó?" Câu trả lời cần tách ba định danh: byte của image, lần chạy của container và nơi thực sự lưu dữ liệu.

Image OCI gồm những thay đổi hệ thống tệp có thứ tự và cấu hình chạy như entrypoint, đối số, biến môi trường. Digest giúp nhận diện nội dung image đã lấy về, nhưng byte của image không tự tạo một namespace hay một cgroup. Runtime nhận bundle và cấu hình, chuẩn bị root filesystem, namespace, quyền và giới hạn, rồi khởi chạy chương trình. Image vẫn còn để dùng cho lượt chạy khác. Lớp ghi, volume và dữ liệu bên ngoài có vòng đời riêng, nên xóa hoặc chạy lại container không có nghĩa phục hồi dữ liệu như cũ.

Builder đọc Dockerfile cùng build context, thực hiện các bước để tạo filesystem và cấu hình image. Theo mô hình OCI, manifest chỉ tới cấu hình và danh sách layer có thứ tự; descriptor có digest để người nhận kiểm tra đúng byte đã tham chiếu. Một layer ghi thay đổi so với lớp trước, không phải một máy ảo thu nhỏ. Lệnh `COPY` đưa dữ liệu của context vào kết quả build; file bị bỏ qua bởi `.dockerignore` không có trong context. Nếu đưa secret vào một bước build rồi xóa ở bước sau, không được mặc định rằng secret đã biến mất khỏi mọi layer hoặc cache. Cần cơ chế secret của builder và xác minh artifact cuối theo quy trình tương ứng.

Khi registry trả về một index đa nền tảng, lựa chọn kiến trúc và manifest con còn phải được ghi nhận: cùng tag có thể dẫn tới nội dung khác trên hai nền tảng.

Giả sử tag `orders:1.8` trỏ tới image index có digest `I`; index ấy tham chiếu manifest `M_amd` cho `linux/amd64` và `M_arm` cho `linux/arm64`. Đây là ký hiệu giả định, không phải digest từ registry đã chạy. Pin `I` cố định danh sách descriptor được index tham chiếu, nhưng node ở hai kiến trúc vẫn chọn hai manifest và các layer có thể khác byte. Một test integration chỉ chạy trên `amd64` xác minh hành vi của nhánh `M_amd` trong môi trường thử ấy; nó không tự xác minh `M_arm`. Hồ sơ phát hành cần nêu platform được hỗ trợ, digest index, digest manifest con đã thử, và ở runtime mỗi node đã lấy manifest nào. Nếu chỉ còn tag hoặc digest index mà thiếu bằng chứng test cho `arm64`, kết luận "mọi kiến trúc đã qua gate" là UNKNOWN. Ngược lại, không được gọi `M_amd` và `M_arm` khác nhau là build không tái lập: chúng là hai đầu ra dành cho hai platform khác nhau; phép tái lập phải so cùng platform và cùng tập input.

### Bài 22.3 — Phân phối ảnh

Registry lưu manifest và blob. Tag như `orders:2.1` là tên thuận tiện trỏ tới manifest và có thể được cập nhật; digest định danh nội dung manifest đã chọn. Dùng digest của artifact đã kiểm thử để triển khai và xác nhận digest ở đích; vẫn cần đánh giá nguồn gốc, lỗ hổng, cấu hình và quyền vì đúng byte không đồng nghĩa hành vi an toàn. Một registry không sẵn sàng có thể làm lần pull mới thất bại dù container hiện hữu còn chạy trên node đã có image; trạng thái cache cục bộ và chính sách pull quyết định chi tiết.

Trong sự cố giả định, hai node `amd64` và `arm64` đều báo tag `orders:2.1`, nhưng mỗi node chạy manifest con khác nhau từ cùng image index. Khi lỗi chỉ xuất hiện trên `arm64`, chỉ đối chiếu tag hoặc digest của index là chưa đủ; ghi cả kiến trúc, digest manifest con, digest config và bản ghi test tương ứng. Nếu tag bị trỏ lại giữa hai lần pull, hai Pod tạo sau có thể chạy nội dung khác Pod cũ dù manifest triển khai không đổi. Cách kiểm là so danh tính đã giải quyết ở runtime với digest kỳ vọng, không suy từ tên tag trong YAML.

Quá trình lấy image có thể nhìn như một đồ thị địa chỉ theo nội dung. Tên repository và tag giúp tìm một manifest hoặc image index. Index chọn manifest phù hợp platform; manifest tham chiếu config và các layer blob. Digest của index, manifest con và layer trả lời những câu hỏi khác nhau, nên hai node cùng index digest vẫn có thể tải các byte thực thi khác nhau theo kiến trúc. Nếu build chỉ tạo biến thể `amd64`, node `arm64` có thể không tìm được manifest phù hợp trước cả khi tiến trình ứng dụng được tạo. Nếu cả hai biến thể tồn tại nhưng một biến thể thiếu binary phù hợp, lỗi chuyển sang giai đoạn khởi động container. Phân biệt thời điểm thất bại làm giảm số giả thuyết cần thử.

Đặt một phép điều tra có dữ kiện giả định: bản triển khai mong index digest `I`, node A báo platform `linux/amd64`, node B báo `linux/arm64`; A chạy ổn, B báo lỗi pull. Đầu tiên xác nhận B đã resolve đúng `I`, sau đó liệt kê descriptor của index và platform từng manifest. Nếu không có descriptor `arm64`, sửa pipeline build/publish; thay tag trên B không chữa được artifact thiếu. Nếu có, kiểm quyền pull, registry, cache và digest manifest con. Trường hợp B pull và khởi động thành công nhưng trả dữ liệu sai lại đòi so config, binary và test theo platform. Một câu báo “cùng tag” không phân biệt được ba trường hợp này.

Thực hành có thể làm với một image công khai, không chứa dữ liệu riêng: ghi lại tag, digest index, platform và digest manifest con bằng công cụ hỗ trợ inspect manifest; sau đó triển khai bằng digest đã ghi trong môi trường thử. So với digest mà runtime thật báo sau pull. Ghi phiên bản công cụ và thời điểm vì tag có thể đổi trong lúc thử. Nếu không có Docker hoặc registry truy cập được, dùng manifest mẫu trong tài liệu và đánh dấu đó là bài suy luận, không nhận là phép chạy. Sau khi xác nhận byte, vẫn phải kiểm provenance của build, phụ thuộc, cấu hình runtime và đường dữ liệu: content addressing xác nhận identity, không cấp chứng nhận an toàn hay tính đúng nghiệp vụ.

**Bài tập 22.1 — Tag mới và container còn chạy ảnh cũ**

Một pipeline build và push `orders:2.2`; máy chủ vẫn chạy container tạo từ digest cũ. Phân biệt trạng thái registry, image cục bộ và tiến trình đang phục vụ. Cần đối chiếu những định danh nào để khẳng định bản mới đã nhận request?

# Chương 23 — Vòng đời container và dữ liệu

Dự đoán dữ liệu sau stop, recreate, remove, mount và đổi image.

### Bài 23.1 — Lần chạy

Trạng thái `running` của runtime cũng không phải readiness của ứng dụng. Tiến trình có thể còn sống nhưng chưa bind socket, đang đợi DB, trả lỗi nghiệp vụ hoặc nghe sai địa chỉ. Việc publish port qua host hay qua bộ điều phối thêm một đường chuyển tiếp ngoài network namespace; thấy socket lắng nghe ở `127.0.0.1` bên trong chưa chứng minh client ngoài namespace đến được. Cần nối các phép quan sát từ tiến trình, route và socket tới phép thử request thực tế, như đã học ở Chương 8, Chương 9 và Chương 11.

Runtime tạo môi trường theo cấu hình, khởi chạy tiến trình và theo dõi trạng thái như `created`, `running`, `stopped`. Dừng tiến trình rồi khởi động lại cùng container có thể giữ writable layer của lần tạo ấy; xóa hoặc recreate container tạo lớp ghi khác. Thay image không tự sửa writable layer của một container đang tồn tại; phải xác định container nào thực sự đang phục vụ. Khi container bị xóa, dữ liệu chỉ nằm trong writable layer của nó không còn được giữ như một kho bền vững. Đặt database hoặc upload quan trọng ở đó là thiết kế dễ mất dữ liệu khi thay container.

Tách vòng đời của runtime khỏi vòng đời của dịch vụ. Theo mô hình OCI, `create` dựng môi trường từ bundle và cấu hình nhưng **chưa chạy chương trình** của người dùng; `start` mới thực thi nó. `created` vì vậy không thể trả lời ứng dụng đã bind cổng hay chưa. `running` chỉ cho biết tiến trình chính đã thực thi và chưa thoát; `stopped` cho biết nó đã thoát, không cho biết tác dụng phụ trước lúc thoát đã được đảo lại. Lỗi khi tạo mount hoặc namespace xảy ra trước mã ứng dụng; một tiến trình đã chạy nhưng thoát với lỗi cấu hình là nhánh khác. Muốn chọn cách sửa, cần đặt trạng thái runtime cạnh sự kiện tạo/khởi động, exit status, log tiến trình và phép thử request từ đúng vị trí client.

Xét ba ảnh chụp **giả định** của cùng image. Ở lượt A, `create` thất bại vì thiếu mount nguồn: chưa có bằng chứng chương trình đã chạy, nên sửa mã HTTP không giải quyết lỗi này. Ở lượt B, trạng thái là `running` và socket chỉ nghe `127.0.0.1` trong network namespace, trong khi client đi qua cổng publish tới địa chỉ khác: cần kiểm cấu hình bind và đường chuyển tiếp, chưa thể đổ cho image. Ở lượt C, tiến trình chính đã thoát sau khi ghi file vào writable layer. Khởi động lại container ấy có thể còn thấy file; xóa rồi tạo container khác từ cùng image thì có lớp ghi khác. Ba lượt dùng chung byte image nhưng khác trạng thái và dữ liệu. Đây là phép suy luận theo cấu hình giả định, không phải output Docker đã chạy.

Trước khi sửa hoặc recreate, ghi lại container ID, image digest, mount source/destination và nơi ứng dụng thực sự ghi; sau đó kiểm dữ liệu bằng một thao tác đọc phù hợp. Runtime state, image identity và data identity là ba phép quan sát độc lập.

Container đang `running` là một tiến trình đã thực thi chương trình trong môi trường ấy; khi tiến trình chính dừng, runtime ghi trạng thái `stopped`. Cách nói này tập trung vào tiến trình chính; bên trong container vẫn có thể có nhiều tiến trình con.

### Bài tập 23.1 — Container Running nhưng ứng dụng chưa sẵn sàng

Một image digest đã được kiểm thử nhưng runtime báo `running`, readiness vẫn đỏ. Giải thích vì sao cả digest lẫn trạng thái runtime không đủ chứng minh request được phục vụ. Nêu vai trò khác nhau của image, namespace và cgroup trong tình huống này.

### Bài 23.2 — Hệ thống tệp khi chạy

Image được tạo từ các layer chỉ đọc; container chạy thêm một lớp ghi riêng để nhìn thấy các thay đổi trong lần chạy ấy. Cách nhìn merged filesystem che chi tiết nguồn gốc: đọc một đường dẫn có thể thấy bản từ layer trên, còn bản cùng tên ở layer dưới vẫn tồn tại trong image. Khi xóa một tệp vốn ở layer dưới, lớp trên ghi dấu xóa (whiteout) để merged view không hiện tệp ấy. Whiteout không xóa byte khỏi layer image đã phân phối. Trong định dạng layer OCI, tên whiteout có dạng `.wh.` gắn trước tên bị che; cơ chế lưu nội bộ của storage driver có thể khác và không nên đọc thẳng thư mục layer của engine như API ổn định.

Theo dõi tình huống giả định từ chương trước: image A chứa chương trình, container A ghi 41 bản ghi vào `/var/lib/orders` mà không có mount. Dừng rồi khởi động lại cùng container có thể còn thấy 41 bản ghi vì lớp ghi của chính container đó vẫn còn. Xóa container A và tạo container B từ cùng image tạo lớp ghi mới, nên B không tự thấy 41 bản ghi. Thay image A bằng image B cũng không chuyển dữ liệu từ lớp ghi cũ sang container mới. Nếu gắn named volume tại `/var/lib/orders`, dữ liệu ghi vào mount thuộc volume và có thể gắn lại vào lần chạy khác bằng đúng volume identity. Một bind mount trỏ tới đường dẫn host, chịu quyền và vòng đời của đường dẫn ấy. Cả volume lẫn bind mount có thể che tệp vốn nằm sẵn ở destination trong image.

Phải xác định đúng nơi ghi trước khi kết luận mất dữ liệu. Với container B thấy thư mục rỗng, các giả thuyết gồm dùng nhầm project tạo volume tên khác, mount sai destination, volume đúng nhưng không có 41 bản ghi, hoặc dữ liệu trước đây chỉ ở lớp ghi của A. Quan sát cần là danh tính container, image digest, loại mount, source và destination, rồi phép đọc bản ghi. `docker ps` cho thấy container đang chạy; nó không chứng minh mount có dữ liệu. Không chạy thao tác xóa volume hoặc prune để “thử” khi chưa biết nguồn dữ liệu thực và có bản sao phục hồi.

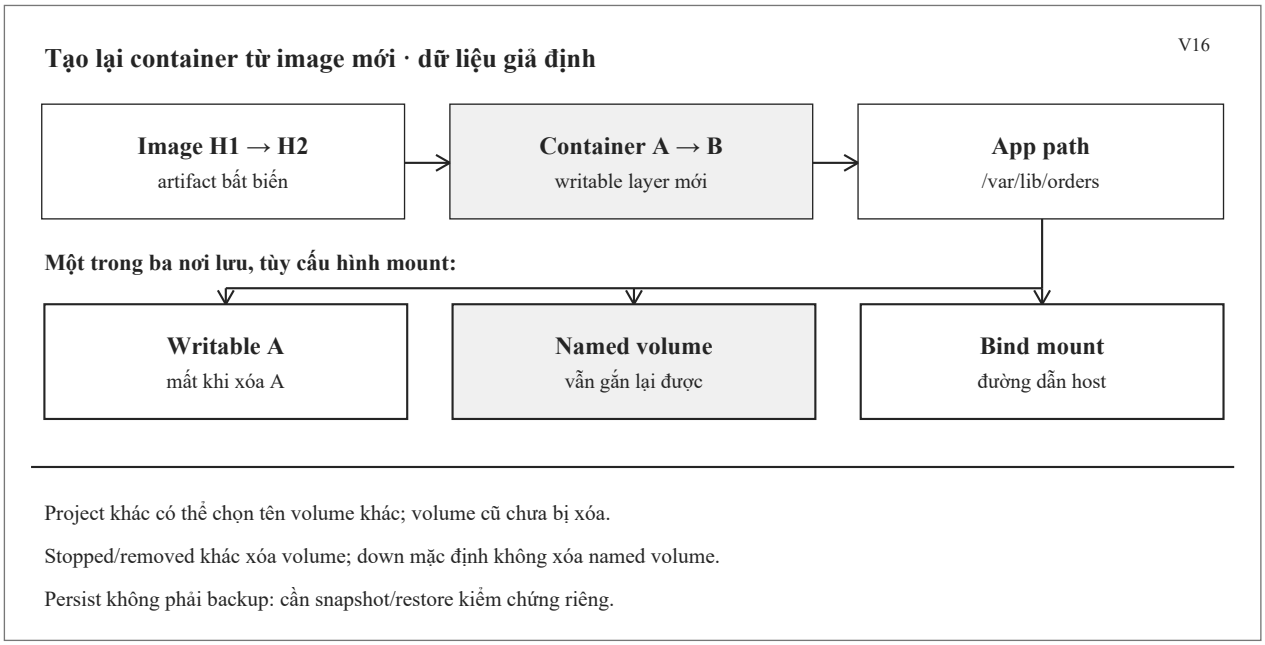
Đổi image và đổi dữ liệu là hai quy trình tách biệt. Nếu schema dữ liệu thay theo phiên bản ứng dụng, giữ nguyên volume có thể làm bản mới không đọc được; rollback image cũng không tự đảo migration. Vì vậy, ngoài digest image phải ghi phiên bản schema, đường lưu dữ liệu, backup và phép thử đọc/ghi sau recreate. Trong ví dụ 41 bản ghi chỉ là dữ kiện giả định để suy luận, không phải kết quả Docker đã chạy.

### Bài tập 23.2 — Dữ liệu trong lớp ghi của container

Container A ghi 41 bản ghi vào /var/lib/orders không mount. Tạo B từ cùng image rồi B thấy thư mục rỗng. Vì sao và cần quan sát gì trước khi xử lý?

## Bài 23.3 — Gắn nơi lưu dữ liệu

Volume là vùng dữ liệu do Docker quản lý, được mount vào đường dẫn trong container. Named volume có định danh để một container khác có thể gắn lại; nó tồn tại độc lập với container cho tới khi bị xóa riêng. Bind mount thay bằng một đường dẫn cụ thể trên host của Docker daemon. Nó hữu ích cho mã nguồn hoặc cấu hình cần đọc từ host, nhưng bị ràng buộc vào đường dẫn, quyền và hệ điều hành của host ấy; khi dùng Docker Desktop, host của daemon có thể là Linux VM phía sau giao diện máy người dùng. Một bind mount có quyền ghi cho tiến trình trong container có thể sửa file host tương ứng, nên cấu hình chỉ cần đọc nên mount read-only. Cả hai loại mount che nội dung có sẵn tại điểm mount trong container; không nên thấy một thư mục "trống" rồi vội kết luận byte cũ đã bị xóa. Với empty named volume, Docker có thể sao chép nội dung sẵn trong image tại điểm mount vào volume theo mặc định; hành vi này khác việc chuyển dữ liệu từng được ghi ở writable layer của container cũ.




Hình VII.3.1 — Vòng đời image, writable layer và nơi lưu độc lập; dữ liệu giả định.

Lần theo một trạng thái **giả định** để phân biệt ba nơi lưu. Image X có `/var/lib/orders/seed.db` chứa ký hiệu `S`. Container A tạo từ X, **không có mount**, ghi thêm `live.db=A41`. Dừng rồi khởi động lại chính A có thể còn thấy `S` và `A41`; tạo container B mới từ X chỉ thấy `S`, vì B có lớp ghi mới. Không có thao tác "đổi image" nào tự chuyển `A41` sang B. Bây giờ tạo container C từ X với **named volume V mới và rỗng** gắn tại `/var/lib/orders`. Theo hành vi sao chép mặc định của Docker, nội dung có sẵn tại điểm gắn trong view của C, ở đây là `seed.db=S`, được đưa vào V; C ghi `live.db=V41` vào V. Một container D gắn lại **đúng V** thấy `S` và `V41`, không thấy `A41`. Nếu tạo volume V2 mới, nó có thể nhận lại `S` từ image nhưng không nhận `V41` của V. Các ký hiệu này là dữ kiện suy luận, không phải kết quả đã chạy trên Docker Engine.

Đổi đúng một điều kiện để kiểm nguyên nhân: dùng `volume-nocopy` khi gắn V mới thì Docker không đưa `S` từ image vào volume; dùng bind mount từ thư mục host rỗng ở cùng destination cũng che `S` mà không có bước sao chép mặc định như named volume. Hai trường hợp đều có thể hiện thư mục trống nhưng nguyên nhân và nơi cần tìm byte cũ khác nhau. Nếu V đã có dữ liệu, gắn lại V

không sao chép lại image mỗi lần chạy; thay image X bằng Y cũng không tự ghi đè dữ liệu trong V. Vì thế cần ghi cả thời điểm tạo volume, trạng thái rỗng ban đầu và tùy chọn mount trước khi giải thích vì sao một tệp có mặt hay vắng mặt.

Đường dẫn ứng dụng không nói nơi lưu. `/var/lib/orders/orders.db` có thể thuộc writable layer, named volume, hoặc một thư mục host. Cần đọc cấu hình mount và danh tính container/volume rồi mới quyết định backup, migration hay xóa. Volume vẫn nằm trên một host hoặc backend cụ thể; "persist" không tự có nghĩa có bản sao, backup, giao dịch nhất quán hay khả năng chạy sang node khác. Database đang ghi cần phương pháp snapshot hoặc backup phù hợp với chính database, cùng phép restore thử. Xóa container, image và volume là ba hành động khác nhau; đặc biệt các tùy chọn xóa volume phải được xem như thao tác xóa dữ liệu.

Khi có một môi trường Docker thử riêng, phép kiểm không bắt đầu bằng `prune`: đối chiếu container ID/image digest, trường `Mounts` của từng container (loại, source, destination), danh tính volume và nội dung đọc qua đúng mount. So một bản ghi kiểm chứng trước và sau khi recreate; nếu giá trị khác, dừng ở giả thuyết "khác nguồn dữ liệu" cho tới khi xác định ứng dụng thật sự ghi đường nào. `docker inspect` cho biết cấu hình mount quan sát được, không chứng minh database nhất quán hay bản sao có thể restore. Nếu thiếu Engine, chỉ giải trace trên giấy và ghi runtime SKIP. Hậu điều kiện của một lần chuyển dữ liệu là bản ghi mong đợi đọc được từ container mới **và** bản backup thử restore được, không chỉ là tên volume còn tồn tại.

**Bài tập 23.3 — Dữ liệu sau khi dừng hoặc xóa container**

Giải thích vì sao 41 bản ghi ở lượt A không tự xuất hiện ở lượt B. Nếu container A chỉ dừng, câu trả lời khác gì trường hợp A đã bị xóa? Hành động nào phải tránh khi chưa có backup?

**Bài tập 23.4 — Ba nơi lưu cho cùng một đường dẫn**

Image X có seed.db=S tại /var/lib/orders. Container A không gắn mount và ghi live.db=A41. Dự đoán tệp nhìn thấy sau khi (a) khởi động lại A, (b) tạo B mới từ X, (c) tạo C từ X với named volume V mới/rỗng rồi C ghi live.db=V41, (d) tạo D từ X gắn lại V. Nếu thay V bằng bind mount từ thư mục host rỗng, seed.db sẽ ra sao? Nêu phép kiểm trước khi xóa A hoặc V.

# Chương 24 — Compose và ứng dụng nhiều dịch vụ

Phân biệt cấu hình project với trạng thái service/volume thực tế.

## Bài 24.1 — Mô hình dự án Compose

Compose file khai báo service, image hoặc cách build, port, network và volume cho một ứng dụng. `docker compose up` đối chiếu cấu hình để tạo hoặc thay container, mạng và volume cần thiết trên Docker Engine được chọn. Compose gán project để phân biệt tập tài nguyên; mặc định có mạng cho các service cùng project, chúng tìm nhau bằng tên service. Cổng host và cổng trong container là hai địa chỉ khác nhau: trong ví dụ dưới, client trên host dùng `127.0.0.1:18080`, còn tiến trình trong container phải nghe cổng 8080 trên địa chỉ có thể nhận kết nối từ mạng container. Đổi IP container sau recreate không nên buộc service khác sửa cấu hình; dùng tên service trong mạng chung. `localhost` trong service vẫn là chính network namespace của nó.

Đoạn Compose sau chỉ minh họa cú pháp. Image `orders:2.1` và thư mục `./config` là đầu vào giả định của dự án học, không có ứng dụng có thể chạy ngay trong repository này.

Trước khi xem tệp YAML đầy đủ ở cuối chương, giữ bốn dữ kiện của cùng ví dụ: project mặc định theo tệp là `orders-case`; service `orders` dùng image `orders:2.1`; host công bố `127.0.0.1:18080` tới cổng 8080 của container; dữ liệu ứng dụng ở `/var/lib/orders` được gắn named volume `orders_data`, còn `./config` được bind vào `/etc/orders` chỉ đọc. Nhờ vậy, khi hỏi "đổi image có mất dữ liệu không?", ta kiểm nơi ghi và mount trước, không suy từ tag. Trường `name:` đặt tên project khi không có lựa chọn ưu tiên cao hơn: `-p` trên lệnh chạy hoặc `COMPOSE_PROJECT_NAME` có thể ghi đè. Do đó phải xem tên project hiệu lực, không mặc định YAML tự khóa danh tính volume trong mọi lần chạy. Các giá trị trên là cấu hình giả định để suy luận, chưa phải output Docker.

Nếu container `orders` bị recreate sau khi đổi image, nội dung chỉ ghi trong writable layer của container cũ không phải một cơ sở bền vững cho lần chạy mới. Dữ liệu đặt trên named volume hoặc bind mount có vòng đời theo cấu hình và đường dẫn host riêng; phải xác nhận container thực sự mount đúng nơi ứng dụng ghi. Một project Compose mới có thể tạo volume mang tên khác dù YAML gần giống, khiến ứng dụng nhìn như "mất dữ liệu" trong khi volume cũ còn trên host. Kiểm danh tính project, tên volume thực, mount target và nội dung trước khi xóa bất cứ tài nguyên nào. Compose không tự cung cấp failover đa node hay backup dữ liệu.

## Bài 24.2 — Khởi động

`orders_data` được mount tại đường ứng dụng ghi; `./config` là bind mount từ host, chỉ đọc trong container. `name: orders-case` cố định tên project trong ví dụ; chạy với project khác có thể tạo volume khác dù phần tên `orders_data` trong file giống nhau. Trước khi triển khai, phải xem cấu hình Compose đã được nội suy và kiểm tra image, mount, project, quyền, port cùng nơi daemon đang chạy. Đoạn này chưa có healthcheck; Compose thấy container chạy không chứng minh API hoặc database ready. `depends_on` mặc định giải quyết thứ tự khởi động, không tự kiểm tra ứng dụng đã sẵn sàng; điều kiện healthcheck có thể chờ một phép kiểm tra được khai báo, nhưng phép kiểm tra ấy vẫn chỉ chứng minh điều nó đo.

Compose hữu ích để mô tả và chạy ứng dụng trên một Docker Engine, kể cả Engine ở máy khác. File Compose tự nó không là bộ lập lịch nhiều node, không tự phân phối bản sao khi host hỏng và không tự nhân bản volume cục bộ. Nếu triển khai trên Swarm hoặc một hệ điều phối khác, phải đánh giá cơ chế scheduler, mạng nhiều host và storage của hệ đó thay vì suy từ cú pháp Compose. Phần XI mới đi sâu vòng điều khiển và placement của Kubernetes.

Đọc cấu hình mẫu từ phía dữ liệu. Container ghi vào `/var/lib/orders`; đường ấy được volume `orders_data` che lên, nên xóa container rồi tạo container khác cùng project/volume không tự xóa dữ liệu ở volume. Ngược lại, nếu ứng dụng thật ghi nhầm sang `/tmp/orders`, dữ liệu đó nằm trong writable layer của container và có vòng đời khác. Bind mount `./config:/etc/orders:ro` lấy file từ phía host nơi Docker daemon truy cập; `:ro` chặn ghi qua mount từ container nhưng không ngăn người có quyền trên host sửa nguồn. Thay image không bảo đảm thay dữ liệu hay cấu hình đang mount. Hãy xác định ba nơi trước khi dự đoán tác động của `up`, `down` hoặc thay tag: path trong container, object lưu trữ thực, và chủ thể đang giữ quyền sửa.

Một tình huống dễ gây "mất dữ liệu": người học chạy Compose với project `orders-case`, tạo record `K1`, rồi chạy lại cùng file nhưng ép project name khác và không thấy `K1`. Chưa có bằng chứng record đã bị xóa; lần chạy mới có thể gắn volume khác. Ghi project name được Compose resolve, tên volume thực, container ID và mount destination ở cả hai lượt; so volume chứa marker bằng cách mount read-only trong fixture. Không dọn volume cũ trước khi xác nhận dữ liệu và chính sách retention. Nếu cả hai lượt thực sự dùng cùng volume mà dữ liệu vẫn khác, kiểm ứng dụng ghi đúng path, quyền trên mount và nguồn dữ liệu mà instance mới đọc.

Kiểm tra readiness riêng với container state. `docker compose config` cho biết cấu hình sau nội suy biến; nó không xác nhận image pull thành công, port không xung đột, ứng dụng đã bind, database nhận kết nối hay nghiệp vụ trả đúng. Trong lab, tạo một record rồi restart container, thay image bằng digest thử đã biết và đọc lại record; ghi cả lần request lỗi để phân biệt container chạy với service sẵn sàng. Nếu Engine không có, giải bài từ file Compose và đánh dấu runtime SKIP. Khi cần phục hồi qua mất host, volume cục bộ còn phải có backup và nơi restore; Compose file không biến dữ liệu cục bộ thành dữ liệu bền vững giữa nhiều máy.

**Bài tập 24.1 — Compose khởi động và trạng thái sẵn sàng**

Một Compose file có `depends_on` và hai service cùng mạng mặc định. Nêu điều nó đảm bảo, điều nó không đảm bảo về readiness, dữ liệu và phục hồi sau khi host hỏng. Đề xuất một phép kiểm chứng từ góc nhìn client và một phép kiểm chứng backup.

**Bài 24.3 — Dữ liệu**

```
name: orders-case
services:
  orders:
    image: orders:2.1
    ports:
      - "127.0.0.1:18080:8080"
    volumes:
      - orders_data:/var/lib/orders
      - ./config:/etc/orders:ro
volumes:
  orders_data:
```

Tình huống sau là dữ liệu giả định để suy luận, không phải output Docker đã chạy. Ở lượt A, image `orders:2.0` không khai báo mount cho `/var/lib/orders`; ứng dụng ghi 41 bản ghi vào `orders.db` trong writable layer của container `orders-a`. Sau khi thay container bằng lượt B dùng image `orders:2.1`, đường dẫn ấy trống. Không có bằng chứng 41 bản ghi bị xóa khỏi một volume: chúng chưa từng ở volume; lớp ghi gắn với container A không phải dữ liệu được tự chuyển sang B. Nếu A còn tồn tại dưới dạng container đã dừng, có thể còn khả năng đọc lại lớp ghi; nếu A đã bị xóa, không được hứa khôi phục mà không có backup hoặc cơ chế lưu khác.

Ở lượt C, dự án dùng cấu hình Compose ở mục trước và ghi 41 bản ghi mới vào named volume `orders-case_orders_data`. Lượt D thay image `orders:2.1` bằng digest của bản `2.2`, recreate container nhưng vẫn gắn đúng volume ấy; ứng dụng nhìn thấy 41 bản ghi. Điều đó chưa chứng minh image `2.2` đã đọc dữ liệu đúng schema hoặc volume được backup. Nếu người vận hành đổi project name thành `orders-test`, Compose có thể gắn một volume mới `orders-test_orders_data`; đường dẫn trong container vẫn `/var/lib/orders` nhưng file cũ không xuất hiện. Volume của project trước có thể vẫn còn. Việc thêm bind mount `./empty-data:/var/lib/orders` vào cùng điểm mount cũng thay nơi nhìn thấy; không được đồng thời gắn hai nguồn vào cùng đích và đoán nguồn nào thắng. Chọn một cấu hình rõ ràng, kiểm tra `Mounts` thực tế và ngừng thay đổi trước khi điều tra.

Phép kiểm chứng tối thiểu bắt đầu bằng container ID, image digest và thời điểm recreate; tiếp đến là loại mount, `Source`, `Destination`, tên project và volume. So số bản ghi và checksum hoặc ID nghiệp vụ từ ứng dụng với bản backup độc lập, rồi thử đọc volume cũ bằng phương thức không ghi nếu cần. `docker compose config` giúp xem cấu hình sẽ áp dụng nhưng không chứng minh trạng thái runtime hoặc byte dữ liệu. Không chạy `down -v`, `volume prune` hay xóa thư mục host để "làm sạch" trước khi đã xác định chính xác nguồn dữ liệu. Trong môi trường thử riêng, có thể tái hiện ba lượt với dữ liệu vô hại và so trạng thái trước/sau. Nếu không có Docker Engine để chạy phép thử, giữ ca này ở mức suy luận và ghi rõ rằng các trạng thái A–D chưa được quan sát thực tế.

**Bài tập 24.2 — Dữ liệu khi đổi image và tên project**

Lượt D đổi image nhưng vẫn thấy 41 bản ghi; lượt kế tiếp đổi project name thì không thấy. Dùng volume identity và mount destination để giải thích hai hiện tượng. Vì sao một bind mount rỗng vào cùng đích không chứng minh volume cũ đã mất dữ liệu?

# PHẦN VIII — Điện toán đám mây và AWS

## Một request qua ranh giới danh tính, mạng, tính toán và dữ liệu của cloud ra sao?

Identity policy và đường mạng là hai bộ lọc độc lập; compute và storage có vòng đời, miền lỗi, chi phí khác nhau.

VIII

LỘ TRÌNH PHẦN


- Chương 25 — Danh tính AWS và đánh giá chính sách IAM
- Chương 26 — VPC, mạng con và đường đi trong AWS
- Chương 27 — EC2, cân bằng tải và miền lỗi
- Chương 28 — Lưu trữ khối, tệp và đối tượng

# Chương 25 — Danh tính AWS và đánh giá chính sách IAM

Lần theo principal từ EC2 tới S3/STS và quyết định Allow/Deny.

## Bài 25.1 — Danh tính

Một ứng dụng chạy được trên EC2 vẫn có thể không đọc được object S3. Từ góc nhìn người dùng, cả kết nối mạng bị chặn lẫn yêu cầu bị từ chối đều là "không lấy được báo cáo", nhưng chúng xảy ra ở những ranh giới khác nhau. Chương này theo một request từ client tới Application Load Balancer (ALB), tới tiến trình trên EC2 trong private subnet, rồi tới S3. Mỗi chặng phải trả lời ba câu hỏi: địa chỉ đích là gì, gói tin có đi tới đó không, và principal nào được phép thực hiện hành động gì trên tài nguyên nào. Ví dụ đặt trong một VPC IPv4 giả định ở Region `us-east-1`; tên subnet, bucket, policy và mọi quan sát bên dưới đều là dữ liệu học tập, không phải cấu hình hay log AWS đã chạy.

EC2 cung cấp máy ảo, network interface và vòng đời instance; ứng dụng là tiến trình chạy trên hệ điều hành của instance. EC2 ở private subnet không tự có quyền gọi API AWS. Khi gắn IAM role ứng dụng qua instance profile, phần mềm có thể lấy temporary credentials từ Instance Metadata Service (IMDS) và ký request tới dịch vụ AWS. Role có trust policy xác định ai được nhận role; permissions policy xác định hành động mà phiên role được phép yêu cầu. STS phát hành phiên tạm thời khi một principal gọi `AssumeRole` theo quan hệ tin cậy và quyền thích hợp. Với role gắn cho EC2, nền tảng cấp và làm mới credentials cho ứng dụng; không cần nhúng access key dài hạn vào image hay tự gọi `AssumeRole` để lấy lại chính role đó. Vẫn phải bảo vệ IMDS và tránh ghi credential vào log. Quyền của người triển khai hạ tầng, quyền của role instance và quyền của client HTTP là ba danh tính khác nhau.

Tự lần theo ba quan sát **giả định** của cùng ứng dụng. Ở A, SDK không lấy được credential từ chuỗi cung cấp nên chưa ký được `GetObject`: kiểm instance profile thực gắn với EC2, khả năng ứng dụng truy cập IMDS theo cấu hình và provider mà tiến trình đã chọn; không in token hoặc access key. Ở B, SDK đã chọn một credential nhưng kết nối TCP tới địa chỉ S3 hết hạn trước HTTP response: kiểm DNS, route, SG/NACL và endpoint của đúng subnet; chưa có căn cứ kết luận policy đã cho phép hay từ chối. Ở C, S3 trả `AccessDenied` cùng request ID: chuyển sang đối chiếu principal phiên role, action, ARN object và các policy áp dụng ở bài tiếp theo. Việc log ứng dụng không ghi request ID **không tự chứng minh** request chưa tới S3 vì logger có thể bỏ trường ấy. Sau khi sửa, thử lại đúng key bằng đúng danh tính, kiểm dữ liệu trả về và một key ngoài phạm vi vẫn bị chặn.

### Bài tập 25.1 — Ba ranh giới khi ứng dụng đọc S3

Trong tình huống giả định, `/health` của EC2 trả 200 nhưng tác vụ đọc S3 có ba quan sát lần lượt: A không lấy được credential, B hết hạn TCP trước HTTP response, C nhận `AccessDenied` kèm request ID của S3. Mỗi quan sát chuyển phép kiểm sang ranh giới nào? Có thể từ B kết luận IAM đúng, hoặc từ C kết luận identity policy gây từ chối không?

### Bài 25.2 — Quyết định

Một lần `s3:GetObject` không được đánh giá bằng tên role đơn lẻ. AWS xác thực chữ ký, xác định principal của phiên, action, ARN object, điều kiện của request, rồi xét các policy có hiệu lực. Mặc định là từ chối; một `Allow` hợp lệ phải áp dụng đúng action, resource và điều kiện. Một explicit `Deny` áp dụng ở bất kỳ policy liên quan nào thắng `Allow`. Identity policy, bucket policy, permissions boundary, session policy, SCP/RCP của tổ chức và endpoint policy có thể cùng tác động, nhưng cách kết hợp phụ thuộc loại principal, cùng hay khác account và dịch vụ cụ thể. Chẳng hạn resource policy cấp quyền trực tiếp cho role session có quy tắc khác với cấp cho role ARN; không thể dạy một phép giao hoặc hợp duy nhất cho mọi trường hợp. Trust policy kiểm soát bước nhận role, không thay cho quyền đọc object; bucket policy và endpoint policy cũng không tự tạo tuyến mạng. `AccessDenied` chỉ nói một yêu cầu đã tới lớp có thể từ chối nó; cần lấy principal thực, action, ARN, Region, request ID và loại policy được chỉ ra nếu thông báo có đủ chi tiết. Không suy chỉ từ chữ `AccessDenied` rằng IAM identity policy là nguyên nhân.

Đọc policy giả định sau như một bài phân tích, không phải policy đã gắn trên AWS. Giả sử đây là identity policy của đúng role session, không có policy khác gây hạn chế, bucket nằm cùng account và yêu cầu đã tới S3:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {"Effect": "Allow", "Action": "s3:GetObject",
     "Resource": "arn:aws:s3:::example-reports/public/*"},
    {"Effect": "Deny", "Action": "s3:GetObject",
     "Resource": "arn:aws:s3:::example-reports/public/raw/*"}
  ]
}
```

Với `public/summary.json`, statement Allow khớp còn Deny không khớp, nên trong các giả định đã nêu request có thể được phép. Với `public/raw/orders.csv`, cả hai khớp và explicit Deny quyết định từ chối. Với `private/orders.csv`, không statement nào khớp: đây là implicit deny, không phải explicit deny. Chỉ đổi tên role hoặc mở Security Group không làm thay đổi ba kết quả logic này. Trong sự cố thật, phải xem thêm điều kiện request, bucket/endpoint/KMS policy, boundary, session policy và chính sách tổ chức đang áp dụng; ví dụ đã cố ý loại chúng để người học tự tính một lớp trước.

#### Bài tập 25.2 — Truy nguyên AccessDenied của S3

Sau khi TCP/TLS đã hoạt động, `GetObject` nhận `AccessDenied` dù role có `Allow`. Hãy lần theo các policy cần xét, xác định kết luận từ explicit `Deny` trong dữ kiện bài và nêu một điều chưa thể kết luận chỉ từ HTTP 403.

### Bài 25.3 — Điều tra

Ở trạng thái sau khi có đường endpoint, giả sử quan sát mới là TCP/TLS hoàn tất và dịch vụ trả HTTP 403 `AccessDenied` cho cùng `GetObject`. Mạng đã đủ để nhận phản hồi; 403 không chứng minh đường đang đi qua endpoint mong muốn, cũng không tự chỉ ra policy nào thắng. Cần đối chiếu principal phiên role, bucket/Region/key, endpoint policy và các policy áp dụng; kiểm tra dấu vết request ở đúng tài khoản nếu có. Với dữ kiện identity policy ở trên, explicit `Deny` khớp object sẽ thắng `Allow`; bỏ hoặc thu hẹp deny phải qua review quyền tối thiểu, rồi kiểm chứng lại bằng `GetObject` cho đúng key và kiểm tra dữ liệu trả về. Từng phép sửa chỉ xử lý ranh giới mà bằng chứng xác nhận.

Tình huống không chạy trên tài khoản AWS. Khi kiểm chứng trên môi trường có thật, ghi thời điểm, Region, AZ ID, subnet/route table/endpoint ID, địa chỉ DNS, kết quả TCP/TLS, HTTP status và request ID mà không ghi secret hoặc payload nhạy cảm. Cấu hình có thể đổi trong khi điều tra; so cùng một cửa sổ thời gian và chạy lại đúng request sau mỗi thay đổi. Bất kỳ `AccessDenied` nào cũng có thể do bucket policy, SCP, session policy, endpoint policy hoặc KMS nếu object mã hóa bằng customer managed key; không được chuyển nguyên nhân giả định của ví dụ thành quy tắc chẩn đoán mọi sự cố.

Đọc `403` như một điểm đổi pha trong điều tra: trước đó chưa có phản hồi S3, sau đó đã có phản hồi cho một request cụ thể. Điều này không xác nhận đường đi qua gateway endpoint. Cần so route table liên kết với subnet của chính instance, prefix list S3 của Region, DNS mà tiến trình dùng và dấu vết mạng/endpoint tương ứng. Nếu ứng dụng dùng proxy HTTP hoặc custom DNS, đường request thực có thể khác sơ đồ VPC tưởng tượng. Chỉ khi identity của request và đường mạng được cố định mới chuyển sang bài toán ủy quyền. Một log ứng dụng chỉ chứa chữ `AccessDenied` không đủ: ứng dụng có thể đã bọc lỗi từ một API khác hoặc tự tạo status cho client.

Ở ranh giới IAM, hãy viết ra bộ đầu vào của quyết định: principal là phiên role nào, action là `s3:GetObject`, resource là ARN của object nào, request context có điều kiện gì, và các policy nào được áp dụng. Ví dụ giả định có `Allow` cho `arn:aws:s3:::orders-archive-example/reports/*` nhưng một `Deny` cũng khớp `reports/day-01.json`; kết quả là từ chối dù tuyến mạng đã thông. Xóa `Deny` quá rộng có thể làm lộ cả prefix khác. Phương án sửa phải giải thích vì sao statement đó tồn tại, thu hẹp điều kiện nếu phù hợp và kiểm tra cả ca được phép lẫn ca phải tiếp tục bị từ chối. Policy simulator có thể hỗ trợ suy luận policy, nhưng request thật còn phụ thuộc context, resource policy và trạng thái dịch vụ; không lấy một mô phỏng thành chứng cứ cuối.

Hậu kiểm có hai phần tách biệt: đúng object được đọc qua request nghiệp vụ từ client, và quyền ngoài phạm vi vẫn bị chặn. Ghi request ID phía S3, identity của role session và phiên cấu hình dùng trong lần thử. Nếu object có mã hóa bằng khóa do khách hàng quản lý, thêm quyền KMS phù hợp vào giả thuyết thay vì kết luận ngay “S3 Deny”. Đây là lý do phép sửa route và phép sửa policy phải được kiểm riêng, không gộp thành một lần thay đổi rồi suy nguyên nhân từ kết quả cuối.

# Chương 26 — VPC, mạng con và đường đi trong AWS

Chứng minh private workload có hoặc không có đường tới S3/STS.

## Bài 26.1 — Cấu trúc mạng VPC

VPC là không gian mạng của tài khoản trong một Region; subnet là một dải địa chỉ thuộc một Availability Zone (AZ). Route table gắn với subnet quy định đích IP/prefix đi tới target nào, theo tuyến khớp dài nhất; subnet không gắn tường minh dùng main route table. Public subnet trong mô hình IPv4 có tuyến mặc định tới Internet Gateway (IGW), nhưng một EC2 muốn giao tiếp trực tiếp ra internet còn cần địa chỉ public phù hợp và các rule cho phép. Private subnet không có đường trực tiếp ấy. Public ALB có thể nhận client ở các subnet public rồi kết nối tới private IP của target EC2; target không cần public IP. Health check xanh chỉ chứng minh phép thử được cấu hình, không chứng minh `s3:GetObject` hoặc tác vụ nghiệp vụ đã thành công.

IGW là cổng nối VPC với internet. Public NAT Gateway theo kiến trúc theo AZ nhận lưu lượng IPv4 outbound từ private subnet qua tuyến `0.0.0.0/0` của subnet ấy; subnet đặt NAT cần đường ra IGW. NAT cho phép kết nối do phía private khởi tạo và phản hồi đi về, không biến instance private thành server nhận kết nối inbound tùy ý. NAT Gateway là một phụ thuộc sẵn sàng và có chi phí tài nguyên/lưu lượng; nếu nhiều AZ cùng dựa vào một NAT theo AZ, AZ chứa NAT trở thành điểm lỗi chung. AWS còn có kiểu NAT và lựa chọn kiến trúc khác; kết luận này dành cho public NAT Gateway theo AZ đang xét, không áp dụng máy móc cho mọi kiểu. Endpoint nằm trong VPC và Region cụ thể; tạo endpoint nhưng quên liên kết route table của subnet vẫn không tạo đường đi cho subnet đó. Endpoint policy là một lớp lọc quyền bổ sung cho lưu lượng đi qua endpoint, không thay IAM/bucket policy.

Một phép lần đường giả định làm rõ route table. VPC có CIDR `10.20.0.0/16`; EC2 A ở private subnet AZ A có IP `10.20.1.10`, EC2 B ở private subnet AZ B có IP `10.20.2.10`. Cả hai subnet có route `10.20.0.0/16 → local` và `0.0.0.0/0 → nat-a`, trong đó `nat-a` là public NAT Gateway ở AZ A; subnet của NAT có route ra IGW. Từ A tới B, đích `10.20.2.10` khớp `/16` cụ thể hơn `/0`, nên đi theo route local, không đi NAT. Từ B tới địa chỉ internet IPv4 `198.51.100.10`, route mặc định chọn `nat-a`; đường về thuộc kết nối B đã khởi tạo. Đây là sơ đồ địa chỉ dùng để suy luận, không phải kết quả chạy AWS hoặc bảo đảm firewall cho phép.

Nếu AZ A mất, B vẫn có thể chạy ứng dụng nhưng đường outbound qua `nat-a` bị mất. Có thể đặt một public NAT Gateway theo AZ và cho mỗi private subnet dùng NAT cùng AZ để tránh phụ thuộc chéo ấy, rồi kiểm chi phí và route thật. Một S3 gateway endpoint được gắn vào đúng route table có thể tạo đường riêng cho lưu lượng S3 cùng Region theo prefix list; điều đó không sửa đường tới mọi dịch vụ AWS hay internet. Khi thấy timeout, đọc địa chỉ đã phân giải, route table hiệu lực của đúng subnet, target của route được chọn và các bộ lọc hai chiều trước khi kết luận NAT hỏng.

## Bài 26.2 — Chính sách gói

VPC Flow Logs, nếu đã bật ở đúng network interface và khoảng thời gian, có thể giúp xét ACCEPT/REJECT ở tầng mạng, nhưng không giải thích được nội dung policy IAM hay mọi nguyên nhân timeout.

Với một request từ instance, hãy tách hai câu hỏi: gói tin có đi qua ranh giới mạng hay không, và danh tính gọi API có được phép hay không. Security Group (SG) gắn với network interface và gồm các luật cho phép; nó theo dõi kết nối, nên gói phản hồi của kết nối được phép không cần một luật chiều ngược riêng. NACL xét lưu lượng ở ranh giới subnet theo các luật và hướng riêng; vì vậy cần kiểm tra cả cổng đích lẫn cổng tạm thời của chiều phản hồi trong cùng đường đi. Route table quyết định next hop, không tự tạo một luật cho phép ở SG hoặc NACL.

Nếu phép quan sát ở network interface cho thấy luồng bị từ chối, đó là bằng chứng để tiếp tục đối chiếu luật và thời điểm, chưa phải bằng chứng về IAM. Ngược lại, khi request tới được endpoint và nhận `AccessDenied`, ta phải xét principal, action, resource cùng các lớp policy áp dụng. Giữ nguyên Region, subnet, địa chỉ đích và thời gian khi so các phép đo; đổi đồng thời nhiều yếu tố sẽ làm mất quan hệ nhân quả.

Với kết nối **giả định** từ `10.0.2.10:49152` tới `10.0.1.20:443`, gói trả lời đi từ cổng `443` về cổng tạm `49152`. Một SG cho phép chiều đi đã theo dõi kết nối có thể nhận gói trả lời, nhưng NACL ở mỗi subnet là bộ lọc không trạng thái: luật chỉ mở `443` theo một chiều chưa đủ cho cả hai chiều. Đọc luật NACL theo subnet nguồn và đích, hướng vào/ra và cổng của từng gói, rồi so với flow log cùng khoảng thời gian. Nếu vẫn chỉ thấy timeout, không quy nguyên nhân cho NACL từ sơ đồ này; DNS sai, route thiếu, endpoint không phục vụ hoặc gói trả lời đi đường khác vẫn là những giả thuyết cần loại trừ.

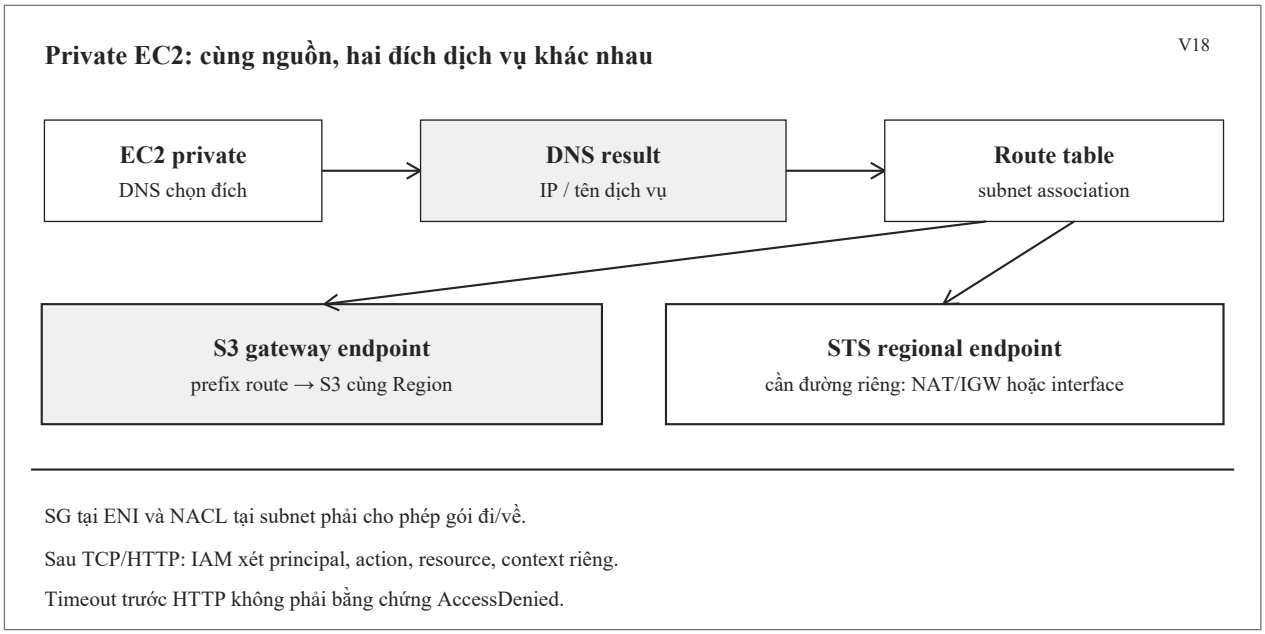
Lần theo đúng kết nối ấy với **bộ luật giả định**: route hai chiều có sẵn; SG của client cho outbound TCP 443, SG của server cho inbound TCP 443. NACL subnet client cho outbound tới đích 443 nhưng chặn inbound có **cổng đích 49152** từ subnet server; NACL subnet server cho inbound đích 443 và outbound đích 49152. Giả sử không có luật số nhỏ hơn khớp trước, SYN có thể tới server, nhưng SYN-ACK từ `10.0.1.20:443` về `10.0.2.10:49152` bị chặn khi vào subnet client. Client thấy timeout dù bản ghi flow ở phía server cho chiều SYN là `ACCEPT`. NACL so luật theo thứ tự và xét từng chiều, còn SG theo dõi kết nối; vì vậy không mở thêm SG inbound 49152 trên client để sửa nhầm lớp. Đối chiếu tuple, interface, hướng, thời gian và luật NACL hiệu lực tại thời điểm lỗi; Flow Logs gộp theo cửa sổ và có thể trễ, còn `REJECT` không tự định danh một luật cụ thể hay luôn có nghĩa SG/NACL đã từ chối một kết nối mới. Khi sửa, cho phép chiều trả về theo dải cổng tạm thực của client và phạm vi nguồn cần thiết, thử lại TCP/TLS từ chính client rồi mới phân tích HTTP hoặc IAM nếu có phản hồi. Một cổng 49152 trong ví dụ không phải dải ephemeral mặc định của mọi hệ điều hành.

**Bài tập 26.1 — Lần theo gói trả về qua hai subnet**

Client 10.0.2.10:49152 gọi server 10.0.1.20:443. Route và SG cho phép; NACL phía server cho inbound 443/outbound 49152, NACL phía client cho outbound 443 nhưng chặn inbound đích 49152 từ server. Giả sử không có luật ưu tiên cao hơn: SYN, SYN-ACK đi tới đâu, vì sao client timeout dù server có Flow Log ACCEPT? Nêu thay đổi hẹp và hậu kiểm trước khi xét IAM.

## Bài 26.3 — Truy cập riêng tới dịch vụ AWS

DNS đổi tên thành địa chỉ; nó không chứng minh TCP tới địa chỉ ấy kết nối được. Route 53 Resolver/AmazonProvidedDNS có thể trả lời trong VPC, còn DNS tùy chỉnh, private DNS của endpoint, Region trong SDK và cache có thể làm ứng dụng nhận một đích khác với dự tính. Network ACL (NACL) áp cho subnet, có thứ tự luật allow/deny và không giữ trạng thái: cả chiều đi lẫn chiều về, gồm cổng ephemeral của client, phải được cho phép. “Route đúng” chưa chứng minh SG/NACL đúng; “DNS trả IP” chưa chứng minh route hay quyền đúng.




Hình VIII.2.1 — Đường tới S3 và STS khác nhau; DNS, mạng và IAM là các ranh giới riêng.

Ở trạng thái đầu, giả sử DNS trên instance trả địa chỉ cho S3 regional endpoint, ứng dụng bắt đầu TCP 443 nhưng hết hạn kết nối; log ứng dụng chỉ ghi timeout và không có HTTP response từ S3. Đó là bằng chứng về đích DNS và việc chưa thiết lập được kết nối, không phải bằng chứng IAM đã cho phép hoặc từ chối. Đọc subnet association và route table cho thấy chỉ có tuyến local; endpoint được tạo nhưng không gắn với table ấy. Kiểm tra SG outbound, NACL outbound 443/inbound ephemeral, trạng thái endpoint và đường đi vẫn cần làm trước khi quy nguyên nhân duy nhất. Trong bài toán đã cho, thiếu tuyến tới S3 là lời giải cho timeout; sửa bằng cách liên kết đúng route table theo thay đổi được phê duyệt, không mở rộng `0.0.0.0/0` hoặc thêm NAT chỉ để lấp một khoảng trống bằng suy đoán.

Với S3 cùng Region, gateway VPC endpoint có thể thêm tuyến prefix list S3 vào route table đã liên kết, để lưu lượng đi tới S3 qua endpoint mà không cần NAT/IGW. Nếu ứng dụng còn gọi STS regional endpoint, S3 gateway endpoint không tự mở đường tới STS; cần xét NAT hoặc interface endpoint tương ứng cùng DNS và quyền. Nếu request tới STS regional endpoint vẫn timeout trong cùng private subnet, đó là đường mạng khác: S3 gateway endpoint không cấp kết nối cho STS.

Một điểm dễ chẩn đoán sai là gateway endpoint **không đòi DNS trả địa chỉ riêng của endpoint**. Trong mô hình này, workload vẫn dùng tên public của S3 cùng Region; route table chọn đích theo prefix list do AWS quản lý và đưa lưu lượng qua gateway endpoint. Giả sử cùng câu trả lời DNS là địa chỉ `S` nằm trong prefix list S3 của Region: subnet A có route `pl-S3 → vpce-1`, subnet B chỉ có

route `local`. Từ A, phép chọn route có đường tới endpoint; từ B, cùng địa chỉ `S` không có đường ra nếu B không có NAT/IGW. `S` là ký hiệu giả định, không phải IP hay output DNS thật. Vì thế đổi resolver hoặc thêm một bản ghi private thủ công không giải quyết thiếu association của route table B. Phải xác định instance thuộc subnet nào, subnet đó dùng route table nào, route prefix list có target nào, rồi mới so DNS của chính ứng dụng.

Đổi đúng một điều kiện để thấy giới hạn: nếu ứng dụng gọi STS thay vì S3, route prefix list S3 không khớp đích STS dù trước đó `GetObject` đã thành công. Nếu ứng dụng gọi S3 ở Region khác, không được mặc nhiên lấy route S3 cùng Region làm bằng chứng đường tới endpoint kia. Nếu route đã khớp mà kết nối vẫn timeout, đối chiếu SG outbound, NACL ở subnet theo chiều đi và cổng tạm của chiều về, đích DNS thực, và trạng thái endpoint; không thêm IAM Allow để thử một lỗi chưa tới bước ủy quyền. Sau khi có HTTP response của S3, mới chuyển sang phân tích principal, action, resource, endpoint policy và bucket policy. Chuỗi này giúp tách "đã chọn tuyến", "đã nối TCP", "dịch vụ đã trả lời" và "được cấp quyền" thành bốn hậu điều kiện quan sát được, thay vì coi một lệnh kiểm DNS là phép thử toàn bộ.

**Bài tập 26.2 — S3 timeout trong mạng riêng**

Với cấu hình ở Bài 26.3, DNS trả địa chỉ S3 nhưng TCP 443 timeout. Hãy nêu ba phép kiểm tra phân biệt route, SG/NACL và DNS. Vì sao tăng quyền role chưa phải phép sửa có căn cứ?

# Chương 27 — EC2, cân bằng tải và miền lỗi

Phân biệt instance running, target healthy và request nghiệp vụ đúng.

## Bài 27.1 — Tài nguyên tính toán EC2

EC2 cung cấp một máy ảo theo cấu hình instance type và Amazon Machine Image (AMI) được chọn. AMI là đầu vào để tạo root filesystem, không phải bản ghi của tiến trình và dữ liệu mới sinh sau khi máy chạy. Một launch request còn chọn subnet, security group, loại root volume, user data và IAM instance profile. Mỗi yếu tố trả lời câu hỏi khác: subnet và ENI quyết định địa chỉ cùng đường mạng; security group lọc lưu lượng; profile đưa credential tạm thời tới workload; user data là cơ chế khởi tạo, không phải bằng chứng ứng dụng đã sẵn sàng.

Theo tình huống `orders-api` ở chương này, theo dõi một request từ ALB tới target `i-demo-01` trong private subnet. Trước hết kiểm chứng instance ID, AMI ID, AZ, ENI và profile thực tế; sau đó xem hệ điều hành đã boot, tiến trình đã nạp cấu hình, socket đã listen và probe của target group đã qua chưa. Trạng thái EC2 `running` chỉ mô tả vòng đời compute. Nó không chứng minh script bootstrap thành công, credential đủ quyền `s3:GetObject`, hay endpoint `/reports` đọc được S3. Health check `/health` trả 200 cũng chỉ chứng minh hợp đồng của chính đường `/health`. Giữ các phép kiểm theo ranh giới này tránh quy lỗi IAM cho một máy chưa có route hoặc quy lỗi ALB cho ứng dụng đang trả AccessDenied.

Instance profile gắn role vào EC2 để phần mềm lấy credential từ metadata service theo cơ chế của AWS; role chỉ có hiệu lực khi trust và policy phù hợp, còn ứng dụng phải dùng credential đúng chuỗi cung cấp. Đừng chép access key tĩnh vào AMI hay user data. Nếu cần thay profile hoặc policy, xác nhận lại role mà request thực sự đang dùng và quyền trên resource đích; việc gắn profile vào instance không tự chứng minh một call API đã được cho phép.

Dừng rồi khởi động một instance EBS-backed khác với terminate và tạo instance thay thế. Ở đường stop/start, instance ID và EBS root có thể giữ lại, còn địa chỉ public tự cấp có thể đổi; ở đường replace, ID mới xuất hiện và số phận root volume cũ phụ thuộc cờ xóa lúc terminate. Dữ liệu nghiệp vụ không nên chỉ sống trên root volume nếu có yêu cầu phục hồi. Trước mọi phép thay máy, ghi ID, vùng, volume, snapshot và phép thử đọc dữ liệu cần giữ. Những thông số của `i-demo-01` ở đây là giả định để phân tích, không phải AWS đã được truy vấn.

### Bài tập 27.1 — EC2 chạy nhưng API bị từ chối quyền

EC2 đã running và ALB /health trả 200 nhưng /reports lỗi AccessDenied. Hai phép quan sát ấy chứng minh điều gì, và cần kiểm gì tiếp?

### Bài 27.2 — Luồng lưu lượng

Giả sử ở `us-east-1`, client gọi HTTPS tới public ALB; target EC2 `orders-api` nằm trong subnet `10.24.2.0/24` của một AZ, không có public IPv4. Tiến trình đọc `reports/day-01.json` từ bucket giả định `orders-archive-example`, rồi trả báo cáo. ALB và target có đường tới nhau; health check chỉ gọi `/health`, không đọc S3. Role ứng dụng cho phép `s3:GetObject` trên prefix báo cáo, nhưng một statement khác trong chính identity policy giả định `Deny` đúng action và object ấy. VPC có S3 gateway endpoint, song route table của subnet `10.24.2.0/24` chưa liên kết với endpoint; subnet cũng không có tuyến NAT. Đây là dữ kiện thiết kế giả định, không phải kết quả AWS đã đo.

Lần theo request thay vì nhìn riêng mã lỗi: ALB có thể đánh dấu target healthy vì `/health` không đụng S3, rồi request báo cáo lại treo ở bước gọi S3 do không có đường ra. Liên kết đúng route table với gateway endpoint chỉ sửa đường mạng tới S3; nó không xóa `Deny`. Sau khi đường mạng được sửa, `AccessDenied` có thể mới lộ ra. Dùng dấu thời gian và request ID của cùng lượt gọi để tách timeout kết nối, phản hồi từ S3 và phản hồi do ứng dụng tự sinh. Phép thử chỉ kết luận về bucket, key, role session, endpoint policy và Region đang dùng; nó không chứng minh mọi truy cập AWS từ subnet đều hoạt động.

Hãy đặt ba dấu mốc vào cùng một request giả định: `r17` tới ALB lúc 10:00:00, target nhận lúc 10:00:01, ứng dụng bắt đầu `GetObject` lúc 10:00:02. Nếu không có log phản hồi từ S3 rồi ứng dụng timeout ở 10:00:12, chưa thể kết luận IAM cho phép hay từ chối: yêu cầu có thể chưa tới dịch vụ. Đầu tiên xác nhận DNS của tên S3 trả địa chỉ phù hợp cho Region và target có thể tạo kết nối; sau đó đọc route table thật được liên kết với subnet target, không chỉ route table mang tên “private”. Với gateway endpoint S3, tuyến tới prefix list của S3 phải hiện trong route table phù hợp. Security Group, NACL, endpoint policy và cấu hình DNS vẫn có thể là điều kiện bổ sung; không lấy một sửa route làm chứng minh mọi lớp còn lại đúng.

Giả sử sau khi liên kết endpoint, cùng phép gọi trả `AccessDenied` với request ID của S3. Đó là một bước tiến về khả năng quan sát: request đã đi tới lớp ủy quyền và bị từ chối trong phạm vi yêu cầu này. Đối chiếu role session thực sự của EC2, action `s3:GetObject`, ARN của đúng key, identity policy, bucket policy và endpoint policy. Trong dữ kiện giả định của bài, explicit `Deny` trong identity policy đủ để chặn dù có `Allow` khác. Việc nới Security Group hoặc cấp thêm một `Allow` sẽ không thắng `Deny`; sửa policy phải ở đúng statement sau khi xác nhận chủ thể và phạm vi object. Không in credential hoặc nội dung object vào log chẩn đoán.

Một bảng chứng cứ nhỏ tự lập nên có bốn cột: thời điểm, lớp kiểm tra, phép quan sát và điều còn chưa biết. Ví dụ `ALB healthy` chỉ loại bớt giả thuyết target không qua `/health`; `S3 request ID` mới cho biết đã có phản hồi từ S3; `GetObject` thành công với key thử chỉ chứng minh key và role session ấy trong thời điểm ấy. Sau sửa, gửi lại request báo cáo qua ALB, đối chiếu record được trả với object kỳ vọng và xác nhận không còn timeout hoặc AccessDenied cho cùng hành trình. Nếu không có tài khoản AWS thử, đây là tình huống suy luận từ cấu hình giả định; không tạo tài nguyên tốn phí để có vẻ như đã thực nghiệm.

### Bài tập 27.2 — Tách đường ALB, S3 và STS

ALB `/health` trả 200; API báo cáo lỗi, còn STS timeout. Phân biệt đường client–ALB–EC2, EC2–S3, EC2–STS và nêu phép kiểm chứng sau sửa.

### Bài 27.3 — Miền lỗi

Availability Zone (AZ) là ranh giới vận hành để bố trí bản sao compute; một Region có nhiều AZ, nhưng hai instance cùng AZ vẫn cùng chịu một số sự cố vùng cục bộ. Đưa hai target của `orders-api` vào hai AZ, cùng cấu hình và cùng được ALB health check, làm giảm phụ thuộc vào một AZ. Điều đó chỉ có ích khi subnet, route, security group, capacity, dependency và dữ liệu cho phép target ở AZ còn lại phục vụ. Hai máy cùng đọc một database chỉ có một điểm lỗi thì sơ đồ "multi-AZ" của compute chưa tạo ứng dụng chịu lỗi toàn tuyến.

Giả định target ở AZ A rời nhóm lúc 09:02, ALB chuyển request mới sang target ở AZ B. Muốn kết luận dịch vụ còn khả dụng, cần quan sát target health theo từng AZ, số target sẵn sàng, request thật qua ALB, latency và lỗi nghiệp vụ, cùng khả năng AZ B nhận tải bổ sung. Một probe trả 200 không kiểm được đường đọc S3 của `/reports`; ngược lại một request `/reports` lỗi AccessDenied ở cả hai AZ gợi ý dependency hoặc quyền chung, không phải riêng máy A. ALB routing cũng có độ trễ phát hiện theo chu kỳ và ngưỡng health check; không hứa failover tức thời.

Khi Auto Scaling hoặc người vận hành tạo instance thay thế, AMI và launch template tạo cấu hình khởi đầu, không di chuyển writable state từ instance cũ. Root EBS gắn với AZ và cờ DeleteOnTermination; EFS, S3 hay database có vòng đời và failure domain khác. Trước khi rút target A, xác định 41 đơn giả định nằm ở đâu: trên root volume, trong hàng đợi, hay đã commit vào kho dữ liệu bền vững. Nếu chỉ ở root A, một request mới đi sang B có thể thiếu dữ liệu dù B healthy. Nếu cần phục hồi, phải có bản sao nhất quán và thử đọc lại, không chỉ tin số bản sao hoặc tên dịch vụ storage.

Chạy thử kế hoạch chuyển lưu lượng trong môi trường được phép: ghi đường request và số bản ghi trước, cô lập một target có kiểm soát, đo lại tỷ lệ lỗi và dữ liệu sau, rồi phục hồi target. Không tạo hay dừng tài nguyên AWS chỉ để chứng minh ví dụ này; các mốc giờ và số đơn là dữ kiện giả định. Kết luận hợp lệ từ mô hình là danh sách hậu điều kiện cần kiểm: ít nhất một target Ready với đủ capacity, request đại diện thành công, và dữ liệu có thể đọc được sau chuyển AZ. Nó chưa chứng minh một hệ thật đạt mục tiêu khôi phục.

### Bài tập 27.3 — Mất một AZ khi health check vẫn xanh

Hai EC2 ở hai AZ đều /health xanh. Có thể kết luận /reports vẫn phục vụ sau khi mất AZ A không?

# Chương 28 — Lưu trữ khối, tệp và đối tượng

Chọn EBS/EFS/S3 theo cách truy cập và vòng đời dữ liệu.

## Bài 28.1 — Lưu trữ khối

EC2 instance có CPU, RAM, hệ điều hành, ENI và trạng thái chạy; dừng hoặc thay instance không tự quyết định số phận mọi dữ liệu. Instance store, nếu loại instance có cung cấp, gắn với vòng đời phần cứng/instance và không phải nơi đặt dữ liệu cần tồn tại sau mất instance. EBS là block volume: hệ điều hành thấy thiết bị, tạo filesystem rồi ứng dụng đọc/ghi như đĩa. Volume EBS thuộc một AZ và chỉ attach cho instance tương thích trong cùng AZ; không thể coi nó như filesystem dùng chung giữa các AZ. Dữ liệu EBS có thể còn sau khi instance dừng, nhưng khi terminate phải xem thuộc tính `DeleteOnTermination` của từng volume. Redundancy của volume trong AZ giúp chịu lỗi thiết bị, không tự tạo bản khôi phục khỏi xóa nhầm, lỗi ứng dụng hay mất AZ. Snapshot là bản sao theo thời điểm cần được lên lịch, bảo vệ và thử restore; snapshot của volume đang ghi không tự chứng minh nhất quán giao dịch của database.

Trong một thay instance giả định, root volume đặt `DeleteOnTermination=true`, còn data volume đặt `false`. Terminate có thể xóa root volume nhưng giữ data volume; dữ liệu còn đó không có nghĩa instance mới ở AZ khác attach được ngay, vì EBS volume thuộc AZ. Lập kế hoạch phải ghi ID volume, AZ, trạng thái attachment, filesystem và khóa mã hóa liên quan trước khi thay, rồi đọc lại dữ liệu ứng dụng sau khi gắn. Nếu cần chuyển AZ, phải dùng cơ chế sao chép hoặc snapshot/restore phù hợp và thử tính nhất quán. Đây là phân tích cấu hình giả định, không phải một thao tác AWS đã thực hiện.

Điều kiện `DeleteOnTermination` thuộc từng ánh xạ EBS của instance; không nên đoán giá trị của data volume theo chữ "data" vì mặc định có thể khác theo cách nó được gắn lúc tạo. Với ca trên, nếu instance ở AZ A bị terminate và data volume thật sự được giữ, volume có thể được gắn sang một instance tương thích khác trong A sau khi trạng thái cho phép; hệ điều hành mới còn phải nhận đúng thiết bị, mount filesystem và ứng dụng đọc được bản ghi. Nếu cả AZ A không dùng được, volume ấy không thể attach trực tiếp ở AZ B. Một snapshot đã hoàn tất trong cùng Region có thể dùng tạo volume mới ở B, rồi mới gắn và kiểm dữ liệu. EC2 mới ở trạng thái `running` chỉ hoàn tất phần tính toán, chưa hoàn tất phục hồi ứng dụng.

Đặt mốc giả định: snapshot nhất quán ở mức ứng dụng gần nhất chốt lúc 09:45, AZ A ngừng phục vụ lúc 10:00, bản sao ở B khởi động lúc 10:08. Nếu không có log hoặc nguồn giao dịch khác để replay, điểm khôi phục đang cách sự cố 15 phút; đây là **khoảng dữ liệu có nguy cơ**, không phải số đơn đã mất được đo. Không thể gọi RTO là 8 phút chỉ vì EC2 đã chạy: phải chờ volume từ snapshot, mount, database phục hồi và request nghiệp vụ đọc/ghi đúng, rồi mới lấy mốc phục vụ để tính. Nếu snapshot được chụp khi database còn ghi mà không phối hợp ứng dụng, mốc 09:45 có thể chỉ nhất quán ở mức block; trước khi quảng bá lưu lượng phải thử recovery và đối soát dữ liệu. Một phương án có snapshot nhưng không có khóa giải mã, quyền tạo/gắn volume hoặc bài thử restore cũng chưa chứng minh được khả năng phục hồi.

### Bài 28.2 — Lưu trữ tệp

EFS cung cấp filesystem chia sẻ qua NFS, không phải block device. Client cần DNS, tuyến tới mount target, SG cho NFS và quyền ở lớp filesystem/ứng dụng. EFS Regional giữ dữ liệu qua nhiều AZ trong Region; tạo mount target ở các AZ cần truy cập giúp tránh phụ thuộc vào đường liên AZ. EFS One Zone đặt dữ liệu và mount target trong một AZ, nên không có cùng mô hình chịu mất AZ. Hai loại này có phạm vi sẵn sàng khác nhau, và quyền NFS không thể thay bằng IAM đơn giản. Một mount thành công vẫn chưa xác nhận dữ liệu được backup hay ứng dụng ghi đúng file.

Giả sử hai EC2 ở hai AZ cùng ghi vào `/reports/daily.csv` trên EFS Regional. Cả hai nhìn cùng filesystem, nhưng điều đó không tự bảo đảm hai tiến trình phối hợp ghi an toàn: ứng dụng phải định nghĩa lock, cách ghi tạm rồi công bố, và cách xử lý người ghi đồng thời. Nếu một máy báo `permission denied`, tách lỗi thiết lập NFS ở cổng 2049 khỏi lỗi UID/GID hoặc quyền ở đường dẫn đã mount. Chỉ sau khi ghi từ một máy, đọc lại từ máy kia và so nội dung mới có bằng chứng về đường dữ liệu của ca thử. Thử nghiệm ấy vẫn không thay bài kiểm tra phục hồi bản sao sau mất dữ liệu hoặc hỏng Region.

Cùng là "không đọc được file", nhưng ba thời điểm đòi ba phép đo khác nhau. Nếu mount chưa tạo được, kiểm DNS của tên EFS, mount target trong AZ, route và Security Group hai chiều cho NFS; quyền mode trên `/reports` chưa có cơ hội được xét. Nếu mount đã hiện nhưng `open()` trả `EACCES`, kiểm identity UID/GID mà NFS nhìn thấy, quyền thư mục và access point nếu dùng, rồi mới xét policy IAM cho NFS khi cơ chế ấy được bật. Nếu `open()` thành công mà nội dung cũ, kiểm ứng dụng đã đóng/đồng bộ thao tác ghi theo hợp đồng close-to-open và đang đọc đúng path; "EFS Regional" không tự bảo đảm hai writer ghi chồng một file thành một bản ghi hợp lệ.

Ví dụ giả định: EC2 A ghi một dòng `42` rồi đóng file; EC2 B mở lại cùng đường dẫn và đọc `42`. Phép thử ấy chứng minh đường đọc/ghi của hai máy trong lúc thử, chưa chứng minh các cập nhật đồng thời được tuần tự hóa. Nếu A và B cùng append một record với cùng khóa, cần cơ chế ứng dụng ngăn trùng hoặc lock được hai bên tuân thủ. EFS hỗ trợ khóa NFS, nhưng khóa advisory chỉ có tác dụng khi client phối hợp; tiến trình bỏ qua lock vẫn có thể ghi. Ghi nhận thứ tự mở, ghi, flush/close và đọc khi đo; không suy từ một lần đọc đúng rằng dữ liệu đã chịu được mất AZ, xóa nhầm hoặc ransomware.

Khi thiết kế, chọn EFS Regional nếu cần filesystem dùng chung qua AZ với mô hình chịu lỗi của dịch vụ ấy; One Zone có miền lỗi hẹp hơn. Dù chọn loại nào, phân biệt tính sẵn sàng của filesystem với khả năng phục hồi phiên bản dữ liệu. Có thể tạo file kiểm thử trong một thư mục tạm riêng, đọc từ hai client rồi xóa đúng file thử theo chính sách lab; phép restore phải dùng bản backup riêng, không thử phá filesystem thật. Ghi rõ chi phí truyền liên AZ và yêu cầu độ trễ khi đặt client khác AZ với mount target gần nhất, nhưng không bịa con số giá hoặc độ trễ cố định.

### Bài 28.3 — Lưu trữ đối tượng

S3 lưu object trong bucket ở Region đã chọn; ứng dụng gọi API theo key, không mount mặc định như block device hay NFS. S3 Standard được thiết kế phân tán qua nhiều AZ, còn một số storage class có mô hình một AZ hoặc thời gian truy xuất khác; phải đọc lớp lưu trữ thực tế thay vì suy từ chữ "S3". Durability là xác suất giữ byte theo thiết kế, availability là khả năng truy cập dịch vụ khi cần, còn backup là khả năng trở về một trạng thái lịch sử đã chọn. Replication có thể chép cả thay đổi sai hoặc xóa tùy cấu hình; không thay cho versioning, lưu bản sao độc lập theo chính sách, kiểm tra quyền khôi phục và diễn tập restore. EBS tính theo dung lượng/hiệu năng được cấp và snapshot; EFS theo lớp lưu trữ, thông lượng/truy cập; S3 theo lớp, dung lượng, request và truy xuất; NAT và truyền liên AZ có thể thêm chi phí. Không chọn dịch vụ chỉ bằng giá mỗi GB, cũng không bịa giá cố định cho một Region chưa kiểm tra bảng giá hiện hành.

Nếu ứng dụng ghi nhầm object `reports/current.json`, việc ba AZ đều giữ bản sai không khôi phục bản đúng. Hậu điều kiện của chiến lược phục hồi là tìm được phiên bản lịch sử mong muốn, đọc được byte và metadata cần thiết, rồi chứng minh ứng dụng thực sự dùng bản đã phục hồi; chỉ nhìn trạng thái replication xanh chưa đủ. Khi so chi phí, mô tả workload bằng kích thước object, số request đọc/ghi, dữ liệu ra ngoài và thời hạn giữ bản cũ rồi tính theo bảng giá của Region và storage class đang chọn. Không dùng một tỷ lệ durability được quảng bá làm ước lượng RTO hay RPO của ứng dụng.

Xét bucket **đã bật Versioning trước sự cố** với key `reports/current.json`. Lúc 09:00 ứng dụng ghi nội dung đúng thành version `v1`; lúc 09:10 job ghi sai thành `v2`; lúc 09:12 một lệnh xóa key không chỉ định version ID tạo delete marker `m3` làm phiên bản hiện hành. Một lần đọc key thông thường sau 09:12 có thể báo không tìm thấy dù `v1` và `v2` vẫn là các phiên bản cũ. Phép điều tra là liệt kê version và marker của đúng key, đọc `v1` theo version ID, so nội dung với nguồn nghiệp vụ độc lập rồi chọn cách đưa byte đúng trở lại thành phiên bản hiện hành theo quy trình được duyệt. Các ID và thời điểm chỉ là dữ liệu giả định. Xóa **chỉ định** version ID có thể xóa vĩnh viễn chính version ấy; lifecycle hết hạn phiên bản cũ cũng có thể lấy mất đường phục hồi. Bật Versioning sau 09:12 không thể tạo lại `v1` đã không được giữ từ trước.

Trong ca này, `v1` có thể giúp trả báo cáo đúng tới 09:00, nhưng các thay đổi hợp lệ giữa 09:00 và 09:10 cần được đối soát riêng; "đã restore một object" không xác nhận RPO của cả hệ thống. Trước khi đổi phiên bản hiện hành, kiểm ứng dụng có cache hay đang đọc đúng key/bucket/Region không, ai có quyền đọc version cũ và ghi bản phục hồi, và replication/lifecycle có thể ảnh hưởng bản nào. Sau khôi phục, đọc key **không kèm version ID** qua đúng đường ứng dụng, so byte và metadata mong đợi, rồi xác nhận log của một request thật đã dùng bản ấy. Nếu không có version hoặc bản sao độc lập được bảo vệ, trạng thái là chưa có bằng chứng phục hồi, không thể suy từ durability của S3 rằng byte cũ vẫn tồn tại.

#### Bài tập 28.1 — Độ sẵn sàng và phục hồi của ba loại lưu trữ

Một nhóm đặt database trên EBS cùng AZ với EC2, file dùng chung trên EFS Regional và báo cáo trên S3 Standard. AZ của EC2 hỏng, sau đó một job xóa nhầm object báo cáo. Mỗi lớp có thể tiếp tục phục vụ hoặc phục hồi theo điều kiện nào? Giải thích vì sao redundancy không đủ thay backup.

# PHẦN IX — Terraform và trạng thái hạ tầng

## Cấu hình, state và hạ tầng thực lệch nhau thì quyết định thay đổi thế nào?

Provider dựng DAG rồi đọc state/API và thực hiện các tác vụ có thể thất bại từng phần; state là ánh xạ nhạy cảm.

IX

**LỘ TRÌNH PHẦN**


- Chương 29 — HCL, nhà cung cấp và đồ thị phụ thuộc
- Chương 30 — Kế hoạch thay đổi, trạng thái và kho lưu trữ trạng thái
- Chương 31 — Sai lệch hạ tầng, nhập tài nguyên và tái cấu trúc
- Chương 32 — Thay đổi hạ tầng đang chứa dữ liệu

# Chương 29 — HCL, nhà cung cấp và đồ thị phụ thuộc

Đọc tài nguyên, data source và cạnh implicit/explicit trong plan.

## Bài 29.1 — Ngôn ngữ

Một `resource` khai báo object Terraform quản lý và vòng đời tạo, đọc, sửa, xóa của nó thông qua provider.

Một cấu hình HCL diễn đạt quan hệ mong muốn, không phải bản ghi đầy đủ về thứ đang tồn tại. `variable` nhận đầu vào của lần chạy; expression kết hợp các giá trị; `output` công bố giá trị sau khi Terraform tính được. `data` đọc một đối tượng do nơi khác quản lý để dùng trong đồ thị, còn `resource` yêu cầu provider quản lý một đối tượng trong state của cấu hình này. Các tên trong HCL là địa chỉ logic để các khối tham chiếu nhau; chúng không tự chứng minh rằng API phía xa đã tạo đúng đối tượng.

Khi một expression trong resource tham chiếu thuộc tính của resource khác, Terraform có cơ sở suy ra cạnh phụ thuộc. Nếu thứ tự cần thiết không thể hiện qua dữ liệu tham chiếu, người viết phải diễn đạt quan hệ ấy rõ ràng và giải thích vì sao. Đọc plan bằng cách đối chiếu địa chỉ logic, ID từ state và hành động dự kiến. Chỉ sau apply và phép kiểm tra hậu điều kiện tại hạ tầng thật mới được kết luận thay đổi đã đạt mục tiêu.

Ví dụ HCL nhỏ sau dùng resource tích hợp `terraform_data`, không tạo cloud object. Địa chỉ `terraform_data.deploy` phụ thuộc `terraform_data.config` vì expression của `input` đọc `output` của resource trước. Đó là cạnh dữ liệu mà Terraform tự suy ra, không cần `depends_on` chỉ để đặt thứ tự:

```
resource "terraform_data" "config" {
  input = "v2"
}

resource "terraform_data" "deploy" {
  input = terraform_data.config.output
}
```

Nếu thay dòng cuối bằng `input = "v2"`, hai khối tình cờ cùng giá trị nhưng không còn cạnh phụ thuộc từ expression. Trong ví dụ chỉ có dữ liệu giả định nên điều này chưa tạo lỗi vận hành; ở hệ thật, một phụ thuộc ẩn như “policy phải tồn tại trước khi API cho tạo resource” cần được mô tả rõ. `depends_on` thêm cạnh thứ tự nhưng không kiểm policy có hiệu lực thật sau khi API trả thành công. Muốn tự kiểm, có thể chạy `terraform init` và `terraform plan` trong thư mục tạm chứa hai khối trên, xem địa chỉ và action dự kiến; không khẳng định output nào đã được chạy trong sách.

Đồ thị có **cạnh** và mỗi đỉnh có **hành động dự kiến**; hai thứ không đồng nhất. Với `config → deploy` trong ví dụ, đổi `config.input` từ `v2` sang `v3` làm Terraform phải tính lại giá trị mà `deploy.input` tham chiếu. Cạnh cho biết thứ tự khi giá trị cần thiết, nhưng không tự nói `deploy` sẽ update hay replace; hành động phụ thuộc hợp đồng của resource đích. Nếu thêm một `terraform_data.audit` có `input = "kiểm tra"` và `depends_on = [terraform_data.deploy]`, cạnh `deploy → audit` không biến chuỗi chữ “kiểm tra” thành một phép health check, cũng không mặc nhiên khiến audit thay đổi mỗi lần deploy. Muốn có kiểm

chứng, phải dùng một tác vụ ngoài Terraform hoặc cơ chế có hậu điều kiện thật và đọc kết quả của nó.

Khi một dependency ngoài HCL là **sẵn sàng theo thời gian**, thêm cạnh chưa đủ. Ví dụ API tạo policy trả thành công ở thời điểm T nhưng dịch vụ tiêu thụ policy chỉ bắt đầu áp nó sau T+Δ. Terraform có thể đã đi qua cạnh và gọi API kế tiếp; lỗi trong khoảng Δ không chứng minh đồ thị sai, mà chứng minh "đã tạo" khác "đã có hiệu lực". Đặt retry có giới hạn và phép quan sát đúng ở tầng tiêu thụ, thay vì chèn sleep cố định rồi gọi là bảo đảm. Ghi ai sở hữu phép kiểm readiness và khi nó thất bại thì dừng ở đâu; plan chỉ mô tả quan hệ và hành động, không đo sự hội tụ của mọi dịch vụ ngoài.

**Bài tập 29.1 — Khi nào cần depends_on?**

Trong HCL của bài, `deploy.input = terraform_data.config.output` tạo cạnh phụ thuộc nào? Nếu đổi thành `deploy.input = "v2"` nhưng một resource thật vẫn cần một policy được tạo trước mà không đọc thuộc tính của policy, người viết phải khai báo gì? Vì sao cạnh thứ tự đó và một plan xanh vẫn chưa chứng minh policy đã có hiệu lực? Nêu phép kiểm trước khi apply và hậu kiểm sau apply.

**Bài 29.2 — Nhà cung cấp Terraform**

Terraform Core không gọi thẳng mọi API của cloud; provider plugin khai báo schema và thực hiện thao tác đọc, tạo, cập nhật hoặc xóa cho từng resource type. Core phân tích cấu hình, state và đồ thị phụ thuộc rồi yêu cầu provider đọc trạng thái thực và thực thi thay đổi. Vì thế hai resource trông tương tự trong HCL có thể khác hẳn về thuộc tính nào buộc replacement. Không suy hành vi update từ tên trường; xem schema, tài liệu và mã provider đúng phiên bản, rồi kiểm plan trên chính cấu hình và state cần đổi.

`required_providers` khai báo địa chỉ nguồn và ràng buộc phiên bản; `provider` block cấu hình một instance của plugin như Region và cách xác thực. Một provider alias tạo cấu hình khác, có thể trỏ Region hoặc account khác. Một resource không khai báo `provider = ...` sẽ dùng cấu hình mặc định phù hợp; nếu người vận hành tưởng nó dùng alias, plan có thể nhắm sai nơi. Khi đọc một thay đổi AWS, ghi cùng lúc provider source/version, alias, account/Region dự kiến và ID hạ tầng của resource. Credential hợp lệ chỉ chứng minh API có thể được gọi, không chứng minh target account đúng.

Khóa dependency ở `.terraform.lock.hcl` ghi phiên bản và checksum provider đã chọn; ràng buộc phiên bản trong HCL xác định tập phiên bản được phép. Hai thứ cùng tồn tại: constraint như `~> 6.19` không thay cho lockfile, còn lockfile không khóa module từ xa. Review thay đổi lockfile như một thay đổi mã vì phiên bản plugin mới có thể đổi phép đọc, diff, retry hoặc replacement. Kết hợp exact module version/commit và môi trường chạy nếu cần tái lập hành vi plan. Không nhầm cache plugin cục bộ với khóa phiên bản đã kiểm toán.

Một ví dụ chẩn đoán giả định: HCL của `aws_instance.orders` không đổi, nhưng một pipeline dùng provider 6.19.0 còn pipeline khác dùng phiên bản mới hơn vì thiếu lockfile. Hai plan khác nhau chưa đủ kết luận cloud drift. So provider version/checksum, biến đầu vào, workspace, backend,

state serial và account/Region trước; sau đó mới đối chiếu API thực. Nếu mọi đầu vào đã khớp mà diff còn khác, đọc schema/changelog và tạo tái hiện tối thiểu không phát sinh tài nguyên. Chương này không chạy Terraform với AWS thật, nên không khẳng định một phiên bản cụ thể sẽ sinh diff giả định ấy.

Lần theo **sai provider alias** trong một cấu hình giả định. Root module có `aws` mặc định ở Region A và `aws.dr` ở Region B; child module `backup` dự định tạo bản sao tại B. Nếu module không được truyền alias một cách tường minh và tài nguyên trong child dùng provider mặc định, cùng một HCL có thể hướng về A theo cấu hình được kế thừa. Tên module `backup` hoặc biến `region = "B"` riêng của ứng dụng không đổi provider đã gắn với resource. Child module cần khai báo alias mà nó sử dụng, root module truyền đúng mapping `providers`, rồi plan phải được đọc với account/Region và ID API dự kiến của **từng** resource. Aliased configuration không tự chảy vào child như default provider. Đây là lỗi định tuyến quyền và hạ tầng, không thể phát hiện chỉ bằng cách đếm `Plan: 1 to add`.

Trước một apply có chi phí hoặc tác dụng phụ, đóng sổ **danh tính thực thi**: commit cấu hình, provider source/version/checksum, backend/workspace, credentials đã chọn, account/Region sau khi resolve, và các resource address dự kiến. Không in secret hoặc credential vào hồ sơ. Nếu một pipeline chỉ kiểm alias trong mã nhưng runner đã đổi biến môi trường xác thực, target account vẫn có thể khác. Khi không thể xác minh identity bằng phương thức đọc chỉ, dừng trước apply và ghi UNKNOWN; không tạo một object rẻ để "thử xem" trên cloud thật. Một plan trống ở account sai càng không phải bằng chứng account đúng đã hội tụ.

**Bài tập 29.2 — Hai plan khác nhau từ cùng HCL**

Root module có provider aws mặc định ở Region A và alias aws.dr ở Region B. Child module backup dự định dùng B nhưng không nhận mapping providers. Vì sao tên module và biến region không chứng minh đích là B? Cần kiểm gì trước apply?

**Bài 29.3 — Đồ thị phụ thuộc**

Một tệp HCL mô tả điều ta muốn quản lý; state ghi Terraform đang gắn địa chỉ nào với object nào; API của nhà cung cấp trả về trạng thái hạ tầng tại thời điểm được hỏi. Ba thực thể ấy có thể khác nhau. `plan` đối chiếu chúng để đề xuất hành động, không phải bản ghi đã thay đổi thành công. Muốn dùng Terraform an toàn, người vận hành phải đọc cả nguyên nhân và tác dụng phụ của mỗi hành động, nhất là `replace` trên tài nguyên giữ dữ liệu.

`data source` đọc object có sẵn để làm đầu vào, nhưng không nhận quyền sở hữu vòng đời object ấy. Provider là chương trình riêng kết nối API, diễn giải schema và quyết định thuộc tính nào cập nhật tại chỗ, thuộc tính nào buộc thay object. Cấu hình không tự gọi API; Terraform khởi tạo provider rồi lập đồ thị tác vụ. Tham chiếu thuộc tính như `terraform_data.schema.output` tạo cạnh phụ thuộc suy diễn. `depends_on` tạo cạnh tường minh khi phụ thuộc có thật mà không biểu đạt được qua giá trị. Thêm cạnh tùy tiện có thể trì hoãn phép đọc data source và làm plan có nhiều giá trị chưa biết; thứ tự đồ thị không chứng minh dịch vụ đã sẵn sàng cho request.

Ví dụ HCL dưới đây chỉ dùng `terraform_data` của Terraform, không kết nối cloud và không tạo máy, mạng hay lưu trữ. Nó cho phép quan sát riêng quy tắc về đồ thị. `schema.output` là giá trị từ resource thứ nhất; `triggers_replace` khiến resource `service` được thay khi giá trị đó đổi. Cạnh `schema` tới `service` được suy ra từ tham chiếu. Cạnh `service` tới `audit` là chủ ý mô phỏng một bước kiểm tra phải chờ bước dịch vụ, nên khai báo tường minh. `audit` chỉ lưu chuỗi `ready`, không chạy health check thật; đừng diễn giải tên ấy thành bằng chứng readiness.

```
terraform {
  required_version = ">= 1.5, < 1.17"
}

variable "schema_revision" {
  type    = string
  default = "v1"
}

resource "terraform_data" "schema" {
  input = var.schema_revision
}

resource "terraform_data" "service" {
  triggers_replace = terraform_data.schema.output
}

resource "terraform_data" "audit" {
  input      = "ready"
  depends_on = [terraform_data.service]
}

output "service_id" {
  value = terraform_data.service.id
}
```

Với Terraform CLI tương thích, sao chép ví dụ vào một thư mục tạm riêng rồi chạy `terraform fmt -check`, `terraform init -backend=false`, `terraform validate` và `terraform plan`. Chỉ `apply` nếu muốn thử state cục bộ của ba `terraform_data` này. Đổi `schema_revision` từ `v1` sang `v2` rồi đọc plan: `schema` đổi `input`, `service` có thể phải replace do `triggers_replace`; `audit` chỉ có cạnh thứ tự. Giá trị ID có thể chưa biết trước apply. Đây là dự đoán từ cấu hình và hợp đồng resource, không phải transcript chạy thực tế; kết quả kiểm tra trên máy dựng được ghi riêng ngoài sách.

Một **data source** thêm ranh giới thời gian vào đồ thị. Nếu mọi đối số đã biết, Terraform có thể đọc object ở pha lập plan; nếu giá trị đầu vào còn phụ thuộc resource sẽ thay trong lần apply, phép đọc có thể bị hoãn và thuộc tính đầu ra hiện là `known after apply`. Thêm `depends_on` rộng vào data source cũng có thể đẩy phép đọc sang apply mà không làm API cuối sẵn sàng hơn. Khi output của data source quyết định ID tài nguyên quan trọng, một plan chứa giá trị chưa biết là một giới hạn review: người duyệt chưa thấy đích cuối, phải giải thích vì sao hành động vẫn an toàn hoặc tách thay đổi thành hai lượt có điểm kiểm giữa chúng. Không lấp unknown bằng ID đoán từ môi trường khác.

Khi pipeline lưu plan bằng `-out`, file plan giữ cấu hình, giá trị đầu vào và có thể giữ **bí mật ở dạng đọc được** dù giao diện terminal che chúng. Xử lý nó như artifact nhạy cảm: giới hạn người đọc, thời

hạn giữ, nơi tải lên và việc dùng lại; chỉ apply đúng plan đã được xem trong đúng backend/workspace đã khóa, rồi kiểm trạng thái API và hậu điều kiện ứng dụng. Saved plan tránh việc pipeline tự tính lại một tập hành động khác sau review, nhưng không làm API thành giao dịch và không tự ngăn drift ngoài Terraform trong khoảng chờ. Nếu state đã đổi vì một lượt apply khác, plan có thể bị coi là cũ; dù apply được, vẫn phải đọc kết quả thật và plan tiếp theo. Không mang plan sang account khác như một "bản lệnh" độc lập với backend và credentials.

**Bài tập 29.3 — Cạnh suy diễn và cạnh khai báo**

Trong HCL mẫu, cạnh schema→service và service→audit khác nhau thế nào? Một data source bị hoãn đọc tới apply khiến ID đích hiện known after apply; saved plan chứa secret dù terminal che giá trị. Có thể coi plan xanh là đủ để duyệt không?

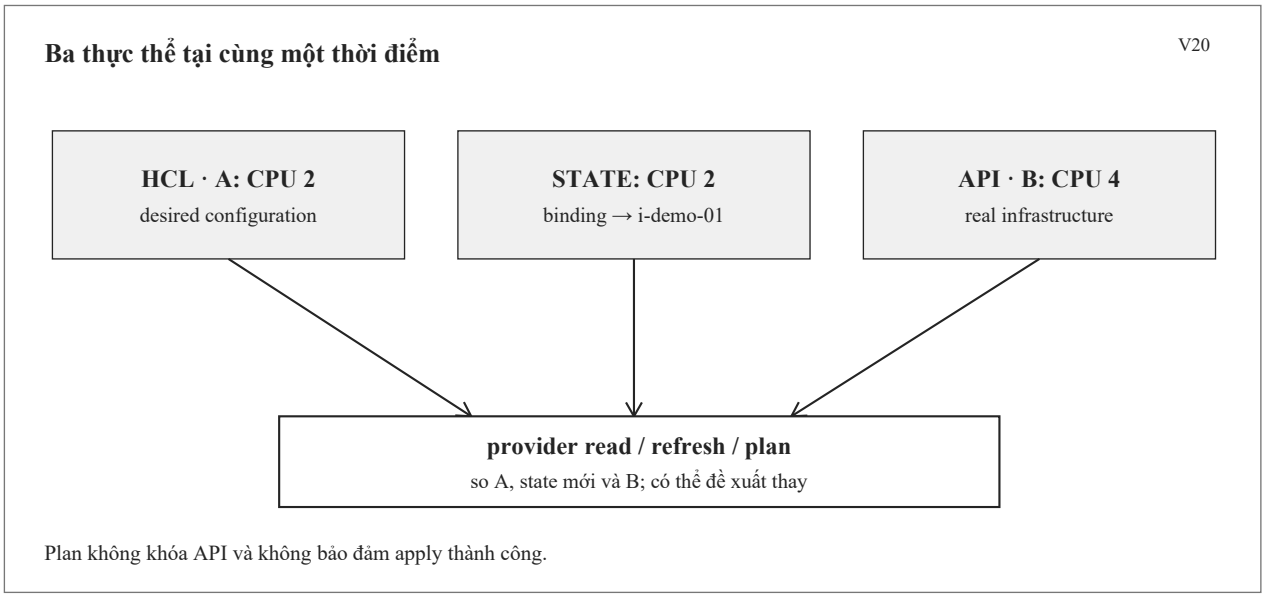
# Chương 30 — Kế hoạch thay đổi, trạng thái và kho lưu trữ trạng thái

Giải thích tại sao một plan có thể thất bại giữa chừng và state nhạy cảm.

## Bài 30.1 — Ba trạng thái

State ánh xạ địa chỉ `resource` trong cấu hình tới ID object thực và giữ metadata cần thiết để cập nhật hoặc xóa. Nó không phải bản sao đầy đủ của hạ tầng, cũng không phải cấu hình mong muốn. Trong plan thông thường, Terraform đọc state và hỏi API để làm mới góc nhìn về object đang quản lý, sau đó so với cấu hình. Bởi thế, sửa object ngoài Terraform tạo drift có thể xuất hiện khi refresh; thuộc tính không được provider đọc hoặc API che giấu có thể không hiện.

Giả sử cấu hình muốn instance loại `t3.small`, state ghi ID `i-example`, còn console đã đổi loại instance thành `t3.medium`. Nếu provider đọc lại thuộc tính ấy, plan có thể đề xuất kéo thực tế về cấu hình; nó không có nghĩa state "là" hạ tầng thực. Trước khi apply, xác nhận ai đổi, vì sao đổi, tài nguyên có dữ liệu gì, và action cụ thể trong plan là update hay replace. Nếu thay đổi ngoài hệ thống là chủ ý, sửa cấu hình hoặc quy trình import/state phù hợp rồi lập plan lại; chỉnh state bằng tay để ép plan đẹp sẽ che giấu nguyên nhân. Không chạy apply từ một plan cũ sau khi người khác đã thay đổi hạ tầng hoặc state.




Hình IX.2.1 — Terraform: cấu hình, state và API thực có thể lệch nhau; CPU là dữ kiện giả định.

Hãy đọc plan như một phép đối chiếu ba mốc, không như bản ghi sự thật duy nhất. Cấu hình ghi địa chỉ `aws_instance.api` và loại `t3.small`; state giữ ID gắn với địa chỉ đó cùng các thuộc tính provider đã lưu; API hiện trả loại `t3.medium`. Nếu phần loại máy được provider quan sát và cấu hình vẫn là `t3.small`, plan có thể đề xuất sửa thuộc tính về giá trị mong muốn. Nhưng quyết định update tại chỗ hay replace phụ thuộc schema của resource, các cờ lifecycle và thuộc tính thay đổi. Không được suy ra action chỉ từ chữ "drift"; đọc ký hiệu action và trường gây thay đổi trong plan cụ thể trước khi duyệt.

Một lần đổi tên địa chỉ trong cấu hình là biến thể nguy hiểm. Nếu `aws_instance.api` thành `aws_instance.frontend` mà state vẫn gắn object với địa chỉ cũ, Terraform có thể xem đây là một object bị bỏ và một object mới cần tạo. ID hạ tầng có thể không đổi, nhưng quan hệ địa chỉ

trong state đã không còn khớp. Trước khi apply, phân biệt "đổi tên đối tượng quản lý" với "thay đối tượng thực": dùng cơ chế khai báo di chuyển địa chỉ hoặc thao tác state được kiểm soát, rồi lập plan mới và xác nhận không có destroy/create ngoài ý muốn. Không sửa state bằng trình soạn thảo văn bản; state có quan hệ phụ thuộc và dữ liệu nhạy cảm, một bản sửa thiếu đồng bộ dễ làm lần apply sau hỏng thêm.

Quy trình điều tra an toàn cho tình huống giả định là ghi commit cấu hình, workspace/backend, phiên bản provider, state serial và ID remote trước khi thay đổi; tạo plan mới sau khi đồng bộ với người đã thao tác ngoài Terraform. So sánh action theo từng địa chỉ, đánh dấu resource giữ dữ liệu và thuộc tính buộc replacement. Nếu plan muốn replace một instance có EBS chứa dữ liệu, dừng ở plan để xác nhận backup có thể restore, chiến lược giữ volume và cửa sổ gián đoạn; không dùng `-auto-approve` để bỏ qua câu hỏi đó. Sau apply, kiểm API và ứng dụng thật, rồi lập plan lại để tìm sai lệch còn tồn tại. Một plan sạch ngay sau apply chỉ nói cấu hình và những thuộc tính provider đọc được đang đồng thuận ở thời điểm ấy; nó không chứng minh dữ liệu hoặc dịch vụ đã phục hồi.

### Bài 30.2 — Lập kế hoạch và áp dụng

Plan là dự báo dựa trên cấu hình, state, dữ liệu provider trả về và quyền tại thời điểm lập. Từ plan đến apply, người khác hoặc dịch vụ khác có thể đổi object; API có thể từ chối, hết quota, timeout hoặc trả kết quả không chắc chắn. Terraform thực hiện các tác vụ theo dependency graph, thường có song song giữa các nhánh độc lập, chứ không mở một giao dịch chung trên các API. Nếu ba resource đã tạo mà resource thứ tư thất bại, ba resource đầu có thể còn nguyên; phải đọc kết quả apply, state và API thực rồi lập kế hoạch tiếp. Exit code lỗi không đồng nghĩa mọi hành động đã rollback. Saved plan giúp ràng buộc lựa chọn đã review, nhưng vẫn không đóng băng hạ tầng ngoài Terraform hoặc bảo đảm API sẽ thành công.

Ví dụ giả định có hai tài nguyên A (mạng) và B (máy), trong đó B tham chiếu ID của A nên đồ thị buộc tạo A trước B. Apply tạo A và ghi nhận vào state, sau đó API tạo B từ chối vì quota: lệnh thoát lỗi, nhưng A vẫn tồn tại và state có thể đã lưu A. Phép kiểm tiếp theo là đối chiếu từng địa chỉ resource trong state với object thực và nguyên nhân B thất bại; sau khi xử lý quota, lập plan mới để thấy Terraform còn định làm gì, rồi mới quyết định apply tiếp hay xóa A theo quy trình. Nếu API B **đã** tạo object nhưng phản hồi về provider bị mất, trạng thái B có thể không chắc chắn; phải tìm object theo định danh/nhãn đã thiết kế và đối chiếu state trước khi thử lại, tránh tạo trùng hoặc import nhầm object. Không thể suy trường hợp nào chỉ từ dòng `Error` hoặc từ exit code 1. Kịch bản này mô tả các nhánh có thể xảy ra, không phải output Terraform đã chạy.

`lifecycle` điều chỉnh vòng đời, không thay phép phân tích tác động. `create_before_destroy` đổi thứ tự replacement khi API và tên/định mức cho phép; nó không tự chuyển dữ liệu. `prevent_destroy` chặn plan phá object khi rule còn trong cấu hình, nhưng không bảo vệ object trước thao tác ngoài Terraform hay khi resource block bị loại bỏ. `ignore_changes` hợp lý khi một thuộc tính được giao cho bộ điều khiển khác, song nó có thể giấu drift có ý nghĩa; phải ghi rõ chủ sở hữu thuộc tính và đường giám sát. `destroy` là yêu cầu xóa các resource còn được quản lý trong state, không phải thao tác dọn thử nghiệm vô hại.

Data source có thể được đọc lúc plan hoặc bị hoãn tới apply nếu đầu vào chưa biết; không nên hiểu mọi giá trị hiện trong plan là một snapshot nguyên tử của toàn hệ thống. `-refresh=false` tránh phép đọc trong lượt đó nhưng có thể làm plan dựa trên state cũ; `refresh-only` cập nhật state theo thực tế mà không kéo thực tế về cấu hình, nên vẫn phải review trước apply.

### Bài 30.3 — Kho lưu trữ trạng thái

State cục bộ thường là tệp JSON; state từ backend dùng chung cho phép nhiều người và CI cùng làm việc trên một bộ ánh xạ. Remote backend phải được chọn theo khả năng kiểm soát truy cập, mã hóa, lưu phiên bản, khôi phục, tính sẵn sàng và cơ chế khóa. Backend nào có locking thì Terraform có thể khóa state trước thao tác ghi để ngăn hai lượt Terraform cùng sửa một state; khóa không ngăn người sửa trực tiếp trên console, không khóa tất cả API và không phải backup. Không tắt khóa hoặc force-unlock khi chưa biết chủ khóa và tình trạng lượt chạy. Nếu state remote ghi thất bại sau khi API đã nhận thay đổi, phải bảo toàn bản state Terraform giữ lại để phục hồi và đối chiếu, thay vì chạy apply lần hai một cách mù quáng.

State và tệp plan có thể chứa secret, kể cả giá trị được đánh dấu `sensitive`; cờ này chủ yếu che hiển thị trong output chứ không làm state biến thành kho bí mật. Tệp cấu hình backend cũng có thể để lộ credential nếu nhúng trực tiếp. Giới hạn quyền đọc/ghi state, dùng cơ chế cấp credential ngắn hạn và lưu phiên bản/backup có thử restore. Một bản sao state hỏng hoặc cũ có thể khiến Terraform tưởng object chưa tồn tại, đề xuất tạo trùng hoặc phá nhầm; khi phục hồi cần xác định lineage, serial, thời điểm và object ID, kiểm tra khóa, lưu bản hiện tại an toàn rồi mới khôi phục bản đúng. Không chỉnh JSON state bằng tay khi còn công cụ và quy trình chính thức.

`import` nối một object đã tồn tại với một địa chỉ resource, không tự tạo thiết kế HCL đúng và không tự bảo vệ dữ liệu. Với import block, khai báo địa chỉ và ID, bảo đảm cấu hình resource khớp object, review plan và kiểm tra chỉ có một địa chỉ quản lý mỗi object trước apply. Nếu object vẫn thuộc một state khác, import vào state mới sẽ tạo hai chủ quản lý cạnh tranh. `state rm` chỉ bỏ ánh xạ quản lý, không xóa object thực; dùng nó như cách "sửa" một plan nguy hiểm mà không tìm nguyên nhân sẽ để lại hạ tầng mồ côi. Mỗi thao tác phục hồi hay chuyển state cần ghi quyền sở hữu và hậu điều kiện ở cả state lẫn API.

#### Bài tập 30.1 — Apply dở dang và lỗi ghi state

Một apply tạo thành công network và volume nhưng API tạo instance timeout; đồng thời remote backend báo lỗi ghi state. Vì sao không chạy lại ngay? Hãy nêu thứ tự đối chiếu và hậu điều kiện trước lượt tiếp theo.

# Chương 31 — Sai lệch hạ tầng, nhập tài nguyên và tái cấu trúc

Phân biệt thay đổi ngoài Terraform với đổi địa chỉ cấu hình.

## Bài 31.1 — Phát hiện sai lệch

Giả sử Region `us-east-1`, Terraform CLI 1.16.0 và AWS provider 6.19.0 quản lý `aws_instance.orders` có `user_data` cấu hình là phiên A, `user_data_replace_on_change = true`, và root EBS `delete_on_termination = true`. Ứng dụng còn giữ 41 đơn chưa được sao lưu trên root volume; đó là thiết kế nguy hiểm có chủ ý để phân tích, không phải mẫu nên triển khai. State lưu instance ID `i-demo-01` và user data A. Người vận hành dừng instance, sửa user data thành B bằng console rồi khởi động lại; cấu hình HCL vẫn A. Các ID và số liệu là giả định, không phải output Terraform/AWS đã chạy. Nếu provider không phát hiện thuộc tính ấy, hoặc object đã đổi cách khác, kết luận về plan phải theo kết quả thực, không áp dụng máy móc.

Trong mô hình provider đã nêu, nếu refresh đọc được B và plan hiển thị thay đổi `user_data` yêu cầu replacement, apply có thể terminate instance cũ.

Nếu B đúng và cần quản lý lâu dài, sửa HCL cho khớp B qua review, refresh/plan lại và chỉ apply khi không còn replacement ngoài chủ đích. Nếu B sai nhưng dữ liệu cần giữ, thiết kế đường chuyển dữ liệu có snapshot nhất quán và restore thử, hoặc tách dữ liệu khỏi root volume trước khi tính tới thay instance. Nếu state trỏ sai ID vì một lượt apply trước bị gián đoạn, đối chiếu API và lịch sử state trước khi cân nhắc import/khôi phục; không nhập một object đang được quản lý ở địa chỉ khác. Nếu vẫn phải replace, ghi kế hoạch dừng ghi, chuyển lưu lượng, backup, restore, kiểm tra đơn 41/41 và phương án rollback ứng dụng lẫn dữ liệu; rollback mã không tự trả lại EBS đã bị xóa. Sau apply, so instance ID, root volume ID/chính sách xóa, trạng thái ứng dụng, dữ liệu đơn, request đại diện và plan mới; plan trống riêng nó chưa chứng minh nghiệp vụ đúng.

## Bài 31.2 — Ánh xạ

State nối một địa chỉ Terraform với ID object ngoài đời. `import` dùng khi object đã tồn tại nhưng chưa được Terraform ở workspace này quản lý; nó không tạo object và không tự sinh cấu hình đúng ý định. `moved` diễn đạt đổi địa chỉ của object đã được quản lý khi refactor HCL. `terraform state mv` sửa ánh xạ state một lần bằng CLI; nó hữu ích cho thao tác đặc biệt nhưng dễ để code và state khác lịch sử nếu không có bản ghi thay đổi trong cấu hình. Ba công cụ giải ba tình huống khác nhau, không phải ba cách gọi của cùng một phép.

Giả định bucket `orders-archive-example` đã tồn tại ngoài Terraform. Trước import, xác nhận account, Region, tên/ID và ai đang quản lý bucket; sao lưu state hiện tại, giữ lock của backend và đối chiếu policy chặn destroy. Viết `resource` đúng địa chỉ dự định, thêm import block trỏ ID tương ứng, rồi đọc plan. Nếu plan vừa import vừa muốn sửa policy hoặc xóa versioning, đó là cảnh báo cấu hình chưa khớp trạng thái mong muốn; điều chỉnh HCL hoặc quyết định rõ thay đổi cần thiết trước apply. Sau import, đối chiếu state address với API object và chạy plan mới. Không import cùng object vào hai địa chỉ quản lý mà không chủ đích.

Nếu resource `aws_instance.orders` đang được quản lý nhưng đổi tên logic thành `aws_instance.api`, dùng `moved { from = aws_instance.orders to = aws_instance.api }` trong cấu hình. Plan phải thể hiện chuyển địa chỉ mà không định terminate instance chỉ vì đổi tên. Nếu plan vẫn đề xuất replace vì thuộc tính provider thay đổi, `moved` không chữa nguyên nhân ấy. Khi dùng `state mv` trong một sự cố, đóng băng các actor apply khác, sao lưu/version state, ghi lệnh và thời điểm, cập nhật HCL cùng review, rồi plan lại. Lock chỉ tuần tự hóa các actor dùng đúng backend; nó không khóa console hoặc workspace khác.

Địa chỉ là một phần của hợp đồng: module path, resource name và instance key đều phải chính xác. Thêm `count` hay `for_each` biến một địa chỉ đơn thành địa chỉ có index/key. Trước mỗi phép chuyển, viết bảng ánh xạ cũ → mới và ID ngoài đời, kiểm nó một-một; thiếu một dòng có thể khiến Terraform hiểu object cũ bị bỏ và object mới cần tạo. Một plan "0 to destroy" là điều kiện cần cho refactor giữ object, nhưng chưa chứng minh ứng dụng còn phục vụ đúng.

**Bài tập 31.1 — Chọn import hay moved**

Bucket có sẵn chưa ở state; một instance đã ở state chỉ đổi tên HCL. Dùng import hay moved cho từng việc?

**Bài 31.3 — Tái cấu trúc địa chỉ tài nguyên**

Refactor Terraform là đổi cấu trúc mã mà vẫn giữ quan hệ giữa địa chỉ trong state và object thực. Việc đổi `resource "aws_instance" "orders"` thành `module.orders.aws_instance.this` chỉ là đổi tên trong mắt người đọc; nếu không diễn đạt phép move, Terraform có thể dự báo destroy địa chỉ cũ rồi create địa chỉ mới. Với object chứa dữ liệu, khác biệt giữa "đổi địa chỉ" và "thay object" quyết định rủi ro mất dữ liệu. `moved` block trong module lưu lịch sử ánh xạ để các workspace chưa nâng cấp cũng có thể đi qua refactor.

`count` đánh số instance bằng chỉ số nguyên, nên xóa phần tử giữa một list có thể làm các chỉ số còn lại đổi nghĩa. `for_each` dùng key ổn định hơn nếu key là danh tính nghiệp vụ bền. Nhưng đổi từ `count` sang `for_each` không tự ánh xạ `[0]` sang `["api-a"]`; viết từng `moved` block tương ứng và kiểm ID thật. Giả định `[0]` đang là `i-demo-a`, `[1]` là `i-demo-b`: ánh xạ `[0] → ["a"]`, `[1] → ["b"]` phải được chứng minh bằng tags/API và state, không suy theo thứ tự hiển thị ở console. Nếu key bị tái sử dụng cho object khác, plan "không destroy" cũng có thể che sai ý nghĩa quản lý.

Đưa resource vào module còn thêm module path. Chạy plan ở mỗi workspace có phiên bản state khác nhau; một workspace có thể đang ở địa chỉ cũ, workspace khác đã nâng cấp. Không xóa `moved` block ngay sau khi một môi trường apply xanh, vì môi trường trễ sẽ mất đường nâng cấp. Khi module từ xa đổi version, khóa phiên bản module phù hợp và review cả output, provider alias, dependency ngoài HCL; move chỉ giải quyết địa chỉ, không bảo đảm hành vi của resource không đổi.

Preflight cho refactor gồm snapshot state có thể phục hồi, inventory workspace, bảng address–ID cũ/mới, kiểm quyền provider và plan rõ từng action. Stop nếu có delete/replace ngoài ý định hoặc ID không khớp. Sau apply, so ID, cấu hình API và request nghiệp vụ, rồi plan lại để thấy drift còn không. Không dùng `ignore_changes` để giấu một diff chưa hiểu; nó có thể che hỏng cấu hình

quan trọng và đẩy rủi ro sang lần thay đổi sau.

Một phép thử refactor không cần tài nguyên cloud có thể dùng `terraform_data` trong thư mục tạm. Giả sử state hiện có địa chỉ `terraform_data.service`, giá trị `input = "blue"`, còn cấu hình mới chuyển cùng resource vào module ở địa chỉ `module.service.terraform_data.this`. Khối sau diễn đạt *đổi địa chỉ quản lý*, không tự đổi object hay nội dung `input`:

```
moved {
  from = terraform_data.service
  to   = module.service.terraform_data.this
}
```

Trước khi áp dụng, đọc plan để xác nhận địa chỉ cũ được ánh xạ tới địa chỉ mới và không có destroy/create ngoài ý định; cũng kiểm cấu hình resource ở module vẫn giữ `input = "blue"`. Nếu workspace A đã có object ở địa chỉ cũ nhưng workspace B chưa từng tạo nó, cùng một `moved` block không có nghĩa hai workspace sẽ có cùng số object sau plan: A có phép chuyển trong state, B có thể cần create theo cấu hình. Nếu thuộc tính mới còn thay đổi và kích hoạt replacement, `moved` chỉ giữ liên kết định danh trước khi Terraform tính diff, không chặn replacement. Hậu kiểm phải so ID state và kết quả đọc từ provider, không chỉ thấy thông báo “moved” rồi dừng.

**Bài tập 31.2 — Đổi địa chỉ tài nguyên đang giữ dữ liệu**

Đổi count [0]/[1] thành for_each [a]/[b] cho hai EC2 giữ dữ liệu. Nêu điều kiện trước apply và lúc nào dừng.

# Chương 32 — Thay đổi hạ tầng đang chứa dữ liệu

Quyết định dừng khi plan đề xuất replace instance chứa dữ liệu.

## Bài 32.1 — Thay thế tài nguyên

Đừng áp ngay chỉ vì plan là "đưa về cấu hình". Trước tiên đóng băng lượt triển khai, xác định workspace/backend/state serial, phiên provider, đúng account/Region/instance ID và trạng thái apply khác. Đọc plan có refresh để biết thuộc tính nào đổi, replacement do schema provider hay lifecycle nào, object nào sẽ bị destroy. Đối chiếu thuộc tính thực qua API và lịch sử thay đổi; hỏi người sửa B vì có thể B là bản sửa lỗi cần giữ. Kiểm tra root block mapping thực tế, vị trí 41 đơn, cơ chế ghi đang hoạt động, snapshot/backup và phép restore độc lập trước khi quyết định. `prevent_destroy` có thể là rào chắn bổ sung khi block còn hiện diện; không lấy nó làm bằng chứng dữ liệu đã an toàn. Không dùng `-refresh=false`, `-target` hoặc `ignore_changes` để che nguyên nhân rồi áp plan.

Tình huống chưa được chạy trên AWS và không có chi phí cloud phát sinh để chứng minh. Thuộc tính `user_data` theo provider này mặc định dẫn tới stop/start khi đổi; chính lựa chọn `user_data_replace_on_change = true` mới làm thay instance. Bỏ qua chi tiết ấy sẽ dạy sai nguyên nhân. Ngay cả khi replacement tạo máy mới trước, root volume cũ vẫn phải được bảo vệ và dữ liệu phải được chuyển/kiểm tra riêng.

Nếu instance cũ bị terminate, root EBS với cờ xóa theo termination có nguy cơ mất 41 đơn.

Từ plan tới quyết định cần một bảng trạng thái ngắn do người vận hành tự ghi, không cần thêm tài nguyên AWS để thử. Trước apply: instance ID nào đang nhận request, root volume ID nào chứa 41 đơn chưa đối soát, `DeleteOnTermination` của mapping là gì, snapshot/backup nào đã được restore thử, và plan định `update`, `stop/start` hay `replace` resource nào. Sau một replace giả định: instance mới có ID khác, nhưng 41 đơn có ở đâu và ứng dụng có đọc đúng không? Nếu câu sau chưa có bằng chứng, không gọi replace là an toàn chỉ vì máy mới Ready. Root volume có thể đi theo vòng đời instance; tạo máy mới trước không tự chuyển dữ liệu từ volume cũ.

Phân biệt ba phản ứng với thay đổi ngoài Terraform. Nếu B sửa khẩn để chữa lỗi và thay đổi đó phải trở thành trạng thái mong muốn, đưa nó vào configuration sau review rồi lập plan mới. Nếu B sửa nhầm, plan đưa object về cấu hình cũ có thể phù hợp, nhưng vẫn cần phân tích tác dụng phụ và cổng dữ liệu. Nếu state gắn sai object hoặc địa chỉ, sửa quan hệ quản lý bằng cơ chế import/moved/state phù hợp, có khóa và backup state; không dùng `ignore_changes` để che một object mà đội không còn hiểu. Trong cả ba trường hợp, action trong plan là dữ kiện để ra quyết định, không phải mệnh lệnh tự động.

Hậu kiểm cho phương án không replace phải ghi API attribute, instance đang phục vụ và phép đọc/ghi đơn thử, rồi lập plan mới để tìm drift còn lại. Hậu kiểm cho phương án replace còn phải chứng minh dữ liệu cũ được bảo toàn hoặc phục hồi, đường request chuyển sang máy mới và máy cũ được xử lý đúng theo chính sách giữ dữ liệu. 41 đơn là dữ kiện giả định của ca học, không phải số đo AWS thực. Nếu thiếu backup có thể restore hoặc thiếu thẩm quyền thay đổi, dừng ở plan và ghi rõ điều chưa biết thay vì dùng một apply làm thí nghiệm.

**Bài tập 32.1 — Replacement ngoài dự kiến**

Plan với refresh báo `aws_instance.orders` cần replace sau khi user data bị sửa ngoài Terraform. Hãy phân biệt cấu hình, state và thực tế; xác định điều kiện có thể mất 41 đơn, ba bằng chứng phải thu trước apply và hai hướng xử lý tùy B đúng hay sai.

**Bài 32.2 — Điều phối**

Một backend có khóa state giúp các lượt Terraform cùng sử dụng backend đó tránh ghi state đồng thời. Nó không khóa API của cloud và không ngăn người có quyền sửa security group trên console. Nếu một người sửa ngoài Terraform sau lúc plan được tạo, hạ tầng thực đã khác điều kiện đầu vào của plan. Dùng saved plan mà không kiểm lại phạm vi, state và thay đổi bên ngoài có thể áp một hành động khác điều người duyệt đã hình dung.

Trong tình huống hai pipeline cùng hướng đến một workspace, cần biết chúng có thật sự dùng cùng backend, cùng khóa và cùng state hay không. Nếu có, một lượt có thể phải đợi lượt kia; đó chỉ là bảo vệ tuần tự cho state. Sau khi khóa được nhả, lượt tiếp theo vẫn cần đọc trạng thái mới và đánh giá lại plan. Sửa đổi trực tiếp ở console là một actor thứ ba ngoài khóa ấy. Hãy ghi rõ object bị sửa, account/Region, thời điểm và lý do, rồi đối chiếu configuration, state và API thực trước khi quyết định nhập thay đổi vào cấu hình hay hoàn nguyên drift.

State có thể chứa giá trị nhạy cảm. `sensitive = true` chủ yếu hạn chế hiển thị ở một số đầu ra, không thay kiểm soát truy cập vào state. Versioning giúp tìm lại phiên bản cũ nhưng không tự mã hóa, giới hạn người đọc hoặc bảo đảm bản khôi phục đúng. Cần thiết kế quyền, bảo vệ lưu trữ, khóa và phương án phục hồi như các cơ chế riêng.

Hãy xét lịch giả định: pipeline A lấy lock của workspace lúc 10:00, pipeline B đợi; A apply xong và nhả lock lúc 10:05. B không được dùng kết luận từ plan đã duyệt lúc 09:55 mà bỏ qua trạng thái mới. Saved plan được lập theo configuration và state tại thời điểm lập plan; sau khi state đổi, bản duyệt cũ không còn là căn cứ vận hành đủ tin cậy. Người C sửa object trực tiếp qua console lúc 10:06 mà không ghi vào state: state serial có thể chưa đổi, trong khi remote object đã khác. Lượt B cần refresh/plan mới và đọc action trước khi apply, nhất là với object có dữ liệu. Không suy "lock đã chạy" thành "không có drift".

Trường hợp khác nguy hiểm hơn là A và B tưởng cùng quản lý môi trường nhưng trỏ hai backend hoặc workspace khác nhau. Mỗi bên có thể giữ một lock hợp lệ của riêng mình, rồi cùng ghi lên một remote object. Kiểm backend URL, workspace, state lineage/serial và resource ID trong hồ sơ thay đổi; không chỉ so tên nhánh Git. Nếu phát hiện hai state cùng nhận cùng object, dừng cả hai pipeline, chọn một chủ quản và lập kế hoạch chuyển giao có backup state, thay vì để hai bộ điều khiển tranh nhau.

Phép khôi phục state cần được tập dượt như khôi phục dữ liệu: biết phiên bản state nào tương ứng thời điểm trước lỗi, ai có quyền đọc/ghi, khóa có đang được giữ không, và remote object đã đổi gì kể từ snapshot. Đưa một bản state cũ lên không đảo hạ tầng thực; nó có thể làm Terraform tưởng object còn ở trạng thái cũ rồi đề xuất thay đổi sai. Trong môi trường thử không có cloud, có thể dùng provider cục bộ và hai workspace độc lập để quan sát lock/serial, nhưng kết quả ấy chỉ minh họa cơ

chế state; không chứng minh API AWS được khóa. Bảo vệ cuối là plan mới, review hành động và hậu kiểm resource thật.

**Bài tập 32.2 — State lock và sửa ngoài Terraform**

Hai pipeline cùng dùng một backend có locking; một người sửa security group trực tiếp trên console. Khóa bảo vệ được gì, drift nào nó không ngăn, và vì sao `sensitive = true` hoặc state versioning riêng lẻ chưa đủ bảo vệ dữ liệu?

**Bài tập 32.3 — Khóa state có ngăn sửa qua console?**

State lock có ngăn sửa security group qua console không?

### Bài 32.3 — Hậu kiểm

Apply thành công chỉ cho biết Terraform hoàn tất các hành động mà nó thực hiện, không chứng minh request người dùng hoặc dữ liệu đã phục hồi. Trong sự cố `orders` giả định, trước apply phải ghi instance ID `i-demo-01`, root volume ID, cờ DeleteOnTermination, 41 đơn chưa sao lưu và đường request qua ALB. Nếu plan yêu cầu replace, dừng để đưa dữ liệu ra khỏi root hoặc tạo bản sao nhất quán có thử restore. Sau apply, ghi instance ID mới, volume ID mới, profile, target health, số đơn đọc được và request `/reports` thực tế. Mỗi phép quan sát gắn với một hậu điều kiện; `terraform plan` trống không thay thế phép đọc 41/41 đơn.

Nếu rollout hỏng, tách hai loại phục hồi. Phục hồi mã hoặc cấu hình có thể là chuyển về artifact/plan trước sau khi kiểm tác động; phục hồi dữ liệu cần bản sao đúng thời điểm và quy trình replay hoặc reconcile các ghi phát sinh. Một state snapshot cũ là bản ghi ánh xạ object, không chứa đủ dữ liệu root EBS và không phải bản backup ứng dụng. Rollback Terraform bằng cách checkout HCL cũ rồi apply có thể lại tạo replacement khác. Đối chiếu state, API và dữ liệu thật trước khi chọn hành động; không tự chạy destroy để "quay lại".

Một apply bị ngắt giữa chừng có thể đã tạo resource mới nhưng chưa cập nhật mọi địa chỉ state, hoặc đã xóa object cũ trong khi dependency chưa xong. Đọc log, state version/serial và API inventory dưới cùng account/Region. Nếu một object ngoài state đang tồn tại, điều tra ownership rồi dùng import hoặc cleanup có phê duyệt, không vội chạy apply lần hai khi chưa biết tác dụng phụ. Cần ghi ID và thời điểm từng hành động để phân biệt trạng thái từng bước với mong muốn ban đầu.

Kết thúc sự cố bằng bằng chứng độc lập: request qua đường người dùng, lượng dữ liệu sau phục hồi, backup có thể restore, plan mới không còn thay đổi bất ngờ và alert không báo lỗi kéo dài. Nếu chỉ có `apply complete` và target `/health` xanh, kết luận đúng là hạ tầng đã tiến thêm một bước và probe hẹp đã qua; không được kết luận toàn bộ giao dịch đơn hàng an toàn. Các ID và số liệu trong ví dụ là giả định phục vụ suy luận, không phải log AWS/Terraform thật.

**Bài tập 32.4 — Apply xanh nhưng mất đơn hàng**

Terraform apply complete và target /health xanh sau replace. Có thể đóng sự cố mất 41 đơn chưa?

# PHẦN X — Ansible và hội tụ cấu hình

## Làm sao áp trạng thái mong muốn lên các máy đang tồn tại mà không gây restart thừa?

Inventory chọn host, module quan sát rồi đổi trạng thái, handler hoãn tác động; kết quả theo từng host không phải giao dịch.

X

**LỘ TRÌNH PHẦN**


- Chương 33 — Danh sách máy và đường điều khiển Ansible
- Chương 34 — Hội tụ cấu hình, handler và lỗi từng máy
- Chương 35 — Vai trò, bí mật và triển khai theo đợt

# Chương 33 — Danh sách máy và đường điều khiển Ansible

Chỉ ra ai kết nối tới ai, bằng quyền nào và biến nào được dùng.

## Bài 33.1 — Sơ đồ kết nối Ansible

Terraform ở chương trước quản lý ánh xạ giữa resource và object hạ tầng bằng state. Khi máy đã tồn tại, Ansible thường làm việc ở lớp khác: nối tới máy, kiểm tra file, package hoặc service hiện có rồi yêu cầu chúng đạt cấu hình mong muốn. Không có một state tập trung tương đương Terraform ghi toàn bộ máy đã như thế nào; mỗi module tự quan sát trạng thái liên quan ở lúc chạy và báo `ok`, `changed` hoặc `failed`. Vì vậy chạy lại playbook là một phép thử quan trọng, nhưng chỉ thấy `changed=0` chưa chứng minh ứng dụng phục vụ đúng.

Inventory liệt kê managed node và nhóm, đồng thời có thể gắn biến như địa chỉ, người dùng hoặc cách kết nối. Một play chọn nhóm trong inventory, đặt biến và tập task; playbook chứa một hoặc nhiều play. Control node đọc playbook, nạp module và plugin, rồi kết nối tới managed node. Với máy Linux, kết nối thường qua SSH; `ansible_connection=local` là lựa chọn khác khi chạy trên chính máy điều khiển. Kết nối thành công chỉ chứng minh đường điều khiển đã mở, không chứng minh ứng dụng đã sẵn sàng. Module là đơn vị thực hiện hành động cụ thể như `file`, `template` hoặc `systemd_service`; task gọi module cùng tham số và điều kiện. Module chạy với danh tính của connection user, trừ khi task hoặc play dùng `become` để nâng quyền.

Ansible và Terraform có thể cùng có mặt: Terraform tạo instance, network và volume; Ansible cấu hình ứng dụng trên instance được chọn. Cần xác định một chủ quản lý cho từng thuộc tính. Nếu Terraform viết cùng file qua user data còn Ansible cũng ghi file ấy, hai bộ điều khiển có thể thay nhau tạo drift. Inventory có thể lấy từ hạ tầng nhưng tên host trong inventory không phải ID object trong Terraform state; cần kiểm tra đích thật trước khi chạy. Không dùng một kết quả `ping` của Ansible làm bằng chứng dịch vụ HTTP hay cơ sở dữ liệu hoạt động.

Xét inventory INI **giả định** dưới đây. Các địa chỉ `192.0.2.x` chỉ dùng để minh họa, không phải máy cần kết nối. `atlas-a` là tên Ansible dùng để chọn host; `ansible_host` là địa chỉ kết nối, không biến tên ấy thành danh tính EC2 hoặc bằng chứng IP vẫn thuộc đúng máy.

```
[app]
atlas-a ansible_host=192.0.2.11
atlas-b ansible_host=192.0.2.12

[canary]
atlas-a

[db]
atlas-db ansible_host=192.0.2.21
```

Một play có `hosts: app` chọn `atlas-a` và `atlas-b`; thêm giới hạn `canary` chỉ giữ `atlas-a` trong tập đã chọn, không đưa `atlas-db` vào play. Trước khi cho task sửa tệp chạy, đọc inventory đã phân giải, so alias, địa chỉ và nhóm với danh sách máy được duyệt; trên một bản sao của đoạn INI, `ansible-inventory -i inventory.ini --list` chỉ hiển thị cấu trúc inventory, không kết nối tới ba địa chỉ. Khi có playbook thật, còn phải xác nhận host pattern và giới hạn của **chính lượt chạy**, vì inventory đúng nhưng chọn nhầm nhóm vẫn sửa nhầm máy. Nếu `atlas-b` báo

`unreachable`, chưa có kết quả module trên máy đó: kiểm địa chỉ hiệu lực, đường SSH và danh tính kết nối trước khi quy lỗi cho cấu hình ứng dụng. Một `ok` trên `atlas-a` cũng không phải kết quả của `atlas-b`; báo trạng thái và hậu kiểm theo từng host.

**Bài tập 33.1 — Chọn đúng máy trước khi chạy Ansible**

Trong inventory của bài, play chọn `hosts: app` và lượt chạy bị giới hạn `canary`. Host nào nhận task? Nếu bỏ giới hạn, `atlas-b` báo `unreachable` còn `atlas-a` báo `ok`, có thể kết luận cấu hình trên B đã hội tụ không? Nêu hai phép kiểm trước khi chạy task sửa tệp trên máy thật.

**Bài 33.2 — Đơn vị chạy**

Facts là dữ liệu Ansible thu được từ managed node, chẳng hạn hệ điều hành hoặc network interface, nếu bật `gather_facts` hoặc gọi module thu thập; fact không phải một nguồn luôn mới khi dùng cache.

Một play chọn tập host từ inventory rồi chạy các task theo thứ tự đối với các host đó. Mỗi task gọi module với tham số đã tính từ biến và facts; module so trạng thái quan sát được với yêu cầu của task, sau đó trả về `ok`, `changed` hoặc `failed`. Vì vậy bản playbook là ý định hội tụ, còn recap là báo cáo của lần chạy cụ thể, không phải bản sao state lưu lâu dài như Terraform.

Khi dùng fact trong điều kiện, cần biết fact được lấy tại thời điểm nào. Một fact đã cache có thể phản ánh trạng thái trước drift; một host không truy cập được có thể không có fact mới để đánh giá. Nếu một task được báo `changed`, tìm đúng thuộc tính đã đổi và xem handler nào được thông báo. Nếu lượt chạy thứ hai vẫn `changed` dù đầu vào không đổi, phải kiểm tra module, template, nguồn biến và điều kiện task trước khi gọi playbook là idempotent.

Hãy dự đoán một lượt chạy giả định trên host A và B. Task 1 chép template và trả `changed` ở A, `ok` ở B. Task 2 kiểm cú pháp cấu hình và trả `failed` ở A, `ok` ở B. Theo chiến lược mặc định, A bị loại khỏi các task còn lại của play, còn B tiếp tục. Recap vì vậy không phải "cả playbook thành công" chỉ vì B có `failed=0`; phải đọc kết quả theo từng host và xác định A đã bị để lại với tệp cấu hình mới nhưng chưa qua hậu kiểm. Nếu task trước có thể gây thay đổi một phần, cần thiết kế kiểm trước khi kích hoạt thay đổi và đường phục hồi riêng cho host lỗi.

Để tự kiểm mà không đụng host thật, dùng inventory chỉ tới fixture Linux tạm với `connection: local`, `become: false`, rồi so recap lần một và lần hai. Kiểm tệp, mode và nội dung sau mỗi lượt; `changed=0` lần hai là bằng chứng hẹp về những task đã chạy, không chứng minh ứng dụng ngoài play đã sẵn sàng. Nếu host bị `unreachable`, phân biệt lỗi kết nối của control node với `failed` do module chạy trên managed node; hai trạng thái đòi phép sửa khác nhau.

**Bài tập 33.2 — Fact cache và drift thực tế**

Fact cache có đủ để xác nhận host sau drift không?

**Bài 33.3 — Quyền và biến**

`become` thay danh tính thực thi trên đích, không tự cấp quyền sudo và không thay chính sách phân quyền của hệ điều hành.

Một task có thể đọc biến từ nhiều mức như inventory, play, role và lời gọi thực thi. Giá trị cuối cùng cần được xác định theo thứ tự ưu tiên thực sự của phiên bản Ansible đang dùng, thay vì đoán từ vị trí xuất hiện trong tệp. Trước khi sửa quyền một đường dẫn, ghi rõ danh tính kết nối, danh tính sau privilege escalation, chủ sở hữu mong muốn và mode hậu điều kiện. Nếu thiếu quyền, task phải báo lỗi; chạy lại với quyền rộng hơn mà không xét ranh giới truy cập có thể che giấu sai cấu hình.

`become` chỉ yêu cầu thực thi dưới danh tính khác nếu cơ chế trên máy đích cho phép. Nó không biến một lệnh tùy ý thành thao tác an toàn để chạy lặp lại. Secret dùng cho kết nối hoặc escalation cần được chuyển qua kênh quản lý phù hợp và tránh in vào log hoặc diff. Khi kiểm tra một lần hội tụ, đối chiếu mode và nội dung file thực tế sau cả lượt đầu lẫn lượt lặp lại; nhãn `ok` một mình không chứng minh hậu điều kiện quan trọng đã đúng.

Trong một ca sửa quyền **giả định**, control node kết nối bằng tài khoản `deploy`, task đặt `become: true` để ghi file do `root` sở hữu. Nếu policy sudo của host từ chối chuyển danh tính, module chưa sửa file: lỗi nằm ở bước escalation, không phải bằng chứng mode mong muốn đã sai. Nếu task báo `changed`, kiểm tra lại owner, group, mode và nội dung file trên đúng host; `changed` mô tả việc module phát hiện và thực hiện thay đổi, chưa đủ để chứng minh dịch vụ đọc được file hay handler đã chạy. Khi cần thử an toàn, trỏ play vào fixture do mình sở hữu trên filesystem POSIX và giữ `become: false`; chỉ dùng escalation trên đích thật khi phạm vi quyền và hậu điều kiện đã được phê duyệt.

**Bài tập 33.3 — Thiếu quyền và become**

Nếu task thiếu quyền, bật become có tự giải quyết không?

# Chương 34 — Hội tụ cấu hình, handler và lỗi từng máy

Chứng minh lượt đầu changed, lượt hai ok và drift sửa đúng phạm vi.

### Bài 34.1 — Tính lặp lại không gây thay đổi

Task `file` sửa quyền thư mục dữ liệu nhưng không notify: thay quyền riêng nó không phải bằng chứng cần restart.

Một task idempotent có thể chạy lại với cùng đầu vào và trạng thái quan sát mà không gây thêm thay đổi ngoài ý muốn. `file` với `mode: '0750'` chỉ sửa quyền khi khác; `template` so nội dung tạo ra với file đích rồi báo `changed` khi cần ghi. `ok` nghĩa là module không báo thay đổi ở lượt đó, không phải lời bảo đảm toàn hệ thống đúng. `failed` ngăn các task tiếp theo trên host theo quy tắc mặc định, nhưng không tự đảo những task trước đã thành công. `changed` là kết quả do module hoặc `changed_when` báo, không phải phép đo độc lập rằng người dùng được phục vụ tốt hơn. Một `command` hoặc `shell` gọi utility thường không có hiểu biết trạng thái đích: nó có thể báo changed mỗi lần và lặp tác dụng phụ, trừ khi người viết tạo điều kiện, kiểm tra và ranh giới lỗi đúng.

Trong thư mục chứa playbook, `ansible-playbook -i localhost, playbook.yml --syntax-check` kiểm tra cú pháp; dấu phẩy khiến `localhost,` là danh sách host một phần tử. Chạy với `-i localhost,` mà không dùng inventory production. Để kiểm thử quyền trên Linux, có thể thêm `-e sandbox=<thư_mục_tạm_riêng>` trỏ tới filesystem hỗ trợ mode POSIX, không cần chuyển repository hoặc dùng root. Sau lượt đầu, đọc recap từng task, nội dung `atlas.conf`, mode thực của `data` và marker. Chạy lại cùng biến: nếu đã hội tụ, task template và quyền là `ok`, handler không được thông báo. Để thử drift một cách cô lập, đổi `atlas.conf` sang `listen_port=9090`, chạy lại và so marker; chỉ đổi mode thư mục `data` rồi chạy lại phải sửa quyền mà không chạm marker. Trên filesystem không biểu diễn quyền POSIX đúng, thí nghiệm mode phải ghi SKIP thay vì diễn giải sai kết quả. Đây là thí nghiệm về module và handler, không chứng minh một daemon đã reload thành công.

#### Bài tập 34.1 — Hai lượt chạy hội tụ cấu hình

Trong ví dụ cục bộ, lượt đầu tạo file và thư mục có thể báo gì? Lượt thứ hai cùng đầu vào nên có trạng thái nào cho template và handler? Vì sao `changed=0` vẫn không chứng minh Atlas phục vụ HTTP đúng?

### Bài 34.2 — Thông báo và xử lý thay đổi

Task chạy theo thứ tự đã khai báo trên từng host; các host có thể tiến hành song song theo chiến lược và số fork. `when` bỏ qua task khi điều kiện không đúng, nhưng điều kiện dựa trên biến đăng ký từ task trước có thể khác trong check mode. `notify` chỉ xếp handler khi task báo changed; cùng một handler được thông báo nhiều lần thường chỉ chạy một lần tại điểm flush. Handler mặc định chạy sau nhóm task thích hợp, hoặc sớm hơn qua `meta: flush_handlers`. Bởi vậy phải thiết kế thứ tự kiểm tra, điểm restart và hậu điều kiện, không chỉ bật `force_handlers` để “chắc ăn”.

**Cùng playbook trên fixture POSIX · tình huống giả định** V22

| Lượt | file: mode 0750 | template | handler |
|---|---|---|---|
| 1 · tạo mới | changed | changed → notify | chạy 1 lần |
| 2 · chạy lại | ok | ok | không chạy |
| Drift quyền | changed | ok | không chạy |
| Drift config | ok | changed → notify | chạy 1 lần |

check mode chỉ dự báo thay đổi; apply thật và request vẫn cần hậu kiểm.

Hình X.2.1 — Handler chỉ chạy khi task phù hợp báo changed; các lượt là tình huống giả định.

Playbook dưới đây chỉ nhắm `localhost`, dùng connection local, `become: false` và ghi trong `.qa-ch14/fixture` thuộc giáo trình. `atlas.conf` là cấu hình mẫu; handler chỉ chạm marker `reload-requested`, không restart dịch vụ. Template gán cổng và thư mục dữ liệu từ biến. Task template mới notify handler khi nội dung hoặc quyền của file cấu hình đổi. Phép `stat`/`assert` kiểm tra hậu điều kiện quyền sau lượt thật; nó được bỏ qua trong check mode vì lúc đó thay đổi chưa xảy ra.

```
---
- name: Converge an isolated Atlas configuration
  hosts: localhost
  connection: local
  gather_facts: false
  become: false
  vars:
    sandbox: "{{ playbook_dir }}/../../.qa-ch14/fixture"
    listen_port: 8080
  tasks:
    - name: Keep the sandbox directory
      ansible.builtin.file:
        path: "{{ sandbox }}"
        state: directory
        mode: '0750'
    - name: Restore data directory access
      ansible.builtin.file:
        path: "{{ sandbox }}/data"
        state: directory
        mode: '0750'
    - name: Converge application configuration
      ansible.builtin.template:
        src: templates/atlas.conf.j2
        dest: "{{ sandbox }}/atlas.conf"
        mode: '0640'
      notify: Record reload request
    - name: Observe final data directory mode
      ansible.builtin.stat:
        path: "{{ sandbox }}/data"
      register: data_directory

    - name: Reject a failed permission repair
      ansible.builtin.assert:
        that: data_directory.stat.mode == '0750'
      when: not ansible_check_mode

  handlers:
    - name: Record reload request
      ansible.builtin.file:
        path: "{{ sandbox }}/reload-requested"
        state: touch
      when: not ansible_check_mode

listen_port={{ listen_port }}
data_dir={{ sandbox }}/data
```

Trước khi chạy, hãy dự đoán recap theo ba lần thay đổi trên filesystem POSIX tạm. Lượt đầu chưa có `sandbox`: hai task thư mục và task template phải tạo đối tượng, `stat` chỉ quan sát, `assert` kiểm mode sau khi tạo; handler được notify bởi template và chạm marker một lần ở cuối play. Lượt hai không sửa input: các task file/template phải trả `ok`, không có notify mới, marker không được chạm lại. Sau đó đổi riêng mode của thư mục `data` từ `0750` sang `0770`: task sửa thư mục phải báo `changed`, nhưng template vẫn `ok`, nên handler không chạy. Nếu handler lại chạy, thiết kế notify hoặc biến đầu vào không khớp dự đoán; đừng giải thích bằng câu “Ansible luôn restart khi có bất kỳ thay đổi nào”.

Biến thể khác là đổi `listen_port` của template. Task template phải báo changed khi nội dung đích đổi và notify handler. Nếu một task sau template thất bại trước điểm flush, handler mặc định có thể không chạy như kỳ vọng; phải quyết định có cần flush sớm, `force_handlers` và cơ chế bảo đảm an toàn của dịch vụ hay không. Chạy handler sớm cũng có giá: service có thể nạp cấu hình trước khi các task phụ thuộc hoàn tất. Với playbook thật, kiểm syntax/config trước khi publish file mới và chỉ reload khi hậu điều kiện đã đạt, thay vì dùng handler như cách che mọi lỗi task.

Fixture trong ví dụ có thể đặt ở filesystem Linux hỗ trợ mode POSIX bằng cách truyền biến `sandbox` khi chạy trong môi trường tạm do người thử sở hữu. Không suy quyền thật từ mount 9p hoặc filesystem không biểu diễn đúng mode; `stat.mode` phải được đối chiếu sau lượt apply thật. Không cần `become` hay sửa host service. Ghi recap `ok/changed/failed`, mode, nội dung cấu hình và dấu thời gian marker sau từng lượt để phân biệt hội tụ với việc chỉ chạy không lỗi. Check mode hữu ích để dự báo thay đổi, nhưng vì task có thể bỏ qua hậu kiểm và module có hành vi mô phỏng khác apply thật, kết quả check mode không thay thế phép chạy trên fixture cô lập.

**Bài tập 34.2 — Sửa quyền mà không restart thừa**

Chỉ mode thư mục `data` thành `0700`; `atlas.conf` không đổi. Playbook sửa gì, handler có chạy không? Vì sao `command: chmod 0750 ...` có `notify` lại có thể restart thừa?

**Bài 34.3 — Xử lý lỗi**

`--syntax-check` kiểm tra cấu trúc playbook, chưa kết nối và chưa xác nhận đường file, quyền hoặc hành vi module trên host thật. `--check` dự đoán thay đổi mà không ghi trên managed node với module hỗ trợ; module không hỗ trợ có thể bị bỏ qua, task dùng kết quả task trước có thể không được đánh giá như lúc apply. `--diff` hiển thị chênh lệch khi module hỗ trợ và có thể làm lộ dữ liệu nhạy cảm trong template, nên không bật tùy tiện cho secret. Check mode không dự báo được API sẽ từ chối, ổ đĩa hết chỗ, handler restart thất bại hay request sau restart sai. Chạy trên một host đại diện và kiểm tra sau thay đổi vẫn cần thiết.

Nếu host lỗi trước lúc handler được chạy, handler thường không chạy trên host đó; `force_handlers` thay hành vi này nhưng có thể khiến restart trên cấu hình chưa được xác minh.

Xét một play giả định có ba task: render template, đổi mode thư mục, rồi validate cấu hình; handler restart được notify chỉ khi template changed. Trong `--check`, task validate có thể không nhìn thấy file mới vì bước render chưa ghi thật; vì vậy "check xanh" không chứng minh handler sẽ restart an toàn khi apply. Trên một host thử có dữ liệu riêng, ghi checksum template, mode và trạng thái service trước/sau lượt đầu; lượt thứ hai phải `ok` ở các task hội tụ, không notify restart ngoài ý muốn. Nếu validation fail trước handler, cần xem lại file đang hiện hữu và trạng thái tiến trình, không áp `force_handlers` như một thuốc sửa tổng quát.

Một cách đọc recap có giá trị hơn chỉ nhìn `failed=0` là so với dự đoán trước chạy. Nếu chỉ mode thư mục drift, task file phải `changed`, task template phải `ok` và không có notify restart. Nếu template đổi nội dung, handler sẽ được xếp hàng; trước khi cho nó tác động dịch vụ, kiểm cú pháp file đích và điều kiện quyền/truy cập dữ liệu. `--check` không ghi file, nên một task validate đọc file hiện hữu có thể đang kiểm bản cũ chứ không phải bản sắp áp; đó là sự khác nhau giữa dự đoán thay

đổi và kiểm thực trạng sau apply. Đặt tên từng bước kiểm trong log để biết cổng nào đã chạy thật.

Trên host thật, đưa thay đổi qua một máy thử hoặc một nhóm nhỏ, ghi phiên bản playbook, inventory, biến hiệu lực và checksum cấu hình trước khi sửa. Sau lượt đầu, kiểm mode bằng `stat`, cú pháp cấu hình bằng công cụ của dịch vụ, trạng thái process và một request nghiệp vụ. Sau lượt hai với cùng input, so checksum, recap và marker reload: nếu còn `changed` không dự kiến, truy task nào không hội tụ thay vì chấp nhận restart lặp. Với cấu hình có secret, không đưa `--diff` hoặc output module lên hệ log mở; dùng hash hoặc phép kiểm chức năng phù hợp mà không in giá trị.

Nếu task validate thất bại sau khi template đã thay, trạng thái cần báo là **partial change**: file trên host có thể đã mới nhưng handler chưa chạy, service có thể còn dùng cấu hình cũ trong bộ nhớ. Phương án sửa tùy hợp đồng dịch vụ: khôi phục file cũ đã lưu, sửa file mới rồi validate lại, hoặc cô lập host khỏi traffic trước khi reload. Không tự suy rollback đã xảy ra vì playbook trả lỗi. Hậu điều kiện phải nói rõ file trên đĩa, process đang dùng bản nào và request của người dùng ra sao.

**Bài tập 34.3 — Check mode và lỗi apply từng phần**

Một lượt `--check --diff` báo sẽ sửa config, nhưng apply thật thất bại sau khi một host đã restart. Hãy nêu ba giới hạn của check/diff, trạng thái từng phần cần ghi lại và hậu điều kiện trước khi chạy tiếp.

# Chương 35 — Vai trò, bí mật và triển khai theo đợt

Tổ chức cấu hình nhiều host với blast radius kiểm soát được.

## Bài 35.1 — Tái sử dụng

Variables có thể đến từ play, inventory, group/host vars, role, CLI và nguồn khác; ưu tiên của chúng phải được kiểm tra khi cùng tên xuất hiện nhiều nơi. Role gom task, handler, template và biến theo cấu trúc tái sử dụng; nó không tạo ranh giới quyền mới, và handler của role tham gia phạm vi handler của play. Một role dùng lại rộng rãi vẫn cần review quyền, biến mặc định và tác dụng phụ trên nhóm host thực tế.

Trong một play giả định, role đặt `listen_port: 8080`, group vars đặt `9090`, còn người vận hành truyền `listen_port=7070` ở dòng lệnh. Trước khi đoán port cuối từ trí nhớ về thứ tự ưu tiên, kiểm đúng phiên bản ansible-core, loại biến được dùng và giá trị đã render cho host mục tiêu; một extra var và một tuỳ chọn CLI thông thường không có cùng luật ưu tiên. Với fact về địa chỉ mạng, ghi thời điểm thu thập và host identity: một fact cache cũ có thể làm template đúng cú pháp nhưng sai địa chỉ. Nếu role notify handler, đọc nơi handler được định nghĩa và các tên `listen` trùng nhau để biết tác vụ nào thật sự chạy. Đây là tình huống lập luận, không phải output của playbook đã thực thi.

Với **ansible-core 2.16**, giả sử `listen_port` là `8080` trong `roles/atlas/defaults/main.yml`, `9090` trong `group_vars` và `7070` từ `-e listen_port=7070`. Template nhận `7070`: extra vars có ưu tiên cao nhất trong ba nguồn. Bỏ `-e` thì nhận `9090`; bỏ cả group var thì nhận `8080`. Nếu chuyển khai báo sang `vars/main.yml`, role var thắng group var nên nhận `8080` khi không có `-e`. Dạng `-e key=value` cho giá trị chuỗi; nếu cần số nguyên, dùng JSON/YAML hoặc chuyển kiểu có kiểm chứng. Kiểm file render trên đúng host và cổng ứng dụng thực lắng nghe trước khi restart; precedence không chứng minh ứng dụng đã mở cổng.

Trong một lượt chạy nhiều host, "giá trị cuối cùng của playbook" chưa phải một khái niệm đủ chính xác. Giả sử `atlas-a` thuộc nhóm `web`, `atlas-b` thuộc nhóm `worker`; `group_vars/web` đặt `listen_port=9090`, còn `atlas-b` chỉ nhận role default `8080`. Không có `-e`, hai host có thể render hai cổng khác nhau trong cùng play. Nếu inventory động đã đổi membership mà người vận hành vẫn đọc bản inventory cũ, họ có thể đổ lỗi cho precedence dù đầu vào host/group đã khác. Trước khi thay role, cố định inventory source, phiên bản playbook, tên host được chọn và danh sách group của từng host; xem giá trị hiệu lực trên đúng host bằng một phép debug chỉ cho biến **không nhạy cảm**, rồi so template đã render. `ansible-inventory` giúp kiểm đầu vào inventory, nhưng nó không chứng minh task sau này không gọi `set_fact`, `include_vars` hoặc truyền extra vars làm giá trị đổi thêm.

Một phép thử đối chứng nhỏ dùng fixture local và hai host giả không kết nối production: chạy cùng role với hai bộ inventory đã biết, ghi cổng dự đoán trước khi render và so file kết quả. Sau đó đưa `-e listen_port=7070` vào một lượt thử riêng: cả hai host phải thấy override ấy ở task liên quan nếu không có lỗi scope/type. Nếu `atlas-a` vẫn nghe 8080 sau khi file của nó ghi 7070, lỗi đã chuyển từ chọn biến sang nạp cấu hình hoặc vòng đời service; kiểm handler, trạng thái socket và một request qua đúng host. Nếu file vẫn ghi 9090, tiếp tục truy biến hiệu lực, điều kiện task và đường file mà template thực sự ghi, không restart hàng loạt để tìm may rủi. Với fact cache, một giá trị "đã quan

sát" có thể thuộc lượt trước; ghi thời điểm thu thập và làm mới fact cần thiết trước quyết định. Không debug toàn bộ `hostvars` khi nó có thể chứa credential; chỉ xuất các trường vô hại phục vụ giả thuyết.

**Bài tập 35.1 — Giá trị biến hiệu lực trên từng máy**

Role default đặt listen_port=8080, group_vars/web đặt 9090. Host atlas-a thuộc web, atlas-b chỉ nhận default. Dự đoán cổng render trên mỗi host khi không có -e, rồi khi chạy với -e listen_port=7070. Nếu file của atlas-a ghi 7070 nhưng service vẫn nghe 8080, cần điều tra ranh giới nào tiếp theo?

**Bài 35.2 — Phân đợt**

Giả sử hai máy `atlas-a` và `atlas-b` phục vụ sau load balancer. Config đúng là `listen_port=8080`; service chạy dưới user `atlas`, cần quyền đọc file config và đi qua thư mục dữ liệu. Một playbook lỗi chạy `command: chmod 0700 /srv/atlas/data` ở mọi lượt rồi `notify` restart, còn task template đặt `owner: root`, `group: root`, `mode: '0600'` cho config. Trên máy chỉ cần đổi quyền thư mục, `command` vẫn có thể báo changed và kích hoạt handler không cần thiết; restart sau đó có thể khiến service user không đọc được config hoặc data. Nếu cả hai host restart cùng lúc, khả năng phục vụ có thể mất. Đây là chuỗi giả định, không phải log đã đo; `chmod` có thể thành công về exit status dù tạo quyền sai cho ứng dụng.

Phân tích theo thứ tự: xác nhận inventory thực sự chọn hai máy, connection user và `become` dùng quyền nào; xem task nào báo changed và vì sao; xác nhận mode/owner/group cùng UID/GID của service, không chỉ nhìn ký hiệu `0700` hoặc `0600`. So file trước và sau, số lần notify, điểm handler chạy, trạng thái systemd và request từ client. Nếu template bị ghi sai quyền, sửa ownership/mode trước khi restart. Dùng module `file` và `template` để hội tụ về chủ sở hữu/quyền đã kiểm tra; chỉ task đổi nội dung config mới notify handler `ansible.builtin.systemd_service` với `state: restarted` khi ứng dụng thực sự cần đọc lại config. Task bảo đảm service đang chạy dùng `state: started`, không dùng `restarted` ở mọi lượt. Với nhiều máy, dùng nhóm thử trước rồi `serial: 1` hoặc chiến lược phù hợp, rút từng máy khỏi lưu lượng nếu cần và kiểm tra request đại diện trước máy tiếp theo. `serial: 1` giới hạn số host trong một batch, không tự rút lưu lượng hoặc bảo đảm health check đúng.

Một sửa đổi an toàn bắt đầu từ backup cấu hình và đường khôi phục, kiểm tra syntax của cấu hình bằng chính ứng dụng nếu nó có chế độ validate đáng tin, kiểm tra quyền file và dữ liệu, rồi mới restart. Nếu validate task thất bại, không nên ghi đè file tốt hoặc gọi handler. Sau restart, kiểm tra tiến trình, log, cổng lắng nghe và request qua đúng load balancer; trạng thái `active` của systemd chỉ là một phần bằng chứng. Khi một host lỗi giữa play, các task đã hoàn tất trên host khác không tự rollback. Ghi lại host nào đã đổi nội dung, host nào mới sửa quyền, handler nào đã chạy và phiên bản config hiện tại; khoanh tác động rồi phục hồi từng host theo trạng thái thực. Secrets trong biến, template và diff cần được quản lý bằng cơ chế bảo mật riêng, không đặt mật khẩu vào inventory Git hay in diff/log.

### Bài tập 35.2 — Chmod gây restart không cần thiết

Playbook lỗi ở Bài 35.2 báo task `chmod` changed và restart hai máy. Hãy dựng chuỗi nguyên nhân có thể dẫn đến gián đoạn, nêu bằng chứng cần thu để phân biệt lỗi quyền với lỗi nội dung config, và đề xuất thứ tự sửa trên từng máy.

### Bài 35.3 — Quản lý bí mật

Ansible Vault mã hóa biến hoặc tệp ở trạng thái lưu trữ; controller giải mã để đưa vào module hoặc template. Vault tránh plaintext trong Git, nhưng giá trị có thể lộ qua output, diff, log, command line hoặc file đích. `no_log: true` chỉ giảm lộ trong output Ansible. Lần theo secret từ kho khóa, controller, SSH, managed node tới ứng dụng.

Với `atlas-a` và `atlas-b` đổi credential, kiểm ai giữ khóa Vault/KMS, đọc repo, chạy playbook và xem log. Để khóa giải mã cạnh repo phá ranh giới bảo vệ. `ansible-playbook --check --diff` có thể lộ secret trong template; kiểm syntax và hậu điều kiện không in giá trị, không dùng secret thật trong fixture.

Xoay secret cần bên cấp và bên tiêu thụ cùng chuyển: nếu chỉ một credential hợp lệ, đổi bên nào trước cũng có thể gây lỗi. Khi hỗ trợ chồng lấp, tạo khóa mới, cập nhật từng host, kiểm request rồi thu hồi khóa cũ; nếu không, lập kế hoạch gián đoạn. `serial: 1` hạn chế số host cùng đổi, không bảo đảm ứng dụng reconnect.

Sau rollout, kiểm owner/mode, ứng dụng đọc được khóa mới, request, log/diff và thu hồi khóa cũ. Nếu host lỗi, ghi phiên bản từng host rồi cô lập host lỗi; Ansible không rollback host đã chạy. Tình huống giả định, không dùng credential thật hay host hiện hữu.

Để thấy trạng thái từng phần, xét một lần xoay **giả định**: bên cấp nhận cả `K0` và `K1` đến 10:20. Lúc 10:05, `atlas-a` đã nhận `K1` và xử lý request thử; `atlas-b` vẫn dùng `K0` vì task ghi file trả lỗi quyền. Nếu thu hồi `K0` ngay vì lượt đầu của playbook xanh, `atlas-b` mất khả năng xác thực. `serial: 1` chỉ giới hạn host được xử lý trong mỗi batch, không kiểm chứng hộ rằng host còn lại đã nhận khóa hay sẽ được load balancer loại ra. Tại 10:05 phải giữ `K0`, ghi rõ A=`K1`, B=`K0`, nguyên nhân lỗi quyền và thời hạn chồng lấp; sửa B trong phạm vi đã kiểm rồi thử request qua từng host trước khi thu hồi. Nếu không thể kéo dài thời gian chồng lấp, cần quyết định dừng hoặc gián đoạn có kiểm soát, không hứa xoay khóa không downtime.

Biên bản không cần ghi byte khóa hoặc hash của một mật khẩu yếu. Dùng định danh phiên bản bí mật do hệ cấp quản lý, host identity, thời điểm thay, mode/owner của file đích, kết quả xác thực thử và thời điểm thu hồi. `no_log` cho task nhạy cảm và tắt diff của template chỉ giới hạn một số đường output; vẫn phải kiểm log ứng dụng, file tạm, quyền người chạy và nơi giữ khóa giải mã. Hậu điều kiện là mọi host còn nhận traffic đã dùng `K1`, request qua từng host thành công, `K0` đã bị thu hồi đúng lúc và không có giá trị bí mật trong bản ghi vận hành. Nếu một host không thể chứng minh trạng thái, ghi UNKNOWN và giữ phương án cô lập hoặc quay lui; không suy từ recap chung rằng cả cụm đã xoay xong.

### Bài tập 35.3 — Vault, diff và rò bí mật

File Vault trong Git và task template chạy với --diff. Có thể khẳng định secret không rò không? Nêu kiểm soát cần thiết.

# PHẦN XI — Kubernetes và ứng dụng đám mây

## Khai báo biến thành Pod sẵn sàng phục vụ nhờ những vòng điều khiển nào?

API lưu spec, controller/scheduler/kubelet/dataplane hội tụ bất đồng bộ; status, endpoint và request là các chứng cứ khác nhau.

XI

**LỘ TRÌNH PHẦN**


- Chương 36 — API, etcd và vòng điều khiển hội tụ
- Chương 37 — Pod, Deployment và trạng thái sẵn sàng
- Chương 38 — Lập lịch và ngân sách tài nguyên
- Chương 39 — Service, EndpointSlice và NetworkPolicy
- Chương 40 — PVC, CSI và dữ liệu của ứng dụng
- Chương 41 — HPA và điều khiển mở rộng
- Chương 42 — Gateway API và ranh giới yêu cầu ngoài cụm
- Chương 43 — Gói Helm (Helm chart) và vòng đời bản phát hành

# Chương 36 — API, etcd và vòng điều khiển hội tụ

Theo một spec từ apply tới các controller và node.

## Bài 36.1 — Mặt phẳng điều khiển

Tôi chọn mở bài Kubernetes bằng một câu hỏi nhỏ: sau khi `kubectl apply` trả về, chính xác điều gì đã xảy ra? Câu trả lời dễ nghe nhất — "ứng dụng đã chạy" — cũng là câu trả lời dễ đưa người vận hành đi lạc. Bạn vừa giao một ý định cho API; từ đó đến một request thành công còn là hành trình của nhiều tác nhân độc lập. Chúng ta sẽ lần theo hành trình ấy, thay vì học thuộc tên các thành phần.

Ở Chương 34, một lần chạy Ansible quan sát rồi sửa cấu hình trên các máy đã chọn. Kubernetes đặt khai báo trong API để nhiều bộ điều khiển liên tục quan sát và sửa phần việc của mình. Người vận hành gửi `spec` mô tả điều muốn có; `status` là quan sát mà hệ thống cập nhật sau đó. Khoảng cách giữa hai phần là chuyện bình thường trong một thời gian, không phải bằng chứng `kubectl apply` đã thất bại. Ngược lại, API chấp nhận object cũng không chứng minh ứng dụng đã phục vụ. Muốn chẩn đoán phải lần theo đối tượng nào đã xuất hiện, ai chịu trách nhiệm bước kế tiếp, và điều kiện nào còn chưa đạt.

Hãy giữ hai bản sao `atlas` trong đầu, nhưng đừng vội nhìn chúng như hai tiến trình đã tồn tại. Ở thời điểm đầu tiên, chúng mới là một yêu cầu được ghi nhận. Tôi cố tình dừng lâu ở khoảnh khắc này vì hầu hết nhầm lẫn về control plane bắt đầu từ việc bỏ qua nó.

Giả sử ta khai báo Deployment `atlas` có hai bản sao. `kubectl` gửi yêu cầu đến API server. API server xác thực người gọi, kiểm tra quyền và qua các bước admission/validation áp dụng, rồi lưu object bằng kho trạng thái của cluster, thường là etcd. etcd giữ dữ liệu API; nó không chạy container và không phải kho dữ liệu của ứng dụng. Phản hồi apply thành công xác nhận yêu cầu khai báo đã được API chấp nhận, không phải xác nhận hai Pod đã chạy. Bản sao etcd và backup của nó liên quan đến khả năng khôi phục control plane; chúng không tự backup database bên trong workload.

Controller manager chạy các vòng điều khiển. Deployment controller thấy Deployment mới thì tạo hoặc điều chỉnh ReplicaSet; ReplicaSet controller tạo Pod để số bản sao tiến gần `spec.replicas`. Controller thường đọc/ghi qua API server thay vì trực tiếp mở tiến trình ứng dụng. Chọn node không có nghĩa image đã tải hoặc ứng dụng đã mở cổng. Runtime thực thi container, còn kubelet điều phối vòng đời Pod trên node. Network plugin và triển khai Service trên node tham gia đường truyền gói tin; không thể lấy kết quả scheduler làm bằng chứng đường dữ liệu đã thông.

## Bài 36.2 — Vòng điều khiển hội tụ

Một object Kubernetes không phải tấm ảnh chụp hiện trường. Nó giống một cuộc trao đổi đang diễn ra: phía khai báo nói điều mình muốn, phía điều khiển báo điều mình đã quan sát. Nếu bạn đọc hai vế như thể chúng được viết cùng lúc, mọi độ trễ bình thường đều trông giống lỗi. Bài này dạy cách đọc cuộc trao đổi ấy theo thời gian.

`spec` và `status` có chủ thể ghi khác nhau. Khi sửa `spec`, `metadata.generation` của object phù hợp tăng; trường `status.observedGeneration` của một số controller, khi được công bố, giúp nhận ra controller đã quan sát thế hệ khai báo nào. Không phải mọi kind/status đều cung cấp cùng

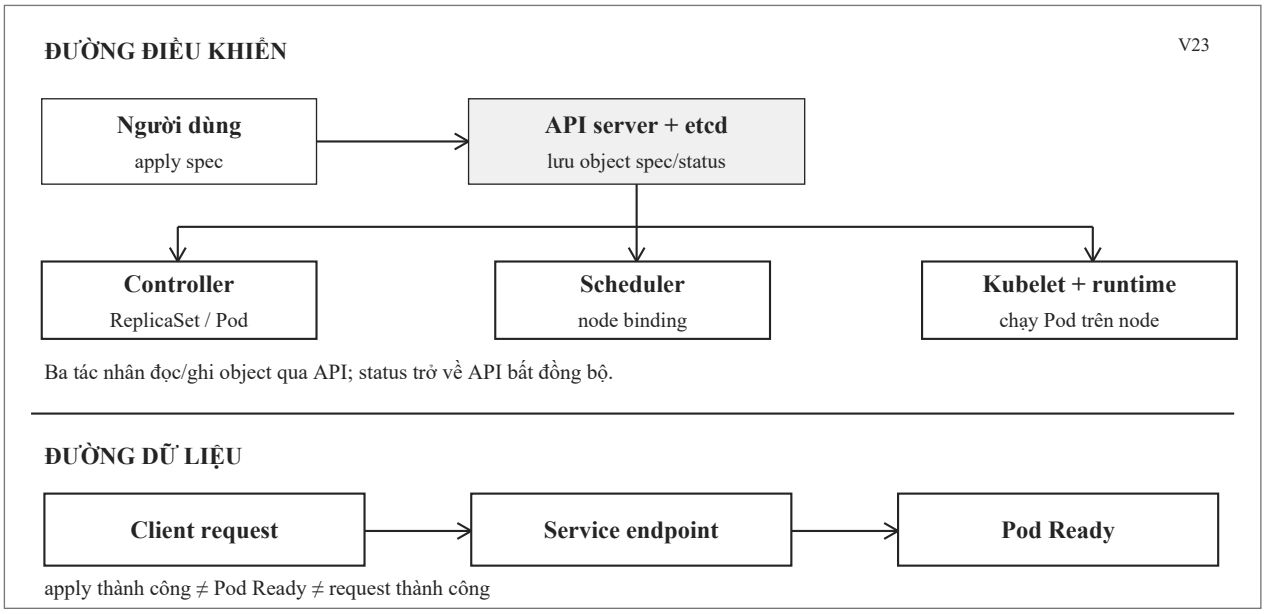
một trường hoặc cập nhật cùng lúc, nên phải đọc theo hợp đồng của kind. `metadata.resourceVersion` là chuỗi phiên bản nội bộ để watch và cập nhật có điều kiện; chỉ so bằng nhau hoặc gửi lại nguyên dạng theo API, không biến nó thành đồng hồ hoặc lấy số lớn hơn để kết luận Pod đã mới hơn Deployment. Một lần `get` là ảnh chụp tại thời điểm đọc; quan sát thay đổi cần watch hoặc đọc lại, không suy từ một snapshot.

Tôi tách hai câu hỏi trước khi nhìn số Pod: controller đã thấy bản khai báo mới chưa, và sau khi thấy, nó đã làm được đến đâu? Hai câu hỏi có thể cho hai câu trả lời khác nhau trong cùng một phút. Con số generation giúp trả lời câu đầu; trạng thái các đối tượng liên quan mới giúp điều tra câu sau.

Trong tình huống giả định, image của Deployment vừa đổi và `metadata.generation=7`, còn `status.observedGeneration=6`. Tôi chưa gọi đó là rollout hỏng: ảnh chụp chỉ cho thấy status chưa chứng minh controller đã xử lý thế hệ mới. Bạn đặt một hạn chờ hợp lý, đọc lại generation, condition và ReplicaSet; nếu controller đã quan sát thế hệ 7 mà `availableReplicas` vẫn thấp, mới đi xuống Pod event, probe và selector. Kể cả hai thế hệ đã khớp, request từ ngoài cluster còn phải được thử riêng. Controller báo về phần việc của nó, chứ không ký giấy bảo đảm cho toàn bộ đường client.

Vòng reconciliation không phải một lệnh `apply` kéo dài. Controller đọc ý định và đối tượng liên quan qua API, tính chênh lệch, gửi thay đổi cần thiết rồi quan sát lại ở lượt sau. Với Deployment muốn hai replica, việc đã có ReplicaSet nhưng chỉ một Pod không khiến API server tự tạo Pod thứ hai: controller của ReplicaSet phải quan sát phần thiếu, yêu cầu tạo Pod; scheduler và kubelet còn các bước riêng. Nếu API hoặc quota từ chối tạo Pod, controller có thể thử lại mà trạng thái thực vẫn chưa hội tụ. Muốn xác định vòng nào đang kẹt, lần theo owner reference, số lượng mong muốn và hiện có tại từng đối tượng, cùng event của chính đối tượng ấy; không chỉ đọc một điều kiện ở Deployment.

`resourceVersion` còn bảo vệ người ghi khỏi ghi đè trạng thái vừa đổi. Trong một ví dụ giả định, người A đọc object ở phiên bản mờ đục `rA`, người B cập nhật cùng object và API trả phiên bản mới `rB`. Nếu A gửi một bản cập nhật `PUT` toàn object vẫn kèm `rA`, API từ chối với `409 Conflict`; A phải đọc lại và quyết định thay đổi của mình còn hợp lệ hay không. Không thay `rA` bằng `rB` một cách mù quáng: B có thể đã sửa chính trường A định sửa. Đây là cơ chế phát hiện xung đột của cập nhật có điều kiện, không phải bằng chứng controller đã hoàn thành công việc. `generation` trả lời spec nào đã được khai báo, `observedGeneration` trả lời controller tuyên bố đã thấy spec nào, còn phép đọc request thật mới trả lời người dùng nhận gì.




Hình XI.1.1 — Vòng điều khiển Kubernetes và đường request tách biệt.

### Bài tập 36.1 — Apply thành công nhưng chưa có Pod sẵn sàng

`kubectl apply` trả thành công, Deployment mong muốn hai Pod nhưng `availableReplicas=0`. Hãy nêu chuỗi object và thành phần cần quan sát để phân biệt controller chưa tạo Pod, scheduler chưa chọn node và kubelet chưa chạy container. Mỗi bước có thể kết luận đến đâu?

### Bài 36.3 — Ranh giới tại nút

Khi một khách hàng nói “dịch vụ vẫn không vào được”, sơ đồ thành phần của control plane chưa đủ để trả lời. Tôi muốn bạn đặt ngón tay lên đường đi của chính request ấy, rồi đặt ngón tay còn lại lên đường đi của bản khai báo. Hai đường có thể cắt nhau ở Pod và Service, nhưng không thay thế nhau.

Có hai đường cần vẽ riêng trong đầu. Đường điều khiển là client → API server → object trong kho trạng thái → controller/scheduler → kubelet, kèm các lần ghi status ngược về API. Đường dữ liệu của một request là client → địa chỉ/port Service → cơ chế chuyển tiếp trên node → IP/port Pod → ứng dụng, tùy kiểu Service và cách cluster triển khai mạng. API server không chuyển mọi HTTP request của ứng dụng qua nó. Một control plane khỏe có thể đi kèm Service không có backend; một Pod chạy được cũng không chứng minh client đi tới được nó.

Scheduler theo dõi Pod chưa có node, chọn node theo tài nguyên được yêu cầu và các ràng buộc, rồi ghi quyết định gán node vào API.

Kubelet trên node nhận Pod được gán, làm việc với container runtime qua Container Runtime Interface để chuẩn bị sandbox, tải image và chạy container; nó báo trạng thái quan sát về API.

Một lệnh `kubectl apply` thành công chỉ nói API server đã chấp nhận object theo quyền và validation tại thời điểm đó. Nếu Pod chưa có node, theo dõi scheduler và event của Pod; nếu đã có node nhưng `ImagePullBackOff`, xem đường kubelet tới registry và thông tin xác thực. Chỉ khi Pod Ready, EndpointSlice chọn đúng Pod và một request từ client đi qua Service có phản hồi mới có

chứng cứ đầu cuối. Đường điều khiển có thể tiếp tục hội tụ sau khi lệnh apply kết thúc, nên đặt timeout và phân biệt “chưa kịp” với “không thể” bằng chuỗi trạng thái và event, không bằng một ảnh chụp đơn lẻ.

Hãy lần theo một triển khai giả định, không phải log thu từ cluster. Ở bước A, API server nhận Deployment yêu cầu ba replica và trả lời thành công, nhưng chưa có Pod nào; phải đợi Deployment controller tạo ReplicaSet rồi ReplicaSet tạo Pod. Ở B, ba Pod đã xuất hiện nhưng một Pod chưa có `nodeName`: controller đã đi qua bước tạo object, còn scheduler chưa gán node cho Pod ấy; đọc event để phân biệt thiếu tài nguyên, ràng buộc node và việc controller chỉ chưa kịp xử lý. Ở C, Pod có `nodeName` nhưng container đang `ImagePullBackOff`: quyết định gán node đã có, vấn đề chuyển sang kubelet, image reference, registry hoặc credential. Tăng số replica lúc này không chứng minh đã sửa đường kéo image.

Đến mốc D, container đã chạy nhưng Pod vẫn `Ready=False`. Một người nhìn cột `Running` có thể thở phào quá sớm; EndpointSlice của Service chọn đúng nhãn còn phải phản ánh điều kiện endpoint, và trong cấu hình thường backend chưa Ready không phải đích phục vụ bình thường. Ở mốc E, Pod Ready và EndpointSlice đã có backend phù hợp, nhưng tôi vẫn giữ lại phép thử cuối: gửi request từ đúng vị trí client, qua đúng Service, rồi đối chiếu response. Nếu hồ sơ chỉ còn ảnh C và E mà thiếu timeline, bạn không thể biết lỗi image pull tự hết, được sửa hay đang xuất hiện ở Pod khác. Event, status và thời điểm phải đi cùng nhau; chúng không phải biên bản của một giao dịch duy nhất.

# Chương 37 — Pod, Deployment và trạng thái sẵn sàng

Phân biệt Running, Ready, Available và rollout complete.

## Bài 37.1 — Đối tượng điều khiển ứng dụng

Ba Pod đều hiện màu xanh trên màn hình; người dùng vẫn nhận lỗi. Cảnh ấy không mâu thuẫn nếu ta biết ai sở hữu Pod và ai chỉ chọn Pod bằng nhãn. Tôi thích bắt đầu từ quan hệ này, vì nó giải thích được cả hai kiểu sai: Service không thấy backend nào, hoặc Service vô tình thấy thêm một backend lạ.

Pod là đơn vị được lập lịch; các container của Pod chia sẻ ngữ cảnh mạng và các volume được khai báo cho Pod. Pod có danh tính và vòng đời riêng. Deployment giữ ý định cập nhật một tập Pod qua các ReplicaSet; ReplicaSet giữ số Pod theo selector và Pod template. Quan hệ quản lý được ghi ở `metadata.ownerReferences`: Pod thường thuộc ReplicaSet, ReplicaSet thuộc Deployment. Khi Pod mất, controller có thể tạo Pod khác, với UID/IP khác. Selector là phép chọn theo label, không phải quan hệ sở hữu: Service chọn Pod có label khớp dù Pod đó thuộc controller nào. Nếu selector của Service sai, việc Deployment đủ bản sao không sửa được đường nối này.

Pod `Running` nói Pod đã gắn vào node và container đã được tạo, với ít nhất một container còn chạy hoặc đang khởi động lại.

Trước khi sửa YAML, bạn hãy dự đoán số backend bằng đúng phép so nhãn, không bằng số replica của Deployment. Một phép tính tập hợp nhỏ ở đây thường tiết kiệm nhiều vòng restart vô ích.

Hãy xét Deployment muốn ba bản sao và ReplicaSet đã tạo ba Pod, cả ba `Running`; Service selector lại dùng `app=invoice` trong khi Pod mang `app=orders`. Controller của Deployment có thể hoàn thành phần việc của nó, còn Service không chọn endpoint nào. Kiểm nhãn Pod và selector Service ở cùng namespace, sau đó kiểm EndpointSlice, thay vì tăng `replicas` hoặc khởi động lại node. Trạng thái `Ready` của từng Pod trả lời một câu hỏi khác: các probe của Pod đã đạt tại thời điểm báo cáo. Cả `Running` lẫn `Ready` không khẳng định nhãn khớp với Service hoặc request đi qua proxy sẽ thành công.

Sự cố ngược lại còn khó nhận ra hơn. Trong một cấu hình giả định, ReplicaSet của ba Pod API chọn `app=orders,component=api`, còn Service chỉ chọn `app=orders`. Một Pod debug độc lập có `app=orders,component=debug`, không thuộc ReplicaSet và đang `Ready`. Deployment báo đủ ba bản sao là đúng trong phạm vi nó quản; Service có thể nhận bốn backend khớp nhãn, gồm Pod debug. Tôi sẽ không tăng replica để chữa phép chọn sai, cũng không chỉ xóa Pod debug nếu nó có thể được tạo lại. Bạn đối chiếu UID, `ownerReferences`, label, selector và `targetRef` của EndpointSlice; sau đó mới thu hẹp selector và thử request từ đường client thật. Thay selector trên hệ thống đang phục vụ có thể loại cả backend hợp lệ. Lập luận này dành cho Service có selector do controller quản lý, không áp sang endpoint khai báo riêng.

### Bài 37.2 — Phép kiểm tra sức khỏe

Buổi sáng, bảng điều khiển báo hai Pod đang chạy; điện thoại hỗ trợ lại nhận tin người dùng không mở được trang. Tôi dùng tình huống giả định ấy để đặt một câu hỏi cụ thể cho bạn: trạng thái nào chứng minh tiến trình tồn tại, trạng thái nào quyết định Pod có được đưa vào tập backend? Câu trả lời sẽ nằm ở probe và EndpointSlice, không nằm trong màu của cột trạng thái.

Một Pod có thể đã được tạo và có tiến trình đang chạy. Nó không nói request thành công. Readiness probe kiểm tra điều kiện để đưa Pod vào tập backend bình thường; probe thất bại có thể giữ Pod `Running` nhưng `Ready=False`. Trong EndpointSlice, một endpoint có các điều kiện như `ready`, `serving`, `terminating`; không đếm mọi địa chỉ tìm thấy như backend đang phục vụ. Một Service có selector được EndpointSlice controller cập nhật theo các Pod khớp; địa chỉ Service và `targetPort` vẫn phải đi tới đúng cổng ứng dụng. `containerPort` trong Pod là khai báo thông tin cổng, không tự làm ứng dụng lắng nghe.

Ca `atlas` dưới đây có hai lỗi độc lập. Trước khi đọc tiếp, thử dự đoán: nếu chỉ sửa probe thì Service sẽ có backend không; nếu chỉ sửa selector thì request sẽ được chuyển tới Pod nào? Đặt hai giả thuyết riêng giúp ta thấy vì sao một thay đổi đúng có thể chưa làm hệ thống tốt lên ngay.

Đây là dữ liệu giả định để luyện đọc trạng thái, không phải output từ một cluster đã chạy. Ở namespace `demo`, bản sửa đổi khai báo Deployment `atlas` hai bản sao; Pod template có label `app=atlas`, container dự kiến nghe TCP 8080, readiness HTTP gọi `/ready` trên 8080. Service `atlas` công bố port 80, `targetPort: 8080`, nhưng selector bị gõ `app=atals`. Ứng dụng vừa đổi đường readiness thành `/health/ready`, còn `/ready` trả 404. Hai lỗi độc lập cùng ngăn lưu lượng. Manifest minh họa không đặt `metadata.namespace`; nếu thử trong môi trường cô lập đã được cho phép, dùng `kubectl -n demo apply -f` với đúng đường dẫn manifest, hoặc khai báo namespace trong từng object. Không sửa cả hai lỗi theo trực giác trước khi biết từng ranh giới đã xảy ra gì.

Manifest tối giản dưới đây cho thấy trực tiếp hai sai khác ấy. Container minh họa tạo file ở `/health/ready` rồi phục vụ thư mục qua HTTP; probe lại gọi `/ready`, còn Service chọn nhãn khác nhãn Pod. Đây là cấu hình cố ý lỗi để đọc spec, không phải kết quả đã chạy trong cluster; image tag cũng chưa được xác minh bằng một lần pull.

```
# Minh họa giả định trong namespace demo; cố ý sai probe và Service selector.
# Image tag dùng để đọc spec, chưa được pull hoặc chạy trên cluster.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: atlas
spec:
  replicas: 2
  selector:
    matchLabels:
      app: atlas
  template:
    metadata:
      labels:
        app: atlas
    spec:
      containers:
        - name: atlas
          image: python:3.12-alpine
          command: ["/bin/sh", "-ec"]
          args:
            - |
              mkdir -p /tmp/site/health
              printf 'ready\n' > /tmp/site/health/ready
              cd /tmp/site
              exec python -m http.server 8080
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /ready # Sai: file phục vụ ở /health/ready.
              port: 8080
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
---
apiVersion: v1
kind: Service
metadata:
  name: atlas
spec:

  selector:
    app: atals # Sai: Pod mang app=atlas.
  ports:
    - name: http
      port: 80
      targetPort: 8080
```

Ở bước đầu, apply trả thành công và `get deployment` cho thấy `spec.replicas=2` nhưng `status.availableReplicas=0`. Đây chỉ xác nhận khai báo và trạng thái chưa hội tụ. Xem ReplicaSet và Pod có `ownerReferences`/label đúng để biết Deployment controller và ReplicaSet controller đã tạo gì. Nếu không có Pod, kiểm tra Deployment/ReplicaSet events và quota hoặc admission; đừng nhảy ngay sang mạng. Nếu Pod có `spec.nodeName` rỗng và event `FailedScheduling`, xét requests CPU/memory, node affinity/taint và năng lực node theo đúng

event. Nếu đã có node, scheduler không còn là giả thuyết chính cho lần đó.

Trong dữ kiện tiếp theo, hai Pod có node, container state `Running`, không có event `FailedScheduling` hay `Failed` khi pull image; image ID đã được kubelet ghi. Những quan sát này loại bớt lỗi gán node và tải image cho hai Pod được xem, nhưng không chứng minh mọi lần pull trong tương lai hoặc mọi node đều tốt. Nếu gặp `ErrImagePull` hay `ImagePullBackOff` trong sự cố khác, kiểm tra image reference/digest, credential registry, quyền và đường mạng từ node; event và container waiting reason quan trọng hơn việc đoán từ tên Deployment. Ở ca này, điều kiện Pod `Ready=False` và event readiness báo HTTP 404 cho `/ready`. Ứng dụng trả 200 tại `/health/ready` trong kiểm tra từ đúng network context của Pod. Quan hệ nhân quả có thể kiểm được: kubelet đang thử sai path, nên Pod không trở thành endpoint `ready`. Sửa probe theo hợp đồng ứng dụng, áp dụng bản sửa và đợi Pod thế hệ mới đạt Ready; đừng xóa probe để có màu xanh giả.

Một thứ tự quan sát ngắn là: đọc object và generation → tìm ReplicaSet/Pod do controller tạo → xem `spec.nodeName`, events và container state → so Pod `Running` với điều kiện `Ready` và probe → so Service selector với label → xem tất cả EndpointSlice của Service và điều kiện endpoint → gửi request qua đúng Service. Lệnh thích hợp có thể là `kubectl -n demo describe pod <pod>`, `kubectl -n demo get pod <pod> -o yaml`, `kubectl -n demo get svc atlas -o yaml` và `kubectl -n demo get endpointslices -l kubernetes.io/service-name=atlas -o yaml`. Kết quả từng lệnh là bằng chứng ở thời điểm chạy, không phải suy luận thay cho cluster chưa có. Khi thu log và YAML cần tránh chia sẻ Secret hoặc token.

### Bài 37.3 — Triển khai phiên bản mới

Tôi dè chừng chữ “xong” trên một rollout. Nó có thể chỉ nói controller đã hoàn tất phần việc mà nó được giao, trong khi khách hàng đi qua một Service sai selector hoặc ứng dụng vừa mở cổng đã trả lỗi. Để đọc trạng thái trung thực, bạn cần biết mỗi điều kiện được tính từ gì và điều gì nằm ngoài phạm vi của nó.

Deployment `Available=True` phản ánh mức sẵn sàng tối thiểu theo chiến lược và thời gian `minReadySeconds`, không đồng nghĩa tất cả Pod đều mới. `Progressing=True` có thể còn đang rollout; chỉ riêng giá trị True không chứng minh rollout hoàn tất. Với Deployment v1.34, điều kiện `Progressing=True` kèm reason `NewReplicaSetAvailable` biểu thị rollout đã hoàn tất vào thời điểm controller báo; `kubectl rollout status` cho một phép đợi/kiểm tra phù hợp hơn nhìn một cột. Ngay cả khi rollout hoàn tất, Service selector, cổng, DNS, policy mạng và request thực từ client vẫn cần kiểm chứng. `Available` có thể đổi sau đó khi Pod gặp lỗi; không lấy một ảnh chụp tốt làm cam kết vĩnh viễn.

Các vòng điều khiển chạy bất đồng bộ; có thể tạm thời thấy Deployment mới, ReplicaSet mới nhưng Pod cũ, hoặc status chưa theo kịp spec. Watch có thể đứt, API có thể quá tải, node có thể hết tài nguyên, registry không truy cập được hoặc quyền bị từ chối. Controller có thể thử lại và báo điều kiện lỗi, nhưng không thể tạo tài nguyên vượt quota hay làm ứng dụng lỗi trở nên đúng. `progressDeadlineSeconds` giúp nhận biết rollout không tiến triển; nó không tự rollback một Deployment đã lỗi. Cần định nghĩa ngưỡng dừng, cách sửa hoặc rollback theo bằng chứng và kiểm

tra request sau can thiệp. Mỗi API write và bước điều khiển có thể thành công một phần; reconciliation không phải giao dịch trên cả cluster.

Ngân sách rollout là một bài toán năng lực, không phải nghi thức đổi phiên bản. Hãy thử dừng ở từng bước và hỏi: hiện có bao nhiêu Pod thực sự `Available`, bao nhiêu Pod mới đang tiêu thụ chỗ, và ta được phép mất bao nhiêu khả năng phục vụ? Những con số sau đây chỉ có nghĩa khi gắn với ba câu hỏi đó.

Tính một lượt thay thế giả định để hiểu giới hạn thay vì chỉ nhìn cờ trạng thái. Deployment muốn bốn bản sao, đang có bốn Pod cũ `Available`; cấu hình `RollingUpdate` đặt `maxSurge: 1`, `maxUnavailable: 0`, `minReadySeconds: 10`. Bộ điều khiển có thể tạo một Pod mới, nên có tối đa năm Pod chưa tính Pod đang kết thúc, nhưng phải giữ ít nhất bốn Pod `Available`. Nếu Pod mới `Running` mà readiness thất bại, số bản sao khả dụng vẫn là bốn và không thể loại Pod cũ chỉ để đẩy nhanh rollout. Sau khi Pod mới giữ `Ready` liên tục đủ 10 giây, nó mới được tính `Available`; khi ấy bộ điều khiển có thể giảm một bản sao cũ rồi tiếp tục thay thế. Đây là diễn giải giới hạn và một đường tiến triển có thể xảy ra, không phải log hay thứ tự sự kiện được bảo đảm trên mọi cluster. Pod đang `Terminating` vẫn có thể tiêu thụ tài nguyên, nên ngân sách chỗ trống trên node cần tính cả giai đoạn chồng lấn.

Thử đổi ngân sách thành `maxSurge: 0`, `maxUnavailable: 1`. Controller được phép giảm một Pod cũ trước khi đưa Pod mới vào, nên trong lúc chuyển đổi chỉ cần giữ ba trong bốn Pod khả dụng. Cách ấy tiết kiệm chỗ chồng lấn, nhưng bạn chưa biết ba Pod còn lại có gánh nổi traffic không. Nếu Pod mới trượt readiness, tôi sẽ đặt `updatedReplicas`, `readyReplicas`, `availableReplicas`, ReplicaSet cũ/mới và event cạnh cùng một mốc thời gian. Hết `progressDeadlineSeconds` có thể cho `Progressing=False`, reason `ProgressDeadlineExceeded`; controller vẫn thử tiếp, không tự quay về bản cũ. Quyết định dừng hay rollback còn cần tỷ lệ lỗi, latency và tương thích dữ liệu. Một ngân sách Pod đẹp không chữa được response nghiệp vụ sai.

**Bài tập 37.1 — Available không đồng nghĩa rollout hoàn tất**

Deployment muốn 4 bản sao, ban đầu 4 Pod cũ Available; RollingUpdate đặt maxSurge=1, maxUnavailable=0, minReadySeconds=10. Pod mới Running nhưng readiness thất bại: tối đa bao nhiêu Pod mới có thể được tạo trong đợt đầu và có được giảm Pod cũ không? Nếu sau đó Deployment báo Available=True, Progressing=True, có thể kết luận rollout hoàn tất và người dùng nhận response đúng chưa? Nêu bằng chứng cần xem.

# Chương 38 — Lập lịch và ngân sách tài nguyên

Giải thích Pending vì requests/taint dù CPU tức thời thấp.

### Bài 38.1 — Lựa chọn nút chạy

Một node trông gần như rảnh nhưng Pod vẫn `Pending` là chuyện khiến người mới học hoài nghi scheduler. Tôi lại thấy đây là dịp tốt để phân biệt tài nguyên đã hứa với tài nguyên vừa dùng. Scheduler phải đặt chỗ cho một cam kết tương lai; ảnh chụp CPU hiện tại không xóa được các cam kết ấy.

Chương 36 theo một khai báo từ API tới Pod và Service. Nhưng một Pod chỉ phục vụ ổn định khi đồng thời tìm được node phù hợp, có CPU và bộ nhớ để làm việc, có volume cần thiết và nhận được request qua đường mạng thực. Bốn điều kiện này do những thành phần khác nhau xử lý; `Running` hay một con số replica không gộp chúng thành một bảo đảm. Chương này lần theo từng ranh giới rồi mới dùng metric CPU để hiểu HPA. Các số liệu trong tình huống là dữ liệu giả định có thể tính lại, không phải output từ cluster của người học.

Node công bố `capacity` và `allocatable`. `allocatable` là phần dành được cho Pod sau khi trừ ngân sách dự trữ phù hợp cho hệ thống và kubelet, không phải CPU đang rảnh tức thời. Scheduler lọc các node khả thi bằng requests CPU/bộ nhớ của Pod, tài nguyên còn có thể cấp theo các Pod đã được đặt, ràng buộc volume, taint, affinity và các quy tắc khác; sau đó chấm điểm để chọn node. Một node đang dùng CPU thực tế thấp vẫn có thể bị loại nếu tổng requests đã chiếm hết allocatable. Ngược lại, Pod được đặt không có nghĩa nó luôn có đủ CPU khi nhiều workload cùng tranh chấp.

Đọc dữ kiện A và B như một phép loại trừ, không như bài tìm node có CPU thấp nhất. Với mỗi node, bạn có thể viết một lý do chấp nhận hoặc loại bỏ mà người khác kiểm lại được. Nếu cả hai bị loại, tăng replica chỉ làm hàng đợi dài thêm.

Ví dụ giả định: node A có `allocatable.cpu=2000m`, các Pod đã đặt yêu cầu tổng `1700m`, nên chỉ còn `300m` để nhận request mới. Node B có `2000m`, đã yêu cầu `1400m` nên còn `600m`, nhưng mang taint `dedicated=infra:NoSchedule`. Pod mới yêu cầu `500m` và không có toleration tương ứng: A thiếu `200m`, B bị taint chặn. Kể cả đo usage của A và B chỉ `200m`, Pod vẫn chưa thể đặt theo dữ kiện này. Cần đọc `FailedScheduling` event, requests thực của mọi container, `status.allocatable`, taint và affinity của chính Pod; một event tại một thời điểm không chứng minh nguyên nhân duy nhất của mọi lần thử sau. Toleration chỉ bỏ một rào cản taint, không ép Pod lên B; node affinity dạng `requiredDuringSchedulingIgnoredDuringExecution` là ràng buộc bắt buộc lúc đặt, còn `preferred` chỉ ảnh hưởng lựa chọn khi có node phù hợp. Không nới ràng buộc an toàn hoặc sửa taint của node production chỉ để biến cột Pending thành Running.

Xét một chuỗi quan sát giả định của dịch vụ `atlas` trong namespace `demo`; mỗi ảnh chụp dưới đây là dữ kiện học tập, không phải log thật. Đầu tiên, client thấy timeout. Deployment mong muốn ba replica, một Pod mới Pending với event `FailedScheduling`. Node A còn `300m` theo requests, Pod cần `500m`; node B còn `600m` nhưng có taint mà Pod không toleration. Việc `kubectl top node` cho usage thấp không phủ định hai rào cản này. Hướng xử lý là bố trí năng lực/node phù hợp hoặc xem lại ràng buộc theo yêu cầu vận hành, rồi kiểm tra Pod thực sự được gán node; không vội tăng

HPA max.

**Bài tập 38.1 — Pod Pending dù CPU thực còn rảnh**

Node A còn `300m` allocatable chưa được request; B còn `600m` nhưng có taint `NoSchedule`. Pod yêu cầu `500m` và không có toleration. Vì sao usage thực 10% không giúp scheduler đặt Pod? Nêu quan sát cần kiểm tra trước khi đổi ràng buộc.

**Bài 38.2 — Ngân sách tài nguyên khi chạy**

Sau khi Pod được đặt, một bộ luật khác bắt đầu có tiếng nói: kernel cấp CPU time và bộ nhớ cho tiến trình. Tôi muốn bạn giữ ranh giới này thật rõ. Scheduler có thể chấp nhận Pod rồi container vẫn bị throttle; một Pod đang chạy cũng có thể chết vì OOM. Cùng một biểu hiện chậm từ phía khách hàng, nhưng hồ sơ chứng cứ khác hẳn.

`requests.cpu` dùng để đặt chỗ trên node và thường ảnh hưởng trọng số chia CPU khi cạnh tranh; `limits.cpu` là trần CPU time được kernel áp bằng cgroup trong các khoảng quota. Nếu chương trình cần thêm CPU nhưng bị trần, counter `cpu.stat.nr_throttled` và `throttled_usec` tăng trên cgroup tương ứng; đó là bằng chứng đã bị quota chặn trong cửa sổ đo, chưa chứng minh mọi request chậm đều do CPU. Kết hợp hiệu `usage_usec` để tính số lõi trung bình theo thời gian như Chương 5, cùng latency từng chặng và cửa sổ metric tương ứng. `limits.memory` khác: khi cấp phát không thể đáp ứng dưới giới hạn, kernel có thể giết tiến trình; đối chiếu `OOMKilled`, restart count, event và counter `memory.events` của đúng cgroup, không đọc một giá trị `memory.current` sau khi process đã chết rồi kết luận chưa từng thiếu bộ nhớ. Memory request chủ yếu tham gia scheduling và đánh giá áp lực node; vượt request không tự động bị OOM như vượt limit có thể dẫn tới OOM.

Với QoS truyền thống ở mức container, Pod `Guaranteed` có request và limit CPU lẫn memory bằng nhau cho từng container; `BestEffort` không khai báo chúng; trường hợp còn lại có ít nhất một khai báo phù hợp là `Burstable`. QoS tham gia ưu tiên khi node chịu áp lực và xử lý OOM/eviction, nhưng không thay thế requests/limits, dữ liệu sử dụng thực hoặc policy của kernel. Kubernetes v1.34 còn có tùy chọn tài nguyên ở mức Pod sau feature gate; khi dùng tính năng ấy phải đối chiếu hợp đồng phiên bản thay vì áp máy móc tiêu chí container trên mọi cluster. Pod có thể `Pending` vì không đủ requests, `Running` nhưng bị CPU throttling, hoặc restart vì OOM; cả ba có thể biểu hiện cùng một latency từ phía client nhưng cần bằng chứng khác nhau.

Một ca giả định cho thấy thay một trường khai báo có thể đổi cả quyết định đặt Pod lẫn cách đọc metric. Pod chỉ có một container, khai báo `limits.cpu=1000m` nhưng bỏ `requests.cpu`; giả sử không có admission policy điền request trước đó. Kubernetes dùng chính limit `1000m` làm request. Node chỉ còn `700m` theo tổng requests đã đặt, dù CPU thực dùng lúc ấy chỉ `100m`, nên Pod có thể nhận `FailedScheduling: Insufficient cpu`. Số `100m` không phản bác scheduler: nó xét ngân sách đã hứa, không xét ảnh chụp mức dùng. Nếu sau khi đo nhu cầu và dự phòng, người vận hành đặt request tường minh `250m` và giữ limit `1000m`, rào cản CPU trên node ấy được gỡ; vẫn phải kiểm những ràng buộc khác và sự ổn định khi nhiều Pod cùng tranh CPU. Giảm request chỉ để hết `Pending` mà không đo nhu cầu có thể tạo quá tải về sau.

Nhìn ba counter của cùng cgroup như ba phép đo độc lập, đừng vội biến chúng thành một lời buộc tội CPU. Usage nói đã dùng bao nhiêu CPU time; chu kỳ throttle nói quota từng chặn; latency của request mới nói người dùng chịu hậu quả gì. Cơ chế có thể liên quan, nhưng quan hệ nhân quả cần các cửa sổ thời gian khớp nhau.

Giả sử Pod đã chạy và hai mẫu cách nhau 10 giây của **cùng cgroup** cho hiệu `usage_usec=8.500.000`, `nr_periods=100`, `nr_throttled=15`. Mức dùng trung bình là `8,5/10=0,85` lõi; với request `250m`, tỷ lệ usage/request của mô hình một container là `850/250=340%`, không phải 340% năng lực node. Mức trung bình dưới limit một lõi vẫn tương thích với 15 chu kỳ bị throttle: nhiều luồng có thể dùng hết quota sớm trong một chu kỳ rồi phải đợi phần còn lại. `15/100` đếm chu kỳ có throttle, không phải phần trăm request chậm hay phần trăm CPU đã dùng. Muốn quy latency cho quota, đối chiếu cửa sổ request chậm với hiệu `throttled_usec`, thời gian từng chặng và tải dependency; một counter tăng chỉ củng cố giả thuyết, chưa đóng điều tra.

Nếu cùng dịch vụ sau đó restart và `lastState.terminated.reason` là `OOMKilled`, chuyển sang nhánh bộ nhớ: kiểm request/limit memory của container, event, log trước khi chết và hiệu `memory.events.local` trên **cgroup của lượt chạy bị ảnh hưởng** nếu còn thu được. Counter của cgroup mới sau restart không thể phủ định lần kill ở cgroup cũ; `memory.events.max` tăng cũng chưa tương đương `oom_kill` tăng. Bài tự kiểm là lần lượt dự đoán Pod sẽ bị chặn ở scheduling, bị chậm khi đang chạy hay bị kết thúc; với mỗi dự đoán ghi một quan sát có thể bác bỏ nó. Không đổi cả request, limit và số replica cùng lúc rồi gán hiệu quả cho một nguyên nhân.

**Bài tập 38.2 — Pod chờ lịch chạy và CPU bị giới hạn**

Một Pod một container chỉ khai báo limits.cpu=1000m, không khai báo requests.cpu; không có admission policy điền request. Node còn 700m theo tổng requests. Dự đoán vì sao Pod có thể Pending dù CPU đang dùng thấp. Sau khi đặt request=250m, hai mẫu cùng cgroup cách nhau 10 giây cho hiệu usage_usec=8.500.000 và nr_throttled tăng 15. Tính CPU trung bình, tỷ lệ usage/request và nêu điều 15 lần throttle chưa chứng minh.

**Bài 38.3 — Số đo tài nguyên tối thiểu**

Một con số CPU không có cửa sổ thời gian giống một chiếc đồng hồ chỉ có kim phút mà thiếu mặt số. Tôi muốn bạn hỏi "trong bao lâu?" trước khi hỏi "bao nhiêu phần trăm?". Từ phép chia CPU time cho thời gian thực, ta mới có một đại lượng đủ rõ để so với request và để hiểu HPA ở chương sau.

CPU usage là mức CPU time tiêu thụ trong một khoảng đo: hiệu counter CPU time chia cho thời gian thực. Nếu một container tích lũy thêm 3 giây CPU trong 10 giây tường, trung bình nó dùng `0,3` lõi, tức `300m`. Một mẫu không cho biết request nào chờ lâu, mức dùng giữa hai mẫu hay khả năng sẽ tăng tải sau đó. Metrics Server thường thu resource metrics từ kubelet và trình bày qua `metrics.k8s.io`; HPA lấy metric qua API này, còn `kubectl top` là một cách xem nhanh. Metrics Server là add-on có thể thiếu hoặc trễ; timestamp/cửa sổ mẫu phải được đối chiếu với sự cố. Thiếu request thích hợp khiến utilization của Pod không xác định và HPA không thể quyết định từ metric đó theo cách dự kiến.

Nếu `kubectl top` trả `300m` cho Pod có tổng request CPU thích hợp là `200m`, tỷ lệ theo request trong ví dụ đơn giản là `300/200 = 150%`. Đó không phải 150% của node và cũng không nói Pod cần đúng 1,5 node. Đối chiếu cửa sổ đo, container nào được cộng, request của từng container và trạng thái metric thiếu trước khi liên hệ với HPA. Nếu latency tăng nhưng CPU time giảm, giả thuyết nghẽn CPU yếu đi; cần tìm thêm thời gian chờ I/O, dependency hoặc throttling. Số đo giả định này chỉ dạy phép tính và phép phản chứng, không phải snapshot của cluster thật.

Có một trò đánh lừa rất hiền của phép lấy trung bình: dữ liệu đúng nhưng cửa sổ quá dài sẽ giấu một lần tăng tải vừa xảy ra. Hai cửa sổ ngắn dưới đây cho bạn thấy chính xác phần thông tin bị làm phẳng. Hãy tính trước khi đọc kết quả, rồi tự hỏi HPA đã nhận được mẫu nào.

Đọc hai cửa sổ liên tiếp sẽ rõ chiếc bẫy của counter tích lũy. Với **cùng một cgroup**, `usage_usec` lần lượt là 2.000.000 ở giây 0, 5.000.000 ở giây 10 và 13.000.000 ở giây 20. Bạn tính được `(5−2)/10=0,3` lõi rồi `(13−5)/10=0,8` lõi; trung bình `0,55` lõi trên 20 giây che lần tăng gần đây. Nếu container đã được tạo lại giữa hai mẫu, phép trừ xuyên hai đời cgroup vô nghĩa, kể cả khi nó cho số âm trông rất lạ mắt. Với chỉ container này trong Pod, cửa sổ sau dùng `0,8` lõi: request `200m` cho `800/200=400%`, request `1000m` cho `800/1000=80%`. Công việc không đổi mà tín hiệu HPA đã đổi vì mẫu số. Trước khi scale, tôi muốn xem tuổi mẫu, độ phủ Pod, request có hợp lý và latency cùng cửa sổ; một tỷ lệ cao tự nó chưa chỉ ra nút thắt.

# Chương 39 — Service, EndpointSlice và NetworkPolicy

Lần theo tên Service tới Pod socket và policy chặn.

### Bài 39.1 — Tìm điểm phục vụ

DNS trả lời một địa chỉ, nhưng chưa hề hứa rằng sau địa chỉ ấy có người tiếp khách. Tôi đưa bạn trở lại `atlas`: tên Service, selector, EndpointSlice và socket của ứng dụng là bốn mối nối khác nhau. Đứt ở đâu thì bằng chứng phải lấy ở đó; sửa DNS khi selector sai chỉ làm câu chuyện rối hơn.

Sau khi hai Pod `Ready=True`, Service vẫn không phục vụ. So trực tiếp `spec.selector` của Service với `metadata.labels` của Pod trong cùng namespace: `app=atals` khác `app=atlas`. EndpointSlice gắn với Service không có endpoint khớp. Đây là một lỗi ranh giới chọn backend, không phải lỗi TLS hay CPU. Sửa selector thành `app=atlas`; chờ EndpointSlice có hai địa chỉ Pod với `ready=True`, rồi thử kết nối qua Service port 80 và kiểm tra request ứng dụng từ vị trí client thực. Nếu đã có endpoint `ready` mà request vẫn lỗi, mới tiếp tục xem `port`/`targetPort`, network policy, DNS, đường mạng node và ứng dụng ở cổng đích. Service port 80 chuyển tới 8080 theo cấu hình, không phải do tên `atlas` hay readiness tự suy ra.

Trong cluster có DNS phù hợp, tên Service được phân giải theo namespace thành địa chỉ Service, thường là ClusterIP cho Service thông thường. DNS trả lời chỉ chứng minh bước tìm tên; nó không chứng minh Service có backend. Service selector chọn Pod theo label, EndpointSlice chứa địa chỉ/port và điều kiện endpoint, rồi thành phần dataplane trên node chuyển tiếp kết nối tới Pod phù hợp. `ready=True` của endpoint là tín hiệu để chọn backend bình thường; vẫn phải kiểm tra ứng dụng nghe đúng `targetPort` và request đại diện thực sự thành công. Các triển khai có thể dùng kube-proxy hoặc dataplane khác; không suy ra mọi cluster dùng cùng luật iptables hay cùng kiểu eBPF từ API Service.

Khi rollout đang xóa một Pod, đọc thêm `serving` và `terminating` của EndpointSlice thay vì chỉ đếm địa chỉ. Trong trạng thái thường, `ready` tương ứng với endpoint còn phục vụ và chưa terminating; một endpoint đang terminating có thể `serving=True` nhưng `ready=False`. Service proxy thường bỏ endpoint đang terminating, song có thể đưa traffic tới endpoint vừa `serving` vừa `terminating` nếu mọi endpoint khả dụng đều đang terminating. Ngoại lệ `publishNotReadyAddresses=true` còn làm `ready` không phản ánh phép thử readiness theo cách thông thường. Vì vậy phép đếm hai địa chỉ trong slice không tự chứng minh có hai backend bình thường.

Trong một ảnh chụp **giả định** sau khi sửa selector, endpoint A có `ready=True, serving=True, terminating=False`; endpoint B có `ready=False, serving=True, terminating=True`. Tôi dự đoán traffic mới thông thường sẽ ưu tiên A, nhưng không đóng câu “mọi kết nối chỉ tới A”: proxy có quy tắc riêng nếu tất cả endpoint khả dụng đều đang terminating, còn kết nối đã mở có vòng đời của nó. Nếu client vẫn lỗi, bạn ghép timestamp của slice với timestamp request, rồi kiểm `port`/`targetPort`, backend thực nhận và policy mạng. Một snapshot chụp muộn có thể đã bỏ lỡ trạng thái gây lỗi. Ta có thể luyện phép đọc trên manifest lưu lại; muốn khẳng định hành vi của một cluster cụ thể thì phải đo ở client và dataplane của chính cluster ấy.

### Bài tập 39.1 — Probe đúng nhưng Service vẫn sai selector

Hai Pod `Running`, điều kiện `Ready=False`, readiness HTTP trả 404. Service cũng có selector không khớp label Pod. Sau khi sửa probe, vì sao request qua Service vẫn lỗi? Nêu bằng chứng cần có trước khi kết luận đường Service đã hoạt động.

### Bài 39.2 — Đường truyền dữ liệu

Đi từ tên Service tới Pod không có nghĩa gói tin được đi qua mọi ranh giới. Sau DNS và EndpointSlice còn có dataplane, policy của nguồn và policy của đích. Tôi muốn bạn theo một request cụ thể, vì từ "mạng" quá rộng: nó dễ biến mọi timeout thành một lời giải thích nghe có vẻ đúng mà không kiểm được.

Pod network do plugin tương thích CNI triển khai, gồm cấp địa chỉ và kết nối giữa Pod/node theo mô hình mạng của cluster. CNI là giao diện giữa runtime và plugin, không chỉ định một sản phẩm, một tuyến gói tin duy nhất hay tự mang chính sách firewall. Mặc định, Pod không bị cô lập ở một chiều nếu chưa có policy chọn nó cho chiều đó. Khi đã bị cô lập, các allow rule cộng lại, không phải danh sách lệnh deny theo thứ tự. DNS có thể trả đúng Service IP, EndpointSlice có backend Ready nhưng một policy vẫn chặn client → Pod hoặc Pod → dependency; ngược lại một timeout không tự chứng minh policy là nguyên nhân. So namespace/label, port, chiều ingress và egress, khả năng của plugin, rồi thử từ đúng nguồn tới đúng đích. Có thể có độ trễ giữa API nhận policy và dataplane áp dụng; trạng thái object policy không phải bằng chứng gói tin đã bị chặn tại thời điểm ấy.

Trước khi xem policy, bạn có thể dự đoán hai cánh cửa độc lập: client có được rời nguồn không, và backend có nhận lưu lượng ấy không? Đặt câu hỏi theo từng chiều sẽ tránh cách sửa nửa vời — mở ingress rồi ngạc nhiên vì egress vẫn khóa.

Một đường request giả định bắt đầu từ Pod `client` ở namespace `front`, gọi tên `orders.back.svc` trên TCP 8080. Phân giải DNS trước tiên cần đường egress từ client tới DNS của cluster; nếu tên đã phân giải đúng, bước tiếp theo là Service chọn EndpointSlice có Pod `orders` Ready. Từ đó dataplane chuyển gói tới địa chỉ Pod đích theo triển khai của cluster. Nếu `client` bị cô lập egress, phải có rule cho lưu lượng tới đích; nếu `orders` bị cô lập ingress, phía đích cũng phải cho phép nguồn. Hai chiều này độc lập: mở ingress của `orders` không sửa egress của `client`. Gói trả lời của một kết nối được cho phép có quy tắc xử lý riêng theo NetworkPolicy, không nên tự viết một rule ngược chỉ vì nhìn thấy hai hướng gói.

Bạn chỉ nên chọn phép sửa sau khi biết gói tin dừng ở đâu. Tên không phân giải thì xem DNS và egress tới DNS; tên đúng nhưng Service không có endpoint thì xem selector và readiness. Có endpoint mà TCP timeout, hãy thử từ đúng nguồn tới Pod IP, rồi đối chiếu policy ingress/egress, route giữa node và plugin. Nếu Pod IP đi được còn đường Service lỗi, nghi vấn nghiêng sang Service selector hoặc dataplane của Service, chưa đủ để kết án CNI. Một Pod debug ở namespace khác hoặc mang label khác không đại diện cho client bị policy chọn. Tôi sẽ ghi cả nguồn, đích, cổng và thời điểm cho mỗi phép thử trước khi nói "mạng đã thông".

### Bài 39.3 — Chính sách mạng

Một dấu gạch đầu dòng trong YAML có thể đổi tập đích được phép. Tôi coi NetworkPolicy là bài học về logic tập hợp trước khi coi nó là bài học về firewall. Bạn sẽ thấy cùng hai selector, đặt chung một phần tử thì là giao; tách thành hai phần tử thì là hợp. Sự khác biệt trên giấy có thể là một ranh giới bảo mật ngoài đời.

Ở ảnh chụp thứ ba, DNS trả ClusterIP, EndpointSlice của Service có ba địa chỉ `ready=True`, request trực tiếp từ một Pod khác tới backend đáp, nhưng client Pod ở namespace `web` kết nối qua Service bị timeout. Các phép đo này thu hẹp lỗi về đường từ client tới Service/backend; chúng chưa chỉ ra plugin mạng hay policy nào. Đối chiếu policy chọn client/đích theo đúng namespace và chiều, `targetPort`, route và dataplane thực của cluster. Chỉ khi phép thử trước/sau trên cùng nguồn/đích và quan sát policy implementation khớp mới quy nguyên nhân cho NetworkPolicy. Sau khi request đi được, latency vẫn cao là vấn đề mới, không thể tiếp tục kết luận "mạng lỗi" từ timeout cũ.

`NetworkPolicy` là API chọn Pod và cho phép lưu lượng ingress/egress theo những điều kiện khai báo; nó chỉ có tác dụng khi plugin mạng của cluster hỗ trợ thực thi chính sách.

Ví dụ giả định một egress policy chọn Pod client ở namespace `web` nhưng chỉ cho kết nối tới Pod mang `app=api` trong namespace có nhãn `env=staging`; backend thật lại nằm ở `env=prod`. DNS vẫn có thể trả ClusterIP, còn TCP từ client bị chặn trước khi tới backend. Kiểm selector thực tế của cả Pod lẫn namespace, policy ingress ở phía đích và quy tắc DNS nếu client cần phân giải tên. Một thử nghiệm từ Pod ở namespace khác không thay cho thử từ nguồn bị chọn. Nếu plugin không thực thi NetworkPolicy, YAML tồn tại mà traffic vẫn qua; phải xác nhận triển khai dataplane trước khi quy nguyên nhân.

Đến đây nên tạm bỏ chữ "rule" và lấy bút khoanh các tập Pod. Làm phép giao hoặc hợp trước, sau đó mới hỏi ingress phía đích và plugin thực thi ra sao. Cách đọc ấy chậm hơn nhìn YAML lướt qua vài giây, nhưng nhanh hơn điều tra một quyền truy cập đã mở quá rộng.

Đọc policy như phép toán trên tập Pod, không như danh sách luật firewall chạy theo thứ tự. Trong một phần tử `to` chứa **cả** `namespaceSelector: env=prod` và `podSelector: app=api`, đích là giao của hai tập: chỉ Pod `app=api` thuộc namespace `env=prod`. Nếu tách chúng thành **hai** phần tử của mảng `to`, đích trở thành hợp: mọi Pod trong namespace `env=prod` **hoặc** Pod `app=api` trong namespace của chính policy. Một dấu gạch đầu dòng YAML đặt sai có thể mở rộng quyền truy cập. Khi nhiều policy cùng chọn client theo hướng egress, tập được phép là hợp các rule cho hướng ấy; policy thứ hai không "ghi đè" để thu hẹp policy thứ nhất. Kết nối Pod–Pod chỉ qua được nếu egress của nguồn và ingress của đích đều cho phép, hoặc hướng tương ứng chưa bị cô lập. Vì vậy trong ca trên, sau khi sửa nhãn `staging` thành `prod`, vẫn phải kiểm ingress ở backend trước khi tuyên bố đường đã thông.

Ta thử phép toán bằng dữ kiện giả định: `prod` có Pod `api` và `worker`; `web` có `api` và `frontend`. Policy nằm ở `web`, chọn `frontend`. Nếu một phần tử `to` chứa cả `namespaceSelector` và `podSelector`, tập egress chỉ có `prod/api`. Tách thành hai phần tử, tập ấy nở thành `prod/api`, `prod/worker` và `web/api`. Bạn vừa đổi quyền truy cập mà không đổi một tên Pod nào. Đây mới là kết quả xét selector, chưa phải kết quả kết nối: ingress của đích, port và plugin còn quyết định.

Trong cluster cô lập, có thể đọc nhãn/policy rồi thử cùng một cổng từ đúng `frontend` tới từng Pod. Tôi không suy thứ tự thực thi policy trước hay sau dịch địa chỉ Service từ một lượt gọi ClusterIP, vì dataplane có thể khác nhau.

**Bài tập 39.2 — Timeout từ namespace khác**

DNS trả Service IP, ba endpoint `ready=True`, request từ một Pod nội bộ thành công, nhưng client namespace khác timeout. Hãy thiết kế phép kiểm tra phân biệt DNS, selector, cổng, NetworkPolicy và dataplane; nêu điều không thể suy từ một timeout.

# Chương 40 — PVC, CSI và dữ liệu của ứng dụng

Phân biệt PVC Bound với volume mount và dữ liệu phục hồi được.

## Bài 40.1 — Tài nguyên

Tôi thích hỏi "dữ liệu đang ở đâu?" trước khi hỏi "Pod đã chạy chưa?". Kubernetes có thể tìm chỗ cho tiến trình, cấp volume, gắn volume vào node và mount vào container qua những bước khác nhau. Chữ `Bound` chỉ kể được một đoạn của câu chuyện; nó chưa kể ứng dụng đã đọc đúng file hay chưa.

PersistentVolume (PV) là object tài nguyên lưu trữ ở phạm vi cluster; PersistentVolumeClaim (PVC) là yêu cầu của workload trong namespace, nêu dung lượng, access mode và StorageClass. StorageClass mô tả cách provision volume và chính sách liên quan; provisioner/CSI driver có thể tạo PV động cho PVC. Binding nối một PVC với một PV đáp ứng điều kiện, nhưng sau đó node còn phải attach hoặc mount bằng driver và hạ tầng lưu trữ thật. Khi Pod kẹt `ContainerCreating`, cần xem events `FailedAttachVolume` hoặc `FailedMount`, CSI controller/node plugin và topology của PV trước khi đổ lỗi ứng dụng.

Access mode nói về cách volume được dùng hoặc ghép claim với volume: `ReadWriteOnce` cho phép read-write từ một node, không nhất thiết chỉ một Pod nếu nhiều Pod ở cùng node. `ReadWriteOncePod` giới hạn một Pod, cần CSI driver và sidecar hỗ trợ; `ReadOnlyMany`/`ReadWriteMany` còn tùy năng lực provider. Access mode không tự mã hóa dữ liệu, không tạo quyền file ở trong container, và không phải bản sao dữ liệu. Với `WaitForFirstConsumer`, binding/provisioning được trì hoãn để xét đồng thời yêu cầu của Pod và topology volume; một PVC Pending trước khi có consumer có thể là trạng thái chủ ý. Đặt thẳng `spec.nodeName` bỏ qua scheduler có thể khiến PVC kiểu này Pending mãi; cần chọn cơ chế ràng buộc node qua scheduler. Nếu topology không có node phù hợp, thêm replica chỉ tạo thêm Pod Pending.

Với PVC `ledger`, mỗi mốc chuyển trạng thái là một cửa kiểm tra. Bạn hãy dự đoán tác nhân tiếp theo phải làm gì trước khi xem event; cách đó giúp ta không sửa CPU request khi lỗi thực nằm ở mount, và không xóa dữ liệu khi mới thấy một trạng thái Pending hợp lệ.

Đọc một sự cố theo các bước của vòng đời lưu trữ. PVC `ledger` yêu cầu `10Gi`, StorageClass chọn `WaitForFirstConsumer`, Pod cần volume ấy và có ràng buộc node ở một zone cụ thể. Trước khi Pod được scheduler xét, PVC Pending chưa đủ kết luận thiếu dung lượng: hệ thống đang đợi consumer để chọn topology. Sau khi có Pod, nếu event báo không tìm được node đáp ứng cả yêu cầu Pod lẫn volume, kiểm affinity, taint, zone và năng lực provision của StorageClass. Nếu PVC đã Bound nhưng Pod còn `ContainerCreating`, chuyển sang attach/mount và CSI events; sửa request CPU sẽ không chữa một lỗi `FailedMount`. Nếu Pod đã Ready nhưng ứng dụng đọc dữ liệu cũ, kiểm mount path, volume name, PVC UID và nội dung tại mount trong container; trạng thái Bound không chứng minh ứng dụng dùng đúng đường dẫn.

Một phép dự đoán quan trọng: hai Pod ở cùng node dùng chung PVC `ReadWriteOnce` có thể được mount cùng lúc tùy driver và workload; chuyển Pod thứ hai sang node khác có thể làm attach thất

bại vì giới hạn một node. Không lấy "hai Pod đều chạy" làm bằng chứng rằng storage hỗ trợ nhiều node, cũng không lấy `ReadWriteOnce` làm khóa ghi giữa hai tiến trình. Nếu ứng dụng cần một writer duy nhất, cần ràng buộc workload và cơ chế đồng bộ dữ liệu thích hợp. Nếu cần truy cập đồng thời nhiều node, phải chọn loại volume và access mode thật sự được storage backend hỗ trợ, rồi thử failover trong môi trường riêng.

Để kiểm chứng mà không chạm dữ liệu production, tạo PVC và Pod thử với tên riêng, ghi StorageClass, binding mode, access mode, `volumeHandle`, zone/node được chọn và events theo thứ tự thời gian. Viết một marker trong volume thử, thay Pod rồi đọc lại marker; sau đó thử khôi phục từ backup riêng để phân biệt persistence với recoverability. Không xóa PVC thật để chứng minh chính sách reclaim: hành vi xóa phụ thuộc PV reclaim policy và backend, có thể phá dữ liệu. Khi thiếu cluster hoặc CSI driver phù hợp, chỉ giải tình huống từ object và event mẫu, ghi rõ không có kiểm chứng runtime.

### Bài 40.2 — Thực thi

Có những trạng thái nhìn rất yên tâm trên bảng điều khiển mà vẫn chưa chạm được dữ liệu. PVC `Bound` là một ví dụ: quan hệ với PV đã được ghi, còn đường từ thiết bị đến thư mục ứng dụng vẫn có thể đứt. Bài này đi chậm qua từng mối nối, bởi một thao tác "thử xóa rồi tạo lại" ở đây có giá đắt.

Sau khi có node phù hợp, PVC dùng StorageClass `WaitForFirstConsumer` mới được chọn theo topology. PVC Bound nhưng Pod tiếp tục kẹt `ContainerCreating`, event giả định báo `FailedMount` từ CSI. Điều đã biết: binding hoàn tất; điều chưa biết: volume đã attach/mount và chứa dữ liệu đúng. Kiểm tra volume handle, node/topology, driver và sự kiện theo UID Pod/PVC. Không xóa claim hoặc PV để "thử lại" khi chưa có bản sao và chính sách reclaim rõ ràng. Sau khi mount được xác nhận, Pod có thể Running và Ready, nhưng timeout của client vẫn cần chẩn đoán đường dữ liệu.

`PVC Bound` không chứng minh file đã mount, quyền đọc/ghi đúng, dữ liệu có nội dung mong đợi hay có bản backup.

Một phép kiểm an toàn theo lớp là ghi lại UID của PVC/PV/Pod và StorageClass, rồi đối chiếu event `FailedMount` với node đã được chọn; không dùng tên hiển thị đơn lẻ vì tài nguyên cùng tên có thể đã được tạo lại. Khi lỗi mount hết, xác nhận mount path trong container và đọc một file kiểm chứng không phá dữ liệu. Nếu muốn thử ghi, dùng vị trí và nội dung test tách biệt đã được ứng dụng cho phép. Chỉ khi đường lưu trữ, quyền, nội dung và đường request đều được kiểm tra mới nói phục hồi dịch vụ; hành động xóa PV để ép tạo mới có thể làm mất bằng chứng và dữ liệu của sự cố ban đầu.

Tôi đề nghị bạn đọc timeline này như hai câu chuyện song song: control plane biết gì về volume, và node thật đã làm gì với volume? Mốc 09:01 thuộc câu chuyện thứ nhất; event 09:02 mới mở câu chuyện thứ hai. Ghép nhầm chúng là cách rất nhanh để tự tin sai.

Giả sử timeline của một Pod mới như sau: 09:00 PVC `ledger` ở Pending, 09:01 scheduler chọn node cùng zone rồi PVC chuyển Bound, 09:02 Pod kẹt `ContainerCreating` với event `FailedMount`, 09:04 người vận hành thấy Deployment chưa Available. Mốc 09:01 cho biết một PV đã được gắn về mặt control plane; nó không chứng minh CSI node plugin đã mount được thiết bị

hoặc đường filesystem có dữ liệu đúng. Nếu sự kiện nêu timeout khi mount, hỏi CSI node plugin và đường tới storage; nếu nêu permission khi ứng dụng mở file sau khi Pod đã chạy, chuyển sang UID/GID và mode trong mount. Hai triệu chứng ở hai lớp khác nhau, không nên cùng chữa bằng tăng dung lượng PVC.

Khi đọc event, ghép theo UID và timestamp chứ không ghép theo tên Pod một mình. Controller có thể xóa Pod lỗi và tạo Pod mới cùng nhãn, làm event cũ dễ bị áp nhầm vào lần chạy mới. Ghi PVC UID, PV name, volume handle, node, topology và driver; so trạng thái attach trên controller với trạng thái mount trên node. Nếu Bound nhưng driver trên node không có hoặc topology không phù hợp, kết luận cần sửa cơ chế gắn volume, không sửa readiness probe. Ngược lại, mount đã thành công nhưng client vẫn timeout thì kiểm EndpointSlice, Service, route và ứng dụng như các bài mạng trước.

Tôi chỉ gọi Pod đã phục hồi sau ba phép kiểm riêng: volume được mount vào đúng path, ứng dụng đọc được marker hoặc bản ghi kỳ vọng, và Pod thay thế trong lab vẫn đọc lại được sau khi Pod đầu dừng. Ba phép ấy chứng minh persistence qua vòng đời Pod trong điều kiện thử, chưa chứng minh backup. Recoverability cần một lượt restore từ bản sao độc lập vào volume thử, rồi đối chiếu checksum và nội dung ở đường đọc ứng dụng. Không có cluster hoặc CSI phù hợp thì chỉ có thể kết luận trên timeline giả định; đừng biến một bài suy luận thành lời kể rằng mount đã chạy thành công.

**Bài tập 40.1 — PVC Bound nhưng Pod không gắn được volume**

Một Pod có PVC Bound nhưng kẹt `ContainerCreating` với event `FailedMount`. Có thể kết luận dữ liệu đã an toàn và application đã lỗi không? Phân biệt binding, attach/mount, access mode và kiểm tra dữ liệu.

**Bài 40.3 — Vòng đời**

Một Pod mới có thể giống Pod cũ ở tên ứng dụng mà không mang theo một byte dữ liệu nào của lượt chạy trước. Điều ấy không làm Kubernetes kém hữu ích; nó nhắc ta rằng vòng đời tiến trình và vòng đời dữ liệu là hai hợp đồng khác nhau. Tôi muốn bạn kiểm cả hai hợp đồng trước khi gọi một lần thay Pod là "phục hồi".

Thay Pod chỉ phục hồi số bản sao của tiến trình theo template. File trong writable layer của container mới không tự mang dữ liệu container cũ; `emptyDir` sống theo Pod và mất khi Pod bị loại khỏi node. PersistentVolume/PVC có vòng đời và chính sách thu hồi riêng, nhưng gắn volume không tự tạo backup nhất quán hoặc đảo được một lần ghi sai. Trước khi xóa/recreate Pod hoặc rollback image trong hệ thống có dữ liệu, xác nhận dữ liệu nằm ở đâu, bản sao/backup có thể restore, và phiên bản ứng dụng cũ có đọc được dữ liệu mới hay không. Kiểm chứng cuối cùng là request và dữ liệu nghiệp vụ được phục vụ từ đúng đường đi, không chỉ một hàng `Running` hoặc `Available`.

StatefulSet cho Pod danh tính ổn định và có thể tạo PVC riêng theo ordinal qua `volumeClaimTemplates`, hữu ích khi mỗi replica cần dữ liệu riêng. Nó không tự biến một volume một AZ thành lưu trữ HA, không tự backup, không bảo đảm database replica đã đồng bộ. Reclaim policy như `Retain` hoặc `Delete`, chính sách giữ PVC của StatefulSet, snapshot và backup phải được kiểm tra riêng trước khi scale down hoặc xóa. Phục hồi phải thử ở đường đọc dữ liệu nghiệp

vụ, không dừng ở việc PVC chuyển sang Bound.

Với `ledger-1`, tên ổn định rất dễ tạo cảm giác an toàn. Hãy thử coi nó chỉ là một đầu mối để điều tra: PVC nào, UID nào, volume handle nào, bản ghi nào? Bằng chứng dữ liệu phải đi đến tận đường đọc của ứng dụng, không dừng ở việc Pod thứ hai xuất hiện lại.

Ví dụ **giả định** StatefulSet `ledger` có hai Pod `ledger-0`, `ledger-1`; mỗi ordinal có một PVC riêng từ template `data`. Scale từ hai xuống một làm Pod `ledger-1` ngừng chạy, nhưng không được suy rằng PVC và bản ghi của ordinal 1 đã mất hoặc chắc chắn được giữ: kiểm chính sách `persistentVolumeClaimRetentionPolicy` của StatefulSet và chính sách thu hồi của PV, rồi ghi lại UID của claim/PV trước thao tác. Khi tăng lại lên hai, cùng tên `ledger-1` chưa đủ chứng minh nó đang đọc đúng bản ghi cũ; đối chiếu UID claim, volume handle, mount và một bản ghi kiểm chứng qua ứng dụng. Nếu bản ghi mới từng được ghi chỉ trên ordinal 1 mà chưa nhân bản, Pod 0 vẫn Ready cũng không chứng minh dữ liệu đó còn phục vụ. Đây là phép suy luận trên cấu hình giả định, không phải một lượt scale đã chạy trên cluster.

**Bài tập 40.2 — Pod mới không mang theo emptyDir**

Một Pod bị xóa rồi ReplicaSet tạo Pod khác; dữ liệu chỉ nằm trong `emptyDir` của Pod cũ. Hệ thống đã tự phục hồi điều gì, chưa phục hồi điều gì? Đề xuất hậu điều kiện cho một ứng dụng giữ dữ liệu trước khi cho phép thay Pod trên production.

# Chương 41 — HPA và điều khiển mở rộng

Tính replica từ CPU utilization khi biết request và chất lượng metric.

## Bài 41.1 — Phép đo

HPA không nhìn vào một chiếc đồng hồ CPU chung của node rồi tự biết nên thêm bao nhiêu Pod. Nó đọc metric của workload và so với một mẫu số đã khai báo. Tôi nhắc lại phép chia này từ đầu vì một mẫu số sai có thể khiến toàn bộ câu chuyện scale nghe hợp lý mà vẫn sai.

Không lấy CPU `limits.cpu` hoặc tổng CPU node làm mẫu số của CPU utilization trong HPA: với target kiểu utilization, mẫu số là CPU `requests.cpu` của các container liên quan.

Trước khi giải thích một quyết định tăng replica, phải đặt cùng một cửa sổ đo cho ba đại lượng: CPU đã dùng của Pod, CPU request dùng làm mẫu số và số replica đủ điều kiện trong phép tính. Một mức sử dụng bằng 80% request không có nghĩa dùng 80% toàn node. Nếu request đặt quá thấp, tỷ lệ có thể cao dù lượng CPU tuyệt đối nhỏ; nếu metric vắng hoặc đã cũ, kết quả không thể được xem như số 0.

HPA điều chỉnh số replica theo metric và cấu hình mục tiêu; hành động đó vẫn phải đi qua Deployment, scheduler và khả năng khởi động Pod mới. Một đề xuất replica cao hơn chưa chứng minh request của người dùng đã nhanh hơn. Khi truy lỗi, kiểm tra dữ liệu metric và thời điểm lấy mẫu trước, sau đó xem desired/current/ready replica cùng sự kiện của Pod. Nếu Pod mới không Ready, nguyên nhân có thể nằm ở đường khởi động, probe hoặc phụ thuộc ứng dụng, không tự nằm trong công thức HPA.

Với metric tài nguyên, HPA hỏi API metric của cluster; một cách triển khai thường gặp là Metrics Server cung cấp `metrics.k8s.io`. Đường này khác với việc người vận hành nhìn thấy số CPU trong dashboard Prometheus: dashboard có dữ liệu không chứng minh HPA đọc được API metric mà nó cần. Theo chiều ngược lại, HPA đọc được metric cũng chưa chứng minh metric có độ trễ và độ phủ thích hợp cho quyết định vận hành. Hãy kiểm lần lượt sự hiện diện của API metric, mẫu theo từng Pod/container, thời điểm mẫu, CPU request khai báo và condition của HPA.

Bạn hãy thử tự điền mẫu số của từng container trước khi đọc điều kiện HPA. Nếu thiếu một request, đó không phải ô trống để thay bằng limit hoặc số không. Chính sự thiếu dữ kiện ấy là một trạng thái cần chẩn đoán.

Giả sử Deployment có ba Pod và một sidecar không khai báo CPU request. Khi target HPA dùng CPU utilization ở mức Pod, phép chia usage/request không xác định cho Pod liên quan; không được coi request thiếu là 0m rồi tính ra vô cực, cũng không được lấy CPU limit thay vào mẫu số. Cần bổ sung request có cơ sở đo, hoặc chọn metric và phạm vi container phù hợp nếu thiết kế workload cho phép. Khi HPA còn một metric khác hợp lệ, kết luận "toàn bộ HPA dừng" cũng quá mạnh; phải đọc điều kiện và quyết định thật của controller. Trong lab cô lập, đối chiếu `kubectl top pods -n demo` với `kubectl describe hpa -n demo`, rồi kiểm các `resources.requests.cpu` trên Pod template; không diễn giải output mẫu thành bằng chứng cluster đã được chạy ở đây.

**Bài tập 41.1 — CPU theo request không phải CPU của node**

CPU 80% request có nghĩa dùng 80% node không?

**Bài 41.2 — Vòng tính**

Tôi không muốn công thức HPA trở thành phép bói số replica. Nó hữu ích nhất khi ta biết dữ liệu đầu vào, cửa sổ đo và các bước bảo thủ mà controller có thể áp sau phép tính cơ sở. Vì vậy hãy tự tính trường hợp đủ ba mẫu trước; sau đó bỏ đi đúng một mẫu và xem trực giác của mình có còn đúng không.

Ví dụ tính toán giả định, tất cả ba Pod đã Ready và có metric hợp lệ trên cùng cửa sổ 10 giây. Mỗi Pod khai báo `requests.cpu=500m`, target utilization là 50%, giới hạn HPA từ 2 đến 5 replica. Hiệu CPU time của ba Pod lần lượt là 3,0; 4,5 và 1,5 giây. Chia cho 10 giây cho usage `300m`, `450m`, `150m`; chia cho request `500m` cho utilization 60%, 90%, 30%. Trung bình là `(60+90+30)/3=60%`. Công thức cơ sở cho ba replica là `ceil(3 × 60/50)=ceil(3,6)=4`. Tỷ số 1,2 nằm ngoài tolerance mặc định 10% của controller v1.34; 4 cũng nằm trong khoảng 2–5. Nếu `limits.cpu=1000m` cho mỗi Pod, con số limit ấy không đổi phép tính utilization. Đây là một phép tính lý tưởng trên dữ kiện đầy đủ, không phải lời hứa controller đã tạo đúng một Pod mới ở lượt quan sát ấy.

HPA là vòng điều khiển chạy theo chu kỳ và ghi số replica mong muốn qua `scale` subresource; Deployment/StatefulSet và scheduler mới tạo/đặt Pod. Trước quyết định cuối, controller loại Pod đang xóa hoặc Failed, xét Pod chưa Ready và metric thiếu, tính lại theo giả định bảo thủ để giảm mức scale, rồi áp tolerance, min/max, chính sách tốc độ scale và cửa sổ ổn định khi scale down. Status HPA có thể hiển thị trung bình metric ban đầu, khác tỷ số đã điều chỉnh để quyết định. Nếu metric thiếu, sample cũ, Pod vừa khởi động hay nhiều metric cùng cấu hình, không lấy mỗi công thức cơ sở để đoán số replica cuối. HPA không tăng node allocatable, không sửa PVC, DNS hoặc NetworkPolicy; nó có thể tạo thêm Pod Pending khi ràng buộc đặt chỗ vẫn tồn tại. Khi HPA quản lý số replica, tránh liên tục apply một `spec.replicas` cố định trong manifest vì hai bộ điều khiển sẽ tranh cùng giá trị.

Đến đây tôi mời bạn dừng bút: nếu chỉ còn hai mẫu, bạn có lấy trung bình hai Pod rồi scale cả ba không? Thuật toán không được phép tự xem Pod thứ ba là bằng chứng của 0 CPU thực. Nó dùng một giả định bảo thủ cho quyết định, và sự khác nhau ấy giải thích nhiều màn hình HPA tưởng như tự mâu thuẫn.

Thử mất **riêng metric của Pod thứ ba** trong đúng ví dụ trên; cả ba Pod vẫn tồn tại, Ready, request `500m` và chỉ hai Pod đầu có mẫu hợp lệ `60%`, `90%`. Nếu lấy trung bình hai mẫu rồi áp công thức cơ sở cho ba replica, ta được `75%` và `ceil(3 × 75/50)=5`: nhìn riêng status HPA có thể tưởng cần tăng mạnh. Nhưng trong nhánh scale-up, thuật toán v1.34 giả định Pod thiếu metric dùng **0% giá trị mục tiêu** khi tính lại một cách bảo thủ. Trung bình điều chỉnh thành `(60+90+0)/3=50%`, tỷ số với target `50%` bằng 1, nên **không có đề xuất tăng từ phép tính này**. Số 0 ở đây chỉ là giả định điều khiển để tránh scale quá mức, không phải phép đo chứng minh Pod thứ ba không dùng CPU. Nếu metric của nó trở lại ở `30%` với cùng cửa sổ và không có điều kiện khác cản trở, phép tính đầy đủ lại đề xuất bốn replica như trên. Một status vẫn hiện `75%` có thể cùng tồn tại với quyết định chưa scale vì status báo

trung bình ban đầu, không phải giá trị đã điều chỉnh.

Phép chẩn đoán phải lấy đúng thời điểm: liệt kê Pod thuộc scale target, CPU request và timestamp metric từng Pod; kiểm API metric, điều kiện/sự kiện HPA rồi đọc desired/current/ready replicas sau một chu kỳ điều khiển. Dashboard Prometheus có hai đường CPU không chứng minh Metrics API đã cấp đủ ba mẫu cho HPA. Nếu API thiếu mẫu của Pod ba, ưu tiên phục hồi độ phủ metric và so quyết định sau đó; không tăng `maxReplicas` hoặc đổi target chỉ vì status `75%`. Ví dụ chỉ tính nhánh một metric CPU, không dự đoán chính xác hành vi của cluster có metric khác, policy scale riêng hoặc cấu hình tolerance khác mặc định.

**Bài tập 41.2 — Tính số replica từ CPU HPA**

Ba Pod cùng request `500m` CPU; trong 10 giây mỗi Pod dùng 3,0; 4,5; 1,5 giây CPU. Target HPA 50%, min=2, max=5. Tính usage, utilization và desired replicas cơ sở; giải thích vì sao controller có thể chưa tạo đủ Pod hoặc request vẫn chậm.

**Bài tập 41.3 — HPA thiếu một mẫu CPU**

Ba Pod cùng request 500m, HPA target 50% và hiện có ba replica. Pod A/B có utilization 60%/90%, Pod C Ready nhưng thiếu metric CPU. Tính trung bình và replica cơ sở từ hai mẫu, rồi tính lại theo giả định bảo thủ của HPA v1.34 cho nhánh scale-up. Status có thể hiện gì, và phép quan sát nào phân biệt "thiếu mẫu" với "CPU bằng 0"?

**Bài 41.3 — Hội tụ**

Scale là một đề nghị tăng năng lực, không phải năng lực đã có. Tôi muốn bạn đi theo Pod thứ tư từ quyết định HPA đến node, volume, readiness, endpoint và request thật. Chỉ ở cuối đường ấy mới có cơ sở hỏi người dùng đã bớt chờ chưa.

Cuối cùng, ba Pod Ready có các hiệu CPU time 3,0; 4,5; 1,5 giây trên 10 giây như Bài 41.2. Phép tính cho khuyến nghị cơ sở bốn replica, nhưng phải kiểm tra timestamp metric, điều kiện HPA, hiệu counter throttling và latency từng chặng trước khi nói CPU là nút thắt. Nếu HPA ghi desired=4 mà Pod thứ tư Pending vì requests hoặc PVC, HPA đã đổi ý định nhưng chưa tăng năng lực phục vụ. Hậu điều kiện là Pod mới Scheduled, volume mount, Ready, EndpointSlice cập nhật và request từ client có latency/response đạt yêu cầu qua nhiều cửa sổ, cùng với dữ liệu vẫn nguyên vẹn. Một giá trị `desiredReplicas=4` riêng nó chưa chứng minh điều đó.

Từ cùng cửa sổ giả định 10 giây, ba Pod đã dùng tổng cộng 9 giây CPU, tức 0,9 lõi cộng gộp; chia cho ba Pod được 0,3 lõi mỗi Pod. Ở cấu hình chính vừa nêu, mỗi Pod request 0,5 lõi nên utilization trung bình là 60% và đề xuất cơ sở là bốn replica. **Biến thể riêng:** nếu chỉ thay request thành 0,2 lõi mỗi Pod, giữ nguyên usage, target 50% và giới hạn 2–5 replica, utilization trung bình thành 150%; công thức cơ sở cho `ceil(3 × 150/50)=9`, rồi bị `maxReplicas=5` chặn ở 5 trước các điều chỉnh khác. Con số 150% không cho biết node còn CPU trống hay request người dùng bị chậm ở I/O. HPA dùng metric và request theo thuật toán, còn scheduler xét từng Pod với requests khi đặt chỗ. Vì vậy một thay đổi HPA có thể hợp lý theo CPU mà không tạo năng lực khi node thiếu chỗ. Đây là phép tính trên số liệu giả định của bài, không phải số đo cluster thật.

Đổi request là một thao tác hai mặt: nó đổi tỷ lệ utilization mà HPA nhìn thấy, đồng thời đổi ngân sách scheduler dùng để đặt Pod mới. Ta sẽ giữ nguyên CPU time trong hai biến thể để bạn nhìn rõ tác động của riêng mẫu số, rồi kiểm xem quyết định scale có chuyển thành khả năng phục vụ không.

Hãy nối ba bộ điều khiển thay vì dừng ở công thức. HPA có thể ghi desiredReplicas=4; Deployment tạo Pod thứ tư; scheduler không đặt được vì requests mới cộng với các Pod hiện tại vượt allocatable của node có zone phù hợp; kubelet chưa bao giờ chạy container đó. Trong trường hợp ấy `currentReplicas`, `readyReplicas` và số EndpointSlice thực phục vụ phải được đọc riêng. Nếu Pod được đặt nhưng `FailedMount`, nút thắt chuyển sang CSI như bài trước. Nếu Pod Ready nhưng latency không giảm, kiểm phân phối traffic, throttling của Pod cũ, thời gian upstream và p95 theo route. Mỗi quan sát loại một lớp giả thuyết, không tự chứng minh CPU là nguyên nhân ban đầu.

Nếu thiếu Metrics API hoặc cluster, hãy dừng ở hai biến thể tính trên giấy và ghi rõ runtime chưa kiểm. Khi có lab thật, tôi sẽ đặt timestamp của từng mẫu CPU cạnh request của từng container, target, tolerance và condition HPA; rồi chụp desired/current/ready cùng event của Pod mới trước và sau cửa sổ scale. Hiệu `nr_throttled`, thời gian throttle và latency request phải được so trên cùng khoảng, không biến một counter tích lũy thành tỷ lệ. Hậu điều kiện đủ sức thuyết phục là Pod mới Scheduled, Ready, có endpoint nhận request và latency nghiệp vụ tốt lên qua vài cửa sổ. Dòng `desired=4` chỉ là đầu một hành trình.

**Bài tập 41.4 — HPA tăng Pod nhưng thiếu nơi chạy**

Pod mới do HPA yêu cầu Pending do thiếu node phù hợp, trong khi một Pod cũ tăng `nr_throttled` và `oom_kill` không đổi. Hãy tách hai ranh giới tài nguyên, những phép đo tiếp theo và một quyết định vận hành có hậu điều kiện rõ.

# Chương 42 — Gateway API và ranh giới yêu cầu ngoài cụm

Phân biệt controller conditions với đường request của client.

## Bài 42.1 — Đối tượng Gateway API

Một request từ ngoài cluster không quan tâm ta đã đặt tên object đẹp đến đâu; nó chỉ đi theo địa chỉ mà client biết, bắt tay với listener thật và tìm một backend nhận được nó. Tôi dùng Gateway API để chia rõ người quản lý cửa vào và người khai báo tuyến, còn Helm để giữ một nhóm cấu hình thành bản phát hành. Hai công cụ giải hai việc khác nhau.

Chương 36, Chương 38 và Chương 39 cho thấy object được API nhận, Pod được đặt và Service có endpoint vẫn là những sự kiện khác nhau. Khi người dùng ở ngoài cluster gửi request, còn có ranh giới định tuyến vào cluster. Helm giúp tạo và cập nhật một nhóm object theo bản phát hành; Gateway API khai báo nơi lưu lượng đi vào và cách chọn backend. Helm không phải bộ định tuyến, còn Gateway API không tự chạy proxy. Để một request thành công, bản phát hành, controller Gateway, dataplane, Service, Pod và ứng dụng phải cùng thỏa điều kiện tại thời điểm thử.

Gateway API là nhóm API mở rộng: cluster cần CRD tương ứng và một triển khai controller hỗ trợ phiên bản, loại Route và tính năng đang dùng. `GatewayClass` là object phạm vi cluster, chỉ ra `controllerName` của triển khai. Quản trị viên có thể chuẩn bị class; tên class trong bản khai báo không làm controller xuất hiện. `Gateway` là object có namespace, chọn class và khai báo các `listener`: cổng, protocol, hostname, chính sách Route được phép gắn, và khi cần thì cấu hình TLS. Controller được chọn có thể tạo load balancer mới, sửa proxy hiện có hoặc lập trình hệ thống mạng khác; API không quy định một proxy cố định cho mọi triển khai.

Hãy thử đọc Route từ phía request thay vì từ phía YAML: nó đến địa chỉ nào, mang Host nào, khớp listener nào, rồi mới tới luật path và backend. Mỗi câu trả lời có một object hoặc một phép đo riêng. Một Route được API lưu chưa tự trả lời được cả chuỗi.

`HTTPRoute` mô tả một hay nhiều luật HTTP. `parentRefs` chọn Gateway, có thể chỉ rõ listener qua `sectionName`; Gateway listener cũng phải cho phép Route gắn vào. `hostnames` và `matches.path` thu hẹp request phù hợp, còn `backendRefs` chỉ nơi gửi tiếp, thường là Service cùng namespace và cổng Service. Địa chỉ DNS của client phải tới địa chỉ thực của Gateway; HTTP `Host` hoặc TLS SNI phải phù hợp với listener và Route. Có Route đúng mà DNS trỏ tới một proxy cũ thì request vẫn không đi qua nó. Có Gateway nhận request mà không có Route phù hợp thì phản hồi có thể là 404 do dataplane; cần kiểm tra status, request và cấu hình triển khai trước khi quy nguyên nhân từ mã HTTP.

Với HTTPS termination, listener của Gateway dùng TLS Secret để bắt tay với client; sau điểm giải mã, HTTPRoute có thể so hostname/path và chuyển request tới backend. Secret thường ở cùng namespace với Gateway; tham chiếu Secret ở namespace khác cần grant phù hợp và hỗ trợ từ triển khai. SNI, chứng chỉ, tên host và nơi chấm dứt TLS phải được xét cùng nhau. TLS ở Gateway không mặc nhiên mã hóa chặng Gateway → Service/Pod. Ví dụ chart dưới đây cố ý chỉ dùng HTTP để không giả vờ có chứng chỉ hoặc bảo đảm bảo mật của chặng bên trong; muốn dùng trong hệ thống thật phải thiết kế TLS và ranh giới tin cậy riêng.

Gateway tạo listener HTTP cổng 80, giới hạn Route cùng namespace. Namespace lấy từ release để tên Gateway và Route không bị tách nhầm.

**Bài tập 42.1 — Thiếu GatewayClass sau khi render chart**

Chart đã render Gateway và HTTPRoute hợp lệ về YAML, nhưng GatewayClass chưa tồn tại. Dự đoán thứ tự bằng chứng cần đọc trước khi chẩn đoán 404 từ client. Điều gì chưa thể kết luận từ `helm template`?

**Bài 42.2 — Gắn tuyến vào cổng**

Có hai quyền dễ bị trộn lẫn: quyền cho Route gắn vào Gateway, và quyền để Route tham chiếu backend ở namespace khác. Tôi muốn bạn giữ chúng như hai chiếc khóa khác nhau. Mở khóa thứ nhất không tự mở khóa thứ hai; mở quá rộng cả hai chỉ để hết lỗi cấu hình thì lại biến một bài routing thành sự cố phân quyền.

Quan hệ gắn Route có hai phía: Route gọi tên parent, còn listener giới hạn loại Route và namespace qua `allowedRoutes`. Mặc định thông dụng là chỉ Route cùng namespace; trong ví dụ dưới đây `from: Same` được viết tường minh. Nếu Route ở namespace khác, Gateway có thể cho phép namespace bằng `allowedRoutes`, nhưng điều đó không tự cho Route quyền trỏ tới Service ở namespace thứ ba. Tham chiếu chéo namespace tới backend hoặc TLS Secret cần `ReferenceGrant` tại namespace chứa object đích, với nguồn và đích được khai báo cụ thể. Trường hợp Route ở namespace khác gắn Gateway là ngoại lệ của cơ chế ReferenceGrant và do `allowedRoutes` điều khiển. Không mở `allowedRoutes: All` hoặc tạo grant rộng chỉ để làm mất lỗi cấu hình; quyền gắn Route là quyền chiếm một phần bề mặt lưu lượng.

`GatewayClass.status.conditions` cho biết class có được triển khai chấp nhận hay không. Trên Gateway, đọc `Accepted`, `Programmed`, status của từng listener và địa chỉ được cấp nếu có. `Accepted=True` nghĩa cấu hình được controller nhận ở mức tương ứng; `Programmed=True` nói controller đã lập trình cấu hình xuống dataplane theo hợp đồng của nó, không chứng minh DNS, firewall, TLS hay Pod phía sau hoạt động. Trên HTTPRoute, đọc `status.parents` đúng parent/controller, điều kiện `Accepted` và `ResolvedRefs`; `ResolvedRefs=False` có thể chỉ backend hoặc tham chiếu khác không tồn tại hay không được phép. Một Route có nhiều parent có thể được parent này nhận nhưng parent kia từ chối. Luôn so `observedGeneration` trong condition với `metadata.generation` trước khi dùng status để đánh giá bản spec mới.

HTTPRoute gắn đúng listener qua `parentRefs`, so hostname và chuyển path bắt đầu bằng `/` tới cổng Service. `backendRefs.port` là cổng Service, không phải `targetPort` của Pod:

Đặt ba namespace `edge`, `shop`, `payments` lên giấy trước khi đọc tiếp. Vẽ một mũi tên Route → Gateway và một mũi tên Route → Service; ghi ai cấp quyền cho mỗi mũi tên. Khi status báo lỗi, bản vẽ nhỏ này sẽ giúp bạn hỏi đúng nơi thay vì thay đổi bừa mọi grant.

Ví dụ giả định có Gateway `edge/public`, HTTPRoute `shop/orders` và Service `shop/orders-api`. Nếu listener chỉ cho Route cùng namespace, `parentRefs` trỏ đúng tên Gateway vẫn chưa đủ vì Route thuộc `shop`, Gateway thuộc `edge`; phải cho phép namespace `shop` theo `allowedRoutes` phù hợp. Sau khi Route được gắn, backend cùng namespace `shop` không cần

`ReferenceGrant` cho chính tham chiếu ấy. Nếu đổi backend thành Service ở namespace `payments`, quyền gắn Route vào Gateway vẫn không cấp quyền trỏ backend: namespace `payments` phải có grant thích hợp cho nguồn `shop`. Mở quyền một phía không thay phía còn lại.

Đọc `status.parents` của đúng Gateway sau mỗi lần đổi: `Accepted=False` ở parent này gợi ý kiểm điều kiện gắn listener; `ResolvedRefs=False` gợi ý kiểm tên, cổng, namespace và grant của tham chiếu. Một parent khác có thể có kết quả khác. Chỉ dùng condition có `observedGeneration` tương ứng với spec vừa áp dụng; kể cả các condition đã True, cần kiểm request thật và đường tới backend để kết luận ứng dụng phục vụ. Các trạng thái trên là nhánh dự đoán để đọc status thực, không phải output đã quan sát từ cluster.

**Bài tập 42.2 — HTTPRoute khác namespace và ReferenceGrant**

HTTPRoute ở namespace `app` trỏ tới Gateway ở namespace `edge`, listener chỉ cho Route cùng namespace. Một kỹ sư thêm ReferenceGrant ở `edge` cho Route này. Vì sao Route có thể vẫn không attach? Nếu Route còn trỏ Service ở namespace `data`, cần cơ chế quyền nào khác?

**Bài tập 42.3 — Route được nhận nhưng backend chưa hợp lệ**

Sau upgrade, `HTTPRoute.status.parents` cho parent đúng có `Accepted=True`, `ResolvedRefs=False`, còn Service dự kiến không có trong namespace của Route. Suy luận nhánh lỗi hợp lý và phép kiểm tra tiếp; vì sao không nên chỉ tăng timeout?

### Bài 42.3 — Đường đi yêu cầu

Một mã 503 ở mép hệ thống nghe giống câu trả lời, nhưng thực ra mới là lời mời điều tra. Tôi muốn biết nó do dataplane nào trả, sau khi đã chọn Route hay trước khi tới backend, và ở thế hệ cấu hình nào. Chỉ khi gắn mã lỗi với đường đi cụ thể, ta mới tránh việc sửa Pod trong khi DNS còn dẫn khách đến một cổng cũ.

Ingress vẫn là API Kubernetes đang tồn tại và vẫn được controller tương ứng triển khai; Gateway API mở rộng mô hình vai trò Gateway/Route và cho phép chia quyền cấu hình rõ hơn. Không đổi một Ingress thành Gateway chỉ bằng cách đổi `kind`. Cần kiểm tra loại Route, tính năng được controller cụ thể hỗ trợ, đối tượng TLS và cách dataplane nhận lưu lượng. Cả Ingress lẫn Gateway đều cần một controller thực thi; sự tồn tại của object trong API server chỉ xác nhận bản khai báo được lưu.

Nếu host và attachment đều đúng nhưng nhận 503, lần tiếp theo là `backendRefs` → Service/port → EndpointSlice có endpoint Ready → Pod readiness → ứng dụng. Trong nhiều triển khai, không có backend khỏe có thể hiện thành 503, nhưng mã ấy không tự định danh lỗi: proxy khác, policy, timeout kết nối backend hoặc ứng dụng cũng có thể góp phần. `ResolvedRefs=False` gợi ý Service/Secret sai tên, sai namespace hoặc thiếu grant; `Accepted=False` trên Route gợi ý Route chưa attach, `Programmed=False` trên Gateway gợi ý controller/dataplane chưa hoàn tất. Nếu tất cả condition đều hiện True nhưng request timeout, kiểm tra địa chỉ và DNS, đường client → Gateway, firewall, listener thực tế, rồi thử từ vị trí gần backend để tách chặng lỗi. Một status True có `observedGeneration` cũ không được dùng để minh oan revision 5.

Hai địa chỉ `192.0.2.20` và `192.0.2.10` làm bài toán bớt mơ hồ, nhưng khác nhau chưa đủ kết tội DNS. Hãy giữ nguyên Host, SNI, path và vị trí client khi đổi riêng địa chỉ đích. Một cặp thử như thế nói nhiều hơn mười lần nhìn lại status xanh.

Ví dụ chẩn đoán giả định: Gateway báo địa chỉ đã bind `192.0.2.20`, còn client phân giải `api.example.test` thành `192.0.2.10` và timeout. Tôi sẽ ghi DNS tại chính client, địa chỉ kết nối thực, Host/SNI và timestamp trước khi đổi cấu hình. `Programmed=True` ở generation mới không kiểm được những dữ kiện ấy. Nếu được phép, thử cùng Host/SNI và path qua `192.0.2.20`; đường trực tiếp thành công nhưng đường theo tên thất bại làm DNS hoặc tuyến tới địa chỉ cũ đáng nghi hơn. Khác địa chỉ chưa phải bằng chứng root cause: NAT, DNS chia vùng và lớp cân bằng tải vẫn có thể can dự. Cả hai đường timeout thì quay sang listener, firewall và đường client; đã tới Gateway nhưng nhận HTTP lỗi thì đọc Route, Service và backend.

**Bài tập 42.4 — Gateway xanh nhưng client vẫn timeout**

Người dùng báo timeout nhưng Gateway `Programmed=True` ở generation mới, HTTPRoute Accepted/ResolvedRefs True và Service có endpoint Ready. Hãy thiết kế phép thử phân biệt đường client tới Gateway, Gateway tới Pod và lỗi ứng dụng; chỉ rõ giới hạn của mỗi condition.

# Chương 43 — Gói Helm (Helm chart) và vòng đời bản phát hành

Theo chart/values đến manifest/revision rồi phân biệt với trạng thái thật.

## Bài 43.1 — Gói mô tả Helm

Một chart được render tốt có thể là một bản thiết kế đẹp nằm trên bàn: chưa ai cấp đất, chưa ai lắp đường điện, chưa ai thử mở cửa. Tôi dùng hình ảnh ấy đúng một đoạn rồi quay về các hợp đồng thật: Helm tạo manifest, API chấp nhận object, controller lập trình Gateway, client mới gửi request. Đừng để một bước giả làm bốn bước.

Một chart là gói đầu vào. `Chart.yaml` chứa tên và version của chart, `values.yaml` đặt giá trị mặc định, còn `templates/` là mẫu được Helm render bằng Go template và values cuối cùng. Giá trị do người dùng truyền qua `-f` hoặc `--set` có thể ghi đè mặc định; `--reuse-values` khi upgrade có thể kéo theo một giá trị cũ mà người sửa chart tưởng đã bỏ. Bản YAML ấy chưa được API server chấp nhận, chưa có CRD/controller và chưa thể nói request sẽ đi đâu.

Chart `atlas-gateway` chỉ tạo Gateway và HTTPRoute trong namespace của release; nó trỏ tới Service `atlas:80` đã có từ Bài 37.2. Đây là ví dụ cấu hình, chưa có kết quả chạy trên cluster. Trước khi thử, quản trị viên phải có Gateway API CRD, `GatewayClass` tên `example-gateway` do một controller thật quản lý, namespace `demo`, Service `atlas` và backend Ready. Chữ `example-gateway` không phải tên class mặc định của Kubernetes. Manifest Bài 37.2 không đặt namespace; nếu thử nó trong cluster cô lập đã được phép, chạy `kubectl -n demo apply -f` với đường dẫn tệp đó, sau khi đã hiểu hai lỗi cố ý và sửa trên bản sao thử nghiệm. Đừng áp manifest lỗi lên workload đang phục vụ.

`Chart.yaml` và `values.yaml` định nghĩa đầu vào, không chứa credential:

```
apiVersion: v2
name: atlas-gateway
description: Teaching example for an existing Atlas Service
type: application
version: 0.1.0
appVersion: "1.0"

# A matching GatewayClass and the atlas Service must already exist.
gatewayClassName: example-gateway
hostname: atlas.example.test
backend:
  service: atlas
  port: 80
```

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: {{ .Release.Name }}-web
  namespace: {{ .Release.Namespace }}
spec:
  gatewayClassName: {{ .Values.gatewayClassName | quote }}
  listeners:
    - name: http
      protocol: HTTP
      port: 80
      hostname: {{ .Values.hostname | quote }}
      allowedRoutes:
        namespaces:
          from: Same

apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: {{ .Release.Name }}-route
  namespace: {{ .Release.Namespace }}
spec:
  parentRefs:
    - name: {{ .Release.Name }}-web
      sectionName: http
  hostnames:
    - {{ .Values.hostname | quote }}
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
        - name: {{ .Values.backend.service | quote }}
          port: {{ .Values.backend.port }}
```

Trên máy có Helm 3, `helm lint` và `helm template atlas-edge ./examples/gateway-helm --namespace demo` kiểm tra chart và cho manifest dự kiến. Thử ghi đè hostname bằng một values file thử nghiệm rồi so bản render; kiểm tra `metadata.namespace=demo`, `atlas-edge-web`, tên listener `http`, host và backend. Không dùng render cục bộ như bằng chứng API/CRD/controller chấp nhận. Nếu triển khai trong cluster được cho phép, cần kiểm tra GatewayClass đã Accepted, Gateway/listener đã Programmed, HTTPRoute parent Accepted/ResolvedRefs, địa chỉ Gateway, DNS trỏ đúng địa chỉ và request có `Host: atlas.example.test` từ một client xác định. Không tự cài CRD, controller hay tạo Gateway ngoài môi trường được giao chỉ để chứng minh ví dụ sách.

Tôi luôn hỏi “values nào?” khi nghe câu “chart đã đúng”. Default, file ghi đè và giá trị kéo lại từ revision cũ có thể cho ba manifest khác nhau dù template không đổi. Bài toán hostname dưới đây là một cách kiểm tra câu hỏi ấy mà chưa cần cluster.

Giá trị hiệu lực là một **đầu vào phiên bản hóa của release**, không chỉ `values.yaml` trong chart. Trong fixture, chart mặc định có `hostname: atlas.example.test`; revision 4 từng được cài với override `hostname: old.example.test`. Khi revision 5 đổi chart nhưng người triển khai dùng

`--reuse-values` và không truyền hostname mới, giá trị cũ có thể được mang sang bản render mới. Vì vậy câu "source chart đã đặt host đúng" chưa đủ để dự đoán Route. Ghi chart version/digest, các file `-f` theo thứ tự, cờ `--set`, chế độ reuse/reset và giá trị cuối được Helm dùng; so `helm template` với đúng đầu vào ấy và `helm get manifest` của revision sau triển khai. `helm get values --all` giúp thấy giá trị tính cả mặc định, nhưng manifest đã render mới cho biết trường Kubernetes thực được gửi.

Một phép thực hành không cần cluster có thể render ba lần trên bản sao chart: mặc định, override host bằng file, rồi override lần nữa bằng tham số dòng lệnh; so `spec.listeners[*].hostname` và `spec.hostnames` của HTTPRoute. Nếu listener và Route nhận hai host khác nhau do template lấy hai key khác nguồn, YAML vẫn có thể hợp cú pháp nhưng không phục vụ host mong muốn. Bài thử render chứng minh logic template theo đầu vào đã ghi, chưa chứng minh controller hỗ trợ GatewayClass hoặc DNS thật. Giữ file values fixture không chứa credential để output render có thể được review an toàn.

**Bài tập 43.1 — Giá trị hiệu lực khi nâng cấp chart**

Chart mặc định hostname atlas.example.test; revision 4 ghi đè thành old.example.test. Revision 5 dùng chart mới với --reuse-values nhưng không truyền hostname. Dự đoán host được render và nêu ba chứng cứ cần so trước khi nói client sẽ dùng được host mới.

**Bài 43.2 — Bản phát hành Helm**

Trong một lần nâng cấp, chữ `deployed` của Helm có thể xuất hiện trước khi khách hàng nhìn thấy trang đúng. Tôi không xem đó là lời hứa sai của Helm; công cụ đã báo trong phạm vi nó theo dõi. Việc của người vận hành là nối bản render, object live và request thật bằng cùng revision và cùng thời điểm.

`--wait` có thể chờ một số điều kiện Ready của tài nguyên Kubernetes trong thời hạn cấu hình, nhưng không thay thế request đại diện qua Gateway, kiểm tra dữ liệu hay xác nhận backend nghiệp vụ. Một chart render đúng vẫn có thể fail khi API server không có CRD hoặc cấm quyền; một upgrade được ghi nhận vẫn có thể để controller Gateway từ chối Route hay Pod chưa Ready. Vì vậy cần giữ riêng bốn ảnh chụp: chart/values đã chọn, manifest của revision, trạng thái object hiện tại và phản hồi từ đúng client.

`helm lint` kiểm tra chart ở mức tĩnh; `helm template` cho thấy YAML render từ release name, namespace và values đã chọn mà không cần cluster.

Trong một lần nâng cấp giả định, `helm template` tạo HTTPRoute hợp lệ về cú pháp, nhưng Gateway controller đặt điều kiện `Accepted=False` vì listener không cho phép Route từ namespace ấy. `helm upgrade --wait` có thể hoàn tất phạm vi chờ của các tài nguyên mà nó biết mà client vẫn nhận lỗi. Chứng cứ cần nối revision Helm với manifest đã render, điều kiện của Route theo `observedGeneration`, endpoint backend và một request đi qua đúng hostname/listener. Nếu thiếu quyền đọc status của Route, ghi UNKNOWN thay vì suy từ Pod Ready. Việc rollback chart cũng không tự đảo các ghi dữ liệu do phiên bản mới đã thực hiện.

Bốn lớp này là một phương pháp ghi chép sự cố, không phải bốn lệnh phải chạy cho đủ. Ở mỗi lớp, bạn viết một câu có thể bị chứng cứ bác bỏ. Nếu bằng chứng dừng ở lớp hai, đừng lấy màu xanh của Helm để tô tiếp lớp ba và bốn.

Lần theo bốn lớp với một release giả định `orders-r7`. Lớp thứ nhất là đầu vào render: chart version, release name, namespace và values hiệu lực; đổi một value có thể đổi selector hoặc hostname dù source template không đổi. Lớp thứ hai là manifest đã lưu cho revision r7, phải đối chiếu với YAML đã xem trước vì môi trường và tham số render có thể khác. Lớp thứ ba là trạng thái API sau controller xử lý: HTTPRoute có `metadata.generation` mới nhưng status `observedGeneration` còn cũ thì điều kiện Accepted chưa mô tả spec mới nhất. Lớp thứ tư là traffic thật qua Gateway tới backend. Mỗi lớp trả một câu hỏi; không dùng kết quả `helm template` để thay cho status hoặc request.

Nếu `Accepted=False` ở generation mới và lý do chỉ vào namespace listener không cho phép, sửa phạm vi listener/Route theo chính sách quản trị, rồi chờ controller quan sát generation mới. Nếu `Accepted=True` nhưng request vẫn lỗi, kiểm `ResolvedRefs`, Service và EndpointSlice của backend; đừng tiếp tục đổi listener khi bằng chứng đã chuyển lớp. Nếu mọi status xanh nhưng body sai, kiểm digest workload và dữ liệu mà bản mới đọc. Một phép request thành công từ pod nội bộ có thể bỏ qua hostname, TLS và listener công khai, nên hậu kiểm phải đi qua đúng đường của client mục tiêu.

Để rút ngắn điều tra, dùng **cặp phép thử chỉ đổi một ranh giới**. Với cùng Host, path và payload, thử qua địa chỉ Gateway mà DNS công bố và qua địa chỉ mà Gateway status báo, từ cùng client được phép. Nếu chỉ đường theo DNS lỗi, kiểm DNS/cache/tuyến trước khi sửa chart. Nếu cả hai tới Gateway nhưng Route `Accepted=False` cho generation mới, sửa attachment theo đúng parent; nếu Route Accepted và ResolvedRefs nhưng EndpointSlice không có backend Ready, điều tra selector/probe của Service–Pod. Nếu EndpointSlice có backend nhưng request qua Gateway lỗi còn request trực tiếp tới Service trong cluster đúng, nghi listener/dataplane/network giữa Gateway và Service. Các nhánh là giả thuyết có điều kiện, không suy nguyên nhân chỉ từ một mã 404/503.

Một condition cần được đọc kèm **thời điểm nó mô tả**. Helm revision 7 có thể ghi manifest mới ở T1, Gateway controller vẫn báo `observedGeneration` cũ ở T2, còn DNS TTL giữ địa chỉ cũ tới T3. Chụp ba màn hình khác thời điểm rồi ghép thành câu "mọi thứ xanh" là sai. Ghi revision, `metadata.generation`, `observedGeneration`, địa chỉ thực client kết nối, endpoint Ready và timestamp cho cùng một cửa sổ. Nếu một bước chưa quan sát thế hệ mới, trạng thái là chờ/UNKNOWN trong giới hạn timeout, chưa phải bằng chứng lỗi của application. Nếu timeout qua mà trạng thái không tiến, chuyển sang event/log của đúng controller, không tăng `--wait` vô hạn để biến thiếu chứng cứ thành PASS.

Bạn có thể bắt đầu mà chưa có cluster: render chart với hai bộ values, so Route, Service và selector, rồi dự đoán condition controller sẽ kiểm. Khi có cluster cô lập, tôi sẽ ghi Helm revision, generation/observedGeneration, condition và response của một request có ID. Thiếu Gateway controller thì đánh dấu SKIP cho thử nghiệm ấy; thiếu quyền đọc status thì ghi UNKNOWN, không tự điền một output tưởng tượng. Rollback Helm đưa manifest về nội dung cũ trong phạm vi nó quản lý, còn schema và dữ liệu do revision mới tạo cần phương án riêng. Cách ghi chép này có vẻ chậm

hơn một câu “upgrade đã xong”, nhưng nó cho phép người kế tiếp kiểm lại kết luận.

**Bài tập 43.2 — Lần theo lỗi qua Gateway và backend**

Revision 7 đã ghi manifest; Route Accepted=True nhưng observedGeneration còn cũ, EndpointSlice có backend Ready và DNS client trỏ địa chỉ khác Gateway status. Có thể kết luận backend lỗi không? Sắp thứ tự phép quan sát để thu hẹp nguyên nhân.

**Bài 43.3 — Phục hồi bản phát hành**

Rollback nghe như quay ngược thời gian, nhưng hệ thống không có chiếc đồng hồ ấy. Helm có thể đưa manifest do nó quản lý về nội dung cũ bằng một revision mới; request đã trả, message đã phát và dữ liệu đã ghi vẫn thuộc hiện tại. Tôi muốn bạn phân biệt điều gì thực sự được đảo với điều gì chỉ được hy vọng sẽ ổn.

Khi `helm install` tạo release, Helm gửi manifest đã render đến API server và ghi lại lịch sử release. `helm upgrade` render lại với chart/values mới và tạo revision tiếp theo; `helm history` và `helm get manifest` giúp phân biệt revision đã ghi với cấu hình đang có trong cluster. Chart version, `appVersion`, revision và image digest là bốn danh tính khác nhau; một revision mới có thể chỉ đổi hostname, không thay image. `helm rollback` tạo một revision mới dựa trên revision cũ, rồi cập nhật các object Helm quản lý; nó không quay ngược cơ sở dữ liệu, request đã xử lý, dữ liệu trên volume hay thay đổi bên ngoài chart. `helm uninstall` bỏ các object trong phạm vi release theo quy tắc Helm, không phải lệnh dọn mọi dữ liệu liên quan. Giá trị, template và manifest có thể được giữ trong release metadata, thường là Kubernetes Secret của Helm 3; Secret không phải nơi an toàn để ghi credential vào chart/values rồi đưa output `helm template` hoặc `helm get` lên log công khai.

Trước khi chọn rollback, hãy ghi lại bốn danh tính: chart, revision, manifest và image digest. Chúng không đồng nghĩa. Trong ca hostname này, đổi revision có thể chỉ đổi đường vào, trong khi ứng dụng phía sau giữ nguyên; bằng chứng phục hồi vì thế phải là request từ đúng client, không phải chỉ là số revision mới.

Giả sử revision 4 của release `atlas-edge` dùng host `atlas.example.test` và Service `atlas:80`; một request kiểm thử qua địa chỉ Gateway đã thành công. Revision 5 được tạo bằng `helm upgrade` với file values mới đặt `hostname: api.example.test`. Lệnh hoàn tất và hai object mới tồn tại, nhưng DNS của khách hàng vẫn đưa `atlas.example.test` tới đúng Gateway. Đây là dữ liệu giả định, không phải log Helm, kubectl hoặc proxy đã chạy. So chart và values được dùng cho revision 5 với `helm get manifest` của revision đó: cả listener lẫn Route nay chọn `api.example.test`. Request mang Host cũ không khớp cấu hình mới, nên một phản hồi 404 tại Gateway là giả thuyết mạnh; vẫn phải xác định chính gateway nào đã trả mã và điều kiện listener/Route ở generation mới. Sửa values cho đúng host và upgrade có kiểm tra, hoặc rollback sang revision đã biết rồi xác nhận request cũ; chỉ nhìn chữ `deployed` của Helm không đủ.

Khi rollback, ghi rõ revision đích, manifest cũ, các object live trước/sau và thời điểm lưu lượng chuyển. Nếu revision 5 kèm migration schema hay đã ghi dữ liệu vào backend, đổi Route về host cũ không khôi phục dữ liệu. Hậu điều kiện cần có request từ client ở đúng hostname/SNI và path,

response mong đợi, EndpointSlice còn backend Ready, và kiểm tra dữ liệu nếu ứng dụng có ghi. Đây là phép xác nhận một lần tại thời điểm đo, không phải bảo đảm lâu dài.

`--atomic` trên một lần `helm upgrade` yêu cầu Helm tự quay lại thay đổi khi **lượt nâng cấp báo thất bại theo điều kiện mà Helm theo dõi**; nó cũng bật cơ chế chờ tài nguyên tương ứng. Từ đó không suy "giao dịch ứng dụng nguyên tử". Nếu manifest của revision 5 được API nhận và các tài nguyên thuộc phạm vi chờ đều đạt trạng thái Helm xét, nhưng hostname mới làm request người dùng thất bại, lệnh có thể vẫn thành công và không có rollback tự động. Ngay cả khi Helm phát hiện lỗi và rollback manifest, request đã trả cho client, bản ghi đã commit và message đã phát không quay lại. Vì vậy gate ngoài Helm phải gửi request đại diện qua đúng Gateway và so dữ liệu; nếu gate thất bại, người vận hành chọn rollback/forward fix cùng kế hoạch dữ liệu riêng.

Hãy lập **sổ revision** cho tình huống host: revision 4 có chart/values, manifest và request qua `atlas.example.test` đã biết tốt; revision 5 có đầu vào render, manifest host mới, điều kiện Gateway/Route theo generation và request từ client đã thất bại; revision 6 có thể là kết quả rollback về nội dung revision 4 nhưng mang số revision mới. Không nói "quay lại revision 4" theo nghĩa lịch sử bị xóa. So manifest revision 6 với cấu hình mong muốn, rồi so object live và request thực; nếu một tài nguyên ngoài chart đã đổi trong lúc đó, rollback Helm không sở hữu nó. Bản ghi này cho phép giải thích chính xác điều gì được phục hồi ở lớp khai báo và điều gì còn chờ ở lớp traffic/dữ liệu.

**Bài tập 43.3 — Helm rollback và tương thích dữ liệu**

Revision 7 đổi hostname và image, ứng dụng ghi schema mới. helm upgrade --atomic hoặc helm rollback báo hoàn tất. Có thể kết luận request và dữ liệu đã phục hồi không? Nêu ba hậu điều kiện và giới hạn của --atomic.

# PHẦN XII — Quan sát hệ thống và dữ liệu đo

## Dữ liệu quan sát được tạo, vận chuyển và diễn giải có giới hạn nào?

Metric tổng hợp, log sự kiện và trace đường request là các phép chiếu có sai lệch lấy mẫu, mất dữ liệu và khác thời gian.

XII

**LỘ TRÌNH PHẦN**


- Chương 44 — Phép đo, nhãn và chuỗi thời gian
- Chương 45 — Prometheus, PromQL và Grafana
- Chương 46 — Nhật ký, vết truy cập và OpenTelemetry
- Chương 47 — Cảnh báo và sức khỏe hệ thống quan sát

# Chương 44 — Phép đo, nhãn và chuỗi thời gian

Định nghĩa phép đo và rủi ro sai lệch trước khi xem dashboard.

## Bài 44.1 — Đo

Một metric chỉ có ý nghĩa khi biết loại, đơn vị, nhãn và điểm tăng của nó. Counter diễn tả số tích lũy có thể reset khi tiến trình khởi động lại; để hỏi tốc độ trong một cửa sổ phải xét chênh lệch theo thời gian theo cách nhận biết reset. Gauge mô tả giá trị có thể tăng hoặc giảm, ví dụ số công việc đang chờ ở thời điểm lấy mẫu. Không gọi tổng của các gauge ở các thời điểm khác nhau là một tốc độ.

Classic histogram gồm các bucket tích lũy theo ngưỡng `le`, `_sum` và `_count`. Vì bucket tích lũy, số ở ngưỡng lớn đã bao gồm quan sát của các ngưỡng nhỏ hơn; cộng tất cả bucket sẽ đếm lặp. Có thể tổng hợp bucket cùng ranh giới từ nhiều target trước khi ước lượng quantile, nhưng kết quả phụ thuộc các ngưỡng đã chọn. Summary có thể xuất quantile do client tính; không thể cộng trực tiếp các quantile của nhiều target để có quantile toàn hệ thống. Trước khi vẽ biểu đồ, kiểm tra đơn vị giây/millisecond và population của nhãn để tránh một con số đẹp nhưng sai nghĩa.

Ví dụ histogram giả định trong một cửa sổ: `_count` tăng 100, `_sum` tăng 22 giây; bucket `le="0.1"` tăng 60, `le="0.5"` tăng 90 và `le="+Inf"` tăng 100. Thời gian trung bình là `22/100=0,22` giây. Có 90/100 request không quá 0,5 giây trong cửa sổ; không cộng `60+90+100`, vì các tập con lồng nhau. Ta chỉ biết ít nhất 10 request vượt 0,5 giây, chưa biết p95 chính xác: không có ranh giới bucket tiếp theo để xác định nó nằm ở đâu. Kể cả khi có thêm bucket, quantile từ histogram là ước lượng phụ thuộc vị trí biên và giả định nội suy.

Trước khi dùng con số ấy để quyết định SLO, xác nhận counter không reset giữa hai ảnh chụp, các bucket cùng nhãn và cùng cửa sổ, và request lỗi có được tính hay không. Một dashboard lọc bỏ `status="500"` có thể cho latency đẹp hơn trong khi độ sẵn sàng xấu đi. Đây là bộ đếm tự đặt để luyện tính, không phải metric đã scrape từ hệ thống thật.

### Bài tập 44.1 — Vì sao không lấy trung bình p95?

Vì sao không lấy trung bình p95 của hai Summary?

## Bài 44.2 — Tập hợp

Với classic histogram, mỗi bucket đếm tích lũy số quan sát tới ngưỡng của nó.

Mỗi tổ hợp tên metric và giá trị nhãn tạo một time series. Gauge có thể tăng hoặc giảm theo trạng thái hiện tại, như số request đang chạy. Counter cộng dồn và có thể về giá trị nhỏ khi tiến trình khởi động lại; không lấy một giá trị counter đơn lẻ làm tốc độ. Một request nằm trong mọi bucket có ngưỡng lớn hơn hoặc bằng thời gian của nó; bucket `+Inf` phải bằng `_count`. Chương này chỉ tính p95 từ classic histogram, không trộn với native histogram.

Chọn nhãn để phân nhóm hữu ích như `service`, `route`, `status_class`; tránh `request_id` hoặc user ID thay đổi theo từng request. Nếu 50.000 request tạo 50.000 giá trị `request_id` cho một metric, riêng chiều nhãn đó có thể tạo khoảng 50.000 chuỗi thay vì vài chuỗi theo status; nhân thêm instance, method và bucket còn tăng số chuỗi, bộ nhớ và chi phí truy vấn. Đây là phép đếm tổ hợp

giả định, không phải benchmark Prometheus. Không đưa secret hay định danh người dùng vào nhãn chỉ để lọc dashboard.

Giả sử một counter được scrape ở ba thời điểm: `120` rồi `180` rồi `5` sau khi tiến trình khởi động lại. Hiệu `5-120` âm không phải tốc độ request âm; đó là dấu hiệu cần xét reset và khoảng thời gian. Với hai mẫu đầu cách nhau 60 giây, phép tính thô `(180-120)/60=1` request/giây chỉ mô tả đúng cửa sổ và chuỗi ấy. Muốn đánh giá cả một dịch vụ, phải chọn cùng tập nhãn, xử lý reset theo hàm truy vấn phù hợp và xem scrape có thiếu mẫu không. Số liệu này hoàn toàn giả định; nó dạy cách đọc counter, không phải kết quả đo production.

Để kiểm một histogram, trước tiên đọc một snapshot của cùng một tập nhãn, không trộn instance hoặc route. Ví dụ giả định các bucket `le="0.1"`, `le="0.5"`, `le="+Inf"` lần lượt là 12, 40, 50 và `_count=50`. Dãy phải không giảm và `+Inf` bằng `_count`. Có 12 request ở tối đa 0,1 giây, thêm 28 request trong khoảng `(0,1;0,5]`, và 10 request trên 0,5 giây. Cộng thẳng `12+40+50` sẽ đếm lặp một request nhiều lần. `_sum` lưu tổng thời gian quan sát để tính trung bình khi đủ dữ kiện; nó không cho biết riêng p95. Nếu bucket cuối khác `_count`, phải kiểm tên metric, nhãn, cửa sổ và exporter trước khi tính quantile.

Tại cửa sổ có reset `180 → 5`, giá trị thật sau reset không xác định từ hai mẫu nếu không biết thời điểm reset và số sự kiện sau đó. Một hàm như `rate()` ước lượng tốc độ từ nhiều mẫu và điều chỉnh reset của từng series, nhưng thiếu scrape hoặc cửa sổ quá ngắn vẫn làm ước lượng kém ổn định. Tính tốc độ riêng theo `instance`, `route` và `le` trước khi cộng nhóm; cộng counter của hai tiến trình rồi mới đưa vào `rate()` có thể làm reset của một tiến trình bị che bởi tiến trình kia. Ghi cả tập target tham gia để không hiểu nhầm lưu lượng giảm do một target biến mất thành người dùng ít đi.

Cardinality là phép nhân, không chỉ là một nhãn xấu. Nếu có 5 route, 4 status class, 3 instance và 6 bucket, riêng bucket series có thể lên tới `5×4×3×6=360` trước các nhãn khác; thêm 50.000 `request_id` độc nhất biến thiết kế thành tập chuỗi tăng theo request. Các số là phép tính giới hạn trên với mọi tổ hợp có mặt, không phải lượng series Prometheus đã đo. Chọn nhãn theo câu hỏi vận hành cần trả lời, rồi đo số chuỗi thật và ngân sách lưu trữ sau khi instrument.

**Bài tập 44.2 — Request ID làm tăng số chuỗi metric**

Một thay đổi thêm `request_id` vào metric HTTP và Grafana báo chậm. Tính bậc số chuỗi nếu có 50.000 ID khác nhau, hai instance, hai status class và bốn bucket; đề xuất nhãn thay thế và cách kiểm chứng sau sửa. Giả sử mọi tổ hợp đều xuất hiện; giải thích vì sao tích này chỉ là ước lượng trần theo giả định.

### Bài 44.3 — Thu thập

Ở Chương 41, HPA dùng CPU utilization tính từ CPU usage và request. Đó chỉ là một tín hiệu điều khiển. Người vận hành cần biết phép đo được tạo ở đâu, có tới nơi lưu trữ không, câu truy vấn biến mẫu rời rạc thành đại lượng nào và lúc nào dữ liệu không còn đủ để kết luận. Một biểu đồ đẹp không sửa được một lần scrape thất bại hay một nhãn sai.

Ứng dụng hoặc exporter đo một sự kiện rồi giữ giá trị theo tên metric và tập nhãn. Target công bố snapshot qua HTTP; Prometheus chọn target bằng cấu hình hoặc service discovery, định kỳ scrape

endpoint và gắn timestamp cho mẫu nhận được. Metric `up` do Prometheus tạo cho kết quả scrape: 1 khi scrape thành công, 0 khi thất bại với target vẫn còn trong danh sách. `up=1` chỉ xác nhận việc lấy metric, không chứng minh request nghiệp vụ qua Gateway thành công. Target bị loại khỏi discovery là trường hợp khác với target còn được theo dõi nhưng `up=0`.

Prometheus lưu mẫu thành các chuỗi thời gian trên đĩa cục bộ với write-ahead log và block; thời hạn lưu do cấu hình retention chi phối. Retention không biến dữ liệu thành backup: lỗi đĩa hoặc xóa TSDB vẫn cần kế hoạch sao lưu/độ sẵn sàng riêng. Rule cũng chịu ảnh hưởng của dữ liệu thiếu và thứ tự đánh giá, không xác nhận nguyên nhân gốc.

Hãy tách bốn câu hỏi khi một panel bỗng trống: ứng dụng có phát metric không, target còn trong discovery không, scrape có thành công không, và truy vấn có chọn đúng chuỗi không. Giả sử lúc 12:00 job `api` có hai target A và B. Lúc 12:05 A vẫn được discovery liệt kê nhưng endpoint metric từ chối kết nối: `up` của A sẽ phản ánh scrape thất bại. Cùng lúc B bị loại khỏi discovery: không còn một chuỗi `up` hiện hành cho B để so với 0. Một panel lọc `up == 0` có thể báo một lỗi dù thực tế có hai đường quan sát bị mất theo cơ chế khác nhau. Phải đối chiếu danh sách target mong đợi từ cấu hình/service discovery với danh sách target hiện tại, không dùng `up` một mình làm mẫu số của mức phủ giám sát.

Một metric nghiệp vụ cũng có vòng đời riêng. Counter `http_requests_total` tăng theo request được ứng dụng ghi nhận; `rate` ước lượng tốc độ từ các mẫu trong một khoảng thời gian, không khôi phục request chưa hề được instrument. Nếu một Pod restart, counter của instance có thể về 0; cộng các `rate` theo instance thích hợp hơn lấy hiệu hai mẫu thô trên dashboard. Nếu scrape interval là 30 giây nhưng cửa sổ truy vấn chỉ 30 giây, một lần scrape trễ có thể để quá ít mẫu cho ước lượng ổn định. Trước khi giải thích một đường biểu đồ rơi xuống 0 là traffic hết, xem số chuỗi đang đóng góp, nhãn `job`/`instance`, thời điểm mẫu cuối và tình trạng target. `up=1` không chứng minh metric nghiệp vụ là đúng đơn vị, đúng nhãn hoặc đúng chỗ ghi.

Phép kiểm trong môi trường thử là cố ý dừng endpoint metric của một target đã được theo dõi, ghi thời điểm, quan sát trạng thái target và `up`; tiếp đó bỏ target ấy khỏi discovery và so sự khác biệt. Không làm trên hệ thống production để "thử alert". Với dữ liệu mẫu giả định, yêu cầu người học giải thích vì sao ba hiện tượng "`up=0`", "không có chuỗi `up`" và "`up=1` nhưng không có request" cần ba điều tra khác nhau. Hậu kiểm là khôi phục target, thấy sample mới trở lại và xác nhận metric của một request thử tăng đúng như hợp đồng instrument; vẫn chưa suy ra mọi đường request được đo đầy đủ.

# Chương 45 — Prometheus, PromQL và Grafana

Tính rate, tỷ lệ lỗi và p95 có giới hạn từ mẫu thật/giả định.

## Bài 45.1 — PromQL

Dữ liệu CSV dưới đây là ví dụ tính toán giả định, không phải output scrape trên cluster. Hai instance `a` và `b` được lấy mẫu ở giây 0, 60, 120, 180; trong ví dụ, tên instance được gắn khi scrape thay cho địa chỉ target mặc định. `ok` và `err` là hai counter HTTP; bốn cột `le_*` là bucket tích lũy của thời gian request tính bằng giây; `cpu_s` là CPU time tích lũy. Để số dễ tự kiểm, từng phút mỗi instance xử lý 54 request 2xx và 6 request 5xx; số đếm bucket tăng 20, 45, 58, 60. Mọi request của bảng thuộc một trong hai status class và đi vào histogram. Trong vận hành, hãy xác nhận phạm vi instrument trước khi lấy histogram count làm mẫu số HTTP.

```
t_s,instance,ok,err,le_0_1,le_0_5,le_1,le_inf,cpu_s
0,a,95,5,30,70,90,100,100
60,a,149,11,50,115,148,160,112
120,a,203,17,70,160,206,220,124
180,a,257,23,90,205,264,280,136
0,b,190,10,60,140,180,200,200
60,b,244,16,80,185,238,260,212
120,b,298,22,100,230,296,320,224
180,b,352,28,120,275,354,380,236
```

Tại giây 180, biểu thức `rate(http_requests_total[3m])` dùng mẫu trong khoảng `(0,180]`, tức giây 60, 120, 180 theo quy tắc range selector. Trên chuỗi đều và không reset này, `rate` của 5xx ở mỗi instance là `(23−11)/120 = 0,1` và `(28−16)/120 = 0,1` request/giây. Cộng theo instance cho 0,2 lỗi/giây. Tương tự tổng request mỗi instance là `(280−160)/120 = 1` request/giây, cả hai là 2 request/giây; tỷ lệ lỗi bằng 0,2/2 = 10%. `increase(counter[3m])` là `rate` nhân 180 giây, có thể cho số lẻ vì Prometheus ngoại suy tới biên cửa sổ. Không đọc nó như hiệu thô 120 giây giữa hai mẫu đầu/cuối. Nếu cần kiểm phép đếm thô của cả 0→180, tổng lỗi tăng 36 trên tổng request tăng 360, cũng là 10% trong bộ dữ liệu tuyến tính này.

Trong hệ có nhãn `job="atlas"` được gắn lúc scrape, các biểu thức sau gom theo status và instance sau khi tính tốc độ riêng từng chuỗi:

```
sum(rate(http_requests_total{job="atlas",status_class="5xx"}[3m]))
/
sum(rate(http_requests_total{job="atlas"}[3m]))

histogram_quantile(0.95,
  sum by (le) (rate(http_request_duration_seconds_bucket{job="atlas"}[3m]))
)
```

Phép cộng trước `rate` là sai khi hai tiến trình reset counter khác lúc: reset của một instance có thể bị tổng của instance khác che khuất. `rate` và `increase` nhận biết bước giảm counter như reset trong phạm vi mẫu, nhưng không thể cứu cửa sổ quá ít mẫu hoặc instrument sai. Khi chọn status class, tử số và mẫu số phải cùng service, route, loại request và cửa sổ; một truy vấn thiếu nhãn lọc có thể cho tỷ lệ số học đúng mà sai đối tượng.

Recording rule định kỳ lưu kết quả biểu thức thành chuỗi mới để giảm chi phí truy vấn lặp lại.

Một **counter reset** cho thấy vì sao thứ tự phép toán quan trọng. Giả sử instance `a` báo tổng request ở ba lần scrape lần lượt 110, 3, 13 vì process restart trước mẫu thứ hai; instance `b` báo 200, 210, 220. Lấy hiệu hai đầu của `a` cho số âm, còn cộng `a+b` trước khi xử lý reset có chuỗi 310, 213, 233; việc gộp đã xóa ranh giới tiến trình và có thể khiến thuật toán nhận sai bước giảm. Tính `rate` riêng cho từng series rồi `sum` để mỗi reset được xét theo đúng identity. Không gán con số rate chính xác từ ba mẫu giả định này nếu chưa chỉ rõ thời điểm scrape, cửa sổ và phép ngoại suy; mục đích của ví dụ là nhận biết **thứ tự và phạm vi**. Nếu `instance` được tái sử dụng cho tiến trình mới, vẫn cần kiểm timestamp restart và nhãn target để giải thích counter giảm.

Tỷ lệ lỗi là phép chia của **hai quần thể cùng phạm vi**. Nếu tử số `sum by (route)` giữ nhãn `route` còn mẫu số `sum` bỏ nhãn ấy, một phép chia vector–vector trong PromQL không tự chia từng route cho tổng fleet: hai phía thiếu bộ nhãn khớp, kết quả có thể rỗng. Muốn tỷ lệ theo route, giữ `route` ở cả hai tổng và dùng cùng bộ lọc `job`, status và thời gian. Muốn tỷ lệ toàn service, bỏ `route` ở cả hai. Nếu một route chỉ tồn tại ở tử số do instrument thiếu ở mẫu số, không được điền 0 hoặc 1 theo trực giác; kiểm tập nhãn và metric gốc. Trường hợp chia cho mẫu số 0 cũng cần chính sách hiển thị “không có traffic” riêng, không gọi đó là 0% lỗi.

**Bài tập 45.1 — Rate và p95 tính sai thứ tự**

Một kỹ sư tính `rate(sum(http_requests_total)[3m:])` rồi lấy trung bình `p95` của hai Pod. Nêu hai lỗi cơ chế và truy vấn/quan sát phù hợp hơn. Nếu một Pod mất scrape, giới hạn của kết quả mới là gì?

**Bài 45.2 — Phân bố độ trễ bằng histogram**

Ở `(0,180]`, tổng tốc độ bucket của hai instance lần lượt là 0,667; 1,5; 1,933; 2 request/giây cho ngưỡng 0,1; 0,5; 1; `+Inf`. Vì p95 của 2 request/giây nằm tại 1,9 request/giây, nó thuộc bucket `(0,5;1]`. Với giả định nội suy đều trong bucket của `histogram_quantile`, kết quả là `0,5 + (1,9−1,5)/(1,933…−1,5) × 0,5 ≈ 0,962 giây`. Có thể kiểm bằng 120 giây quan sát: tổng 240 request, 180 ở ≤0,5 giây và 232 ở ≤1 giây; vị trí 228 cho `0,5+(228−180)/(232−180)×0,5 = 0,9615` giây. Đây là ước lượng từ ranh giới bucket, không phải p95 chính xác của từng request; bucket rộng làm sai số nội suy lớn. Giữ `le` khi `sum` bucket; bỏ nó làm mất biên để tính quantile.

Thử phản chứng bằng hai tập dữ liệu giả định đều có 240 request, 180 request ở ≤0,5 giây, 52 request ở `(0,5;1]` và 8 request ở >1 giây. Ở tập A, cả 52 request giữa nằm gần 0,51 giây; ở tập B, chúng nằm gần 0,99 giây. Nếu giữ nguyên cách phân bố 180 request đầu và 8 request cuối, mọi bucket đã công bố ở hai tập giống nhau, nên phép nội suy vẫn trả khoảng 0,962 giây cho cả hai. Nhưng quan sát thứ 228, tức p95 theo thứ tự mẫu trong ví dụ này, gần 0,51 giây ở A và 0,99 giây ở B. Khi ngưỡng SLO nằm trong bucket rộng ấy, không được gọi chênh lệch nhỏ của `histogram_quantile` là bằng chứng chắc chắn của cải thiện hoặc suy thoái. Cần ranh giới bucket dày hơn quanh ngưỡng quyết định hoặc dữ liệu latency có độ phân giải phù hợp, đồng thời cân nhắc số series và chi phí thu thập. Phản ví dụ giữ nguyên count ở từng bucket, không giả vờ có 240 latency thô trong CSV hiện tại.

CPU time tăng 24 giây trên **mỗi** instance trong 120 giây mẫu, tổng 48 giây CPU time trên hai instance, tức 0,4 lõi trung bình; con số ấy không loại trừ chờ I/O, lock, backend, mạng hoặc một

nhóm request chậm. p95 tăng cùng lúc CPU bình thường là dấu hiệu cần điều tra latency theo route và upstream; không được gán nguyên nhân chỉ bằng hai đường biểu đồ.

Snapshot exposition giả định của target `a` ở giây 180 dưới đây có cú pháp metric, bucket và count nhất quán với dòng `a` cuối CSV. Endpoint ứng dụng không tự xuất nhãn `instance` hoặc `job`: Prometheus gắn chúng theo target khi scrape; `up` cũng do Prometheus tạo. Snapshot không có các mẫu quá khứ nên riêng nó không thể trả lời truy vấn `rate`.

```
# HELP http_requests_total Completed HTTP requests.
# TYPE http_requests_total counter
http_requests_total{status_class="2xx"} 257
http_requests_total{status_class="5xx"} 23
# HELP http_request_duration_seconds Request duration in seconds.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{le="0.1"} 90
http_request_duration_seconds_bucket{le="0.5"} 205
http_request_duration_seconds_bucket{le="1"} 264
http_request_duration_seconds_bucket{le="+Inf"} 280
http_request_duration_seconds_sum 130
http_request_duration_seconds_count 280
# HELP process_cpu_seconds_total CPU time consumed by the process.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 136
```

Histogram còn cho một phép quyết định **không cần nội suy p95** nếu ngưỡng SLO trùng ranh giới bucket. Trong fixture 120 giây, có 240 request và 232 request ở bucket `le="1"`. Tỷ lệ request không quá 1 giây theo cách instrument này là `232/240 ≈ 96,7%`; nếu SLO yêu cầu 99% request ≤1 giây, dữ liệu fixture không đạt. Ngược lại, muốn kiểm ngưỡng 0,8 giây thì bucket 0,5 và 1 giây chỉ cho cận: ít nhất 180/240 = 75% và nhiều nhất 232/240 ≈ 96,7% ở ≤0,8 giây. Không thể từ các bucket ấy quyết định một mục tiêu 95% tại 0,8 giây. Cần bucket ranh giới 0,8 giây hoặc dữ liệu latency phù hợp, thay vì dựa vào p95 nội suy 0,962 giây như một phép đo chính xác.

Nếu cộng bucket từ nhiều instance, phải kiểm **độ phủ đo** trước khi tính phân vị hoặc SLI. Một instance ngừng scrape có thể làm tốc độ bucket và count cùng giảm; phép chia bucket/count vẫn trông hợp lý nhưng bỏ người dùng của instance đó. Dùng `up`, số target dự kiến, `histogram_count` hoặc bucket `+Inf`, lượng request ở edge và traffic split để kiểm mẫu. Bucket `+Inf` phải nhất quán với tổng quan sát của histogram trong cùng phạm vi; nếu không, lỗi instrument, nhãn hoặc cửa sổ cần được giải thích trước khi ra quyết định. Đây là kiểm tra dữ liệu đầu vào, không phải một phép hiệu chỉnh bằng cách thêm giá trị 0 cho target vắng mặt.

### Bài tập 45.2 — Tính rate, lỗi và p95 từ CSV

Từ CSV, tại giây 180 hãy tính rate 5xx, tổng rate, tỷ lệ lỗi, CPU trung bình và p95 classic histogram cho hai instance. Nêu vì sao `increase(...[3m])` không nhất thiết bằng hiệu thô giữa mẫu đầu và cuối trong cửa sổ.

### Bài 45.3 — Bảng quan sát

Tình huống giả định: sau một thay đổi nhãn, dashboard báo p95 tăng, CPU giữ quanh 0,4 lõi, target `b` có `up=0` ở giây 240 và số series tăng nhanh. Trước hết khóa cùng data source, time range, step, filter `service/route` và múi giờ trong Grafana; panel vẽ `rate(...[3m])` ở các mốc đánh giá khác nhau sẽ nhìn những cửa sổ khác nhau. Xem Explore hoặc truy vấn thô để so số mẫu, nhãn và series tham gia, rồi kiểm tra target discovery, endpoint `/metrics`, kết quả scrape và log exporter. Nếu `b` vẫn được discovery và scrape lỗi, `up{instance="b"}=0`; sau failed scrape, series do `b` xuất có thể stale và biến mất khỏi instant vector. Range vector có thể còn mẫu cũ một thời gian. Tỷ lệ lỗi hoặc p95 chỉ từ `a` có thể trông tốt lên dù người dùng của `b` vẫn lỗi. Không biến "không có mẫu" thành 0 request hay 0 latency.

Để thấy tác động lên mẫu số, xét **hai cửa sổ giả định** có cùng độ dài, sau khi cửa sổ truy vấn đã hết các mẫu cũ của `b`. Ở cửa sổ đầu, `a` ghi 1 lỗi trên 100 request, `b` ghi 50 lỗi trên 100 request: tỷ lệ gộp là `(1+50)/(100+100) = 25,5%`. Ở cửa sổ sau, chỉ `a` còn đủ mẫu cho truy vấn và vẫn ghi 1 lỗi trên 100 request; panel tính được 1%. Chênh lệch 25,5% → 1% là thay đổi **phạm vi quan sát**, chưa phải bằng chứng hệ thống hồi phục. Không có dữ liệu mới của `b`, ta không biết nó còn phục vụ 100 request với 50 lỗi, đã dừng nhận request, hay chỉ hỏng endpoint đo. Truy vấn `rate(...[3m])` có thể vẫn dùng các mẫu cũ của `b` trong một phần cửa sổ chuyển tiếp; phải kiểm thời điểm đánh giá và tập series thực sự có mặt trước khi áp dụng phép tính minh họa này.

Song song, xem metric/label nào tạo nhiều tổ hợp mới; nếu có `request_id`, bỏ nhãn ấy ở điểm instrument, giới hạn nhãn được phép và đánh giá ngân sách chuỗi trước khi triển khai lại. Sau đó xác minh cả hai target scrape được, số series ổn định và request nghiệp vụ qua đúng Route; xem latency/error theo route và so với log/trace của upstream để tìm nguyên nhân p95. Tăng retention hay đổi panel không sửa được explosion ở nguồn. Grafana hiển thị câu trả lời của truy vấn cho cửa sổ đã chọn, nên một panel "No data" là tín hiệu kiểm tra scrape, selector và range, không phải bằng chứng dịch vụ khỏe hoặc chết.

Grafana truy vấn Prometheus qua data source rồi vẽ panel; nó không phải nguồn gốc của phép đo.

Ước lượng **cardinality trước khi thêm nhãn**. Giả sử một metric counter có 10 route template, 5 lớp status và 2 instance; tối đa `10×5×2 = 100` tổ hợp đang hoạt động nếu mọi tổ hợp xuất hiện. Một classic histogram có 12 bucket cho cùng ba nhãn có thể có tới 1.200 bucket series, cộng series `_sum` và `_count` nếu được xuất đầy đủ. Con số thực phụ thuộc tổ hợp nào xuất hiện và quá trình stale, nhưng phép nhân cho thấy mỗi chiều nhãn có chi phí. Nếu thêm `request_id` gần như riêng cho từng request, lượng series mới tăng theo số ID được ghi, không còn bị chặn bởi 100 tổ hợp hữu hạn. Trước rollout, dự tính giới hạn series, chi phí ingest và truy vấn; dùng route template thay raw URL, còn ID ở log/trace có retention và quyền phù hợp.

Một panel "p95 tốt lên" nên đi kèm **bảng kiểm phạm vi**: số target dự kiến và `up`, số series bucket từng target, tốc độ request ở edge so với ứng dụng, selector của panel, cửa sổ và bước đánh giá. Nếu `b` biến mất, so tỷ lệ lỗi trên tập `a` cố định *trước và sau* để hỏi `a` có cải thiện không; song song đánh dấu SLI toàn service là UNKNOWN cho khoảng thiếu `b` nếu edge không cung cấp mẫu bù. Khi `b` trở lại, không nối hai đoạn thời gian như một population liên tục nếu nhãn hoặc route mapping đã đổi. Grafana có thể vẽ đường mượt qua dữ liệu đã được tổng hợp; bản điều tra phải giữ các điểm thiếu và

sự kiện thay đổi instrument để không biến lỗi thu thập thành thành tích vận hành.

**Bài tập 45.3 — Panel lỗi giảm khi target mất scrape**

Ở giây 240, `up{instance="b"}=0` nhưng panel lỗi 5xx giảm. Liệt kê các phép kiểm để phân biệt dịch vụ `b` hồi phục, `b` không được scrape và truy vấn bị lọc nhãn; chỉ rõ điều chưa thể kết luận từ `up`.

# Chương 46 — Nhật ký, vết truy cập và OpenTelemetry

Nối log và span của một request, kể cả khi một nhánh timeout nhưng cuối trả 200.

## Bài 46.1 — Nhật ký

Bài 45.2 cho thấy p95 tăng có thể cùng tồn tại với CPU bình thường và tỷ lệ 5xx không đổi. Metric trả lời "bao nhiêu request chậm trong tập được đo", nhưng không chứa đường đi của từng request. Log ghi một sự kiện tại một điểm: khi nào một lần gọi timeout, module nào báo lỗi, biến trạng thái nào đã quan sát. Trace nối các công đoạn thực thi thành các span có quan hệ cha con. Ba tín hiệu hỗ trợ nhau; một log `WARN`, một span dài hoặc một p95 cao tự nó chưa chứng minh nguyên nhân gốc.

Một log record cần thời gian, mức nghiêm trọng, nội dung sự kiện và các thuộc tính đủ để định vị nguồn phát. OpenTelemetry phân biệt thời điểm sự kiện được ghi với thời điểm hệ thu nhận quan sát nó; hai thời điểm có thể khác vì hàng đợi hoặc truyền mạng. `INFO`, `WARN` hay `ERROR` thể hiện quyết định của nơi phát log, không thay cho HTTP status hoặc trạng thái span. Log có cấu trúc giữ các trường riêng như `service`, `event`, `request_id` và `trace_id`, để lọc chính xác hơn việc tìm chuỗi trong một câu thông báo. Không đặt payload, token hoặc người dùng cụ thể vào trường chỉ để việc tìm kiếm dễ hơn.

`request_id` do Gateway hoặc ứng dụng tạo giúp tìm log cùng một request; nó có thể xuất hiện ở header ứng dụng nhưng không phải định dạng propagation chuẩn của trace. `trace_id` định danh một trace, còn `span_id` gắn log với đúng công đoạn nếu logger biết context hiện hành. Một trace có thể phục vụ nhiều log và nhiều span; khi context bị mất, log vẫn được tạo nhưng thiếu ID hoặc mang ID mới. Không đưa `request_id` vào nhãn metric Prometheus: một ID khác cho mỗi request gây cardinality lớn như đã phân tích ở Bài 45.2.

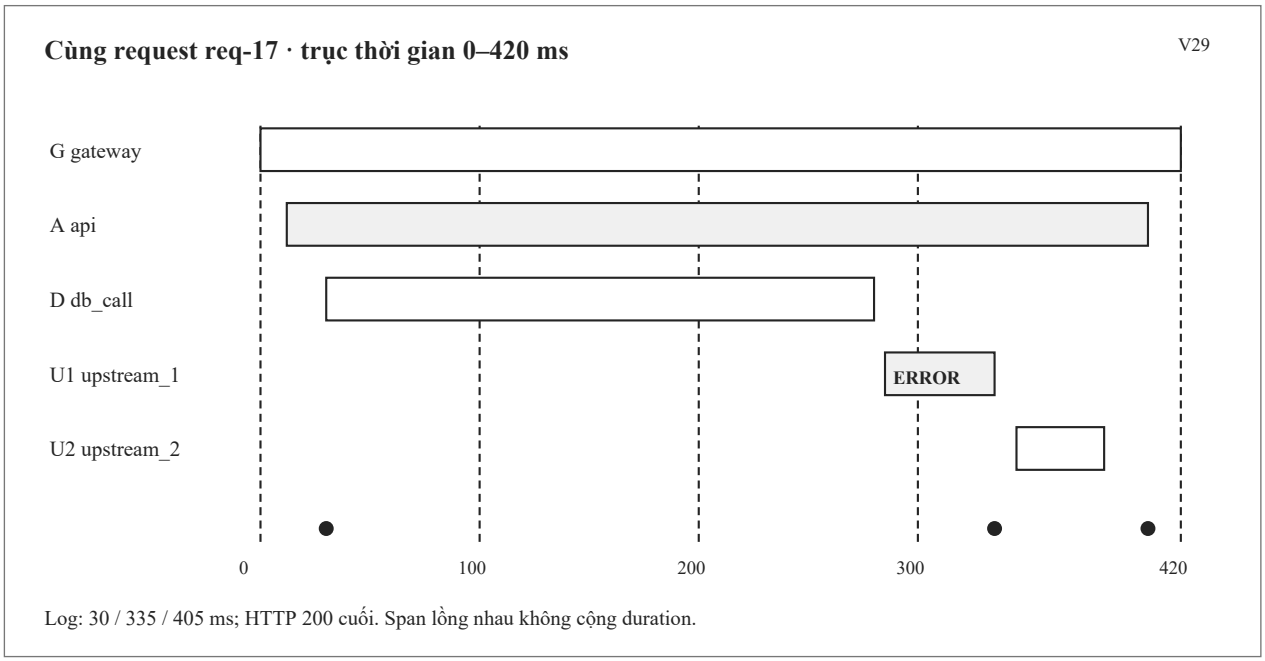
```
[
  {
    "time": "2026-10-19T10:00:00.030Z",
    "severity": "INFO",
    "service": "api",
    "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
    "span_id": "c1c1c1c1c1c1c1c1",
    "request_id": "req-17",
    "event": "db_call_started"
  },
  {
    "time": "2026-10-19T10:00:00.335Z",
    "severity": "WARN",
    "service": "api",
    "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
    "span_id": "d2d2d2d2d2d2d2d2",
    "request_id": "req-17",
    "event": "upstream_timeout",
    "attempt": 1
  },
  {
    "time": "2026-10-19T10:00:00.405Z",
    "severity": "INFO",
    "service": "api",
    "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
    "span_id": "b7ad6b7169203331",
    "request_id": "req-17",
    "event": "response_sent",
    "http_status": 200
  }
]
```

Giả sử log ứng dụng ghi `db query completed` kèm `trace_id`, `span_id` và thời điểm sự kiện; collector nhận nó 3 giây sau. Dùng event time để ghép với span, rồi kiểm observed time để thấy độ trễ pipeline log. Nếu Gateway log một `request_id` nhưng API mất trace context ở chặng gọi nội bộ, có thể tìm hai log theo request ID đã truyền, song không được gán span API mới vào trace cũ chỉ vì timestamp gần nhau. Phải kiểm header/context propagation tại ranh giới hai dịch vụ. Việc log body hay token để "nối dễ hơn" vừa không sửa propagation vừa tăng nguy cơ lộ dữ liệu.

Khi điều tra, bắt đầu từ cửa sổ metric và route bị chậm, chọn request đại diện có trace hoặc request ID, rồi đối chiếu log của từng chặng. Ghi rõ chặng nào không có bằng chứng: không có log có thể do ứng dụng không ghi, pipeline mất log hoặc phép lọc sai; không có span có thể do sampling. Một kết luận đạt yêu cầu phải nêu phép quan sát sẽ bác bỏ giả thuyết, ví dụ DB span vẫn ngắn trong các trace chậm và API chờ pool lâu thì phải chuyển giả thuyết từ thời gian query sang thời gian cấp connection.

## Bài 46.2 — Vết truy cập

Span biểu diễn một thao tác có thời điểm bắt đầu/kết thúc, tên, thuộc tính, event và status; `trace_id` chung nối nó vào trace. `span_id` định danh span trong trace, `parent span ID` cho biết công đoạn gọi trực tiếp. Span event là dấu thời gian gắn trong một span, chẳng hạn lúc lên lịch retry; nó không phải một log record độc lập, và có thể không tồn tại trong trace được lưu nếu span bị bỏ. Status tóm tắt kết quả của thao tác theo instrumentation, không phải severity của một log event. Các span lồng nhau nên không được cộng mọi duration để ra latency của request. Nếu Gateway span chạy 420 ms và API span nằm bên trong 393 ms, tổng hai số đã đếm trùng phần lớn thời gian. Với các nhánh chạy song song, đường quyết định thời gian hoàn tất còn phụ thuộc việc chờ và thứ tự, không phải tổng độ dài mọi span. Span client gọi DB chỉ đo thời gian nhìn từ client, không tự phân rã được thời gian DB xử lý, chờ connection pool và mạng nếu không có instrument tương ứng.




Hình XII.3.1 — Năm span trên một trục 420 ms; số liệu từ fixture trace và log.

Khi Gateway gọi API, nó inject trace context vào carrier HTTP; API extract rồi tạo server span nhận parent từ request. Tiếp đó API phải truyền context hiện hành tới client DB/upstream và qua hàng đợi hoặc tác vụ bất đồng bộ khi có. Nếu chỉ tạo span mới mà không truyền context, trace có thể tách thành hai cây dù request vẫn chạy thành công. Context propagation truyền định danh và cờ xử lý, không vận chuyển toàn bộ span hay bảo đảm span đã được lưu.

W3C Trace Context quy định header `traceparent` gồm version, trace ID 16 byte, parent ID 8 byte và trace flags. Trong ví dụ này, Gateway gửi tới API `traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01`. API tạo span mới có cùng trace ID và parent là span Gateway. Bit sampled trong `01` là tín hiệu propagation/sampling, không phải biên nhận của backend. Header `tracestate` tùy chọn mang trạng thái của hệ tracing theo cú pháp chuẩn; không dùng nó để chuyển secret hoặc payload. Nếu không có header hợp lệ, dịch vụ có thể tạo root trace mới theo cấu hình, làm đứt mạch tương quan cần điều tra.

Sampling giảm lượng trace được ghi, gửi và lưu. Quyết định đầu trace có thể bỏ qua cả một request chậm nếu tỷ lệ lấy mẫu thấp; quyết định ở Collector sau khi thấy thêm span có thể chọn giữ trace

chậm/lỗi nhưng phải giữ dữ liệu tạm và chờ đủ lâu. Một `traceparent` có cờ sampled không bảo đảm mọi SDK, exporter, Collector và backend thực sự nhận được span. Khi mẫu trace không đại diện ngẫu nhiên cho toàn bộ lưu lượng, tỷ lệ lỗi hoặc p95 tính từ trace đã lưu có thể lệch so với metric HTTP được instrument đầy đủ. Muốn suy ra phân phối người dùng, phải hiểu chính sách sampling của từng tuyến.

Ví dụ sau là dữ liệu giả định có thể tính độc lập, không phải trace lấy từ cluster hay giao diện backend. Các mốc là mili giây tương đối từ khi Gateway nhận request; giả định đồng hồ các nguồn đã được chuẩn hóa cho bài toán. `G` là Gateway server span; `A` là API server span; `D` là DB client span do API tạo; `U1` và `U2` là hai lần gọi upstream do API tạo. Cột `status` là span status, không phải HTTP status. Không có DB server span hoặc upstream server span trong dữ liệu này.

```
alias,parent,span_id,stage,start_ms,end_ms,status
G,-,00f067aa0ba902b7,gateway,0,420,UNSET
A,G,b7ad6b7169203331,api,12,405,UNSET
D,A,c1c1c1c1c1c1c1c1,db_call,30,280,UNSET
U1,A,d2d2d2d2d2d2d2d2,upstream_1,285,335,ERROR
U2,A,e3e3e3e3e3e3e3e3,upstream_2,345,385,UNSET
```

Gateway mất `420−0=420 ms`, API nằm trong khoảng `12..405` và mất 393 ms. DB client span `30..280` mất 250 ms. Lần gọi upstream đầu `285..335` mất 50 ms và bị timeout, nghỉ 10 ms rồi retry `345..385` mất 40 ms; đoạn retry quan sát được từ 285 tới 385 chiếm 100 ms. Tổng duration của năm span là `420+393+250+50+40=1153 ms`, không phải latency client. Chuỗi công việc đã thấy nằm trên đường hoàn tất của request 420 ms; phần `0..12`, `12..30`, `280..285`, `335..345`, `385..405` và `405..420` chưa được phân rã đủ để gán nguyên nhân. DB call dài nhất trong các child span được đo, nhưng có thể chứa chờ pool hoặc mạng; chưa có chứng cứ DB server là nơi xử lý chậm.

Ba log JSON giả định dưới đây dùng cùng trace ID; `span_id` nối lần lượt tới `D`, `U1` và `A`. Tại 335 ms, API ghi timeout lần thứ nhất; tại 405 ms nó trả HTTP 200 sau retry. `U1` có span status `ERROR`, còn `A` và `G` là `UNSET`: một thao tác con thất bại rồi được phục hồi không buộc request cuối cùng thành 5xx. Ngược lại HTTP 500 là trường HTTP; trạng thái span là phân loại riêng của instrumentation/semantic conventions và phải kiểm tra theo kind, ngữ cảnh và phiên bản. Một timeout do client chủ động hủy cũng cần phân biệt với timeout lỗi upstream.

Mẫu này có thể phù hợp với một phần request chậm của hiện tượng Bài 45.2, nhưng một trace 420 ms không chứng minh p95 toàn hệ, không chứng minh CPU bình thường là nguyên nhân hoặc loại bỏ các request không được sampling. Chỉ khi cùng route, phiên bản, cửa sổ thời gian và nhóm người dùng với metric p95, nhiều trace đại diện cùng log tương ứng mới hỗ trợ giả thuyết DB hoặc upstream đóng góp. Trace không có span DB server vẫn để ngỏ phân rã 250 ms; cần query latency/pool wait, log DB hoặc instrument thêm với ngân sách dữ liệu phù hợp.

**Bài tập 46.1 — Tính độ trễ trên cây span**

Từ CSV, tính duration của `G`, `A`, `D`, `U1`, `U2` và khoảng của hai lần gọi upstream. Vì sao cộng năm duration không cho latency của client? Chỉ ra hai đoạn thời gian chưa có span riêng.

### Bài tập 46.2 — Log timeout nhưng HTTP trả 200

Log ở 335 ms báo timeout, nhưng log ở 405 ms ghi HTTP 200. Hãy lần theo parent/child và phân biệt HTTP status, span status, severity. Có thể kết luận DB server chậm 250 ms không?

### Bài 46.3 — Đường thu thập tín hiệu

Instrumentation gọi OpenTelemetry API để tạo span, thêm event hoặc phát log; SDK thực thi chính sách sampling, processor và exporter. Instrumentation tự động của framework có thể tạo span HTTP/DB mà không cần viết tay mọi thao tác, nhưng độ phủ, tên thuộc tính và hành vi khi lỗi phụ thuộc ngôn ngữ, thư viện, phiên bản và cấu hình. Resource attributes như `service.name` và `service.version` mô tả nguồn phát chung; thuộc tính span như `http.route` mô tả thao tác riêng. Semantic conventions giúp nhiều dịch vụ gọi cùng một khái niệm bằng tên nhất quán, nhưng phiên bản convention và chế độ ổn định phải được kiểm tra trước khi ghép truy vấn qua nhiều SDK.

SDK có thể xuất OTLP qua HTTP hoặc gRPC tới Collector. Collector dùng `receiver` nhận tín hiệu, `processor` biến đổi/lọc/batch rồi `exporter` gửi tới backend; thành phần đã khai báo nhưng chưa gắn vào `service.pipelines` không xử lý dữ liệu. Có thể có pipeline riêng cho traces, logs và metrics. Exporter helper của Collector hỗ trợ queue, timeout và retry theo cấu hình; batch giảm số lần gửi nhưng tạo độ trễ và phần dữ liệu chưa chuyển đi. Khi backend chậm, queue hữu hạn có thể đầy, retry hết hạn hoặc dữ liệu bị từ chối. Queue trong bộ nhớ mất theo tiến trình; hàng đợi bền cần storage được cấu hình và cũng không thay cho chính sách lưu dữ liệu của backend.

Cần phân biệt bốn ranh giới: ứng dụng đã tạo span; SDK đã đưa span cho exporter; Collector đã nhận yêu cầu OTLP; backend đã lưu và truy vấn thấy span. Một kết quả gửi thành công đến Collector chỉ xác nhận mức chấp nhận của chặng đó. OTLP còn có phản hồi partial success cho phần bị từ chối và lỗi có/không thể retry; ngay cả Collector đã nhận đủ cũng có thể lỗi khi xuất đi. Chẩn đoán mất trace cần đo queue, dropped items, retry/export failures và trạng thái backend ở đúng cửa sổ. Đừng biến lỗi Collector thành lỗi request nghiệp vụ: chọn xử lý bất đồng bộ và giới hạn queue/timeout để đường chính không mặc nhiên chờ telemetry. Tuy nhiên exporter đồng bộ hoặc cấu hình chặn khi queue đầy có thể kéo dài request; phải kiểm tra SDK và Collector cụ thể, không khẳng định khả năng chịu lỗi nếu chưa thử.

Khi p95 tăng mà CPU và error rate bình thường, trước tiên cố định time range, route, status và số request của metric. Chọn một request chậm đại diện, tìm log theo request ID và trace ID rồi kiểm tra timeline, parent, HTTP response và các lần retry. So thời điểm timeout với metric độ trễ DB/upstream và log tại đúng dịch vụ; kiểm tra queue và pool wait trước khi quy chậm cho CPU. Nếu thiếu span, kiểm tra header propagation qua Gateway, sampling, SDK/exporter, Collector receiver/processor/queue, OTLP rejection và backend retention. Không có span chỉ chứng minh không thấy span ở vị trí tìm kiếm, không chứng minh thao tác chưa chạy. Timestamp từ nhiều máy có thể lệch; đối chiếu duration trong span, đồng hồ hệ thống và thứ tự sự kiện từ cùng nguồn trước khi suy một lời gọi kết thúc trước khi bắt đầu.

### Bài tập 46.3 — Trace chỉ có span Gateway

API trả request thành công nhưng tìm trace chỉ thấy span Gateway. Nêu ít nhất bốn ranh giới cần kiểm tra theo thứ tự từ propagation đến backend. Cờ sampled `01` trong `traceparent` cho phép kết luận gì và không cho phép kết luận gì?

# Chương 47 — Cảnh báo và sức khỏe hệ thống quan sát

Tạo tín hiệu hành động thay vì báo động từ một dashboard.

## Bài 47.1 — Quy tắc cảnh báo

Với alert, tách "không lấy được metric" (`up`, target discovery/absence) khỏi "ứng dụng có lỗi" (tỷ lệ 5xx và lưu lượng tối thiểu). Recording rule có thể lưu tốc độ đã tổng hợp theo service/route, nhưng tên và nhãn của rule phải giữ đúng phạm vi; nếu mất target, kết quả tổng hợp cũng thiếu. Một alert p95 nên có điều kiện lưu lượng và chất lượng dữ liệu, ngưỡng theo SLO đã xác định, thời gian `for` phù hợp; ví dụ này không ấn định SLO cho hệ thật.

Một alerting rule đánh giá điều kiện và chuyển cảnh báo sang trạng thái chờ/phát theo `for`.

Giả sử trong năm phút có 1.000 request hợp lệ và 20 phản hồi 5xx: tỷ lệ lỗi tính trên cửa sổ ấy là 2%. Nếu cùng lúc một nửa target biến mất, tỷ lệ đo được có thể bỏ sót request lỗi; alert "tỷ lệ lỗi dưới ngưỡng" không còn đáng tin chỉ vì biểu đồ xanh. Đặt một kiểm tra riêng cho tính đầy đủ của target và volume trước khi diễn giải SLI. Một alert ứng dụng cần route tới đúng người trực, kèm runbook và nhãn giới hạn phạm vi; không gộp cảnh báo mất telemetry với cảnh báo vi phạm SLO thành một nguyên nhân. Những số trên là dữ liệu giả định để tập suy luận, không phải ngưỡng áp cho dịch vụ thật.

Trong bộ số giả định ấy, target A có 500 request/2 lỗi, B có 500/18: tổng 20/1.000 = 2%. Ngưỡng minh họa 1% với `for: 10m` chỉ firing nếu biểu thức vượt ngưỡng liên tục; lần đầu là pending. Khi B mất scrape và chuỗi của nó rời cửa sổ, số còn thấy là 2/500 = 0,4%: alert có thể hết dù người dùng B vẫn lỗi. Áp `rate()` lên từng counter trước khi cộng để xử lý reset; việc ấy không bù được B bị mất. Kiểm discovery, `up`, số series và lưu lượng trước khi đọc tỷ lệ; không coi thiếu dữ liệu là 0 lỗi.

Để chuyển phép tính trên thành quy tắc cảnh báo, giả sử ứng dụng thực sự xuất counter `checkout_requests_total` có nhãn `code` cho **mọi** request hợp lệ, cả thành công lẫn lỗi, ở cả hai target. Đây là rule YAML minh họa ngưỡng 1%, không phải cấu hình đã triển khai:

```
groups:
- name: checkout
  rules:
  - alert: CheckoutHigh5xx
    expr: |
      sum(rate(checkout_requests_total{code=~"5.."}[5m]))
      / sum(rate(checkout_requests_total[5m])) > 0.01
    for: 10m
```

Phải lọc cùng service, route và population ở tử lẫn mẫu nếu metric thực chứa nhiều nhóm; mẫu số phải có request. `rate()` xử lý counter reset theo từng chuỗi rồi `sum` mới gộp, nhưng nó ngoại suy giữa các lần scrape nên không hứa ra đúng `20/1.000` từ hai con số tổng hợp của fixture. Nếu B vẫn nằm trong discovery nhưng scrape hỏng, `up` của B là `0`; nếu B đã biến khỏi discovery, chuỗi `up` của B cũng có thể không còn. Kiểm `up` mà không đối chiếu danh sách target **phải có** vẫn bỏ sót nhánh thứ hai. Một rule về mất telemetry phải có nguồn kỳ vọng độc lập với chính chuỗi đang thiếu, hoặc ít nhất phép đối chiếu discovery và số target theo cấu hình vận hành.

Khi thấy tỷ lệ tính được hạ xuống 0,4%, người trực cần thử giả thuyết "dịch vụ đã hồi phục" bằng request từ phía người dùng hoặc edge cùng cửa sổ, đồng thời kiểm B có còn được scrape hay không. Nếu B vắng, kết luận đúng là SLI quan sát được **không đủ mẫu** để xác nhận hồi phục. `for: 10m` đo thời gian biểu thức liên tục hiện diện và đúng qua các lượt đánh giá; nó không làm dữ liệu bị thiếu tự trở lại. Tách cảnh báo thiếu target khỏi cảnh báo lỗi ứng dụng, rồi xác minh đường gửi notification ở bài kế tiếp trước khi dựa vào alert để quyết định triển khai.

**Bài tập 47.1 — Tỷ lệ lỗi đẹp lên khi mất target**

Trong fixture hai target, A có 500 request/2 lỗi và B có 500/18. Sau khi B biến khỏi discovery, chỉ còn A trong phép tính năm phút. Tính tỷ lệ trước/sau, phân biệt up=0 với chuỗi up vắng và nêu phép kiểm cần có trước khi thông báo dịch vụ đã hồi phục.

**Bài 47.2 — Định tuyến cảnh báo**

Prometheus đánh giá rule và gửi alert sang Alertmanager; Alertmanager deduplicate, group và route thông báo tới receiver. Alert "firing" không có nghĩa người trực đã thấy nó. Giữa rule và con người còn mạng, cấu hình route, group wait, receiver, kênh giao hàng và quy trình trực. Trong một sự cố, tách phép kiểm metric/rule đang đúng, alert đã tới Alertmanager, route nào khớp, notification có được gửi và người trực đã nhận. Một lỗi ở bước sau không thể chữa bằng cách hạ ngưỡng metric ở bước đầu.

Grouping gộp các alert có nhãn chung thành một thông báo; nếu group theo `cluster` và `alertname`, mười Pod lỗi cùng kiểu có thể thành một page, nhưng chi tiết từng Pod vẫn cần được giữ trong thông báo hoặc truy cập từ đó. Group quá rộng có thể chôn hai sự cố khác nhau; group quá hẹp tạo bão thông báo. Inhibition chỉ chặn notification của alert mục tiêu khi alert nguồn tương ứng đang firing và nhãn so khớp theo cấu hình. Silence chặn thông báo theo matcher trong khoảng thời gian có chủ đích. Cả hai không làm bệnh của hệ thống biến mất hay sửa metric gốc.

Giả định lúc 10:05, toàn cụm mất đường scrape; 60 alert "target down" và một alert "cluster unreachable" xuất hiện. Có thể inhibit các alert con khi alert cụm cùng `cluster` đang firing để người trực nhận tín hiệu chính, nhưng phải kiểm matcher thiếu nhãn: so khớp nhãn rỗng hoặc vắng có thể làm một rule inhibit rộng hơn ý định. Nếu `cluster unreachable` không tới Alertmanager, các alert con có thể vẫn gửi hoặc tất cả mất tùy đường lỗi. Không mặc định "không có notification" là healthy. Thử route với nhãn representative, cả trường hợp thiếu nhãn, mất receiver và silence hết hạn; lưu kết quả như bằng chứng vận hành, không dựa trên ảnh cấu hình.

Escalation là hợp đồng bên ngoài Alertmanager: ai nhận trước, bao lâu phải xác nhận, kênh dự phòng và ai thay thế khi không phản hồi. Một route đúng chỉ chứng minh quyết định chọn receiver, chưa chứng minh giao hàng hoặc hành động. Với các sự cố nghiêm trọng, dùng một tín hiệu kiểm tra đường cảnh báo định kỳ trong môi trường được phép, kiểm nhận thực tế và ghi thời điểm. Không dùng dữ liệu giả định 10:05 hay 60 alert làm kết quả benchmark. Hệ thực còn cần chính sách retention, quyền truy cập và bảo vệ thông tin nhạy cảm trong label/annotation.

### Bài tập 47.2 — Mất scrape nhưng không có cảnh báo

Không có page khi cluster mất scrape. Có thể kết luận healthy không? Truy vết các ranh giới nào?

### Bài 47.3 — Đường thu thập tín hiệu

Một pipeline telemetry gồm nhiều chặng: ứng dụng tạo tín hiệu, SDK hoặc agent chọn và gửi, collector nhận và xử lý, backend lưu, rồi công cụ truy vấn hiển thị. Mỗi chặng có miền lỗi riêng. Request thành công không chứng minh span đã được lưu; một trace xuất hiện cũng không chứng minh toàn bộ request đã được lấy mẫu. Vì vậy, khi p95 metric tăng nhưng các trace đã lưu chủ yếu nhanh, không được kết luận metric sai chỉ từ tập trace đó.

Trước hết xét sampling: head sampling có thể bỏ qua nhiều request trước khi biết độ trễ cuối cùng; tail sampling có thể có quy tắc khác cho trace lỗi hoặc chậm. Đối chiếu cấu hình và số lượng trace theo cùng cửa sổ với metric toàn population. Tiếp theo xét đường vận chuyển: queue đầy, retry hoặc drop ở exporter/collector có thể làm thiếu span; đọc counter của chính pipeline và log lỗi gửi. Cuối cùng xét truy vấn và retention: time range, nhãn dịch vụ, sampling của backend và thời gian lưu có thể khiến tập nhìn thấy không đại diện. Kiểm từng giả thuyết bằng số vào/ra của chặng tương ứng.

Liên kết metric, log và trace qua service, route, thời gian và trace ID trong log/trace. Không đặt request ID vào nhãn metric có cardinality cao; cũng không đưa dữ liệu cá nhân vào thuộc tính chỉ để tiện tìm kiếm. Nếu tín hiệu thiếu, ghi trạng thái đo là chưa đủ bằng chứng thay vì biến khoảng trống thành số 0.

Ví dụ giả định: metric đếm đủ 1.000 request trong một cửa sổ, có 50 request mất trên 0,9 giây; backend chỉ hiện 20 trace, đều nhanh. Hai tập không cùng cỡ và chưa biết cách chọn 20 trace, nên 20 trace ấy không thể bác bỏ tail latency của 1.000 request. Với head sampling, quyết định giữ có thể được đưa ra trước khi biết request cuối cùng chậm hay nhanh; với tail sampling, phải biết collector có giữ được toàn trace đủ lâu để áp quy tắc hay không. Nếu queue exporter đầy, cả trace được chọn cũng có thể biến mất sau sampling. Phân biệt tỷ lệ được chọn, tỷ lệ gửi thành công và tỷ lệ truy vấn thấy được.

Lập cân đối số lượng trong cùng cửa sổ theo từng chặng: request hoàn tất ở ứng dụng, span root được tạo, trace được chọn, collector nhận, exporter gửi thành công, backend lưu và truy vấn trả về. Các counter ở chặng khác nhau có thể không bằng nhau vì sampling, retry, batch, mất kết nối và định nghĩa đếm khác; cần nêu đơn vị của từng counter trước khi chia tỷ lệ. Ví dụ 100 trace được chọn nhưng chỉ 80 gửi thành công là bằng chứng thiếu 20 ở đường xuất, chưa cho biết 20 trace ấy chậm hay nhanh. Nếu thiếu counter, ghi UNKNOWN cho chính chặng đó. Không nội suy phân phối latency của phần mất từ phần còn lại.

Phép kiểm tiếp theo là chọn vài request kiểm thử có ID đã biết trong một môi trường cô lập, xác nhận chúng xuất hiện ở ứng dụng, collector và backend với cùng trace ID và thời gian hợp lý. Sau đó so lại truy vấn metric theo cùng service, route, instance và khoảng thời gian; nếu trace sampling làm lệch tập, dùng metric toàn population để tính SLI và trace để giải thích ca cụ thể. Không tăng sampling lên 100% trên production khi chưa tính tải và dữ liệu nhạy cảm mà pipeline sẽ thu.

### Bài tập 47.3 — Metric p95 cao nhưng trace thấy nhanh

p95 metric tăng, còn tập trace đã lưu có phần lớn request nhanh. Nêu ba cách dữ liệu quan sát bị sai lệch và một phép kiểm cho từng cách. Đề xuất cách liên kết log/trace/metric mà không đưa request ID vào nhãn Prometheus hoặc lộ PII.

### Bài tập 47.4 — Metric và trace mâu thuẫn

Metric p95 cao nhưng trace thấy nhanh: được kết luận metric sai không?

# PHẦN XIII — Độ tin cậy và phục hồi dịch vụ

## Độ tin cậy được định nghĩa, bảo vệ và khôi phục ra sao?

SLI/SLO định nghĩa tác động người dùng; response dùng chứng cứ, DR dùng bản sao có thử khôi phục, capacity dùng tải và giới hạn.

XIII

LỘ TRÌNH PHẦN


- Chương 48 — Chỉ số, mục tiêu và ngân sách lỗi
- Chương 49 — Ứng phó sự cố và phân tích sau sự cố
- Chương 50 — Sao lưu, khôi phục thảm họa và dữ liệu
- Chương 51 — Hiệu năng, năng lực tải và thử giới hạn

# Chương 48 — Chỉ số, mục tiêu và ngân sách lỗi

Chọn phép đo gần trải nghiệm người dùng và tính burn rate.

## Bài 48.1 — SLI

**Các số liệu trong chương là ví dụ giả định để học, không phải phép đo từ hệ thống production.**

Một API trả HTTP 200 có thể vẫn gây mất tiền nếu giá trị đơn hàng sai; ngược lại HTTP 503 ở một tác vụ nền không thuộc hành trình mua hàng có thể không làm hỏng thao tác khách hàng đang quan tâm. Vì thế trước khi chọn biểu đồ, ta viết hợp đồng sự kiện: ai là người dùng, hành động nào quan trọng, điểm đo ở client, edge hay server, và cách xử lý timeout, retry, cache, request trùng, bot, dữ liệu lỗi. **SLI** là phép đo có định nghĩa đó, không phải tên của một metric có sẵn. Availability thường là tỷ lệ thao tác hợp lệ thành công; latency là tỷ lệ thao tác hợp lệ hoàn tất dưới ngưỡng thời gian. Error rate là phần bù của một SLI nhị phân nếu định nghĩa "bad" bao trùm mọi trường hợp không good. Throughput đo lượng công việc trên đơn vị thời gian; nó mô tả tải, không tự là chất lượng phục vụ.

Với request-based SLI, tập `valid` gồm các lần thử thuộc phạm vi đã công bố; `good` là tập con thỏa cả kết quả nghiệp vụ và ngưỡng thời gian nếu SLI gộp hai điều kiện. `bad = valid - good`. Tỷ lệ good là `good/valid`, có đơn vị 1; không cộng các tỷ lệ của từng Pod rồi chia đều khi lưu lượng khác nhau. Chỉ loại request có lý do sản phẩm rõ (ví dụ endpoint nội bộ hoặc đầu vào bị từ chối đúng theo hợp đồng), ghi riêng số bị loại. Timeout phía client sau khi request rời máy thường phải được tính là bad nếu SLI hứa đáp ứng cho lần thử ấy, ngay cả khi server không có log; đo chỉ ở server có thể làm mẫu số co lại trong lúc mạng hỏng. Retry chỉ là good của một *lần thử* nếu SLI đo lần thử; nếu SLI đo một hành trình người dùng, phải gom các lần thử thành một thao tác và công bố quy tắc gom. Không đổi mẫu số sau sự cố để làm SLI đẹp hơn.

Time-based SLI chia lịch thành các khoảng đo (chẳng hạn phút) và xét phút nào thỏa điều kiện phục vụ. Nó hữu ích khi mục tiêu là thời gian sẵn sàng, nhưng một phút có một request được cân ngang phút có mười nghìn request. Request-based SLI cân theo số thao tác, nên phản ánh tác động tại đỉnh tải tốt hơn nhưng im lặng khi không có request. Hai cách không đổi chéo bằng một phép nhân tùy tiện; nếu dùng probe tổng hợp, giữ nó thành luồng đo riêng và nói rõ phần hành vi người dùng mà probe không kiểm được.

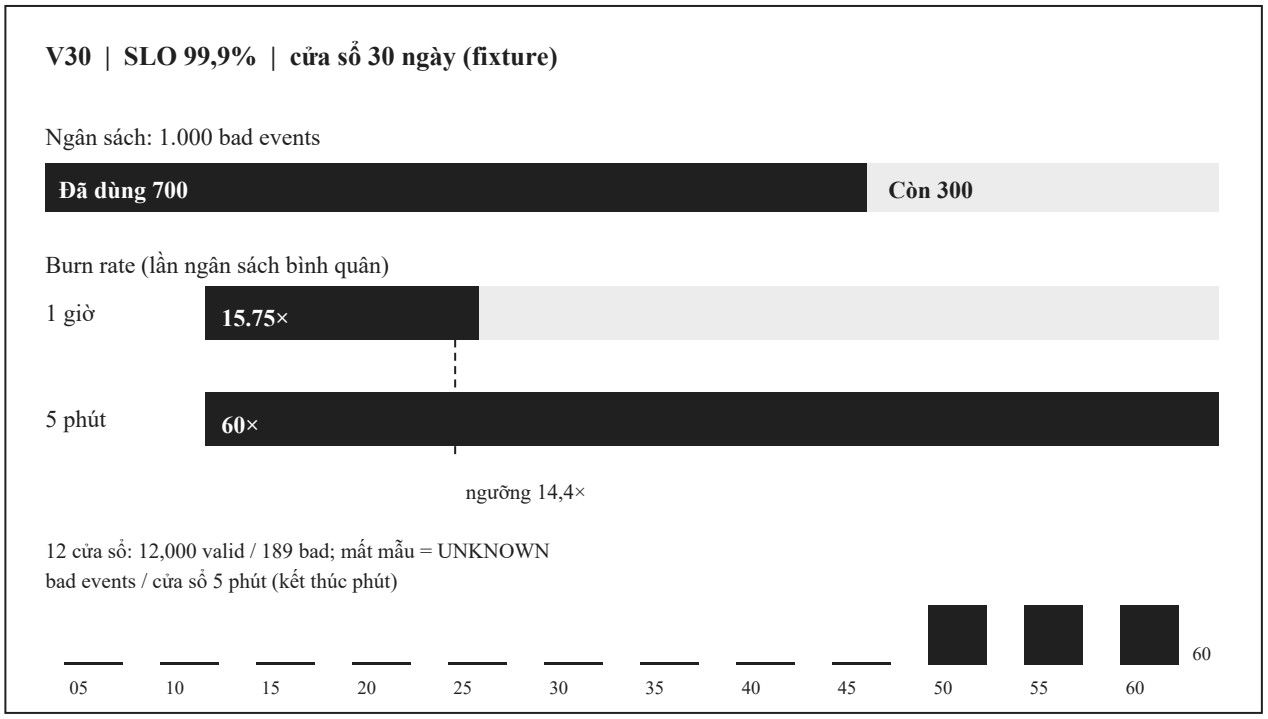
## Bài 48.2 — SLO

**SLO** là mục tiêu định lượng áp lên SLI, population và cửa sổ: ví dụ "ít nhất 99,9% request checkout hợp lệ trong 30 ngày gần nhất trả kết quả đúng trong 500 ms, đo tại edge". **SLA** là thỏa thuận với bên nhận dịch vụ có hậu quả khi vi phạm; chỉ có SLO nội bộ chưa tự thành SLA. SLO cũng không chứng minh mỗi request đúng: một mục tiêu 99,9% vẫn cho phép một phần thao tác lỗi. Rolling 30 ngày tại thời điểm `t` dùng `(t−30 ngày, t]` và dịch mỗi khi đánh giá; tháng lịch cố định đóng ở ranh giới tháng. Cùng số lượng bad nhưng budget còn lại có thể khác vì population, cửa sổ và lượng request khác. Cần chốt múi giờ, độ trễ nạp dữ liệu và quy tắc event sát biên trước khi so số liệu.

Với SLO dạng tỷ lệ `S` và `V` valid events trong cửa sổ, ngân sách lý thuyết là `B=(1−S)×V` bad events; đã dùng `E=V−G`, còn `B−E`. Nếu `B` không nguyên, chính sách làm tròn phải được định nghĩa trước, không tự nâng trần để nới mục tiêu. Với một cửa sổ có ít lưu lượng, tỷ lệ có phương sai lớn: một bad trong 10 request là 10% lỗi, không đủ để suy cả hệ thống hỏng liên tục. Với SLO theo thời gian, ngân sách là `(1−S)×độ dài cửa sổ` theo đơn vị thời gian, không lấy số request làm mẫu số. Với latency SLI, bad là request quá ngưỡng (hoặc timeout theo hợp đồng), không lấy trung bình các p95 rời rạc làm phần trăm good toàn hệ thống.

`monthly.csv` là **toàn bộ dữ liệu giả định** của rolling 30 ngày tại một thời điểm: 988.000 valid/511 bad trước giờ cuối, 12.000 valid/189 bad ở giờ cuối. Tổng `V=1.000.000`, `E=700`, `G=999.300`, SLI `99,93%`. Với `S=99,9%`, `B=1.000` bad và còn `300` bad. Phần giờ cuối khớp 12 dòng năm phút trong `five_minute.csv`; con số này chứng minh phép tính trên fixture, không cho biết lưu lượng tháng sau. Nếu có request hoàn toàn mất khỏi instrumentation, các con số vẫn có thể đẹp một cách sai lệch.

### Bài 48.3 — Tốc độ tiêu hao ngân sách lỗi




Hình XIII.1.1 — V30: 700 bad đã dùng và 300 còn lại; burn 1h/5m từ dữ liệu giả định.

Trong một cửa sổ ngắn, `error_ratio = bad/valid`. **Burn rate** của SLO request-based là `error_ratio/(1−S)`, không có đơn vị. Burn 1 nghĩa tỷ lệ lỗi hiện tại bằng phần lỗi được phép bình quân; nó không nói budget hiện đã cạn. Giờ cuối trong fixture có `189/12.000 = 1,575%` lỗi, burn `15,75`; năm phút cuối có `60/1.000 = 6%`, burn `60`. Cảnh báo 1 giờ/5 phút với ngưỡng burn `14,4` sẽ thỏa cả hai phía (`15,75 > 14,4` và `60 > 14,4`). Đây là một tín hiệu can thiệp nhanh, **không phải** bằng chứng duy nhất của nguyên nhân hay xác suất tháng sẽ vi phạm SLO. Công thức "burn × độ dài cửa sổ / 30 ngày" ước lượng tỷ lệ ngân sách thời gian bị tiêu nếu traffic và error ratio đại diện, ổn định; với request budget và traffic biến thiên, phải dùng bad events thật trên toàn population thay cho ước lượng thời gian ấy.

Ví dụ PromQL sau giả sử ứng dụng xuất hai counter riêng `sli_valid_total` và `sli_bad_total`, cùng nhãn `service` và `route`, counter bad tăng đúng một lần cho mỗi valid bad. Đây là **mẫu biểu thức chưa chạy trên Prometheus**, không phải output thật:

```
(
  sum(rate(sli_bad_total{service="checkout",route="write"}[1h]))
  /
  sum(rate(sli_valid_total{service="checkout",route="write"}[1h]))
  > (14.4 * 0.001)
)
and
(
  sum(rate(sli_bad_total{service="checkout",route="write"}[5m]))
  /
  sum(rate(sli_valid_total{service="checkout",route="write"}[5m]))
  > (14.4 * 0.001)
)
```

`rate()` phải áp trên từng counter trước `sum` để nhận ra reset từng instance. Hai cửa sổ dài/ngắn cùng vượt ngưỡng giúp xác nhận lỗi còn đang tiếp diễn và giảm thời gian alert treo sau khi hồi phục. Trên Prometheus thật, cửa sổ phải có đủ sample, vector matching/labels phải đúng, mẫu số dương và target phải còn được scrape; thiếu chuỗi hoặc mất scrape là trạng thái đo **unknown**, không là lỗi bằng 0. Cần cảnh báo độc lập cho tính đầy đủ telemetry và kiểm từ phía client. Không tính trung bình tỷ lệ của các replica; lấy tổng tốc độ bad chia tổng tốc độ valid cùng population. SLO 6h/30m là một nhánh alert khác, không được kết luận từ fixture chỉ có 1h.

Một page burn nhanh và chỉ còn 300 bad làm việc mở rộng canary không hợp lý cho tới khi phân biệt lỗi thật và lỗi đo. Người trực có thể dừng tăng traffic, so request/dữ liệu theo release, giảm tác động bằng rollback hoặc flag nếu hậu điều kiện đã biết. Quyết định tạm dừng release là chính sách sản phẩm/SRE đã thống nhất, không phải tính năng tự động của phương trình. Khi ít lưu lượng, một bad có thể tạo burn rất lớn; đánh giá giá trị giao dịch, probe riêng, số sự kiện và độ tin cậy dữ liệu trước khi đánh thức cả đội. Khi không có valid event, tỷ lệ không xác định chứ không bằng 100% good.

### Bài tập 48.1 — Thiếu dữ liệu đo trong cửa sổ ngắn

Trong fixture 30 ngày, giả sử chỉ giữ 11 dòng đầu của `five_minute.csv` vì dòng cuối chưa được scrape, nhưng monthly summary vẫn được dashboard giữ nguyên. Có được báo “không còn burn” hay không? Hãy nêu phép đối chiếu, trạng thái đo và quyết định trước khi tiếp tục canary.

### Bài tập 48.2 — Gộp tỷ lệ lỗi của hai replica

Một API có hai replica: A xử lý 900 valid/9 bad, B xử lý 100 valid/10 bad trong cùng cửa sổ. Tính tỷ lệ lỗi toàn dịch vụ và giải thích vì sao trung bình cộng 1% và 10% là sai. Nếu SLO 99%, còn ngân sách trong cửa sổ 1.000 valid là bao nhiêu bad?

### Bài tập 48.3 — Burn rate và cảnh báo hai cửa sổ

Với năm phút cuối trong fixture, tính burn và so ngưỡng 14,4. Nếu 1 giờ trước đó chỉ có 12.000 valid/100 bad (thay cho 189), alert hai cửa sổ có page không? Giải thích vì sao không thể tự suy nhánh 6h/30m.

# Chương 49 — Ứng phó sự cố và phân tích sau sự cố

Tổ chức phát hiện, phân vai, giảm tác động và học từ sự cố.

## Bài 49.1 — Phản ứng

**Tình huống, log tóm tắt và số liệu trong chương là fixture giả định, không phải kết quả chạy production.**

Một alert chỉ là tín hiệu được một quy tắc phát ra. Một incident bắt đầu khi có tác động hoặc rủi ro đủ để cần phối hợp ứng cứu; incident có thể được khai báo từ cuộc gọi khách hàng dù alert im lặng. **Impact** mô tả ai bị ảnh hưởng, hành động nào hỏng, bao nhiêu sự kiện và trong bao lâu. **Severity** là mức tác động đã xác định theo chính sách tổ chức; **priority** là thứ tự xử lý có thể thay đổi vì thời gian, nghĩa vụ pháp lý hoặc rủi ro tiếp diễn. Không gán P1 chỉ vì một biểu đồ màu đỏ. Ngược lại, sai số tiền đã lưu là lý do hợp lý để đề xuất P1 dù HTTP 5xx bằng 0; người trực vẫn phải đối chiếu tiêu chí P1 thực tế của tổ chức.

Fixture `observations.csv` theo cùng một cửa sổ năm phút: v1 phục vụ 900 request, 1 số tiền sai; v2 phục vụ 100 request, 30 số tiền sai. Cả hai có `http_5xx=0`. Nếu SLI chỉ đếm status thành công thì sẽ báo 100% good, trong khi phép đối soát nghiệp vụ cho thấy 31/1.000 request sai, tức 3,1%. Trước canary, v1 ở cửa sổ trước có 1/900 sai; đây là baseline nhỏ, không chứng minh v1 hoàn hảo. Tỷ lệ v2 sai 30%, v1 khoảng 0,111% cùng thời điểm. Ta mới có một tương quan mạnh với release, chưa có root cause. Việc tập v2 là 10% lưu lượng cần kiểm chứng phân tuyến thực tế, không chỉ cấu hình cân bằng tải.

Tại 10:07, trong bài toán giả định, người trực tuyên bố incident và mở một tài liệu trạng thái sống: tác động đã biết, phần chưa biết, giả thuyết, người phụ trách, thời điểm cập nhật tiếp theo. Incident commander (IC) giữ mục tiêu và quyền điều phối; Ops lead thực hiện thay đổi kỹ thuật trong phạm vi được giao; Communications lead báo đều cho khách hàng và các nhóm liên quan, nêu điều đã xác minh, điều đang điều tra và mốc cập nhật. Nếu chỉ có một người, họ có thể tạm kiêm vai nhưng phải ưu tiên gọi hỗ trợ khi đồng thời vừa đổi hệ thống vừa báo cáo trở nên không an toàn. Escalation không phải chuyển trách nhiệm cho “ai đó”: nêu rõ cần quyền gì, quyết định nào, thời hạn nào. Nhật ký chung ghi cả thử nghiệm thất bại và lý do dừng để người sau không lặp thao tác nguy hiểm.

### Bài 49.2 — Giảm tác động

Từ `request_evidence.csv`, hai request v2 mang 4.100 và 5.900 cents được lưu thành 41 và 59 cents, nhưng đều trả HTTP 200; request v1 cùng đầu vào 4.100 giữ 4.100. DB time 42–49 ms và p95 toàn cửa sổ 180–190 ms không cho thấy tăng chậm lớn, nhưng vài mẫu không đủ loại bỏ mọi vấn đề database. Giữ bốn giả thuyết: DNS/edge đưa sai đích; DB làm chậm hoặc ghi sai; adapter v2 đổi đơn vị; feature flag chỉ áp cho nhóm canary. DNS sai toàn service khó giải thích khác biệt v1/v2 trong cùng cửa sổ, nhưng fixture chưa có edge destination hay revision cho từng request; cần đo chúng trước khi loại giả thuyết này. DB chậm không giải thích nhân 1/100 nhất quán; vẫn có thể có lỗi ghi logic ở tầng DB. Feature flag và adapter đều còn khả dĩ cho tới khi so config, diff và tái hiện cặp input–output trong fixture độc lập. Một diff **giả định** `store(amount_cents)` → `store(amount_cents // 100)` khớp hai mẫu v2; đây là bằng chứng cơ chế của tình huống, không là log/code production đã quan sát. Aggregate chỉ phân loại **30 request v2 sai**; số order hoặc bản ghi phân biệt bị ảnh hưởng là **UNKNOWN** vì retry và cập nhật lặp có thể cùng trỏ một order. Cần nối từng request ID với order ID rồi đối soát trạng thái lưu trước khi khẳng định phạm vi dữ liệu.

Đường giảm tác động chọn theo ranh giới dữ liệu. Tại 10:09 runbook **đề xuất** giữ digest, manifest, config flag, log/trace, phân tuyến và snapshot dữ liệu; hạn chế truy cập bản sao có thông tin khách hàng. Tại 10:12 **đề xuất** dừng đường ghi mới của v2 và đưa traffic về v1; chưa có bằng chứng thao tác đã hoàn tất. Hậu kiểm phải chứng minh không còn worker/queue consumer v2; chỉ rollback routing mà bỏ qua job nền là chưa đủ. Nếu lỗi nằm sau một flag độc lập, tắt flag có thể nhanh hơn, nhưng phải chứng minh phiên v2 không còn ghi sai. Failover sang vùng khác sẽ vô ích nếu cùng binary/config sai hoặc dữ liệu đã được sao chép; nó chỉ phù hợp nếu failure domain được chứng minh độc lập. Traffic shifting, rollback, flag và failover là các lựa chọn có tiền điều kiện/hậu điều kiện, không phải bốn nút “chữa lỗi” tương đương.

Mốc 10:16 là **phép hậu kiểm cần làm**, chưa có request ID, revision, input/stored hay kết quả đọc lại sau mitigation. Cần thử request mới qua v1 từ nhiều client/route, kiểm digest/revision đang phục vụ và xác nhận không còn v2 writer; chỉ sau đó mới có thể nói đường request mới phục hồi. Tại 10:22, dữ liệu hiện có vẫn chỉ cho biết 30 request v2 sai trong cửa sổ; chưa biết số order/bản ghi phân biệt, chưa có đối soát hay sửa dữ liệu. Vì vậy incident đang ở trạng thái *mitigation được đề xuất; phục vụ mới và phục hồi dữ liệu chưa xác minh*. Dashboard 5xx và CPU xanh không thay thế hai phép kiểm ấy. Một SLI nghiệp vụ đúng phải tiếp tục báo phần bad đã xảy ra; chính sách SLO xác định khi nào mở lại release, không được xóa dữ liệu lịch sử để “reset” ngân sách.

## Bài 49.3 — Học lại

**Incident timeline | fixture, không phải production**

| | | |
|---|---|---|
| **09:55** | **QUAN SÁT** | Baseline v1 ghi nhận 900 valid và 1 sai số tiền |
| **10:00** | **GIẢ ĐỊNH** | change rel-84 tạo revision 5 đưa v2 nhận 10 phần trăm request |
| **10:03** | **QUAN SÁT** | Hỗ trợ nhận báo cáo số tiền sai |
| **10:05** | **QUAN SÁT** | Chốt cửa sổ 5 phút v1/v2 và lấy mẫu K-26 đến K-28 |
| **10:07** | **ĐỀ XUẤT** | Tuyên bố incident P1 và phân IC/Ops/Comms |
| **10:09** | **ĐỀ XUẤT** | Giữ manifest digest config log trace và bản sao dữ liệu |
| **10:12** | **ĐỀ XUẤT** | Đề xuất chặn mọi v2 writer và chuyển request mới về v1; chưa có hậu kiểm |
| **10:16** | **CẦN XÁC MINH** | Cần thử request mới qua v1 và xác minh không còn v2 writer; chưa có kết quả |
| **10:22** | **CẦN ĐỐI SOÁT** | Đã phân loại 30 request v2 sai; số order hoặc bản ghi phân biệt chưa rõ |

31/1.000 request sai; số order phân biệt UNKNOWN; hậu kiểm 10:16 chưa chạy.

Hình XIII.2.1 — Trạng thái bằng chứng trong incident; 10:16 còn cần xác minh.

Timeline trong `timeline.csv` tách observation, giả định, đề xuất thao tác và verification còn cần làm. Cùng một câu “rollback xong” có thể có ba timestamp: lệnh hoàn tất, request mới đúng, dữ liệu cũ được sửa. Postmortem không ép ba mốc đó thành một. Khi điều tra sau giảm tác động, hỏi vì sao một thay đổi đổi đơn vị lọt qua: phép thử nào thiếu, canary gate chỉ nhìn status nào, telemetry có dấu hiệu gì nhưng không có cảnh báo, ai có quyền nâng traffic, và dữ liệu nào đáng lẽ được đối soát trước khi ghi. Giả thuyết root cause là adapter v2 đổi cents thành đơn vị nhỏ hơn; cần tái hiện trên input cụ thể và so cả diff/config trước khi ghi là xác nhận. Các điều kiện đóng góp có thể gồm test chỉ xác nhận 200, thiếu SLI số tiền, không có stop rule cho canary 10%; đó là lỗ hổng hệ thống cần khắc phục, không là lời buộc tội cá nhân.

Action item phải có chủ sở hữu, thời hạn và phép kiểm: (1) test âm cho 4.100 cents và kiểm dữ liệu lưu; (2) canary gate tính tỷ lệ sai nghiệp vụ theo release với mẫu số và missing-data policy; (3) runbook chặn mọi v2 writer và đối soát toàn bộ order ID; (4) diễn tập rollback app **và** kiểm sửa dữ liệu. “Cẩn thận hơn” không kiểm chứng được. Incident chỉ được đóng khi người phụ trách dữ liệu xác nhận phạm vi bản ghi, kế hoạch/sửa chữa, hậu điều kiện và thông báo cho người bị ảnh hưởng theo quy trình. Một postmortem không phải tài liệu để tìm người chịu lỗi; nó là hồ sơ bằng chứng và các thay đổi giảm xác suất hoặc tác động của lần sau.

### Bài tập 49.1 — Lỗi canary và giả thuyết nguyên nhân

Tính tỷ lệ sai của v1, v2 và toàn canary window. Một người kết luận “DB chậm là root cause” vì có vài request chậm; người khác kết luận “DNS sai” vì khách hàng báo lỗi. Dữ liệu nào chống lại hai kết luận ấy, dữ liệu nào còn thiếu? Hãy đề xuất thứ tự ba phép kiểm ít gây tác động nhất.

### Bài tập 49.2 — 5xx bằng không có nghĩa đã hồi phục?

Dashboard ghi HTTP 5xx bằng 0 trong cửa sổ quan sát. Runbook đề xuất chuyển về v1 lúc 10:12; phép thử request mới lúc 10:16 chưa có kết quả. Aggregate có 30 request v2 sai nhưng chưa biết số order phân biệt. IC có được đánh dấu incident `resolved` và mở lại v2 không? Tách bốn việc: mitigation, xác minh request mới, đối soát/khôi phục dữ liệu và điều kiện đóng incident.

### Bài tập 49.3 — Chọn rollback, tắt cờ hay failover

So sánh rollback ứng dụng, tắt feature flag và failover trong fixture. Nêu tiền điều kiện và một hậu điều kiện có thể đo cho mỗi lựa chọn; nếu dữ kiện thiếu thì ghi rõ không thể chọn an toàn.

# Chương 50 — Sao lưu, khôi phục thảm họa và dữ liệu

Phân biệt availability, durability, RPO, RTO và restore kiểm chứng.

## Bài 50.1 — Mục tiêu

**Toàn bộ thời điểm, đơn hàng và kết quả trong chương là fixture giả định. Chưa chạy restore PostgreSQL hay diễn tập trên hệ thống thật.** Mô hình trung tâm: chỉ một bản sao có thể khôi phục, đủ nhất quán và nằm ngoài miền lỗi cần chống mới có giá trị đối với mục tiêu phục hồi.

Khi ứng dụng mất kết nối database, khởi động lại Pod có thể khôi phục process nhưng không tái tạo một đơn hàng đã bị xóa. **High availability** giữ dịch vụ tiếp tục phục vụ khi một thành phần hỏng, thường nhờ bản sao hoặc chuyển vai; **disaster recovery** khôi phục sau biến cố vượt khả năng HA. Replication đưa thay đổi tới bản sao để giảm gián đoạn, đồng thời có thể sao cả thao tác xóa sai hoặc dữ liệu hỏng. Backup là một điểm dữ liệu có thể chọn để phục hồi; nhãn "backup thành công" chỉ nói đã tạo được bản sao theo một quy trình, chưa chứng minh đọc lại đúng nội dung. Snapshot là cách ghi trạng thái ở một thời điểm, nhưng tính nhất quán phụ thuộc tầng tạo snapshot và ứng dụng. Những cơ chế này bổ sung cho nhau; không suy ra backup từ số replica hay từ việc storage tuyên bố durable.

Mục tiêu giả định là **RPO không quá 15 phút**, **RTO không quá 45 phút**. RPO đo khoảng từ thời điểm mất khả năng ghi lúc 10:00 lùi đến commit cuối cùng thực sự khôi phục được. RTO đo từ 10:00 đến lúc đường nghiệp vụ đọc **và ghi** được kiểm chứng, không dừng ở thông báo "database started". Nếu chỉ đọc được một trang health check, chưa đạt hậu điều kiện. Kịch bản dùng base backup nhất quán lúc 09:00 và chuỗi WAL đã lưu liên tục đến commit 09:52. Sự cố lúc 10:00; phát hiện 10:04, bắt đầu restore 10:08, replay xong 10:27, xác minh dữ liệu và ghi/đọc ứng dụng xong 10:37. Theo fixture, RPO thực tế là 8 phút, RTO thực tế là 37 phút. Đây là phép tính từ timeline, không phải bảo đảm RPO/RTO cho mọi lần sự cố. Nếu WAL thực sự thiếu một đoạn trước 09:52, recovery point phải lùi hoặc restore thất bại; không được ghi 09:52 chỉ vì dashboard archive xanh.

Miền lỗi quyết định bản sao nào còn sống. Hai file trên cùng volume không bảo vệ khi volume mất; replica đồng bộ cùng account và quyền xóa không bảo vệ trước một lệnh xóa lan truyền. Bản sao phục hồi cần được kiểm tra ở miền lỗi tách biệt theo rủi ro đang xét, với chính sách quyền, phiên bản, thời hạn lưu và khóa giải mã còn dùng được. "Offsite" không tự động nghĩa là độc lập trước credential bị chiếm; cần thử kịch bản mất quyền, mất key và xóa nhầm bên cạnh lỗi máy.

Để chọn chiến lược, trước hết viết ra **mất mát nào phải chống**. Mất một tiến trình yêu cầu khởi động lại hoặc chuyển replica; lỗi một AZ yêu cầu một bản sao và đường chuyển lưu lượng ở miền còn sống; xóa nhầm lúc 09:57 yêu cầu một recovery point *trước* 09:57, thay vì replica đã nhận cùng lệnh xóa. Nếu cùng credential có thể xóa dữ liệu gốc lẫn mọi bản backup, tách AZ chỉ chống được lỗi hạ tầng, chưa chống được tài khoản bị chiếm. Một bảng rủi ro nên có sự kiện, miền ảnh hưởng, bản sao còn đọc được, người có quyền phục hồi và thời gian thử được. Không gộp tất cả vào chữ "có backup".

RPO và RTO còn phụ thuộc **điểm đo**. Giả sử người dùng nhận xác nhận tạo O-105 lúc 09:59 nhưng WAL khả dụng mới đến 09:52: cơ sở dữ liệu sau restore thiếu đơn ấy, dù ứng dụng trước sự cố đã trả

thành công. Sổ yêu cầu bên ngoài, thông điệp trong queue hoặc hóa đơn thanh toán có thể giữ dấu vết để đối soát, nhưng cũng có thể là bản sao không đầy đủ. Cần định nghĩa đơn vị cam kết: giao dịch đã commit, request đã được ACK, hay sự kiện đã xuất ra hệ khác. Nếu ba mốc không trùng nhau, chỉ số RPO tính từ commit database không diễn đạt hết mất mát mà khách hàng nhìn thấy. Quyết định phục hồi phải ghi các mốc ấy riêng và nêu dữ liệu nào có thể tái dựng, dữ liệu nào phải nhờ nghiệp vụ xử lý.

### Bài 50.2 — Bản sao

Một snapshot filesystem có thể **crash-consistent**: các byte giống lúc máy tắt đột ngột, database phải replay log và vẫn có thể phát hiện giao dịch chưa commit. Nó không mặc nhiên **application-consistent**: ví dụ hai hệ thống cùng ghi một đơn, một snapshot chụp sau DB commit nhưng trước khi đối soát message có thể giữ hai phía ở hai thời điểm khác nhau. Một base backup PostgreSQL hợp lệ cộng WAL liên tục cho phép khôi phục đến điểm mong muốn trong phần WAL còn có; việc sao chép riêng data directory của database đang chạy mà không theo quy trình backup không thay thế cơ chế ấy. Nếu cần nhất quán xuyên database, object store và queue, phải xác định invariant nghiệp vụ và cách đối soát hoặc đóng băng ghi; một snapshot từng hệ thống không tự tạo giao dịch liên hệ thống.

Trong fixture `orders.csv`, O-101 đến O-103 commit chậm nhất 09:52 và còn ở bản khôi phục. O-104 lúc 09:53 và O-105 lúc 09:59 không có trong recovery point; chúng nằm trong khoảng mất dữ liệu 8 phút đã chấp nhận theo RPO, nhưng vẫn cần đối chiếu nguồn giao dịch bên ngoài để xử lý. Một bản sao khác đến 09:59 có thể ít mất hơn, nhưng chỉ chọn được nếu kiểm tra tính toàn vẹn, thứ tự log và miền lỗi. Không đổi điểm khôi phục chỉ dựa vào timestamp của file hoặc “latest” trên UI.

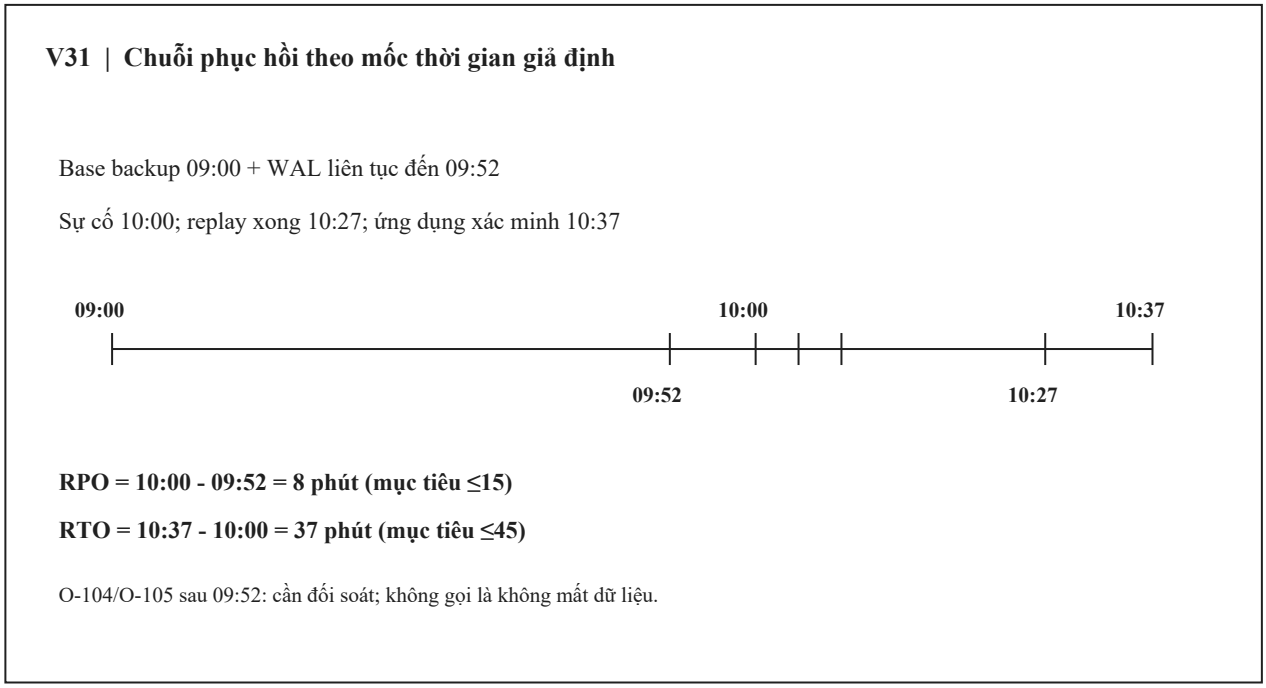
Retention xác định ta có thể quay về bao xa trước một lỗi xóa âm thầm được phát hiện muộn. Versioning hoặc bản sao khó sửa giúp giữ một phiên bản cũ, nhưng không loại rủi ro hết hạn retention, phá khóa mã hóa hoặc cấu hình cho cùng principal được xóa mọi phiên bản. Cần ghi danh mục backup, checksum hoặc kiểm tính toàn vẹn tương đương, chain WAL, phiên bản schema, key và quyền đọc. Hash khớp cho file chỉ xác nhận byte được so khớp; nó không chứng minh giao dịch nghiệp vụ đúng hoặc ứng dụng dùng được.

Hãy lần theo một lỗi ở ranh giới **database → hàng đợi**. Ứng dụng commit O-104 lúc 09:53, rồi bị dừng trước khi gửi sự kiện `OrderCreated`. Restore database tới 09:52 làm mất O-104; replay queue từ 09:50 có thể lại chứa sự kiện của đơn đã bị mất nếu producer khác đã gửi nó. Chiều ngược lại cũng có thể xảy ra: database giữ đơn nhưng queue không giữ sự kiện. Không có thứ tự phục hồi chung tự động giữa hai hệ. Phương án outbox đặt bản ghi sự kiện cùng giao dịch với đơn giải quyết khe hở commit–publish trong database, nhưng publisher vẫn có thể gửi lặp khi crash sau publish và trước đánh dấu đã gửi. Consumer vì thế cần một khóa idempotency và phép thử “gửi hai lần chỉ tạo một tác dụng nghiệp vụ”. Đây là phân tích giả định của giao thức; fixture hiện tại chưa chứng minh hệ `orders` triển khai outbox.

Khi chọn một mốc PITR, lập **bản kê quyết định** thay vì chỉ chọn file mới nhất: ID base backup, khoảng WAL liên tục có thể đọc, recovery target, timeline sau lần promote trước, bản ứng dụng/schema tương thích và bản sao còn giữ nguyên để quay lại. Nếu thiếu đúng một đoạn WAL

cần thiết, không nối hai đoạn ở hai phía rồi tuyên bố phục hồi liên tục. Chạy restore trên bản sao tách biệt, kiểm các invariant như "một ID đơn có đúng một trạng thái cuối" và "tổng thanh toán có thể đối soát"; sau đó mới quyết định chuyển writer. PostgreSQL chỉ khôi phục những gì trong phạm vi cluster và WAL được lưu; cấu hình máy, khóa, object bên ngoài và hành động nghiệp vụ vẫn cần quy trình riêng.

## Bài 50.3 — Diễn tập




Hình XIII.3.1 — V31: WAL, điểm khôi phục và hậu kiểm ứng dụng; RPO/RTO giả định.

Runbook bắt đầu bằng **dừng thao tác phá dữ liệu**: cô lập writer và job ghi, khóa quyền xóa trên bản sao, giữ ảnh hiện trạng và log trước khi thử restore. Nếu chưa xác định được backup chain, khóa giải mã, recovery target hoặc người phê duyệt ghi đè, dừng trước bước thay production. Diễn tập trên môi trường tạm độc lập: liệt kê base backup và WAL, kiểm tính liên tục, dựng một instance sạch, replay đến commit 09:52, kiểm tra trạng thái database, các order ID và tổng tiền. Sau đó thử bằng tài khoản ứng dụng một phép đọc và ghi có thể hoàn tác, đối soát message/queue, xác nhận route tới instance đúng và chỉ mới cân nhắc mở traffic. Nếu bài kiểm gây thay đổi dữ liệu, dùng fixture độc lập rồi dọn theo chính sách của môi trường thử; không chạy thử trực tiếp trên bản sao duy nhất.

Ở 10:27 chỉ có kết quả replay; 10:37 mới có hậu điều kiện ứng dụng. Báo người chịu trách nhiệm dữ liệu rằng O-104/O-105 chưa có trong điểm khôi phục và cần quy trình tái tạo/đền bù, tránh tuyên bố "không mất dữ liệu". Nếu client đã nhận xác nhận ghi sau 09:52, phải đối soát với nguồn ngoài DB để không tự phát hành trùng đơn. Khi chỉ có binary/app phục hồi còn nội dung sai, incident vẫn mở. Sau diễn tập, lưu thời điểm, hash, phiên bản, query kiểm tra, sai lệch, quyền đã dùng, thời gian từng bước và kết luận theo từng mục tiêu. Một lần PASS trong môi trường nhỏ không bảo đảm thời gian restore khi dữ liệu lớn hơn hoặc miền lỗi khác; lặp diễn tập với kích thước và điều kiện gần thực tế, không dùng nhãn PASS vĩnh viễn.

Diễn tập nên đo thời gian ở **từng pha**, vì cùng RTO 37 phút có thể che hai vấn đề khác nhau. Với fixture, 10:00–10:04 là phát hiện, 10:04–10:08 là ra quyết định, 10:08–10:27 là dựng và replay,

10:27–10:37 là kiểm chứng. Nếu dữ liệu lớn lên gấp bốn, không được nhân cơ học 19 phút replay với bốn: tốc độ đọc backup, giải nén, I/O ghi, WAL phát sinh và tranh chấp trên môi trường khôi phục có thể đổi khác. Hãy đặt giới hạn thời gian từng pha, đo kích thước byte và tốc độ thực trong diễn tập, rồi tính khoảng dự phòng cho kiểm ứng dụng. Một dashboard chỉ báo “replay hoàn tất” sẽ bỏ sót pha quyết định và hậu kiểm, nơi RTO thực có thể bị tiêu hết.

Sau chuyển lưu lượng, giữ một **cửa sổ xác nhận**: đọc O-101–O-103, từ chối hoặc ghi nhận đúng trạng thái O-104/O-105, tạo một đơn thử có khóa idempotency, đọc lại qua đường client, đối chiếu sự kiện đã phát và đo lỗi/độ trễ. Nếu đơn thử được ghi hai lần sau retry, restore có thể đã thành công ở tầng database nhưng đường nghiệp vụ chưa an toàn. Khi một invariant thất bại, dừng mở rộng traffic, giữ nguyên bản gốc để điều tra và ghi UNKNOWN cho mục tiêu chưa xác minh; không tiếp tục bằng cách sửa tay trên bản sao duy nhất.

**Bài tập 50.1 — Tính RPO và RTO từ dòng thời gian**

Từ `timeline.csv`, tính RPO/RTO thực tế và đánh giá hai mục tiêu. Nếu dashboard chỉ ghi “restore xong” lúc 10:27, vì sao không thể dùng 27 phút làm RTO? Nếu thiếu WAL 09:45–09:46, có còn chứng minh được recovery point 09:52 không?

**Bài tập 50.2 — Đơn hàng ở điểm khôi phục**

Đối chiếu `orders.csv`: đơn nào chắc chắn hiện diện ở recovery point, đơn nào cần đối soát bên ngoài? Một kỹ sư đề xuất chuyển sang replica cùng account rồi xóa bản chính để tiết kiệm thời gian. Nêu ít nhất ba điều kiện cần xác minh trước bất kỳ thao tác hủy dữ liệu nào.

**Bài tập 50.3 — Hậu kiểm một lần restore**

Thiết kế hậu kiểm cho một restore: phân biệt kiểm byte, kiểm database, kiểm ứng dụng và kiểm nghiệp vụ. Nêu một tình huống mà checksum PASS nhưng vẫn chưa được mở ghi production.

# Chương 51 — Hiệu năng, năng lực tải và thử giới hạn

Phân biệt nghẽn CPU, I/O, hàng đợi và giới hạn dữ liệu khi tải tăng.

## Bài 51.1 — Tải công việc

**Mọi số liệu trong `load.csv` là fixture tính toán giả định, không phải benchmark đã chạy.** Mô hình trung tâm: request đến, chờ, dùng tài nguyên rồi hoàn tất hoặc bị từ chối; capacity là mức tải giữ được SLO và khả năng phục hồi với biên an toàn đã kiểm, không phải một giá trị CPU.

Một request có **latency** từ điểm người dùng bắt đầu đo tới kết quả họ nhận; một service có **throughput** là số request hoàn tất mỗi giây, còn **arrival/offered rate** là số request đi vào mỗi giây. Khi hàng đợi tăng, hai rate này khác nhau. **Concurrency** là số request đang ở trong hệ thống, gồm đang chờ và đang phục vụ; utilization là phần tài nguyên bận theo một định nghĩa quan sát cụ thể. Một CPU 70% không cho biết thread đang đợi DB, khóa, đĩa, network hay một shard nóng. Cần đo cùng cửa sổ và cùng population; trung bình latency có thể ổn dù p95/p99 vi phạm mục tiêu của một nhóm người dùng.

Trong fixture, mỗi pha dài 60 giây và không có retry, reject hay timeout ẩn. Pha A nhận và hoàn tất 100 request/s, latency trung bình 0,120 s, p95 0,180 s. Pha B nhận và hoàn tất 160 request/s, trung bình 0,250 s, p95 0,520 s. Nếu hai pha đã ổn định và số vào bằng số ra, Little's Law cho số request trung bình trong hệ thống **L = λW**: A là 100 request/s × 0,120 s = 12 request; B là 160 × 0,250 = 40 request. `L` không phải số thread hay CPU core; nó gồm cả chờ. Cùng định nghĩa "vào hệ thống" và "rời hệ thống" phải áp cho λ, W và L. Từ hai con số này không suy được phân bố độ trễ hay bottleneck; dữ liệu p95 và counter tài nguyên vẫn cần thiết.

Pha C nhận 220/s nhưng chỉ hoàn tất 180/s trong 60 giây. Nếu không từ chối, timeout, hủy hoặc retry ẩn và cùng ranh giới đo, **số request chưa hoàn tất** tăng `(220−180)×60 = 2.400 request`. `outstanding_delta` là chênh lệch tổng request đang chờ **và** đang xử lý, không phải phép đo riêng độ dài queue. Hệ thống chưa ổn định: không lấy 220×0,550 để gọi là concurrency trung bình theo Little's Law; 0,550 s chỉ đo trên nhóm đã hoàn tất và bỏ lại các request chưa hoàn tất. Ngay cả 180×0,550 cũng không tự cho tổng request đang có khi outstanding thay đổi. Phải đo riêng độ dài queue đầu/cuối, số in-flight, drop, timeout và retry; nếu một counter thiếu, số 2.400 chỉ là hệ quả **có điều kiện** của fixture.

Phân biệt **thời gian phục vụ** với thời gian chờ bằng một mô hình có thể tự tính. Giả sử một worker xử lý đúng 10 request/s khi luôn bận, và có 12 worker độc lập không tranh chấp; trần lý tưởng là 120/s. Nếu 160/s đến đều trong 10 giây mà không reject, số chờ tăng ít nhất `(160−120)×10 = 400`, trước khi tính request còn đang chạy. Đây là cận từ giả định về năng lực, không phải dự đoán p95: scheduling, phân bố thời gian phục vụ và shared DB có thể làm trần thấp hơn. Nếu queue chỉ giữ 200 phần tử, sau khi đầy phải có nơi ghi nhận số bị từ chối; nếu load generator bỏ qua các phản hồi ấy thì biểu đồ latency của request thành công sẽ đẹp lên một cách đánh lừa. Theo cùng ranh giới, kiểm phương trình bảo toàn `đến = hoàn tất + từ chối + hủy + Δ đang xử lý/chờ`; lệch số là dấu hiệu instrument hoặc cách đếm retry chưa rõ.

Little's Law áp cho trung bình dài hạn của hệ ổn định, không đòi từng request có cùng latency. Nó không cho biết tail latency, vị trí chờ hay nguyên nhân chờ. Trước khi dùng `L=λW`, xác định cửa vào có bao gồm retry không, timeout được tính là rời hệ ở đâu, và "hoàn tất" là trả response hay ghi thành công xuống dependency. Nếu số đến 160/s gồm 20/s retry của cùng giao dịch, throughput 160/s không đồng nghĩa 160 giao dịch người dùng/s; capacity nghiệp vụ phải dùng mẫu số khác.

### Bài 51.2 — Năng lực phục vụ

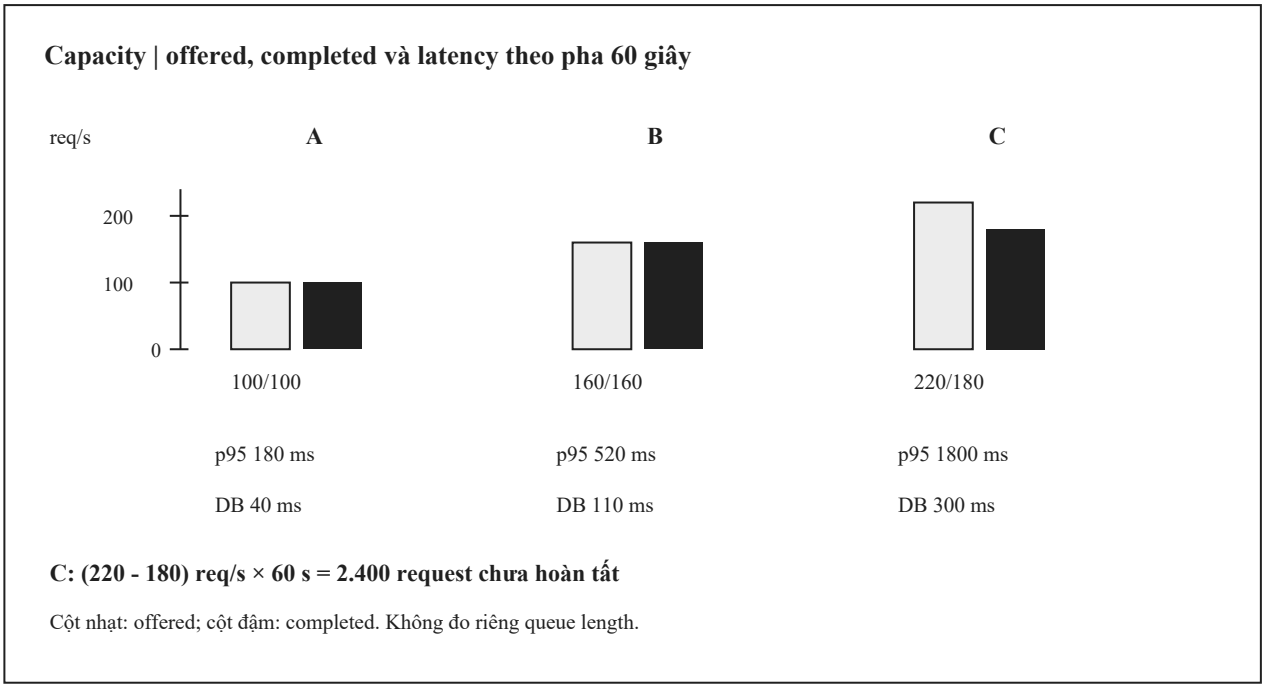
Ở A→B, CPU giả định tăng 45%→65% nhưng p95 từ 180→520 ms; DB wait p95 từ 40→110 ms. Ở C, CPU chỉ 70% trong khi DB wait p95 300 ms, p95 request 1.800 ms và số chưa hoàn tất tăng. Tương quan cho thấy cần kiểm tra DB/connection pool và queue trước khi thêm replica; không chứng minh DB là root cause vì client mix, lock, hot key hoặc đo sai cũng có thể tạo dấu hiệu giống vậy. CPU phải được đọc kèm quota/throttling, per-core skew, run queue và thời gian chờ I/O. Memory phải phân biệt RSS/page cache, reclaim, OOM và GC; I/O cần độ trễ, queue depth, throughput; network cần loss/retransmit, RTT và connection errors. Một graph CPU thấp là một observation, không phải chứng thư còn capacity.

Capacity forecast bắt đầu từ nhu cầu và hậu điều kiện. Nếu peak hiện tại là 160 request/s và dự báo tăng 25%, peak mới là **200 request/s**. Fixture chỉ có pha B ổn định 160/s; pha C 220/s có số request chưa hoàn tất tăng. Vì vậy không thể nội suy rằng 200/s đạt SLO hoặc suy ra cần đúng 1,25 lần số replica. Nếu SLO giả định là p95 ≤600 ms và error rate <0,1% tại peak, B qua được điều kiện latency trong fixture, nhưng chưa có số liệu error rate và độ dài chạy nên cả SLO chưa được chứng minh. C cần stop vì số chưa hoàn tất tăng và p95. Biên an toàn phải xét mất một instance, rollout, autoscaler trễ, burst, skew tenant và shared dependency. Tăng replica có chi phí và có thể đẩy thêm tải vào DB; đặt queue hữu hạn và shed tải có kiểm soát có thể giữ phần request được phục vụ đúng hạn, nhưng sẽ ảnh hưởng availability SLI. Quyết định cần xem cả ngân sách lỗi P13.C01 và chi phí, không tối ưu p95 bằng cách lặng lẽ bỏ request khỏi mẫu số.

Để biến tương quan thành một chẩn đoán có thể bác bỏ, dựng **hai dự đoán đối nghịch**. Giả thuyết H1: pool DB giới hạn số query song song. Khi tăng arrival 160→200/s, số request chờ lấy connection, p95 thời gian acquire và chiều dài wait queue sẽ tăng trước khi CPU API tăng mạnh; tăng replica API không làm p95 giảm nếu tổng pool connection tới DB vẫn bị chặn. Giả thuyết H2: một replica API bị CPU throttle; khi tách latency theo replica, chỉ replica ấy có thời gian runnable nhưng không chạy và p95 cao, còn DB acquire không đổi. Đo cùng cửa sổ `connection acquire`, DB query execution, số connection đang dùng/chờ, CPU throttled time và latency theo replica. Nếu số chờ pool không tăng và query time ổn trong lúc throttling tăng ở đúng replica chậm, H1 yếu đi. Nếu mọi replica cùng chờ DB và throttling không đổi, H2 yếu đi. Một biểu đồ p95 toàn fleet không phân biệt được hai trường hợp.

Thử tăng một replica chỉ khi đặt **dự đoán định lượng trước**: ví dụ tại 180/s, pool wait p95 phải giảm ít nhất 30% và error rate không tăng trong ba cửa sổ ổn định. Giữ traffic mix, dữ liệu và số connection tối đa cố định để biết phép thử thay cái gì; nếu tăng replica cũng tăng tổng connection, kết quả không cô lập nguyên nhân. So cả DB CPU/lock và request timeout. Nếu API khỏe hơn nhưng DB bắt đầu reject connection, rollback giả thuyết "thêm replica tăng capacity" và xem giới hạn ở dependency. Không chỉnh đồng thời pool, replica và query rồi gán cải thiện cho một nút.

## Bài 51.3 — Thử nghiệm giới hạn




Hình XIII.4.1 — Tải A/B/C: offered và completed; chênh lệch C là tổng outstanding.

Một **load test** chạy mức tải gần dự báo để kiểm SLO; **stress test** tăng vượt vùng dự kiến để tìm ngưỡng bão hòa và hành vi từ chối; **soak test** giữ tải đủ lâu để lộ leak, tích queue, hết connection/disk hay drift chậm. Tên test không tự chứng minh kết quả: dữ liệu phải có traffic mix, kích thước payload, think time hoặc arrival model, trạng thái cache, phiên bản artifact, topology, warm-up, thời lượng và điểm đo phía client. Một máy phát tải bão hòa có thể tạo số liệu giả; đo cả generator. Histogram p95 phải giữ population và bucket phù hợp; không lấy trung bình p95 giữa các pod như p95 toàn fleet.

Đề xuất thí nghiệm **chưa chạy** cho peak 200/s: dùng môi trường cô lập với cấu hình và dữ liệu đại diện nhưng không chứa dữ liệu khách hàng; kiểm baseline 160/s, tăng từng bậc 180→200→220/s và giữ mỗi bậc đủ lâu để thấy queue đầu/cuối. Ghi offered, accepted, completed, rejected, timed-out, retries, latency client p50/p95/p99, in-flight, queue, CPU quota/throttle, memory/GC, disk/network và DB wait theo cùng cửa sổ. Stop ngay nếu p95 vượt 600 ms hai cửa sổ liên tiếp, backlog tăng quá giới hạn định trước, dữ liệu sai, hoặc generator bão hòa; giữ khả năng phục hồi trước khi tăng tiếp. Sau giảm tải, xác nhận queue rút, request mới đúng và không có job nền tiếp tục gây tải. Chỉ khi dữ liệu thật đạt hậu điều kiện và thử thêm mất một replica mới chốt mức capacity. Không failure-inject production để "cho chắc" nếu chưa định blast radius và đường quay lui.

Mô hình phát tải thay đổi câu hỏi thí nghiệm. Một **closed-loop client** chờ response rồi mới gửi request tiếp; khi server chậm, nó tự giảm offered rate. Vì vậy một bài thử đặt 100 client có thể che vùng bão hòa: throughput đứng lại nhưng số "người dùng ảo" vẫn đủ. Mô hình phát theo lịch arrival cố định tiếp tục ghi request đến, timeout, reject và độ trễ ngay cả khi server chậm; nếu generator không đủ năng lực hoặc backlog của chính generator tăng, lịch phát thực khác lịch định trước. Ghi cả thời điểm request **đáng lẽ** phát và thời điểm thực phát để thấy phần trễ do generator. Không so p95 giữa hai mô hình khi population và cách xử lý timeout khác nhau.

Một báo cáo thí nghiệm nên chứa một bảng cho từng bậc với `arrived`, `accepted`, `completed`, `rejected`, `timed_out`, `in_flight_start/end` và `generator_lag`. Dùng phương trình bảo toàn để tìm mẫu bị bỏ sót; ghi cả response body sai, không chỉ HTTP status. Sau bậc 200/s, hạ xuống 160/s: nếu backlog không rút hoặc p95 không trở lại baseline, hệ có thể đang giữ công việc nền, leak hoặc DB chưa phục hồi. Khi tắt tải, chờ và đo thời gian hồi phục trước khi tuyên bố PASS; bài thử chỉ chứng minh cấu hình, dữ liệu và cửa sổ vừa dùng, chưa chứng minh mọi burst production.

**Bài tập 51.1 — Concurrency theo Little's Law**

Từ A và B tính concurrency trung bình theo Little's Law. Ở C tính mức tăng số request chưa hoàn tất trong 60 giây, rồi giải thích vì sao 220×0,550 không là concurrency đã chứng minh. Có thể kết luận riêng độ dài queue tăng bao nhiêu không? Nêu hai counter có thể bác bỏ giả định của phép tính.

**Bài tập 51.2 — CPU 70% có đủ để tăng replica?**

Một người nói “CPU 70% nên chỉ cần thêm 25% replica để chịu peak 200/s”. Dựa vào dữ liệu A–C, chỉ ra ba lỗ hổng trong suy luận, một giả thuyết bottleneck và phép đo có thể bác bỏ nó. Kết luận có được triển khai 200/s chưa?

**Bài tập 51.3 — Thiết kế phép thử năng lực tải**

Thiết kế một phép thử capacity cho peak 200/s với tiêu chí dừng, hậu kiểm và thông tin cần ghi để tái lập. Phân biệt load, stress và soak; nêu một ví dụ khi kết quả PASS ở môi trường thử vẫn chưa đủ để công bố production capacity.

# PHẦN XIV — Bảo mật và DevSecOps

## Mối đe dọa đi qua identity, artifact, secret và runtime thế nào?

Đặt trust boundary và tài sản trước công cụ; giảm quyền, chứng minh provenance, kiểm soát secret và đo hiệu lực policy.

XIV **LỘ TRÌNH PHẦN**


- Chương 52 — Mô hình đe dọa và quyền tối thiểu
- Chương 53 — Bí mật và bảo vệ dữ liệu
- Chương 54 — Chuỗi cung ứng phần mềm
- Chương 55 — Gia cố môi trường đám mây và Kubernetes

# Chương 52 — Mô hình đe dọa và quyền tối thiểu

Nhận diện tài sản, actor, trust boundary và đường lạm dụng.

## Bài 52.1 — Bề mặt

Thiết kế bảo mật bắt đầu bằng tài sản và đường đi, không bắt đầu bằng danh sách công cụ. Với dịch vụ `orders-api`, tài sản là đơn hàng, credential đọc kho dữ liệu, artifact triển khai và khả năng sửa policy. Actor gồm client có tài khoản, người vận hành, pipeline CI, Pod và đối thủ có thể điều khiển request hoặc một dependency. Vẽ luồng client → edge → API → database/S3, thêm luồng CI → registry → cluster và operator → control plane. Mỗi mũi tên ghi dữ liệu, identity, giao thức, điểm xác thực và điểm ủy quyền; nếu không biết một mũi tên dùng credential nào, đó là câu hỏi cần điều tra chứ chưa phải một ranh giới đã được bảo vệ.

Một trust boundary xuất hiện khi quyền kiểm soát thay đổi: Internet sang edge, Pod sang Kubernetes API, workload sang cloud API, hoặc CI sang registry. Private subnet chỉ đổi khả năng định tuyến trực tiếp; nó không chặn một request hợp lệ qua ALB khai thác lỗi ủy quyền ở API. TLS bảo vệ một chặng truyền có cấu hình đúng, nhưng không quyết định user A có được đọc đơn của user B. Một image được ký có thể vẫn chứa lỗ hổng logic. Đặt control đúng vị trí bằng cách nêu giả thuyết tấn công và hậu điều kiện quan sát được.

Tình huống giả định: client gửi `GET /orders/41` với token hợp lệ của user 7, nhưng đơn 41 thuộc user 8. Edge kiểm chữ ký token và chuyển request; API chỉ kiểm "đã đăng nhập" rồi đọc ID từ URL. Sự cố là thiếu object-level authorization ở API, dù WAF, TLS và private subnet đều hoạt động. Phép thử âm phải yêu cầu user 7 bị từ chối, user 8 đọc được; thêm kiểm audit record không lộ payload. Nếu API dùng cache, cache key cũng cần chứa phạm vi chủ sở hữu hoặc kiểm lại quyền trước khi trả. Các user/ID chỉ là fixture, không phải dữ liệu khách hàng thật.

Threat model là giả thuyết có thể cập nhật: ghi tài sản, actor, đường vào, lạm dụng, control, phép thử và rủi ro còn lại. Khi đổi đường dữ liệu hoặc thêm service account, cập nhật sơ đồ và phép thử. Một sơ đồ đẹp không đủ nếu không chỉ ra ai có thể vượt ranh giới và bằng chứng nào sẽ bác bỏ giả định an toàn.

Phân rã đường tấn công của O-41 theo **điều kiện cần**, thay vì liệt kê lỗ hổng. Đối thủ cần (1) token được edge chấp nhận, (2) khả năng yêu cầu ID của chủ khác, (3) API tra đơn chỉ theo ID hoặc cache trả bản ghi không gắn owner, và (4) response lộ dữ liệu. Chặn ở bước (3) bằng truy vấn ràng buộc `order_id` **và** `owner_id` lấy từ danh tính đã xác thực; kiểm quyền ở tầng đáng tin trước khi cache response. Nếu ID khó đoán nhưng bước (3) vẫn thiếu, rủi ro chưa bị loại bỏ: ID có thể lộ qua link, log hoặc chia sẻ. Unit test chỉ gọi `GET /orders/41` với chủ sở hữu không kiểm đường tấn công; cần ca âm dùng user 7 và quan sát không có nội dung O-41 trong body, cache và log.

Thay một giả định để thấy threat model đổi thế nào: edge cache theo URL và phục vụ response trước khi API chạy kiểm quyền. Khi đó sửa truy vấn API chưa đủ; cache key phải phân biệt phạm vi danh tính hoặc cache không được lưu response riêng tư. Với mỗi control ghi rõ chủ sở hữu triển khai, vị trí thực thi, dữ liệu đầu vào đáng tin, ca âm và tín hiệu khi thất bại. Một `403` từ edge không chứng minh API an toàn nếu client nội bộ có đường đi vòng qua edge. Thử đường nội bộ trong môi trường

được phép, hoặc đánh dấu phạm vi chưa kiểm. Đây là cách biến sơ đồ ranh giới thành các dự đoán có thể bác bỏ.

**Bài tập 52.1 — Quyền truy cập từng đơn hàng**

Token user 7 hợp lệ nhưng đơn 41 thuộc user 8. TLS, WAF và private subnet đều hoạt động. Control nào còn thiếu, và hai phép thử nào cần có?

**Bài 52.2 — Quyền**

Quyền truy cập là quan hệ giữa principal, action, resource và context. Trong AWS, workload giả định chạy dưới role session của `orders-api`; một policy cho `s3:GetObject` trên prefix báo cáo chưa đủ nếu explicit Deny từ policy áp dụng hoặc điều kiện context không khớp. Trong Kubernetes, ServiceAccount xác định identity của Pod với API server; Role/ClusterRole và binding quyết định các verb trên loại resource. Hai hệ quyền này khác nhau: được đọc ConfigMap không suy ra được đọc S3, và được đọc S3 không suy ra được sửa Deployment. Phải kiểm từng cạnh qua đúng identity mà tiến trình đang dùng.

Áp dụng least privilege bằng cách xuất phát từ một thao tác nghiệp vụ: `orders-api` cần đọc object trong prefix báo cáo, không cần liệt kê mọi bucket, ghi policy hay tạo access key. Tách role chạy ứng dụng khỏi role CI triển khai và role con người xử lý sự cố. Người duyệt quyền và người tự cấp quyền không nên là cùng một principal nếu yêu cầu kiểm soát thay đổi. Nhưng chia role theo tên không đủ; cần kiểm trust policy, policy được gắn, đường assume role và các quyền gián tiếp như cập nhật code hoặc template của workload có role mạnh.

Một ma trận nhỏ cho fixture có thể ghi bốn phép: user 8 đọc đơn 41 được; user 7 đọc đơn 41 bị chặn; Pod `orders-api` đọc đúng object được; Pod đó ghi object hoặc đọc prefix khác bị chặn. Mỗi ô có action/resource/identity chính xác và kết quả kỳ vọng. Thử bằng credential tạm của đúng workload trong môi trường cô lập, không dùng credential admin để "đại diện". Nếu một phép denial thất bại, dừng rollout và thu hẹp policy; log phải có request ID và quyết định ủy quyền nhưng không chứa token.

Một quyền có thể hợp lệ hôm nay nhưng thừa sau thay đổi kiến trúc. Review định kỳ theo usage, đường tấn công và owner, đồng thời giữ đường khẩn cấp có thời hạn và audit. Không tự rút quyền trên production chỉ dựa vào một log thiếu event; sampling, vùng lưu log và retention có thể làm bằng chứng không đầy đủ. Least privilege là vòng điều chỉnh có phép thử dương và âm, không phải một mẫu JSON bất biến.

Xét một **đường leo quyền gián tiếp**: role A không có `s3:GetObject` cho prefix của user khác nhưng được sửa Deployment của Pod B; Pod B lại chạy bằng identity có quyền đọc prefix đó. Khi A thay image hoặc command của B, quyền sửa workload có thể trở thành đường tiếp cận credential hoặc dữ liệu của B. Tương tự, quyền sửa policy, gắn role, cập nhật trust relationship hoặc đọc secret cấp credential có thể mạnh hơn tập action API trực tiếp nhìn thấy. Vì vậy ma trận quyền phải gồm cả cạnh "có thể thay đổi principal nào" và "có thể khiến code nào chạy dưới principal ấy", không chỉ liệt kê API cuối cùng. Mức khai thác thực tế phụ thuộc admission policy, audit, quyền đọc output và cấu hình runtime.

Một thiết kế kiểm thử có ba lớp. Lớp 1 kiểm policy tĩnh: role ứng dụng chỉ chứa action/resource dự kiến, trust chỉ cho đúng workload assume. Lớp 2 kiểm identity **thực** trong Pod/runner: request tới tài nguyên cho phép thành công, request sang prefix khác bị từ chối. Lớp 3 kiểm quyền quản trị gián tiếp: principal CI có thể cập nhật Deployment nào, gắn ServiceAccount nào, sửa trust/policy nào. Nếu lớp 1 PASS nhưng lớp 2 sử dụng credential khác dự kiến, lớp 1 không còn chứng minh least privilege của tiến trình. Ghi account/namespace, session identity, action, resource và context điều kiện cho mỗi phép thử; che token trong artifact kiểm thử.

**Bài tập 52.2 — Quyền ConfigMap không phải quyền S3**

Pod đọc được ConfigMap. Có thể kết luận Pod cũng đọc được S3 không? Nêu ca âm least privilege.

**Bài 52.3 — Bằng chứng**

Một control chỉ đáng tin khi có chứng cứ nó đã chặn hành vi thuộc phạm vi thiết kế. Audit log cho biết một sự kiện được ghi, không tự chứng minh mọi request đều đã được ghi hoặc rule đã chặn đúng. Với `orders-api`, liên kết thay đổi policy tới commit, người duyệt, deployment revision và một bộ request có danh tính rõ. Phép thử dương xác nhận user/Pod hợp lệ vẫn làm được việc; phép thử âm xác nhận user/Pod ngoài phạm vi bị từ chối. Cả hai cần ghi thời điểm, identity và resource giả định, status thực và nơi tìm log; nếu log thiếu, kết quả "không thấy sự kiện" là UNKNOWN cho claim về audit.

Giả định một PR thêm `s3:*` cho role ứng dụng để chữa lỗi AccessDenied. Review không nên chỉ hỏi JSON hợp cú pháp. Tái hiện request lỗi, đối chiếu action và object, kiểm route/endpoint trước, rồi đề xuất quyền nhỏ nhất. Một ca âm thử `PutObject` vào prefix khác phải vẫn bị từ chối. Sau áp dụng, so policy hiệu lực và truy vết một request thật; một simulator chỉ mô phỏng những policy/context được nhập, không thay request ở đường thực. Nếu policy sửa rộng làm ca âm PASS trái kỳ vọng, rollback policy theo phiên bản đã biết và kiểm lại workload.

Chứng cứ còn có giới hạn thời gian: role session có thể giữ credential tạm, cache ủy quyền có thể tồn tại, và log có thể đến trễ. Vì vậy hậu kiểm gắn với thời điểm thay đổi và ít nhất một request sau thời điểm ấy. Không lấy policy trong Git làm bằng chứng duy nhất cho trạng thái cloud hoặc cluster hiện tại. Đối chiếu configuration, object thực và request. Sự khác nhau giữa ba thứ là drift cần phân tích, không che bằng một dashboard xanh.

Trước khi mở rộng quyền trong sự cố, ghi stop rule, chủ sở hữu, thời hạn và phương án thu hồi. Nếu thiếu quyền làm gián đoạn dịch vụ, quyết định có thể là cấp tạm trong phạm vi hẹp; sau đó phải review nguyên nhân gốc, ca âm và xóa ngoại lệ. Một quy trình có log đẹp nhưng không kiểm quyền phủ định vẫn có thể để lại lối đi tắt lâu dài.

Đặt **hậu điều kiện có thể kiểm** cho ngoại lệ kéo dài 30 phút trong fixture. Trước khi cấp: lưu policy hiệu lực và phiên bản workload, xác định đúng object báo lỗi và owner duyệt. Trong cửa sổ ngoại lệ: chỉ cho `GetObject` trên prefix cần thiết, gắn thời điểm hết hạn ở hệ quản lý thay đổi, ghi request ID của ca dương và ca âm. Khi hết hạn: thu hồi, thử lại bằng session mới và xác minh ca âm vẫn bị chặn; đồng thời xem session cũ còn dùng credential đến khi hết TTL hay không. Nếu chưa kiểm được session cũ, không tuyên bố quyền đã bị thu hồi tức thời. Sau đó so request lỗi ban đầu với log ở

điểm ra quyết định: nếu hóa ra object không tồn tại hoặc route sai, ngoại lệ là chữa nhầm nguyên nhân và phải xem dữ liệu nào đã bị truy cập ngoài mục đích.

Một phép thử bảo mật cũng có thể **tự gây hại**. Dùng ID khách hàng thật để thử deny dễ đưa dữ liệu vào log; thử `PutObject` hoặc xóa trên bucket thật có tác dụng phụ ngay cả khi định kiểm quyền. Dùng fixture riêng, action vô hại theo scope được giao, và cleanup có chủ sở hữu. Khi không thể chạy ca âm trên production, tách kết luận "policy đã review" khỏi "runtime denial đã được chứng minh"; lập kế hoạch kiểm ở môi trường có cùng đường identity. Phân biệt chứng cứ tĩnh, mô phỏng và request thật làm hồ sơ an toàn hữu ích hơn một nhãn PASS chung.

**Bài tập 52.3 — Chứng cứ trước khi mở rộng quyền S3**

PR thêm s3:* để sửa AccessDenied. Chứng cứ nào cần trước và sau khi cấp quyền?

# Chương 53 — Bí mật và bảo vệ dữ liệu

Lần theo secret từ tạo tới sử dụng, log, rotation và thu hồi.

## Bài 53.1 — Vòng đời

Secret có vòng đời gồm phát hành, phân phối, lưu trữ, sử dụng, xoay và thu hồi. Gọi mọi byte bí mật là "mật khẩu" che mất khác biệt: API token có scope, private key dùng ký, database credential có quyền theo user, và khóa mã hóa bảo vệ nhiều bản sao dữ liệu. Mỗi loại cần owner, TTL, nơi sinh, nơi được phép đọc, cơ chế rotate và bằng chứng revoke. Nếu không biết credential nào đang được ứng dụng sử dụng, đổi giá trị trong kho bí mật chưa phải rotation hoàn tất.

Theo dõi một token giả định `orders-api` cần gọi S3 qua role session. Đường ít lưu secret tĩnh hơn là workload nhận credential tạm từ identity provider đúng scope. Nhưng credential vẫn xuất hiện trong bộ nhớ tiến trình và có thể bị lộ qua crash dump, biến môi trường, log debug hoặc truy vấn metadata trái phép. Tại mỗi chặng, ghi ai được phép đọc, dữ liệu được mã hóa khi nào và key thuộc quyền ai. Encryption at rest của state hoặc volume chỉ che một lớp lưu trữ; code đang chạy vẫn cần plaintext đúng thời điểm, nên quyền runtime và audit là các control riêng.

Rotation phải xét trạng thái hai đầu. Nếu app dùng secret A mà backend chuyển thẳng sang B, request đang chạy có thể lỗi; nếu backend chấp nhận A và B trong một cửa sổ, rollout app B theo từng đợt, kiểm request rồi revoke A sẽ giảm gián đoạn. Không mọi hệ hỗ trợ hai credential đồng thời; khi không có, cần kế hoạch dừng ghi hoặc giao thức chuyển phù hợp. Ghi thời điểm issue/revoke, version tham chiếu và số instance còn dùng A. Một biến đã đổi trong CI không chứng minh container cũ đã restart hoặc database đã từ chối A.

Thử trong fixture không dùng secret thật: token A/B giả và API mô phỏng trả trạng thái theo version. Ca dương kiểm B dùng được sau rollout; ca âm kiểm A bị từ chối sau revoke; ca lỗi kiểm rollback ứng dụng về A sau khi A đã bị thu hồi sẽ không được coi là phục hồi. Nếu ca âm không chạy được vì quyền môi trường, ghi SKIP và không tuyên bố revoke thành công. Chính vòng đời và hậu điều kiện mới là kết quả cần bảo vệ.

Để không lẫn **ý định xoay** với trạng thái thực, theo dõi bốn cột ở mỗi mốc: backend chấp nhận credential nào, kho bí mật đang cấp version nào, từng instance đang dùng version nào, và request qua instance đó có thành công không. Ví dụ giả định 10:00 backend chấp nhận A; 10:05 backend chấp nhận A+B; 10:08 hai trong ba Pod dùng B; 10:12 Pod cuối vẫn giữ A vì chưa reload. Thu hồi A lúc 10:09 sẽ tạo lỗi ở Pod cuối dù hai Pod đầu đều xanh. Chỉ khi mọi endpoint còn nhận traffic đã dùng B và phép thử A bị từ chối theo hợp đồng mới đóng việc xoay. Nếu không xác định được version của một Pod, cô lập nó khỏi traffic hoặc giữ cửa sổ chồng lấp theo quyết định rủi ro có thời hạn; không đoán từ `Deployment Available`.

TTL còn tạo **đường lỗi sau thời điểm rollout**. Credential tạm có thể hợp lệ tại request thử 10:08 nhưng hết hạn 10:30; nếu tiến trình không renew hoặc lấy credential mới, lỗi chỉ xuất hiện về sau. Với cơ chế lease, ghi lease ID/version, thời điểm hết hạn thực do hệ cấp trả về, khả năng renew và cách ứng dụng phản ứng khi renew thất bại. Đừng giả định TTL yêu cầu trong cấu hình là TTL được cấp thực. Thử bằng fixture thời gian ngắn: request trước hạn qua, renew bị chặn thì tiến trình chuyển

sang credential mới hoặc fail closed theo thiết kế, không fallback sang secret tĩnh trong image. Kết quả ấy chứng minh một nhánh lỗi của fixture, chưa chứng minh mọi backend revoke tức thời.

**Bài tập 53.1 — Đổi token khi container cũ còn chạy**

10:05 backend nhận A+B; hai Pod đã dùng B, Pod thứ ba còn A vì chưa reload. Token B thử thành công; lease của B hết hạn lúc 10:30. Có thể thu hồi A và báo rotation xong lúc 10:08 không? Nêu phép kiểm tại 10:12 và sau 10:30.

**Bài 53.2 — Bề mặt lộ**

Telemetry có thể lộ dữ liệu nhạy cảm dù ứng dụng chính không trả nó cho client. Chọn thuộc tính có cardinality hữu hạn như route template, service và nhóm lỗi; giữ request ID chỉ ở log/trace có kiểm soát truy cập, không thêm token, mật khẩu, cookie, payload, email hay raw URL chứa định danh vào log/span. Cấu hình lọc ở điểm instrument trước khi dữ liệu rời tiến trình, thêm processor kiểm tra ở Collector, phân quyền và retention tại backend; lọc muộn không thu hồi dữ liệu đã gửi hoặc bản sao đã lưu. Sampling và retention là đánh đổi giữa chi phí và khả năng điều tra: tăng chi tiết chỉ khi có mục tiêu kiểm chứng và biết dữ liệu nào bị bỏ. Kiểm tra cả sức khỏe pipeline quan sát, nhưng không coi nó là thay thế cho probe/request nghiệp vụ.

Đường lộ không chỉ ở telemetry. CI có thể in biến khi bật debug hoặc khi một utility echo argv; môi trường của subprocess có thể kế thừa credential dù tác vụ không cần; Terraform state có thể chứa giá trị nhạy cảm ngay khi output được đánh dấu sensitive; một Docker image layer có thể giữ secret đã COPY vào rồi xóa ở layer sau. Với mỗi bản sao, hỏi ai đọc được byte thật, nơi nó tồn tại bao lâu và có bị nhân vào cache, artifact hoặc backup không. Dòng **** trong một log chỉ chứng minh một formatter đã che tại đó, không chứng minh mọi kênh khác sạch.

Tình huống giả định: token thử nghiệm `TOK-DEMO-41` đi từ kho bí mật vào job build, được viết nhầm vào `.env`, rồi file bị COPY vào image trước khi một lệnh RUN xóa nó. Kiểm filesystem cuối của container có thể không thấy `.env`, nhưng layer cũ trong image vẫn chứa byte. Đó là lỗi thiết kế đường build; không giải bằng cách chỉ xóa file ở layer mới. Cần dựng lại image từ context sạch, invalidate artifact/digest cũ, revoke token, kiểm registry/cache và truy vết nơi image đã được phân phối. Trong sách chỉ dùng chuỗi giả; tuyệt đối không thử bằng secret thật.

Một phép kiểm âm quét repository, CI log, state được phép đọc, image layers và telemetry theo marker giả. Kết quả âm có phạm vi: chỉ các artifact và thời đoạn đã kiểm, không chứng minh tuyệt đối không còn bản sao. Khi gặp dấu hiệu rò thật, dừng phát hành, xoay/revoke credential, giới hạn quyền truy cập bằng chứng và ghi timeline. Không đưa chính secret vào báo cáo sự cố; dùng hash có cân nhắc, ID và vị trí được kiểm soát để định danh.

Điều tra sự cố `TOK-DEMO-41` phải tách **thu hồi quyền** khỏi **xóa bản sao**. Revoke token làm giá trị cũ mất khả năng gọi API trong phạm vi backend thực hiện revoke; xóa image tag không xóa các digest đã pull, cache registry, layer trên runner hay bản backup. Ngược lại, dọn artifact mà chưa revoke vẫn để token dùng được ở nơi đã sao chép. Vì vậy trình tự khẩn cấp là giới hạn quyền token, revoke/rotate và kiểm ca âm qua API; song song khóa đường phân phối, liệt kê digest và job đã dùng, làm sạch theo chính sách lưu giữ. Mọi kết luận “không còn bản sao” phải nêu những kho đã

kiểm và những nơi chưa truy cập được.

Trước khi build lại, sửa **điểm đưa secret vào**: không truyền credential qua `ARG`/`ENV` để nó đi vào image metadata/history hoặc build context; dùng cơ chế secret mount thích hợp cho build, giới hạn quyền tải dependency và tránh ghi token vào file/lệnh echo. Sau build, tìm marker giả trong layer, build log và artifact đầu ra, rồi kiểm ứng dụng vẫn lấy credential runtime từ identity dự kiến. Một scan chuỗi không phát hiện trong image mới là phép kiểm hữu ích nhưng có thể bỏ qua dạng đã mã hóa, copy khác hoặc cache ngoài phạm vi. Mục tiêu là xóa nguyên nhân tạo bản sao, không chỉ thay tên token.

**Bài tập 53.2 — Bí mật còn trong layer của image**

Token giả TOK-DEMO-41 từng được COPY vào image rồi xóa ở layer sau. Image mới không còn marker, nhưng digest cũ đã pull trên runner và có trong registry cache. Nêu hai việc độc lập phải làm và giới hạn của phép quét.

**Bài 53.3 — Kiểm soát**

KMS và Vault giải các phần khác nhau của bài toán: một hệ quản lý khóa có thể tạo, bảo vệ và kiểm soát thao tác mã hóa/giải mã; một kho secret có thể cấp, lưu phiên bản và phân phối credential. Dùng sản phẩm nào cũng phải xác định principal được quyền giải mã hoặc đọc secret, chính sách audit, TTL, backup và phương án mất hệ cấp khóa. Nếu workload có quyền đọc mọi secret trong cùng namespace, dữ liệu được mã hóa at rest vẫn lộ khi workload bị chiếm. Quyền theo đúng tên secret và workload identity là điều kiện vận hành, không chỉ cấu hình storage.

Mô hình envelope encryption phân biệt data key dùng mã hóa payload và key mã hóa data key. Lưu ciphertext và encrypted data key cạnh dữ liệu không cho phép tự giải mã nếu không có quyền KMS, nhưng app được cấp quyền decrypt vẫn nhận plaintext trong bộ nhớ. Rotation của wrapping key không tự đổi mọi token đã bị lộ, và revoke quyền KMS có thể làm hệ đang chạy không đọc được backup. Trước khi đổi key, thử restore một bản sao trong môi trường tách biệt và ghi rõ key version, quyền cần thiết, thời gian phục hồi. Backup có byte nhưng thiếu key hữu dụng không đạt mục tiêu phục hồi.

Giả định `orders-api` cần đọc database password `db/orders/v3`. Thiết kế cho Pod identity chỉ đọc đúng đường đó, CI chỉ có quyền triển khai reference chứ không đọc giá trị, và operator khẩn cấp dùng quyền có thời hạn. Hậu kiểm gồm request dùng v3 thành công, v2 bị từ chối sau revoke, log/trace không chứa giá trị, audit ghi lượt đọc hợp lệ và bất hợp lệ. Một alert audit im lặng có thể do pipeline hỏng; thử ca âm bằng fixture để chứng minh đường quan sát, không dùng credential thật.

Khi hệ quản lý secret lỗi, chọn rõ chế độ: app đang chạy có thể tiếp tục với credential đã cache trong TTL, nhưng instance mới không lấy được secret; hoặc fail closed và ngừng xử lý. Không nên âm thầm fallback sang plaintext trong image hay Git. Runbook ghi thời điểm hết TTL, số instance ảnh hưởng, đường khôi phục key và quyết định dừng rollout. Thiết kế này là giả định để học phân tích; không phải chứng nhận một cấu hình KMS/Vault cụ thể đã được triển khai.

Trong envelope encryption, phân biệt **xoay khóa bao** với **thay data key**. Nếu ciphertext của một bản backup được mã hóa bằng data key D, còn D được bao bằng key K1, việc đổi phiên bản key

quản lý sang K2 không tự tạo lại ciphertext backup hoặc thay D. Có thể tái bao D dưới K2 theo giao thức sản phẩm, nhưng phải giữ khả năng giải mã bản cũ cho đến khi đã chuyển và thử restore tất cả bản còn trong retention. Nếu hủy K1 trước khi một backup cũ còn cần K1, byte backup vẫn nguyên mà mục tiêu phục hồi thất bại. Ngược lại, nếu D hoặc plaintext đã lộ, chỉ tái bao D không thu hồi dữ liệu đã bị đọc; cần đánh giá lại phạm vi rò và có thể phải mã hóa lại dữ liệu bằng data key mới. Đây là mô hình cơ chế, còn hành vi key rotation tự động tùy dịch vụ cần đối chiếu tài liệu đúng phiên bản.

Một phép diễn tập **mất hệ cấp khóa** nên kiểm ba nhóm instance. Nhóm đang chạy với data key/credential còn cache có thể tiếp tục trong TTL; nhóm restart có thể không giải mã được cấu hình; bản restore ở môi trường khác có thể không có quyền KMS dù ứng dụng cũ vẫn chạy. Ghi thời điểm của từng trạng thái và request đọc/ghi được phép. Nếu chỉ kiểm service hiện hữu rồi tuyên bố DR đạt, nhánh “dựng lại từ backup trong miền lỗi khác” vẫn chưa được thử. Hậu điều kiện là bản phục hồi mới giải mã được dữ liệu bằng đúng identity, ứng dụng đọc/ghi đúng và audit không lộ plaintext; thiếu quyền hoặc key version thì ghi FAIL/UNKNOWN theo phép thử, không chuyển sang lưu plaintext để cứu tiến độ.

**Bài tập 53.3 — Backup không còn khóa giải mã**

Backup cũ dùng data key D được bao bằng K1. Hệ chuyển sang K2, sau đó định hủy K1 trước khi thử restore. Có thể kết luận backup còn phục hồi được không? Nếu chính D đã lộ, tái bao D dưới K2 có đủ không?

# Chương 54 — Chuỗi cung ứng phần mềm

Truy vết source→dependency→build→image→deploy không đánh tráo artifact.

## Bài 54.1 — Nguồn

Một commit ID không mô tả toàn bộ đầu vào build nếu dependency được tải động. Trước khi build `orders-api`, ghi commit, lockfile của ngôn ngữ, digest base image, phiên bản toolchain và nguồn các gói tải về. Constraint như ">=" cho phép nhiều phiên bản hợp lệ; cache của hai runner có thể chọn byte khác nếu không khóa hoặc không kiểm checksum. Dependency pin làm đầu vào xác định hơn, nhưng không tự chứng minh gói không có mã độc. Khi nâng gói, review diff API, license, advisory và phạm vi test; đừng chỉ nhìn tên version tăng.

SBOM là bản kê các component theo một định dạng có cấu trúc, ví dụ SPDX; nó cho phép hỏi "artifact này khai báo chứa gì?" nhưng câu trả lời phụ thuộc thời điểm tạo, độ bao phủ và cách nhận diện component. Một SBOM của source tree có thể khác image cuối do build stage, thư viện hệ thống hoặc gói tải runtime. Gắn SBOM với digest artifact, generator/version và thời điểm. Nếu SBOM chỉ gắn tag có thể di chuyển, không thể chắc nó mô tả image đang chạy. Quét vulnerability theo SBOM cần so phiên bản và ngữ cảnh reachability; một scan xanh chỉ nói database/rule tại thời điểm scan không phát hiện, không là chứng minh vô lỗ hổng.

Giả định commit `c84` pin thư viện A v1 và base image digest D1; build sau đó vô tình dùng base tag `stable` đã chuyển sang D2. Manifest image digest thay đổi dù commit không đổi. Điều tra đầu vào trước khi đánh giá code: so lockfile, base digest, build parameters và provenance. Nếu image D2 đã được ký nhưng policy chỉ kiểm chữ ký của builder, chữ ký không bác bỏ việc builder đã dùng dependency ngoài kỳ vọng. Stop promotion, ghi digest thực, tái build với đầu vào đã xác định và chạy gate trên byte mới. Các mã c84/D1/D2 là giả định, không phải image thật đã quét.

Để kiểm toán, đưa vào ledger đường commit → dependency selection → build inputs → digest image → SBOM subject. Mỗi mũi tên có bằng chứng riêng. Nếu một mũi tên thiếu, ghi UNKNOWN và phạm vi ảnh hưởng; không lấp bằng suy luận từ tên tag hoặc log "success". Bài này chuẩn bị cho provenance và admission ở hai bài tiếp theo.

Xét một **phụ thuộc bắc cầu** trong fixture: ứng dụng pin gói A phiên bản 1.4, A khai báo gói B với dải phiên bản cho phép, còn lockfile của job phát hành bị bỏ qua. Hai runner cùng checkout `c84` có thể lấy B khác nhau; cùng tên commit vì thế tạo hai image digest. Ghi bản khóa của toàn bộ cây phụ thuộc, checksum của gói, nguồn registry và toolchain thực đã dùng. Một build lại cho digest khác là tín hiệu điều tra chứ chưa tự chứng minh tấn công: timestamp, thứ tự file, base image hoặc compiler cũng có thể làm byte khác. Muốn kết luận "build tái lập", phải loại hoặc cố định nguồn bất định rồi so digest trên hai môi trường độc lập đủ tương đương.

SBOM cần được đọc theo **vị trí tạo**. SBOM từ source trước build có thể ghi gói chỉ dùng cho test mà image runtime không chứa; SBOM lấy từ image có thể thấy thư viện hệ điều hành mà source scan bỏ qua. Ngược lại, plugin tải tại runtime có thể vắng khỏi cả hai bản kê nếu không quan sát đường tải ấy. Khi một advisory nêu B, hỏi: B nằm trong artifact digest nào, được nạp ở runtime nào, và chức năng bị lỗi có đường vào từ request không? `not affected` phải có lý do kiểm được, không chỉ vì

một scanner không báo. Nếu chưa xác định được vị trí B, giữ trạng thái UNKNOWN và chặn promotion theo mức rủi ro đã định.

**Bài tập 54.1 — Base image đổi dù commit không đổi**

Cùng commit c84 nhưng base tag stable chuyển D1 sang D2 và lockfile bị bỏ qua khiến gói bắc cầu B thay phiên bản. Hai image khác digest. Scan/chữ ký của bản đầu có áp cho bản sau không, và cần khóa những đầu vào nào?

**Bài 54.2 — Dựng sản phẩm phần mềm**

Provenance mô tả artifact được tạo bởi builder nào, từ input và tham số nào, với subject digest nào. Attestation là bản ghi có thể được ký về một phát biểu; chữ ký kiểm được byte bản ghi và key ký, không tự chứng minh builder trung thực. Nếu người tấn công điều khiển builder hoặc key, một attestation hợp lệ vẫn có thể chứng thực dữ liệu sai. Vì vậy cần bảo vệ builder, phân quyền trigger, cách lấy source/dependency và key custody; provenance là bằng chứng mạnh hơn log tùy tiện khi những giả định ấy được kiểm soát.

Một build cô lập giảm khả năng job A đọc secret hoặc cache của job B, nhưng phải định nghĩa mức cô lập: filesystem, network, credential, runner lifetime và artifact store. Đầu ra của `orders-api` cần có digest tính từ byte, provenance subject trùng digest đó, source commit đã resolve và dependency theo bản khóa. Nếu build lại cho production, digest mới cần gate mới; ký một tag không cố định không bảo đảm image sau này cùng byte. Tính tái lập độc lập, đã học ở P06, là phép kiểm khác với chữ ký: một bên so byte của hai build, bên kia chứng minh ai đã ký một phát biểu.

Tình huống giả định: provenance ghi subject H84, registry hiện tag `prod` trỏ H84b. Kiểm chữ ký provenance của H84 PASS không cho phép deploy H84b. So digest trong admission request với subject, kiểm identity builder và policy nguồn/branch được phép, rồi mới cho triển khai. Nếu mismatch, stop và điều tra tag bị dời hoặc artifact bị rebuild; đừng sửa provenance thủ công để khớp tag. H84/H84b là ký hiệu giả định, không phải output registry đã đọc.

Ngay cả khi registry và subject đều là H84, còn một phép đối chiếu độc lập. Theo mô hình SLSA v1.1, `runDetails.builder.id` nêu build platform mà ta phải quyết định có tin hay không; `buildDefinition.buildType` cho biết cách hiểu quy trình; `externalParameters` ghi đầu vào do bên gọi điều khiển, còn `resolvedDependencies` có thể giữ commit đã giải ra từ một ref. Giả sử policy phát hành yêu cầu commit `c84` của repository đã duyệt, nhưng attestation ký hợp lệ cho H84 ghi ref `release` và commit thực được resolve là `c83`. Không được suy từ tên ref `release` hay từ chữ ký rằng policy nguồn đã đạt: phải so commit đã resolve với `c84` và từ chối hoặc điều tra. Nếu attestation không ghi đủ dependency cần để làm phép so, trạng thái kiểm nguồn là UNKNOWN chứ không tự chuyển thành PASS. Ví dụ này chỉ kiểm tính nhất quán của bằng chứng với policy đã định; nó không chứng minh builder không bị xâm nhập hoặc `c84` không chứa lỗi.

Giới hạn bảo đảm phải ghi cùng kết quả. Provenance không chứng minh ứng dụng không có lỗ hổng logic; SBOM không bảo đảm mọi dependency runtime đã được liệt kê; signature không bảo đảm key chưa lộ. Đặt thêm test hành vi, scan có dữ liệu cập nhật, review quyền và phép quan sát production. Trong báo cáo, tách “digest đã khớp attestation” khỏi “hệ thống an toàn”, vì mệnh đề sau rộng hơn

nhiều.

Hãy kiểm một attestation như một **mệnh đề có ba cạnh**, thay vì dấu tích chữ ký. Cạnh thứ nhất là subject: digest H84 trong attestation phải bằng digest image định triển khai, không phải tag `prod`. Cạnh thứ hai là nguồn: repository và commit đã resolve phải thuộc danh sách được phép, không chỉ có ref tên `release`. Cạnh thứ ba là builder: danh tính/ký danh của môi trường build phải đúng policy và bản ghi phải do khóa đang được tin cậy phát ra. Nếu H84 khớp nhưng commit là c83, từ chối vì sai nguồn; nếu c84 khớp nhưng builder lạ, từ chối vì sai môi trường; nếu cả hai khớp nhưng subject H84b, từ chối vì sai byte. Mỗi nhánh là một phép kiểm độc lập và cho nguyên nhân điều tra khác nhau.

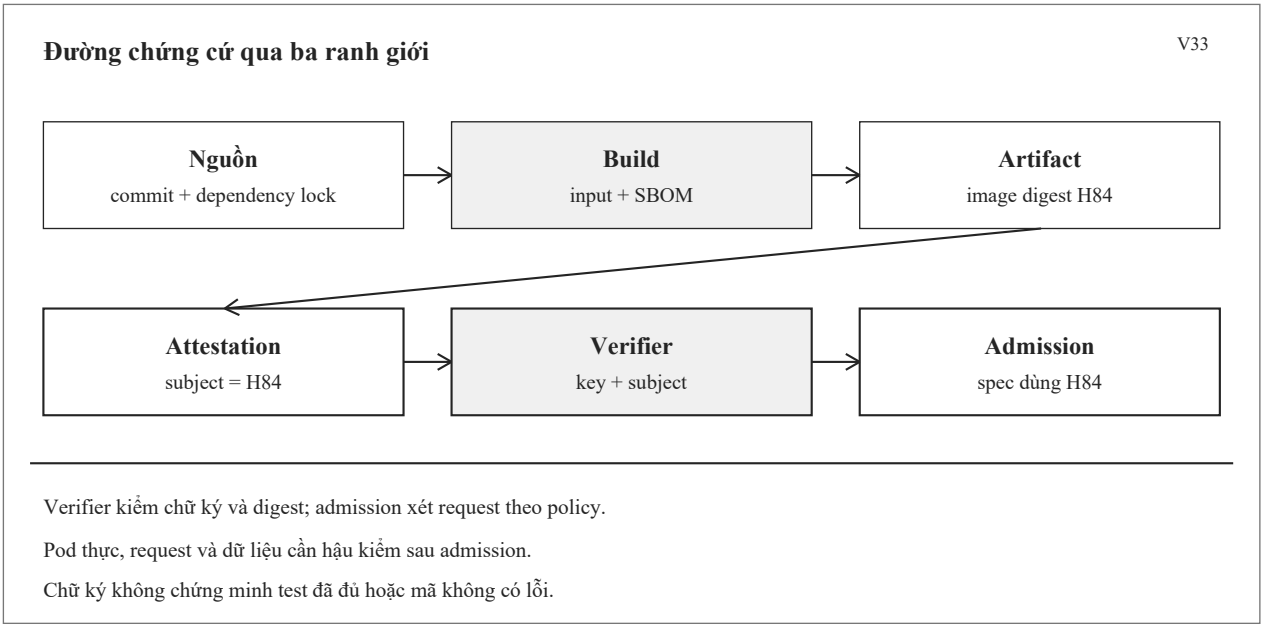
Định nghĩa rõ **ai có quyền chọn đầu vào**. Một workflow có thể chỉ build từ nhánh được bảo vệ, nhưng nếu job nhận URL dependency hay tham số build tùy ý từ một PR chưa tin cậy thì provenance "builder hợp lệ" vẫn có thể ghi byte do người ngoài chỉ định. Đối chiếu `externalParameters` với policy và ghi `resolvedDependencies` thành digest/commit cụ thể khi phần mềm build cho phép. Không phải mọi trường của provenance đều chứng minh tính đầy đủ; thiếu một dependency quan trọng là giới hạn bằng chứng. Cô lập runner và không đưa credential phát hành vào job nhận code chưa tin cậy làm giảm rủi ro *trước* khi chữ ký được tạo, thay vì chờ admission phát hiện mọi vấn đề.

**Bài tập 54.2 — Chữ ký đúng nhưng tag trỏ digest khác**

Attestation ký hợp lệ có subject H84 và tag prod cũng trỏ H84, nhưng commit đã resolve là c83 trong khi policy yêu cầu c84. Có được deploy không? Phân biệt ba phép kiểm độc lập.

**Bài 54.3 — Triển khai sản phẩm**

Deployment an toàn dùng image digest đã kiểm thử, không dùng một tag có thể di chuyển làm identity cuối. Registry trả manifest/index với digest; khi kéo image theo digest, client yêu cầu đúng byte manifest ấy, còn nội dung layer được tham chiếu bằng digest riêng. Policy admission có thể từ chối Pod không chỉ rõ digest hoặc không có attestation phù hợp. Nhưng admission chỉ xét request tạo/cập nhật theo policy tại thời điểm đó; nó không tự kiểm mọi process đã chạy, không tự sửa image cũ và không thay runtime monitoring.




Hình XIV.3.1 — Chứng cứ nguồn, build và deploy: chữ ký không chứng minh chất lượng ứng dụng.

Gắn chuỗi từ CI tới cluster: commit `c84` → image H84 → test result có H84 → attestation subject H84 → Deployment spec dùng H84 → Pod thực tế báo imageID tương ứng. Mỗi bước ghi timestamp và namespace. Nếu spec dùng H84 nhưng một Pod cũ còn chạy H83 trong rollout, không được nói toàn bộ lưu lượng đã chuyển; xem ReplicaSet/Pod đang Ready, endpoint và request theo revision. Nếu tag `prod` sau đó chuyển H84b, Pod đã pull H84 không nhất thiết đổi, còn Pod recreate theo tag có thể dùng byte khác. Suy luận phải theo digest Pod thực và thời điểm pull.

Một policy còn phải có đường cập nhật và revoke. Khi phát hiện key ký bị lộ, chỉ kiểm "signature valid" là vô nghĩa cho các bản ký bởi key ấy. Thu hồi trust của key, xác định artifact đã được chấp nhận, pin digest đã xác minh bằng kênh độc lập, và quyết định dừng rollout hoặc thay Pod. Nếu chứng cứ không đủ để phân biệt H84/H84b, ghi UNKNOWN, giới hạn blast radius và kiểm request/data; không tự xóa mọi workload đang chạy khi chưa có kế hoạch phục hồi.

Phép thử âm cho fixture là một Pod spec dùng digest khác subject của attestation phải bị từ chối; phép thử dương là digest đúng được chấp nhận và sau đó request nghiệp vụ hoạt động. Nếu không có policy engine trong môi trường, chỉ kiểm tĩnh hợp đồng và ghi SKIP cho admission runtime. Kết quả scan, chữ ký và policy là các control theo ranh giới; chúng không thay hậu kiểm dữ liệu sau triển khai.

Một **race giữa policy và trạng thái chạy** cần được xét trong hậu kiểm. Admission có thể từ chối Pod mới dùng H84b sau khi policy siết lại, nhưng Pod H84b đã được tạo trước đó vẫn có thể tiếp tục phục vụ cho tới khi bị thay thế. Vì vậy khi có thông báo digest hoặc key bị thu hồi, liệt kê workload spec và Pod thực đang chạy theo imageID, map tới Service endpoint và lượng traffic, rồi lên kế hoạch thay hoặc cô lập theo blast radius. Không xóa tức thì mọi Pod chỉ vì một scanner báo tên gói nếu chưa biết đường phục hồi và tác động dữ liệu; cũng không coi policy mới là đã chữa workload cũ.

Một rollout chứng minh chuỗi artifact phải có **hai phép đóng sổ**. Đóng sổ kỹ thuật: spec trỏ H84, tất cả Pod phục vụ mục tiêu có imageID H84, endpoint thuộc revision dự kiến, request canary đi đúng

Pod và dữ liệu đúng. Đóng sổ bằng chứng: lưu digest, attestation, kết quả test và policy version đã dùng ở thời điểm cho phép. Nếu một Pod cũ H83 vẫn nằm ở endpoint, ghi phạm vi còn lại thay vì nói "toàn bộ đã cập nhật". Nếu một Pod H84 không Ready, chữ ký và SBOM của H84 không giải thích readiness; quay lại probe, cấu hình và log ứng dụng. Việc liên kết hai sổ này giúp tránh nhầm an toàn chuỗi cung ứng với thành công vận hành.

**Bài tập 54.3 — Pod cũ còn nhận request khi rollout**

Admission policy mới từ chối Pod H84b, nhưng một Pod H84b tạo trước policy vẫn Ready và có Endpoint. Có thể coi rủi ro đã hết không? Nêu cách đóng sổ rollout.

# Chương 55 — Gia cố môi trường đám mây và Kubernetes

Đặt ranh giới quyền và mạng trên workload thật.

## Bài 55.1 — Kiểm soát lúc tiếp nhận

Admission xử lý một request tới Kubernetes API sau xác thực và ủy quyền, trước khi object mới được lưu. Pod Security Admission có thể áp mức baseline/restricted cho namespace theo chế độ enforce, audit hoặc warn; policy engine khác có thể thêm điều kiện như image digest, registry cho phép hay label bắt buộc. RBAC trả lời caller có quyền gửi request hay không; admission trả lời object cụ thể có đạt chính sách hay không. Cả hai không thay kubelet, runtime hoặc network policy ở đường chạy.

Giả định namespace `orders` yêu cầu container không đặc quyền và image tham chiếu digest. Một Pod spec thêm `privileged: true` phải bị chặn trước khi tạo nếu policy enforce áp đúng namespace và version. Nếu chỉ ở chế độ warn, API có thể vẫn chấp nhận object và trả cảnh báo; một thông báo "policy violation" không chứng minh đã chặn. Kiểm nhãn namespace, version policy, kết quả request, audit event và object có thật hay không. Ca dương dùng Pod hợp lệ để chắc policy không vô tình làm mọi deployment dừng; ca âm thử đặc quyền hoặc tag không digest trong fixture.

Admission webhook hoặc policy engine là dependency của control plane: timeout, failurePolicy và availability quyết định request được phép đi tiếp hay fail closed khi webhook lỗi. Không đặt một webhook chưa thử vào đường production rồi coi "deny" là an toàn tuyệt đối; nó có thể chặn cả recovery. Trước rollout policy, kiểm trường hợp API không gọi được webhook, workload hiện hữu, ngoại lệ hệ thống và đường break-glass có owner/thời hạn. Một policy chỉ áp cho object mới không tự sửa Pod cũ đang chạy; cần inventory và kế hoạch thay chúng.

Admission có phạm vi hạn chế: image được phê duyệt có thể vẫn chứa lỗi logic; spec hợp lệ không chứng minh app Ready; và một controller có quyền mạnh có thể sửa object sau đó. Kết nối policy với provenance digest, RBAC của người đổi policy, audit và request nghiệp vụ sau rollout. Không bịa kết quả từ cluster thật; các ca trên là hợp đồng kiểm thử cho môi trường cô lập được cấp quyền.

Tách hai cơ chế trong một ca cụ thể. Namespace `orders` có nhãn PSA `pod-security.kubernetes.io/warn=restricted` nhưng chưa có `pod-security.kubernetes.io/enforce=restricted`; giả sử không có admission khác chặn, Pod đặc quyền được API chấp nhận và object có UID mới, đồng thời client nhận warning. Đó là cảnh báo có tác dụng quan sát, không phải chặn. Khi chuyển sang enforce ở version đã chọn, cùng Pod spec mới phải nhận response từ chối và không có object mới. Object cũ được tạo trước khi đổi nhãn không tự biến mất. Để xác nhận chính sách vận hành, kiểm response, UID/object sau request và audit cùng thời điểm; chỉ nhìn dòng warning hoặc một Pod đang Running sẽ dễ nhầm thời điểm áp policy.

Có thêm một ranh giới khi Pod do controller tạo. Với Kubernetes v1.34, `enforce` của PSA áp lúc **tạo Pod**, không từ chối trực tiếp object Deployment chỉ vì Pod template của nó vi phạm. Giả sử namespace `orders` đặt `enforce=restricted` và `warn=restricted`, người gọi được phép tạo Deployment `orders-web` nhưng template có `privileged: true`. API có thể nhận Deployment

và trả warning; ReplicaSet sau đó thử tạo Pod và bị admission từ chối. Khi ấy `kubectl apply` trả thành công nhưng số Pod mới bằng không, ReplicaSet có sự kiện `FailedCreate`, và Deployment chưa đạt số replica mong muốn. Đó là enforce đang chặn đúng ở ranh giới Pod, không phải policy thất bại. Đọc response của lần tạo Deployment, sự kiện ReplicaSet và danh sách Pod theo thời điểm; đừng lấy exit status của `apply` làm hậu điều kiện triển khai. Với một Pod tạo trực tiếp có cùng spec, request tạo Pod phải bị từ chối. PSA kiểm các Pod Security Standards, không kiểm image có được pin bằng digest; muốn chặn image tag phải có một chính sách admission khác và kiểm nó bằng ca âm riêng.

Với validating webhook tùy chỉnh, failurePolicy là một nhánh khác: nếu webhook không trả lời trước timeout, `Fail` làm API request bị từ chối vì không lấy được quyết định, còn `Ignore` cho request đi tiếp qua webhook đó. Đây là xử lý lỗi webhook, không phải kết quả đánh giá policy "đạt". Các plugin admission khác vẫn có thể từ chối request. Khi thử rollout, tạo ca policy vi phạm và ca webhook không sẵn sàng trong cluster cô lập, ghi response và object sau request cho cả hai. Chọn fail closed hay fail open cần xét độ sẵn sàng control plane và đường cứu hộ, không suy từ tên cờ mà hệ thống luôn an toàn.

Trước khi bật enforce diện rộng, đo số workload sẽ bị ảnh hưởng, phân biệt Pod do controller tạo với Pod do người dùng tạo, ghi owner và thời hạn cho ngoại lệ, rồi thử rollback policy khi webhook lỗi. Giữ một đường quan sát độc lập về request bị từ chối; nếu audit pipeline cũng hỏng cùng webhook, việc mất log không chứng minh admission vẫn thông. Hậu điều kiện là workload hợp lệ vẫn triển khai, spec bị cấm không được tạo mới, và ứng dụng hiện hữu vẫn phục vụ trong suốt thay đổi.

**Bài tập 55.1 — Deployment được nhận nhưng Pod bị từ chối**

Namespace orders bật PSA enforce=restricted và warn=restricted trên Kubernetes v1.34. Apply Deployment có Pod template privileged=true trả thành công kèm warning; ReplicaSet báo FailedCreate và không có Pod mới. Enforce đã thất bại chưa? Nếu một Pod khác chỉ dùng image tag nhưng đạt restricted, PSA có từ chối không? Nêu bằng chứng cần xem.

**Bài 55.2 — Gia cố lúc chạy**

Pod chạy dưới ServiceAccount mặc định hoặc được chọn rõ; token của nó có thể cho tiến trình gọi Kubernetes API nếu được mount và được RBAC cho phép. `runAsNonRoot` giảm một lớp quyền trong container, nhưng không chặn ứng dụng đọc token được gắn vào Pod hoặc gọi API với quyền đã cấp. Role cho phép `get secrets` trong namespace là đường truy cập khác với khả năng root trong container. Hãy xem SA name, automount, RoleBinding và phép thử `can-i` cho đúng verb/resource/namespace, rồi kiểm request thật trong môi trường tạm nếu được phép. `can-i` là phép dự đoán ủy quyền, chưa thay kiểm ứng dụng và volume.

Tách rõ hai đường tới Secret. Giả sử Pod `orders-api` dùng ServiceAccount `orders-app` không có quyền API `get` hay `list` Secret, đặt `automountServiceAccountToken: false`, nhưng Pod spec khai báo Secret `db-credential` dưới dạng volume và container mount tại `/run/credentials`. Kubelet trên node lấy Secret được Pod yêu cầu và đưa byte vào volume; tiến trình không cần gọi API bằng token của `orders-app` để đọc tệp đã mount. Vì vậy `kubectl auth can-i get secrets --as=system:serviceaccount:orders:orders-app -n orders`

trả `no` **không mâu thuẫn** với việc tiến trình đọc được tệp, nếu quyền tệp cho UID của nó đọc. `automountServiceAccountToken: false` chỉ bỏ tự động gắn credential API của SA, không tự bỏ Secret volume khai báo riêng. Ngược lại, có RBAC `get` cũng không tự đặt Secret thành tệp trong container. Người có quyền tạo Pod tham chiếu Secret có thể làm lộ giá trị qua Pod dù họ không được `get` Secret trực tiếp; vì thế phải xem cả quyền tạo/sửa workload, không chỉ quyền đọc Secret của SA. Đây là tình huống giả định, chưa chạy trên cluster.

Để phân biệt hai đường mà không in bí mật ra log, kiểm Pod spec và volume mount, danh tính tiến trình cùng quyền đọc đường tệp bằng phép thử chỉ trả status, rồi kiểm quyền API của đúng SA bằng `can-i` và audit request nếu được bật. Nếu ứng dụng không cần Secret, bỏ volume và triển khai lại Pod; thu hồi credential đã lộ nếu từng có process không tin cậy đọc nó. Nếu ứng dụng cần Secret, giới hạn mount vào container cần dùng, giới hạn ai được tạo/sửa Pod trong namespace, và kiểm lại request nghiệp vụ sau thay đổi. Không thể vừa để chính tiến trình ấy dùng Secret dạng tệp vừa khẳng định nó không đọc được byte trong tệp.

Security context còn kiểm capabilities, privilege escalation, root filesystem ghi được và seccomp profile. Giảm capabilities và dùng seccomp RuntimeDefault có thể giảm syscall/đặc quyền có sẵn, nhưng profile cụ thể phụ thuộc runtime/node; một lệnh bị chặn cần đọc lỗi từ tiến trình, event và cấu hình node, không đoán chỉ từ YAML. Mount volume chứa credential ở chế độ read-only hạn chế ghi vào mount ấy, không bảo vệ secret khỏi một process đã có quyền đọc. Nếu ứng dụng cần thư mục ghi tạm, cấp một mount đúng đường và quyền thay vì bật cả root filesystem ghi được.

Giả định attacker khai thác endpoint `/orders/41` để chạy mã trong process. Với SA chỉ đọc ConfigMap cần thiết, không mount token nếu không dùng API, user không root, capability tối thiểu và filesystem hạn chế, các đường leo quyền giảm nhưng chưa biến lỗi ứng dụng thành vô hại. Process vẫn có thể đọc dữ liệu ứng dụng được cấp và gửi qua kết nối mạng cho phép. Threat model cần nối identity, file/volume, network egress và secret scope; mỗi control ngăn một cạnh cụ thể.

Ca âm của fixture: workload không thuộc nhóm operator không thể `list secrets`; ca dương: app vẫn đọc config và phục vụ request. Nếu thiếu cluster, chỉ kiểm schema/YAML và ghi SKIP cho hành vi RBAC/runtime. Sau thay đổi security context, kiểm startup, readiness và request, vì chặn một syscall mà app cần có thể gây gián đoạn. Không xem Pod Running là bằng chứng chính sách đã thực thi đúng mọi ranh giới.

**Bài tập 55.2 — Secret qua API và volume là hai đường quyền**

Pod orders-api chạy non-root, SA orders-app không được API get/list Secret và automountServiceAccountToken=false, nhưng process đọc được tệp từ Secret volume db-credential. Có mâu thuẫn không? Phân biệt hai đường cấp quyền, nêu phép kiểm không làm lộ giá trị và cách giới hạn truy cập khi ứng dụng vẫn cần Secret.

## Bài 55.3 — Mạng và đám mây

NetworkPolicy chọn Pod theo label và mô tả lưu lượng ingress/egress được phép, nhưng hiệu lực phụ thuộc CNI triển khai hỗ trợ policy. Một namespace "private" hoặc một policy deny-all trên giấy không đủ nếu plugin không thực thi, selector không khớp hoặc traffic đi qua đường khác. Với `orders-api`, thử riêng client → Service → Pod, Pod → DNS, Pod → database và Pod → cloud endpoint; DNS egress bị chặn có thể hiện ra như lỗi tên, còn thiếu route/SG có thể hiện ra timeout sau khi DNS đã đúng. Đọc kết quả từng tầng thay vì gán mọi timeout cho policy.

Workload identity nối SA/Pod với credential cloud theo cơ chế nền tảng; quyền Kubernetes API và quyền AWS vẫn là hai chính sách khác. Gắn role hẹp cho `orders-api`, kiểm token/session thực và policy tới S3 object cụ thể. Security group của ENI hoặc node lọc ở ranh giới mạng cloud; NetworkPolicy lọc ở ranh giới Pod mà CNI hỗ trợ. Cho phép TCP/443 ở một lớp không cấp quyền `s3:GetObject` ở lớp IAM. Ngược lại IAM Allow không tạo route qua NAT/endpoint hay mở firewall. Bảng kiểm phải ghi từng ranh giới, owner và phép quan sát tương ứng.

Tình huống giả định: Pod Ready, DNS S3 giải được, TCP kết nối được qua endpoint, nhưng request trả AccessDenied. Đường mạng cơ bản đã tiến xa hơn, nên điều tra role session, explicit Deny, policy bucket và key object; không gỡ NetworkPolicy hoặc mở SG 0.0.0.0/0 để "chữa" AccessDenied. Nếu TCP timeout trước khi có response, ưu tiên kiểm route/endpoint, SG, NACL và egress policy, đồng thời nhớ một capture chỉ cho thấy điểm đo được. Không có CloudTrail event không tự chứng minh request chưa tới AWS vì phạm vi logging có thể thiếu.

Kiểm policy trong môi trường cho phép bằng ca dương tới đúng dependency, ca âm tới host/port ngoài danh sách và request nghiệp vụ. Ghi CNI/version, namespace labels, SA, role, route và kết quả; nếu không có cluster hoặc cloud account, ghi SKIP thay vì dựng output giả. Sau thay đổi, xem audit, flow log trong phạm vi đã bật và telemetry ứng dụng, rồi quyết định có giữ policy hay rollback. Mục tiêu là chứng minh đường được phép vẫn hoạt động và đường bị cấm thực sự bị chặn.

### Bài tập 55.3 — AccessDenied không sửa bằng mở Security Group

Pod Ready, DNS và TCP tới endpoint đã qua nhưng S3 trả AccessDenied. Mở SG 0.0.0.0/0 có chữa đúng nguyên nhân không?

# PHẦN XV — Thiết kế và vận hành hệ thống thực tế

## Làm sao ghép các cơ chế thành hệ thống đáng tin, đo được và có chi phí hợp lý?

Luồng request, dữ liệu và control plane phải cùng được vẽ với miền lỗi, ngân sách, chủ sở hữu và kế hoạch phục hồi.

XV

**LỘ TRÌNH PHẦN**


- Chương 56 — Thiết kế một dịch vụ vận hành thực tế
- Chương 57 — Mở rộng và tối ưu chi phí có ràng buộc
- Chương 58 — Bài tổng hợp: thay đổi, sự cố và phục hồi

# Chương 56 — Thiết kế một dịch vụ vận hành thực tế

Bảo vệ đường request, dữ liệu và control plane trong một kiến trúc.

## Bài 56.1 — Yêu cầu

Một thiết kế production bắt đầu từ hợp đồng người dùng và dữ liệu. Với dịch vụ học tập `orders-api`, giả định 120 request/giây ở giờ thường, đỉnh 360 request/giây trong mười phút, mỗi request ghi tạo một ID đơn và có đường đọc lại. Mục tiêu minh họa là ít nhất 99,9% request hợp lệ thành công trong cửa sổ 30 ngày, p95 dưới 400 ms khi tải trong phạm vi đã công bố, và mất dữ liệu tối đa bằng không cho đơn đã xác nhận commit. Những số này là giả định của bài toán, không phải SLO hay benchmark của hệ thật. Phải định nghĩa request hợp lệ, timeout, loại lỗi nào tính vào tử số, điểm đo client/edge và đơn nào được coi là commit trước khi tính ngân sách lỗi.

Viết ràng buộc như các đại lượng có đơn vị: lưu lượng thường/đỉnh, kích thước bản ghi, tăng trưởng mỗi ngày, retention, RPO/RTO, thời gian triển khai cho phép và ngân sách tiền theo Region/tháng. Nếu 360 request/giây tồn tại 600 giây và mỗi request ghi 2 KiB, lượng payload thô là 360 × 600 × 2 = 432.000 KiB, chưa kể index, replication và backup. Đây là phép tính tải giả định để lộ scale dữ liệu, không phải dung lượng provision cần thiết. Từ đó mới chọn cơ chế lưu trữ và capacity; không suy ngược từ một sơ đồ cloud có sẵn.

Ràng buộc compliance cũng phải được chuyển thành control có thể kiểm: dữ liệu nào là nhạy cảm, nơi được phép lưu, ai đọc, retention và đường xóa. Không tự tuyên bố đạt một chuẩn pháp lý chỉ vì dùng dịch vụ có chứng nhận. Nếu yêu cầu giữ đơn trong 90 ngày và audit trong 365 ngày, hai loại dữ liệu có vòng đời khác nhau; log không được sao chép payload bí mật để "dễ debug". Thêm owner cho mỗi ràng buộc: product quyết định SLO/chấp nhận rủi ro, data owner quyết định retention, security duyệt quyền, vận hành chịu runbook và thử phục hồi.

Bước kiểm đầu vào là viết một trang hợp đồng có giả định, nguồn dữ liệu đo nếu có, phạm vi chưa biết và stop rule. Nếu chưa đo phân phối tải hoặc chưa thử restore, ghi UNKNOWN và thiết kế thí nghiệm tạm; không điền giá trị "chuẩn ngành". Một yêu cầu thiếu định nghĩa không thể được bảo vệ bằng thêm replica hay mua máy mạnh hơn.

### Bài tập 56.1 — Định nghĩa SLO có thể kiểm chứng

Vì sao 99,9% request thành công chưa là SLO kiểm được nếu chưa định nghĩa request hợp lệ và điểm đo?

## Bài 56.2 — Kiến trúc

Vẽ ba đường riêng cho `orders-api`: đường request client → DNS/TLS edge → load balancer → API → database; đường dữ liệu ghi/backup/restore; và đường điều khiển commit → CI → registry → IaC/cluster API → controller → runtime. Các đường dùng chung vài thành phần nhưng có failure domain và identity khác. Một request thành công sau deploy không chứng minh đường backup tốt; một plan/apply xanh không chứng minh Pod Ready; một Pod Ready không chứng minh dữ liệu ghi đã durable. Ghi trust boundary, owner và bằng chứng tại từng chuyển tiếp.

Một phương án giả định dùng edge/ALB ở hai AZ, hai replica API phân tán, database có cơ chế replication và backup đã thử, object storage cho artifact, gateway/private endpoint cho dependency, và service identity với quyền hẹp. “Hai AZ” phải được kiểm cả đường dữ liệu: nếu database writer chỉ ở AZ A và failover cần thao tác ngoài giờ, thời gian khôi phục là tổng phát hiện, quyết định, chuyển writer, cập nhật routing và xác nhận dữ liệu. EBS của một instance không tự attach xuyên AZ; volume cục bộ không thể là nơi duy nhất giữ đơn. Chọn một topology đơn giản hơn nếu tải và RTO cho phép, nhưng viết rõ rủi ro còn lại.

Đường điều khiển cũng có thể hỏng khi data plane còn chạy: API server tạm mất làm rollout đứng, registry lỗi làm Pod mới không pull image, CI sai digest đưa artifact khác vào cluster. Người dùng có thể vẫn được phục vụ bởi Pod cũ; không vội restart mọi Pod để “khắc phục” control plane. Ngược lại, control plane khỏe không chứng minh request qua edge và dữ liệu ổn. Mỗi mũi tên cần một phép quan sát cụ thể, ví dụ digest imageID ở Pod, EndpointSlice có target Ready, request ID gắn revision, và đọc lại đơn sau khi ghi.

Sơ đồ phải có legend cho đường request, dữ liệu và điều khiển, tên môi trường/Region, ranh giới AZ, storage và identity. Không thêm biểu tượng như firewall, cache hoặc queue chỉ để lấp hình; mỗi thành phần phải có chức năng, state, owner và chế độ lỗi. Khi chưa có cluster thật, đây là kiến trúc tham chiếu để kiểm bằng fixture cục bộ, không phải chứng nhận production.

Hãy lần theo một đơn hàng giả định thay vì chỉ vẽ hộp. Client gửi POST có request ID `r-42`; edge chấm dứt TLS và chuyển qua load balancer; replica API ở AZ B xác thực, ghi giao dịch vào writer database rồi trả mã đơn `o-42`; client GET lại từ đường công khai. Ở mỗi bước ghi rõ điểm nào giữ identity của request, điểm nào có thể retry, ai sở hữu timeout và kết quả nào được coi là commit. Nếu edge retry POST sau timeout mà API không có khóa idempotency, có thể tạo hai đơn dù request cuối cùng trả lỗi cho client. Đường đọc lại kiểm hậu điều kiện nghiệp vụ; một probe `/health` không bao quát lỗi ghi hoặc đọc lệch replica.

Với giả định mất AZ A, hai replica phân tán mỗi AZ một cái chỉ bảo đảm B còn một tiến trình, chưa bảo đảm B đủ capacity hoặc database còn writer. Viết phép kiểm tách: target B có Ready và đang nhận request; số request/giây trên B trong giới hạn đã đo; đường database ở B đọc được dữ liệu đã commit; thao tác failover, nếu có, kết thúc trước RTO. Nếu phải chờ phục hồi từ backup, ghi rõ RPO theo thời điểm bản sao và RTO theo thời điểm request thành công trở lại. Replication và backup không thay nhau: replication có thể truyền cả xóa nhầm, còn backup không tự phục vụ request khi writer hỏng.

Một bảng trách nhiệm ngắn đi kèm sơ đồ: edge owner kiểm TLS/routing, platform owner kiểm target và Pod image digest, data owner kiểm commit/restore, release owner kiểm manifest và rollback, incident commander quyết định dừng traffic. Không cần thêm sản phẩm cho từng hàng; cần điểm đo và quyền thao tác. Trên fixture cục bộ, có thể mô phỏng request ID, timeout và đọc lại đơn để bác bỏ sơ đồ sai, nhưng không suy từ đó ra tính chịu lỗi AZ của AWS thật.

**Bài tập 56.2 — Rollout xanh nhưng đơn hàng không đọc lại được**

Control plane rollout xanh, Pod Ready, nhưng POST /orders không đọc lại được. Ba trạng thái cho biết gì và thiếu phép kiểm nào?

### Bài 56.3 — Đánh giá

Đánh giá kiến trúc bằng các sự cố đối nghịch với giả định đã ghi. Mất AZ A: target ở B còn capacity và dữ liệu có thể đọc không? Chứng chỉ TLS hết hạn: edge trả lỗi trước khi request tới API, alert nào phát hiện trước người dùng? Image H84 triển khai lỗi: có bản H83 đã kiểm thử, nhưng schema dữ liệu sau H84 có còn đọc được bởi H83 không? Ba đường này chạm network, certificate, artifact và data khác nhau; một "rollback" chung không giải hết.

Lập bảng quyết định gồm failure, dấu hiệu, điểm đo, owner, stop rule, hành động đầu tiên và hậu kiểm. Với AZ A, dấu hiệu target health giảm chưa đủ; kiểm request qua B và backlog. Với TLS, kiểm chain, tên, thời hạn, SNI trên đường client thật, rồi quyết định đổi certificate hoặc chuyển traffic. Với deploy, so commit/digest, revision, error budget và request dữ liệu; nếu dữ liệu đã migrate không tương thích, revert image có thể làm lỗi nặng hơn. Mỗi hành động phải có điều kiện tiền đề và đường phục hồi nếu chính hành động thất bại.

Tình huống giả định: H84 có lỗi ghi khiến 2% request tạo đơn thất bại trong 5 phút, nhưng `/health` vẫn xanh. Với 360 request/giây, đó là khoảng 360 × 300 × 0,02 = 2.160 request lỗi nếu tải đều; con số chỉ minh họa cách tính, không là đo. Dừng tăng canary, giữ evidence và chuyển quyết định theo SLO đã định. Nếu H84 đã ghi schema mới, kiểm khả năng H83 đọc đơn mới trước rollback; nếu không, dừng ghi và dùng kế hoạch dữ liệu riêng. Sau đó xác minh request mới và bản ghi trước/sau, không đóng sự cố chỉ vì deployment đã "rolled back".

Runbook tốt chỉ rõ một người có quyền ra quyết định, người thao tác, người liên lạc và nguồn sự thật của thời gian/ID. Diễn tập trên fixture với stop rule và blast radius hạn chế; ghi thời gian phát hiện, quyết định, khôi phục và mất dữ liệu thực. Nếu một nhánh chưa thử, đánh dấu UNKNOWN và ưu tiên thí nghiệm, không gán điểm PASS cho sơ đồ. Thiết kế production là tập giả thuyết có thể bác bỏ bằng bằng chứng.

Đặt các mốc giả định để buộc tính RTO/RPO đúng: AZ A ngừng phục vụ lúc 10:00, cảnh báo đủ tin cậy lúc 10:02, quyết định chuyển writer lúc 10:05, request POST và GET qua AZ B thành công lúc 10:09. Nếu hợp đồng đo RTO từ lúc người dùng mất khả năng hoàn thành đơn tới lúc đường nghiệp vụ phục hồi, RTO quan sát trong ca này là 9 phút, không phải 4 phút từ lúc bấm chuyển writer. Nếu chứng minh được bản sao ở B nhất quán và đầy đủ tới 09:58, recovery point của B cách sự cố 10:00 hai phút; đó là khoảng trễ điểm khôi phục, chưa phải số đơn thực sự mất. Không có bằng chứng về các ghi đã xác nhận trong 09:58–10:00 thì không được gọi mục tiêu RPO đã đạt chỉ vì replica báo healthy. Yêu cầu ở đầu phần là không mất đơn đã xác nhận: chỉ được chấp nhận phương án này khi đối soát ID đơn chứng minh không có đơn xác nhận bị thiếu, hoặc có nguồn độc lập để replay chúng rồi kiểm đọc lại. Nếu còn thiếu đơn mà không phục hồi được, phương án FAIL yêu cầu dữ liệu dù request qua B đã phục vụ lại trong chín phút.

Dấu hiệu dừng trước hành động phải phụ thuộc loại lỗi. Với chứng chỉ TLS hết hạn, không đổi image API khi request chưa qua được edge; thử SNI/chain từ phía client và kiểm chứng certificate mới trên cùng hostname. Với H84 lỗi 2% ghi đơn, dừng tăng canary và giữ digest/status trước khi rollback; nếu H84 đã viết dữ liệu mới H83 không đọc được, bảo vệ dữ liệu và chuyển sang quy trình phục hồi tương thích. Với mất AZ, không tăng traffic sang B khi chưa đo capacity và writer; có thể phải chấp nhận chế độ giảm chức năng theo quyết định đã chuẩn bị. Mỗi nhánh cần một phép thử âm

và một phép hậu kiểm nghiệp vụ.

Sau diễn tập, ghi những gì còn UNKNOWN: thời gian phát hiện từ alert thật, độ trễ cấp quyền cho người trực, thời gian cập nhật route, độ trễ replica dữ liệu và khả năng restore backup. Đừng cộng các ước lượng tốt nhất rồi gọi là RTO cam kết; các chặng có thể chồng lấn hoặc chặn nhau. Chỉ một phép chạy trên môi trường đại diện, với timestamp đồng bộ và log request/dữ liệu, mới cho phân bố thời gian đủ để so với mục tiêu vận hành.

**Bài tập 56.3 — Canary lỗi khi health check vẫn xanh**

H84 gây 2% lỗi trong 5 phút ở 360 rps nhưng /health xanh. Tính số lỗi theo giả định và nêu stop rule. Trong nhánh mất AZ lúc 10:00, B chỉ chứng minh dữ liệu nhất quán đến 09:58; request phục hồi 10:09. Có thể tuyên bố đạt yêu cầu không mất đơn đã xác nhận chưa? Nêu phép đối soát.

# Chương 57 — Mở rộng và tối ưu chi phí có ràng buộc

Chọn năng lực đủ SLO trong giới hạn tiền và vận hành.

## Bài 57.1 — Mô hình tải

Capacity không phải chỉ là số replica ở tải trung bình. Cho bài toán giả định 360 request/giây ở đỉnh, một replica xử lý ổn định 120 request/giây ở ngưỡng p95 đã định trong thí nghiệm sẽ phải thực hiện, và chính sách chỉ dùng 75% mức ấy để giữ headroom. Mức mục tiêu là 90 request/giây/replica; cần ít nhất `ceil(360/90)=4` replica khi mọi AZ khỏe. Nếu yêu cầu chịu mất trọn một AZ mà vẫn giữ cùng headroom, AZ còn lại cần 4 replica; với hai AZ cân bằng, tổng đặt trước 8 replica. Đây là tính toán từ giả định, không phải benchmark; nếu thí nghiệm cho năng lực khác, phải tính lại.

Hàng đợi làm tail latency tăng mạnh trước khi CPU luôn đạt 100%. Nếu thời gian phục vụ hoặc dependency DB kéo dài, số request đồng thời tăng ngay cả khi arrival rate chưa đổi. HPA nhìn metric có cửa sổ lấy mẫu và cần thời gian tạo Pod, pull image, startup, readiness; burst mười giây có thể kết thúc trước khi replica mới phục vụ. Với workload burst, cân nhắc capacity dự trữ, queue có giới hạn và backpressure. Queue không tự làm việc nhanh hơn; nó dời thời gian chờ và có thể tăng độ trễ hoặc mất request nếu đầy. Đặt giới hạn queue, timeout và retry budget để tránh khuếch đại tải khi dependency chậm.

Đo capacity bằng request hợp lệ, latency histogram, error rate, concurrency, CPU throttling, memory/OOM, DB connection pool và queue depth cùng thời gian. Một điểm đo CPU thấp trong khi p95 cao có thể là DB/lock/network wait; tăng replica API khi DB đã bão hòa có thể làm tình hình xấu hơn. Thí nghiệm cần baseline, bước tăng tải, thời gian đủ dài, dữ liệu không làm bẩn production, và stop rule khi error budget hoặc tài nguyên vượt ngưỡng. Không lấy một bài benchmark trên laptop làm cam kết cho cluster/cloud khác.

Sau autoscale, hậu kiểm là request và dữ liệu qua đường thật, không phải `desiredReplicas` đã tăng. Nếu scale-up chưa Ready trước burst, ghi autoscale lag và điều chỉnh dự trữ hoặc mô hình tải. Nếu một AZ mất, kiểm số target thực ở AZ còn lại thay vì lấy tổng replica trong spec. Mức 8 replica ở trên là ví dụ cho ràng buộc chịu lỗi rất chặt; quyết định thật cần so SLO và chi phí của dự trữ.

Tính tiếp fixture hai AZ để thấy **năng lực đặt trước và năng lực hữu dụng** khác nhau. Bình thường bốn replica ở mỗi AZ nhận xấp xỉ 180/s, tức 45/s mỗi replica nếu cân đều. Khi AZ A mất, B phải nhận 360/s: bốn replica còn Ready cần giữ 90/s mỗi replica, đúng trần mục tiêu đã chọn. Nếu B chỉ có ba replica Ready do một Pod đang rollout, mức mục tiêu còn 270/s; thiếu 90/s ngay cả khi HPA sau đó yêu cầu thêm Pod. Bài tính `ceil(360/90)=4` là điều kiện cần dưới giả định phân tải đều và năng lực mỗi replica độc lập; nó chưa kiểm quota node, pool DB, DNS, LB health hay thời gian phát hiện lỗi AZ. Thử failover phải đo request từ client trước, trong và sau chuyển vùng, đồng thời kiểm tỉ lệ lỗi và dữ liệu đúng.

Một kiểm tra burst giả định: lưu lượng tăng từ 180/s lên 360/s trong 20 giây, capacity hiện có xử lý 270/s trong 12 giây trước khi Pod mới Ready. Nếu không có reject và mọi request được giữ, backlog tăng `(360−270)×12 = 1.080 request`; queue 500 phần tử buộc phải reject, timeout hoặc nghẽn trước khi scale hoàn tất. Sau giây 12, nếu capacity chỉ vừa bằng 360/s, backlog cũ **không rút**; phải

có capacity lớn hơn offered rate trong giai đoạn hồi phục hoặc giảm arrival. Ví dụ là phép tính có điều kiện, không phải kết quả HPA thực. Nó giải thích vì sao chọn số replica theo peak sau autoscale mà bỏ qua thời gian khởi động vẫn có thể vi phạm p95.

**Bài tập 57.1 — Số replica để chịu đỉnh và mất AZ**

Giả định đỉnh 360 rps, mỗi replica 120 rps đã đo và chỉ dùng 75% để giữ headroom. Cần mấy replica khỏe, và nếu muốn mất một AZ vẫn chịu đỉnh với hai AZ cân bằng?

**Bài 57.2 — Mô hình tiền**

Mô hình tiền phải có đơn vị và ranh giới: chi phí tháng giả định `C = C_compute + C_storage + C_transfer + C_observability + C_backup + C_operations`. Compute gồm thời gian instance/Pod node và phần dự trữ để chịu lỗi; storage gồm byte-tháng, request hoặc I/O theo dịch vụ; transfer gồm đường giữa AZ/Region, Internet và dịch vụ; observability gồm ingest, index, retention và truy vấn; backup gồm bản sao, chuyển và thử restore. Mỗi hạng mục có Region, ngày giá, mức sử dụng và giả định tính tiền. Không lấy giá của một Region hoặc một tháng áp cho nơi khác.

Ví dụ bằng biến để tránh bịa giá: phương án A luôn chạy 8 replica với giá hiệu dụng `p` cho mỗi replica-giờ, 730 giờ/tháng thì compute là `8 × 730 × p`. Phương án B chạy 4 replica nền và thêm 4 replica trong 2 giờ/ngày thì compute xấp xỉ `(4 × 730 + 4 × 2 × 30) × p = 3.160p`, so với `5.840p` của A; chưa tính node bin-packing, autoscale lag, egress, storage và phí tối thiểu. Chênh `2.680p` là đại số trên giả định, không là hóa đơn AWS. B có thể không đáp ứng mất AZ tức thời; không so tiền mà bỏ ràng buộc SLO/DR.

Telemetry có chi phí kép: ingest/storage và chi phí vận hành khi cardinality nổ. Một label `request_id` trên metric tạo chuỗi gần như theo từng request, làm tăng bộ nhớ và lưu trữ; giữ ID ở log/trace có kiểm soát truy cập. Giảm sampling hoặc retention tiết kiệm tiền nhưng có thể làm thiếu bằng chứng khi sự cố hiếm xảy ra. Ghi loại tín hiệu nào được giữ đủ để tính SLI và điều tra, loại nào được sampling, và tác động UNKNOWN khi pipeline mất dữ liệu. Backup cũng không nên bị cắt chỉ để đạt ngân sách tháng nếu RPO/RTO còn là yêu cầu.

Đầu ra bảng giá chỉ đáng tin khi có ngày, Region, loại dịch vụ, cấu hình, đơn vị và link tới bảng giá chính thức ở hồ sơ ngoài PDF; sách này không in giá thời gian thực. Chạy phép nhạy cảm: tải tăng 2 lần, lưu trữ tăng 3 lần, egress tăng 50%, hoặc giữ log lâu hơn. Nếu không biết một biến, ghi khoảng giá trị và tác động lên quyết định, không điền số tròn như kết quả đo.

Phép so A/B cần tính **đơn vị năng lực phục vụ**, không chỉ replica-giờ. Trong fixture, A tốn `5.840p`, B `3.160p` cho compute, tức B giảm `2.680p` trước các phí khác. Nhưng nếu hợp đồng chịu mất AZ tức thời đòi bốn replica còn Ready ở AZ sống, B với bốn replica *tổng cộng* có thể chỉ còn hai, công suất mục tiêu 180/s, thiếu 180/s ở peak. Để so công bằng, định cấu hình B thỏa cùng ràng buộc hoặc ghi rõ B **không đủ điều kiện**, rồi mới tính chi phí. Một phương án rẻ nhưng không đạt ràng buộc cứng không nằm trên cùng biên lựa chọn. Nếu có dữ liệu chứng minh autoscale/failover trong thời gian chấp nhận, ghi cửa sổ mất dịch vụ và đưa nó vào SLO thay vì ngầm coi là không có.

Ví dụ khác về tổng chi phí: giảm tần suất scrape làm phí ingest thấp hơn nhưng biến một đợt lỗi 40 giây thành điểm khó nhìn thấy nếu cửa sổ lấy mẫu quá thưa hoặc pipeline mất mẫu. Cắt retention từ

30 xuống 7 ngày có thể xóa tín hiệu cần cho sự cố phát hiện sau 10 ngày. Trước khi tối ưu, phân loại tín hiệu theo quyết định nó phục vụ: đo SLI, phát hiện, chẩn đoán, pháp lý và audit. Với mỗi loại, ghi độ phân giải và thời hạn tối thiểu; thử truy vấn một sự cố giả định ở tuổi 10 ngày. Chỉ phần dữ liệu vượt yêu cầu đã kiểm mới là ứng viên cắt giảm. Giá trị tiết kiệm phải tính theo byte ingest, byte lưu và query thực, không suy từ một công tắc cấu hình.

**Bài tập 57.2 — Tính chi phí hai phương án replica**

Với p là giá mỗi replica-giờ, so compute 8 replica cả 730 giờ và 4 nền cộng 4 thêm 2 giờ/ngày trong 30 ngày.

**Bài 57.3 — Quyết định**

Quyết định tối ưu dùng "chi phí cho một kết quả hợp lệ" và rủi ro vi phạm SLO, không chỉ tổng hóa đơn. Với hai phương án A/B ở bài trước, số request thành công trong cửa sổ phải được tính cùng định nghĩa SLI. Nếu B rẻ hơn nhưng mất 2.160 request trong một burst do autoscale lag, chi phí tiết kiệm có thể không chấp nhận được theo error budget hoặc tác động nghiệp vụ. Không biến một request lỗi thành một giá tiền giả nếu product chưa định nghĩa; ghi tác động và người chấp nhận rủi ro.

Điểm hòa vốn cần nêu biến. Giả định lựa chọn thêm một replica dự trữ tốn `730p` mỗi tháng và giảm xác suất lỗi khi burst từ `q0` xuống `q1` theo thí nghiệm có giới hạn. Nếu chi phí một sự cố được chủ sản phẩm ước lượng là `L`, lợi ích kỳ vọng đơn giản là `(q0-q1) × L`; mua dự trữ chỉ hợp lý theo mô hình này khi lợi ích vượt `730p`, cùng ràng buộc SLO/DR khác. Đây không phải phép tính giá trị chính xác: q và L khó đo, phân phối sự cố có đuôi, nhiều lỗi tương quan và quyết định pháp lý có thể là hard constraint. Dùng phép tính để lộ giả định, không thay phê duyệt.

Thử một thay đổi chi phí theo từng bước: chọn candidate, ghi baseline tải/SLO/chi phí, dự đoán lợi ích, giới hạn rollout và stop rule, đo bằng cùng bộ chỉ số rồi quyết định giữ hoặc hoàn nguyên. Ví dụ giảm retention trace từ 30 xuống 7 ngày cần kiểm khả năng điều tra sự cố kéo dài và nghĩa vụ lưu trữ trước; không chỉ đọc bảng tiết kiệm. Giảm replica cần thử mất một AZ và thời gian scale-up, không chỉ thử tải trung bình. Sau thay đổi, kiểm cả lượng dữ liệu ghi/đọc và đường restore.

Số liệu giá phải được lấy lại khi ra quyết định thật: Region, ngày, loại instance/storage, transfer direction và discount/commitment. Tài liệu dạy dùng `p`, `q`, `L` để không giả vờ biết giá hoặc xác suất. Nếu không có đo thực hoặc quyền truy cập billing, ghi UNKNOWN; quyết định tốt hơn có thể là chờ một thí nghiệm nhỏ với stop rule thay vì tối ưu theo một spreadsheet thiếu dữ kiện.

Một **quyết định có điều kiện** dễ kiểm toán hơn khẩu hiệu "scale thông minh". Với phương án giảm replica, viết trước: được giảm khi hai tuần traffic mix đại diện có p95 và error budget còn dư, thử mất một AZ giữ hậu điều kiện, thời gian Pod Ready đo được nhỏ hơn cửa sổ burst chấp nhận, và backend DB chưa bị dồn connection. Rollout trước ở một phần traffic, dừng nếu rejected/timeout hoặc queue tăng hai cửa sổ liên tiếp; hoàn nguyên số replica và xác nhận queue rút, request mới qua, dữ liệu đúng. Nếu không có môi trường để thử mất AZ, đánh dấu rủi ro còn mở và giữ cấu hình cũ cho yêu cầu chịu lỗi ấy. Đây là gate quyết định, không phải khẳng định phép thử đã chạy.

Khi xem “chi phí trên request thành công”, kiểm cả **mẫu số và phần việc bị dời**. Giả sử A hoàn tất 1 triệu request hợp lệ trong ngày, B cũng báo 1 triệu response 2xx nhưng thêm 20.000 request bị từ chối ở edge và 5.000 job phải chạy lại hôm sau. Dùng chỉ 2xx của backend làm mẫu số khiến B trông rẻ dù người dùng nhận ít kết quả hơn và chi phí job chuyển ngày. Định nghĩa kết quả hợp lệ từ điểm client và trạng thái dữ liệu cuối, gộp retry theo operation ID, rồi mới so chi phí cùng thời kỳ. Các con số chỉ là fixture để minh họa ranh giới kế toán; chúng không chứng minh A hay B ngoài thực tế tốt hơn.

**Bài tập 57.3 — Giá rẻ hơn nhưng mất SLO**

Phương án B rẻ hơn 2.680p nhưng có thể mất SLO khi một AZ lỗi. Dữ kiện gì cần trước khi chọn?

# Chương 58 — Bài tổng hợp: thay đổi, sự cố và phục hồi

Chứng minh toàn hành trình từ commit đến phục hồi bằng artifact và phép đo.

## Bài 58.1 — Chuẩn bị

Capstone dùng một dịch vụ API cục bộ có thao tác `POST /orders` tạo bản ghi và `GET /orders/{id}` đọc lại từ kho dữ liệu bền vững trong fixture. Hai phiên bản S1/S2 phải khác nhau về commit và artifact; một thay đổi đã biết có rủi ro vận hành để thử stop/rollback. Không cần AWS hay tài nguyên có phí. Nếu có container/Kind thì dùng chúng trong môi trường tách biệt; nếu không, tiến trình và filesystem tạm vẫn có thể chứng minh chuỗi source → artifact → request → dữ liệu. Không chạm lab/state đang dùng. Mỗi lần chạy ghi tool version, host/namespace, thời điểm và đường dẫn fixture; output của sách là hợp đồng cần thực hiện, không phải log đã chạy.

Tạo `change_id` cho đợt thay đổi, ví dụ `chg-demo-84`; ghi commit object ID đã resolve, dependency lock và digest artifact tính từ byte. `deployment_revision` chỉ sinh khi triển khai, ví dụ `rev-s1` và `rev-s2`; không dùng chung ID với `change_id`. Request ID chỉ sinh khi phát request thử; mỗi request có ID riêng và được nối với revision thực sự phục vụ qua log/trace hoặc header tương quan có thời điểm. Nếu không chứng minh được request đi qua revision nào, ghi UNKNOWN; không suy từ tag hoặc từ việc Deployment desired đã đổi.

Trước deploy, ghi baseline: S1 tạo và đọc được bản ghi 41, checksum hoặc nội dung đã chuẩn hóa của bản ghi, snapshot/backup và phép restore nhỏ vào thư mục khác. Một file backup tồn tại chưa đủ; cần đọc được sau restore. Vẽ đường request và trust boundary, ghi principal nào được đọc dữ liệu và artifact. Kiểm config/schema S2 có tương thích với S1 khi rollback hay không. Nếu S2 làm migration một chiều, phải chuẩn bị đường dữ liệu riêng và stop rule trước rollout; rollback binary không tự đảo schema.

Gate chuẩn bị gồm kiểm digest lại, test chức năng/âm trên fixture, review diff cấu hình/IaC, backup có kiểm restore, SLO/alert và phân quyền. Thiếu một mắt xích quan trọng thì dừng và ghi SKIP/FAIL đúng phạm vi; không tự nâng từ "source build được" thành "đủ an toàn để deploy". Đây là bài thực hành có rubric, không là chứng nhận production.

Để tự kiểm đường request–dữ liệu trước khi dùng cluster, chạy `python examples/capstone_orders.py` từ thư mục `giaotrinh` bằng Python 3.12 trở lên. Fixture chỉ mở HTTP trên `127.0.0.1`, tạo SQLite trong thư mục tạm và xóa khi kết thúc. Trước khi chạy, dự đoán bản backup chứa những đơn nào và S1 có đọc được bản ghi do S2 tạo hay không. Kết quả JSON ghi riêng `request_id`, revision phục vụ, status, nội dung đọc lại và hash của mã fixture/bản backup. Hash mã ở đây chỉ định danh **một script mô phỏng cả hai revision**; nó không chứng minh có hai commit, hai artifact độc lập hoặc một chuỗi CI. Trong capstone thật, người học phải bổ sung các mắt xích ấy bằng artifact và bằng chứng của chính lần làm bài.

### Bài tập 58.1 — Thiếu digest và phép thử restore

Có commit và file backup nhưng chưa tính digest artifact hay thử restore. Có được deploy theo rubric không?

### Bài 58.2 — Thực thi

Thực thi bắt đầu bằng một bản ghi quyết định: `change_id`, digest H2 đã kiểm, cấu hình triển khai, người duyệt và stop rule. Triển khai `rev-s2` theo canary có giới hạn; giữ `rev-s1` để so. Khi control plane báo apply/rollout thành công, vẫn phải theo đường dữ liệu: Pod/process nào đang chạy H2, readiness nào đã qua, request qua endpoint thật nào tới S2, và POST tạo bản ghi nào đọc lại được. Nếu chỉ `/health` xanh, chưa đạt hậu điều kiện ghi/đọc. Một request có ID `req-201` đi qua S1 không thể được ghi là bằng chứng cho S2.

Với mỗi request thử, ghi `request_id`, thời điểm, revision thực tế, endpoint, status, latency, record ID và kết quả đọc lại. Metric cần có cửa sổ và population; log/trace cần cùng request ID nhưng không chứa token hoặc payload nhạy cảm. Nếu chạy HTTP/TLS thật, phân biệt DNS, TCP, TLS, HTTP, proxy và upstream; nếu chạy local process, ghi rõ ranh giới tương đương và phần mạng đã SKIP. Quan sát một vài request không đủ tính SLO dài hạn; đó là smoke test với giới hạn mẫu nhỏ.

Cho ví dụ dữ liệu giả định để tập tính: trong canary 5 phút có 1.000 request hợp lệ, 990 thành công và 10 lỗi, SLI thành công của cửa sổ là `990/1000 = 99%`. Nếu SLO minh họa là 99,9%, cửa sổ canary này không đạt; dừng tăng traffic và điều tra. Không coi 990/1000 là log đã thu. Nếu 50 request không có telemetry nhưng server có bằng chứng đã nhận, không tự bỏ chúng khỏi mẫu; cần định nghĩa missing data và đối chiếu nguồn log/edge. Một alert firing hay dashboard trống không thay chứng cứ request.

Khi test S2 qua, ghi revision/digest/endpoint và số bản ghi trước-sau, rồi mới cân nhắc tăng canary. Nếu gateway trả 502, truy lần lượt upstream health, connection, timeout và log; status code không tự chỉ nguyên nhân. Nếu dữ liệu đọc sai dù status 200, đó là FAIL nghiệp vụ, stop ngay và giữ evidence. Không tự động rollback khi schema/data chưa được kiểm tương thích. Kết quả của capstone phải cho thấy cả quyết định dừng lẫn lý do tiếp tục.

Trong fixture cục bộ, hãy lần theo `events`: `s1_write` và `s1_read` tạo rồi đọc đơn 41 với `amount_cents=100`; bản sao được chụp sau bước đó. S2 cố ý ghi đơn 42 được yêu cầu là 200 nhưng lưu thành 201. `s2_health` trả 200, `s2_write` trả 201 và `s2_read` trả 200; cả ba status đều không phát hiện giá trị nghiệp vụ sai. So input với nội dung đọc lại, rồi nối từng request ID với `revision` trả từ chính tiến trình phục vụ. Đây là lỗi được tiêm có chủ đích, không phải số liệu đo production. Server của bài chỉ dùng thư viện HTTP tối giản để kiểm ranh giới; không có TLS, xác thực, proxy, đa node hoặc canary phân phối lưu lượng. Ghi SKIP cho các chặng đó nếu chỉ chạy fixture.

#### Bài tập 58.2 — Canary dưới SLO có được tăng tải?

Canary có 990/1000 request hợp lệ thành công, SLO giả định 99,9%. Có tăng traffic không?

### Bài 58.3 — Bảo vệ

Tiêm lỗi trong fixture với blast radius rõ: S2 ghi một trường mới khiến S1 không đọc được bản ghi mới, hoặc cấu hình của S2 trả 200 nhưng đọc sai một đơn. Trước khi sửa, lưu `change_id`, revision/digest đang phục vụ, request ID lỗi, nội dung bản ghi đã chuẩn hóa, metric và thời điểm. Nếu nguyên nhân chưa chắc, ghi giả thuyết và phép kiểm phân biệt; không xóa dữ liệu thử để làm bài "xanh". Stop canary khi vượt ngưỡng đã định hoặc có sai dữ liệu, rút S2 khỏi lưu lượng theo đường được thử rồi kiểm request mới.

Rollback ứng dụng từ `rev-s2` về `rev-s1` chỉ đổi code đang phục vụ. Dữ liệu do S2 ghi vẫn còn và có thể không tương thích với S1. Trước rollback, thử S1 đọc bản ghi cũ và bản ghi mới trên bản sao; nếu bản ghi mới không đọc được, chọn dừng ghi và đường migration ngược/reconcile đã kiểm hoặc phục hồi backup rồi replay các giao dịch sau backup. Quyết định phục hồi dữ liệu phải xác định mất mát chấp nhận được theo RPO, thời gian theo RTO, và ai duyệt. Một restore từ snapshot không có phép đọc nội dung không đủ PASS.

Ví dụ timeline giả định: backup hợp lệ ở 10:00, dịch vụ bắt đầu trả dữ liệu sai lúc 10:04, phát hiện 10:06, S2 rút khỏi lưu lượng và mọi đường ghi được chặn lúc 10:07, dữ liệu được khôi phục và request đọc/ghi lại thành công lúc 10:13. Nếu chỉ restore backup 10:00 mà không replay, mọi ghi đã xác nhận trong khoảng 10:00–10:07 nhưng không nằm ở nguồn khác đều cần đối soát; cửa sổ ghi có nguy cơ mất dài tối đa 7 phút, còn số bản ghi thực sự mất chưa biết nếu không có ledger. Theo định nghĩa RPO, khoảng từ recovery point 10:00 đến lúc sự cố bắt đầu 10:04 là 4 phút; RPO là mục tiêu tổ chức phải đặt trước, không thể suy ra từ một incident. Tuổi của backup lúc phục vụ lại là 13 phút, nhưng không được cộng thời gian sửa dịch vụ vào khoảng mất dữ liệu rồi gọi đó là RPO. Nếu sai dữ liệu từ 10:04 được tính là dịch vụ không đạt hợp đồng và 10:13 là lúc hậu kiểm thành công, thời gian phục hồi đo được là 9 phút; RTO là mục tiêu tối đa được quy định riêng. Đây là dữ liệu tập tính, không phải phép đo incident thật.

Sau phục hồi, dùng request ID mới để chứng minh revision đang phục vụ, checksum/danh sách bản ghi cũ-mới và quyền/secret vẫn đúng. Kiểm lại alert và telemetry pipeline; nếu chúng đã hỏng trong sự cố, ghi UNKNOWN cho khoảng không quan sát. Postmortem ghi điều kiện gây lỗi, tín hiệu trễ, quyết định và thay đổi test/guardrail, không quy lỗi cá nhân. Ca âm dữ liệu là cốt lõi: rollback binary thành công nhưng đọc bản ghi mới sai phải được đánh FAIL.

Fixture còn hai phép đối chứng: mở cùng database bằng S1 sau khi S2 dừng thì `rollback_old` đọc đơn 41, còn `rollback_new` trả 409 vì S1 không hiểu phiên bản bản ghi 42; đọc bản backup vào database tách biệt cho `restored_old` trả 200 và `restored_new` trả 404. Việc đổi revision của server không xóa hàng 42 trong database hoạt động; việc khôi phục bản sao cũ làm hàng 42 biến mất khỏi bản phục hồi. Hai kết quả này loại bỏ hai kết luận sai thường gặp: "rollback code đã phục hồi dữ liệu" và "restore thành công nên không mất đơn đã xác nhận". Bài chạy trong thư mục tạm và xóa nó sau khi xuất JSON; nếu cần hồ sơ nộp, lưu riêng stdout cùng phiên bản Python, hash script và thời điểm chạy, không gọi đó là phép thử mất điện hay đo RPO/RTO thực tế.

### Bài tập 58.3 — RPO và RTO khi chỉ restore backup

Backup 10:00; dữ liệu bắt đầu sai 10:04; chặn toàn bộ ghi 10:07; request đọc/ghi đúng trở lại 10:13. Chỉ restore backup, không replay: phân biệt khoảng đến recovery point lúc sự cố, cửa sổ ghi có nguy cơ mất, tuổi backup lúc phục vụ lại và thời gian phục hồi. Có suy ra RPO/RTO mục tiêu hoặc số bản ghi mất được không?

### Bài 58.4 — Kiểm toán

Mỗi mục trong ledger ghi file/log và hash hoặc ID bất biến, thời điểm, môi trường, phiên bản công cụ, chủ thể tạo và PASS/FAIL/SKIP/UNKNOWN. Chuỗi cần chứng minh là `change_id → commit → build input → artifact digest → deployment_revision → running instance`; mỗi request nối `request_id → revision đang phục vụ → record → telemetry`. Tên tag hay lời kể không đủ chứng minh một mắt xích.

**Một thay đổi · mỗi hàng có chứng cứ và quy tắc dừng** V43

| Bước | Artifact/ID | Phép kiểm | Trạng thái |
|---|---|---|---|
| Build | commit → digest | tính lại SHA-256 | CHƯA ĐO |
| Deploy | revision → imageID | Pod + request | CHƯA ĐO |
| Quyền | role + secret ID | ca âm/ca dương | CHƯA ĐO |
| Quan sát | request_id | metric/log/trace | CHƯA ĐO |
| Sự cố | injection ID | stop / rollback | CHƯA ĐO |
| Dữ liệu | backup ID | restore + SLI | CHƯA ĐO |

Thiếu identity, restore hoặc hậu trạng thái dữ liệu → DỪNG; SKIP có lý do.

Hình XV.3.1 — Ledger capstone: mỗi hậu điều kiện cần phép kiểm và trạng thái riêng.

Người thẩm định tính lại digest từ byte, nối request với revision và record, đọc backup sau restore, rồi tính SLI từ tử số/mẫu số. Nếu thiếu telemetry, ghi rõ population và cách xử lý request thiếu; dashboard xanh không thay phép tính. Ca âm tìm marker secret giả trong log/state/image/trace phải ghi phạm vi quét; kết quả âm không chứng minh mọi bản sao đều sạch.

Rubric có ca dương digest H2 đúng và request ghi/đọc đúng; ca âm digest sai phải bị chặn, Ready nhưng dữ liệu sai phải FAIL, rollback app mà schema không tương thích phải dừng. Thiếu Docker/Kind/Go/Terraform/OTel backend thì ghi SKIP theo phép và dùng process/filesystem tạm khi tương đương; AWS không là điều kiện đạt. SKIP không biến thành PASS; test chưa chạy không phải bằng chứng không có lỗi.

Quyết định cuối là chấp nhận giới hạn có owner, sửa và thử lại, hoặc dừng. Phải dừng khi chạm dữ liệu/lab thật, plan replace/destroy thiếu backup và duyệt, canary vượt ngưỡng, quyền/secret sai, restore thất bại, hoặc thiếu identity artifact/hậu trạng thái dữ liệu. Báo cáo phân biệt fixture với phép

chạy thật. Capstone hoàn chỉnh là chuỗi bằng chứng có thể kiểm lại và quyết định phù hợp, không phải số công cụ trong sơ đồ.

Một ledger tốt phải cho phép người khác tái dựng quyết định mà không hỏi người viết. Giả sử build từ commit `C` tạo digest `H2`, deployment revision `D7` khai báo `H2`, nhưng node phục vụ request `q17` báo digest `H1`. Ba bản ghi đầu có thật vẫn chưa chứng minh request đi qua artifact đã duyệt. Trước khi ký đạt, kiểm xem `q17` tới node nào, revision nào, có instance cũ trong giai đoạn rolling không, và digest runtime lấy từ nguồn quan sát nào. Nếu thiếu liên kết request–instance, ghi UNKNOWN; không tự lấp bằng digest trong file cấu hình. Nếu giai đoạn rolling cho phép cả `H1` và `H2`, ledger phải ghi rõ khoảng thời gian và tỷ lệ traffic được phép, chứ không ép mọi request vào một phiên bản duy nhất.

Tương tự, một SLI giả định ghi 19.900 request thành công trên 20.000 request trong cửa sổ: tỷ lệ là 99,5%. Nhưng nếu 500 request không có nhãn kết quả và bị loại khỏi mẫu số, phép chia chỉ mô tả tập đã quan sát; không thể gọi là mức tin cậy của toàn bộ traffic. Ghi định nghĩa population, nguồn đếm, xử lý missing data và ngưỡng quyết định trước khi nhìn số. Nếu đối tượng kiểm toán là "đọc được dữ liệu sau rollback", một request `/health` thành công không thay cho phép đọc record đã ghi bởi bản mới và phép phục hồi từ backup thử. Các hậu điều kiện này phải gắn với thời điểm và revision, vì hệ thống có thể đổi trạng thái sau khi test.

Khi gặp bằng chứng trái nhau, ledger không cần làm đẹp thành một trạng thái duy nhất. Ghi PASS cho phép kiểm digest ở build, FAIL cho phép đối chiếu digest runtime, SKIP cho restore do thiếu môi trường tách biệt, UNKNOWN cho request không truy được instance. Quyết định phát hành phụ thuộc cổng bắt buộc: nếu identity runtime hoặc hậu trạng thái dữ liệu chưa đạt, dừng dù những phép khác xanh. Người thẩm định có thể tái tính hash, mẫu số SLI và mở các record được nêu; nếu không thể, bằng chứng chưa đủ cho capstone. Đó là một bài tập về chất lượng suy luận vận hành, không phải bài thi thuộc tên công cụ.

**Bài tập 58.4 — Ledger xanh nhưng thiếu bằng chứng capstone**

Một ledger có tag prod, CI xanh và dashboard xanh nhưng thiếu digest, request ID và restore test. Đạt capstone không?

# Lời giải bài tập

## Đáp án chương 1 — Công việc, thời gian và đường đi dữ liệu

**Bài tập 1.1 — Thời gian CPU và thời gian chờ**
Wall time 100 ms, CPU time 32 ms. Giữ nguyên 68 ms chờ và giảm CPU xuống 16 ms cho 84 ms. Đây là tính toán theo giả định; không dự báo phép đo thật.

**Bài tập 1.2 — CPU time, thời gian chờ và độ trễ**
Giữ nguyên 60 ms chờ thì wall time còn 20+60=80 ms trong mô hình; lịch chạy và I/O thật có thể thay đổi. R của ps gộp running và runnable, một snapshot không cho thời lượng mỗi trạng thái. Hai luồng cộng 80 ms CPU time của tiến trình trong 50 ms tường vì có thể chạy song song trên hai lõi. CPU time của toàn tiến trình không phải độ trễ của request riêng và có thể gồm việc của request khác.

**Bài tập 1.3 — Tên tệp sau mất điện**
Cần kiểm việc đồng bộ nội dung tệp và mục nhập thư mục sau đổi tên, đồng thời kiểm lỗi từng lời gọi trên filesystem mục tiêu. HTTP thành công không chứng minh dữ liệu đã bền vững sau mất điện.

## Đáp án chương 2 — Chương trình, tiến trình và ranh giới nhân

**Bài tập 2.1 — Thay tệp chạy khi dịch vụ còn hoạt động**
Artifact trên đĩa và lần thực thi đang chạy là hai thực thể khác nhau. Việc ghi đè một đường dẫn không chứng minh process image đã đổi. Cần kiểm tra PID và thời điểm khởi động, lệnh và artifact mà dịch vụ đang dùng, rồi so với định danh bản triển khai. Nếu ứng dụng có endpoint phiên bản, chỉ dùng nó khi biết endpoint phản ánh đúng build đang chạy. Khởi động lại phải theo quy trình triển khai, có kiểm tra sức khỏe và đường quay lui.

**Bài tập 2.2 — Nhánh lỗi sau fork và execve**
Sau `fork` thành công có hai tiến trình: cha và con. Nếu con gọi `execve` và lời gọi thất bại, chính nhánh con nhận giá trị lỗi và có thể báo lại; cha chỉ biết nếu có cơ chế thông báo hoặc chờ kết thúc. `execve` thay chương trình trong tiến trình gọi, nên không tăng số tiến trình. Trên đường thành công, lệnh sau `execve` trong chương trình cũ không chạy tiếp.

## Đáp án chương 3 — Lời gọi hệ thống, đường dẫn và hệ thống tệp

**Bài tập 3.1 — Tệp log cũ vẫn nhận dữ liệu**
Descriptor của tiến trình tham chiếu open file description đang mở, không tra cứu lại tên đường dẫn cho mỗi lần ghi. Đổi tên hoặc tạo tệp mới cùng tên không chuyển descriptor cũ. Cách xử lý là yêu cầu dịch vụ mở lại log bằng cơ chế được nó hỗ trợ, hoặc khởi động lại có kiểm soát sau khi bảo đảm log đã được thu thập và khả năng phục vụ không bị gián đoạn ngoài mức chấp nhận. Không xóa cưỡng bức dữ liệu trước khi xác định descriptor và nhu cầu lưu giữ.

**Bài tập 3.2 — Ghi cấu hình bền vững**
Chưa đủ. Lời gọi ghi thành công không tự bảo đảm dữ liệu và metadata đã tới thiết bị bền vững. Khi tên tệp được tạo hoặc đổi, cần kiểm tra hợp đồng đồng bộ tệp và thư mục chứa mục tên đó, cùng hành vi hệ thống tệp thực tế. Không nên biến `fsync` trên tệp thành lời hứa tuyệt đối cho mọi bước của thao tác thay tên.

# Đáp án chương 4 — Danh tính và quyền trong Linux

**Bài tập 4.1 — Quyền đi qua thư mục cha**

Nhóm `atlas` có thể đọc tệp theo bit nhóm của `0640`, nhưng không có quyền tìm kiếm trên `/srv/atlas` khi thư mục là `0700 root:root`; phân giải đường dẫn có thể dừng ở thư mục đó với `EACCES`. Kiểm tra quyền từng thành phần và danh tính thực của dịch vụ; nếu hợp đồng cho phép, cấp quyền tìm kiếm trên thư mục cho đúng nhóm hoặc chuyển cấu hình sang cây thích hợp. Không đổi tệp thành `0666` hay dùng `chmod -R 777`: hai cách đó vừa không sửa quyền đi qua thư mục, vừa mở rộng truy cập.

**Bài tập 4.2 — Tìm điểm gây lỗi EACCES**

Điểm chặn đầu là quyền tìm kiếm ở /srv/atlas. Kiểm dưới đúng danh tính và ngữ cảnh dịch vụ sau một thay đổi quyền hẹp được duyệt. chmod 777 cho tệp cuối không cấp quyền đi qua thư mục cha.

# Đáp án chương 5 — Tài nguyên và chẩn đoán hiệu năng Linux

**Bài tập 5.1 — Bộ nhớ ảo và bộ nhớ cư trú**

Không gian địa chỉ ảo đo vùng địa chỉ mà tiến trình có thể dùng, không phải số trang vật lý đang cư trú. Cần phân biệt kích thước ánh xạ ảo và bộ nhớ cư trú; nếu phân tích áp lực chung, còn phải xét trang dùng chung, cache và cách cgroup tính bộ nhớ.

**Bài tập 5.2 — Dữ liệu ghi và độ trễ I/O**

Writeback chậm là một giả thuyết. Cần ghép thời điểm request và thời gian fsync/I/O cùng ranh giới process, cgroup, thiết bị. Chưa thể quy toàn bộ 80 MiB dirty hay độ trễ cho request này vì các counter có thể gồm tác vụ khác.

**Bài tập 5.3 — Đọc nhiều tín hiệu tài nguyên**

`cpu.stat usage_usec` chia cho 60.000.000 µs cho 0,30 và 0,32 lõi trung bình; `nr_throttled` không tăng nên chưa có dấu vết quota CPU tại cgroup đã đo. `memory.current` cuối kỳ tăng 5 MiB, còn `memory.events high` và `oom` không tăng; chưa thấy chạm hai ngưỡng này, nhưng chưa loại trừ thu hồi ở máy hoặc cgroup tổ tiên. `io.pressure some total` chia cho 60.000.000 µs cho 2% và 30% thời gian có ít nhất một tác vụ nghẽn I/O. Cùng với p95 tăng, đó là lý do ưu tiên điều tra I/O. PSI không gắn thời gian chờ với từng yêu cầu hay thiết bị; bước tiếp theo là đo thời gian thao tác đọc theo yêu cầu, đối chiếu `io.stat` và tình trạng thiết bị trong cùng cửa sổ. Chưa thể quy toàn bộ độ trễ cho I/O.

# Đáp án chương 6 — systemd và vòng đời dịch vụ

**Bài tập 6.1 — systemctl start đã chứng minh gì?**

`Type=exec` xác nhận systemd đã thiết lập và thực thi được binary chính, theo hợp đồng của loại unit này. Nó không chứng minh ứng dụng đọc cấu hình thành công, mở cổng hay trả HTTP đúng. Xem nhật ký và trạng thái tiến trình chính, rồi gọi health endpoint hoặc phép thử chức năng từ đúng đường đi của người dùng. Một PID `active` không đủ thay cho phép thử khả năng phục vụ.

**Bài tập 6.2 — Phân biệt lỗi khởi chạy dịch vụ**

Với binary thiếu, bước chuẩn bị/thực thi của systemd thất bại trước khi mã ứng dụng chạy. Với `openat` trả `EACCES`, binary đã chạy; nhân từ chối một thao tác tệp của tiến trình, ứng dụng phải ghi lỗi và chọn cách thoát. Nếu tiến trình thoát lỗi, `Restart=on-failure` có thể thử lại, rồi bị giới hạn tần suất khởi động. Tăng `RestartSec` chỉ đổi nhịp thử; đường dẫn hoặc quyền sai vẫn sai. Cần sửa đúng artifact, danh tính và quyền sau khi kiểm tra nguyên nhân.

# Đáp án chương 7 — Vận hành máy chủ và phục hồi có kiểm chứng

**Bài tập 7.1 — Hậu kiểm sau khi đổi máy chủ**

User atlas không có quyền search trên /srv/atlas khi không thuộc group root; phân giải đường dẫn có thể trả EACCES trước khi kiểm quyền đọc của orders.db. Không sửa bằng chmod 777 hay chạy service bằng root. Ghi UID/GID thực, mode/ACL của từng thành phần đường dẫn và hash config; xác nhận process dưới user atlas mở được file, request qua đường client trả đúng order, đồng thời principal ngoài phạm vi vẫn không đọc được dữ liệu.

**Bài tập 7.2 — Bản sao tệp khi cơ sở dữ liệu đang ghi**

Chưa. Hash chỉ nói byte của từng file không đổi sau copy; hai file có thể thuộc hai mốc giao dịch khác nhau, và archive có thể thiếu WAL/key/schema. Dùng cơ chế backup nhất quán của ứng dụng hoặc dừng ghi theo hợp đồng; restore vào đường dẫn tạm độc lập, không đè nguồn. Kiểm mở database bằng đúng version/identity, invariant giữa đơn O-7 và outbox, rồi thử đường đọc/ghi ứng dụng. Ghi rõ phần chỉ kiểm byte và phần thực sự kiểm phục hồi.

**Bài tập 7.3 — Dữ liệu sau hoàn nguyên cấu hình**

Không báo phục hồi. HTTP 200 chỉ chứng minh một request có response, còn dữ liệu sai là hậu điều kiện thất bại. Giữ bản hiện tại làm chứng cứ, so revision/config với bản trước, đối chiếu order-7 và các ghi sau thay đổi; nếu process đã mở file nhưng nội dung sai thì chuyển sang runbook dữ liệu, không restart lặp hay mở quyền vô cớ. Sau phục hồi phải đọc đúng order-7, thử ghi/đọc operation mới có khóa idempotency qua đường client, xác nhận quyền còn hẹp và log không còn đường lỗi.

# Đáp án chương 8 — Địa chỉ, mạng con và đường đi gói IP

**Bài tập 8.1 — Chọn tuyến đến địa chỉ đích**

Route `/24` cụ thể hơn route mặc định `/0`, nên được ưu tiên trong mô hình đã nêu. Trên Linux còn phải xét policy routing, bảng route, địa chỉ nguồn và namespace. `ip route get` trong đúng ngữ cảnh cho biết quyết định hiện tại, không bảo đảm gói thực sự qua được gateway.

**Bài tập 8.2 — MAC của chặng kế tiếp**

Client chọn gateway `10.20.1.1` làm next hop vì đích nằm ngoài prefix trực tiếp. Nó cần MAC của gateway trên liên kết cục bộ để đóng frame đầu tiên. MAC của server ở mạng xa không được ARP trực tiếp qua các router; mỗi chặng có frame và phép phân giải láng giềng riêng.

**Bài tập 8.3 — MTU và gói có cờ DF**

Router bỏ gói quá lớn và có thể trả ICMP fragmentation needed; nguồn phải giảm ước lượng PMTU. Payload tối đa 1400 - 20 - 20 = 1360 byte theo giả định. Ping nhỏ không thử ngưỡng 1400 và không chứng minh ICMP cần thiết quay về được.

# Đáp án chương 9 — Tầng vận chuyển: TCP, UDP và NAT

**Bài tập 9.1 — ACK cộng dồn và truyền lại**

SYN chiếm một vị trí nên sau `seq=1000`, byte hoặc số thứ tự tiếp theo của client là 1001; server báo `ack=1001`. Bốn byte ở 1001–1004 làm số tiếp theo là 1005, nên ACK cộng dồn là 1005. Nếu ACK mất rồi cùng đoạn dữ liệu được gửi lại với `seq=1001`, server vẫn báo `ack=1005` vì không xuất hiện byte mới liên tục; TCP nhận diện đoạn lặp và không giao bốn byte lần hai cho ứng dụng. Không thể suy từ một capture đơn lẻ rằng mất ACK là nguyên nhân duy nhất của retransmission.

**Bài tập 9.2 — SYN-ACK thất lạc trên đường về**
Kiểm tra route trả từ server tới địa chỉ nguồn mà server thực sự thấy, rồi kiểm tra ánh xạ NAT và lọc trên gateway hoặc các chặng về. Có thể đối chiếu capture hai phía của NAT theo thời gian và tuple trước/sau dịch. SYN-ACK đã rời server nhưng chưa tới client nghĩa là bắt tay TCP chưa hoàn tất ở client; HTTP chưa bắt đầu nên chưa thể quy nguyên nhân cho ứng dụng HTTP.

**Bài tập 9.3 — Kết nối bị từ chối trên địa chỉ loopback**
Giả thuyết mạnh nhất là server chỉ bind trên loopback, nên không có listener cho `198.51.100.20:443`. Vẫn kiểm tra quy tắc lọc có thể chủ động gửi RST. Sau khi sửa cấu hình bind trong môi trường được phép, xác nhận `ss -ltn` có đúng địa chỉ/cổng, thấy SYN, SYN-ACK, ACK và thử yêu cầu ứng dụng. `systemctl active` chỉ phản ánh trạng thái unit/process, không chứng minh socket nhận từ xa hay ứng dụng phục vụ đúng.

**Bài tập 9.4 — NAT và đường trả gói**
Server trả tới `203.0.113.5:40001`, không gửi thẳng tới `10.20.1.10:51514`. Gateway cần ánh xạ còn hiệu lực và đường đưa gói về mạng trong để dịch đích trở lại. Ánh xạ NAT là trạng thái ở thiết bị dịch; `SYN-SENT` hay `ESTABLISHED` là trạng thái TCP của hai endpoint. Có mục NAT không chứng minh server đã chấp nhận kết nối.

**Bài tập 9.5 — Phản hồi DNS không về client**
Đã có bằng chứng request tới resolver và resolver phát response ở điểm capture ấy. Cần kiểm tuple sau NAT, route/filter chiều về và capture gần client; timeout chưa định vị nơi mất. Retry là chính sách ứng dụng hoặc thư viện, không là bảo đảm của UDP.

## Đáp án chương 10 — Tên miền, TLS và ngữ nghĩa HTTP

**Bài tập 10.1 — Địa chỉ DNS cũ trong bộ nhớ đệm**
Ghi tên và loại bản ghi mà ứng dụng hỏi, resolver nó dùng, thời điểm cùng TTL còn lại, rồi đối chiếu địa chỉ mà chính tiến trình đã kết nối. Có thể kiểm tra dữ liệu authoritative và cache của resolver để định vị chỗ sai. Một máy khác có cache, cấu hình resolver, đường đi hoặc thời điểm khác; kết quả của nó không đại diện cho client đang lỗi.

**Bài tập 10.2 — DNS qua UDP được, qua TCP lỗi**
`TC=1` báo phản hồi UDP bị cắt; resolver cần một đường lấy câu trả lời đầy đủ, trong tình huống này là thử lại qua TCP/53. UDP thành công cho truy vấn nhỏ không kiểm được đường TCP. Giữ cùng tên, loại bản ghi, resolver và thời điểm; xác nhận cờ `TC`, quan sát SYN/SYN-ACK ở hai đầu đoạn TCP, route và quy tắc lọc có hiệu lực để xác định nơi kết nối bị chặn. Sau sửa, cùng truy vấn phải nhận phản hồi đầy đủ qua đường được chọn và ứng dụng dùng đúng địa chỉ; DNS thành công vẫn chưa chứng minh TLS/HTTP. Không thể quy lỗi cho dữ liệu authoritative chỉ từ timeout TCP.

**Bài tập 10.3 — Chứng chỉ TLS không khớp tên**
Đối chiếu tên URL, SNI mà client gửi, chứng chỉ proxy chọn, tên trong chứng chỉ, chuỗi tin cậy, thời hạn và đồng hồ của client. `Host` là thông tin HTTP sau TLS; đổi nó không làm chứng chỉ đã chọn hợp lệ cho tên yêu cầu. Tắt xác minh biến lỗi rõ ràng thành một kết nối không được xác thực, không sửa binding hoặc chứng chỉ sai.

# Đáp án chương 11 — Reverse proxy và điều tra yêu cầu qua hai chặng

**Bài tập 11.1 — Client hết hạn nhưng đơn vẫn được ghi**

Client hết hạn lúc 1.500 ms nên không đọc được 201, dù upstream trả trước hạn 2.000 ms của proxy. Đơn của lượt đầu có thể đã commit ở 1.700 ms; retry ở 1.600 ms có thể chạy đồng thời và tạo đơn thứ hai. Cần ghép request ID/log hai chặng với sổ đơn để biết lượt đầu đã tới và commit hay chưa; kiểm hợp đồng khóa idempotency hoặc cách xác minh lượt đầu chắc chắn chưa áp dụng trước khi tự thử lại. Timeout phía client không hoàn nguyên giao dịch.

**Bài tập 11.2 — Định vị lỗi theo từng tầng kết nối**

Timeout trước TLS sau khi DNS có địa chỉ mới chỉ cho biết chưa có kênh TCP hoàn tất; xem SYN/SYN-ACK ở client và proxy. Certificate error chứng minh đã tới giai đoạn xác thực TLS, chưa có HTTP request hợp lệ; xem SNI và chứng chỉ. `502` chứng minh một HTTP response mang mã ấy đã tới client, cần log proxy xác định ai tạo và phản hồi upstream nào bị coi không hợp lệ. `504` cũng là HTTP response; đối chiếu thời gian proxy, upstream và nguồn tạo status. Không suy upstream chết từ `502`: ứng dụng có thể còn chạy nhưng trả header sai, đóng kết nối, hoặc chính upstream trả mã đó.

**Bài tập 11.3 — Thử lại yêu cầu tạo đơn hàng**

Lần đầu có thể đã tạo đơn hàng dù proxy hết chờ, nên retry sang upstream khác có nguy cơ tạo bản ghi thứ hai. Chỉ retry tự động khi thao tác idempotent theo hợp đồng hoặc có khóa chống lặp và khả năng truy lại kết quả lần đầu; cần giới hạn số lần, thời gian và quan sát cả hai lần. Tăng timeout đổi ngưỡng proxy chờ, có thể giữ kết nối lâu hơn và che triệu chứng, nhưng không làm truy vấn dữ liệu hoặc phụ thuộc upstream nhanh hơn.

# Đáp án chương 12 — Đối tượng Git, vùng chờ và tham chiếu

**Bài tập 12.1 — Blob, tree và commit khi đổi tên tệp**

Từ C1 sang C2, blob nội dung giữ nguyên, tree đổi do tên entry đổi và commit đổi do tree/parent mới. Từ C2 sang C3, blob, tree và commit đều đổi. Git lưu snapshot qua object; khi so snapshot, diff có thể suy luận rename theo độ giống, không có object sự kiện rename riêng. Kết luận giả định không có sửa mode hay byte trong bước C2.

**Bài tập 12.2 — HEAD, index và cây làm việc**

`HEAD` trỏ tới `B` với giá trị `1000`; index có snapshot `2000` tại lúc `git add`; working tree đã đổi thành `3000`. Commit ngay lấy index nên ghi `2000`, còn `3000` vẫn là thay đổi chưa stage. `git diff --cached` cho thấy `1000 → 2000`, `git diff` cho thấy `2000 → 3000`; stage lại rồi xem diff nếu định commit `3000`.

**Bài tập 12.3 — Commit, tag và nhánh từ xa**

Không tên nào trong ba ref xác định cùng commit ở snapshot này: main=D, tag=B, origin/main=C. origin/main là bản ghi cục bộ nên chưa thể kết luận branch trên server hiện trỏ C; cần fetch/đối chiếu remote rồi ghi object ID.

# Đáp án chương 13 — Tích hợp nhánh và sửa lịch sử

**Bài tập 13.1 — Xung đột khi gộp hai nhánh**

`D` và `C` đều là con của `B`, không commit nào là tổ tiên của commit kia; dịch `main` thẳng tới `C` sẽ bỏ `D` khỏi lịch sử đi theo `main`. Merge sau giải conflict tạo `M` có cha thứ nhất `D`, cha thứ hai `C`; `main` trỏ tới `M`, feature vẫn trỏ tới `C`. Khi dừng vì conflict, `HEAD` còn ở `D`; index giữ stage 1 từ `B`, stage 2 từ `D` và stage 3 từ `C` cho đường dẫn xung đột, thay cho một bản đã giải ở stage thường.

**Bài tập 13.2 — Viết lại commit bằng rebase**
Rebase tạo `C'` có cha `D`, nên nhánh feature đi theo `A → B → D → C'`; `C'` có định danh mới vì parent của commit đã đổi. Nếu `C` đã được người khác dùng, dịch ref tới `C'` tạo lịch sử khác với lịch sử họ đang có và buộc phối hợp khi cập nhật nhánh chung. Merge giữ cả `C` lẫn `D` và ghi quyết định tích hợp ở `M`. Cả hai cách đều có thể cho snapshot sai; cần test timeout và retry trên kết quả cuối.

**Bài tập 13.3 — Hoàn nguyên merge trên nhánh chung**
`git revert -m 1 M` tạo một commit mới đảo tác động merge so với cha thứ nhất `D`; phải chọn mainline vì `M` có hai cha và "đảo merge" phụ thuộc cha nào được giữ làm nền. Xem diff vì nó không đơn giản đưa toàn kho về `B`, và merge lại nhánh cũ có thể không tự phục hồi thay đổi đã đảo. `git reset D` dịch ref `main` về `D`; nếu đã push `M`, lịch sử branch chia sẻ bị viết lại và bản sao của người khác có thể lệch. Cần review diff, xác nhận branch/ref và cha, chạy kiểm thử liên quan trên snapshot sau sửa, rồi kiểm tra trạng thái trước push.

## Đáp án chương 14 — Bash và ngữ nghĩa lỗi của shell

**Bài tập 14.1 — Tách từ và ký tự đại diện trong Bash**
`tar -C $src` nhận hai word `/tmp/du` và `lieu` sau word splitting trong trường hợp giá trị đã cho; `tar -C "$src"` nhận một word `/tmp/du lieu`. Nếu biến chứa ký tự glob như *, lần mở rộng tên tệp có thể biến một word thành nhiều đường dẫn. `set -u` chỉ phát hiện một số lần dùng biến chưa đặt, không chặn tách word hay globbing. Lưu danh sách bằng mảng và truyền `"${files[@]}"` để bảo toàn ranh giới từng phần tử.

**Bài tập 14.2 — Pipeline báo thành công giả**
Không có `pipefail`, status pipeline là 0 từ `tee`, nên `set -e` không thoát; `printf` in `BACKUP_OK` và script kết thúc 0. Có `pipefail`, pipeline trả 2 trong giả định này, `set -e` ở vị trí top-level khiến script thoát trước `printf`. Không nên chỉ dựa vào tương tác `set -e` vì nó có ngoại lệ ngữ cảnh; kiểm tra status tường minh rõ hơn. `tar` trả 0 chứng minh thao tác của `tar` đạt hợp đồng trả status của nó trong lần chạy ấy, không chứng minh restore hoàn chỉnh, nguồn nhất quán hoặc bản ghi đã bền vững qua mất điện.

**Bài tập 14.3 — Giới hạn của set -e trong hàm**
Hàm được gọi làm điều kiện `if`, nên `errexit` bị bỏ qua trong thân hàm ở ngữ cảnh này: `false` trả 1 nhưng `printf` vẫn chạy và trả 0. Hàm kết thúc 0, do đó in `done` rồi `ok`. Có thể viết `f() { if bước_đầu; then :; else return $?; fi; printf 'done\n'; }`, trong đó `bước_đầu` là lệnh thật cần kiểm tra. Cách khác là `bước_đầu || return $?` trong hàm, với chú ý lan truyền lỗi rõ ràng.

## Đáp án chương 15 — Python và Go điều phối tiến trình

**Bài tập 15.1 — Trạng thái từng bước của chương trình Python**
`precheck` trả 0; `apply_fail` ghi `partial` rồi trả 7; `verify` đọc trạng thái ấy và trả 8. `Popen` khởi chạy helper, `communicate` chờ và thu dữ liệu; sau khi con kết thúc, `returncode` mang status cần xét. Nhánh `bad` ghi hai lỗi ra stderr nhưng bỏ qua status trong quyết định cuối, vẫn in `DEPLOY_OK` và thoát 0. Nhánh `checked` dùng status 7 để dừng với mã 1, nên không chạy `verify`; file vẫn chứa `partial`, vì dừng điều phối không đảo tác dụng phụ đã có.

**Bài tập 15.2 — Đối số argv và nguy cơ chèn lệnh**
Khi truyền `prod; echo LEAK` là một phần tử argv với API không gọi shell, dấu chấm phẩy là dữ liệu trong đối số; với `sh -c` và chuỗi ghép không được bảo vệ, nó có thể trở thành dấu tách lệnh. Dù dùng argv, utility vẫn có thể diễn giải giá trị bắt đầu bằng `-` như tùy chọn hoặc chấp nhận tên ngoài schema; cần xác thực và

dùng ranh giới tùy chọn theo hợp đồng utility. Secret có thể lộ qua argv trong danh sách tiến trình, environment được kế thừa, hoặc log in nguyên command/exception.

**Bài tập 15.3 — Timeout và trạng thái còn dở trong Go**
Helper phát `READY` sau khởi tạo và đợi `go`; cha chỉ bắt đầu bộ đếm 0,5 giây sau khi gửi `go`. `apply_slow` ghi `partial` trước khi ngủ hai giây; khi bộ đếm hết, cha hủy context, đợi `Wait` và trả nhãn `TIMEOUT`. Lỗi `Wait` sau khi kill không phải exit status 7 của nhánh `apply_fail`; `ctx.Err()` khi chủ động hủy là `context.Canceled`, nên không dùng nó một mình để suy ra nguyên nhân. Kill không hoàn nguyên file. `CommandContext` mặc định tác động tới tiến trình con trực tiếp; không thể kết luận tiến trình cháu hay yêu cầu từ xa đã dừng nếu không có cơ chế và quan sát riêng.

## Đáp án chương 16 — Quy trình tự động hóa an toàn và chạy lại

**Bài tập 16.1 — Hai lượt công bố bản sao lưu**
Kiểm tồn tại và `rename()` là hai thao tác tách rời: A và B đều có thể thấy tên chưa có trước khi một bên công bố; `rename()` thông thường có thể thay tên đích đã tồn tại. Trong trạng thái đề bài, A không được coi tệp tạm cũ là hợp lệ hoặc ghi đè bản B. Dùng cơ chế công bố không thay thế tên đích trên filesystem hỗ trợ, hoặc giao thức có bảo đảm tương đương; nếu tên đã tồn tại, so danh tính/manifest của bản cuối với lượt mong đợi và báo xung đột khi khác. Bản được chấp nhận phải đọc và trích xuất dữ liệu mong đợi trong môi trường riêng, đồng thời giữ dấu vết lượt tạo. Tính nguyên tử của tên chưa chứng minh byte bền sau mất điện hay dữ liệu nguồn nhất quán.

**Bài tập 16.2 — Chạy lại bản sao lưu an toàn**
Lần thứ hai vẫn tạo và kiểm tra file tạm; tới `ln`, tên `backup.tar` đã có nên GNU `ln` mặc định trả khác 0 và script thoát 5. `EXIT` trap xóa file tạm của lần ấy; hard link của bản đầu còn nguyên. `tar -tf` chỉ kiểm tra khả năng đọc cấu trúc archive; cần thử khôi phục và so dữ liệu, kiểm tra snapshot nhất quán và bền vững theo hệ thống tệp. `SIGKILL` không chạy trap, có thể để lại file tạm; nguồn bị sửa đồng thời có thể cho bản sao không nhất quán dù cleanup đúng. Việc phát hiện và xử lý hai rủi ro đó đòi hỏi cơ chế khác.

## Đáp án chương 17 — Dựng phần mềm và định danh sản phẩm

**Bài tập 17.1 — Định danh artifact khi chuyển môi trường**
Build lại tạo một tập byte mới; kết quả test staging chỉ gắn với digest cũ, còn tag có thể bị trỏ lại. Ghi commit, đầu vào và phiên bản build, digest artifact được test, test-run, approval, môi trường và digest thực sự được triển khai; promotion phải dùng lại digest đã test hoặc chạy lại cổng phù hợp cho artifact mới. Dependency không khóa, timestamp, toolchain hoặc cờ build khác có thể đổi byte dù commit giống nhau. Muốn kết luận tái lập từng byte phải so digest từ các lượt build độc lập có đầu vào kiểm soát.

**Bài tập 17.2 — Cùng commit nhưng khác digest**
Không. Hai digest khác nhau là hai chuỗi byte khác nhau trong phạm vi hash; test của H84 chưa chứng minh H84b. Ghi môi trường và tập đầu ra, so byte/metadata để tìm biến thiên, tái lập hoặc chạy lại gate trên chính digest sẽ promote.

# Đáp án chương 18 — Cổng kiểm thử và CI/CD

**Bài tập 18.1 — Kiểm thử bỏ sót sai lệch nghiệp vụ**
Unit test không phủ đơn giá trị 1234; readiness chỉ chứng minh điều kiện của probe, không kiểm tra phép ghi tiền. Gửi một đơn canary có đầu vào biết trước, đối chiếu bản ghi và sổ độc lập, gắn kết quả với `H84`; 123400 thay vì 1234 là bằng chứng lỗi cụ thể. Bốn mẫu cho phép dừng rollout nhưng không ước lượng chính xác tỷ lệ lỗi toàn hệ thống, nhất là khi luồng canary hoặc phân phối đơn hàng không đại diện.

**Bài tập 18.2 — Readiness xanh nhưng dữ liệu sai**
So giá trị đã ghi với input và quy tắc nghiệp vụ trên cùng request/artifact; readiness chỉ chứng minh probe đã đáp ứng.

# Đáp án chương 19 — Triển khai theo đợt và quyết định dừng

**Bài tập 19.1 — Chọn chiến lược triển khai khi thiếu tài nguyên**
Blue-green đầy đủ cần giữ hai tập có năng lực phục vụ, nên thiếu tài nguyên nếu chỉ thêm được một Pod. Rolling có thể tăng một Pod rồi giảm dần bản cũ; canary có thể dùng chính Pod đó ở tỷ lệ lưu lượng nhỏ nếu bộ định tuyến hỗ trợ, sau đó mở rộng từng bước. Trong cả hai, 1.8.3 và 1.8.4 cùng truy cập schema nên việc thêm cột nullable phải giữ bản cũ đọc/ghi được; thông điệp, job và cache cũng cần xét. Sai một đơn tiền đã đối chiếu là lý do dừng ngay, dù tỷ lệ lỗi HTTP tổng thể vẫn đẹp.

**Bài tập 19.2 — Hai bản phát hành ghi đè nhau**
Approval cho phép từng lượt nhưng không đảm bảo thứ tự và không xác nhận phiên bản đích tại lúc ghi. Cần serialize release theo môi trường và dùng kiểm tra điều kiện: chỉ thay nếu digest hiện tại đúng phiên bản mà lượt này dự kiến. Nếu `R2` đã thay trước, `R1` phải dừng hoặc được đánh giá lại, không ghi đè im lặng. Khóa CI chỉ quản lý các lượt đi qua CI; công cụ hoặc người triển khai trực tiếp vẫn có thể thay đích, nên hệ thống đích cũng phải kiểm tra danh tính bản hiện tại và quyền thao tác.

**Bài tập 19.3 — Phê duyệt không thay thế điều phối**
Phê duyệt cho phép từng lượt đi tiếp nhưng không xếp thứ tự ghi vào môi trường. Nếu R2 hoàn tất trước rồi R1 ghi sau, bản cũ có thể thay bản mới. Cần điều phối theo đích và kiểm phiên bản/digest hiện tại ngay trước khi thay; nếu khác kỳ vọng thì dừng để đánh giá lại. Khóa trong CI không bảo vệ trước công cụ triển khai khác.

# Đáp án chương 20 — Phục hồi phiên bản và tương thích dữ liệu

**Bài tập 20.1 — Pod Ready nhưng đơn hàng sai**
Gắn request với artifact và kiểm tra hậu điều kiện bản ghi; Ready không chứng minh tính đúng của phép ghi.

**Bài tập 20.2 — Rollback ứng dụng và dữ liệu**
`ROLLBACK` SQL chỉ hủy giao dịch còn mở, trong khi migration và các đơn đã commit. Chia toàn bảng cho 100 phá các đơn đúng do `H83` ghi xen kẽ và không xử lý thanh toán bên ngoài. Hậu điều kiện ứng dụng: chỉ `H83` nhận ghi, xác nhận digest ở đường phục vụ và đơn mới đúng. Hậu điều kiện dữ liệu: tập đơn do `H84` ảnh hưởng được khoanh bằng chứng cứ, từng giá trị đối soát và sửa có audit; cột mới còn tương thích. Hậu điều kiện bên ngoài: mỗi thanh toán liên quan có trạng thái xác minh bằng ID, xử lý bù nếu hợp đồng cho phép, các trường hợp chưa biết được ghi riêng.

# Đáp án chương 21 — Namespace, cgroup v2 và ranh giới thực thi

**Bài tập 21.1 — Quan sát PID và namespace mạng**

`NSpid` cho thấy cùng tiến trình có PID 3142 ở góc nhìn tổ tiên và PID 1 ở góc nhìn bên trong; danh tính network namespace khác host; listener chỉ bind loopback của ngăn mạng đang quan sát. Chưa thể kết luận client có hay không có đường publish/forward, cũng chưa biết request đã tới ứng dụng. Kiểm tra cấu hình chuyển tiếp và socket/route ở đúng hai đầu, rồi thử từ vị trí client: nếu TCP không thiết lập, điều tra đường mạng và listener; nếu có HTTP 500, kết nối TCP và tầng HTTP đã hoạt động đủ để có phản hồi, tiếp tục xem proxy/upstream và log ứng dụng. HTTP 500 riêng nó chưa chỉ rõ tiến trình nào sinh lỗi.

**Bài tập 21.2 — Quota CPU và số chu kỳ bị giới hạn**

Hiệu lần lượt là 4.700.000 microsecond CPU, 100 chu kỳ, 80 chu kỳ throttled và 2.000.000 microsecond throttling; mức dùng trung bình là 4,7/10 = 0,47 lõi. `50000/100000 = 0,5` lõi là trần trung bình theo chu kỳ. Throttling trong 80 chu kỳ cho thấy quota thực sự chặn công việc ở cửa sổ đo, nhưng không đo thời gian chờ của từng request và không loại trừ I/O, khóa hay upstream. Cần đối chiếu cửa sổ metric với trace/latency từng chặng trước khi quy nguyên nhân.

**Bài tập 21.3 — Sự kiện bộ nhớ và nguyên nhân OOM**

`max` tăng 2 nghĩa là hai lần mức dùng chạm giới hạn; `oom` tăng 1 ghi nhận tình huống cấp phát sắp thất bại có xét OOM; `oom_kill` tăng 1 xác nhận một tiến trình trong cgroup bị OOM killer giết. Muốn xác định nạn nhân cần ghép timestamp, PID/lượt chạy, log kernel và runtime; một số đo `memory.current` sau sự kiện có thể đã giảm. Lần khởi động sau thoát 2 với log `config missing` và counter OOM của cgroup mới không đổi thuộc nhánh lỗi cấu hình ứng dụng; sự kiện OOM cũ không tự giải thích lần thoát này.

# Đáp án chương 22 — Ảnh OCI, quy trình dựng và kho lưu trữ

**Bài tập 22.1 — Tag mới và container còn chạy ảnh cũ**

Build tạo artifact, push đưa manifest/blob lên registry, pull đưa nội dung về máy chạy; các bước ấy không tự thay tiến trình. Ghi commit nguồn, digest manifest đã kiểm thử, digest image được dùng tạo container mới, container ID, thời điểm chuyển lưu lượng và kết quả request từ client. Tag `orders:2.2` có thể trỏ lại nên tên tag riêng lẻ không đủ. Nếu máy chủ vẫn phục vụ container từ digest cũ, bản mới chưa được chứng minh là đang nhận request dù registry đã có nó.

# Đáp án chương 23 — Vòng đời container và dữ liệu

**Bài tập 23.1 — Container Running nhưng ứng dụng chưa sẵn sàng**

Digest nhận diện byte image đã kiểm thử, không chứng minh cấu hình runtime, dependency ngoài, mạng hay dữ liệu hiện thời đúng. `running` chỉ xác nhận chương trình đã được thực thi và chưa dừng; readiness đỏ cần phép thử ứng dụng cụ thể, ví dụ bind, phụ thuộc và request đại diện. Image cung cấp root filesystem và tham số mặc định; namespace đặt góc nhìn tài nguyên của tiến trình; cgroup giới hạn/phân phối tài nguyên. Bất kỳ tầng nào cũng có thể góp phần làm request thất bại, nên cần quan sát đúng tầng rồi mới kết luận.

**Bài tập 23.2 — Dữ liệu trong lớp ghi của container**

B có lớp ghi mới, không kế thừa lớp ghi của A. Kiểm tra identity A/B, digest image, mount type/source/destination và đọc dữ liệu của A hoặc bản sao trước khi xóa. Không suy rằng image mới đã xóa dữ liệu cũ; nếu từng dùng volume còn phải kiểm đúng volume identity.

**Bài tập 23.3 — Dữ liệu sau khi dừng hoặc xóa container**

Lượt A ghi trong writable layer riêng của container A. B được tạo mới từ image nên có lớp ghi mới, không tự nhận dữ liệu từ A. Nếu A chỉ dừng và còn container, lớp ghi có thể còn đọc được để điều tra; nếu A bị xóa,

không được hứa khôi phục từ lớp ghi ấy. Tránh xóa container cũ, chạy `down -v`, prune hoặc ghi đè đường dẫn nguồn trước khi xác định dữ liệu và có bản sao đã thử restore.

**Bài tập 23.4 — Ba nơi lưu cho cùng một đường dẫn**

A sau restart có thể thấy S và A41; B mới chỉ thấy S. V mới/rỗng được Docker sao chép seed.db=S từ view của image theo mặc định, nên C và D gắn đúng V thấy S và V41, không thấy A41. Bind mount thư mục host rỗng che seed.db trong container, không có bước sao chép mặc định ấy. Trước khi xóa, kiểm ID/image digest, Mounts.Type/Source/Destination, tên volume, nơi ứng dụng thật ghi và đọc bản ghi qua đúng mount; tạo backup rồi thử restore. Nếu dùng volume-nocopy hoặc V đã có nội dung, phải tính lại trạng thái ban đầu của V.

## Đáp án chương 24 — Compose và ứng dụng nhiều dịch vụ

**Bài tập 24.1 — Compose khởi động và trạng thái sẵn sàng**

Mạng mặc định cho các service cùng project đường kết nối và tên để tìm nhau; `depends_on` mặc định sắp thứ tự khởi động, không đảm bảo API hay database đã ready. Ngay cả healthcheck chỉ bao phủ phép đo được khai báo. Compose file cũng không tự backup volume hoặc tự lập lịch bản sao sang host khác khi host chết. Gửi request đại diện từ đúng vị trí client và kiểm tra dữ liệu trả về; tạo backup nhất quán của dữ liệu thật rồi thử restore trong môi trường riêng, so ID và checksum hoặc hậu điều kiện nghiệp vụ.

**Bài tập 24.2 — Dữ liệu khi đổi image và tên project**

D vẫn gắn `orders-case_orders_data` tại `/var/lib/orders`, nên đổi byte image không thay byte của named volume. Project `orders-test` tạo tài nguyên tên khác theo project; ứng dụng nhìn cùng đường dẫn nhưng có thể đọc volume mới. Bind mount rỗng đặt tại `/var/lib/orders` che nội dung ở điểm mount; volume cũ có thể vẫn tồn tại, chỉ không còn là nguồn dữ liệu hiện được gắn. Xác minh loại mount, `Source`, `Destination`, tên volume và đọc dữ liệu cũ theo cách không ghi trước khi sửa cấu hình.

## Đáp án chương 25 — Danh tính AWS và đánh giá chính sách IAM

**Bài tập 25.1 — Ba ranh giới khi ứng dụng đọc S3**

`/health` chỉ kiểm endpoint đó. A cần kiểm profile của instance, provider/IMDS và lỗi lấy credential của chính tiến trình; chưa có request S3 được ký theo dữ kiện. B chuyển sang DNS, route, bộ lọc mạng và endpoint của subnet; chưa có HTTP response nên không thể kết luận IAM cho phép hay từ chối. C cho biết S3 đã trả một response cho request ấy, nên phải ghi principal phiên role, action, ARN, điều kiện và đối chiếu các policy áp dụng; `AccessDenied` chưa chỉ đích danh identity policy. Thiếu request ID trong log ở ca khác cũng chưa đủ chứng minh request không tới S3. Sau sửa, thử đúng key và một key ngoài quyền bằng cùng danh tính, không ghi secret vào log.

**Bài tập 25.2 — Truy nguyên AccessDenied của S3**

Ghi principal phiên role, action `s3:GetObject`, ARN bucket/key, Region, điều kiện request và request ID. So identity policy, bucket policy, endpoint policy cùng session policy, boundary, SCP/RCP nếu áp dụng; nếu mã hóa bằng KMS key của khách hàng thì xét cả quyền và key policy thích hợp. Statement `Deny` trong identity policy của ví dụ khớp cùng object, nên thắng `Allow`. HTTP 403 riêng nó chưa xác định policy nào gây từ chối trong một sự cố khác, cũng không chứng minh request dùng đúng endpoint. Sau thay đổi quyền được review, phải đọc đúng key và kiểm tra nội dung, không chỉ thấy hết lỗi.

# Đáp án chương 26 — VPC, mạng con và đường đi trong AWS

**Bài tập 26.1 — Lần theo gói trả về qua hai subnet**
SYN tới server qua hai NACL và SG đã cho; SYN-ACK đi ra subnet server nhưng bị NACL inbound của subnet client chặn ở cổng đích 49152, nên bắt tay TCP không xong. ACCEPT ở interface server chỉ mô tả luồng tại điểm và cửa sổ ghi ấy, không chứng minh chiều về, TLS, HTTP hay IAM. Kiểm tuple/hướng/interface/thời điểm với luật NACL, rồi cho phép chiều trả về từ phạm vi nguồn cần thiết trên dải cổng tạm thực của client; SG đã theo dõi kết nối nên không mở inbound 49152 chỉ để sửa ca này. Thử lại TCP/TLS và request từ đúng client; chỉ khi nhận phản hồi API mới xét principal, action, resource và policy nếu có AccessDenied.

**Bài tập 26.2 — S3 timeout trong mạng riêng**
Xác nhận instance dùng đúng DNS resolver và đúng S3 regional endpoint; so địa chỉ trả về với Region cấu hình. Đọc route table thực sự gắn với subnet và liên kết gateway endpoint S3, không suy từ việc endpoint tồn tại trong VPC. Kiểm tra SG outbound TCP 443 và NACL ở cả hai chiều, đặc biệt inbound ephemeral khi instance là client; nếu có flow logs ở đúng ENI/cửa sổ, dùng chúng làm dấu vết phụ. DNS đã trả địa chỉ chỉ loại bớt lỗi phân giải ở lượt ấy; timeout xảy ra trước khi có phản hồi dịch vụ, nên chưa có bằng chứng IAM đã được xét. Trong dữ kiện đề, thiếu tuyến endpoint và không có NAT giải thích timeout.

# Đáp án chương 27 — EC2, cân bằng tải và miền lỗi

**Bài tập 27.1 — EC2 chạy nhưng API bị từ chối quyền**
Chúng chỉ chứng minh compute đang chạy và hợp đồng /health đã qua. Kiểm profile/role của request, policy trên action/object, đường mạng tới S3 và request /reports; không suy từ health check rằng S3 đọc được.

**Bài tập 27.2 — Tách đường ALB, S3 và STS**
`/health` xanh chứng minh ALB có target đáp ứng phép thử ấy, không chứng minh bước S3 của báo cáo. EC2–S3 có thể qua gateway endpoint của route table đúng, còn EC2–STS cần đường riêng như NAT hoặc interface endpoint phù hợp; timeout STS chưa chứng minh role thiếu quyền. Đối chiếu DNS, route, SG/NACL và kết quả TCP/TLS từng đích; với phản hồi `AccessDenied` mới chuyển sang kiểm tra principal và policy. Hậu điều kiện là request báo cáo từ client thực nhận đúng object/phiên bản, không có lỗi ẩn trong log và quyền chỉ mở cho action/resource cần thiết; ghi Region, request ID, thời điểm và kiểm chứng dữ liệu mà không lộ secret.

**Bài tập 27.3 — Mất một AZ khi health check vẫn xanh**
Chưa. Cần chứng minh ALB route tới target B, B đủ capacity, dependency và quyền S3 hoạt động, dữ liệu nghiệp vụ ở nơi B đọc được; thử request thật và đọc dữ liệu sau failover.

# Đáp án chương 28 — Lưu trữ khối, tệp và đối tượng

**Bài tập 28.1 — Độ sẵn sàng và phục hồi của ba loại lưu trữ**
Instance ở AZ hỏng không phục vụ; EBS volume của AZ ấy không thể attach trực tiếp cho instance AZ khác, cần snapshot đã có hoặc cơ chế sao chép/phục hồi khác để dựng volume ở AZ hoạt động, cộng với dữ liệu nhất quán và ứng dụng sẵn sàng. EFS Regional có dữ liệu phân tán nhiều AZ nhưng client ở AZ khác cần mount target, đường NFS, SG/quyền và ứng dụng thay thế hoạt động. S3 Standard được thiết kế qua nhiều AZ, nhưng object xóa nhầm cần versioning/bản sao lịch sử hoặc backup đã bảo vệ và thử restore; redundancy của trạng thái hiện tại không tự giữ một phiên bản đúng trong quá khứ. Khả năng phục hồi còn phụ thuộc RPO/RTO thực đo.

# Đáp án chương 29 — HCL, nhà cung cấp và đồ thị phụ thuộc

**Bài tập 29.1 — Khi nào cần depends_on?**
Ở cấu hình đầu, tham chiếu `config.output` cho Terraform suy ra cạnh `config → deploy`; thêm `depends_on` cho cùng quan hệ ấy là thừa. Khi dùng chuỗi literal, cạnh dữ liệu biến mất. Nếu resource triển khai thật có phụ thuộc vận hành vào một resource policy nhưng không đọc giá trị của nó, khai báo `depends_on` tới đúng resource policy để thể hiện thứ tự; không thêm cạnh chỉ vì hai resource cùng xuất hiện trong tệp. Cạnh chỉ ràng buộc thứ tự thao tác trong DAG: API báo tạo policy xong có thể chưa đồng nghĩa quyền đã lan truyền hoặc workload đã sẵn sàng. Trước apply, đọc plan, địa chỉ resource, action và provider/account đích; nếu plan dự kiến replace tài nguyên có dữ liệu thì dừng để xét hậu quả. Sau apply, đọc trạng thái policy tại API và thử request đại diện theo đúng principal/action/resource, gồm một ca phải bị từ chối. Nếu thiếu quyền hoặc môi trường để thử, ghi giới hạn thay vì coi plan thành bằng chứng readiness.

**Bài tập 29.2 — Hai plan khác nhau từ cùng HCL**
Provider gắn với resource quyết định API đích; alias không tự được child module kế thừa như default provider. Child phải khai báo alias cần dùng và root truyền mapping đúng; resource trong child phải chọn cấu hình đó. Trước apply, so provider source/version/checksum, mapping từng resource, credential thực, account/Region đã resolve, workspace/backend và ID dự kiến trong plan. Một plan chỉ báo 1 to add có thể hợp cú pháp mà nhắm sai Region hoặc account; nếu không xác minh được identity, dừng trước khi tạo tài nguyên.

**Bài tập 29.3 — Cạnh suy diễn và cạnh khai báo**
Cạnh schema→service suy từ tham chiếu giá trị; service→audit do depends_on tường minh, chỉ đặt thứ tự. Audit.input='ready' không chạy health check. Data source có đầu vào phụ thuộc thay đổi có thể bị đọc ở apply, nên người duyệt chưa biết ID đích; phải giải thích rủi ro hoặc tách thành hai lượt có kiểm chứng giữa chúng. Saved plan có thể chứa input nhạy cảm dạng đọc được, phải giới hạn quyền đọc/lưu và áp đúng backend/workspace. Sau apply vẫn kiểm object API và request nghiệp vụ; plan không là giao dịch hay bằng chứng readiness.

# Đáp án chương 30 — Kế hoạch thay đổi, trạng thái và kho lưu trữ trạng thái

**Bài tập 30.1 — Apply dở dang và lỗi ghi state**
Terraform không có giao dịch chung trên API và backend. Network/volume có thể đã tồn tại; timeout của API instance không đủ kết luận instance không được tạo; lỗi ghi state còn khiến ánh xạ lưu trữ khác thực tế. Giữ log và state phục hồi, tạm ngừng lượt triển khai, xác định khóa và phiên state cuối cùng. Đọc API theo ID/tag và thời điểm, đối chiếu từng object với state, tránh tạo thêm object trùng. Sửa ánh xạ bằng quy trình phục hồi/import đã review nếu cần, rồi lập plan mới và kiểm tra hậu điều kiện: đúng một chủ quản lý cho mỗi object, đúng dependency, volume/instance nhận diện chính xác, dữ liệu còn nguyên và request đại diện hoạt động. Chỉ tiếp tục khi nguyên nhân lỗi backend/API đã được xử lý.

# Đáp án chương 31 — Sai lệch hạ tầng, nhập tài nguyên và tái cấu trúc

**Bài tập 31.1 — Chọn import hay moved**
Bucket: import vào địa chỉ resource đúng sau khi xác minh ownership. Instance: moved block cũ→mới; đọc plan để bảo đảm không có delete/replace ngoài ý định.

**Bài tập 31.2 — Đổi địa chỉ tài nguyên đang giữ dữ liệu**
Xác minh bảng ánh xạ ID thật [0]→a, [1]→b; viết moved blocks, backup state, plan từng workspace; dừng nếu plan có delete/replace ngoài ý định hay ID không khớp.

# Đáp án chương 32 — Thay đổi hạ tầng đang chứa dữ liệu

**Bài tập 32.1 — Replacement ngoài dự kiến**
HCL vẫn muốn user data A, state ghi liên kết `aws_instance.orders` với `i-demo-01` và giá trị đã lưu, còn instance thực có B nếu API xác nhận. Chỉ khi provider đọc được khác biệt và thuộc tính `user_data_replace_on_change = true` áp dụng thì thay user data có thể dẫn tới replacement; mặc định provider chỉ stop/start. Nếu instance bị terminate trong khi root EBS thật có `delete_on_termination = true` và 41 đơn chỉ ở đó, dữ liệu có thể mất. Trước apply phải xác nhận đúng account/Region/ID và plan có refresh, root block mapping/chính sách xóa và bản sao nhất quán đã thử restore; thêm lịch sử ai sửa B. Nếu B là cấu hình mong muốn, cập nhật HCL rồi plan lại. Nếu B sai, bảo vệ/chuyển dữ liệu và lập kế hoạch phục hồi được kiểm tra trước khi thay máy; không che drift bằng `ignore_changes` chỉ để plan xanh.

**Bài tập 32.2 — State lock và sửa ngoài Terraform**
Locking của backend, nếu hỗ trợ và cùng dùng đúng state, ngăn hai lượt Terraform cùng sửa state; nó không khóa console hoặc tất cả API AWS, nên thay security group ngoài Terraform vẫn là drift. Lượt plan kế tiếp có refresh và quyền đọc phù hợp mới có cơ hội thấy đổi khác. `sensitive = true` che hiển thị nhưng state/plan vẫn có thể chứa giá trị; cần giới hạn quyền, mã hóa và tránh nhúng credential. Versioning giữ bản cũ để khôi phục nhưng không ngăn đọc trái phép hay chứng minh restore chạy được; phải bảo vệ bản sao, kiểm tra quyền và thử phục hồi.

**Bài tập 32.3 — Khóa state có ngăn sửa qua console?**
Không; nó chỉ điều phối những lượt dùng cùng cơ chế backend/state. Phải refresh và xét lại plan sau thay đổi ngoài.

**Bài tập 32.4 — Apply xanh nhưng mất đơn hàng**
Chưa. So instance/root volume ID, đọc đủ 41/41 đơn từ nơi khôi phục, thử request nghiệp vụ, kiểm backup restore và plan mới; state snapshot cũ không chứa dữ liệu EBS.

# Đáp án chương 33 — Danh sách máy và đường điều khiển Ansible

**Bài tập 33.1 — Chọn đúng máy trước khi chạy Ansible**
Giao của nhóm `app` với `canary` chỉ có `atlas-a`; `atlas-db` không thuộc play. Bỏ giới hạn thì A và B được chọn, nhưng `unreachable` nghĩa task/module chưa chạy được trên B, nên không có bằng chứng hội tụ của B dù A báo `ok`. Trước thao tác thật, xem inventory đã phân giải và tập host của chính lượt chạy; đối chiếu từng alias, địa chỉ, danh tính kết nối với danh sách máy được duyệt. Sau khi chạy mới kiểm nội dung/quyền và request ứng dụng trên từng host.

**Bài tập 33.2 — Fact cache và drift thực tế**
Không. Fact cache là ảnh chụp tại thời điểm thu thập, nên một địa chỉ hay thuộc tính host có thể đã đổi sau đó. Xác định tuổi fact và host identity, thu thập lại khi cần, rồi kiểm trực tiếp nội dung, quyền và trạng thái dịch vụ là hậu điều kiện của task. Một recap ok chỉ phản ánh kết quả module ở lượt chạy ấy.

**Bài tập 33.3 — Thiếu quyền và become**
Không. become chỉ yêu cầu dùng cơ chế escalation sẵn có trên host; nó không cấp thêm quyền nếu policy sudo từ chối. Xác định connection user, become user, phương thức escalation và lỗi xảy ra trước hay trong module. Sau khi task chạy, kiểm owner, mode và khả năng dịch vụ đọc file, thay vì suy từ recap.

# Đáp án chương 34 — Hội tụ cấu hình, handler và lỗi từng máy

**Bài tập 34.1 — Hai lượt chạy hội tụ cấu hình**

Lượt đầu trên filesystem POSIX có thể báo changed cho việc tạo sandbox, thư mục dữ liệu, config và handler marker; `stat`/`assert` chỉ quan sát nên ok nếu hậu điều kiện đúng. Lượt thứ hai cùng biến, khi không có drift, task template là ok, handler không được notify và không chạy. `changed=0` chỉ nói các module không báo sửa ở phạm vi chúng quản lý; không thử kết nối HTTP, dữ liệu, phụ thuộc hay sức khỏe tiến trình Atlas. Trên ổ gắn không giữ mode POSIX, thí nghiệm quyền có thể thất bại dù module đặt mode và không được tính là kiểm chứng hội tụ.

**Bài tập 34.2 — Sửa quyền mà không restart thừa**

`file` sửa mode thư mục từ `0700` về `0750`; template không đổi nên handler không được notify, marker không bị chạm. Lệnh `command: chmod 0750 ...` không tự so trạng thái trước/sau theo hợp đồng cấu hình; nếu chạy và báo changed ở mọi lượt, handler bị gọi không cần thiết dù mode đã đúng. Cần dùng module có ý thức trạng thái hoặc điều kiện cùng kiểm chứng rõ ràng, và xác nhận service user thực sự cần quyền nào.

**Bài tập 34.3 — Check mode và lỗi apply từng phần**

Syntax/check không chứng minh kết nối, quyền hoặc API thật; module có thể không hỗ trợ check hay task phụ thuộc kết quả trước cho dự báo khác; diff chỉ hiển thị phần module hỗ trợ và còn có nguy cơ lộ secret. Khi apply lỗi sau một restart, ghi host nào đã đổi file/quyền, handler nào đã chạy, dịch vụ nào còn phục vụ và config đang có. Khoanh host lỗi, kiểm tra log và request, phục hồi từ bản cấu hình đúng nếu cần; trước lượt tiếp phải chứng minh file và quyền đúng, service healthy qua request đại diện, inventory/biến đúng, và kế hoạch rollout còn đủ năng lực phục vụ. Lỗi trên host này không tự rollback host trước.

# Đáp án chương 35 — Vai trò, bí mật và triển khai theo đợt

**Bài tập 35.1 — Giá trị biến hiệu lực trên từng máy**

Không có -e: atlas-a render 9090, atlas-b render 8080 theo dữ kiện. Có extra var 7070: cả hai render 7070 ở task dùng biến ấy. Khi file đã ghi 7070 mà socket còn 8080, precedence không còn là giả thuyết chính; kiểm file mà process thật đọc, handler/reload, lỗi cấu hình và socket/request từ đúng host. Bài toán giả định không có nguồn biến ưu tiên cao khác hoặc điều kiện task bỏ qua.

**Bài tập 35.2 — Chmod gây restart không cần thiết**

Task `chmod 0700` có thể bỏ quyền đi qua của group service; template `root:root 0600` có thể khiến user `atlas` không đọc được config. Task command báo changed rồi notify restart; khi process cũ còn chạy nó có thể chưa cần mở lại file, nhưng sau restart sẽ gặp lỗi quyền. Nếu hai máy cùng restart, không còn target phục vụ. Thu mode/owner/group và UID/GID tại từng host, diff config, recap task/handler, log systemd và thử request; không suy từ mỗi chữ changed. Sửa quyền và nội dung có kiểm tra trước, chỉ restart máy thử khi config thật sự đổi, xác nhận request rồi lần lượt tới máy khác; nếu host lỗi, dừng rollout và ghi trạng thái từng máy.

**Bài tập 35.3 — Vault, diff và rò bí mật**

Không. Vault chỉ mã hóa at rest; cần bảo vệ khóa giải mã, no_log/không in diff ở task nhạy cảm, quyền file đích và log ứng dụng, rồi kiểm đường đi secret và quy trình rotation.

# Đáp án chương 36 — API, etcd và vòng điều khiển hội tụ

**Bài tập 36.1 — Apply thành công nhưng chưa có Pod sẵn sàng**

Đọc Deployment ở đúng namespace, phân biệt `spec.replicas=2` với status và generation mà controller đã quan sát. Tìm ReplicaSet thuộc Deployment qua owner reference; nếu chưa có, xem điều kiện/events Deployment và khả năng controller/API bị chặn. Nếu ReplicaSet có nhưng không có Pod, xem selector, quota, lỗi tạo Pod và events của ReplicaSet. Với Pod đã có, `spec.nodeName` rỗng cùng `FailedScheduling` đưa chẩn đoán sang scheduler, requests và ràng buộc node; khi đã có node, đọc container waiting reason/events để tách image pull hoặc lỗi runtime. Không bước nào riêng lẻ chứng minh Service có backend hay ứng dụng đáp request.

# Đáp án chương 37 — Pod, Deployment và trạng thái sẵn sàng

**Bài tập 37.1 — Available không đồng nghĩa rollout hoàn tất**

Giới hạn surge cho phép tổng 5 Pod chưa tính Pod đang kết thúc, nên lúc đầu có thể tạo thêm 1 Pod mới. Vì phải giữ ít nhất 4 Pod Available, Pod mới chưa qua readiness chưa cho phép loại Pod cũ chỉ để tiến rollout. Nó chỉ được tính Available sau khi Ready liên tục đủ 10 giây; khi đó controller có thể giảm 1 bản sao cũ. Available=True chỉ cho biết ngưỡng khả dụng, Progressing=True vẫn có thể đang cập nhật. Xem reason NewReplicaSetAvailable, updated/ready/available replicas, ReplicaSet cũ/mới, status của thế hệ đã quan sát và rollout status. Sau cùng kiểm selector, EndpointSlice ready, cổng và response từ client cùng cửa sổ; các cờ không bảo đảm nghiệp vụ hay dữ liệu đúng.

# Đáp án chương 38 — Lập lịch và ngân sách tài nguyên

**Bài tập 38.1 — Pod Pending dù CPU thực còn rảnh**

Scheduler xét requests và node allocatable đã trừ phần được request bởi Pod khác, không dùng mức CPU tức thời 10% để lấp chỗ. A thiếu `500-300=200m`; B đủ theo requests nhưng taint `NoSchedule` loại Pod không toleration. Đọc event `FailedScheduling`, requests từng container, allocatable, tổng requests đã đặt, taint/toleration, affinity và ràng buộc volume ở đúng thời điểm. Nếu đổi ràng buộc, phải xác nhận node mới thật sự thích hợp về quyền, topology và năng lực rồi thấy Pod được gán và phục vụ; toleration riêng nó không bảo đảm đặt lên B.

**Bài tập 38.2 — Pod chờ lịch chạy và CPU bị giới hạn**

Kubernetes lấy limit 1000m làm request khi request vắng và không có mặc định khác, nên 700m còn lại không đủ để đặt Pod. Request 250m có thể gỡ riêng rào cản CPU nhưng chưa chứng minh các ràng buộc khác hoặc hiệu năng. CPU trung bình là 8,5/10=0,85 lõi=850m; usage/request=850/250=340%. 15 chu kỳ throttle chứng minh quota đã chặn cgroup trong cửa sổ, không cho biết 15% request chậm hay quota là nguyên nhân duy nhất của latency; cần đối chiếu thời gian, throttled_usec và từng chặng.

# Đáp án chương 39 — Service, EndpointSlice và NetworkPolicy

**Bài tập 39.1 — Probe đúng nhưng Service vẫn sai selector**

Probe sửa đúng path làm Pod có thể đạt Ready, nhưng Service vẫn chọn `app=atals` trong khi Pod mang `app=atlas`. Controller không thể sinh backend khớp selector sai. So selector và labels trong cùng namespace, rồi quan sát EndpointSlice gắn với Service có địa chỉ Pod với `ready=True`; sau đó kiểm tra Service port 80 tới `targetPort` 8080 và gửi request từ đúng vị trí client. `Running` và thậm chí Pod Ready riêng lẻ chưa chứng minh đường Service.

**Bài tập 39.2 — Timeout từ namespace khác**
DNS trả Service IP loại bớt lỗi phân giải ở lượt đó. So Service selector với nhãn Pod, `port`/`targetPort` và tất cả EndpointSlice/điều kiện Ready; thử kết nối từ chính client và từ nguồn đã hoạt động tới cùng đích/port. Đọc policy ingress ở đích và egress ở nguồn theo namespace/label, xác nhận plugin thực thi NetworkPolicy, rồi quan sát route/dataplane của cluster nếu còn lỗi. Một timeout không định danh policy, CNI hay ứng dụng; backend đáp từ một nguồn cũng chưa chứng minh nguồn khác được phép tới.

## Đáp án chương 40 — PVC, CSI và dữ liệu của ứng dụng

**Bài tập 40.1 — PVC Bound nhưng Pod không gắn được volume**
`Bound` chỉ xác nhận PVC đã ghép với PV, không xác nhận CSI attach/mount, quyền file, nội dung hay backup. `FailedMount` yêu cầu xem event theo UID, CSI node/controller, volume handle, topology và trạng thái attach; không nên xóa PVC/PV khi chưa hiểu reclaim policy và dữ liệu. RWO giới hạn read-write theo node chứ không đồng nghĩa một Pod; driver/storage vẫn có thể từ chối mount vì nguyên nhân riêng. Sau khi mount thành công, đọc dữ liệu/hậu điều kiện nghiệp vụ và thử restore bản sao riêng trước khi tuyên bố an toàn dữ liệu.

**Bài tập 40.2 — Pod mới không mang theo emptyDir**
ReplicaSet có thể phục hồi số Pod theo template bằng một Pod mới, nhưng `emptyDir` gắn với Pod cũ đã mất khi Pod bị loại khỏi node; dữ liệu ứng dụng không được tái tạo bởi replica count. Trước khi cho phép thay Pod, xác định dữ liệu nào cần bền, chọn lưu trữ phù hợp và thử restore bản sao nhất quán; kiểm tra ràng buộc truy cập volume, tính tương thích phiên bản/schema và request nghiệp vụ sau khi Pod mới Ready. PVC/PV cũng không tự chứng minh backup hoặc rollback dữ liệu.

## Đáp án chương 41 — HPA và điều khiển mở rộng

**Bài tập 41.1 — CPU theo request không phải CPU của node**
Không. Với CPU utilization, mẫu số là CPU request của container được xét, không phải tổng CPU node hay CPU limit. Ví dụ usage 80 millicore trên request 100 millicore cho 80% request, dù node có thể còn nhiều CPU rảnh. Muốn dự đoán HPA cần kiểm tra request, metric có đủ mẫu, trạng thái Pod và chính sách scale.

**Bài tập 41.2 — Tính số replica từ CPU HPA**
Hiệu CPU time chia cửa sổ 10 giây cho `300m`, `450m`, `150m`. Với request `500m` mỗi Pod, utilization lần lượt 60%, 90%, 30%, trung bình 60%; tỷ số với target 50% là 1,2, nên `ceil(3×1,2)=4` trong khoảng 2–5. CPU limit nếu có không nằm ở mẫu số. Đây là tính toán cơ sở: metric có thể cũ/thiếu, Pod chưa Ready, tolerance, chính sách tốc độ và ổn định thay quyết định cuối; Pod thứ tư còn cần scheduler, volume, readiness và đường mạng. Đối chiếu status HPA, timestamp metric, Pod/events, EndpointSlice và latency client trước khi kết luận mở rộng đã giúp.

**Bài tập 41.3 — HPA thiếu một mẫu CPU**
Trung bình hai mẫu là 75%; lấy nó làm tỷ số cơ sở cho ba replica sẽ cho ceil(3×75/50)=5. Khi scale-up, controller tính lại với Pod C thiếu metric ở 0% giá trị mục tiêu: (60+90+0)/3=50%, tỷ số 1 nên phép tính này không đề xuất tăng. Status có thể còn báo 75% từ các mẫu ban đầu. 0 là giả định điều khiển, không phải CPU đo được của C. Kiểm danh sách Pod, thời điểm và độ phủ metric từng Pod qua API metric mà HPA dùng, cùng điều kiện/sự kiện và desired/current/ready replicas ở cùng cửa sổ; không suy chỉ từ dashboard hay status.

**Bài tập 41.4 — HPA tăng Pod nhưng thiếu nơi chạy**
Pod mới Pending là thất bại ở bước đặt chỗ: xem requests, allocatable, taint, affinity và topology PVC.

`nr_throttled` tăng ở Pod cũ cho thấy quota CPU có chặn trong cửa sổ đọc counter; cần so `throttled_usec`, `usage_usec`, CPU limit và latency cùng cửa sổ để đánh giá tác động. `oom_kill` không đổi không chứng minh mọi bộ nhớ đều đủ nhưng không có bằng chứng OOM kill trong counter này. Có thể tăng năng lực node phù hợp hoặc điều chỉnh request/limit dựa trên nhu cầu đo và quyền thay đổi; hậu điều kiện gồm Pod mới Scheduled/Ready, request qua Service đạt latency mục tiêu, dữ liệu nguyên vẹn, và counter throttling/metric được đọc lại ở cùng thời gian.

## Đáp án chương 42 — Gateway API và ranh giới yêu cầu ngoài cụm

**Bài tập 42.1 — Thiếu GatewayClass sau khi render chart**
Đọc manifest render để biết Gateway dùng class nào, rồi xác nhận CRD đã có, `GatewayClass` ấy tồn tại và status Accepted ở generation hiện hành; sau đó mới xem Gateway/listener, HTTPRoute parent/conditions và địa chỉ dataplane. Từ vị trí client, xác nhận DNS, địa chỉ đích, Host/SNI và nơi sinh 404. `helm template` chỉ tạo YAML cục bộ với giả định capabilities, không hỏi API server và không chứng minh CRD, class, controller, listener hay request thực tồn tại.

**Bài tập 42.2 — HTTPRoute khác namespace và ReferenceGrant**
Tham chiếu Route → Gateway khác namespace là quan hệ attachment do listener `allowedRoutes` quyết định; ReferenceGrant ở `edge` không mở cửa khi listener còn `from: Same`. Người quản lý Gateway phải cho phép namespace `app` theo chính sách hẹp phù hợp, rồi đọc status parent của Route. Tham chiếu Route → Service ở namespace `data` là quan hệ backend riêng, cần ReferenceGrant trong namespace `data` cho đúng nguồn HTTPRoute và đích Service; sau đó xác minh Service/EndpointSlice. Grant không thay thế selector, endpoint hay khả năng kết nối.

**Bài tập 42.3 — Route được nhận nhưng backend chưa hợp lệ**
`Accepted=True` chỉ nói parent nhận đủ phần cấu hình để có tác dụng; `ResolvedRefs=False` báo ít nhất một tham chiếu sai hoặc không được phép. Với Service vắng mặt, kiểm tra `backendRefs.name`, namespace mặc định là namespace Route, port và message/reason của condition theo parent, cùng khả năng một ReferenceGrant cần thiết. Đối chiếu Service thực và endpoint sau khi sửa. Tăng timeout không tạo object bị thiếu và không đổi quyền tham chiếu; cũng chưa được suy từ condition này rằng dataplane đã phản hồi mã HTTP cụ thể.

**Bài tập 42.4 — Gateway xanh nhưng client vẫn timeout**
So DNS với địa chỉ Gateway và thử TCP/TLS/HTTP từ chính client, ghi IP, Host/SNI, thời gian và nơi sinh timeout. Từ điểm gần Gateway kiểm tra listener/dataplane và request tới Service, rồi từ trong cluster so Service/EndpointSlice với Pod, thử trực tiếp cổng ứng dụng và đọc log/latency; thay từng điểm quan sát để khoanh chặng bị ngắt. Gateway Programmed là báo cáo controller về lập trình dataplane, Route Accepted/ResolvedRefs về attachment/tham chiếu, endpoint Ready về chọn backend; không điều kiện nào kiểm tra đường client, policy/firewall, socket ứng dụng hay response nghiệp vụ tại thời điểm ấy.

## Đáp án chương 43 — Gói Helm (Helm chart) và vòng đời bản phát hành

**Bài tập 43.1 — Giá trị hiệu lực khi nâng cấp chart**
Với --reuse-values, giá trị đã dùng ở release trước có thể được mang sang nên host cũ old.example.test còn hiệu lực nếu không có override mới. Phải ghi chart/version, bộ values hiệu lực và tham số -f/--set; so manifest render dự kiến với helm get manifest của revision 5; cuối cùng kiểm Gateway/Route status ở generation mới, DNS và request thật mang Host/SNI cần phục vụ. Chart source mặc định đúng không chứng minh release đang dùng đúng giá trị.

**Bài tập 43.2 — Lần theo lỗi qua Gateway và backend**
Chưa. Ghi revision, generation hiện tại và observedGeneration của đúng parent/controller; trạng thái cũ chưa mô tả manifest mới. Từ cùng client, so địa chỉ DNS thực và địa chỉ Gateway status với cùng Host/SNI/path theo quyền thử; nếu chỉ đường DNS lỗi, kiểm DNS/cache/tuyến. Khi condition đã thấy generation mới, đọc Accepted/ResolvedRefs, listener và backend, rồi kiểm EndpointSlice và request tới Service từ nguồn phù hợp. Endpoint Ready không chứng minh đường client→Gateway→backend; không suy nguyên nhân từ một 404/503 hoặc snapshot khác thời điểm.

**Bài tập 43.3 — Helm rollback và tương thích dữ liệu**
Không. --atomic chỉ tự rollback khi upgrade thất bại theo điều kiện Helm theo dõi; nếu object được nhận và Ready nhưng hostname sai cho khách hàng, gate nghiệp vụ vẫn có thể fail mà Helm không tự đảo. Kiểm revision mới/manifest/values và object live; kiểm Gateway/Route ở generation mới, endpoint Ready và request qua đúng DNS/SNI/Host/path từ client; kiểm schema và dữ liệu do revision 7 đã ghi còn tương thích hoặc phục hồi theo kế hoạch riêng. Rollback tạo revision mới, không xóa request/message/migration đã commit.

## Đáp án chương 44 — Phép đo, nhãn và chuỗi thời gian

**Bài tập 44.1 — Vì sao không lấy trung bình p95?**
p95 của từng Summary là quantile trên hai phân phối riêng, không cộng tuyến tính; trung bình hai con số ấy không cho p95 của tập request gộp, nhất là khi cỡ mẫu khác nhau. Nếu cần quantile gộp, dùng histogram với cùng ranh giới bucket và tập nhãn phù hợp, cộng tốc độ bucket trước rồi ước lượng quantile. Kết quả vẫn bị giới hạn bởi độ rộng bucket.

**Bài tập 44.2 — Request ID làm tăng số chuỗi metric**
Nếu mọi tổ hợp đều có dữ liệu, 50.000 × 2 × 2 × 4 = 800.000 chuỗi bucket; còn `_sum`, `_count` và request counter làm tăng thêm. Thực tế mỗi ID thường chỉ gắn một status và một instance, nên số này là trần giả định của phép nhân, không phải phép đo. Thay ID bằng nhãn hữu hạn như route template, method, status class; giữ request ID ở log/trace. Sau sửa so số series, scrape duration, bộ nhớ và độ trễ truy vấn cùng điều kiện tải; xác nhận dashboard vẫn phân nhóm cần thiết.

## Đáp án chương 45 — Prometheus, PromQL và Grafana

**Bài tập 45.1 — Rate và p95 tính sai thứ tự**
`rate` phải xử lý reset trên từng chuỗi trước khi `sum`; tổng counter rồi tính rate có thể che reset riêng lẻ. p95 không cộng hoặc lấy trung bình vì phân phối và số request của Pod khác nhau; dùng `histogram_quantile(0.95, sum by (le)(rate(..._bucket[3m])))` cho classic histogram với cùng phạm vi. Nếu một Pod mất scrape, phép gộp chỉ phản ánh các bucket còn quan sát được, có thể bias p95 và không kết luận toàn dịch vụ; kiểm tra `up`, số target/series tham gia và request thực.

**Bài tập 45.2 — Tính rate, lỗi và p95 từ CSV**
Trong `(0,180]`, mỗi instance tăng 12 lỗi và 120 request từ giây 60 đến 180: mỗi instance 0,1 lỗi/giây và 1 request/giây; gộp thành 0,2 và 2 request/giây, tỷ lệ 10%. CPU tăng 24 giây mỗi instance trên 120 giây, tổng 0,4 lõi. Bucket gộp trong cửa sổ 120 giây tăng 80, 180, 232, 240; p95 là vị trí 228 trong khoảng 180–232, nội suy 0,9615 giây. `increase` ngoại suy `rate` theo toàn cửa sổ 180 giây, nên cho khoảng 36 lỗi và 360 request trong ví dụ này, trong khi hiệu thô giữa giây 60 và 180 là 24 và 240. Khi scrape không đều hoặc reset, chênh lệch còn khác.

**Bài tập 45.3 — Panel lỗi giảm khi target mất scrape**
Kiểm tra target có còn trong service discovery, `up` của đúng job/instance, thời điểm scrape cuối, lỗi HTTP/TLS/timeout của endpoint, series `http_requests_total` có mẫu mới không, và selector/job/route/time range của panel. Thử request nghiệp vụ tới `b` qua đường thật, quan sát log và health của ứng dụng. `up=0` chỉ xác nhận scrape thất bại cho target còn được theo dõi; không chứng minh dịch vụ đã ngừng hoặc đã hồi phục. Khi `b` vắng mẫu, tổng lỗi của `a` có thể giảm giả tạo so với trước dù tình trạng người dùng xấu hơn.

## Đáp án chương 46 — Nhật ký, vết truy cập và OpenTelemetry

**Bài tập 46.1 — Tính độ trễ trên cây span**
`G=420−0=420 ms`, `A=405−12=393 ms`, `D=280−30=250 ms`, `U1=335−285=50 ms`, `U2=385−345=40 ms`; hai lần gọi cùng khoảng chờ giữa chúng kéo dài từ 285 đến 385, tức 100 ms. Tổng các duration là 1153 ms vì `A`, `D`, `U1`, `U2` nằm trong `G` và các child nằm trong `A`, nên thời gian bị đếm lặp. Có thể chọn 0..12 ms (Gateway trước API), 335..345 ms (chờ trước retry) hoặc 385..405 ms (API trước khi gửi response) là đoạn chưa có span riêng. Không được tự gán các đoạn ấy cho mạng, queue hoặc CPU.

**Bài tập 46.2 — Log timeout nhưng HTTP trả 200**
`G` là root, `A` là child của `G`; `D`, `U1`, `U2` là child của `A`. Log timeout gắn span `U1` ở 335 ms, severity `WARN`, span status `ERROR` do lần gọi này lỗi; retry `U2` kết thúc ở 385 ms và API ghi response HTTP 200 ở 405 ms. HTTP status của response cuối, span status và mức log là ba trường với hợp đồng khác nhau. `D` là DB client span 250 ms: nó đo thời gian từ API quan sát, có thể gồm chờ pool, mạng và xử lý DB; chưa có DB server span để khẳng định DB server xử lý 250 ms.

**Bài tập 46.3 — Trace chỉ có span Gateway**
Kiểm tra header `traceparent` đi khỏi Gateway và vào API; API extract context và tạo server span đúng parent; chính sách sampling và việc SDK kết thúc/flush span; exporter gửi OTLP; Collector receiver, processor, queue, retry và dropped items; cuối cùng backend ingest, index, retention và truy vấn đúng time range/trace ID. `01` biểu thị sampled flag trong context được truyền, không phải xác nhận span API đã được ghi, gửi, nhận hoặc lưu. Request thành công không thay thế kiểm tra pipeline telemetry.

## Đáp án chương 47 — Cảnh báo và sức khỏe hệ thống quan sát

**Bài tập 47.1 — Tỷ lệ lỗi đẹp lên khi mất target**
Trước mất B: 20/1.000=2%; nếu chỉ A còn trong mẫu: 2/500=0,4%. up=0 nghĩa target còn được khám phá nhưng scrape thất bại; B không còn trong discovery có thể không còn chuỗi up. So danh sách target kỳ vọng với discovery và dữ liệu edge/người dùng cùng cửa sổ, rồi xác minh request thật trước khi kết luận hồi phục. Phép chia trên fixture không thay thế giá trị rate() từ mẫu scrape thật.

**Bài tập 47.2 — Mất scrape nhưng không có cảnh báo**
Không. Kiểm target/discovery, rule firing, alert tới Alertmanager, route/group/inhibition/silence, receiver giao hàng và người trực xác nhận; xét trường hợp thiếu nhãn hoặc alert nguồn không đến.

**Bài tập 47.3 — Metric p95 cao nhưng trace thấy nhanh**
Thứ nhất, head sampling thấp hoặc tail sampling thiên về một loại request có thể làm trace đã lưu không đại diện; đối chiếu cấu hình/chỉ số sampling và metric toàn lưu lượng. Thứ hai, mất context hoặc span ở exporter/Collector/backend có thể làm trace ngắn giả; kiểm tra parent, queue, dropped/export failures và truy vấn cùng time range. Thứ ba, khác route, instance, phiên bản, cửa sổ hoặc đồng hồ lệch khiến hai tập không so được; khóa selector, thời gian và kiểm tra đồng hồ. Dùng nhãn metric hữu hạn như route template và

status class để định vị nhóm; từ log có quyền truy cập lấy request ID/trace ID của request đại diện, rồi truy trace. Không đưa ID hay PII vào nhãn metric hoặc payload telemetry.

**Bài tập 47.4 — Metric và trace mâu thuẫn**
Chưa thể kết luận metric sai. Trace lưu có thể bị thiên lệch do sampling, mất span ở Collector hoặc chỉ bao gồm route, instance, phiên bản và cửa sổ thời gian khác. Đối chiếu cùng population và time range, kiểm tỷ lệ sampling, queue/drop và request đại diện trước khi quy lỗi cho phép đo.

## Đáp án chương 48 — Chỉ số, mục tiêu và ngân sách lỗi

**Bài tập 48.1 — Thiếu dữ liệu đo trong cửa sổ ngắn**
Không. Tổng 11 dòng là 11.000 valid và 129 bad; summary nói giờ cuối 12.000/189, nên thiếu 1.000 valid/60 bad chứ không có bằng chứng lỗi đã hết. Trước tiên kiểm scrape/target, thời điểm sample, counter reset và đối chiếu log/edge từ đúng population. Đánh dấu SLI ngắn hạn `UNKNOWN`, giữ nguyên đánh giá rủi ro canary và không tăng traffic cho tới khi phép đo hoặc tín hiệu độc lập đủ tin cậy. Không dùng dòng vắng như 0 bad. Monthly summary cũng cần provenance: nếu được cập nhật từ một đường dữ liệu khác, nó không tự sửa được khoảng mù năm phút.

**Bài tập 48.2 — Gộp tỷ lệ lỗi của hai replica**
Tổng 19 bad/1.000 valid = 1,9%; phép tính `(1%+10%)/2=5,5%` trao trọng số 50/50 dù A nhận 90% lưu lượng. SLO 99% cho phép `0,01×1.000=10` bad, nên đã vượt 9 bad. Không đổi SLO hay loại B khỏi population sau khi nhìn số.

**Bài tập 48.3 — Burn rate và cảnh báo hai cửa sổ**
Năm phút có `60/1.000=0,06`; burn `0,06/0,001=60`, vượt 14,4. Nếu giờ có 100/12.000 = 0,008333…; burn `8,333…`, không vượt 14,4, nên điều kiện AND không page dù năm phút xấu. Cần dữ liệu 6h và 30m cùng population mới xét nhánh kia; một mẫu 1h không tái tạo được hai cửa sổ ấy. Ngay cả khi page, nguyên nhân và độ tin cậy đo vẫn phải kiểm tra.

## Đáp án chương 49 — Ứng phó sự cố và phân tích sau sự cố

**Bài tập 49.1 — Lỗi canary và giả thuyết nguyên nhân**
v1 có 1/900 ≈ 0,111%; v2 có 30/100 = 30%; chung là 31/1.000 = 3,1%. DB p95 45–50 ms và các mẫu K-26..28 42–49 ms không khớp giải thích "chậm" cho sai số tiền nhân 1/100; tuy nhiên chưa loại được trigger/logic ghi trong DB. DNS sai toàn đường khó tạo kết quả v1 đúng/v2 sai trong cùng cửa sổ; fixture chưa có edge destination hay revision từng request nên chưa loại được lỗi chọn đích theo nhóm. Ba kiểm tra ưu tiên: xác thực request ID→release/config/edge destination; đối chiếu input và dữ liệu lưu gắn với từng ID trong 30 request v2 sai, không suy số order phân biệt; đọc diff và chạy một input 4.100 cents trên hai bản trong fixture cô lập. Không tạo thêm đơn thật để thử.

**Bài tập 49.2 — 5xx bằng không có nghĩa đã hồi phục?**
Chưa. (1) **Mitigation:** 10:12 mới là đề xuất chặn v2 writer và chuyển traffic, cần bằng chứng thao tác hoàn tất cùng hậu kiểm không còn writer v2. (2) **Request verification:** 10:16 còn pending; cần request ID mới, revision/digest, input và giá trị đọc lại đúng qua các route. (3) **Data reconciliation:** 30 là số request sai, không phải số bản ghi phân biệt; phải ánh xạ request→order, đối soát nguồn độc lập rồi sửa hoặc đền bù theo chính sách dữ liệu. (4) **Incident closure:** cần người chịu trách nhiệm xác nhận tác động, hậu điều kiện phục vụ mới, xử lý dữ liệu và thông báo; thiếu một trong bốn không ghi `resolved`. Mở v2 chỉ sau test hồi quy, canary gate nghiệp vụ và quyết định theo risk/budget; không dùng 5xx=0 làm bằng chứng.

**Bài tập 49.3 — Chọn rollback, tắt cờ hay failover**
Rollback/reroute về v1 cần v1 tương thích schema và mọi writer v2 được dừng; hậu kiểm revision/digest, traffic share v2=0, đơn mới đúng. Flag chỉ dùng nếu có bằng chứng flag là nguyên nhân và mọi đường ghi đọc giá trị flag mới; hậu kiểm hai request ở nhánh flag và trạng thái worker. Failover chỉ nếu vùng mới không mang binary/config sai và dữ liệu nhất quán; hậu kiểm từ client tới đúng vùng và đối soát số tiền. Fixture chưa cho bằng chứng cô lập vùng, nên failover không phải lựa chọn an toàn. Cả ba không tự sửa dữ liệu liên quan tới 30 request sai; số bản ghi phân biệt vẫn UNKNOWN.

## Đáp án chương 50 — Sao lưu, khôi phục thảm họa và dữ liệu

**Bài tập 50.1 — Tính RPO và RTO từ dòng thời gian**
Từ 10:00 lùi đến commit khôi phục cuối 09:52 là 8 phút, đạt RPO ≤15 phút trong fixture. Từ 10:00 đến 10:37 là 37 phút, đạt RTO ≤45 phút trong fixture. 10:27 mới replay xong; app chưa chứng minh đọc/ghi, route và invariant nghiệp vụ. Một lỗ WAL trong chuỗi cần thiết có thể chặn replay qua lỗ hoặc buộc chọn điểm sớm hơn; không có bằng chứng 09:52 còn khôi phục được, nên kết luận 8 phút phải chuyển thành UNKNOWN cho đến khi thử chain thật.

**Bài tập 50.2 — Đơn hàng ở điểm khôi phục**
O-101, O-102, O-103 có commit ≤09:52 và được fixture gắn `expected_restored=true`. O-104/O-105 ở 09:53/09:59 cần đối soát receipt, payment, queue hay event log độc lập, không tự tái phát hành. Trước khi xóa bản chính: xác minh backup/replica độc lập miền lỗi và có quyền/khóa đọc; thử restore và nội dung đơn; giữ ảnh hiện trạng và bằng chứng; kiểm tra replica chưa sao lỗi xóa; phê duyệt mục tiêu phục hồi và kế hoạch quay lui. Dữ kiện hiện có không cho phép xóa bản chính.

**Bài tập 50.3 — Hậu kiểm một lần restore**
Hash/size giúp kiểm byte hoặc manifest; `pg_verifybackup` dùng với bản `pg_basebackup` dạng plain có manifest, và kiểm WAL bằng bản công cụ cùng phiên bản, giúp phát hiện một số hư hỏng backup nhưng không là phép kiểm nghiệp vụ. DB phải mở được, replay đúng target, query/order count và constraint hợp lệ; ứng dụng phải đọc/ghi qua đúng endpoint với quyền đúng; nghiệp vụ phải đối soát amount, receipt và tác dụng phụ. Checksum PASS nhưng schema ứng dụng không tương thích hoặc O-104 đã thanh toán bên ngoài mà DB khôi phục không có đơn: chưa mở ghi production vì nguy cơ mất/trùng giao dịch. Cần điều tra và quy trình đền bù trước.

## Đáp án chương 51 — Hiệu năng, năng lực tải và thử giới hạn

**Bài tập 51.1 — Concurrency theo Little's Law**
A: 100 request/s × 0,120 s = 12 request; B: 160 × 0,250 s = 40 request, với điều kiện cửa sổ ổn định, cùng ranh giới vào/ra và latency trung bình của toàn population hoàn tất. C: `(220−180) request/s × 60 s = 2.400 request` tăng trong tổng chưa hoàn tất (đang chờ + đang xử lý) **nếu** không có reject, drop, timeout, hủy hay retry ẩn. Không biết bao nhiêu trong 2.400 đang chờ riêng ở queue; phải đo queue và in-flight đầu/cuối. `220×0,550=121` không là L đã chứng minh vì arrival≠departure, số chưa hoàn tất tăng và W chỉ đo những request đã hoàn tất. Counter reject/timeout hoặc đo tổng outstanding đầu/cuối khác +2.400 sẽ bác bỏ giả định; retry có thể làm offered bị đếm hai lần.

**Bài tập 51.2 — CPU 70% có đủ để tăng replica?**
CPU tổng 70% che per-core/quota skew và waiting; C đã có số request chưa hoàn tất tăng dù completion chỉ 180/s; thêm replica có thể tăng áp lực lên DB hoặc connection pool, còn 200/s chưa được đo. Giả thuyết DB wait/connection pool bão hòa phù hợp p95 DB wait 300 ms; bác bỏ bằng phân rã trace theo stage, pool wait,

DB active/lock/IO và so cùng traffic mix. Không tuyên bố 200/s đạt SLO khi không có error rate, thời lượng ổn định, tail ở 200 hay test mất một replica.

**Bài tập 51.3 — Thiết kế phép thử năng lực tải**
Chốt artifact/topology/payload, generator đủ dư địa, offered/accepted/completed/rejected và SLI client. Baseline 160 rồi 180/200, stress có kiểm soát đến 220 và stop nếu p95 >600 ms hai cửa sổ, backlog tăng, lỗi dữ liệu hoặc generator bão hòa. Soak giữ 200 sau khi đã chứng minh ổn định để xem leak/queue/GC; hậu kiểm giảm tải, queue rút và nghiệp vụ đúng. PASS trong môi trường thử không chứng minh production nếu DB nhỏ hơn, tenant mix khác, cache nóng hơn hoặc không có mất replica/rollout như thực tế. Không đem fixture A–C làm kết quả test thật.

## Đáp án chương 52 — Mô hình đe dọa và quyền tối thiểu

**Bài tập 52.1 — Quyền truy cập từng đơn hàng**
API thiếu kiểm chủ sở hữu từng object. Thử user 8 đọc được đơn 41 và user 7 bị từ chối; kiểm cache không dùng kết quả của user khác.

**Bài tập 52.2 — Quyền ConfigMap không phải quyền S3**
Không. K8s RBAC và AWS IAM là hai chính sách. Kiểm role session thực và thử Pod bị từ chối PutObject hoặc prefix S3 ngoài phạm vi, đồng thời vẫn đọc object cần thiết.

**Bài tập 52.3 — Chứng cứ trước khi mở rộng quyền S3**
Trước: request lỗi, identity/action/object và đường mạng. Sau: policy hẹp, request hợp lệ thành công, ca âm PutObject/prefix khác bị chặn, audit và thời điểm được ghi; không duyệt wildcard chỉ vì status hết lỗi.

## Đáp án chương 53 — Bí mật và bảo vệ dữ liệu

**Bài tập 53.1 — Đổi token khi container cũ còn chạy**
Chưa. Pod thứ ba còn nhận traffic bằng A nên revoke ngay có thể gây lỗi; B thành công ở hai Pod không chứng minh tất cả Pod đã chuyển. Ghi version theo từng Pod, tách hoặc cập nhật Pod còn A, thử request qua từng endpoint, rồi revoke A và thử ca âm A bằng đúng backend. Sau 10:30 phải quan sát renew hoặc cấp credential mới và request tiếp tục đúng; nếu renew thất bại, app fail closed theo thiết kế chứ không fallback sang secret tĩnh. TTL yêu cầu trong config không thay TTL thực hệ cấp trả về.

**Bài tập 53.2 — Bí mật còn trong layer của image**
Thứ nhất revoke/rotate token và kiểm ca âm để giá trị cũ không còn gọi API trong phạm vi backend; thứ hai ngăn phân phối và truy các bản sao ở digest cũ, runner, registry/cache, log và backup theo phạm vi được phép. Xóa tag hay file ở layer mới không xóa layer cũ; dựng lại từ context sạch và dùng build secret mount thay ARG/ENV phù hợp. Quét marker âm chỉ chứng minh không tìm thấy trong artifact/thời đoạn đã quét, không chứng minh mọi bản sao biến mất.

**Bài tập 53.3 — Backup không còn khóa giải mã**
Không. Byte backup còn nguyên nhưng có thể không giải mã được nếu bản cũ vẫn cần K1; giữ khả năng decrypt theo retention, kiểm key version/quyền ở môi trường restore, tái bao theo giao thức sản phẩm và thử restore bằng identity dự kiến trước khi hủy K1. Nếu D đã lộ, chỉ đổi wrapping key không thu hồi dữ liệu có thể đã bị đọc; phải đánh giá phạm vi rò và khả năng mã hóa lại bằng data key mới. Hậu kiểm gồm giải mã, dữ liệu đúng và ứng dụng đọc/ghi, không chỉ lệnh KMS thành công.

# Đáp án chương 54 — Chuỗi cung ứng phần mềm

**Bài tập 54.1 — Base image đổi dù commit không đổi**

Không. Commit không cố định base image hoặc cây dependency khi tag và phiên bản bắc cầu còn động; chữ ký/scan gắn với byte của bản đầu. Dừng promotion, ghi digest base, lockfile toàn cây, checksum và nguồn gói, toolchain, tham số build; tạo SBOM và gate lại cho digest image thực định phát hành. Nếu build lại vẫn khác digest, điều tra nguồn bất định trước khi tuyên bố tái lập; SBOM/scan xanh không tự chứng minh dependency an toàn.

**Bài tập 54.2 — Chữ ký đúng nhưng tag trỏ digest khác**

Không. Subject khớp chỉ xác nhận attestation nói về đúng digest đích; chữ ký hợp lệ chỉ xác nhận bản ghi đến từ khóa đang tin cậy. Policy còn yêu cầu nguồn/commit đã resolve là c84 và builder identity được phép. Vì c83 sai, từ chối và điều tra ref/đầu vào workflow; không đổi tay provenance. Nếu commit đúng nhưng builder lạ hoặc subject khác, cũng từ chối theo cạnh tương ứng. Thiếu trường nguồn quan trọng là UNKNOWN, không phải PASS.

**Bài tập 54.3 — Pod cũ còn nhận request khi rollout**

Chưa. Admission kiểm request tạo/cập nhật mới, không tự dừng Pod đang phục vụ. Liệt kê Pod imageID thực, ReplicaSet, Service endpoint và traffic theo revision; xác định blast radius rồi thay hoặc cô lập Pod theo kế hoạch giữ dịch vụ/dữ liệu. Chỉ đóng sổ kỹ thuật khi spec và mọi Pod phục vụ mục tiêu dùng digest đã duyệt, request đại diện và dữ liệu đúng. Lưu riêng digest, attestation, test result và policy version; chữ ký hoặc policy không thay phép kiểm runtime.

# Đáp án chương 55 — Gia cố môi trường đám mây và Kubernetes

**Bài tập 55.1 — Deployment được nhận nhưng Pod bị từ chối**

Enforce chưa thất bại: PSA warn áp lên template của Deployment, còn enforce xét lúc ReplicaSet tạo Pod và từ chối Pod đặc quyền. Đối chiếu response tạo Deployment, sự kiện FailedCreate của ReplicaSet, Pod mới vắng mặt và số replica chưa đạt; không dùng exit code apply làm bằng chứng workload đã chạy. PSA không kiểm image tag/digest, nên Pod chỉ vi phạm quy tắc digest sẽ không bị PSA từ chối vì lý do ấy. Muốn chặn tag, cần policy admission khác và ca âm riêng chứng minh nó từ chối request tạo Pod.

**Bài tập 55.2 — Secret qua API và volume là hai đường quyền**

Không. RBAC của SA quyết định request API bằng identity ấy; kubelet đưa Secret mà Pod spec tham chiếu vào volume, còn process đọc tệp theo mount và quyền tệp, không cần token API của SA. Kiểm Pod spec/volumeMount, UID và khả năng đọc tệp chỉ bằng status, rồi kiểm can-i get/list của đúng SA và quyền tạo/sửa Pod trong namespace; không in byte Secret. Nếu ứng dụng cần Secret, chỉ mount cho container cần dùng và hạn chế ai được tạo/sửa workload hoặc tham chiếu Secret. Nếu bỏ quyền đọc tệp của chính process mà không đổi cách cấp credential, ứng dụng không thể tiếp tục dùng Secret ấy.

**Bài tập 55.3 — AccessDenied không sửa bằng mở Security Group**

Không. Kiểm role session, action/object, explicit Deny và resource policy; mở SG rộng không cấp IAM Allow và làm tăng bề mặt tấn công.

# Đáp án chương 56 — Thiết kế một dịch vụ vận hành thực tế

**Bài tập 56.1 — Định nghĩa SLO có thể kiểm chứng**

Thiếu population, cửa sổ, numerator/denominator, loại timeout/lỗi và điểm đo. Cần hợp đồng chỉ số trước khi chọn capacity hoặc tính error budget.

**Bài tập 56.2 — Rollout xanh nhưng đơn hàng không đọc lại được**
Chúng xác nhận các ranh giới điều khiển và probe hẹp; cần theo request thật tới database, xác nhận commit và GET đọc lại record cùng revision/digest đang phục vụ.

**Bài tập 56.3 — Canary lỗi khi health check vẫn xanh**
360×300×0,02=2.160 request lỗi giả định. Dừng tăng canary, giữ evidence, kiểm request/data và tương thích schema trước rollback. Nhánh AZ cho thời gian phục hồi 9 phút nếu hợp đồng coi 10:00–10:09 là gián đoạn; B có recovery point cách sự cố 2 phút, không chứng minh đã mất đúng 2 phút dữ liệu hay mất bao nhiêu đơn. So tập ID đơn đã xác nhận trước sự cố với tập đọc được ở B, phục hồi/replay mọi đơn thiếu từ nguồn độc lập và kiểm đọc lại. Khi chưa chứng minh đủ, yêu cầu zero-loss chưa đạt, dù request mới qua B đã thành công.

## Đáp án chương 57 — Mở rộng và tối ưu chi phí có ràng buộc

**Bài tập 57.1 — Số replica để chịu đỉnh và mất AZ**
Mục tiêu 90 rps/replica, cần 4 replica cho hệ khỏe. Để AZ còn lại một mình có 4 replica, đặt 4 mỗi AZ, tổng 8; vẫn phải thử năng lực thật và dependency.

**Bài tập 57.2 — Tính chi phí hai phương án replica**
A=8×730p=5.840p. B=(4×730+4×2×30)p=3.160p; chênh 2.680p, chưa gồm node/egress/storage và rủi ro autoscale lag.

**Bài tập 57.3 — Giá rẻ hơn nhưng mất SLO**
Cần năng lực và autoscale lag đo thật, ràng buộc SLO/DR, error budget, chi phí đầy đủ theo Region/ngày, tác động nghiệp vụ và chủ rủi ro; chênh compute riêng chưa đủ.

## Đáp án chương 58 — Bài tổng hợp: thay đổi, sự cố và phục hồi

**Bài tập 58.1 — Thiếu digest và phép thử restore**
Không. Ghi digest từ byte, khóa input, restore thử và kiểm nội dung trước; thiếu mắt xích thì dừng hoặc ghi SKIP/FAIL đúng phạm vi.

**Bài tập 58.2 — Canary dưới SLO có được tăng tải?**
Không. SLI cửa sổ là 99%, dưới 99,9%; dừng tăng traffic, điều tra lỗi và kiểm missing data/request-data trước quyết định tiếp.

**Bài tập 58.3 — RPO và RTO khi chỉ restore backup**
Recovery point 10:00 cách thời điểm sự cố 10:04 là 4 phút. Nếu có ghi đã xác nhận sau backup đến khi chặn ghi lúc 10:07, khoảng ghi cần đối soát dài tối đa 7 phút; số bản ghi mất phải kiểm ledger và nguồn có thể replay. Lúc phục vụ lại 10:13, backup đã 13 phút tuổi, nhưng 13 phút không phải khoảng dữ liệu mất theo RPO. Nếu dữ liệu sai từ 10:04 được tính là dịch vụ không đạt và hậu kiểm hoàn tất 10:13, thời gian phục hồi đo được là 9 phút. RPO/RTO là các mục tiêu tối đa do tổ chức định trước; timeline không tự cung cấp mục tiêu hay chứng minh đã đạt chúng.

**Bài tập 58.4 — Ledger xanh nhưng thiếu bằng chứng capstone**
Không. Thiếu identity artifact, liên kết request–revision–record và bằng chứng phục hồi. Ghi UNKNOWN/SKIP theo nguyên nhân, dừng triển khai nếu thiếu hậu trạng thái dữ liệu.



# HIỂU HỆ THỐNG.
# VẬN HÀNH
# CÓ CHỨNG CỨ.

Từ mã nguồn đến hạ tầng,
triển khai, quan sát và phục hồi,
cuốn sách đi từ cơ chế đến
quyết định vận hành có thể kiểm chứng.